

算法刷题与 C++ 面试教材

数据结构、算法设计、复杂度证明与工程化题解

从暴力枚举到优化推理，从容器原理到面试表达

面向长期能力建设的完整学习稿

前言

这本书的目标不是堆题解，也不是把学习压缩进某个固定周期，而是把数据结构与算法训练成一套可以长期迁移的工程能力。面试题看似零散，背后其实围绕两条主线：一条是数据结构（data structure），也就是信息怎样存、怎样查、怎样更新；另一条是算法（algorithm），也就是在约束下怎样组织计算、怎样减少重复工作、怎样证明结果正确。数据结构和算法不能分开学。只会说“这题用滑动窗口”，但不知道窗口状态应该放在数组、哈希表还是队列里，代码一定会不稳；只会背 `unordered_map`，但不知道它解决的是历史查询问题，也很难在新题里想到它。

本书按“知识完整性”组织，而不是按倒计时组织。前面的章节先建立复杂度、暴力法、复盘方法和 C++ 代码标准；中间把数组、字符串、链表、栈、队列、哈希表、堆、树、图和并查集讲成数据结构主线；后面再进入双指针、滑动窗口、前缀和、二分、搜索、动态规划、回溯、贪心和综合题。训练周期可以按短期集中训练、学期式推进或长期滚动复习来调整，但正文必须把原理、背景、案例、代码、证明、边界和复盘方法讲完整。

每一类算法都必须从暴力方法开始。暴力方法不是低级答案，而是理解问题空间的方式。两数之和先枚举下标对，才能看出慢点是反复查找补数；最短子数组先枚举所有区间，才能看出正数数组存在窗口单调性；岛屿数量先把每个格子当成节点，才能把题目翻译成图的连通块。优化不是凭感觉，而是来自重复工作、单调性、可维护状态、候选淘汰、最优子结构和数据结构查询能力。

代码使用 C++20。示例代码优先使用 `const` 引用、明确边界、清楚的变量名和可测试函数。书里会解释 `vector`、`string`、`deque`、`stack`、`queue`、`priority_queue`、`unordered_map`、`set`、树、图和并查集的用法与底层复杂度，也会说明迭代器失效、哈希退化、扩容、拷贝、溢出和递归栈这些真实面试和线上评测里会遇到的问题。

读法

读这本书时，不要只看最终代码。每道题至少走四遍：第一遍只读题并写暴力思路；第二遍找慢点和可优化性质；第三遍写 C++ 优化代码；第四遍复盘边界、复杂度和面试表达。

每天训练时，把题目记录成固定格式：题型、输入规模、暴力枚举对象、优化来源、不变量、使用的数据结构、复杂度、错因和一周后是否能独立重写。只写“没想到”没有意义，必须写成具体原因：没有发现单调性、哈希表插入顺序错、窗口收缩条件错、二分边界没有排除 `mid`、`int` 溢出、递归深度过大、容器选择导致复杂度不对。

本书目录只显示部和章，小节不进入目录。章内标题用于串联正文，不把内容切碎成大量编号。你应该把每章当成一条推理链来读，而不是把每个小标题当成孤立知识点。

目录

第一部分	基础方法：从题目到复杂度预算	
第 1 章	刷题方法：从题目到可复用能力	2
第二部分	数据结构：算法能力的骨架	
第 2 章	线性数据结构：数组、字符串、链表、栈和队列	12
第 3 章	非线性数据结构：哈希表、堆、树、图和并查集	39
第三部分	核心算法：从暴力到优化	
第 4 章	数组、双指针和滑动窗口	67
第 5 章	哈希表和前缀和	82
第 6 章	二分查找和答案空间	94
第 7 章	树、图、BFS 和 DFS	102
第 8 章	回溯、枚举和搜索空间剪枝	118
第 9 章	动态规划：状态、转移和重复子问题	131
第四部分	面试专项：从题型到工程表达	
第 10 章	线性结构专项：链表、栈、队列和单调结构	150
第 11 章	排序、选择和区间问题	164
第 12 章	堆、Top K 和贪心证明	177
第 13 章	图论进阶：并查集、最短路和状态图	190
第 14 章	动态规划进阶、容器原理和源码阅读	208
第五部分	Linux 源码专题：内核数据结构与算法	
第 15 章	Linux 内核数据结构与算法专题	349

第零部分

基础方法：从题目到复杂度预算

第 1 章

刷题方法：从题目到可复用能力

刷题和面试训练的核心不是“见过多少题”，而是能否把陌生题拆成熟悉结构。题目只是外壳，真正要识别的是数据怎么组织、状态怎么变化、答案依赖哪些历史信息、是否存在单调性、是否能淘汰候选、是否有重叠子问题。

面对一道题，第一步不是想模板，而是读约束。输入规模决定复杂度上限，值域决定能不能用计数数组或位图，有序性决定能不能双指针或二分，是否允许重复决定哈希表和去重逻辑，是否有负数决定滑动窗口和前缀和的选择。很多错误题解不是算法错，而是约束没看清。

第二步写暴力方法。暴力方法要明确枚举对象：枚举元素、下标对、子数组、子序列、路径、排列、状态，还是答案值。枚举对象一旦明确，复杂度就能估出来。比如两数之和枚举下标对是 $O(n^2)$ ；所有子数组是 $O(n^2)$ ；所有排列是 $O(n!)$ ；网格 BFS 扫所有格子是 $O(mn)$ 。

第三步找重复工作。重复工作有几种典型形态：反复查询某个历史值，适合哈希表；反复求区间和，适合前缀和；反复维护窗口内统计，适合滑动窗口；反复找当前最大或最小，适合堆或单调队列；反复搜索同一状态，适合记忆化或动态规划。

面试表达原则

不要直接说“这题用哈希表”。更好的表达是：暴力会枚举所有下标对，时间是 $O(n^2)$ ；慢点在于每个数都要反复查找另一个数是否存在；我从左到右扫描，用哈希表保存已经看到的值到下标的映射；处理当前值时查 `target - nums[i]` 是否出现过；这样每个元素只插入和查询一次，平均时间 $O(n)$ ，空间 $O(n)$ 。

第四步写不变量。不变量是优化算法正确性的核心。滑动窗口的不变量可能是“窗口内没有重复字符”；单调栈的不变量可能是“栈内下标对应的值单调递减”；BFS 的不变量是“当前层所有节点距离源点相同”；二分的不变量是“答案一定在当前左右边界之间”。没有不变量，代码只是碰巧通过样例。

第五步落到 C++。C++ 代码要讲容器选择。`vector` 适合连续存储和下标访问；`unordered_map` 适合平均常数查询历史信息；`deque` 适合两端弹入弹出；`priority_queue` 适合动态极值但不能删除任意元素；`set` 适合有序前驱后继；图的邻接表通常用 `vector<vector<int>>`，稀疏图比邻接矩阵更节省空间。

仍然以两数之和为例。题目问的是两个不同下标的数能否凑成目标值，最直接的想法不是哈希表，而是枚举下标对。先固定左端 `left`，再枚举它右边的 `right`，检查二者之和是否等于 `target`。这段暴力代码覆盖了全部候选，因为任意合法答案都是某一对 `left < right` 的下标；它也天然避免同一个下标被用两次。

图示：两数之和从暴力到哈希

暴力枚举	固定 <code>left</code> ，扫描 <code>right</code> ，逐对检查 <code>nums[left] + nums[right]</code> 。
重复工作	每个当前位置都在历史元素里重新寻找补数，历史查询没有被保存。
哈希状态	<code>value_to_index</code> 保存已经见过的“值 -> 下标”，只表示当前下标之前的历史。
循环顺序	先查 <code>target - nums[index]</code> ，再插入 <code>nums[index]</code> ，保证不会使用同一个下标两次。

Listing 1.1 LeetCode 1 两数之和：先写完整暴力基线

```
1 std::vector<int> two_sum_bruteforce(const std::vector<int>& nums, int target)
```

```

2 {
3     for (int left = 0; left < static_cast<int>(nums.size()); ++left) {
4         for (int right = left + 1; right < static_cast<int>(nums.size()); ++right) {
5             const long long sum =
6                 static_cast<long long>(nums[left]) + nums[right];
7             if (sum == target) {
8                 return {left, right};
9             }
10        }
11    }
12    return {};
13 }

```

暴力版本慢在“查找补数”这一步。固定当前数 `nums[right]` 时，我们真正需要知道的是此前是否出现过 `target - nums[right]`，不需要把所有历史下标重新扫一遍。于是优化不是凭空说“用哈希表”，而是把历史值组织成值到下标的映射：扫描到当前下标时，先查询补数是否已经在历史表中；如果在，答案就是历史下标和当前下标；如果不在，再把当前值放进历史表，供后面的元素查询。

Listing 1.2 LeetCode 1 两数之和：哈希表把补数查找变成历史查询

```

1 std::vector<int> two_sum_hash_table(const std::vector<int>& nums, int target)
2 {
3     std::unordered_map<int, int> value_to_index;
4     value_to_index.reserve(nums.size());
5
6     for (int index = 0; index < static_cast<int>(nums.size()); ++index) {
7         const int need = target - nums[index];
8         const auto found = value_to_index.find(need);
9         if (found != value_to_index.end()) {
10            return {found->second, index};
11        }
12        value_to_index.emplace(nums[index], index);
13    }
14    return {};
15 }

```

这段优化代码的不变量是：处理下标 `index` 前，哈希表只保存 `[0, index)` 中已经出现过的值。先查再插入，正是为了保持“历史”这个语义，避免当前元素和自己配对。参数用 `const std::vector<int>&`，避免复制输入数组。`reserve` 不是必须，但能减少扩容和 rehash。返回空数组表示没有答案，真实力扣题如果保证有答案，也可以直接返回。这样讲完以后，读者能看到完整链条：枚举下标对可以保证正确，瓶颈是重复扫描历史，哈希表只负责把历史补数查询降到平均常数。

训练路线：从基础题到综合能力

学习路线不是倒计时。有人一个月集中准备，有人一边工作一边长期训练，也有人先补 C++ 和数据结构基础，再进入高强度题目练习。因此本章只给出能力阶段和训练节奏，不把整本书绑定到固定月份。你可以把每个阶段压缩成一周，也可以展开成一个月；真正不能省略的是每个阶段要掌握的知识、题型、代码能力和复盘方法。

第一阶段建立基础语言和复杂度意识。目标是能读懂输入规模，估计可接受复杂度，写出暴力解，并说清楚暴力解慢在哪里。这个阶段学习数组、字符串、`vector`、`string`、排序、二分库函数、下标边界、引用传参和整数溢出。推荐题包括两数之和、移动零、删除有序数组重复项、合并两个

有序数组、最大子数组和、反转字符串和有效回文。做题时不要急着背模板，先把枚举对象写清楚：枚举元素、下标对、区间，还是答案边界。

第二阶段学习线性结构和窗口类算法。双指针、滑动窗口、前缀和、单调栈和单调队列看起来像技巧，本质都是“线性序列上的状态维护”。窗口题要写出左右边界的含义，说明什么时候扩张、什么时候收缩；单调结构题要说明哪些候选被淘汰，为什么淘汰后不会丢答案。推荐题包括盛最多水的容器、三数之和、长度最小的子数组、无重复字符的最长子串、找到字符串中所有字母异位词、最小覆盖子串、每日温度、柱状图最大矩形和滑动窗口最大值。

第三阶段把哈希、堆、树和图串起来。哈希表负责历史查询，堆负责动态极值，树负责层次和有序关系，图负责任意状态转移。这个阶段要形成建模习惯：节点是什么，边是什么，状态保存在哪里，访问标记何时更新，答案在入队、出队、入栈、出栈还是回溯时产生。推荐题包括字母异位词分组、最长连续序列、和为 K 的子数组、前 K 个高频元素、数组中的第 K 个最大元素、二叉树层序遍历、验证二叉搜索树、最近公共祖先、岛屿数量、腐烂的橘子、课程表和课程表 II。

第四阶段进入搜索、回溯和动态规划。回溯题要写清选择列表、路径状态、终止条件和撤销动作；动态规划题要写清状态定义、转移来源、初始化、遍历顺序和空间优化条件。不要先背方程，先问一个状态是否足够描述未来。推荐题包括全排列、子集、组合总和、括号生成、单词搜索、爬楼梯、打家劫舍、零钱兑换、最长递增子序列、最长公共子序列、编辑距离、分割等和子集和目标。

第五阶段处理贪心、并查集和综合题。贪心题的关键是证明局部选择为什么能推出全局最优；并查集题的关键是把动态连通关系建模成集合合并；综合题则常常把哈希、排序、堆、图和 DP 混在一起。推荐题包括跳跃游戏、加油站、合并区间、冗余连接、账户合并、实现 Trie、网络连接、单词接龙和数据流中位数。这个阶段不要只追求 AC，要训练面试表达：暴力解是什么，瓶颈在哪里，优化依赖哪条性质，代码如何保证边界正确。

如果需要集中训练，可以把这些阶段压成十二周：基础复杂度与数组字符串，双指针与窗口，哈希与前缀和，栈队列堆与单调结构，二分查找，链表与树，图和拓扑排序，回溯，一维动态规划，二维动态规划和背包，贪心与并查集，综合模拟与查漏补缺。这只是节奏表，不是本书的边界。书的目标是把知识完整讲清楚，训练周期由你的基础、时间和目标岗位决定。

阶段交付

每个阶段至少留下四类材料：一组代表题的暴力解和优化解，一份数据结构选择说明，一份 C++ 容器和边界错误清单，一份一周后能独立重写的回访题表。刷题报告不是形式，它是把题解转化成长期能力的工具。

学习方法和题解复盘的深层要求

刷题教材不能只给最终代码。读者真正需要的是从题目约束到算法选择的推理链：输入规模允许什么复杂度，数据是否有序，值域能否压缩，状态是否会重复，局部选择能否证明，容器操作成本是否符合题目热路径。每道题都要先写暴力解，因为暴力解暴露了最朴素的状态空间；然后再指出哪个查询、哪个枚举、哪个重复子问题最贵。优化不是把模板贴上去，而是把最贵的那一步替换为更合适的数据结构或更强的不变量。

高质量复盘至少包含三层。第一层是题面复述，用自己的话说明输入、输出和约束，尤其要把隐藏条件翻译成算法语言，例如正数数组带来窗口单调性，排序数组带来二分和双指针，树节点唯一性决定能否用值建索引。第二层是复杂度预算，把 n 、 m 、值域、字符串长度、图边数分开，不要只写一个 n 。第三层是正确性说明，用不变量、交换论证、归纳或最短路层次证明解释为什么不会漏答案。

题解里的代码要服务于解释，而不是炫技。变量名应该表达状态含义，例如 `left` 表示窗口左边界，`farthest` 表示当前可达最远位置，`indegree` 表示尚未满足的依赖数量。复杂代码要先拆出检查函数、松弛函数或合并函数。这样做不是为了形式，而是让读者能在面试中逐句讲清楚代码对应的数学状态。代码写完以后，至少用一个最小用例、一个边界用例、一个反例用例和一个规模用例手算。

三个月计划只能作为节奏，不应该成为内容边界。真正的学习路径是循环式的：第一轮建立题型地图，第二轮补齐证明和边界，第三轮把相似题横向比较，第四轮把代码改成工程上也能接受的版本。每一轮都要记录失败原因。失败原因不是笼统的不会，而是要拆成读错题、复杂度预算错误、状态定义不完整、边界初始化错误、容器 API 误用、证明缺失或代码实现失误。

面试表达要像讲工程方案。先说暴力方案，说明它为什么慢；再说关键观察，说明它让哪一步

变便宜；然后给出数据结构职责，说明保存什么、查询什么、删除什么；最后给出不变量和复杂度。这个顺序能防止答案变成模板堆砌。面试官追问时，优先回到约束和不变量，而不是机械背另一段代码。

实验是把知识落地的关键。数组题打印指针区间，窗口题打印计数表和 formed，二分题打印 left、mid、right 和谓词结果，图题打印队列层次，DP 题打印小表格，堆题打印堆顶和过期状态。只要一次实验能解释一个常见错误，它就不是额外负担，而是把题解变成长期能力的工具。

专题复盘要求

读完本节后，不要只记结论。请把每个结论还原成一个可验证的问题：它解决了哪一步重复工作，依赖哪个题目条件，使用哪个容器或状态，边界条件如何破坏错误写法。

复杂度、暴力法和优化来源

复杂度是算法设计的预算。输入规模如果是 $n \leq 100$ ， $O(n^3)$ 可能可以接受；如果是 $n \leq 2 \times 10^5$ ，通常要接近 $O(n \log n)$ 或 $O(n)$ ；如果网格规模是 $m * n \leq 10^5$ ，BFS 或 DFS 扫一遍通常合理；如果状态空间是指数级，就要考虑剪枝、记忆化或动态规划。

大 O 记号首先是一种上界语言。说一个算法是 $O(f(n))$ ，意思不是它一定运行 $f(n)$ 步，也不是它在每台机器上都花同样时间，而是当输入规模足够大时，它的运行成本不会比 $f(n)$ 的常数倍增长得更快。常数、低阶项和机器细节会被隐藏起来，所以 $3n + 20$ 、 $10n$ 、 $n / 2$ 都写成 $O(n)$ ； $n^2 + 100n + 7$ 写成 $O(n^2)$ 。这种写法的价值是让我们先判断增长数量级，再处理实现细节。

只会写大 O 还不够。 $\Theta(f(n))$ 表示紧确界，意思是上下都被同一数量级夹住；如果一个算法必须完整扫描数组一次，它通常是 $\Theta(n)$ ，既不会低于线性，也不会高于线性。 $\Omega(f(n))$ 表示下界，常用于说明任何算法至少要花多少成本。比如无序数组中找最大值，比较模型下至少要看每个元素一次，所以有 $\Omega(n)$ 下界；一个线性扫描算法又能做到 $O(n)$ ，于是它就是 $\Theta(n)$ 。刷题里常写大 O，是因为我们更关心“会不会超时”；教材学习时必须知道上界、下界和紧确界不是同一个概念。

复杂度分析要先定义输入规模。数组题里 n 通常是元素个数；字符串题里可能是字节长度，也可能是字符数量；矩阵题要写成 m 行 n 列；图题要区分顶点数 V 和边数 E ；字典题还要考虑单词个数和单词平均长度。错误的输入规模会直接导致错误结论。比如课程表不是 $O(n^2)$ 起步，而是取决于课程数和先修边数，邻接表拓扑排序是 $O(V + E)$ 。再比如单词接龙如果每次生成邻居都遍历整个词表，复杂度会包含词表规模和单词长度；若用通配模式桶，复杂度结构又会改变。

复杂度不是一句口号

一份合格的复杂度说明至少包含四件事：输入规模用什么符号表示，主循环或状态数量是多少，每个状态或每次循环做了什么成本，空间里是否包含递归栈、哈希桶、队列、输出结果。只写“时间 $O(n)$ ，空间 $O(1)$ ”但说不出 n 是什么，通常说明分析还没有真正完成。

最好、平均、最坏复杂度也要分清。线性查找目标值，最好情况第一个元素就是答案，成本是常数；最坏情况目标不存在或在最后，成本是线性；平均情况要依赖输入分布。哈希表查询平均是 $O(1)$ ，但最坏可能退化；快速排序平均 $O(n \log n)$ ，若枢轴每次极端，最坏 $O(n^2)$ 。工程和面试里通常优先承诺最坏复杂度；如果使用平均复杂度，要说明它依赖哈希均匀、随机化枢轴或输入分布假设。

摊还复杂度是另一种经常被误解的语言。它不是“随机平均”，而是把一串操作的总成本分摊到每次操作上。`vector::push_back` 多数时候只在尾部构造一个元素，是常数成本；偶尔容量满了，要申请新内存并移动所有旧元素，单次可能是线性成本。但如果容量按倍数增长，连续插入 n 次的总搬移成本仍然是线性的，所以每次插入的摊还成本是 $O(1)$ 。单调栈也是同理：某一次循环可能弹出很多元素，但每个元素一生只会入栈一次、出栈一次，整段程序总弹出次数不超过 n 。

递归算法的复杂度要看调用树，而不是看代码里有几个递归调用就机械下结论。爬楼梯的朴素递归每个状态分裂成两个子调用，并且同一个 $f(i)$ 会被反复计算，形成指数级搜索树；加记忆化后，每个 i 只计算一次，状态数变成 n 。归并排序也有两个递归分支，但两个子问题规模各为一半，每层合并成本是 n ，总共有 $\log n$ 层，因此是 $O(n \log n)$ 。递归式分析的核心是“有多少层、每层总工作量是多少、是否存在重复子问题”。本书的代码会尽量给出迭代实现，但理解递归树有助于解释

为什么 DP 和分治成立。

Listing 1.3 从循环结构读出复杂度

```
1 long long count_pairs_with_order(const std::vector<int>& nums)
2 {
3     long long pairs = 0;
4     for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
5         for (int j = i + 1; j < static_cast<int>(nums.size()); ++j) {
6             if (nums[i] <= nums[j]) {
7                 ++pairs;
8             }
9         }
10    }
11    return pairs;
12 }
```

这段程序的内层循环次数不是 $n * n$ ，而是 $(n - 1) + (n - 2) + \dots + 1$ ，等于 $n(n-1)/2$ 。去掉常数和低阶项后仍然是 $\Theta(n^2)$ 。如果把内层改成每次翻倍的 $j *= 2$ ，就会变成对数次数；如果两个独立循环前后相接，不是相乘，而是相加后取最大数量级。读复杂度时要把循环之间的关系先看清楚：嵌套通常相乘，顺序通常相加，二分通常来自每次砍掉一半，图遍历通常来自每个点和每条边被处理有限次。

空间复杂度不是“有没有开数组”这么简单。递归调用栈是空间，BFS 队列是空间，哈希表桶和节点是空间，排序过程可能使用栈或临时数组，输出结果是否计入空间要按题目约定说明。二叉树层序遍历的额外队列空间是最大层宽，不是常数；岛屿数量如果原地改网格，可以省掉 `visited`，但代价是破坏输入；如果题目要求保留输入，就需要额外布尔矩阵。分析空间时要写清“额外空间”和“包含输出的空间”分别是什么。

暴力法的作用是把问题完整展开。两数之和暴力枚举所有下标对，最长无重复子串暴力枚举所有起点和终点，岛屿数量暴力从每个未访问陆地扩展连通块。暴力方法慢，但它通常是正确性的起点。

Listing 1.4 LeetCode 1 两数之和：暴力解

```
1 std::vector<int> two_sum_bruteforce(const std::vector<int>& nums, int target)
2 {
3     for (int left = 0; left < static_cast<int>(nums.size()); ++left) {
4         for (int right = left + 1; right < static_cast<int>(nums.size()); ++right) {
5             if (nums[left] + nums[right] == target) {
6                 return {left, right};
7             }
8         }
9     }
10    return {};
11 }
```

这段代码枚举的是下标对，数量约为 $n * (n - 1) / 2$ ，时间复杂度是 $O(n^2)$ 。它慢在每个数都要重新扫描后面的数。优化思路不是凭空出现，而是把“扫描后面所有数”改成“快速查询某个补数是否出现过”。这就是哈希表。

常见优化来源有六类。第一类是缓存重复计算，例如前缀和、记忆化搜索和动态规划。第二类是单调性，例如二分、双指针和某些滑动窗口。第三类是局部状态可维护，例如字符计数、窗口和、不同元素个数。第四类是候选淘汰，例如单调栈和单调队列。第五类是数据结构查询能力，例如哈希表、堆、平衡树、并查集。第六类是改变建模方式，例如把网格问题建成图，把区间问题排序，把字符串问题转为频次。

写完暴力法后，应该固定问几个问题：我枚举的对象是什么？有没有重复计算同一段信息？是否只需要历史信息的一小部分？是否存在有序或单调？是否可以把区间查询变成前缀差？是否可以用哈希把线性查询变成平均常数？是否可以把指数搜索合并成状态 DP？

面试时要把这些问题讲出来。一个好答案通常不是从“我用某某模板”开始，而是从暴力法、瓶颈和优化依据开始。

复杂度分析首先要分清“输入长度”和“输入数值”。数组长度为 n 时，扫描一遍是 $O(n)$ ；背包容量为 $target$ 时， $O(n \cdot target)$ 并不等于严格意义上的多项式时间，因为 $target$ 是数值大小，不是它在输入中占用的二进制位数。刷题中我们通常接受这种伪多项式复杂度，因为题目约束直接给出了容量上限；但面试表达里要知道它和 $O(n^2)$ 这类只依赖元素个数的复杂度不同。分割等和子集、零钱兑换、目标和都属于这种情况：如果数值总和很大，即使数组长度不大，DP 表也可能开不下。

复杂度还要看隐藏常数和内存访问。两个算法同为 $O(n)$ ，一个顺序扫描 `vector`，另一个沿链表随机跳转，实际速度可能差很多。原因在于 CPU 缓存和内存局部性。顺序数组能让硬件预取连续内存，链表每个节点可能分散在堆上，访问下一个节点前必须等当前节点指针加载完成。这不改变大 O 记号，但会影响工程判断。刷题时大 O 是第一道门槛；写可维护的 C++ 程序时，还要知道容器布局、分配次数和临时对象成本。

深入理解

大 O 记号描述的是增长趋势，不是精确运行时间。它会忽略常数、低阶项和机器细节，所以不能用它解释所有性能差异。比如 `std::sort` 的 $O(n \log n)$ 在中等规模输入上可能比一个写得很差的 $O(n)$ 哈希方案更快，因为排序是连续内存扫描，哈希表可能频繁分配、冲突和 rehash。反过来，当输入规模足够大，数量级差距会压倒常数。高质量答案要同时具备两层意识：先用复杂度判断能不能过，再用实现成本判断写法是否稳。

暴力法不是低级答案，而是问题建模的显微镜。写暴力时，你会被迫明确“枚举对象”是什么：枚举子数组、枚举切分点、枚举路径、枚举排列、枚举状态，还是枚举答案。不同枚举对象决定了后续优化方向。枚举所有子数组时，优化往往来自前缀和或滑动窗口；枚举所有路径时，优化可能来自 BFS、Dijkstra、记忆化或剪枝；枚举所有排列时，优化可能来自排序去重、位掩码状态压缩或贪心证明；枚举答案时，若可行性单调，就可能二分答案。

以“和为 K 的子数组”为例，最朴素暴力枚举左右端点后再求和，三层成本是 $O(n^3)$ 。第一步优化是把求和缓存成前缀和，左右端点枚举仍然存在，但区间和查询变成 $O(1)$ ，总复杂度降到 $O(n^2)$ 。第二步优化是换一个枚举对象：不再枚举所有左端点，而是在扫描右端点时，查询历史中有多少前缀和等于当前前缀减 k 。这一步把“寻找左端点”交给哈希表，复杂度降到平均 $O(n)$ 。

Listing 1.5 LeetCode 560 和为 K 的子数组：从前缀和到哈希计数

```
1 int subarray_sum_equals_k(const std::vector<int>& nums, int k)
2 {
3     std::unordered_map<int, int> prefix_count;
4     prefix_count.reserve(nums.size() * 2);
5     prefix_count[0] = 1;
6
7     int prefix = 0;
8     int answer = 0;
9     for (const int x : nums) {
10         prefix += x;
11         const auto found = prefix_count.find(prefix - k);
12         if (found != prefix_count.end()) {
13             answer += found->second;
14         }
15         ++prefix_count[prefix];
16     }
17     return answer;
```

```
18 }
```

这段代码背后的数学关系是：若 `prefix[right] - prefix[left] == k`，那么 `prefix[left] == prefix[right] - k`。扫描到当前右端时，所有可能的左端都在历史前缀里，所以查询历史计数即可。必须先查再更新当前前缀，否则当 `k == 0` 时会把长度为 0 的区间错误计入。这个例子说明，优化不是把代码写短，而是把问题从“枚举区间”改写成“查询历史状态”。

再看“长度最小的子数组”。如果数组全是正数，滑动窗口能成立，因为右端右移只会让窗口和增加，左端右移只会让窗口和减少。这个单调性让每个元素最多进入窗口一次、离开窗口一次，总复杂度 $O(n)$ 。如果数组允许负数，单调性消失：右端加入负数可能让和变小，左端删除负数可能让和变大。此时同样的窗口代码会漏解。一个成熟的答案必须说明算法成立的输入条件，而不是看到“子数组”就写窗口。

常见误区

很多错误来自把“题型标签”当作证明。看到连续子数组就写滑动窗口，看到最短就写 BFS，看到最优就写 DP，看到局部最大就写贪心，都是危险的。真正的问题是：窗口是否有单调性？边权是否相等或非负？状态是否无后效？局部选择是否有交换论证？如果这些条件不存在，模板就会变成错误答案。

摊还复杂度也是刷题必须掌握的工具。单调栈和单调队列里，某一次循环可能弹出很多元素，看起来像嵌套循环，似乎是 $O(n^2)$ 。但每个元素一生只会入栈一次、出栈一次，所以所有弹出操作加起来最多 n 次，总复杂度是 $O(n)$ 。摊还分析不是平均随机情况，而是把一组操作的总成本平摊到每个元素上。`vector::push_back` 也常用摊还分析：多数插入是常数时间，偶尔扩容搬移很多元素；如果容量按倍数增长，总搬移次数仍然是线性级别。

Listing 1.6 单调栈摊还分析的代码形态

```
1 std::vector<int> next_greater_elements(const std::vector<int>& nums)
2 {
3     std::vector<int> answer(nums.size(), -1);
4     std::vector<int> stack;
5     stack.reserve(nums.size());
6
7     for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
8         while (!stack.empty() && nums[i] > nums[stack.back()]) {
9             const int index = stack.back();
10            stack.pop_back();
11            answer[index] = nums[i];
12        }
13        stack.push_back(i);
14    }
15
16    return answer;
17 }
```

内层 `while` 不是复杂度灾难，因为被弹出的下标不会再次入栈。面试时可以直接说：每个下标至多被压入一次、弹出一次，因此所有 `while` 循环的总迭代次数是 $O(n)$ 。这种解释比简单写“单调栈 $O(n)$ ”更有说服力。

递归搜索的复杂度要看搜索树。全排列的叶子数是 $n!$ ，子集的叶子数是 2^n ，组合总和的搜索树取决于目标值和候选数。剪枝能减少实际访问节点，但通常不改变最坏复杂度，除非剪枝来自强约束或状态合并。记忆化搜索的关键是把搜索树中的重复节点合并成状态图。动态规划表的大小，就是不同状态的数量；每个状态的转移成本乘上状态数，就是总复杂度。

以爬楼梯为例，暴力递归 $f(n) = f(n-1) + f(n-2)$ 会重复计算大量子问题。 $f(5)$ 需要 $f(4)$

和 $f(3)$ ，而 $f(4)$ 又需要 $f(3)$ ，同一个 $f(3)$ 被反复计算。记忆化把每个 $f(i)$ 只算一次，复杂度从指数级降到 $O(n)$ 。表格 DP 只是把“递归加缓存”改成按顺序填表。二者思想相同，表达形式不同。

Listing 1.7 LeetCode 70 爬楼梯：从重递归到滚动状态

```

1 int climb_stairs(int n)
2 {
3     if (n <= 2) {
4         return n;
5     }
6
7     int previous_two = 1;
8     int previous_one = 2;
9     for (int step = 3; step <= n; ++step) {
10         const int current = previous_one + previous_two;
11         previous_two = previous_one;
12         previous_one = current;
13     }
14     return previous_one;
15 }

```

这里的状态依赖只有前两个值，所以不需要保存整个数组。空间优化的前提是你已经知道未来不会再访问更早状态。背包 DP 的一维滚动数组也是这个道理，但它更容易错，因为遍历方向会决定当前物品是否被重复使用。空间优化不是机械删除一维，而是重新检查转移读取的是上一阶段还是当前阶段。

二分答案的复杂度由“答案范围”和“检查函数”共同决定。爱吃香蕉的速度范围是 $[1, \text{maxPile}]$ ，每次检查扫描所有堆，因此复杂度是 $O(n \log M)$ 。分割数组最大值的答案范围是 $[\text{maxElement}, \text{sum}]$ ，检查函数贪心扫描数组，因此复杂度是 $O(n \log S)$ 。如果检查函数本身是 $O(n \log n)$ ，总复杂度就会多一个因子。分析二分题时，不要只写 $O(\log n)$ ，因为很多二分不是在数组长度上二分。

Listing 1.8 LeetCode 1011 在 D 天内送达包裹：容量越大越容易可行

```

1 int ship_within_days(const std::vector<int>& weights, int days)
2 {
3     int left = *std::max_element(weights.begin(), weights.end());
4     int right = std::accumulate(weights.begin(), weights.end(), 0);
5
6     auto can_ship = [&](int capacity) {
7         int used_days = 1;
8         int current_load = 0;
9         for (const int weight : weights) {
10             if (current_load + weight > capacity) {
11                 ++used_days;
12                 current_load = 0;
13             }
14             current_load += weight;
15         }
16         return used_days <= days;
17     };
18
19     while (left < right) {
20         const int mid = left + (right - left) / 2;

```

```
21     if (can_ship(mid)) {
22         right = mid;
23     } else {
24         left = mid + 1;
25     }
26 }
27 return left;
28 }
```

这个题的检查函数也需要证明。给定容量上限时，按顺序尽量把包裹装到当天，装不下再开新一天，是最少天数策略。因为提前开新一天不会让当前天装得更多，只会浪费容量。容量越大，所需天数不会增加，所以可行性具有单调性。二分答案必须同时具备这两个条件：检查函数正确，答案可行性单调。

空间复杂度也不能只看“额外数组”。递归调用栈是空间，哈希表桶和节点是空间，输出答案是否计入空间要按题目约定说明。比如三数之和，如果不计输出，排序后的双指针额外空间可以说是 $O(1)$ 或排序栈空间；如果计输出，最坏可能有 $O(n^2)$ 个三元组。BFS 的队列空间不是 $O(1)$ ，它可能达到图的最大宽度；二叉树层序遍历的队列空间是最大层宽。

算法优化还要避免过度优化。对于输入 $n \leq 100$ 的题，清晰的 $O(n^3)$ DP 可能比复杂的优化更适合面试；对于 $n \leq 2 \times 10^5$ 的题， $O(n^2)$ 通常不可接受，必须寻找排序、堆、哈希、线段树或单调结构。判断一段代码能否通过，先做粗略预算：一秒级评测里，一千万到一亿级简单操作可能可接受，十亿级通常危险。这个数字不是绝对标准，但能帮助你快速排除明显不合适的方案。

复杂度实验：第一，把两数之和的暴力版和哈希版分别跑在一万、十万规模随机数组上，记录时间差。第二，把和为 K 的子数组写成三层、前缀和两层、哈希一层三个版本，观察复杂度如何影响增长曲线。第三，用 `vector` 和 `list` 分别顺序遍历一百万个整数，比较实际时间，理解缓存局部性。第四，用单调栈统计每个元素入栈出栈次数，验证摊还 $O(n)$ 。

复杂度推理的面试表达

先给暴力法，让问题完整展开；再指出慢点来自重复计算、重复扫描、候选过多还是状态爆炸；然后说明优化依据是缓存、单调性、候选淘汰、数据结构查询能力还是重新建模；最后给出时间、空间、成立条件和反例。能完整说出这条链，比直接背最优解更有价值。

第零部分

数据结构：算法能力的骨架

第 2 章

线性数据结构：数组、字符串、链表、栈和队列

数据结构不是算法之前的一张 API 表，而是算法能够成立的物理基础。刷题时遇到一个问题，第一步不是急着套模板，而是判断答案依赖什么信息：需要按位置访问、按最近关系回退、按先进先出扩展，还是只在两端维护候选。数组、字符串、链表、栈、队列和双端队列构成了这条主线。它们的抽象接口很简单，但背后的存储方式、缓存局部性、迭代器失效规则和边界条件，决定了暴力方法能不能被优化成面试可接受的复杂度。

线性结构的共同特点是元素被组织成一条序列。区别在于这条序列是否连续存储，是否支持下标随机访问，插入删除发生在什么位置，算法状态如何随着左右边界移动而更新。真正掌握这一章，意味着看到题目时可以回答四个问题：序列是否固定，窗口是否连续，历史状态能否增量维护，删除或回退是否只发生在某一端。

从能力边界看，**vector** 负责连续存储、随机访问和顺序扫描，是 DP 表、排序数组、邻接表和双指针题的默认选择；**string** 负责字符序列和子串窗口，但要警惕 **substr** 的复制成本；链表负责节点重连，适合反转、合并、环检测和快慢指针；**stack** 负责最近未解决状态，常用于括号、表达式、路径和单调栈；**queue** 负责按层扩展，是 BFS 的自然结构；**deque** 负责两端维护候选，尤其适合滑动窗口极值。把这些结构先放在同一张能力地图里，后面每个算法就不会变成孤立模板。

数据结构如何决定算法边界

学习数据结构时，不能只背 API 名字和复杂度表。真正要问的是：数据放在哪里，访问路径有多长，修改会影响哪些对象，迭代器或指针是否稳定，状态能否按题目需要增量维护。算法优化往往不是突然少写一层循环，而是把一个昂贵动作换成更合适的数据结构动作。线性查找被哈希查询替代，重复区间求和被前缀和替代，反复找最大值被堆或单调队列替代，反复判断连通性被并查集替代。数据结构给算法提供的是“可快速回答的问题集合”。

复杂度表要和前提一起读。数组下标访问是 $O(1)$ ，前提是你已经有合法下标，并且数组存储连续；链表已知节点后插入删除是 $O(1)$ ，前提是已经拿到节点或前驱，查找位置本身仍然可能是 $O(n)$ ；哈希表平均查询是 $O(1)$ ，前提是哈希分布合理并且负载因子受控；堆取堆顶是 $O(1)$ ，但删除任意元素并不是普通 **priority_queue** 的能力。只背结论而不背前提，遇到变体就会选错结构。

内存布局会改变常数和工程风险。**vector** 的元素连续，顺序扫描通常很快，但扩容会搬迁元素；链表节点分散，重连便宜，但遍历有指针追踪成本；**deque** 分段连续，两端插入删除稳定，但随机访问比 **vector** 多一层寻址；哈希表节点常常独立分配，能快速定位 key，却牺牲局部性；树结构能维护顺序，但每次查找要沿指针走多层。面试题通常关注渐进复杂度，工程题还要看分配次数、缓存局部性、对象生命周期和失效规则。

容器选择要从访问模式倒推。若题目主要按下标读取、排序、前后缀扫描，优先用 **vector**；若需要稳定地把已知节点移动到头尾，才考虑链表；若需要最近未匹配对象，用栈；若需要按距离层次扩展，用队列；若需要窗口内最大最小值，用双端队列维护单调候选；若需要按 key 找历史状态，用哈希表；若需要前驱、后继和有序区间，用有序树。不要因为某个容器“看起来高级”就使用它，结构越复杂，维护不变量和边界的成本越高。

递归和迭代也属于数据结构选择。递归把待处理状态交给语言调用栈，代码短，但深度不可控；迭代把状态放进显式栈、队列或工作表，代码略长，却能控制内存和遍历顺序。二叉树遍历、图搜索、回溯和 DP 都可以从这个角度理解：递归不是算法本身，栈帧里保存的节点、下标、路径、剩余目标和访问标记才是算法状态。掌握这些状态后，递归版和迭代版就能互相转换。

数据结构学习还要带着变体意识。数组可以扩展成前缀和、差分、树状数组和线段树；链表可以扩展成双向链表、循环链表、侵入式链表和跳表；栈和队列可以扩展成单调栈、单调队列、双端队列和优先队列；哈希可以扩展成频次表、状态去重、规范 key 分组和前缀状态表；树可以扩展成 BST、堆、Trie、平衡树和区间树。每个变体都对应一个新的查询能力，不是为了记更多名词。

数据结构题的自查

看到一个结构，先写它能快速回答什么问题，不能快速回答什么问题；再写它的内存布局、修改成本和失效规则；最后把题目里的操作频率和结构能力对齐。若题目需要的查询不是这个结构擅长的，再熟悉的 API 也不应该硬用。

数组和 `std::vector` 是刷题默认容器。连续存储带来两个关键优势：下标访问是 $O(1)$ ，相邻元素通常落在相近内存位置，CPU 缓存命中率高。很多初学者只记住第一个优势，忽略第二个优势。实际程序中，同样是线性扫描，`vector` 往往比链表快得多，并不是因为复杂度写法不同，而是因为连续内存更容易被硬件预取。代价也来自连续存储：中间插入和删除必须移动后续元素；容量不足时扩容会重新申请一段更大的空间，把旧元素移动或复制过去，原来的指针、引用和迭代器都可能失效。

从内存角度看，一个 `vector<int>` 的元素区是一段连续的 `int` 对象，`data()` 指向第一个元素，`data() + i` 指向第 `i` 个元素。对象本身通常只保存三个信息：起始位置、当前结束位置、容量结束位置。`size()` 描述已经构造的元素个数，`capacity()` 描述当前内存最多还能容纳多少元素。这个模型解释了四个常见行为：下标访问能直接做地址计算；尾部追加在容量足够时很快；中间插入要移动后续元素；扩容会换一整段内存，旧地址不能再用。

vector 操作表

操作	复杂度	原理说明
下标读取	$O(1)$	起始地址加偏移即可定位元素。
尾部追加	摊还 $O(1)$	容量足够时原地构造，容量不足时整体搬迁。
中间插入	$O(n)$	插入点之后的元素要整体后移。
中间删除	$O(n)$	删除点之后的元素要整体前移。
顺序遍历	$O(n)$	每个元素访问一次，缓存局部性通常很好。

Listing 2.1 `vector` 的容量、大小和失效规则

```
1 std::vector<int> nums;
2 nums.reserve(n);           // 只预留容量，不构造元素，size 仍然是 0。
3 nums.push_back(10);
4 nums.emplace_back(20);
5
6 std::vector<int> dp(n + 1, 0); // 直接创建 n + 1 个元素。
7
8 const int* first = nums.data();
9 nums.push_back(30);         // 如果触发扩容，first 立刻失效。
```

`reserve` 和 `resize` 必须分清。`reserve` 只改变容量，不构造元素，适合已经知道将要追加多少元素的场景；`resize` 改变大小，会构造或销毁元素，适合直接创建 DP 数组、计数数组或矩阵行。写 `vector<int> dp(n + 1)` 后，`dp[i]` 是合法的；只写 `reserve(n + 1)` 后访问 `dp[i]` 是越界。面试里这类错误很隐蔽，因为代码可能在小样例上运行，到了评测环境才崩溃。

数组算法的基本动作只有三类：按位置读取、按顺序扫描、按区间聚合。按位置读取支撑二分、双指针、DP 表索引；按顺序扫描支撑前后缀、计数、最大最小值维护；按区间聚合支撑前缀和、差分数组 and 滑动窗口。不要把这些动作混成一个“数组题模板”。例如前缀和解决的是重复区间求和，差分数组解决的是多次区间修改，滑动窗口解决的是相邻区间状态可增量维护。三者都用数组，但原理完全不同。

Listing 2.2 差分数组：把区间修改变成端点修改

```
1 std::vector<int> apply_range_additions(
2     int n,
```

```

3     const std::vector<std::array<int, 3>>& updates)
4 {
5     std::vector<int> diff(n + 1, 0);
6     for (const auto& update : updates) {
7         const int left = update[0];
8         const int right = update[1];
9         const int value = update[2];
10        diff[left] += value;
11        if (right + 1 < n) {
12            diff[right + 1] -= value;
13        }
14    }
15
16    std::vector<int> result(n, 0);
17    int current = 0;
18    for (int i = 0; i < n; ++i) {
19        current += diff[i];
20        result[i] = current;
21    }
22    return result;
23 }

```

差分数组和前缀和互为逆操作。前缀和把原数组累加成区间查询工具；差分数组把区间修改记录成端点变化，最后再累加还原。暴力做法对每次区间修改都逐个元素加值，若有 q 次修改和 n 个元素，最坏是 $O(qn)$ 。差分写法每次修改只动两个端点，所有修改后扫描一次数组，总复杂度是 $O(q + n)$ 。这个优化不是代码技巧，而是把“区间内每个位置都加 value”改写成“从 left 开始多 value，从 right 后面撤销 value”。

数组题的暴力方法通常是枚举一个区间、一个配对或一个分割点。优化的关键，是让“下一步答案”复用“上一步状态”。例如区间和暴力做法是每次重新累加 `nums[left..right]`，复杂度 $O(n^3)$ 或 $O(n^2)$ ；前缀和把任意区间和转成两个端点差值，使查询变成 $O(1)$ 。滑动窗口则在区间右端加入一个元素、左端删除一个元素，用增量更新维护窗口状态。它们都依赖数组按下标稳定访问。

Listing 2.3 从重新求和到前缀和查询

```

1 std::vector<long long> build_prefix_sum(const std::vector<int>& nums)
2 {
3     std::vector<long long> prefix(nums.size() + 1, 0);
4     for (std::size_t i = 0; i < nums.size(); ++i) {
5         prefix[i + 1] = prefix[i] + nums[i];
6     }
7     return prefix;
8 }
9
10 long long range_sum(const std::vector<long long>& prefix,
11                     std::size_t left,
12                     std::size_t right)
13 {
14     return prefix[right + 1] - prefix[left];
15 }

```

这段代码里 `prefix[i]` 表示前 i 个元素之和，因此区间 `[left, right]` 的和是 `prefix[right + 1] - prefix[left]`。多出来的第 0 位看似绕，但它消除了 `left == 0` 的特殊判断。很多高质量题解都会主动设计这种哨兵位，因为它能让边界条件服从同一条公式。

字符串 `std::string` 可以看成字符数组，但不能只按字符数组理解。第一，`substr` 会创建新字符串，频繁调用会引入大量复制；第二，`size()` 返回字节数，不等于所有自然语言里的“字符数”；第三，题目如果限定小写英文，计数结构可以用 `std::array<int, 26>`，如果字符集不固定，就应使用哈希表。刷题常见输入大多是 ASCII 或小写英文，面试时仍然要主动说明自己的假设。

C++ 的 `std::string` 是拥有字符存储的容器，常见实现会把字符连续放置，并在末尾保留空字符以便兼容 C 风格接口。现代标准库还常用小字符串优化：短字符串直接存在对象内部，不必单独分配堆内存；长字符串才指向堆上的字符区。标准不要求具体的小字符串容量，所以不能写依赖某个阈值的程序，但这个模型能帮助理解为什么短字符串复制有时不慢、长字符串频繁复制却会明显拖慢热循环。

字符串算法要先问清字符集。若题目明确只有小写英文字母，`std::array<int, 26>` 比哈希表更直接，空间固定，访问成本稳定；若包含大小写字母和数字，可以扩展到 128 或 256 个 ASCII 位置；若是 Unicode 文本，`std::string::size()` 统计的是字节数，不是用户眼里的字符数。算法题通常简化成 ASCII 或小写英文，本书的代码也会按题目约束选择最简单结构。工程代码处理自然语言时，必须另外考虑编码、规范化和字素簇，这已经超出普通刷题范围。

字符串操作的成本

操作	成本	注意点
下标访问	$O(1)$	访问的是字节位置，不一定是自然语言字符。
<code>push_back</code> <code>substr</code>	摊还 $O(1)$ $O(k)$	可能扩容，旧指针和视图可能失效。 创建长度为 k 的新字符串，可能分配和复制。
拼接 比较	取决于长度 最坏 $O(k)$	循环里反复 <code>+</code> 可能造成多次分配。 从左到右比较，遇到不同字符才停止。

Listing 2.4 异位词窗口：避免反复 `substr`

```

1 std::vector<int> find_anagrams(const std::string& s, const std::string& p)
2 {
3     constexpr int ALPHABET_SIZE = 26;
4     std::vector<int> answer;
5     if (s.size() < p.size()) {
6         return answer;
7     }
8
9     std::array<int, ALPHABET_SIZE> need{};
10    std::array<int, ALPHABET_SIZE> window{};
11    for (const char ch : p) {
12        ++need[ch - 'a'];
13    }
14
15    for (std::size_t right = 0; right < s.size(); ++right) {
16        ++window[s[right] - 'a'];
17        if (right >= p.size()) {
18            --window[s[right - p.size()] - 'a'];
19        }
20        if (window == need) {
21            answer.push_back(static_cast<int>(right + 1 - p.size()));
22        }
23    }
24    return answer;
25 }

```


暴力版本会枚举每个长度为 `p.size()` 的子串，复制出来再排序或统计。复制子串本身就是 $O(k)$ ，排序还要 $O(k \log k)$ 。窗口版本只维护 26 个计数，右端进入一个字符，左端离开一个字符，每步常数更新。优化成立的原因不是“用了滑动窗口”这个名词，而是题目只关心固定长度窗口内的字符频次，窗口相邻状态只差两个字符。

字符串匹配和字符串 DP 中，`substr` 成本经常被复杂度分析漏掉。单词拆分的朴素 DP 可能写成枚举 `left` 和 `right`，然后用 `s.substr(left, right-left)` 去哈希集合里查。表面上有两层循环，似乎是 $O(n^2)$ 次查询；但每次构造子串要复制长度为 `k` 的内容，最坏总复制量会把实际成本推高。优化方式不一定一开始就上字典树，可以先限制最大单词长度，再考虑 `string_view`、滚动哈希或字典树。选择哪种结构取决于题目规模，而不是为了显得复杂。

Listing 2.5 用下标比较避免创建临时子串

```
1 bool equals_at(std::string_view text,
2               int start,
3               std::string_view word)
4 {
5     if (start < 0 ||
6         start + static_cast<int>(word.size()) >
7             static_cast<int>(text.size())) {
8         return false;
9     }
10
11     for (int i = 0; i < static_cast<int>(word.size()); ++i) {
12         if (text[static_cast<std::size_t>(start + i)] !=
13             word[static_cast<std::size_t>(i)]) {
14             return false;
15         }
16     }
17     return true;
18 }
```

`string_view` 的作用是表达“我只观察这段字符，不拥有它”。它创建便宜，也不会复制子串；但它不延长原字符串生命周期。原字符串销毁、移动或重新分配后，已有视图就可能悬空。因此教材里会在局部比较、局部扫描场景使用视图；如果要把子串长期存进容器，仍然要存拥有型 `string`，或者保证底层字符存储生命周期覆盖所有视图。

链表是线性结构里最容易被误解的一个。教材常说链表插入删除是 $O(1)$ ，但这句话有前提：你已经拿到了要操作位置的前驱节点或当前节点。如果还需要从头查找位置，查找本身就是 $O(n)$ 。链表的价值不在于比数组更快，而在于节点位置稳定、局部重连便宜、结构可以自然表达“下一个”。面试链表题考的不是容器 API，而是指针不丢、边界不漏、连接顺序清楚。

链表的内存分布和数组相反。数组把元素挤在连续内存里，链表的每个节点可以分散在堆上，节点内部保存值和指向下一个节点的指针。单向链表节点至少有 `value` 和 `next`；双向链表还要保存 `prev`。这意味着链表节点通常比单个值更大，遍历时还会产生指针追逐：CPU 必须先读出当前节点地址里的 `next`，才能知道下一个节点在哪里。复杂度都是 $O(n)$ 时，链表遍历往往慢于数组遍历，原因就在缓存局部性和额外指针开销。

链表操作表

操作	复杂度	前提
按下标查找	$O(n)$	必须从头逐节点走。
已知前驱插入	$O(1)$	只改少数指针。
已知前驱删除	$O(1)$	单链表删除需要待删节点前驱。
反转整表	$O(n)$	每个节点指针改一次。
判断环	$O(n)$	快慢指针保持速度差。

Listing 2.6 迭代反转链表：先保存后继，再改变指向

```

1 struct ListNode {
2     int value = 0;
3     ListNode* next = nullptr;
4 };
5
6 ListNode* reverse_list(ListNode* head)
7 {
8     ListNode* previous = nullptr;
9     ListNode* current = head;
10
11     while (current != nullptr) {
12         ListNode* next = current->next;
13         current->next = previous;
14         previous = current;
15         current = next;
16     }
17
18     return previous;
19 }

```

这段代码只有四行核心操作，顺序却不能乱。先保存 `next`，因为一旦执行 `current->next = previous`，原来的后继就找不到了；再把当前节点接到已反转部分前面；最后整体向后推进。写链表题时，建议在纸上画三个指针：已经处理好的部分、当前节点、尚未处理的部分。只要这三段关系清楚，反转、两两交换、K 个一组反转都可以从同一套指针动作推出来。

链表反转也可以递归描述：先反转从第二个节点开始的后半段，再让第二个节点指回当前节点。这个描述很适合理解“子问题返回新的头节点”，但 C++ 实现时要警惕调用栈深度。长度很长的链表会让递归调用层数达到 n ，极端情况下可能栈溢出；迭代版只用三个指针，空间是 $O(1)$ 。因此本书讲递归含义，但默认落地为迭代代码。面试时如果写递归，也要主动说明栈空间是 $O(n)$ ，并确认题目规模不会让栈深失控。

Listing 2.7 迭代合并两个升序链表

```

1 ListNode* merge_two_sorted_lists(ListNode* first, ListNode* second)
2 {
3     ListNode dummy;
4     ListNode* tail = &dummy;
5
6     while (first != nullptr && second != nullptr) {
7         if (first->value <= second->value) {
8             tail->next = first;
9             first = first->next;
10        } else {
11            tail->next = second;
12            second = second->next;
13        }
14        tail = tail->next;
15    }
16
17    tail->next = (first != nullptr) ? first : second;
18    return dummy.next;
19 }

```


合并链表的关键不是创建新节点，而是重连已有节点。**tail** 永远指向结果链表最后一个节点，下一步只需把较小头节点接到 **tail->next**，再推进对应链表。暴力做法可以把所有值取出放进数组、排序、重建链表，但那会丢掉链表节点重连的本质，也引入额外空间。链表题的核心训练就是在不丢节点、不制造断链的前提下改变链接关系。

虚拟头节点是链表题的另一个重要技巧。删除倒数第 N 个节点、合并两个链表、分隔链表时，真实头节点可能被删除或替换。如果每次都单独判断头节点，代码会被边界分支切碎。虚拟头节点把“修改头节点”和“修改普通节点”统一成“修改某个前驱节点的 **next**”。

Listing 2.8 虚拟头节点统一删除逻辑

```

1 ListNode* remove_nth_from_end(ListNode* head, int n)
2 {
3     ListNode dummy;
4     dummy.next = head;
5
6     ListNode* fast = &dummy;
7     ListNode* slow = &dummy;
8
9     for (int step = 0; step < n; ++step) {
10         fast = fast->next;
11     }
12     while (fast->next != nullptr) {
13         fast = fast->next;
14         slow = slow->next;
15     }
16
17     ListNode* removed = slow->next;
18     slow->next = removed->next;
19     return dummy.next;
20 }

```

快慢指针的本质是让两个指针之间保持固定距离或固定速度差。删除倒数第 N 个节点时，快指针先走 N 步，之后快慢一起走；当快指针到尾部，慢指针正好停在待删节点前驱。环检测时，快指针每次走两步，慢指针每次走一步；如果存在环，速度差会让它们在环内相遇。不要把快慢指针背成模板，应该说清楚“两个指针之间的不变量是什么”。

栈是后进先出结构，适合处理“最近出现、最先解决”的关系。括号匹配中，最近的左括号必须先被右括号匹配；路径简化中，最近进入的目录会被 **..** 回退；表达式求值中，最近的运算符和操作数构成局部归约。栈的强大之处在于把嵌套结构压平成一条处理过程。

从实现角度看，栈不是一种必须单独分配节点的数据结构，而是一组受限操作：只能在一端压入、弹出和查看顶部。C++ 的 **std::stack** 是容器适配器，默认底层可用 **deque**；很多算法题直接用 **vector** 模拟栈，是因为 **vector** 可以 **reserve**，也可以在必要时查看下标。抽象上它们都是栈，工程上接口能力和内存布局不同。只需要 LIFO 语义时，用适配器表达意图；需要预留容量、遍历或保存下标时，用 **vector** 更灵活。

栈还可以看作递归调用栈的显式版本。递归 DFS 会把“当前节点、下一步要访问哪里、局部状态”存在语言运行时的调用帧里；显式栈则把这些信息放进自己定义的结构体或容器中。二者思想相同，但显式栈更容易控制内存、避免深度过大导致栈溢出，也更容易在循环里做剪枝和状态检查。本书给出显式栈，不是因为递归思想不重要，而是为了让 C++ 程序在大输入下更稳定。

Listing 2.9 用显式栈替代递归深度优先遍历

```

1 std::vector<int> preorder_iterative(TreeNode* root)
2 {
3     std::vector<int> order;

```

```

4     if (root == nullptr) {
5         return order;
6     }
7
8     std::vector<TreeNode*> stack{root};
9     while (!stack.empty()) {
10         TreeNode* node = stack.back();
11         stack.pop_back();
12         order.push_back(node->value);
13
14         if (node->right != nullptr) {
15             stack.push_back(node->right);
16         }
17         if (node->left != nullptr) {
18             stack.push_back(node->left);
19         }
20     }
21     return order;
22 }

```

前序遍历的递归写法是“访问当前节点，再访问左子树，再访问右子树”。显式栈要反过来先压右孩子、再压左孩子，因为栈后进先出，左孩子要先被弹出。这个例子说明，递归转换为迭代时，不能只把函数调用替换成 `push`；还要保持原先的访问顺序和局部状态语义。树、图、回溯都可以这样转换，只是栈帧里保存的信息会更丰富。

Listing 2.10 有效括号：最近未匹配结构

```

1 bool is_valid_parentheses(const std::string& s)
2 {
3     std::vector<char> stack;
4     stack.reserve(s.size());
5
6     for (const char ch : s) {
7         if (ch == '(' || ch == '[' || ch == '{') {
8             stack.push_back(ch);
9             continue;
10        }
11        if (stack.empty()) {
12            return false;
13        }
14
15        const char left = stack.back();
16        stack.pop_back();
17        const bool matched = (left == '(' && ch == ')') ||
18                             (left == '[' && ch == ']') ||
19                             (left == '{' && ch == '}');
20
21        if (!matched) {
22            return false;
23        }
24    }
25    return stack.empty();
26 }

```

这里直接使用 `std::vector<char>` 作为栈，比 `std::stack<char>` 更方便查看和预留容量。`std::stack` 是容器适配器，只暴露 `push`、`pop`、`top`、`empty` 等接口；`pop()` 不返回元素，因此必须先 `top()` 再 `pop()`。面试写代码时两者都可以，但要明白适配器牺牲了一部分访问能力，换来更明确的抽象边界。

单调栈是栈在刷题中的高频变体。它维护的不是时间顺序本身，而是一组“尚未找到答案的候选”。每日温度中，栈保存还没有找到更高温度的下标，并且这些下标对应的温度从栈底到栈顶单调不减。新温度到来时，只要它高于栈顶下标的温度，就可以确定栈顶答案；因为这是栈顶下标右侧第一个更高温度。

Listing 2.11 每日温度：单调栈保存下标

```
1 std::vector<int> daily_temperatures(const std::vector<int>& temperatures)
2 {
3     std::vector<int> answer(temperatures.size(), 0);
4     std::vector<int> stack;
5     stack.reserve(temperatures.size());
6
7     for (int day = 0; day < static_cast<int>(temperatures.size()); ++day) {
8         while (!stack.empty() && temperatures[day] > temperatures[stack.back()]) {
9             const int previous_day = stack.back();
10            stack.pop_back();
11            answer[previous_day] = day - previous_day;
12        }
13        stack.push_back(day);
14    }
15
16    return answer;
17 }
```

单调栈常保存下标而不是值，因为题目经常需要距离、宽度，或者需要回到原数组访问更多信息。它能把暴力枚举“右侧第一个更大元素”的 $O(n^2)$ 降到 $O(n)$ ，原因是每个下标最多入栈一次、出栈一次。这个摊还分析要会说：虽然某一次循环里可能弹出很多元素，但全局弹出次数不超过 n 。

队列是先进先出结构，适合按层扩展。BFS 的正确性来自队列顺序：先入队的是距离更短或时间更早的状态，因此第一次到达一个未访问状态时，就已经得到了最短步数。二叉树层序遍历、腐烂橘子、多源最短扩散、无权图最短路，都依赖这个性质。与栈不同，队列不会深挖某一条路径，而是把同一层的状态先处理完。

队列的核心不是“有一个容器”，而是“处理顺序等于进入顺序”。这条顺序带来两个重要结论。第一，在无权图最短路中，第一次出队的距离就是最短距离，因为所有更短路径对应的状态已经更早入队并处理。第二，访问标记通常要在入队时设置，而不是出队时设置；否则同一个状态可能被多个前驱重复入队，队列膨胀，甚至让层数统计错误。多源 BFS 只是把多个起点同时放入队列，它们共同构成第 0 层。

Listing 2.12 多源 BFS：所有源点同时进入第 0 层

```
1 int minutes_to_fill(std::vector<std::vector<int>>& grid)
2 {
3     constexpr int FRESH = 1;
4     constexpr int ACTIVE = 2;
5     constexpr std::array<std::pair<int, int>, 4> DIRECTIONS{
6         std::pair{1, 0}, std::pair{-1, 0},
7         std::pair{0, 1}, std::pair{0, -1}
8     };
9 }
```

```

10  const int rows = static_cast<int>(grid.size());
11  const int cols = rows == 0 ? 0 : static_cast<int>(grid[0].size());
12  std::queue<std::pair<int, int>> queue;
13  int fresh = 0;
14  for (int row = 0; row < rows; ++row) {
15      for (int col = 0; col < cols; ++col) {
16          if (grid[row][col] == ACTIVE) {
17              queue.push({row, col});
18          } else if (grid[row][col] == FRESH) {
19              ++fresh;
20          }
21      }
22  }
23
24  int minutes = 0;
25  while (!queue.empty() && fresh > 0) {
26      const int level_size = static_cast<int>(queue.size());
27      for (int i = 0; i < level_size; ++i) {
28          const auto [row, col] = queue.front();
29          queue.pop();
30          for (const auto [dr, dc] : DIRECTIONS) {
31              const int next_row = row + dr;
32              const int next_col = col + dc;
33              if (next_row < 0 || next_row >= rows ||
34                  next_col < 0 || next_col >= cols ||
35                  grid[next_row][next_col] != FRESH) {
36                  continue;
37              }
38              grid[next_row][next_col] = ACTIVE;
39              --fresh;
40              queue.push({next_row, next_col});
41          }
42      }
43      ++minutes;
44  }
45
46  return fresh == 0 ? minutes : -1;
47 }

```

这段代码刻意把所有初始源点一次性入队。若逐个源点单独 BFS，再取最小时间，就会重复扩展大量格子，也可能算错同时扩散的语义。队列里的每一层对应一分钟，`level_size` 固定了当前分钟能影响的所有状态；新增状态属于下一分钟，不能在同一层里继续无限传播。

Listing 2.13 层序遍历：队列保存当前边界

```

1  std::vector<std::vector<int>> level_order(TreeNode* root)
2  {
3      std::vector<std::vector<int>> levels;
4      if (root == nullptr) {
5          return levels;
6      }
7
8      std::queue<TreeNode*> queue;
9      queue.push(root);

```

```

10
11 while (!queue.empty()) {
12     const int level_size = static_cast<int>(queue.size());
13     std::vector<int> level;
14     level.reserve(level_size);
15
16     for (int i = 0; i < level_size; ++i) {
17         TreeNode* node = queue.front();
18         queue.pop();
19         level.push_back(node->value);
20
21         if (node->left != nullptr) {
22             queue.push(node->left);
23         }
24         if (node->right != nullptr) {
25             queue.push(node->right);
26         }
27     }
28     levels.push_back(std::move(level));
29 }
30
31 return levels;
32 }

```

这段代码里的 `level_size` 是关键。没有它，也可以 BFS，但很难知道哪些节点属于同一层。层序遍历、最短时间、多源扩散都需要把“当前层的节点数量”固定下来，然后处理完这一层再增加距离或分钟数。

`std::deque` 支持两端 $O(1)$ 插入删除，是 `std::queue` 的默认底层容器，也是单调队列的常用实现。滑动窗口最大值的暴力方法是每个窗口扫描一遍最大值，复杂度 $O(nk)$ 。如果用堆，需要处理过期下标。单调队列更贴合题意：队头永远是当前窗口最大值下标，队尾负责淘汰不可能再成为最大值的候选。

`deque` 的典型实现不是普通链表，而是分段连续数组。它用一个中控表指向若干固定大小的缓冲区，每个缓冲区内部连续，整体逻辑上形成一个可从两端增长的序列。这个结构解释了它的能力边界：两端插入删除快，随机访问可行但比 `vector` 多一层定位，整体缓存局部性通常不如单段连续的 `vector`。因此普通 DP 表、排序数组、前缀和数组仍应优先用 `vector`；需要频繁从头尾弹出时，才考虑 `deque`。

Listing 2.14 滑动窗口最大值：单调队列

```

1 std::vector<int> max_sliding_window(const std::vector<int>& nums, int k)
2 {
3     std::vector<int> answer;
4     if (nums.empty() || k <= 0) {
5         return answer;
6     }
7     answer.reserve(nums.size());
8
9     std::deque<int> deque;
10    for (int right = 0; right < static_cast<int>(nums.size()); ++right) {
11        while (!deque.empty() && nums[deque.back()] <= nums[right]) {
12            deque.pop_back();
13        }
14        deque.push_back(right);

```



```

15
16     const int left = right - k + 1;
17     if (deque.front() < left) {
18         deque.pop_front();
19     }
20     if (left >= 0) {
21         answer.push_back(nums[deque.front()]);
22     }
23 }
24
25 return answer;
26 }

```

单调队列的优化理由和单调栈相似。一个新元素如果大于等于队尾元素，那么队尾元素在未来任何包含新元素的窗口里都不可能成为最大值，因为它更小且更早过期。于是可以安全删除。每个下标最多进队一次、出队一次，总复杂度 $O(n)$ 。

理解 `vector` 的底层模型，可以解释很多看似奇怪的 C++ 行为。典型实现内部并不是保存“一个数组对象”，而是保存三个位置：起始位置、当前结束位置、容量结束位置。`size()` 等于当前结束减起始，`capacity()` 等于容量结束减起始。`push_back` 时，如果当前结束还没碰到容量结束，就在当前位置构造新元素并移动结束指针；如果容量满了，就申请更大的连续内存，把旧元素移动或复制过去，再释放旧内存。正因为扩容会换一整段内存，旧的指针、引用和迭代器才会失效。

Listing 2.15 观察 `vector` 扩容：地址变化意味着旧指针失效

```

1 std::vector<std::size_t> collect_capacity_changes(int count)
2 {
3     std::vector<int> values;
4     std::vector<std::size_t> capacities;
5     capacities.reserve(static_cast<std::size_t>(count));
6
7     std::size_t last_capacity = values.capacity();
8     for (int i = 0; i < count; ++i) {
9         values.push_back(i);
10        if (values.capacity() != last_capacity) {
11            capacities.push_back(values.capacity());
12            last_capacity = values.capacity();
13        }
14    }
15
16    return capacities;
17 }

```

这段实验代码不是算法题答案，而是学习容器的办法。你可以打印每次容量变化，观察容量通常按倍数增长。标准并不要求具体增长倍数，不同标准库实现可能不同，但“容量不足时整体搬迁”这个语义是理解失效规则的关键。刷题时，如果你要保存 `vector` 中元素的地址，然后继续 `push_back`，就必须先 `reserve` 到足够容量，或者改用不会整体搬迁节点的结构。反过来，如果你只保存下标，扩容不会让下标语义失效，这也是很多算法保存下标而不是指针的原因。

`vector` 的摊还 $O(1)$ 插入也值得讲清楚。一次扩容可能移动很多元素，看起来很贵；但如果容量按倍数增长，元素一生被搬迁的次数是对数级，总搬迁成本在一串 `push_back` 操作中可以摊成线性。因此连续追加 n 个元素的总成本仍然是 $O(n)$ 。这并不意味着每一次 `push_back` 都是严格常数，也不意味着扩容时没有延迟尖峰。面试算法一般关心摊还复杂度，工程系统如果关心实时延迟，就要提前预留容量或使用其他结构。

常见误区

`reserve` 不是 `resize`。前者只改变容量，不创建可访问元素；后者改变大小，会构造元素。写 `values.reserve(n)` 后访问 `values[i]` 是越界；写 `values.resize(n)` 或构造 `vector<int> values(n)` 后才可以按下标写。DP 表、计数数组、距离数组通常需要 `resize` 或直接构造；收集答案、邻接表追加边、构造结果列表通常适合 `reserve` 后 `push_back`。

`string` 的底层也不只是“字符数组”。现代标准库实现通常会使用小字符串优化：短字符串直接存在 `string` 对象内部，长字符串才分配堆内存。标准不规定小字符串容量，因此不能依赖某个具体长度，但这个优化解释了为什么短字符串拷贝有时比想象中便宜。即便如此，热循环中的 `substr` 仍然要谨慎，因为它会构造一个新 `string`，可能复制字符、分配内存，并参与哈希计算。单词拆分、回文切分、字符串匹配中，如果在双层循环里频繁 `substr`，实际成本可能比复杂度公式更高。

Listing 2.16 用 `string_view` 表达非拥有子串视图

```
1 bool starts_with_at(std::string_view text,
2                     int position,
3                     std::string_view pattern)
4 {
5     if (position < 0 ||
6         position + static_cast<int>(pattern.size()) >
7             static_cast<int>(text.size())) {
8         return false;
9     }
10
11     for (int i = 0; i < static_cast<int>(pattern.size()); ++i) {
12         if (text[static_cast<std::size_t>(position + i)] !=
13             pattern[static_cast<std::size_t>(i)]) {
14             return false;
15         }
16     }
17     return true;
18 }
```

`string_view` 不拥有字符，它只是指向已有字符串的一段视图，所以创建便宜，也不会复制内容。代价是生命周期风险：原字符串销毁或修改导致重分配后，视图可能悬空。刷题中，如果只是函数内部临时比较，`string_view` 很合适；如果要把视图存进容器并在原字符串之后继续使用，就必须非常小心。教材里保留一些 `substr` 写法，是为了让主算法清晰；源码阅读和工程优化时，再讨论如何减少复制。

`deque` 经常被误以为是链表。实际上，`deque` 通常是分段连续存储：有一个中控表保存若干固定大小缓冲区的地址，每个缓冲区内部连续。它支持两端高效插入删除，因为在头尾缓冲区还有空间时，只移动段内指针；空间不够时，分配新缓冲区并挂到中控表。它也支持随机访问，但要先定位缓冲区再定位段内偏移，局部性通常不如 `vector`。这解释了为什么滑动窗口最大值适合 `deque`，而普通 DP 表不应该随便用 `deque` 替代 `vector`。

`list` 是双向链表，节点地址稳定，已知迭代器时插入删除是常数时间。它的缺点也明显：每个节点单独分配，额外保存前后指针，缓存局部性差，不支持下标随机访问。绝大多数刷题场景中，`list` 不是首选。LRU 缓存是少数非常适合 `list` 的例子，因为我们需要在已知迭代器的情况下，把任意节点移动到表头，并在容量满时删除表尾。这个需求正好匹配 `list::splice` 的节点重连能力。

Listing 2.17 `list::splice`：节点移动而不是复制元素

```
1 void move_key_to_front(std::list<int>& keys,
2                       std::list<int>::iterator position)
```

```

3 {
4     keys.splice(keys.begin(), keys, position);
5 }

```

`splice` 不复制元素，也不重新分配节点，只改变链表链接关系。LRU 中哈希表保存 key 到链表迭代器，链表维护新旧顺序；访问一个 key 时，用迭代器把节点移动到表头。若用 `vector` 保存顺序，移动到表头会搬移大量元素，且保存的迭代器会失效；若只用哈希表，又无法知道谁最久未使用。这个案例说明，容器不是按“哪个复杂度低”孤立选择，而是按“多个操作是否同时满足约束”来选择。

栈和队列在 C++ 中通常是容器适配器。`std::stack` 默认底层是 `deque`，只暴露栈需要的接口；`std::queue` 默认底层也是 `deque`，只暴露队列接口。适配器让代码表达意图更明确，但也限制访问能力。例如 `std::stack` 不能遍历，也不能 `reserve`。刷题中如果需要遍历、预留容量、查看任意位置，直接用 `vector` 或 `deque` 模拟栈/队列更方便；如果只需要标准栈队列操作，适配器能让语义更干净。

迭代器失效规则是容器原理的考试点。`vector` 扩容会让所有迭代器、指针和引用失效；未扩容时，尾部插入通常只影响尾后迭代器，中间插入删除会影响插入点之后的位置。`deque` 两端插入删除的失效规则比 `vector` 复杂，不应长期保存它的迭代器做复杂操作。`list` 删除某个节点只让该节点迭代器失效，其他节点仍稳定。`unordered_map` rehash 会让迭代器失效，但指向元素的引用在很多实现中可能仍稳定；标准语义和具体实现细节要区分，不要在可移植代码里依赖实现偶然行为。

深入理解

源码阅读建议从行为反推结构。读 `vector`，先找内部表示大小和容量的成员，再跟 `push_back` 的容量足够路径和容量不足路径；读 `string`，先找短字符串和长字符串的分支；读 `deque`，先找段表和段大小；读 `list`，先找节点结构和链接操作；读 `stack` 和 `queue`，先确认它们只是包装底层容器。不要一开始陷进 `allocator` 和模板特化细节，先抓住数据如何存、操作如何移动、什么时候失效。

线性容器源码实验：第一，连续 `push_back` 并打印 `vector` 的 `capacity()` 和 `data()`，观察扩容时地址变化。第二，把一段单词拆分代码中的 `substr` 次数统计出来，再改成下标比较，比较运行时间。第三，用 `deque` 和 `vector` 分别实现滑动窗口最大值，解释为什么 `vector` 不适合从头部弹出。第四，实现一个最小 LRU，只支持 `get` 和 `put`，强制自己说明哈希表和链表各自负责什么。

线性结构选型

需要稳定下标和顺序扫描，优先 `vector`；需要字符窗口，使用 `string` 加计数结构，不要频繁 `substr`；需要局部重连，考虑链表并画清楚前驱、当前、后继；需要最近未解决状态，用栈；需要按层或最短步数扩展，用队列；需要窗口两端同时维护候选，用双端队列。选择结构时同时说出它让哪一步查询、删除或更新变快。

本章建议配套练习：数组与前缀和做 303、304、560；字符串窗口做 3、438、567、76；链表做 19、21、24、141、142、206、92、25；栈做 20、71、150、155、739、84；队列和双端队列做 102、199、239、994。复盘时不要只写最终代码，要写三行说明：暴力方法慢在哪里，数据结构保存了什么状态，为什么每个元素只被处理有限次。

线性结构的教材级展开

线性数据结构看起来简单，实际最容易被学薄。数组、字符串、链表、栈、队列和双端队列不是几组 API 名称，而是不同访问模式的承诺：数组承诺按下标常数访问，链表承诺已知节点附近的局部重连，栈承诺只处理最近未解决对象，队列承诺按进入顺序扩展，双端队列承诺两端候选都能快速进出。算法题里的“优化”常常就是把原本昂贵的访问模式换成某个结构天然支持的访问模式。经典教材会反复强调这一点，因为如果只会背容器名字，遇到变体题时就无法判断模板为什么失效。

学习线性结构时可以用三条线索串起来。第一条是物理存储：元素是否连续，是否会整体搬迁，

节点地址是否稳定，是否能被 CPU 预取。第二条是抽象接口：能不能随机访问，能不能从中间删除，能不能只看一端，能不能同时维护两端。第三条是算法不变量：慢指针左侧是什么，栈内保存什么未解决对象，队列当前层表示什么距离，窗口左端为什么可以丢弃。真正掌握数据结构，就是能把这三条线索同时讲清楚。

访问模式先于容器名称

选择容器前先问访问模式：按下标频繁访问，优先连续数组；只在尾部追加且要排序或 DP，优先 `vector`；需要头尾都弹入弹出，考虑 `deque`；需要已知节点处常数移动，考虑链表；需要最近未匹配结构，用栈；需要按层扩展，用队列。容器不是越复杂越好，而是越贴合访问模式越好。

数组和动态数组：连续内存、下标语义和原地维护

数组的核心能力是“位置稳定且可计算”。第 i 个元素的位置可以由起始地址加上 i 乘以元素大小得到，所以按下标访问是常数时间。这一性质让数组天然适合二分、前缀和、差分数组、排序、双指针、动态规划表和邻接表。数组的弱点也来自连续性：在中间插入或删除一个元素，后面的元素必须整体搬移；容量不足时，动态数组要申请新内存并移动已有元素。

`std::vector` 的概念模型可以理解成三个指针：起始位置、当前结束位置、容量结束位置。`size()` 来自当前结束位置减起始位置，`capacity()` 来自容量结束位置减起始位置。尾部追加时如果容量够，只在尾部构造一个元素；容量不够时，申请更大内存、移动旧元素、释放旧内存。这个过程解释了两个常见问题：为什么 `push_back` 的均摊复杂度是常数，为什么扩容后旧指针、引用、迭代器会失效。

数组算法要特别重视下标区间语义。左闭右闭区间 `[left, right]` 适合描述题目原始边界，左闭右开区间 `[left, right)` 更适合循环和切片，因为长度正好是 `right - left`，空区间表示为 `left == right`。前缀和通常使用左闭右开语义：`prefix[i]` 表示前 i 个元素之和，区间 `[left, right)` 的和就是 `prefix[right] - prefix[left]`。这种设计把下标零的特殊情况变成统一公式，是教材里非常重要的哨兵思想。

原地维护类题目本质是在数组上划分语义区间。删除有序数组重复项中，慢指针左侧是已经写好的唯一值前缀；颜色分类中，数组被划成零区、一号区、未知区和二区；移动零中，慢指针左侧是已经保留相对顺序的非零元素；下一个排列中，后缀是最长非递增区，枢轴和后缀重排决定字典序的下一步。题目不同，但证明方式相同：每一步循环只扩大已经正确的区域，未知区域严格缩小，被排除或交换出去的元素不会再破坏不变量。

先看颜色分类。题目只有三种颜色，最直接的暴力办法是排序，或者先扫描三遍分别统计 0、1、2 的数量，再按数量覆盖回数组。排序当然能得到结果，但它没有利用值域只有三类这个条件；计数覆盖虽然是线性的，却把题目变成“先统计再重写”，没有训练原地分区。进一步观察，扫描过程中真正需要维护的不是完整频次，而是四段区间：左边已经确定为 0，中间已经确定为 1，右边已经确定为 2，剩下的是未知区。于是优化自然变成荷兰国旗式的三指针维护，而不是直接背一段交换代码。

图示：颜色分类的四段区间

区间	语义	变化方式
<code>[0, next_red)</code>	已经确定为 0	只向右扩大
<code>[next_red, current)</code>	已经确定为 1	只向右扩大
<code>[current, next_blue]</code>	尚未检查	每轮缩小
<code>(next_blue, n)</code>	已经确定为 2	只向左扩大

Listing 2.18 颜色分类：四段区间不变量

```

1 void sort_colors(std::vector<int>& nums)
2 {
3     constexpr int RED = 0;
4     constexpr int WHITE = 1;

```



```

5     constexpr int BLUE = 2;
6
7     int next_red = 0;
8     int current = 0;
9     int next_blue = static_cast<int>(nums.size()) - 1;
10
11    while (current <= next_blue) {
12        if (nums[current] == RED) {
13            std::swap(nums[next_red], nums[current]);
14            ++next_red;
15            ++current;
16        } else if (nums[current] == BLUE) {
17            std::swap(nums[current], nums[next_blue]);
18            --next_blue;
19        } else {
20            ++current;
21        }
22    }
23 }

```

这段代码的关键不是三指针本身，而是区间解释： $[0, \text{next_red})$ 全是 0， $[\text{next_red}, \text{current})$ 全是 1， $[\text{current}, \text{next_blue}]$ 未知， $(\text{next_blue}, n)$ 全是 2。遇到 2 时不能推进 `current`，因为从右侧换回来的元素还没有检查；遇到 0 时可以推进，因为换回来的位置属于已经扫描过的一号区或就是当前。这个细节正是很多错误实现的来源。

数组变种还包括稀疏数组、位图、环形缓冲区和压缩坐标数组。位图把布尔集合压到二进制位，适合值域有限且只需要存在性的问题；环形缓冲区用固定数组和头尾下标模拟队列，适合实时系统和滑动窗口；离散化把大值域映射到连续小编号，适合树状数组、线段树和频次统计。刷题时常见的“坐标压缩”不是数学技巧，而是把无法直接开数组的值域转成数组能支持的下标访问。

字符串：字符序列、编码假设和子串成本

字符串是数组的特例，但题目里的字符串又比数组多几个坑。第一，字符集决定计数结构：小写英文可以用 26 长度数组，ASCII 可以用 128 或 256 长度数组，Unicode 则不能简单按字节计数。第二，`substr` 会构造新字符串，热循环中频繁使用会引入复制、分配和哈希成本。第三，字符串算法常常需要区分子串和子序列：子串必须连续，适合窗口、哈希、KMP 和中心扩展；子序列不要求连续，常用动态规划或贪心扫描。

固定长度窗口题的本质是相邻窗口只差一个进入字符和一个离开字符。找到所有字母异位词、排列包含、固定长度最大元音数都属于这一类。变长窗口题则依赖合法性可以通过移动左端恢复，例如最小覆盖子串中右端扩展增加覆盖能力，满足后移动左端尝试变短。若窗口状态不会随左右端单调变化，例如数组含负数且目标是区间和，就不能硬套滑动窗口。

字符串匹配类题目有多条路线。暴力匹配每个起点再逐字符比较，最坏可能 $O(nm)$ ；KMP 通过前缀函数避免主串指针回退；滚动哈希把子串比较转成哈希值比较，但要处理冲突；Trie 把多个模式串的公共前缀合并，适合单词搜索 II、前缀统计和自动补全。面试时不一定要写出所有高级算法，但必须能说明它们解决的是哪种重复工作。

排列包含是固定长度窗口的典型例子。题目问 `text` 中是否存在某个长度等于 `pattern.size()` 的子串，字符频次和 `pattern` 完全相同。暴力做法是枚举每个起点，截出长度为 `m` 的子串，再排序比较或重新统计频次。相邻两个候选子串只差左边移出的一个字符和右边进入的一个字符，如果每次从零统计，就把 `m` 个字符的绝大部分重复计算了。优化点因此很明确：窗口长度固定，右端每前进一步，只增量加入一个字符，并在窗口超过长度时移出一个字符；窗口计数等于需求计数时返回真。

图示：固定长度窗口的增量更新

旧窗口	<code>text[left ... right]</code> 中已经维护好 26 个字符计数。
右端进入	新字符 <code>text[right + 1]</code> 的计数加一。
左端离开	若窗口长度超过 <code>pattern.size()</code> ，旧字符 <code>text[left]</code> 的计数减一。
判断答案	窗口长度固定后，比较 <code>window == need</code> ，不用重新统计整段。

Listing 2.19 字符串窗口：用数组计数表达字符集假设

```

1 bool contains_permutation(const std::string& text, const std::string& pattern)
2 {
3     constexpr int ALPHABET_SIZE = 26;
4     if (text.size() < pattern.size()) {
5         return false;
6     }
7
8     std::array<int, ALPHABET_SIZE> need{};
9     std::array<int, ALPHABET_SIZE> window{};
10    for (const char ch : pattern) {
11        ++need[ch - 'a'];
12    }
13
14    for (std::size_t right = 0; right < text.size(); ++right) {
15        ++window[text[right] - 'a'];
16        if (right >= pattern.size()) {
17            --window[text[right - pattern.size()] - 'a'];
18        }
19        if (window == need) {
20            return true;
21        }
22    }
23    return false;
24 }

```

这段代码必须附带条件：输入只含小写英文字母。如果题目没有这个条件，数组下标 `ch - 'a'` 就没有意义。经典教材会反复强调“前提条件”，因为算法复杂度和正确性都依赖前提。把字符集换成 Unicode 后，代码结构仍然可以是窗口，但计数结构需要换成哈希表或更明确的编码处理。

链表：节点身份、局部重连和哨兵节点

链表的價值不在于“删除一定比数组快”，而在于节点身份稳定和局部重连便宜。已知一个节点或它的前驱时，插入删除只需要改少数指针；但如果要寻找第 k 个节点，仍然必须从头走 k 步。链表题考的是结构不变量：哪些节点已经接好，哪些节点还没有处理，当前节点的后继是否已经保存，操作后是否仍然能到达未处理部分。

单链表、双链表、循环链表和侵入式链表各有不同用途。单链表空间少，适合反转、合并、快慢指针；双链表能从节点向前向后移动，适合 LRU、删除已知节点和双端队列；循环链表适合轮转和约瑟夫类问题；侵入式链表把链接字段放进业务对象本身，Linux 内核大量使用这种方式来减少额外分配并精确控制对象生命周期。刷题通常使用平台给出的 `ListNode`，但理解这些变体能帮助你解释为什么 LRU 需要哈希表加双向链表。

哨兵节点是链表题的统一边界工具。删除头节点、合并空链表、分隔链表、反转前 k 个节点时，如果直接操作真实头节点，经常要写一堆特殊分支；引入虚拟头节点后，所有操作都变成“修改某个前驱的 `next`”。这跟前缀和多开一个 `prefix[0]` 是同一种思想：用一个额外状态把边界并入普

通逻辑。

两两交换节点如果从数组视角出发，暴力办法是把节点值复制到数组里，交换相邻值后再写回链表。这样能得到相同值序列，却没有真正交换节点，也无法迁移到要求保持节点身份的题。若坚持在链表上做，问题可以缩小到一个局部：前驱节点 `previous` 后面有两个待交换节点 `first` 和 `second`，再后面是下一段入口 `after`。优化不是减少时间复杂度，因为暴力重建也是线性；优化的是结构质量和可迁移性：只改三条边，保持已经处理好的前缀不动，未处理后缀仍然可达。

图示：两两交换的局部重连

交换前	<code>previous -> first -> second -> after</code>
第一条边	<code>previous -> second</code> ，让前缀接到交换后的头。
第二条边	<code>second -> first</code> ，完成这一对内部交换。
第三条边	<code>first -> after</code> ，把未处理后缀接回链表。
推进位置	<code>previous = first</code> ，因为 <code>first</code> 已成为已处理前缀的尾部。

Listing 2.20 两两交换链表节点：哨兵和局部三指针

```

1 ListNode* swap_pairs(ListNode* head)
2 {
3     ListNode dummy;
4     dummy.next = head;
5
6     ListNode* previous = &dummy;
7     while (previous->next != nullptr && previous->next->next != nullptr) {
8         ListNode* first = previous->next;
9         ListNode* second = first->next;
10        ListNode* after = second->next;
11
12        previous->next = second;
13        second->next = first;
14        first->next = after;
15
16        previous = first;
17    }
18    return dummy.next;
19 }

```

这段代码的局部不变量是：`previous` 之前的链表已经交换完成，`previous->next` 是当前待交换对的第一个节点，`after` 保存下一段入口。先保存 `after`，再改三条边，最后把 `previous` 移到这一对交换后的尾部。K 个一组反转也是同一模式，只是局部段更长，需要先检查是否有 k 个节点，再在段内反转并接回。

链表常见变种包括环检测、环入口、相交链表、回文链表、链表排序、复制带随机指针链表和 LRU 缓存。环检测用快慢指针，核心是速度差；相交链表比较的是节点地址，不是节点值；回文链表需要找到中点、反转后半段、比较并最好恢复结构；链表排序适合自底向上归并，避免递归栈；随机指针复制需要原节点到新节点的映射，或者把复制节点插入原链表再拆分。每个变种都不是新模板，而是围绕“节点身份和链接关系”展开。

栈和单调栈：最近关系、表达式和候选淘汰

栈保存的是最近尚未解决的结构。括号匹配中，最近的左括号必须先匹配；路径简化中，最近进入的目录可能被 `..` 取消；表达式求值中，最近的运算符和操作数形成局部归约；DFS 中，栈保存待展开路径。普通栈的正确性来自后进先出，单调栈的正确性来自候选淘汰。

单调栈不是为了“保持栈有序”而有序，而是为了让新元素到来时能结算一批旧元素。每日温度、下一个更大元素、柱状图最大矩形、接雨水都可以用这个模型解释。栈里保存尚未找到右边界

的下标；新元素若破坏单调性，就说明某些栈顶元素的右边界已经确定。被弹出的元素不会再回来，因此总复杂度是摊还线性。

表达式类题目则体现栈的另一面：保存上下文。逆波兰表达式已经把优先级编码到顺序里，只需要数字栈；基本计算器 II 需要遇到乘除立即结合、加减延迟求和；带括号的计算器需要在进入括号时保存外层结果和符号。写表达式题时，先定义 token 类型、当前数字、当前运算符、栈中保存内容，再动手编码，错误会少很多。

逆波兰表达式可以先按题意暴力模拟：从左到右读 token，遇到数字暂存，遇到运算符就回头找离它最近的两个尚未参与归约的数字，计算后再把结果放回序列。这个做法的瓶颈是“最近尚未归约对象”的查找和删除很别扭，因为序列中间会不断被替换。观察运算符的定义后会发现，真正需要的永远是最近的两个操作数，且先出现的操作数在左，后出现的操作数在右。这正是栈的职责：数字入栈，运算符弹出右操作数和左操作数，归约结果再入栈。

图示：逆波兰表达式的栈归约

读到数字	压入栈，表示一个尚未被运算符消费的操作数。
读到运算符	弹出 rhs ，再弹出 lhs ，顺序不能反。
完成归约	计算 lhs op rhs ，把结果重新压栈。
最终状态	所有 token 扫完后，栈顶就是整个表达式的值。

Listing 2.21 逆波兰表达式：栈中保存待归约操作数

```

1 int eval_rpn(const std::vector<std::string>& tokens)
2 {
3     std::vector<int> stack;
4     stack.reserve(tokens.size());
5
6     for (const std::string& token : tokens) {
7         if (token == "+" || token == "-" || token == "*" || token == "/") {
8             const int rhs = stack.back();
9             stack.pop_back();
10            const int lhs = stack.back();
11            stack.pop_back();
12
13            if (token == "+") {
14                stack.push_back(lhs + rhs);
15            } else if (token == "-") {
16                stack.push_back(lhs - rhs);
17            } else if (token == "*") {
18                stack.push_back(lhs * rhs);
19            } else {
20                stack.push_back(lhs / rhs);
21            }
22        } else {
23            stack.push_back(std::stoi(token));
24        }
25    }
26    return stack.back();
27 }

```

这里弹出的第一个数是右操作数，第二个数是左操作数。减法和除法不能交换顺序，这是表达式栈题最常见的细节。工程实现中，如果 token 很多且转换成本重要，可以先写一个更明确的解析函数；但刷题训练重点是栈状态语义。

队列、双端队列和环形缓冲区

队列保存的是时间顺序或层次顺序。BFS 第一次到达某个状态即为最短距离，前提是所有边权相等且访问标记在入队时设置。多源 BFS 只是把多个起点同时作为第零层，腐烂橘子、地图中离最近零的距离、从边界反向标记安全区域都可以这样处理。队列不负责“选最优候选”，它只负责“按层推进”；如果边权不同，就需要堆或其他最短路结构。

双端队列比队列多了从两端进出的能力，因此能维护窗口候选。单调队列保存的不是窗口内所有元素，而是仍可能成为答案的候选下标。队头是当前答案，队尾负责删除被新元素支配的旧候选。滑动窗口最大值、和至少为 K 的最短子数组、受限子序列和都依赖类似的支配关系。

环形缓冲区是队列的底层变体。固定数组配合头尾下标，可以避免频繁分配；当尾部走到数组末尾时回到开头。它适合实现固定容量队列、日志缓冲、生产者消费者队列和某些实时系统。刷题中不常要求手写环形队列，但理解它能帮助你明白为什么 `deque` 和 `queue` 的接口强调两端操作，而不是随机插入。

Listing 2.22 固定容量环形队列：头尾下标表达状态

```

1 class CircularQueue {
2 public:
3     explicit CircularQueue(int capacity)
4         : values_(static_cast<std::size_t>(capacity)), capacity_(capacity)
5     {
6     }
7
8     bool push(int value)
9     {
10         if (this->size_ == this->capacity_) {
11             return false;
12         }
13         this->values_[static_cast<std::size_t>(this->tail_)] = value;
14         this->tail_ = (this->tail_ + 1) % this->capacity_;
15         ++this->size_;
16         return true;
17     }
18
19     std::optional<int> pop()
20     {
21         if (this->size_ == 0) {
22             return std::nullopt;
23         }
24         const int value = this->values_[static_cast<std::size_t>(this->head_)];
25         this->head_ = (this->head_ + 1) % this->capacity_;
26         --this->size_;
27         return value;
28     }
29
30 private:
31     std::vector<int> values_;
32     int capacity_ = 0;
33     int head_ = 0;
34     int tail_ = 0;
35     int size_ = 0;
36 };

```

这段代码保留 `size_`，是为了区分空队列和满队列。如果只用 `head_ == tail_`，空和满都会

满足这个条件，需要浪费一个槽位或额外标记。真实工程中还要考虑并发、异常安全和容量为零的输入约束；刷题版本则要把状态不变量讲清楚：`head_` 指向下一个弹出位置，`tail_` 指向下一个写入位置，`size_` 表示当前元素数。

线性结构的实际应用和变种题谱

数组的实际应用包括排序缓冲、DP 表、图邻接表、前缀统计、差分更新、坐标压缩后的频次数组。字符串应用包括日志解析、词典匹配、搜索建议、DNA 序列重复检测、表达式解析。链表应用包括 LRU、内核对象链、空闲块管理、任务队列和需要稳定节点地址的缓存。栈应用包括函数调用、解析器、撤销操作、表达式求值、单调候选。队列应用包括 BFS、任务调度、消息缓冲、多源扩散。双端队列应用包括滑动窗口极值、0-1 BFS、双端搜索和单调 DP 优化。

变种题复盘可以按结构归类。数组原地维护：26、27、75、88、189、283；前缀与差分：303、304、560、1109、1094；字符串窗口：3、76、438、567；链表重连：19、21、24、25、92、206；链表身份：141、142、160、138；普通栈：20、71、150、224、227、394；单调栈：42、84、496、503、739；队列 BFS：102、199、200、994、1091；单调队列：239、862、1438、1425。每组题都要问同一个问题：结构保存什么，什么时候加入，什么时候移除，移除是否会丢答案。

线性结构实验：第一，用 `vector`、`deque`、`list` 分别顺序扫描一百万个整数，比较实际时间并解释缓存局部性。第二，把字符串窗口题写成 `substr` 加排序和计数窗口两版，记录输入长度增长时的差异。第三，把链表反转的每一步打印为节点地址，确认比较的是节点身份。第四，把单调栈中的每个下标入栈出栈次数记录下来，验证摊还复杂度。第五，手写环形队列并测试空、满、绕回和连续弹入弹出。

线性结构变种题谱

数组变种可以分成七类。第一类是原地过滤，例如删除重复项、移除元素、移动零、稳定划分。这类题的共同点是输出仍放在原数组前缀，慢指针表示下一个写入位置，快指针负责扫描未知区。第二类是双向收缩，例如盛水容器、两数之和 II、回文判断。这类题依赖左右边界比较能排除一侧。第三类是前后缀贡献，例如除自身以外数组乘积、接雨水的前后缀解法、最短无序连续子数组。第四类是矩阵边界模拟，例如螺旋矩阵、旋转图像、矩阵置零。第五类是差分和扫描线，例如航班预订、拼车、区间加法。第六类是坐标压缩后用数组维护频次或排名。第七类是环形数组，例如轮转数组、循环队列、下一个更大元素 II。

这些变种应该统一回到区间不变量。原地过滤要说明写入区不会被未来读操作破坏；双向收缩要说明被移走的一侧不可能产生更优答案；前后缀贡献要说明当前位置答案由左右两边已经缓存的信息合成；矩阵模拟要说明四条边界何时收缩，剩一行或一列时不会重复访问；差分数组要说明每个区间更新如何转化为两个端点事件。题目名称不同，但证明方式不是散的。

字符串变种可以按“连续、非连续、多模式”分。连续子串题包括无重复最长子串、最小覆盖子串、异位词窗口、回文子串、重复 DNA 序列。非连续子序列题包括判断子序列、最长公共子序列、不同的子序列。多模式题包括单词搜索 II、替换单词、搜索建议系统。连续子串常用窗口、哈希、中心扩展和 KMP；子序列常用双指针或 DP；多模式常用 Trie。复盘时先判断目标是否要求连续，能直接避免很多错误方向。

链表变种可以按“位置、结构、身份、排序、缓存”分。位置类题包括删除倒数第 N 个、旋转链表、分隔链表，核心是快慢指针和前驱节点。结构类题包括反转链表、两两交换、K 个一组反转、重排链表，核心是保存后继和局部接回。身份类题包括环形链表、环入口、相交链表，核心是比较节点地址。排序类题包括合并两个有序链表、合并 K 个链表、排序链表，核心是节点重连和归并。缓存类题包括 LRU 和 LFU，核心是哈希定位和链表维护顺序。

栈变种可以按“匹配、归约、单调、上下文”分。匹配类题包括有效括号、删除无效括号、最长有效括号；归约类题包括逆波兰表达式、基本计算器、字符串解码；单调类题包括每日温度、下一个更大元素、柱状图最大矩形、接雨水；上下文类题包括路径简化、文件系统解析、嵌套列表解析。判断一个栈题属于哪一类，关键看栈里保存的是符号、操作数、下标、局部字符串，还是上一层上下文。

队列变种可以按“层次、双端、单调、优先级”分。普通队列服务 BFS 层次，双端队列服务两端扩展和 0-1 BFS，单调队列服务窗口极值和 DP 转移最大值，优先级队列服务动态极值。不要把

这些队列混成一个概念。普通 BFS 队列不按权重选点，所以不能处理不同边权；单调队列不是为了先进先出，而是为了删除被支配候选；优先级队列不是按时间顺序，而是按优先级顺序。

线性结构复盘表

每做一道线性结构题，都在题解末尾补一张小表：输入是否连续，是否允许修改，是否需要保持相对顺序，是否要比较节点身份，窗口是否合法依赖什么状态，候选什么时候过期，元素最多进入和离开结构几次。能填完这张表，说明你掌握的是结构，不是题号。

线性结构的底层成本模型

经典教材不会只给出一张复杂度表，因为复杂度表把太多真实代价藏起来了。线性结构的底层成本至少有五个维度：随机访问、顺序扫描、局部插入删除、内存分配、缓存局部性。数组和 `vector` 在随机访问和顺序扫描上很强，因为第 i 个元素的位置可以直接计算；链表在已知节点附近重连很强，但每走一步都要先加载下一个节点地址；`deque` 介于两者之间，分段连续让两端扩张便宜，却让纯顺序扫描的局部性通常弱于 `vector`。

这五个维度会直接改变刷题方案。最长递增子序列的 $O(n \log n)$ 解法需要频繁在一个有序尾数组上二分，尾数组必须支持下标访问，所以 `vector` 合适；LRU 缓存需要把任意已知节点移到表头并删除表尾，`list` 合适；滑动窗口最大值需要从两端删除候选，`deque` 合适；环形日志缓冲不需要无限扩容，只需要固定容量覆盖旧值，数组加头尾下标更合适。容器不是按照“高级程度”选择，而是按照操作组合选择。

再看插入删除的说法。说链表删除是 $O(1)$ ，必须补上前提：已经有待删节点的前驱或迭代器。如果你只有一个值，还要从头找这个值，查找仍然是 $O(n)$ 。说 `vector` 尾部插入是均摊 $O(1)$ ，也必须补上前提：它可能偶尔扩容，扩容时旧地址全部失效。说 `deque` 两端插入是 $O(1)$ ，也不等于它适合所有随机访问密集任务。高质量答案要把这些前提说出来。

深入理解

CPU 缓存解释了为什么很多教材和工程代码偏爱连续数组。处理器从内存读取一个整数时，通常会把相邻的一小段字节一起载入缓存；如果下一次访问正好落在这段缓存行里，就不必再等主内存。数组顺序扫描天然吃到这个收益。链表节点可能散落在堆上，访问下一个节点前必须先读出当前节点里的指针，硬件很难提前知道下一步在哪里。两段代码同为 $O(n)$ ，实际速度可能差出很多，这不是大 O 错了，而是大 O 的抽象层次不同。

数组模型：从区间到事件

数组题要先判断自己维护的是“区间状态”还是“位置事件”。区间状态题关心某个连续范围内的和、最大值、字符频次、不同元素数；位置事件题关心某个下标开始或结束了一段影响。前缀和把区间查询转成两个端点的差，差分数组把区间更新转成两个端点事件。二者是同一思想的两个方向：前缀和适合多次查询，差分适合多次更新后统一还原。

差分数组特别适合讲清“事件化”的思维。航班预订、拼车、区间加法这类题，如果对每个区间逐个位置累加，复杂度会被区间长度拖垮。差分数组只在左端点加一个事件，在右端点后一位减一个事件；最后扫描一次，把事件累积成每个位置的真实值。它把“影响一整段”改写成“从这里开始多一个影响，从那里之后少一个影响”。

Listing 2.23 差分数组：区间更新变成端点事件

```
1 std::vector<int> apply_range_additions(
2     int length,
3     const std::vector<std::array<int, 3>>& operations)
4 {
5     std::vector<int> difference(static_cast<std::size_t>(length + 1), 0);
6
7     for (const auto& operation : operations) {
8         const int left = operation[0];
9         const int right = operation[1];
```



```

10     const int delta = operation[2];
11     difference[static_cast<std::size_t>(left)] += delta;
12     if (right + 1 < length) {
13         difference[static_cast<std::size_t>(right + 1)] -= delta;
14     }
15 }
16
17 std::vector<int> values(static_cast<std::size_t>(length), 0);
18 int current = 0;
19 for (int i = 0; i < length; ++i) {
20     current += difference[static_cast<std::size_t>(i)];
21     values[static_cast<std::size_t>(i)] = current;
22 }
23 return values;
24 }

```

这段代码的关键是端点语义。若操作作用在闭区间 $[left, right]$ ，就在 $left$ 处增加影响，在 $right + 1$ 处撤销影响。若题目使用半开区间 $[left, right)$ ，撤销位置就是 $right$ 。很多差分题错在闭区间和半开区间混用。写题解时先定义区间语义，再写公式，代码才不会靠样例猜。

数组还能表达“排名”和“值域”。计数排序、桶排序、位图、坐标压缩都利用这一点。若值域很小，直接用值作下标，查询和统计都可以很快；若值域很大但实际出现的值不多，就把出现过的值排序后映射到 $0..m-1$ ，这就是离散化。树状数组统计逆序对、区间和个数、动态排名时，经常先离散化，因为原始数值可能达到十亿，不能直接开数组。

字符串模型：窗口、自动机和字典

字符串题的第一分叉是连续还是不连续。连续子串题可以用窗口、哈希、KMP、Manacher、后缀结构；不连续子序列题更多走双指针和动态规划；多个模式串同时匹配时，要考虑 Trie 或 AC 自动机。只要这个分叉判断错，后面的模板就容易全错。比如最小覆盖子串必须连续，适合滑动窗口；最长公共子序列不要求连续，窗口没有意义；单词搜索 II 同时查很多单词，逐个单词 DFS 会重复大量前缀，Trie 才能合并重复工作。

滑动窗口的本质是“窗口合法性可以增量维护”。无重复最长子串中，合法性由每个字符出现次数是否超过一决定；最小覆盖子串中，合法性由所有必需字符是否都达到需求决定；替换后的最长重复字符中，合法性由窗口长度减最高频字符数是否不超过替换次数决定。这些题的状态不同，但都能随着左右端移动做常数或近似常数更新。

窗口题的三句证明

第一，右端扩张时，窗口状态如何更新；第二，窗口不合法时，左端如何移动并恢复合法；第三，左端移动不会错过答案的原因是什么。正数数组求最短和至少为 K 的子数组能用窗口，是因为和随右端扩张不减；数组允许负数时，这条单调性消失，就要换成前缀和加单调队列。

字符串工程应用还要重视编码和生命周期。刷题题面常说只含小写字母，此时 `array<int, 26>` 既快又清晰；日志、路径、URL、自然语言文本则不能这样写。若函数只临时观察一段字符串，`string_view` 能减少复制；若要把子串作为长期 key 存进哈希表，就必须拥有一份真正的 `string`。算法题为了清晰常省略这些工程前提，但教材应该把它们说清楚。

链表模型：缓存、空闲链和节点生命周期

链表题表面是指针题，底层是节点生命周期题。数组的元素位置由下标决定，链表的元素位置由指针关系决定；数组移动元素会改变位置，链表重连节点不会改变节点身份。这个差异让链表适合缓存、内存分配器、内核对象队列和需要稳定节点地址的系统结构。Linux 内核常用侵入式链表：链表节点字段嵌入业务对象中，通过节点地址反推出外层对象。这样做能减少额外分配，也让对象生命周期由系统自己精确控制。

刷题里最能体现工程味的是 LRU。它需要两个操作同时快：按 key 找到缓存项，按新旧顺序删除最久未使用项。哈希表能快速定位 key，却不知道谁最旧；链表能维护顺序，却不能按 key 快速定位。把二者组合起来，哈希表保存 key 到链表迭代器，链表保存从新到旧的缓存项，访问时用 `splice` 把节点移到表头。

如果直接写暴力 LRU，可以用一个数组或链表保存缓存项。每次 `get` 时线性查找 key，找到后把它移到最前；每次 `put` 时也先线性查找，容量满了删除最后一个。这个版本容易理解，但每个操作最坏都是 $O(n)$ ，慢点集中在“按 key 定位”这一步。换成单独哈希表可以把定位降到常数，却失去新旧顺序；换成单独链表能维护顺序，却仍然无法快速定位 key。于是组合结构是从两个瓶颈推出来的：哈希表负责定位，双向链表负责顺序，二者通过迭代器连接。

图示：LRU 的两个索引

哈希表	<code>key -> list iterator</code> ，负责常数时间定位缓存项。
双向链表	<code>newest <-> ... <-> oldest</code> ，负责维护访问新旧顺序。
<code>get</code>	哈希定位节点，再把节点移动到链表头部。
<code>put</code>	新 key 插入头部；容量满时删除链表尾部，并同步删除哈希表 key。

Listing 2.24 LRU 缓存：哈希定位和链表维护顺序

```

1 class LruCache {
2 public:
3     explicit LruCache(int capacity) : capacity_(capacity) {}
4
5     int get(int key)
6     {
7         const auto found = this->position_by_key_.find(key);
8         if (found == this->position_by_key_.end()) {
9             return -1;
10        }
11        this->items_.splice(this->items_.begin(), this->items_, found->second);
12        return found->second->value;
13    }
14
15    void put(int key, int value)
16    {
17        if (this->capacity_ == 0) {
18            return;
19        }
20
21        const auto found = this->position_by_key_.find(key);
22        if (found != this->position_by_key_.end()) {
23            found->second->value = value;
24            this->items_.splice(this->items_.begin(), this->items_, found->second);
25            return;
26        }
27
28        if (static_cast<int>(this->items_.size()) == this->capacity_) {
29            const int expired_key = this->items_.back().key;
30            this->position_by_key_.erase(expired_key);
31            this->items_.pop_back();
32        }
33    }

```

```

34     this->items_.push_front(Entry{key, value});
35     this->position_by_key_[key] = this->items_.begin();
36 }
37
38 private:
39     struct Entry {
40         int key = 0;
41         int value = 0;
42     };
43
44     int capacity_ = 0;
45     std::list<Entry> items_;
46     std::unordered_map<int, std::list<Entry>::iterator> position_by_key_;
47 };

```

这个实现的核心不在代码长度，而在职责分离。`position_by_key_` 回答“这个 key 在哪里”，`items_` 回答“谁最新，谁最旧”。`splICE` 只改链表链接，不复制节点，所以哈希表里保存的迭代器仍然指向同一个节点。如果用 `vector` 存缓存顺序，把元素移到开头会导致大量搬移，也会破坏保存的位置。这个题是数据结构组合的典型教材案例。

链表变体题也要从节点生命周期解释。复制带随机指针链表时，原节点和新节点之间必须建立映射，否则随机边无法接回；回文链表如果反转后半段，比较后最好恢复原结构，避免修改输入的副作用；排序链表适合自底向上归并，因为链表不支持随机访问，快速排序的分区优势不明显；K 个一组反转需要先确认剩余节点够 K 个，再局部反转，否则尾部不足一组的节点不能被破坏。这些细节都来自“节点关系是结构本身”。

栈、队列和调度模型

栈和队列不只是简单容器，它们分别表达两类调度策略。栈表达最近上下文优先处理，适合解析嵌套结构、撤销操作、显式 DFS 和单调候选；队列表达先到状态优先处理，适合 BFS、任务缓冲和层次传播；双端队列表达两端都有特殊含义，适合单调窗口、0-1 BFS 和双向搜索。

0-1 BFS 是双端队列的代表。边权只有 0 和 1 时，普通 Dijkstra 用堆当然能做，但双端队列更贴合结构：走 0 权边不增加距离，应该放到队头尽快处理；走 1 权边距离加一，放到队尾。队列里状态的距离仍然近似按非降顺序展开，因此可以得到最短路。

从暴力角度看，带权最短路可以枚举路径，但路径数量会指数级增长；普通 BFS 会把每条边都当成同样代价，遇到 0 权边和 1 权边混在一起时，层数不再等于距离。Dijkstra 的堆版本是通用优化，但在 0/1 边权下，堆顶排序只需要区分“不增加距离”和“距离加一”两类。于是双端队列成为更轻的结构：0 权转移放队头，1 权转移放队尾，保证更小距离的状态先被继续扩展。

图示：0-1 BFS 的双端队列

队头方向	0 权边不增加距离，新状态放到队头，尽快继续扩展。
队尾方向	1 权边让距离加一，新状态放到队尾，排在当前距离层之后。
核心不变量	队列中的候选距离近似非降，先处理的状态不会被更小距离状态越过。
失效条件	边权不是 0 或 1 时，双端队列不再足够，应改用 Dijkstra。

Listing 2.25 0-1 BFS：双端队列表达两类边权

```

1 std::vector<int> zero_one_bfs(
2     const std::vector<std::vector<std::pair<int, int>>>& graph,
3     int source)
4 {
5     constexpr int INF = 1'000'000'000;
6     std::vector<int> distance(graph.size(), INF);

```

```

7     std::deque<int> deque;
8
9     distance[static_cast<std::size_t>(source)] = 0;
10    deque.push_front(source);
11
12    while (!deque.empty()) {
13        const int node = deque.front();
14        deque.pop_front();
15
16        for (const auto& [next, weight] : graph[static_cast<std::size_t>(node)]) {
17            const int candidate = distance[static_cast<std::size_t>(node)] + weight;
18            if (candidate >= distance[static_cast<std::size_t>(next)]) {
19                continue;
20            }
21            distance[static_cast<std::size_t>(next)] = candidate;
22            if (weight == 0) {
23                deque.push_front(next);
24            } else {
25                deque.push_back(next);
26            }
27        }
28    }
29    return distance;
30 }

```

这段代码比普通 BFS 多了一个判断：边权为 0 的状态放前面，边权为 1 的状态放后面。它也比堆版 Dijkstra 少了对数因子。成立条件非常窄：边权必须只有 0 和 1；如果出现权重 2、5 或任意非负数，就该回到 Dijkstra。教材式学习要把这种边界讲清楚，否则学生会把一个漂亮技巧误用到不满足条件的图上。

线性结构题目的讲解标准

一题讲清楚，至少要包含五层。第一层是暴力方法：枚举什么，复杂度为什么高。第二层是瓶颈：重复扫描、重复计算、候选过多、边界分支太多，还是查询能力不足。第三层是结构选择：为什么数组、链表、栈、队列或双端队列能降低瓶颈。第四层是不变量：哪些区间、节点、候选或层次已经正确。第五层是边界条件：空输入、单元素、重复值、负数、环、容量为零、字符集假设和是否允许修改输入。

以滑动窗口最大值为例，暴力方法是每个窗口扫描 K 个元素；瓶颈是相邻窗口大量重叠，却重新求最大；结构选择是双端队列，因为窗口左端会过期，右端会加入新候选；不变量是队列下标从前到后递增，值从前到后单调不增，队头始终在窗口内；边界条件是 K 非法、数组为空、重复元素是否用小于还是小于等于淘汰。只有这些都讲清楚，题解才像教材，而不是模板注释。

以反转链表为例，暴力方法并不是重点，因为它本身已经是线性；重点在结构不变量。循环开始时，**previous** 指向已经反转好的前缀头，**current** 指向尚未处理部分的第一个节点，**current** 之后的链仍保持原方向。每次先保存 **next**，再把当前节点接到已反转前缀前，最后推进两个指针。边界条件是空链表和单节点链表。能用这套语言解释，两两交换和 K 个一组反转就不再是新题。

最后要把线性结构放进复盘表里。数组题复盘区间语义和是否原地修改；字符串题复盘字符集、连续性和子串复制成本；链表题复盘节点身份、前驱和后继保存顺序；栈题复盘保存的是符号、下标、操作数还是上下文；队列题复盘层次、距离和访问标记时机；双端队列题复盘候选淘汰和过期规则。这样复盘一段时间后，你看到新题会先想到访问模式，而不是先搜索记忆里的题号。

数据结构源码视角和容器选择

数据结构学习要同时看抽象接口和存储模型。vector 的接口像可变长数组，底层却有大小、容量和连续内存三件事；deque 的接口像两端都能高效进出的序列，底层通常是分段连续块；list 的接口保证

节点稳定和常数移动，但牺牲随机访问和缓存局部性；`unordered_map` 的接口给平均常数查询，底层却受哈希函数、桶数量、负载因子和 rehash 影响。理解这些模型后，刷题时选择容器才不是背名字。

`vector` 的扩容是最值得做的实验。连续 `push_back` 时，`size` 每次加一，`capacity` 只在容量不足时跳变，`data` 地址也可能改变。这个现象解释了为什么保存 `vector` 元素指针或迭代器后继续插入很危险，也解释了 `reserve` 和 `resize` 的区别。`reserve` 改容量，不构造新元素；`resize` 改大小，会构造或销毁元素。很多性能问题不是算法复杂度错，而是无意中触发了大量重新分配和拷贝。

`string` 的成本不能只看语法。`substr` 往往会构造新字符串，动态规划中两层循环里反复 `substr` 可能把复杂度悄悄提高一阶。`string_view` 能表达非拥有视图，但它要求底层字符串生命周期足够长。刷题中为了清晰可以先用 `substr`，复盘时必须知道它的复制成本；工程代码中，如果热路径上大量切片，应该考虑下标范围、视图或 `Trie`。

`unordered_map` 的平均常数时间不是无条件承诺。哈希冲突、负载因子过高、频繁 rehash 都会让性能波动。刷题时调用 `reserve` 可以减少 rehash，使用 `find` 可以避免 `operator[]` 的默认插入副作用。当前缀和计数题里，哈希表语义是历史前缀频次，如果用 `operator[]` 随意读取不存在键，就可能把调试输出和真实状态混在一起。

`map` 和 `set` 基于有序结构，常用于前驱、后继、区间查询和动态有序集合。它们的查询是对数复杂度，不如哈希表平均常数快，但能回答哈希表回答不了的顺序问题。区间覆盖、日程安排、滑动窗口有序统计、离线扫描线，都可能需要有序容器。选择 `map` 不是因为它高级，而是因为题目需要顺序语义。

`priority_queue` 是容器适配器，不是完整排序容器。它只承诺堆顶极值，不承诺遍历顺序。需要频繁删除任意元素时，普通 `priority_queue` 不支持直接删除，常见办法是懒删除：另一个哈希表记录应该删除的元素，真正取堆顶前不断弹出过期项。理解这个限制，才能写好滑动窗口中位数、任务调度和带过期时间的事件模拟。

Linux 内核的数据结构会在独立专题中系统展开。这里先只保留一个接口判断：STL 容器强调泛型接口、异常安全和所有权封装；内核结构强调嵌入式节点、明确生命周期、分配控制和并发语义。两者都不是模板清单，核心仍然是访问模式。

读源码时不要从模板细节硬啃。先读接口语义和复杂度承诺，再找核心成员，最后追一条热路径。`vector` 看 `push_back` 和扩容，`deque` 看段表如何定位下标，`unordered_map` 看查找和 rehash，`map` 看 `lower_bound` 如何沿树下降，`priority_queue` 看 `push_heap` 和 `pop_heap`。每次阅读只回答一个问题：这个 API 为什么是这个复杂度，什么时候会让迭代器失效，题目里用它是否合适。

专题复盘要求

读完本节后，不要只记结论。请把每个结论还原成一个可验证的问题：它解决了哪一步重复工作，依赖哪个题目条件，使用哪个容器或状态，边界条件如何破坏错误写法。

非线性数据结构：哈希表、堆、树、图和并查集

线性结构擅长处理顺序，非线性结构擅长处理关系和查询。哈希表回答“这个状态以前出现过吗”；堆回答“当前最极端的候选是谁”；树表达层次、有序性和递归子问题；图表达任意状态之间的连接；并查集维护“这些元素现在是否属于同一组”。这些结构不是刷题技巧的附属品，而是算法优化的来源。很多题从 $O(n^2)$ 变成 $O(n)$ ，并不是因为循环少写了一层，而是因为某个昂贵查询被换成了合适的数据结构查询。

关系结构的查询能力

非线性结构要按查询能力来学习。哈希表擅长等值查询，却不擅长有序边界；堆擅长动态极值，却不擅长删除任意元素和按顺序遍历；平衡树擅长前驱后继和有序迭代，代价是每次操作有对数级路径；Trie 擅长前缀共享，代价是节点数量可能随字符总量增长；图结构擅长表达任意连接关系，但必须选择合适的搜索或最短路算法；并查集擅长维护连通性，却不能回答路径长度和具体路径。看题时先问“我要问结构什么问题”，再决定使用哪一个结构。

关系结构通常用空间换查询速度。两数之和用哈希表保存历史值，换掉反复线性查找；Top K 用堆保存候选边界，换掉完整排序；课程表用邻接表和入度数组保存依赖，换掉反复扫描所有关系；并查集用父指针保存集合代表，换掉每次 DFS 判断连通。空间不是附带成本，而是优化的一部分。面试时要说清新增空间保存了什么，以及它为什么足够回答后续查询。

这些结构也都有失效边界。哈希表一旦题目要求最小大于、最大不超过或区间统计，就要换有序结构；堆一旦题目要求删除滑出窗口的任意旧元素，就需要懒删除、双堆加哈希或改用 `multiset`；普通并查集一旦题目要求边权比例、奇偶关系或距离，就要扩展成带权并查集；普通 BFS 一旦边权不相等，就要改成 Dijkstra、0-1 BFS 或 Bellman-Ford。真正掌握结构，意味着知道它什么时候不能用。

从工程角度看，非线性结构常常牵涉对象身份。哈希表里的 key 必须稳定；树节点比较器必须满足严格弱序；堆中保存的状态可能过期，需要弹出时验证；图的 visited key 必须覆盖完整状态维度；并查集路径压缩会改变父指针，但不能改变集合语义。代码越依赖多个索引协同，越要维护一致性。LRU、LFU、时间键值存储、最小区间覆盖这类设计题，本质上都是多个结构共同维护一个对象集合。

非线性结构选择表

等值历史查询选哈希；动态最大最小选堆；有序前驱后继选平衡树；前缀匹配选 Trie；任意连接关系先建图；只问连通性选并查集；若还要路径长度、顺序统计、范围更新或资源状态，就必须升级结构或重新建模。

很多中高阶题不是单一结构能解决，而是多个结构维护同一批对象的不同索引。LRU 缓存用哈希表按 key 定位节点，用双向链表维护新旧顺序；LFU 缓存再多一层频次索引，既要按 key 找对象，又要按频次和时间淘汰；时间键值存储先用哈希表按 key 分组，再在每组有序数组里二分时间；最小区间覆盖 K 个列表用堆找当前最小元素，同时维护当前最大值。结构组合的难点不在 API 数量，而在一致性：一个对象被移动、删除或更新时，所有索引都必须同步，否则查询会读到旧状态。

因此，设计题要先写对象模型，再写索引模型。对象模型说明一个条目有哪些字段、由谁拥有、什么时候创建和释放；索引模型说明哪些容器能找到它、按什么 key 找、顺序或优先级如何维护。只有对象身份稳定，索引才有意义。若对象存进 `vector` 后还保存指针，扩容可能让指针失效；若链表节点删除后哈希表没删，哈希表会指向悬空节点；若堆中旧优先级状态不清理，弹出时必须用当前表验证它是否过期。把这些一致性规则写清楚，设计题才像工程答案，而不是容器堆叠。

读题时还要把操作分成在线和离线。离线问题可以先收集全部输入，排序、压缩坐标、批量建图或批量建堆；在线问题则要求每次操作后结构立即保持可查询状态。区间合并如果所有区间一次给出，排序扫描就够；日程表动态插入时，必须维护有序结构查前驱后继。Top K 如果数组一次给

出，可以排序、堆或快速选择；数据流不断到来时，就要维护堆或有序集合。在线和离线的区别，会直接改变数据结构选择。

还有一类题处在二者之间：输入一次给出，但查询很多。此时可以接受较重的预处理，换取每次查询更快。二维前缀和、稀疏表、拓扑可达性预处理、时间键值存储中的分组有序数组，都属于这种思路。判断是否值得预处理，要比较构建成本、查询次数和内存上限，而不是只看单次查询是否漂亮。

哈希表是保存历史信息的核心结构。`std::unordered_map` 和 `std::unordered_set` 的平均查询、插入、删除复杂度是 $O(1)$ ，适合把“回头查找”变成“即时查询”。两数之和的暴力方法枚举两个下标，慢在每个数都要向后找补数；哈希表版本边扫描边记录已经见过的值，当前数只需要查询一次补数。这个优化成立的前提是题目只要求找到一对下标，且补数是否出现过可以由历史集合回答。

哈希表的底层思想是把 key 通过哈希函数变成一个整数，再映射到桶。桶里可能有零个、一个或多个元素；多个不同 key 落到同一个桶，就是冲突。标准库通常用链式或等价结构处理冲突，并通过负载因子控制桶数量。当元素增多、负载因子过高时，容器会 rehash：申请更多桶，把已有元素重新分布。平均 $O(1)$ 来自哈希分布足够均匀、桶内元素数量受控；它不是数学上的绝对常数承诺。

哈希表操作表

操作	平均复杂度	注意点
<code>find</code>	$O(1)$	不插入新 key，适合查询并取值。
<code>contains</code>	$O(1)$	只判断存在性，C++20 可用。
<code>operator[]</code>	$O(1)$	key 不存在会插入默认值。
<code>emplace</code>	$O(1)$	构造并尝试插入，可能触发 rehash。
<code>erase</code>	$O(1)$	删除元素后相关迭代器失效。

用哈希表前要先明确保存什么。保存集合时用 `unordered_set`；保存计数时用 `unordered_map<Key, int>`；保存首次位置时用 `unordered_map<Key, int>` 且通常不覆盖旧值；保存最近位置时才覆盖；保存状态访问标记时要保证状态编码无歧义。很多错误不是哈希表 API 不熟，而是把“出现过”“出现几次”“第一次出现在哪里”“当前窗口中出现几次”混成同一个含义。

Listing 3.1 两数之和：哈希表把补数查询变成平均常数时间

```

1 std::vector<int> two_sum(const std::vector<int>& nums, int target)
2 {
3     std::unordered_map<int, int> index_by_value;
4     index_by_value.reserve(nums.size());
5
6     for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
7         const int complement = target - nums[i];
8         const auto found = index_by_value.find(complement);
9         if (found != index_by_value.end()) {
10             return {found->second, i};
11         }
12         index_by_value.emplace(nums[i], i);
13     }
14
15     return {};
16 }
```

代码里先查再插入，是为了避免同一个元素被用两次。例如目标是 6，当前数是 3，如果先插入再查，可能错误地返回同一个下标。这里用 `find` 而不是 `operator[]`，因为 `operator[]` 在 key 不存在时会插入默认值。只想判断存在性时，C++20 可以用 `contains`；需要同时取值时，用 `find`，

这样只查一次。刷题中还应该习惯 `reserve`，它能减少 rehash 次数。rehash 会重新组织桶，导致迭代器失效；如果保存了哈希表元素迭代器，扩容后不能继续使用。

哈希表不仅能存值到下标，也能存频次、前缀和、状态编码和访问标记。前缀和加哈希表很重要的一类优化。和为 K 的子数组如果暴力枚举左右端点，需要 $O(n^2)$ ；如果知道当前位置前缀和是 `current`，那么以当前位置结尾、和为 k 的子数组数量，等于以前出现过多少次 `current - k`。哈希表保存的是“某个前缀和出现次数”，查询的是“能和当前前缀配成目标的历史前缀”。

Listing 3.2 和为 K 的子数组：前缀和频次

```
1 int subarray_sum_equals_k(const std::vector<int>& nums, int k)
2 {
3     std::unordered_map<int, int> prefix_count;
4     prefix_count.reserve(nums.size() + 1);
5     prefix_count.emplace(0, 1);
6
7     int prefix = 0;
8     int answer = 0;
9     for (const int x : nums) {
10         prefix += x;
11         const auto found = prefix_count.find(prefix - k);
12         if (found != prefix_count.end()) {
13             answer += found->second;
14         }
15         ++prefix_count[prefix];
16     }
17
18     return answer;
19 }
```

这个算法容易错在初始化。空前缀和 0 必须先出现一次，否则从下标 0 开始的合法子数组会漏掉。它也解释了为什么有负数时不能随使用滑动窗口：窗口和不再随右端单调增加，左端移动也不保证更接近目标；前缀和加哈希表不依赖正数单调性，所以适用范围更广。

哈希表的限制同样要清楚。第一，它不维护顺序，如果题目要求按排序顺序取最小、最大或区间，就该考虑 `std::map`、`std::set`、堆、树状数组或线段树。第二，平均 $O(1)$ 不等于绝对 $O(1)$ ，糟糕哈希或恶意数据可能退化。第三，自定义 key 需要提供相等比较和哈希函数。面试里一般不会刻意卡哈希攻击，但你要知道 `unordered_map<pair<int,int>, int>` 不能直接在所有标准库环境下工作，需要自定义哈希。

哈希表也不适合回答范围问题。若题目问“小于等于 x 的最大元素”“区间内有多少元素”“按键有序遍历”，哈希表会丢掉顺序信息，通常需要平衡树、树状数组、线段树或排序加二分。若题目只问“是否出现过”“出现次数”“状态是否访问过”，哈希表正好贴合。选择数据结构时不要只看平均复杂度，要看它保存的信息是否支持问题真正需要的查询。

Listing 3.3 为 `pair` 坐标提供哈希函数

```
1 struct PairHash {
2     std::size_t operator()(const std::pair<int, int>& point) const noexcept
3     {
4         constexpr unsigned COORDINATE_BITS = 32U;
5         const auto first = static_cast<std::uint64_t>(
6             static_cast<std::uint32_t>(point.first));
7         const auto second = static_cast<std::uint64_t>(
8             static_cast<std::uint32_t>(point.second));
9         return std::hash<std::uint64_t>{}((first << COORDINATE_BITS) ^ second);
10    }
```

```

11 };
12
13 std::unordered_set<std::pair<int, int>, PairHash> visited;

```

堆解决的是动态极值问题。C++ 里通常使用 `std::priority_queue`，默认是大顶堆；如果要小顶堆，需要显式给出容器和比较器。堆的插入和弹出堆顶都是 $O(\log n)$ ，查看堆顶是 $O(1)$ 。它不适合频繁删除任意元素，也不适合遍历出有序结果而不破坏结构。换句话说，堆只承诺“我能很快告诉你当前最大或最小是谁”。

二叉堆的物理存储通常是数组，而不是指针树。把数组下标从 0 开始看，节点 i 的左孩子是 $2*i+1$ ，右孩子是 $2*i+2$ ，父节点是 $(i-1)/2$ 。堆只维护局部顺序：父节点不小于孩子就是大顶堆，父节点不大于孩子就是小顶堆。它不保证整棵树中序有序，也不能像平衡树那样快速找任意排名。插入新元素时先放到数组末尾，再不断和父节点比较并上浮；弹出堆顶时把末尾元素移到根，再不断和更合适的孩子交换并下沉。

堆操作表

操作	复杂度	原理说明
top	$O(1)$	堆顶元素就在数组第一个位置。
push	$O(\log n)$	新元素沿树高上浮。
pop	$O(\log n)$	根被替换后沿树高下沉。
建堆	$O(n)$	从最后一个非叶子节点向前下沉。
删除任意元素	通常不支持	<code>priority_queue</code> 没有定位句柄。

Listing 3.4 手写小顶堆上浮和下沉的核心动作

```

1 class BinaryMinHeap {
2 public:
3     void push(int value)
4     {
5         this->values_.push_back(value);
6         this->sift_up(static_cast<int>(this->values_.size()) - 1);
7     }
8
9     void pop()
10    {
11        if (this->values_.empty()) {
12            return;
13        }
14        this->values_[0] = this->values_.back();
15        this->values_.pop_back();
16        this->sift_down(0);
17    }
18
19    int top() const
20    {
21        return this->values_.front();
22    }
23
24 private:
25     void sift_up(int index)
26     {
27         while (index > 0) {
28             const int parent = (index - 1) / 2;

```

```

29         if (this->values_[parent] <= this->values_[index]) {
30             break;
31         }
32         std::swap(this->values_[parent], this->values_[index]);
33         index = parent;
34     }
35 }
36
37 void sift_down(int index)
38 {
39     const int n = static_cast<int>(this->values_.size());
40     while (true) {
41         int best = index;
42         const int left = index * 2 + 1;
43         const int right = index * 2 + 2;
44         if (left < n && this->values_[left] < this->values_[best]) {
45             best = left;
46         }
47         if (right < n && this->values_[right] < this->values_[best]) {
48             best = right;
49         }
50         if (best == index) {
51             break;
52         }
53         std::swap(this->values_[best], this->values_[index]);
54         index = best;
55     }
56 }
57
58 std::vector<int> values_;
59 };

```

这段代码不是为了替代标准库，而是为了看清堆为什么是 $O(\log n)$ 。完全二叉树高度约为 $\log n$ ，上浮和下沉每次只向上一层或下一层移动，所以单次插入和弹出堆顶最多走树高。真实写题时应优先用 `std::priority_queue`，除非题目需要 decrease-key、删除任意元素、或者需要同时维护位置索引，这时普通 `priority_queue` 就不够，需要额外结构或改用其他数据结构。

Listing 3.5 `priority_queue` 的大顶堆和小顶堆

```

1 std::priority_queue<int> max_heap;
2
3 std::priority_queue<int,
4                     std::vector<int>,
5                     std::greater<int>> min_heap;
6
7 max_heap.push(3);
8 max_heap.push(10);
9 const int largest = max_heap.top();
10 max_heap.pop();

```

Top K 是堆最经典的用法。求数组中第 K 大元素，直觉做法是排序，复杂度 $O(n \log n)$ 。如果只关心前 K 大，不需要完整排序，可以维护一个大小为 K 的小顶堆。堆顶是当前前 K 大中最小的元素；新元素如果不超过堆顶，就不可能进入前 K；如果超过堆顶，就弹出堆顶并插入新元素。最

终堆顶就是第 K 大。

Listing 3.6 第 K 大元素：大小为 K 的小顶堆

```
1 int find_kth_largest(const std::vector<int>& nums, int k)
2 {
3     std::priority_queue<int,
4                         std::vector<int>,
5                         std::greater<int>> heap;
6
7     for (const int x : nums) {
8         if (static_cast<int>(heap.size()) < k) {
9             heap.push(x);
10        } else if (x > heap.top()) {
11            heap.pop();
12            heap.push(x);
13        }
14    }
15
16    return heap.top();
17 }
```

这个方法的复杂度是 $O(n \log k)$ ，适合 k 远小于 n 的场景。还有一种平均线性复杂度的快速选择算法，后文会讲。面试表达时要说明选择堆的理由：不需要完整有序，只需要维护 K 个候选的边界。类似地，合并 K 个升序链表用小顶堆保存每条链表当前头节点；任务调度用堆保存当前最早结束或最高优先级任务；数据流中位数用一个大顶堆和一个小顶堆分别维护较小一半和较大一半。

树把数据组织成层次关系。二叉树题看似在考递归，实际在考状态如何沿边传播。状态可以从父节点传给子节点，例如验证二叉搜索树时传递取值上下界；也可以从子节点汇总到父节点，例如求树的高度、直径、最大路径和；还可以按层从左到右传播，例如层序遍历。只背前序、中序、后序三个词是不够的，必须知道遍历顺序服务于哪类状态。

二叉树节点通常分散在堆上，每个节点保存值、左孩子指针和右孩子指针。它不像数组堆那样要求完全填满，也不像 `vector` 那样连续存储。这样的好处是结构灵活，能自然表达缺失子树；代价是遍历时存在指针跳转和额外空间开销。二叉搜索树在二叉树上增加顺序约束：任意节点左子树所有值小于它，右子树所有值大于它。若树保持平衡，查找、插入、删除是 $O(\log n)$ ；若退化成链表，就会变成 $O(n)$ 。这也是标准库有序容器通常用红黑树等平衡结构，而不是普通 BST 的原因。

树的状态流向

流向	典型问题	状态含义
自顶向下	验证 BST、路径和	父节点把约束或累计值传给子节点。
自底向上	高度、直径、最大路径和	子树把贡献值返回给父节点。
按层扩展	层序遍历、最小深度	队列保存同一距离或同一层节点。
中序顺序	BST 第 k 小、验证 BST	访问顺序对应有序序列。

递归遍历适合表达树的数学结构，但 C++ 代码要知道递归栈成本。高度为 h 的树，递归栈空间是 $O(h)$ ；平衡树 h 约为 $\log n$ ，极端斜树 h 可以达到 n 。显式栈中序遍历把调用栈摊开，空间仍然是 $O(h)$ ，但栈对象由程序控制，不依赖系统调用栈深度。理解两者等价后，树题就不会被“递归写法更短”遮住真实状态。

Listing 3.7 二叉搜索树验证：显式栈中序遍历

```
1 bool is_valid_bst(TreeNode* root)
2 {
3     std::vector<TreeNode*> stack;
```

```

4   TreeNode* current = root;
5   std::optional<long long> previous;
6
7   while (current != nullptr || !stack.empty()) {
8       while (current != nullptr) {
9           stack.push_back(current);
10          current = current->left;
11      }
12
13      current = stack.back();
14      stack.pop_back();
15
16      if (previous.has_value() && current->value <= previous.value()) {
17          return false;
18      }
19      previous = current->value;
20      current = current->right;
21  }
22
23  return true;
24 }

```

二叉搜索树的关键性质是中序遍历严格递增。验证 BST 的暴力想法可能是对每个节点扫描整棵左子树和右子树，判断左边都小、右边都大，这会重复访问大量节点。中序遍历把全局约束转成相邻访问值的递增关系，每个节点只处理一次。这里用显式栈而不是递归，是为了避免极端深树导致 C++ 调用栈过深，同时也让“遍历状态”更清楚。

Listing 3.8 BST 中查找目标：利用有序性迭代下降

```

1  TreeNode* search_bst(TreeNode* root, int target)
2  {
3      TreeNode* current = root;
4      while (current != nullptr) {
5          if (current->value == target) {
6              return current;
7          }
8          if (target < current->value) {
9              current = current->left;
10         } else {
11             current = current->right;
12         }
13     }
14     return nullptr;
15 }

```

这段代码体现了 BST 和普通二叉树的差别。普通二叉树没有顺序约束，查找目标最坏要遍历所有节点；BST 每一步能排除一整棵左子树或右子树，前提是树没有严重失衡。若输入已经有序，普通插入构造的 BST 会退化成链表，查找复杂度从 $O(\log n)$ 变为 $O(n)$ 。平衡树的旋转、红黑性质和 AVL 高度约束，本质都是为了避免这类退化。

树题经常需要区分“节点为空的返回值”和“答案的初始值”。求最大深度时，空节点深度是 0；求最大路径和时，答案不能初始化为 0，因为所有节点可能都是负数；求最近公共祖先时，返回值表示“当前子树是否找到了目标节点或祖先”。这些不是小细节，而是树形 DP 的语义。每个返回值都应该能用一句话解释：它代表当前子树对父节点贡献什么信息。

图是最通用的关系结构。只要有状态和一步转移，就可以建模成图。图题最重要的不是 DFS 或 BFS 写法，而是建模四问：节点是什么，边是什么，访问状态是什么，队列或栈里保存什么。岛屿数量里，节点是陆地格子，边是上下左右相邻；课程表里，节点是课程，有向边表示先修依赖；单词接龙里，节点是单词，边表示一次字符变化；开锁问题里，节点是四位密码状态，边是转动一位数字。

图的存储方式要跟输入规模匹配。稠密图可以用邻接矩阵，判断两点是否有边是 $O(1)$ ，但空间是 $O(n^2)$ 。大多数面试题是稀疏图，更适合邻接表，空间是 $O(n + m)$ 。如果节点编号连续，用 `vector<vector<int>>`；如果节点是字符串或稀疏编号，用哈希表映射到编号，或者直接用 `unordered_map<string, vector<string>>`。

图的三种常见表示

表示	适用场景	成本和特点
邻接矩阵	点少、图稠密、频繁查边	空间 $O(V^2)$ ，查边快，遍历邻居慢。
邻接表	图稀疏、需要遍历邻居	空间 $O(V + E)$ ，遍历所有边自然。
边表	Kruskal、批量排序边	直接保存边集合，适合按权重排序。

建图时要先决定边方向。课程表的输入 `[course, prerequisite]` 表示先学 `prerequisite` 才能学 `course`，所以边应从先修课指向后续课；如果方向反了，入度含义就会反，拓扑排序结果也会错。无向图要把一条边加入两次；有向图只加入一次。带权图的邻接表元素通常是 `pair<to, weight>`；无权图只存 `to`。把这些语义写清，后面的 BFS、Dijkstra、拓扑排序才不会变成玄学。

Listing 3.9 带权图邻接表：边保存终点和权重

```
1 std::vector<std::vector<std::pair<int, int>>> build_weighted_graph(
2     int node_count,
3     const std::vector<std::array<int, 3>>& edges)
4 {
5     std::vector<std::vector<std::pair<int, int>>> graph(node_count);
6     for (const auto& edge : edges) {
7         const int from = edge[0];
8         const int to = edge[1];
9         const int weight = edge[2];
10        graph[from].push_back({to, weight});
11    }
12    return graph;
13 }
```

图算法的复杂度通常直接来自表示法。邻接表 BFS 每个点入队一次，每条边检查一次，所以是 $O(V + E)$ ；邻接矩阵 BFS 即使边很少，也可能对每个点扫描一整行矩阵，成本接近 $O(V^2)$ 。如果节点是字符串，先映射成整数编号通常能让邻接表更紧凑，也让数组保存距离、入度和访问状态更方便。哈希表适合做从外部 key 到内部编号的转换，不一定适合贯穿整个图算法。

Listing 3.10 课程表：邻接表和入度数组

```
1 bool can_finish(int num_courses,
2                 const std::vector<std::vector<int>>& prerequisites)
3 {
4     std::vector<std::vector<int>> graph(num_courses);
5     std::vector<int> indegree(num_courses, 0);
6
7     for (const auto& edge : prerequisites) {
8         const int course = edge[0];
9         const int prerequisite = edge[1];
10        graph[prerequisite].push_back(course);
```

```

11     ++indegree[course];
12 }
13
14 std::queue<int> queue;
15 for (int course = 0; course < num_courses; ++course) {
16     if (indegree[course] == 0) {
17         queue.push(course);
18     }
19 }
20
21 int learned = 0;
22 while (!queue.empty()) {
23     const int course = queue.front();
24     queue.pop();
25     ++learned;
26
27     for (const int next : graph[course]) {
28         --indegree[next];
29         if (indegree[next] == 0) {
30             queue.push(next);
31         }
32     }
33 }
34
35 return learned == num_courses;
36 }

```

拓扑排序的暴力直觉是反复扫描所有课程，找当前没有前置依赖的课。这样每学一门课都可能重新扫描所有边。入度数组把“还有多少依赖没被满足”变成可增量更新的状态；队列保存当前入度为 0 的课程。每条边只在它的起点被移除时处理一次，复杂度 $O(n + m)$ 。如果最后学完的课程数少于总数，说明剩余节点互相依赖，图中有环。

DFS 和 BFS 都可以遍历图，但语义不同。BFS 适合无权最短步数和层次扩散，因为它按距离递增访问状态；DFS 适合连通块标记、路径搜索、拓扑状态检测和回溯枚举，因为它沿一条路径深入。C++ 里递归 DFS 简洁，但深度过大可能栈溢出。本书讲递归思想，但实现上会经常给出显式栈版本，让遍历过程、访问标记和状态回退更加可控。

Listing 3.11 岛屿数量：显式栈标记连通块

```

1 int num_islands(std::vector<std::vector<char>>& grid)
2 {
3     if (grid.empty() || grid[0].empty()) {
4         return 0;
5     }
6
7     constexpr char LAND = '1';
8     constexpr char WATER = '0';
9     constexpr std::array<std::pair<int, int>, 4> DIRECTIONS{
10         std::pair{1, 0}, std::pair{-1, 0},
11         std::pair{0, 1}, std::pair{0, -1}
12     };
13
14     const int rows = static_cast<int>(grid.size());
15     const int cols = static_cast<int>(grid[0].size());

```

```

16     int islands = 0;
17
18     for (int row = 0; row < rows; ++row) {
19         for (int col = 0; col < cols; ++col) {
20             if (grid[row][col] != LAND) {
21                 continue;
22             }
23
24             ++islands;
25             std::vector<std::pair<int, int>> stack{{row, col}};
26             grid[row][col] = WATER;
27
28             while (!stack.empty()) {
29                 const auto [r, c] = stack.back();
30                 stack.pop_back();
31
32                 for (const auto [dr, dc] : DIRECTIONS) {
33                     const int nr = r + dr;
34                     const int nc = c + dc;
35                     if (nr < 0 || nr >= rows || nc < 0 || nc >= cols ||
36                         grid[nr][nc] != LAND) {
37                         continue;
38                     }
39                     grid[nr][nc] = WATER;
40                     stack.emplace_back(nr, nc);
41                 }
42             }
43         }
44     }
45
46     return islands;
47 }

```

这段代码把访问过的陆地直接改成水，省掉单独的 `visited` 数组。这样做是否合适取决于题目是否允许修改输入。如果不允许，就创建同样大小的布尔矩阵。图题里访问标记必须在入队或入栈时就设置，而不是弹出时才设置；否则同一个节点可能被多个邻居重复加入，轻则浪费时间，重则导致状态爆炸。

并查集维护动态连通性。它有两个核心操作：`find` 找到一个元素所在集合的代表，`unite` 合并两个集合。路径压缩让查找时顺手把沿途节点直接挂到根上，按大小或按秩合并让树保持较浅。两者结合后，单次操作的均摊复杂度接近常数。它适合回答“是否在同一组”“合并后还有几个连通块”，不适合回答两点之间的具体路径或最短距离。

并查集的内存布局非常简单：`parent[i]` 保存元素 `i` 的父节点，根节点满足 `parent[root] == root`；`size[i]` 或 `rank[i]` 只在根节点上有稳定含义，用来决定合并方向。它不是保存一棵业务意义上的树，而是保存集合代表关系。路径压缩会改变很多节点的父指针，但不会改变集合划分；这就是为什么它能快，却不能用来还原真实路径。若题目需要两点之间经过哪些边，应该用图搜索或最短路，而不是并查集。

并查集操作表

操作	均摊复杂度	说明
<code>find(x)</code>	近似 $O(1)$	返回集合代表，同时压缩路径。
<code>unite(a,b)</code>	近似 $O(1)$	合并两个代表，按大小或秩控制树高。
判断同组	近似 $O(1)$	比较两个 <code>find</code> 结果。
统计连通块	取决于维护方式	可在成功合并时减少计数。
查询具体路径	不适合	并查集只保留集合关系，不保留路径。

这里的“近似常数”更准确地说是反阿克曼函数级别，实际输入规模下可以当作常数理解。教材不要求记住这个函数形式，但要知道复杂度结论依赖两件事同时存在：路径压缩和按大小或按秩合并。只做其中一个通常也很快，但理论保证不如二者结合；两个都不做时，父指针链可能退化得很长。

Listing 3.12 并查集：路径压缩与按大小合并

```
1 class DisjointSetUnion {
2 public:
3     explicit DisjointSetUnion(int n)
4         : parent_(n), size_(n, 1)
5     {
6         std::iota(this->parent_.begin(), this->parent_.end(), 0);
7     }
8
9     int find(int x)
10    {
11        int root = x;
12        while (this->parent_[root] != root) {
13            root = this->parent_[root];
14        }
15        while (this->parent_[x] != x) {
16            const int next = this->parent_[x];
17            this->parent_[x] = root;
18            x = next;
19        }
20        return root;
21    }
22
23    bool unite(int a, int b)
24    {
25        int root_a = this->find(a);
26        int root_b = this->find(b);
27        if (root_a == root_b) {
28            return false;
29        }
30        if (this->size_[root_a] < this->size_[root_b]) {
31            std::swap(root_a, root_b);
32        }
33        this->parent_[root_b] = root_a;
34        this->size_[root_a] += this->size_[root_b];
35        return true;
36    }
37
38 private:
```

```

39     std::vector<int> parent_;
40     std::vector<int> size_;
41 };

```

冗余连接是并查集的标准例题。给一棵树额外加一条边，依次扫描边。如果一条边连接的两个点已经在同一集合里，那么再加入它就会形成环，这条边就是冗余边。暴力方法可以每次加边后做一次 DFS 检查连通性，复杂度很高；并查集把“两个点是否已经连通”维护成近乎常数时间查询。

账户合并也能体现并查集的建模能力。邮箱是连接账户的证据：两个账户只要共享一个邮箱，就属于同一个人。可以把账户编号作为并查集元素，用哈希表记录邮箱第一次出现在哪个账户；再次看到同一邮箱时，把两个账户合并。最后按根节点收集邮箱并排序。这里哈希表负责从字符串邮箱找到账户编号，并查集负责维护账户之间的连通分量。一个题常常需要多个数据结构协作，这也是“数据结构和算法不分家”的真实含义。

数据结构和算法不分家

优化通常来自查询模型的改变。哈希表把历史查找变快，堆把动态极值变快，树把层次状态和有序性质显式化，图把状态转移统一成边，并查集把动态连通性维护变快。刷题复盘时必须同时写出算法动作和数据结构职责：保存什么，支持什么查询，删除什么过期状态，为什么不会漏答案。

本章建议配套练习：哈希表做 1、49、128、560、146；堆做 215、347、23、295、621；树做 94、98、102、104、105、124、236、230；图做 200、207、210、417、994、127、752；并查集做 547、684、721、990、1319。每道题至少复盘一次建模：如果不用这个结构，暴力瓶颈在哪里；用了这个结构，哪一步查询或合并被降下来了；结构本身有什么边界和失效条件。

非线性结构的教材级展开

非线性结构解决的是“关系查询”而不只是“顺序扫描”。哈希表把对象映射到桶，堆把候选组织成局部有序的完全二叉树，平衡树维护全局有序关系，Trie 把字符串前缀共享起来，树状数组和线段树把区间信息按层聚合，图把任意状态连接成网络，并查集把动态连通关系压缩成集合代表。经典教材会把这些结构讲得很细，因为它们不仅出现在面试题里，也出现在编译器符号表、数据库索引、操作系统调度、网络路由、文件系统目录和缓存系统中。

学习非线性结构要警惕一个误区：只记复杂度表。复杂度表只告诉你某个操作大概多快，却没有告诉你结构为什么能这么快、在哪些操作上不擅长、变体题需要保存哪些额外信息。比如哈希表平均查询常数，但无法找前驱后继；堆能快速取极值，但不能快速删除任意元素；平衡树能维护有序，但常数比数组扫描大；并查集能判断同组，却不能告诉你最短路径。高质量题解必须把能力边界讲清楚。

哈希表：桶、冲突、负载因子和历史状态

哈希表的核心思想是把 key 通过哈希函数映射到桶。理想情况下，key 均匀分布到桶中，每个桶里元素很少，查询接近常数时间。实际实现需要处理冲突：常见方式有链地址法和开放寻址。C++ 的 `std::unordered_map` 抽象上承诺平均常数时间，具体实现通常围绕桶数组、节点和负载因子展开。当元素数量相对桶数量过高时，容器会 rehash，重新分配桶并把元素放到新桶里。

刷题中，哈希表最常见的职责是保存历史状态。两数之和保存历史值到下标，前缀和题保存历史前缀出现次数，最长连续序列保存所有值的存在性，快乐数保存已经出现过的状态，克隆图保存原节点到新节点的映射。虽然都叫哈希表，保存内容完全不同：有时保存布尔存在，有时保存次数，有时保存最早位置，有时保存对象指针。题目目标决定哈希表的 value 类型。

同一题型的变体常常只改哈希表语义。和为 K 的子数组问数量，哈希表保存前缀和频次；问最长长度，保存前缀和最早出现位置；问是否存在长度至少二的倍数子数组，保存余数最早出现位置；问最短且允许负数，则普通哈希不够，需要前缀和加单调队列。复盘时不能只写“用哈希表”，必须写“哈希表保存什么历史信息”。

Listing 3.13 最长和为 K 的子数组：保存最早前缀位置

```

1 int longest_subarray_sum_k(const std::vector<int>& nums, int target)
2 {
3     std::unordered_map<long long, int> first_index;
4     first_index.reserve(nums.size() + 1);
5     first_index.emplace(0LL, -1);
6
7     long long prefix = 0;
8     int answer = 0;
9     for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
10         prefix += nums[i];
11         const auto found = first_index.find(prefix - target);
12         if (found != first_index.end()) {
13             answer = std::max(answer, i - found->second);
14         }
15         first_index.emplace(prefix, i);
16     }
17     return answer;
18 }

```

这里使用 `emplace` 而不是直接赋值，是为了保留某个前缀和第一次出现的位置。要最长区间，左端越早越好；如果每次更新为最新位置，就会把潜在更长答案破坏掉。这说明同样是前缀和哈希，计数题、最长题、最短题的 `value` 语义完全不同。

哈希表的实际应用远超刷题。编译器符号表用哈希表从名字找到声明信息；解释器运行时环境用哈希表表示对象属性；缓存系统用哈希表定位缓存项；数据库执行层用哈希表做 `hash join`；操作系统内核用哈希表或类似结构快速定位对象。应用越工程化，越要关心哈希函数、内存分配、并发访问和迭代器失效。刷题可以先掌握模型，进阶时必须知道平均常数背后的条件。

堆和优先队列：动态极值、懒删除和多路归并

二叉堆通常用数组表示完全二叉树。下标 i 的左右孩子在 $2*i+1$ 和 $2*i+2$ ，父节点在 $(i-1)/2$ 。大顶堆要求父节点不小于孩子，小顶堆要求父节点不大于孩子。堆不是完全有序结构，只保证堆顶是全局极值。插入时向上调整，弹出堆顶时把末尾元素放到根再向下调整，复杂度都是 $O(\log n)$ 。

优先队列适合“候选集合动态变化，但每次只需要当前最优候选”的问题。Top K、合并 K 个有序链表、数据流中位数、任务调度、Dijkstra、最小区间覆盖 K 个列表、IPO 项目选择都属于这一类。堆不适合的场景是频繁查找或删除任意元素，因为普通 `priority_queue` 只能访问堆顶。

懒删除是堆的重要变体。滑动窗口中位数需要删除窗口左端过期元素，但堆无法直接删除任意元素。常见做法是用哈希表记录某个值应该被删除的次数，真正访问堆顶前，不断弹出已标记删除的堆顶。这种策略把任意删除延迟到元素成为堆顶时执行，保持堆接口简单。Dijkstra 中的过期距离也是同样思想：不更新堆中旧节点，而是压入新距离，弹出时检查是否过期。

Listing 3.14 堆中旧状态懒跳过：Dijkstra 的核心形态

```

1 std::vector<long long> dijkstra(
2     const std::vector<std::vector<std::pair<int, int>>>& graph,
3     int source)
4 {
5     constexpr long long INF = 4'000'000'000'000'000'000LL;
6     using State = std::pair<long long, int>;
7
8     std::vector<long long> distance(graph.size(), INF);
9     std::priority_queue<State, std::vector<State>, std::greater<State>> heap;
10    distance[static_cast<std::size_t>(source)] = 0;
11    heap.emplace(0, source);
12

```

```

13 while (!heap.empty()) {
14     const auto [current_distance, node] = heap.top();
15     heap.pop();
16     if (current_distance != distance[static_cast<std::size_t>(node)]) {
17         continue;
18     }
19
20     for (const auto& [next, weight] : graph[static_cast<std::size_t>(node)]) {
21         const long long candidate = current_distance + weight;
22         if (candidate >= distance[static_cast<std::size_t>(next)]) {
23             continue;
24         }
25         distance[static_cast<std::size_t>(next)] = candidate;
26         heap.emplace(candidate, next);
27     }
28 }
29 return distance;
30 }

```

堆的变种包括二叉堆、d 叉堆、可并堆、斐波那契堆、索引堆和双堆。面试中通常只需要二叉堆和双堆。双堆维护中位数时，一个大顶堆保存较小一半，一个小顶堆保存较大一半；不变量是大顶堆所有值不大于小顶堆所有值，两个堆大小差不超过一。很多错误实现只维护大小不变量，忘了顺序不变量，最终中位数会错。

有序结构：平衡树、前驱后继和区间维护

哈希表不能回答顺序问题。若题目需要找小于等于某值的最大元素、大于等于某值的最小元素、动态维护区间、按时间顺序查询，通常要考虑有序结构。C++ 的 `std::map` 和 `std::set` 通常由红黑树实现，插入、删除、查询和 `lower_bound` 都是 $O(\log n)$ 。它们常数比哈希表大，但能提供顺序语义。

有序结构典型应用包括日程安排、区间合并、滑动窗口内有序统计、在线前驱后继、时间戳键值存储、离线扫描线。比如 My Calendar 类问题中，要插入一个新区间，需要找到第一个起点不小于新区间起点的区间，再检查它和前一个区间是否冲突。哈希表无法回答“相邻区间是谁”，有序 map 正好能回答。

Listing 3.15 有序 map 维护互不重叠区间

```

1 class IntervalSet {
2 public:
3     bool add(int left, int right)
4     {
5         auto next = this->ranges_.lower_bound(left);
6         if (next != this->ranges_.end() && next->first < right) {
7             return false;
8         }
9         if (next != this->ranges_.begin()) {
10             auto previous = std::prev(next);
11             if (previous->second > left) {
12                 return false;
13             }
14         }
15         this->ranges_.emplace(left, right);
16         return true;
17     }

```

```

18
19 private:
20     std::map<int, int> ranges_;
21 };

```

这段代码体现了有序结构的独特能力：通过 `lower_bound` 直接定位可能冲突的相邻区间，而不是扫描所有区间。变体包括闭区间和半开区间的端点差异、允许重叠次数最多为二、插入后合并相邻区间、查询某点被多少区间覆盖。每个变体都要先定义端点语义，再决定比较条件里的等号。

Trie：前缀共享和字符串集合

Trie 把字符串集合组织成字符边组成的树。根到某节点的路径表示一个前缀，节点上的标记表示是否有单词在这里结束。它适合前缀查询、多模式搜索、自动补全、字典树剪枝和按位 Trie 最大异或。相比哈希表，Trie 的优势不是单个完整字符串查询更快，而是前缀共享和前缀剪枝。

实现 Trie 时要根据字符集选择孩子结构。小写英文可以用 `array<int, 26>` 或指针数组；字符集大时可以用哈希表保存孩子；若内存敏感，可以用压缩 Trie 或把节点存进 `vector` 里用整数下标引用。刷题中，单词搜索 II 是 Trie 的代表：如果对每个单词单独 DFS，会重复遍历棋盘；Trie 合并所有单词前缀后，DFS 一旦发现当前前缀不存在，就可以立即剪枝。

Listing 3.16 Trie 节点用数组孩子表达小写字母前提

```

1 class Trie {
2 public:
3     void insert(const std::string& word)
4     {
5         int node = 0;
6         for (const char ch : word) {
7             const int index = ch - 'a';
8             if (this->nodes_[static_cast<std::size_t>(node)].next[index] == -1) {
9                 this->nodes_[static_cast<std::size_t>(node)].next[index] =
10                     static_cast<int>(this->nodes_.size());
11                 this->nodes_.push_back(Node{});
12             }
13             node = this->nodes_[static_cast<std::size_t>(node)].next[index];
14         }
15         this->nodes_[static_cast<std::size_t>(node)].terminal = true;
16     }
17
18     bool starts_with(const std::string& prefix) const
19     {
20         int node = 0;
21         for (const char ch : prefix) {
22             const int index = ch - 'a';
23             node = this->nodes_[static_cast<std::size_t>(node)].next[index];
24             if (node == -1) {
25                 return false;
26             }
27         }
28         return true;
29     }
30
31 private:
32     struct Node {
33         std::array<int, 26> next{};

```



```

34     bool terminal = false;
35
36     Node()
37     {
38         this->next.fill(-1);
39     }
40 };
41
42     std::vector<Node> nodes_{Node{}};
43 };

```

这里使用整数下标而不是裸指针,可以避免手动释放节点,也让节点存储更紧凑。代价是 `vector` 扩容时如果外部保存了节点引用会失效,所以代码只保存下标。字符集假设同样必须说明:输入如果不是小写英文, `ch - 'a'` 就不安全。

树状数组和线段树：区间聚合和动态更新

前缀和适合静态数组区间求和,但如果数组中间会更新,普通前缀和每次更新都要影响后面所有位置。树状数组和线段树解决的是“动态更新 + 区间查询”。树状数组适合可逆前缀聚合,代码短、常数小;线段树更通用,可以维护区间最大、最小、和、懒标记、复杂合并信息。

树状数组把每个位置负责的一段长度设置为最低位一的大小。更新一个点时,沿着 `index += lowbit(index)` 更新所有包含它的树节点;查询前缀时,沿着 `index -= lowbit(index)` 汇总若干不重叠块。它常用于逆序对、动态前缀和、离散化后的频次排名。

Listing 3.17 树状数组：点更新和前缀查询

```

1 class FenwickTree {
2 public:
3     explicit FenwickTree(int n) : tree_(static_cast<std::size_t>(n + 1), 0) {}
4
5     void add(int index, long long delta)
6     {
7         for (int i = index + 1; i < static_cast<int>(this->tree_.size());
8             i += i & -i) {
9             this->tree_[static_cast<std::size_t>(i)] += delta;
10        }
11    }
12
13    long long prefix_sum(int count) const
14    {
15        long long result = 0;
16        for (int i = count; i > 0; i -= i & -i) {
17            result += this->tree_[static_cast<std::size_t>(i)];
18        }
19        return result;
20    }
21
22 private:
23     std::vector<long long> tree_;
24 };

```

这段代码使用外部零基下标,内部一基下标。外部 `add(index, delta)` 更新第 `index` 个元素, `prefix_sum(count)` 查询前 `count` 个元素之和。把接口设计成这种语义,可以减少 `+1` 和 `-1` 在业务代码里到处出现。

线段树把数组按区间递归划分成二叉树。递归概念容易理解，但 C++ 实现可以用迭代线段树避免调用栈。叶子放在底层，父节点保存两个孩子的合并结果。点更新时从叶子改起一路向上重算；区间查询时左右边界向上跳，收集覆盖查询区间的若干节点。它适合动态区间最值、区间和、扫描线、最长连续段、括号匹配信息等。

图表示：邻接矩阵、邻接表、边表和状态图

图结构的核心是节点和边。邻接矩阵适合点数小且边很多的图，判断两点是否相邻是常数，但空间是 $O(n^2)$ 。邻接表适合稀疏图，空间 $O(n+m)$ ，遍历某点邻居高效。边表适合 Kruskal、Bellman-Ford 和离线处理，因为算法按边扫描而不是按点找邻居。网格图可以不显式建图，用方向数组现场生成邻居。

建图题最容易错在边方向和状态维度。课程表中，如果边从课程指向先修课，拓扑输出顺序就和学习顺序相反；单词接龙中，如果每次找邻居都扫描所有单词，BFS 外层正确也会超时；障碍消除最短路中，状态不是位置，而是位置加剩余消除次数；访问所有节点的最短路径中，状态是当前位置加已访问集合。状态图的节点可能不是输入中的一个对象，而是多个变量的组合。

图结构在实际系统中非常常见。编译器构建控制流图和调用图，包管理器维护依赖图，数据库查询优化器处理算子图，网络路由维护拓扑图，操作系统调度和资源等待可以形成等待图。刷题中的 BFS、拓扑排序和最短路，是这些系统问题的简化模型。

并查集：等价关系、扩展权值和回滚

并查集维护的是等价关系。每个集合有一个代表元，`find` 返回代表，`unite` 把两个代表连接起来。路径压缩降低未来查找成本，按大小合并避免树退化。它适合动态合并、判断同组、统计连通块、检测无向环和 Kruskal 最小生成树。

并查集的变体很多。带权并查集维护节点到父节点的权值关系，可用于除法求值、相对距离和势能差；奇偶并查集维护节点到根的奇偶关系，可用于二分图约束和敌友关系；带大小并查集维护集合规模；回滚并查集支持撤销合并，常用于离线动态连通性。每个变体都遵循同一原则：合并代表时，额外信息必须同步更新到新的父子关系上。

Listing 3.18 并查集统计连通块数量

```

1 class ComponentSet {
2 public:
3     explicit ComponentSet(int n)
4         : parent_(static_cast<std::size_t>(n)),
5           size_(static_cast<std::size_t>(n), 1),
6           components_(n)
7     {
8         std::iota(this->parent_.begin(), this->parent_.end(), 0);
9     }
10
11     bool unite(int lhs, int rhs)
12     {
13         int root_lhs = this->find(lhs);
14         int root_rhs = this->find(rhs);
15         if (root_lhs == root_rhs) {
16             return false;
17         }
18         if (this->size_[static_cast<std::size_t>(root_lhs)] <
19             this->size_[static_cast<std::size_t>(root_rhs)]) {
20             std::swap(root_lhs, root_rhs);
21         }
22         this->parent_[static_cast<std::size_t>(root_rhs)] = root_lhs;
23         this->size_[static_cast<std::size_t>(root_lhs)] +=

```

```

24         this->size_[static_cast<std::size_t>(root_rhs)];
25         --this->components_;
26         return true;
27     }
28
29     int components() const { return this->components_; }
30
31 private:
32     int find(int x)
33     {
34         int root = x;
35         while (this->parent_[static_cast<std::size_t>(root)] != root) {
36             root = this->parent_[static_cast<std::size_t>(root)];
37         }
38         while (this->parent_[static_cast<std::size_t>(x)] != x) {
39             const int next = this->parent_[static_cast<std::size_t>(x)];
40             this->parent_[static_cast<std::size_t>(x)] = root;
41             x = next;
42         }
43         return root;
44     }
45
46     std::vector<int> parent_;
47     std::vector<int> size_;
48     int components_ = 0;
49 };

```

这段代码把连通块数量作为并查集状态维护。每次成功合并两个不同集合，连通块数减一；重复合并不改变数量。省份数量、网络连通操作次数、岛屿动态添加都可以使用这个模式。若题目需要知道集合内最大值、边数或权值差，只需要在根节点上维护额外数组，并在合并时更新。

非线性结构的应用谱和实验

哈希表应用：两数之和、字母异位词分组、前缀和计数、克隆图、LRU 索引、字符串到编号映射。堆应用：Top K、动态中位数、Dijkstra、多路归并、任务调度、最小区间。平衡树应用：日程安排、前驱后继、动态区间、滑动窗口有序统计。Trie 应用：前缀搜索、单词搜索 II、最大异或、敏感词过滤。树状数组和线段树应用：逆序对、区间和、扫描线、动态排名、范围更新。图应用：依赖分析、最短路、连通块、拓扑排序。并查集应用：动态连通、冗余连接、账户合并、等式约束、最小生成树。

非线性结构实验：第一，给 `unordered_map` 设置不同 `reserve`，统计 rehash 次数和运行时间。第二，用完整排序、大小为 K 的堆、快速选择分别求第 K 大，比较不同 K 下的表现。第三，用 `map` 实现日程安排，再用无序表尝试，解释为什么无序表无法找相邻区间。第四，用树状数组统计逆序对，并手算离散化后的更新和查询。第五，把并查集的路径压缩关掉，构造链式合并，观察查找路径长度变化。

非线性结构选型

需要历史精确查询，用哈希表；需要当前极值，用堆；需要前驱后继或动态有序，用平衡树；需要大量前缀共享，用 Trie；需要动态前缀或区间聚合，用树状数组或线段树；需要状态转移和可达性，用图；只需要合并集合和判断同组，用并查集。每次选型都要同时说明它不能做什么，避免把结构能力无限放大。

非线性结构变种题谱

哈希表变种可以按 value 语义分类。保存下标：两数之和、同构字符串、最长无重复子串的上次出现位置。保存次数：异位词分组、前缀和计数、优美子数组、频率栈。保存最早位置：最长和为 K、连续子数组和、最长等量零一子数组。保存集合代表：并查集邮箱映射、字符串编号、图克隆映射。保存对象迭代器：LRU 缓存、全 $O(1)$ 数据结构。题解里必须写清 value 语义，否则变体一来就会把次数、位置和布尔存在混用。

堆变种可以按“极值来源”分类。固定大小堆维护 Top K，双堆维护中位数，多路堆维护 K 个有序流的当前头，带过期堆处理滑动窗口和 Dijkstra 旧状态，贪心堆在任务调度和 IPO 中选择当前可执行最优项目。每类堆都要补一个过期或比较器问题：Top K 要说明堆顶代表边界；双堆要说明大小和平衡；多路堆要说明堆元素里保存来自哪一路；带过期堆要说明过期检查发生在使用堆顶之前。

平衡树和有序结构变种按查询目的分类。前驱后继类：存在重复元素 III、时间键值存储、最近座位。区间维护类：我的日程安排、插入区间、合并数据流区间。排名统计类：逆序对、区间和个数、滑动窗口中位数。扫描线类：会议室、天际线、矩形面积。若只需要存在性，哈希表更简单；一旦需要“最近的更大/更小”“第 k 个”“区间相邻关系”，就要转向有序结构、树状数组或线段树。

Trie 变种按边的含义分类。字符 Trie 用于单词前缀和字典匹配；反向 Trie 用于后缀查询和流式字符匹配；压缩 Trie 用于减少长单链空间；二进制 Trie 用于最大异或、按位贪心和网络前缀匹配；AC 自动机把 Trie 和失败指针结合，用于多模式匹配。刷题中最常见的是字符 Trie 和二进制 Trie。二进制 Trie 的边是 0/1 位，不是字符；每次为了最大异或，优先走当前位相反的分支。

树状数组和线段树变种按更新和查询分类。点更新加区间查询是最基础形态；区间更新加点查询可以用差分思想；区间更新加区间查询需要懒标记或两个树状数组；区间最大值、最小值、最大子段和、最长连续一都需要自定义合并信息。线段树难点不在模板长，而在节点信息能否合并。只要父节点信息可以由左右孩子合成，就可能使用线段树；如果合并依赖更远上下文，就需要换建模。

图变种按边权和状态分类。无权图最短路用 BFS，零一权图用 0-1 BFS，非负权图用 Dijkstra，有负权边但无负环可用 Bellman-Ford 或 SPFA 思想，有向无环图可用拓扑 DP。状态图题要明确状态维度：开锁是四位字符串，访问所有节点是节点加集合，障碍消除是位置加剩余次数，带钥匙迷宫是位置加钥匙掩码。状态维度决定 visited 数组维度，也决定复杂度。

并查集变种按附加信息分类。普通并查集维护连通块，带大小并查集维护集合规模，带权并查集维护相对比值或距离，奇偶并查集维护二分关系，网格并查集把二维坐标映射成一维编号，回滚并查集支持撤销。每个变种都要回答：路径压缩是否影响附加信息，合并两个根时附加信息如何更新，查询需要的是根信息还是节点到根的信息。

工程应用：从刷题结构到系统结构

哈希表和链表组合构成缓存系统的基本骨架。LRU 用哈希表定位节点，用双向链表维护新旧顺序；LFU 在 LRU 基础上增加频次维度，用频次到链表的映射维护同频内部顺序。这个结构也解释了为什么单独一个哈希表或单独一个链表都不够：哈希表不知道谁最旧，链表不能常数定位 key。

堆和有序树常用于调度。操作系统调度器、定时器、任务队列、网络包优先级，都需要从动态集合中取出某种最优对象。简单定时任务可以用小顶堆按到期时间排序；需要频繁删除和修改优先级时，可能要用有序树或索引堆；需要公平性时，键可能不是绝对时间，而是虚拟运行时间。刷题中的任务调度器和会议室问题，是这些系统问题的简化模型。

Trie 和哈希索引用于检索系统。搜索建议、自动补全、敏感词过滤、路由前缀匹配，都依赖前缀共享或快速定位。普通哈希表适合完整 key 查询；Trie 适合前缀查询；倒排索引适合从词到文档集合；布隆过滤器适合快速判断“肯定不存在或可能存在”。刷题不一定直接考布隆过滤器，但理解这些结构能扩展你对“查询模型”的认识。

图结构用于依赖和流动。包管理依赖、编译依赖、课程安排、任务 DAG、网络路由、社交关系和知识图谱都可以建成图。拓扑排序回答依赖顺序，最短路回答最小代价，连通块回答分组，可达性回答影响范围。真正做工程时，图的节点和边往往来自业务对象，需要先把业务关系抽象成图，再选算法。

变种题迁移标准

看到变种题时，不要先问“它像哪道力扣题”，先问四个问题：查询对象是什么，更新对象是什么，是否需要顺序，是否需要合并历史状态。答案分别会指向哈希、堆、有序结构、Trie、区间树、图或并查集。能从访问模式推出结构，才算真正掌握数据结构。

哈希结构的深层语义：键、值和状态等价

哈希表题真正难的地方不是调用 `find`，而是定义 `key` 和 `value`。`key` 必须表示“哪些状态在题目里应当被看作同一类”，`value` 必须表示“遇到同一类状态时还需要保留什么信息”。两数之和的 `key` 是数值，`value` 是下标；字母异位词分组的 `key` 是字符频次签名，`value` 是字符串列表；同构字符串的 `key` 可以是字符映射，`value` 是另一侧字符；子数组和的 `key` 是前缀和，`value` 可能是次数、最早位置或最近位置。`key` 定义错，哈希表越快越会快地得到错误答案。

状态等价尤其重要。字母异位词里，“**eat**”和“**tea**”的原字符串不同，但字符频次相同，所以它们在分组问题中等价；快乐数里，一个整数经过平方和转移后如果再次出现，就说明进入循环，所以 `key` 是当前数值状态；克隆图里，原节点地址决定唯一身份，节点值可能重复，所以 `key` 应该是原节点指针或稳定编号，而不是节点值。经典教材讲哈希时会强调抽象等价关系，因为这比容器 API 更接近算法本质。

字母异位词分组先从暴力说起。最直接的做法是拿每个单词和已有分组逐个比较：若两个单词排序后相同，或逐字符计数后相同，就放进同一组。这个版本的瓶颈有两层：第一，单词之间会反复两两比较；第二，同一个单词的规范表示可能被重复计算。优化的关键是把“是否属于同一组”改写成一个状态等价 `key`：异位词共享同一份字符频次签名。每个单词只计算一次签名，然后用哈希表或有序 `map` 直接进入对应桶。这样，比较单词对的问题就变成了计算 `key` 并追加到 `value` 列表。

图示：异位词分组的状态等价

原始单词	<code>eat</code> 、 <code>tea</code> 、 <code>ate</code> 是不同字符串。
规范签名	三者字符频次都为 <code>a:1, e:1, t:1</code> 。
哈希键	频次签名作为 <code>key</code> ，表示“属于同一组”的等价状态。
哈希值	<code>value</code> 保存这一组所有原始字符串，输出时直接取桶。

Listing 3.19 异位词分组：频次签名定义状态等价

```

1 std::vector<std::vector<std::string>> group_anagrams(
2     const std::vector<std::string>& words)
3 {
4     constexpr int ALPHABET_SIZE = 26;
5     std::map<std::array<int, ALPHABET_SIZE>, std::vector<std::string>> groups;
6
7     for (const std::string& word : words) {
8         std::array<int, ALPHABET_SIZE> signature{};
9         for (const char ch : word) {
10             ++signature[ch - 'a'];
11         }
12         groups[signature].push_back(word);
13     }
14
15     std::vector<std::vector<std::string>> answer;
16     answer.reserve(groups.size());
17     for (auto& entry : groups) {
18         auto& group = entry.second;
19         answer.push_back(std::move(group));
20     }
21     return answer;

```


22 }

这里用 `std::map` 是为了避免给 `array` 写自定义哈希，同时让输出顺序稳定；若追求平均性能，可以换成 `unordered_map` 并提供哈希函数。这个例子还说明 `value` 的类型由题目输出决定：如果只问有多少组，`value` 可以是计数；如果要返回每组字符串，`value` 必须保存列表。不要把“哈希表”当成固定模板，它只是实现 `key` 到 `value` 的映射。

哈希表的工程问题也不能省略。第一，扩容会 `rehash`，迭代器失效，复杂对象的移动和分配成本会上升。第二，自定义 `key` 要保证相等对象具有相同哈希值，否则容器行为不符合预期。第三，浮点数不适合作为普通哈希 `key`，精度误差会让数学上相等的状态变成不同 `key`。第四，如果输入可能被恶意构造，哈希退化会影响性能。刷题环境通常不要求处理所有工程问题，但写书应该把边界讲出来。

堆结构的边界：局部有序不是全局有序

堆的最重要边界是：它只保证堆顶最优，不保证内部整体有序。很多学生误以为从 `priority_queue` 里遍历出来就是有序序列，这是错的。堆适合反复取当前极值，不适合查找任意元素，不适合直接删除任意元素，不适合回答前驱后继。如果题目需要“窗口内第 `k` 小”“动态排名”“相邻区间”，堆往往不够，需要有序结构、双堆加懒删除，或树状数组和线段树。

双堆维护中位数是堆边界的经典扩展。一个大顶堆保存较小一半，一个小顶堆保存较大一半。中位数来自两个堆顶，而不是某一个全局有序数组。不变量有两个：大小不变量和顺序不变量。大小不变量保证两个堆元素数量差不超过一；顺序不变量保证左堆每个元素都不大于右堆每个元素。只维护大小会出错，因为堆顶可能交叉。

数据流中位数的暴力版本很自然：每来一个数就插入数组，查询中位数时排序或线性选择。若每次完整排序，旧元素之间已经确定的相对关系被反复计算；若维护一个有序数组，插入位置可以二分找到，但中间插入仍要搬移大量元素。中位数只需要中间边界，而不是完整有序序列。于是把数据切成两半：较小一半只关心最大值，较大一半只关心最小值。左半用大顶堆，右半用小顶堆，插入后重平衡，就把“维护全序”降成“维护两侧边界”。

图示：数据流中位数的双堆边界

左半部分	大顶堆 <code>lower</code> 保存较小一半，堆顶是左半最大值。
右半部分	小顶堆 <code>upper</code> 保存较大一半，堆顶是右半最小值。
顺序不变量	<code>lower.top() <= upper.top()</code> ，左半所有元素不大于右半。
大小不变量	两堆大小差不超过一；本书示例让左堆可以多一个。
中位数	奇数取左堆顶，偶数取两个堆顶平均值。

Listing 3.20 数据流中位数：双堆维护大小和顺序

```

1 class MedianFinder {
2 public:
3     void add_num(int value)
4     {
5         if (this->lower.empty() || value <= this->lower.top()) {
6             this->lower.push(value);
7         } else {
8             this->upper.push(value);
9         }
10        this->rebalance();
11    }
12
13    double find_median() const
14    {
15        if (this->lower.size() == this->upper.size()) {
16            return (static_cast<double>(this->lower.top()) +

```

```

17         static_cast<double>(this->upper_.top())) /
18         2.0;
19     }
20     return static_cast<double>(this->lower_.top());
21 }
22
23 private:
24     void rebalance()
25     {
26         if (this->lower_.size() < this->upper_.size()) {
27             this->lower_.push(this->upper_.top());
28             this->upper_.pop();
29         } else if (this->lower_.size() > this->upper_.size() + 1) {
30             this->upper_.push(this->lower_.top());
31             this->lower_.pop();
32         }
33     }
34
35     std::priority_queue<int> lower_;
36     std::priority_queue<int, std::vector<int>, std::greater<int>> upper_;
37 };

```

这段代码规定左堆元素数可以比右堆多一个，因此奇数个元素时中位数在左堆顶。插入时先按值进入对应堆，再平衡大小。若题目要求删除窗口外元素，就要引入懒删除表，并在访问堆顶前清理过期元素。双堆题的难点不是写堆，而是维护不变量并在删除时保持两个堆的有效大小。

堆在工程里常用于调度和归并。定时器系统可以把任务按到期时间放入小顶堆；日志合并可以把多个有序文件的当前行放入小顶堆；Dijkstra 把候选节点按当前距离放入小顶堆；操作系统或线程池可以用优先级队列选择下一项任务。所有这些场景都有同一结构：候选集合动态变化，但每次只需要当前最优候选。若还需要修改任意候选的优先级，普通堆就不够，要用索引堆、有序树或重新插入加过期检查。

有序结构和区间结构：从相邻关系到聚合信息

有序结构回答的是“在顺序上的邻居是谁”。`lower_bound` 找到第一个不小于目标的元素，配合前一个迭代器，就能检查插入区间是否和相邻区间冲突。会议室、日程安排、数据流区间、最近座位、存在重复元素 III，都依赖相邻关系。哈希表再快，也不能告诉你一个值在有序集合中的前驱和后继。

区间结构回答的是“一个范围内的聚合值是什么”。树状数组擅长前缀可逆聚合，线段树擅长更通用的区间合并。选择它们时先问两个问题：更新是点还是区间，查询是点还是区间。点更新加区间查询，树状数组简单；区间更新加区间查询，线段树懒标记更直接；如果只做静态查询，前缀和或稀疏表可能更合适。不要一看到区间就写线段树，很多题只需要排序加扫描线或前缀和。

区间最大值如果数组不变，暴力每次扫描区间就能回答，只是多次查询会重复访问同一段；前缀和只能处理可逆的和，不能直接由两个前缀最大值相减得到区间最大。若又有点更新，静态稀疏表也不合适。线段树的推导来自“区间答案能由左右子区间合并”：一个区间的最大值等于左右半区最大值的较大者。把这个合并关系按层缓存起来，点更新只影响从叶子到根的一条路径，区间查询只需收集若干完整覆盖的小区间。

图示：线段树的区间合并

叶子节点	保存单个数组位置的值。
父节点	保存左右孩子区间的合并结果，例如 <code>max(left, right)</code> 。
点更新	修改一个叶子，只沿父指针向上重算一条路径。
区间查询	把查询区间拆成若干不相交的完整节点区间，再合并这些节点。
成立前提	父区间答案必须能由左右子区间答案合成。

Listing 3.21 迭代线段树：点更新和区间最大值

```

1  class SegmentTree {
2  public:
3      explicit SegmentTree(const std::vector<int>& values)
4          : size_(static_cast<int>(values.size())),
5            tree_(static_cast<std::size_t>(values.size() * 2), 0)
6      {
7          for (int i = 0; i < this->size_; ++i) {
8              this->tree_[static_cast<std::size_t>(this->size_ + i)] = values[i];
9          }
10         for (int i = this->size_ - 1; i > 0; --i) {
11             this->tree_[static_cast<std::size_t>(i)] =
12                 std::max(this->tree_[static_cast<std::size_t>(i * 2)],
13                         this->tree_[static_cast<std::size_t>(i * 2 + 1)]);
14         }
15     }
16
17     void update(int index, int value)
18     {
19         int position = index + this->size_;
20         this->tree_[static_cast<std::size_t>(position)] = value;
21         for (position /= 2; position > 0; position /= 2) {
22             this->tree_[static_cast<std::size_t>(position)] =
23                 std::max(this->tree_[static_cast<std::size_t>(position * 2)],
24                         this->tree_[static_cast<std::size_t>(position * 2 + 1)]);
25         }
26     }
27
28     int range_max(int left, int right) const
29     {
30         constexpr int NEGATIVE_INF = std::numeric_limits<int>::min();
31         int answer = NEGATIVE_INF;
32         int l = left + this->size_;
33         int r = right + this->size_;
34         while (l < r) {
35             if ((l & 1) != 0) {
36                 answer = std::max(answer, this->tree_[static_cast<std::size_t>(l)]);
37                 ++l;
38             }
39             if ((r & 1) != 0) {
40                 --r;
41                 answer = std::max(answer, this->tree_[static_cast<std::size_t>(r)]);
42             }
43             l /= 2;
44             r /= 2;
45         }
46         return answer;
47     }
48
49 private:
50     int size_ = 0;
51     std::vector<int> tree_;
52 };

```

这个实现使用半开区间 `[left, right)`。查询时，若左边界是右孩子，就把它纳入答案并右移；若右边界是右孩子的后一位，就先左移再纳入答案。迭代线段树比递归版本短，也避免 C++ 调用栈问题，但它要求你非常清楚半开区间语义。若题目需要区间加法和区间最大值，就要增加懒标记；若节点信息不是一个整数，而是最大子段和、前缀最大、后缀最大和总和，就要定义可合并的结构体。

树结构：从遍历顺序到信息流

树题不应该只按前序、中序、后序背模板，而应按信息流分类。自顶向下题把父节点信息传给子节点，例如路径和、验证 BST 上下界、二叉树所有路径；自底向上题把子树信息汇总给父节点，例如高度、直径、平衡二叉树、最大路径和；层次题按距离或深度组织节点，例如层序遍历、右视图、最小深度；结构改造题改变节点链接，例如展开二叉树、反转、删除 BST 节点。遍历顺序服务于信息流，而不是目的本身。

二叉搜索树的中序递增只是它的一种表象。更本质的性质是：每个节点把值域划分成左右两个区间。验证 BST 可以中序，也可以传上下界；第 K 小可以中序计数；删除节点时，如果左右孩子都存在，可以用后继节点替换当前值，再删除后继节点；最近公共祖先在 BST 中可以利用值域方向，而普通二叉树必须在左右子树中寻找目标。把值域约束讲清楚，很多 BST 题就能串起来。

树形 DP 的核心是定义返回值。最大路径和中，返回给父节点的不是当前子树内部最大路径，而是“从当前节点向下延伸到某一侧的最大贡献”；全局答案才记录可能经过当前节点同时连接左右两侧的路径。打家劫舍 III 中，一个节点的状态至少要区分偷和不偷；若只返回一个最大值，会丢失父子不能同时选的约束。树题最常见的错误就是返回值语义太含糊。

图结构：状态空间、边权和访问时机

图题的第一步是定义状态，而不是选择 DFS 或 BFS。状态可能是一个节点，也可能是多个变量的组合。普通岛屿题的状态是坐标；障碍消除最短路的状况是坐标加剩余消除次数；访问所有节点的最短路径是当前节点加访问集合；带钥匙迷宫是坐标加钥匙掩码；单词接龙是一个字符串或字符串编号。状态定义少了维度，会把不同情况错误合并；状态定义多了无用维度，会让复杂度爆炸。

访问标记的时机也很关键。普通 BFS 中，通常在入队时标记已访问，而不是出队时标记。因为一个节点可能被多个邻居发现，如果等出队才标记，它会被重复入队。Dijkstra 则不同，一个节点可以用更短距离再次进入堆，弹出时检查当前距离是否过期。并查集又不同，它没有访问队列，而是用集合代表压缩连通关系。不同结构有不同“状态已经确定”的时刻，不能混用。

障碍消除最短路不能直接把状态写成坐标。暴力搜索会枚举从起点出发的所有路径，并在路径里记录已经消除过多少障碍；普通 BFS 若只用 `visited[row][col]`，会把“到达同一格子但剩余消除次数不同”的两种情况错误合并。瓶颈不是 BFS 本身，而是状态定义不完整导致要么漏解，要么保留太多等价路径。优化后的状态至少包含位置和剩余资源；进一步观察，若两条路径到达同一格子，距离按 BFS 层已经不差，而剩余消除次数更多的状态支配剩余更少的状态。因此每个格子只保存见过的最大剩余次数，就能剪掉未来不可能更好的状态。

图示：状态图 BFS 的支配关系

错误状态	只保存 <code>(row, col)</code> ，会把资源不同的路径合并。
完整状态	保存 <code>(row, col, remaining)</code> ，位置和剩余消除次数共同决定未来能力。
支配剪枝	同一格子上，剩余次数更少且距离不差的状态没有保留价值。
压缩记录	<code>best_remaining[row][col]</code> 保存到达该格子时见过的最大剩余资源。
BFS 语义	队列按步数扩展，第一次到达终点时步数最短。

Listing 3.22 状态图 BFS：位置加剩余资源才是完整状态

```
1 int shortest_path_with_elimination(
```

```

2   const std::vector<std::vector<int>>& grid,
3   int eliminations)
4   {
5       if (grid.empty() || grid[0].empty()) {
6           return -1;
7       }
8
9       constexpr std::array<std::pair<int, int>, 4> DIRECTIONS{
10          std::pair{1, 0}, std::pair{-1, 0},
11          std::pair{0, 1}, std::pair{0, -1}
12      };
13
14      const int rows = static_cast<int>(grid.size());
15      const int cols = static_cast<int>(grid[0].size());
16      std::vector<std::vector<int>> best_remaining(
17          static_cast<std::size_t>(rows),
18          std::vector<int>(static_cast<std::size_t>(cols), -1));
19
20      std::queue<std::array<int, 4>> queue;
21      queue.push({0, 0, eliminations, 0});
22      best_remaining[0][0] = eliminations;
23
24      while (!queue.empty()) {
25          const auto [row, col, remaining, distance] = queue.front();
26          queue.pop();
27          if (row == rows - 1 && col == cols - 1) {
28              return distance;
29          }
30
31          for (const auto& [dr, dc] : DIRECTIONS) {
32              const int nr = row + dr;
33              const int nc = col + dc;
34              if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) {
35                  continue;
36              }
37              const int next_remaining =
38                  remaining - grid[static_cast<std::size_t>(nr)]
39                      [static_cast<std::size_t>(nc)];
40              if (next_remaining < 0 ||
41                  next_remaining <= best_remaining[static_cast<std::size_t>(nr)]
42                      [static_cast<std::size_t>(nc)]) {
43                  continue;
44              }
45              best_remaining[static_cast<std::size_t>(nr)]
46                  [static_cast<std::size_t>(nc)] = next_remaining;
47              queue.push({nr, nc, next_remaining, distance + 1});
48          }
49      }
50      return -1;
51 }

```

这里没有用三维 `visited[row][col][remaining]`，而是为每个格子保存到达它时剩余消除次数的最好值。若再次到达同一格子但剩余次数更少或相等，未来不会比之前更好，可以剪掉。这

个剪枝来自状态支配关系：位置相同、距离不更短、资源更少的状态没有保留价值。状态图题经常有这种优化，但前提是你能证明一个状态支配另一个状态。

边权决定最短路算法。无权边用 BFS；0/1 权边用 0-1 BFS；非负权边用 Dijkstra；存在负权边时，Dijkstra 的贪心确定性失效，需要 Bellman-Ford 或其他算法；DAG 最短路可以按拓扑序 DP。题目如果写的是“每一步代价相同”，才是普通 BFS；如果每条边代价不同，即使看起来也是从一个点走到另一个点，也不能直接用普通队列。

并查集：等价类、势能和离线思维

并查集维护的是等价类。它能快速回答两个元素是否已经连通，却不能告诉你连通路径是什么，也不能处理删除边后的在线连通性。这个能力边界很重要。省份数量、冗余连接、账户合并、等式可满足性，都只需要合并和查询同组；最短路径、路径条数、删除某条边后是否仍连通，就不是普通并查集擅长的范围。

带权并查集把“到父节点的关系”也保存下来。除法求值中，若 $a / b = 2$ ，可以把 a 和 b 放到同一集合，并维护每个节点到根的比例。查询 x / y 时，如果两者根不同，答案不存在；根相同，则用 x/root 除以 y/root 。奇偶并查集也是同理，只是权值从比例变成奇偶关系。难点在合并两个根时，要根据已知约束推导新父子关系上的权值。

回滚并查集和离线动态连通则体现了并查集的高级用法。普通路径压缩会破坏历史，不适合回滚；如果要支持撤销，常用按大小合并并记录被修改的父亲和大小，撤销时恢复。区间时间上的边添加和删除可以用线段树分治把边分配到生效时间段，再配合回滚并查集处理查询。面试不常要求手写完整实现，但理解这个方向能帮助你了解普通并查集为什么不能删除边，以及离线算法如何绕开在线限制。

数据结构组合：一道题往往不只一种结构

很多高质量面试题不是考单个结构，而是考组合。LRU 是哈希表加双向链表；LFU 是哈希表加频次表加链表；Twitter 设计题是哈希表加链表或数组加堆归并；单词搜索 II 是 Trie 加 DFS 回溯；账户合并是哈希表加并查集；滑动窗口中位数是双堆加延迟删除哈希表；最小覆盖区间是排序加堆或多指针；天际线是扫描线加堆或有序表。组合题要求你给每个结构分配明确职责。

设计题可以按数据流拆解。先列出操作：插入、删除、查询、移动、合并、取极值、取前驱后继、按前缀查找。再为每个操作找结构能力。若两个操作互相冲突，就用两个结构互相索引，并维护一致性。LRU 中，哈希表和链表必须同时更新；Twitter 中，用户到推文列表的映射和关注关系必须分开；滑动窗口中位数中，堆里有旧值，延迟删除表必须在取堆顶前清理。组合题的 bug 往往来自一个结构更新了，另一个结构没同步。

组合结构的写法

先写每个结构的职责，再写同步规则。职责包括：谁负责定位，谁负责顺序，谁负责极值，谁负责连通性，谁负责前缀，谁负责区间。同步规则包括：插入时更新哪些结构，删除时更新哪些结构，移动时是否保持迭代器有效，查询前是否要清理过期状态。没有同步规则的组合结构很容易在边界输入上失效。

非线性结构题目的讲解标准

非线性结构题讲解至少要回答六个问题。第一，状态或对象是什么。第二，结构里的 key、节点、边、堆元素或集合代表分别表示什么。第三，暴力方法慢在哪里，是查找历史、找极值、找相邻、查前缀、查区间，还是判断连通。第四，所选结构把哪一步降到了什么复杂度。第五，结构的限制是什么，比如哈希无序、堆不能任意删除、并查集不能给路径。第六，变体来临时如何调整 value、比较器、状态维度或附加信息。

以课程表为例，对象是课程，边是先修关系，暴力慢在反复找没有依赖的课程；邻接表保存从一门课指向依赖它的后续课程，入度数组保存未满足依赖数量，队列保存当前可学课程。每弹出一门课，就减少后续课程入度。若最终弹出数量不足，说明有环。变体课程表 II 需要输出顺序，只需记录弹出序列；若课程有持续时间并要求最短完成时间，就要在拓扑序上做 DP；若依赖会动态增删，普通拓扑排序就不够。

以账户合并为例，对象是账户和邮箱。哈希表把邮箱映射到第一次出现的账户，并查集合并共

享邮箱的账户，最后按根收集邮箱。暴力方法可能两两比较账户邮箱集合，复杂度很高；哈希表让共享邮箱检测变成平均常数，并查集让账户连通分量合并接近常数。变体如果要求按用户名区分，key 就必须包含用户名或先按用户名分组；如果邮箱大小写不敏感，就要先标准化邮箱。题解必须把这些建模前提说清楚。

以单词搜索 II 为例，对象是棋盘路径和单词集合。暴力做法是对每个单词在棋盘上搜索一次，重复遍历大量相同前缀。Trie 把所有单词的公共前缀合并，DFS 当前路径同时对应 Trie 中一个节点；如果当前字符没有对应边，就立即剪枝。这里 Trie 负责前缀查询，DFS 负责空间路径，访问标记负责同一单元格不能重复使用。变体如果允许八方向移动，只改邻居生成；如果允许重复使用格子，访问标记规则要改；如果字符集很大，Trie 孩子结构也要改。

把这些标准坚持下来，非线性结构就不会变成孤立模板。哈希、堆、树、图、并查集只是不同查询模型的工具；教材的任务是让读者知道每个工具解决什么瓶颈、付出什么代价、在什么条件下失效，以及如何和其他结构组合。

第零部分

核心算法：从暴力到优化

第 4 章

数组、双指针和滑动窗口

数组题是刷题的地基。双指针和滑动窗口不是孤立技巧，而是从暴力枚举子结构中优化出来的方法。双指针常用于有序数组、两端收缩和左右边界协作；滑动窗口常用于连续子数组或子串，并且窗口状态能够增量维护。

连续结构的原理地图

数组题先不要急着找“模板”，要先看题目要求的候选对象是什么。候选对象通常只有四类：单个元素、两个元素组成的配对、连续区间、若干个切分点。单个元素题多半是扫描、计数或比较；配对题会走向哈希表、排序双指针或二分；连续区间题会走向前缀和、差分、滑动窗口、单调队列；切分点题会走向 DP、贪心或答案二分。只要候选对象没有分清，后面的优化就会变成背题号。

暴力枚举是数组题的起点，不是低质量答案。枚举所有左右端点，可以覆盖每个连续子数组；枚举所有下标对，可以覆盖每个二元关系；枚举每个切分点，可以覆盖每种分段。真正的问题在于暴力有没有重复工作。例如区间和每次重新累加，就是重复计算相邻区间共有的部分；三数之和每次重新找另外两个数，就是重复比较已经有序的候选；最小覆盖子串每次重新统计子串字符，就是重复维护高度重叠的窗口状态。优化的第一句话应该是“我把哪一类重复工作保存下来了”，而不是“我使用了某某算法”。

数组的物理基础也会影响算法表达。`std::vector` 连续存储，支持下标随机访问，所以二分、前缀和、双指针和 DP 表都天然适合用它。连续内存让顺序扫描具有较好的缓存局部性，也让 `nums[i]` 这种访问能直接做地址计算。代价是中间插入删除要移动元素，尾部追加可能触发扩容，扩容后旧指针、引用和迭代器可能失效。刷题时我们很少在数组中间频繁插入；一旦题目需要稳定节点身份或常数时间移动节点，就应该想到链表或索引结构，而不是继续勉强使用数组。

区间语义必须统一。闭区间 `[left, right]` 的长度是 `right - left + 1`，右端最后一个元素真实参与窗口；半开区间 `[left, right)` 的长度是 `right - left`，右端表示下一个待处理位置。两种写法都可以，但一章代码里最好有主语义。前缀和天然适合半开区间，因为 `prefix[i]` 表示前 `i` 个元素，区间 `[left, right)` 的和就是 `prefix[right] - prefix[left]`。滑动窗口的代码如果写成闭区间，也要明确什么时候把右端加入状态，什么时候把左端移出状态。大多数 off-by-one 错误，都不是算术不好，而是区间含义没有先写清楚。

双指针不是“两个变量相向移动”的同义词。它成立需要一个排除理由。两端收缩型题目里，当前左右端点之外的候选已经处理过，移动哪一侧必须能证明不会丢掉更优答案；排序配对型题目里，和太小移动左端、和太大移动右端，依赖的是有序性；同向快慢指针里，慢指针通常代表已整理前缀的下一个写入位置，快指针代表正在读取的输入位置。三种模型的指针外形相似，但不变量完全不同。复盘时要写“被排除的候选为什么不可能成为答案”，否则只是把循环跑了一遍。

滑动窗口更严格。它需要三个条件同时成立：答案对象是连续区间；边界移动后状态能用常数或近似常数成本增删；移动策略不会漏掉合法答案。正整数数组的最短子数组和至少为目标，可以用窗口，因为右端扩展只会让和变大，左端收缩只会让和变小。含负数数组就不行，因为负数进入可能让和变小，负数离开可能让和变大，窗口的单调判断被破坏。无重复字符子串可以用窗口，不是因为字符问题天然窗口化，而是因为重复字符的上次出现位置能告诉左端至少要跳到哪里，且左端不会回退。

数组窗口题的推理顺序

先写候选空间，再写暴力枚举；接着定位重复工作，判断它来自重复求和、重复查找、重复统计还是重复比较；然后选择保存状态的结构，说明状态如何随边界变化；最后证明每个下标为什么只会被处理常数次。若题目条件变化，例如数组从正数变成可负、从连续子数组变成子序列、从静态数组变成动态更新，原来的窗口或前缀模型必须重新审查。

盛最多水的容器暴力解枚举左右边界。面积由宽度和较短边决定，所以每个下标对都可能成为

候选。

Listing 4.1 LeetCode 11 盛最多水的容器：暴力解

```

1 int max_area_bruteforce(const std::vector<int>& height)
2 {
3     int best = 0;
4     for (int left = 0; left < static_cast<int>(height.size()); ++left) {
5         for (int right = left + 1; right < static_cast<int>(height.size()); ++right) ↵
6             ↵ {
7                 const int width = right - left;
8                 const int bounded_height = std::min(height[left], height[right]);
9                 best = std::max(best, width * bounded_height);
10            }
11    }
12    return best;
13 }
```

暴力法是 $O(n^2)$ 。优化来自候选淘汰。初始取最宽的左右边界，如果移动较高的一边，宽度变小，而较短边没有变高，面积不可能变大。因此只能移动较短边，才有机会提高瓶颈高度。

Listing 4.2 LeetCode 11 盛最多水的容器：双指针解

```

1 int max_area_two_pointers(const std::vector<int>& height)
2 {
3     int left = 0;
4     int right = static_cast<int>(height.size()) - 1;
5     int best = 0;
6
7     while (left < right) {
8         const int width = right - left;
9         const int bounded_height = std::min(height[left], height[right]);
10        best = std::max(best, width * bounded_height);
11
12        if (height[left] <= height[right]) {
13            ++left;
14        } else {
15            --right;
16        }
17    }
18    return best;
19 }
```

不变量是：每一步排除的短边，不会和更内侧的任何边形成更优答案，因为宽度只会变小，短边高度仍然是瓶颈。这个证明比“左右指针往中间走”重要得多。

长度最小的子数组展示滑动窗口。暴力法枚举所有起点和终点，遇到和达到目标时更新答案。它慢在每个起点都重新累加。

Listing 4.3 LeetCode 209 长度最小的子数组：滑动窗口

```

1 int min_subarray_len_sliding_window(int target, const std::vector<int>& nums)
2 {
3     int best = std::numeric_limits<int>::max();
4     int left = 0;
5     int sum = 0;
```



```

6
7   for (int right = 0; right < static_cast<int>(nums.size()); ++right) {
8       sum += nums[right];
9       while (sum >= target) {
10          best = std::min(best, right - left + 1);
11          sum -= nums[left];
12          ++left;
13      }
14  }
15
16  return best == std::numeric_limits<int>::max() ? 0 : best;
17 }

```

这个优化依赖正整数。右边界右移时窗口和只会增加，左边界右移时窗口和只会减少。如果数组包含负数，这个单调性消失，滑动窗口可能失效，需要考虑前缀和、单调队列或其他结构。

无重复字符的最长子串也是窗口题，但窗口状态不是和，而是字符最后出现位置。窗口不变量是 `[left, right]` 内没有重复字符。

Listing 4.4 LeetCode 3 无重复字符的最长子串

```

1 int longest_substring_without_repeat_window(const std::string& s)
2 {
3     constexpr int ASCII_RANGE = 256;
4     std::array<int, ASCII_RANGE> last_seen{};
5     last_seen.fill(-1);
6
7     int left = 0;
8     int best = 0;
9
10    for (int right = 0; right < static_cast<int>(s.size()); ++right) {
11        const auto ch = static_cast<unsigned char>(s[right]);
12        if (last_seen[ch] >= left) {
13            left = last_seen[ch] + 1;
14        }
15        last_seen[ch] = right;
16        best = std::max(best, right - left + 1);
17    }
18
19    return best;
20 }

```

这里用 `std::array<int, 256>` 是因为按 ASCII 字符处理。如果题目要求完整 Unicode 字符，需要先明确字符单元，或使用哈希表保存字符到位置。

三数之和是双指针的第二个标准模型。题目要求找出所有不重复三元组，使得三个数之和为 0。暴力方法枚举三个下标，时间复杂度 $O(n^3)$ ，还要处理去重；如果用哈希表固定两数再找第三数，可以降到 $O(n^2)$ ，但去重和重复值处理仍然复杂。真正稳定的做法是先排序，再固定第一个数，把剩下两个数变成有序数组上的两数之和。

排序后的关键性质是：固定 `first` 后，如果 `nums[left] + nums[right]` 太小，说明左端值不够大，只能右移 `left`；如果和太大，说明右端值太大，只能左移 `right`。这就是有序性带来的候选淘汰。去重也依赖排序：同一层循环里，如果当前值和前一个值相同，就跳过；找到一个答案后，左右指针都要跳过连续重复值。

Listing 4.5 LeetCode 15 三数之和：排序加双指针

```

1 std::vector<std::vector<int>> three_sum(std::vector<int> nums)
2 {
3     std::sort(nums.begin(), nums.end());
4     std::vector<std::vector<int>> answer;
5
6     for (int first = 0; first < static_cast<int>(nums.size()); ++first) {
7         if (first > 0 && nums[first] == nums[first - 1]) {
8             continue;
9         }
10        if (nums[first] > 0) {
11            break;
12        }
13
14        int left = first + 1;
15        int right = static_cast<int>(nums.size()) - 1;
16        while (left < right) {
17            const int sum = nums[first] + nums[left] + nums[right];
18            if (sum == 0) {
19                answer.push_back({nums[first], nums[left], nums[right]});
20                const int left_value = nums[left];
21                const int right_value = nums[right];
22                while (left < right && nums[left] == left_value) {
23                    ++left;
24                }
25                while (left < right && nums[right] == right_value) {
26                    --right;
27                }
28            } else if (sum < 0) {
29                ++left;
30            } else {
31                --right;
32            }
33        }
34    }
35
36    return answer;
37 }

```

这里参数按值传入,是因为函数内部要排序。如果不想修改调用者的数组,按值传参是合理的;如果题目允许原地修改,也可以传引用。复杂度是排序 $O(n \log n)$ 加双指针枚举 $O(n^2)$, 总时间 $O(n^2)$, 额外空间不计答案为 $O(1)$ 或排序栈空间。面试表达要强调: 去重不是最后把答案放进 set, 而是在生成答案时就避免重复, 这样更可控。

找到字符串中所有字母异位词是固定长度窗口。题目给定字符串 *s* 和模式串 *p*, 要求找出 *s* 中所有长度等于 *p* 且字符频次完全相同的起点。暴力方法枚举每个长度为 *p.size()* 的子串, 然后排序比较或统计比较。如果每次排序, 复杂度是 $O(nk \log k)$; 如果每次重新统计, 复杂度是 $O(nk)$ 。慢点在于相邻窗口高度重叠, 却重复计算了大部分字符频次。

优化方法是维护固定长度窗口的频次数组。窗口右端每进入一个字符, 就把它的计数加一; 当窗口长度超过模式串长度, 左端字符离开, 计数减一。由于题目通常限定小写英文, 可以用 `std::array<int, 26>`, 比较两个数组是常数时间。窗口不变量是: 处理到 *right* 时, *window* 正好记录当前长度不超过模式串长度的后缀窗口字符频次。

Listing 4.6 LeetCode 438 找到所有字母异位词: 固定长度窗口

```

1 std::vector<int> find_anagrams_window(const std::string& s,
2                                     const std::string& p)
3 {
4     constexpr int ALPHABET_SIZE = 26;
5     std::vector<int> answer;
6     if (s.size() < p.size()) {
7         return answer;
8     }
9
10    std::array<int, ALPHABET_SIZE> need{};
11    std::array<int, ALPHABET_SIZE> window{};
12    for (const char ch : p) {
13        ++need[ch - 'a'];
14    }
15
16    for (std::size_t right = 0; right < s.size(); ++right) {
17        ++window[s[right] - 'a'];
18        if (right >= p.size()) {
19            --window[s[right - p.size()] - 'a'];
20        }
21        if (window == need) {
22            answer.push_back(static_cast<int>(right + 1 - p.size()));
23        }
24    }
25
26    return answer;
27 }

```

这题容易错在下标。窗口第一次达到目标长度是在 `right + 1 == p.size()`，起点是 `right + 1 - p.size()`。如果用 `int` 写下标，要小心 `s.size()` 是无符号类型；本书示例这里使用 `std::size_t` 控制循环，最终起点再转成 `int`，因为力扣返回 `vector<int>`。

最小覆盖子串是变长窗口里最重要的一题。题目要求在 `s` 中找最短子串，覆盖 `t` 中所有字符及其次数。暴力方法枚举所有子串，再检查是否覆盖，复杂度至少 $O(n^2|\Sigma|)$ 。优化的核心是：右边界扩张直到窗口满足覆盖条件，然后尽可能移动左边界缩短窗口；一旦左移导致窗口不满足，再继续右移。这个过程成立，因为右移只会增加字符，左移只会删除字符，覆盖关系可以被增量维护。

Listing 4.7 LeetCode 76 最小覆盖子串：变长窗口

```

1 std::string min_window_substring(const std::string& s, const std::string& t)
2 {
3     constexpr int ASCII_RANGE = 128;
4     std::array<int, ASCII_RANGE> need{};
5     std::array<int, ASCII_RANGE> window{};
6     int required = 0;
7
8     for (const char ch : t) {
9         const auto index = static_cast<unsigned char>(ch);
10        if (need[index] == 0) {
11            ++required;
12        }
13        ++need[index];
14    }
15

```

```

16     int formed = 0;
17     int left = 0;
18     int best_left = 0;
19     int best_length = std::numeric_limits<int>::max();
20
21     for (int right = 0; right < static_cast<int>(s.size()); ++right) {
22         const auto in = static_cast<unsigned char>(s[right]);
23         ++window[in];
24         if (need[in] > 0 && window[in] == need[in]) {
25             ++formed;
26         }
27
28         while (formed == required) {
29             const int length = right - left + 1;
30             if (length < best_length) {
31                 best_length = length;
32                 best_left = left;
33             }
34
35             const auto out = static_cast<unsigned char>(s[left]);
36             --window[out];
37             if (need[out] > 0 && window[out] < need[out]) {
38                 --formed;
39             }
40             ++left;
41         }
42     }
43
44     if (best_length == std::numeric_limits<int>::max()) {
45         return "";
46     }
47     return s.substr(best_left, best_length);
48 }

```

required 表示需要满足的不同字符种类数，**formed** 表示当前窗口里已经满足需求的字符种类数。不能只用窗口总长度判断，因为 $t = \text{"AABC"}$ 时，两个 **A** 都必须覆盖。复杂度是 $O(n + m + |\Sigma|)$ ，在 ASCII 数组固定时可视为 $O(n + m)$ 。面试表达要说清楚：这题不是固定窗口，因为答案长度未知；它是“先满足，再收缩”的最短覆盖模型。

本章题目复盘模板

数组和窗口题复盘时至少写五句话：暴力枚举什么；慢点是否来自重复扫描、重复统计或候选过多；优化使用了有序性、窗口增量状态还是候选淘汰；左右指针移动条件是什么；每个元素为什么只会被处理常数次数。只写“用双指针”或“用滑动窗口”不算会。

本章力扣主线：11 盛最多水的容器、15 三数之和、209 长度最小的子数组、3 无重复字符的最长子串、438 找到字符串中所有字母异位词、76 最小覆盖子串。复盘时必须写出窗口不变量和指针移动条件。

数组题的第一层能力是下标语义。很多错误不是算法不会，而是区间闭开混乱。推荐统一使用半开区间描述窗口： $[left, right)$ 表示包含 **left**，不包含 **right**；窗口长度是 $right - left$ ；右端扩张时先处理 $s[right]$ 再让 **right** 前进，左端收缩时处理 $s[left]$ 再让 **left** 前进。力扣代码里常写闭区间 $[left, right]$ ，也没问题，但全篇必须保持一致。区间语义一乱，最小覆盖子串、二分边界、前缀和都会出错。

前缀和是数组题最基础的缓存。它适合静态数组的区间求和、区间计数、差值判断。若数组会

频繁更新，普通前缀和就不够，因为一次修改会影响后面所有前缀；这时要考虑树状数组或线段树。二维前缀和则把矩形区域求和变成四个角的组合。它的本质仍然是容斥：目标矩形等于大矩形减去上方和左方，再加回被减了两次左上角。

Listing 4.8 二维前缀和：矩形查询

```

1 class MatrixPrefixSum {
2 public:
3     explicit MatrixPrefixSum(const std::vector<std::vector<int>>& matrix)
4     {
5         const int rows = static_cast<int>(matrix.size());
6         const int cols = rows == 0 ? 0 : static_cast<int>(matrix[0].size());
7         this->prefix_.assign(rows + 1, std::vector<int>(cols + 1, 0));
8
9         for (int row = 0; row < rows; ++row) {
10             for (int col = 0; col < cols; ++col) {
11                 this->prefix_[row + 1][col + 1] =
12                     this->prefix_[row][col + 1] +
13                     this->prefix_[row + 1][col] -
14                     this->prefix_[row][col] +
15                     matrix[row][col];
16             }
17         }
18     }
19
20     int query(int top, int left, int bottom, int right) const
21     {
22         return this->prefix_[bottom + 1][right + 1] -
23             this->prefix_[top][right + 1] -
24             this->prefix_[bottom + 1][left] +
25             this->prefix_[top][left];
26     }
27
28 private:
29     std::vector<std::vector<int>> prefix_;
30 };

```

二维前缀和的边界最容易错，所以建议多开一行一列哨兵。`prefix[row + 1][col + 1]` 表示从原矩阵左上角到 `(row, col)` 的闭矩形和。查询时，`bottom` 和 `right` 是原矩阵闭区间端点，因此访问前缀数组要加一。这个模式和一维前缀和完全一致：多出来的第 0 行、第 0 列负责消除边界分支。

差分数组是前缀和的反向思维。前缀和把多次区间查询变快；差分把多次区间修改变快。若对区间 `[left, right]` 加 `delta`，只需要让变化从 `left` 开始，在 `right + 1` 结束。最后对差分数组做一次前缀累加，得到所有位置的最终值。航班预订、拼车、区间加法都属于这个模型。

Listing 4.9 差分数组：区间更新后一次还原

```

1 std::vector<int> apply_range_additions(
2     int length,
3     const std::vector<std::tuple<int, int, int>>& updates)
4 {
5     std::vector<int> diff(length + 1, 0);
6     for (const auto [left, right, delta] : updates) {
7         diff[left] += delta;

```



```

8     if (right + 1 < length) {
9         diff[right + 1] -= delta;
10    }
11 }
12
13 std::vector<int> values(length, 0);
14 int current = 0;
15 for (int i = 0; i < length; ++i) {
16     current += diff[i];
17     values[i] = current;
18 }
19 return values;
20 }

```

差分数组要求坐标能开数组。如果坐标很大但更新次数不多，就要做坐标压缩或扫描线事件排序。它也不适合频繁在线查询：如果每次更新后立刻问某个点或区间，普通差分每次都还原会很慢。这时树状数组可以看成“动态前缀和”，线段树可以支持更复杂的区间操作。

双指针并不只是一左一右。它至少有三类模型。第一类是两端收缩，如盛水容器和回文判断；第二类是同向快慢，如删除数组重复项、移动零、链表快慢指针；第三类是排序后的配对搜索，如两数之和 II、三数之和、四数之和。每类指针移动依据不同。两端收缩靠候选淘汰，同向快慢靠覆盖写入或距离不变量，配对搜索靠有序数组的和单调。

Listing 4.10 LeetCode 167 两数之和 II：有序数组配对搜索

```

1 std::vector<int> two_sum_sorted(const std::vector<int>& numbers, int target)
2 {
3     int left = 0;
4     int right = static_cast<int>(numbers.size()) - 1;
5     while (left < right) {
6         const int sum = numbers[left] + numbers[right];
7         if (sum == target) {
8             return {left + 1, right + 1};
9         }
10        if (sum < target) {
11            ++left;
12        } else {
13            --right;
14        }
15    }
16    return {};
17 }

```

这题成立的原因是数组有序。若当前和太小，固定右端时，左端再往左只会更小，所以左端必须右移；若当前和太大，固定左端时，右端再往右只会更大，所以右端必须左移。这个推理不能套到无序数组上。无序数组的两数之和通常用哈希表，因为没有单调性可供淘汰。

四数之和是三数之和的自然扩展。暴力四层循环是 $O(n^4)$ 。排序后固定前两个数，后两个数用双指针，复杂度降到 $O(n^3)$ 。去重必须发生在每一层：第一个数重复跳过，第二个数重复跳过，找到答案后左右指针跳过重复。还要注意整数溢出，四个 `int` 相加可能超过 `int` 范围，应该用 `long long` 保存和。

Listing 4.11 LeetCode 18 四数之和：排序、分层去重和长整型求和

```

1 std::vector<std::vector<int>> four_sum(std::vector<int> nums, int target)
2 {

```

```

3     std::sort(nums.begin(), nums.end());
4     std::vector<std::vector<int>> answer;
5     const int n = static_cast<int>(nums.size());
6
7     for (int first = 0; first < n; ++first) {
8         if (first > 0 && nums[first] == nums[first - 1]) {
9             continue;
10        }
11        for (int second = first + 1; second < n; ++second) {
12            if (second > first + 1 && nums[second] == nums[second - 1]) {
13                continue;
14            }
15
16            int left = second + 1;
17            int right = n - 1;
18            while (left < right) {
19                const long long sum =
20                    static_cast<long long>(nums[first]) + nums[second] +
21                    nums[left] + nums[right];
22                if (sum == target) {
23                    answer.push_back(
24                        {nums[first], nums[second], nums[left], nums[right]});
25                    const int left_value = nums[left];
26                    const int right_value = nums[right];
27                    while (left < right && nums[left] == left_value) {
28                        ++left;
29                    }
30                    while (left < right && nums[right] == right_value) {
31                        --right;
32                    }
33                } else if (sum < target) {
34                    ++left;
35                } else {
36                    --right;
37                }
38            }
39        }
40    }
41
42    return answer;
43 }

```

四数之和还可以加剪枝，例如当前最小可能和已经大于目标就 break，当前最大可能和仍小于目标就 continue。但剪枝要谨慎处理溢出和重复值，不要为了少几步让代码变得难以验证。面试中先写稳定版本，再说明可选剪枝，通常更好。

滑动窗口要分固定长度和可变长度。固定长度窗口的长度由题目直接给出，例如找到所有异位词、长度为 K 的最大平均值、滑动窗口最大值。可变长度窗口的长度由约束决定，例如无重复字符、最小覆盖子串、乘积小于 K 的子数组。固定窗口通常每步右进左出；可变窗口通常右端扩展到满足或破坏条件，再移动左端恢复条件或压缩答案。

乘积小于 K 的子数组是可变窗口的另一个典型题。数组元素为正，窗口乘积随着右端扩展会变大，随着左端收缩会变小，因此具有单调性。对每个右端，当窗口乘积小于 K 时，以右端结尾的合法子数组数量是 $\text{right} - \text{left} + 1$ ，因为从 left 到 right 的任意起点都能得到合法乘积。若 $k \leq 1$ ，正整数乘积不可能小于 K，直接返回 0。

Listing 4.12 LeetCode 713 乘积小于 K 的子数组：窗口计数

```

1 int num_subarray_product_less_than_k(const std::vector<int>& nums, int k)
2 {
3     if (k <= 1) {
4         return 0;
5     }
6
7     int left = 0;
8     int product = 1;
9     int answer = 0;
10    for (int right = 0; right < static_cast<int>(nums.size()); ++right) {
11        product *= nums[right];
12        while (product >= k) {
13            product /= nums[left];
14            ++left;
15        }
16        answer += right - left + 1;
17    }
18    return answer;
19 }

```

这个题经常被错写成每次只更新最长长度，但题目问的是子数组数量。窗口恢复合法后，所有以 `right` 结尾、起点在 `[left, right]` 的子数组都合法，因此一次加上窗口长度。计数问题要特别注意“每个答案由哪个时刻统计”。这里每个合法子数组按右端点唯一统计一次，所以不会重复。

有些窗口题不能用普通滑动窗口，但可以用前缀和加单调队列。例如最短子数组和至少为 `K`，数组允许负数。由于负数存在，窗口和不再随右端扩展单调增加。我们改用前缀和：需要找 `prefix[right] - prefix[left] >= K` 且 `right - left` 最小。扫描右端时，用单调队列维护可能的左端前缀下标，并让前缀和递增。若当前前缀减队首前缀已经达到 `K`，就更新答案并弹出队首，因为更靠后的右端只会让这个左端形成更长区间。

Listing 4.13 LeetCode 862 和至少为 K 的最短子数组：前缀和加单调队列

```

1 int shortest_subarray_at_least_k(const std::vector<int>& nums, int k)
2 {
3     const int n = static_cast<int>(nums.size());
4     std::vector<long long> prefix(n + 1, 0);
5     for (int i = 0; i < n; ++i) {
6         prefix[i + 1] = prefix[i] + nums[i];
7     }
8
9     std::deque<int> deque;
10    int answer = n + 1;
11    for (int right = 0; right <= n; ++right) {
12        while (!deque.empty() &&
13            prefix[right] - prefix[deque.front()] >= k) {
14            answer = std::min(answer, right - deque.front());
15            deque.pop_front();
16        }
17        while (!deque.empty() && prefix[deque.back()] >= prefix[right]) {
18            deque.pop_back();
19        }
20        deque.push_back(right);
21    }

```

```

22
23     return answer == n + 1 ? -1 : answer;
24 }

```

为什么队尾前缀和更大时可以删除? 若有两个候选左端 $i < j$ 且 $\text{prefix}[i] \geq \text{prefix}[j]$, 那么对于任何未来右端 r , $\text{prefix}[r] - \text{prefix}[j]$ 不小于 $\text{prefix}[r] - \text{prefix}[i]$, 并且 j 更靠右, 区间更短。因此 i 被 j 完全支配, 可以删除。这是单调队列的本质: 删除永远不可能成为最优的候选。

数组题还要重视“原地”要求。移动零、删除重复、颜色分类、轮转数组都要求在原数组上改。原地算法通常用覆盖写入或交换。覆盖写入适合保留相对顺序, 例如删除重复和移动零; 交换适合不要求稳定顺序或多区域划分, 例如颜色分类。不要为了原地而牺牲可读性: 如果题目没有要求原地, 新建结果数组往往更清楚; 如果题目明确要求 $O(1)$ 额外空间, 就必须说明如何避免额外数组。

Listing 4.14 LeetCode 189 轮转数组: 三次反转

```

1 void rotate_array(std::vector<int>& nums, int k)
2 {
3     if (nums.empty()) {
4         return;
5     }
6
7     k %= static_cast<int>(nums.size());
8     std::reverse(nums.begin(), nums.end());
9     std::reverse(nums.begin(), nums.begin() + k);
10    std::reverse(nums.begin() + k, nums.end());
11 }

```

三次反转的原理是把末尾 K 个元素整体搬到前面, 并保持它们内部顺序。整体反转后, 原末尾 K 个元素到了前面但内部顺序反了, 原前面部分也顺序反了; 分别反转两段后, 两个部分内部顺序恢复。若 k 大于数组长度, 必须先取模。环状替换也能做到原地, 但需要处理循环个数和最大公约数, 面试里三次反转更稳。

数组题的另一个深层主题是“输出规模”。子集、三数之和、区间交集这类题, 算法即使很高效, 输出本身也可能很大。复杂度分析时要区分计算成本和输出成本。三数之和不计输出是 $O(n^2)$, 但答案数量最坏也可能达到 $O(n^2)$ 。如果题目要求输出所有结果, 就不能期望低于输出规模的时间。面试中说出这一点, 能避免不现实的优化承诺。

窗口题的调试方法是打印四个量: `left`、`right`、窗口状态、答案。无重复字符打印字符最后出现位置; 最小覆盖打印 `formed` 和关键字符计数; 乘积小于 K 打印 `product`; 单调队列打印队列中的下标和值。很多窗口错误都是左边界多移或少移一步, 日志能直接暴露不变量何时被破坏。

常见误区

滑动窗口有三个常见误用。第一, 数组含负数却用窗口维护单调性。第二, 题目要求非连续子序列, 却套连续窗口。第三, 窗口状态不能常数时间增删, 却假设移动边界很便宜。使用窗口前必须确认: 答案是连续区间, 边界移动后状态能增量维护, 且移动策略不会漏掉可能答案。

数组窗口实验: 第一, 把长度最小子数组中的正数条件去掉, 构造包含负数的反例, 观察窗口为什么漏解。第二, 比较最小覆盖子串的暴力枚举、窗口版本在长字符串上的时间。第三, 用随机数组验证四数之和的去重逻辑, 把结果排序后检查是否有重复四元组。第四, 打印最短子数组和至少 K 的单调队列, 观察哪些前缀下标被支配删除。

array-window 深度题卡

数组与原地维护

下面只在这里讲一次共性：先写暴力基线，定位最贵的重复工作，再选择能维护不变量的数据结构。每道题目只分析本题自己的建模、瓶颈、边界和变体，复习时不要把家族共性当成答案。

图示：数组与原地维护推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	慢点在于重复搬移和重复求同一段信息。只要一个位置的状态已经由前缀、后缀、慢指针或边界指针确定，再回头扫描就是浪费。
结构选择	优化要把数组切成若干语义明确的区间：已经处理好的前缀、未知区、已经确定的后缀，或者左侧贡献和右侧贡献。双指针、前后缀数组、原地交换和稳定写入，本质都是让每个元素只进入少数几次状态转移。
不变量	不变量必须能落到下标上：慢指针左侧保存什么，右指针右侧保存什么，当前下标之前是否已经满足题目要求。只要这个叙述含糊，代码中的加一减一就会变成猜。
代码落地	C++ 实现优先使用 <code>std::vector<int></code> 和明确的整型下标。涉及乘积、面积、总和时要主动判断是否需要 <code>long long</code> 。如果题目要求原地修改，要说明返回值表示新长度还是数组本身，避免把输出语义和容器容量混在一起。

LeetCode 5 最长回文子串

题目模型。回文围绕中心对称，中心可以是字符也可以是两个字符之间。

推导重点。中心扩展避免枚举子串后再逐个检查；每次扩展只比较新增左右边界。

边界。偶数长度回文、单字符、全相同字符、多个同长答案。

变体追问。若要求回文子序列，应转成区间 DP，不能用中心扩展。

LeetCode 26 删除有序数组重复项

题目模型。有序数组让重复元素相邻，慢指针表示下一个唯一值写入位置。

推导重点。快指针扫描，只有发现新值才写入慢指针并推进。

边界。空数组、全重复、无重复、返回长度不等于物理容量。

变体追问。若允许每个元素最多保留两次，只需改变写入条件为和前两个比较。

LeetCode 27 移除元素

题目模型。原地过滤，慢指针左侧始终是不等于目标值的稳定前缀。

推导重点。保持顺序用快慢指针；不保持顺序可用左右交换减少写入。

边界。所有元素都被移除、没有元素被移除、目标值在尾部密集。

变体追问。工程里要先确认是否要求稳定顺序，不同要求对应不同写法。

LeetCode 31 下一个排列

题目模型。字典序结构由最长非递增后缀决定，枢轴是后缀前一个位置。

推导重点。从右找第一个上升点，再从右找刚好更大的元素交换，最后反转后缀。

边界。完全降序、重复数字、只有一个元素、交换后后缀必须最小化。

变体追问。若要求上一个排列，比较方向和后缀处理对称变化。

LeetCode 54 螺旋矩阵

题目模型。四个边界收缩模拟一圈一圈访问。

推导重点。每走完一条边就收缩对应边界，并在走反方向前检查是否仍合法。

边界。单行、单列、空矩阵、最后一圈只剩一个元素。

变体追问。若改成生成矩阵，边界逻辑相同，只是读变成写。

LeetCode 75 颜色分类

题目模型。三指针维护零区、一号区、未知区和二区。

推导重点。遇到二区时只移动右边界，不移动扫描指针，因为换回来的元素未知。

边界。全零、全二、已经有序、逆序、交换后漏检查。

变体追问。这是荷兰国旗问题，能推广到三类分区的原地维护。

LeetCode 88 合并两个有序数组

题目模型。第一个数组尾部有空位，适合从后往前写入最终最大值。

推导重点。逆向填充避免覆盖尚未读取的 `nums1` 元素。

边界。第二个数组为空、第一个有效元素为空、重复值、尾部剩余。

变体追问。若没有额外空位，只能新开数组或使用更复杂原地归并。

LeetCode 189 轮转数组

题目模型。右移 k 位等价于把数组切成两段后换序。

推导重点。三次反转能原地完成：整体、前 k 、后 n 减 k 。

边界。 k 为零、 k 大于 n 、空数组、单元素。

变体追问。环状替换也可做，但要处理最大公约数个循环。

LeetCode 238 除自身以外数组的乘积

题目模型。不能用除法时，答案由左侧乘积和右侧乘积组成。

推导重点。先写入左侧前缀积，再从右向左乘右侧后缀积。

边界。一个零、多个零、负数、乘积溢出。

变体追问。这个题展示输出数组可作为状态存储的一部分。

LeetCode 283 移动零

题目模型。稳定保留非零元素顺序，把零集中到尾部。

推导重点。慢指针写入下一个非零位置，最后补零或用交换。

边界。全零、无零、零在开头结尾、稳定顺序要求。

变体追问。若目标是移动指定值，模型与移除元素相同。

LeetCode 581 最短无序连续子数组

题目模型。要找破坏全局有序性的最左和最右位置。

推导重点。从左维护历史最大值确定右边界，从右维护历史最小值确定左边界。

边界。已经有序、逆序、重复值、局部乱序。

变体追问。排序比较法更直观但复杂度更高。

LeetCode 647 回文子串

题目模型。每个回文都有唯一中心或中心间隙。

推导重点。对每个中心向两边扩展，扩展成功一次就计数一次。

边界。空串、全相同字符、偶数回文、奇数回文。

变体追问。Manacher 算法可线性解决，但中心扩展更适合面试基础。

滑动窗口与双指针

下面只在这里讲一次共性：先写暴力基线，定位最贵的重复工作，再选择能维护不变量的数据结构。每道题卡只分析本题自己的建模、瓶颈、边界和变体，复习时不要把家族共性当成答案。

图示：滑动窗口与双指针推导链

暴力基线 瓶颈定位	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。 真正的瓶颈不是两层循环本身，而是窗口状态没有被增量维护。右端移动带来的变化只有一个新元素，左端移动移走的也只有一个旧元素，若每次重新统计，就丢掉了题目给的连续性。
结构选择	滑动窗口成立必须有单调性或固定长度边界。右端扩展让某个条件单调变好或变坏，左端收缩可以恢复合法性或缩短答案。若数组含负数、状态会来回震荡，就不能硬套窗口。
不变量	窗口不变量要写成当前窗口满足什么条件，或者当前窗口是第一个可能满足条件的最短前缀。每次移动指针之前先更新状态，移动之后状态要和区间边界一致。
代码落地	C++ 中建议把窗口写成左闭右开或左闭右闭中的一种，并在全题坚持。字符计数用 <code>std::array<int, 128></code> 或 26 长度数组时要说明字符集假设；泛化字符集再换哈希表。

LeetCode 3 无重复字符的最长子串

题目模型。窗口保存当前无重复子串，右端每次加入一个字符。

推导重点。用上次出现位置直接跳过无效左端，左边界只能向右不能回退。

边界。空串、全相同字符、重复字符出现在窗口左侧之外、扩展字符集。

变体追问。若要求恰好包含 k 种字符，窗口条件从无重复变成种类计数。

LeetCode 11 盛最多水的容器

题目模型。左右指针代表当前选取的两条边，面积由短板和宽度共同决定。

推导重点。每次移动短板，因为移动长板只会缩小宽度且不会提高短板。

边界。两端高度相等、数组长度为二、面积乘法可能溢出。

变体追问。接雨水计算每个位置贡献，不能把本题双指针证明直接搬过去。

LeetCode 15 三数之和

题目模型。排序后固定第一个数，剩余两数转成有序双指针。

推导重点。和太小移动左指针，和太大移动右指针；固定值和左右值都要跳过重复。

边界。全零、重复元素很多、没有答案、结果顺序不要求但组合不能重复。

变体追问。四数之和再外层固定一个数，并用 `long long` 防止目标和溢出。

LeetCode 16 最接近的三数之和

题目模型。排序后固定一数，双指针寻找最接近目标的两数和。

推导重点。每次根据当前和与目标的大小移动指针，同时更新最小绝对差。

边界。差值相等、负数、目标很大、三数和溢出。

变体追问。若要求返回所有最接近组合，需要记录同差值并去重。

LeetCode 18 四数之和

题目模型。两层固定加一层双指针，关键是剪枝和去重。

推导重点。排序提供单调性，外层可用最小可能和和最大可能和提前跳过。

边界。重复元素、目标超出 `int`、答案很多导致输出空间主导复杂度。

变体追问。 k 数之和可以递归降维，但工程实现要控制重复和边界。

LeetCode 76 最小覆盖子串

题目模型。窗口内字符计数必须覆盖目标计数，满足后尽量收缩左端。

推导重点。用 `formed` 记录满足需求的字符种类，避免每次全量比较。

边界。目标有重复字符、源串缺字符、最小窗口在开头或结尾、大小写。

变体追问。若字符集固定，数组计数更快；若是 Unicode，哈希表更通用。

LeetCode 209 长度最小的子数组

题目模型。正数数组让窗口和随右端增加、随左端减少。

推导重点。右端扩展到满足目标后，不断左移缩短并更新答案。

边界。不存在答案、单元素达到目标、全正是必要条件。

变体追问。若含负数，需要前缀和加单调队列。

LeetCode 438 找到字符串中所有字母异位词

题目模型。固定长度窗口内字符频次等于目标频次即为答案。

推导重点。右进左出维护计数，窗口长度达到目标长度后检查差异。

边界。目标比源串长、重复字符、字符集固定、答案重叠。

变体追问。维护差异计数可以避免每次比较整个数组。

第 5 章

哈希表和前缀和

哈希表解决“快速查历史信息”，前缀和解决“快速计算区间信息”。两者结合后，可以把很多子数组问题从 $O(n^2)$ 降到 $O(n)$ 。

历史状态、等价类和前缀代数

哈希表题的核心不是“把东西放进 map”，而是把暴力中的某个查询改写成等值查询。暴力两数之和在历史里找补数，哈希表把“历史中是否有某个值”变成 key 查询；字母异位词分组把字符串是否互为异位词，改写成规范 key 是否相等；图搜索的 visited 集合把状态是否访问过，改写成状态编码是否存在。只要不能说清 key 的含义，哈希表就没有真正设计出来。

设计哈希表时要同时确定 key 和 value。key 回答“用什么东西归类”；value 回答“归类后要保存什么信息”。两数之和保存值到下标，因为输出要下标；和为 K 的子数组保存前缀和到次数，因为输出要数量；连续数组保存前缀和到最早下标，因为输出要最长长度；最长连续序列只需要集合存在性，因为输出不依赖每个数的位置。很多错误来自 value 选错：该保存次数却只保存一个下标，该保存最早位置却不断覆盖，该保存集合却用了会默认插入的 `operator[]`。

插入和查询顺序是哈希题的时间语义。扫描数组时，哈希表可以表示“当前之前的历史”，也可以表示“当前窗口内部”，还可以表示“全局所有对象”。不同语义对应不同更新顺序。两数之和先查再插入，是为了让当前元素不能和自己配对；前缀和计数先查再更新，是为了让当前前缀只能和历史前缀构成非空区间；滑动窗口频次表则要在右端加入、左端移出之间保持窗口语义。写代码前先给哈希表写一句中文定义，能避免大部分顺序错误。

前缀和是一种代数化的历史状态。令 `prefix[i]` 表示前 *i* 个元素的聚合值，则区间信息可以由两个前缀相减得到。加法之所以适合前缀和，是因为它有逆运算：知道大前缀和小前缀，就能得到中间区间。异或也有类似性质；乘积在没有零且可除时可以，但通常要谨慎；最大值、最小值没有简单逆运算，不能用两个前缀最大值相减得到区间最大值。遇到区间最大最小，要考虑单调队列、稀疏表、线段树或堆，而不是强行前缀和。

前缀和加哈希表的统一公式是：当前前缀状态和某个历史前缀状态之间满足目标关系。和为 K 是 `prefix[j] - prefix[i] == K`；连续子数组和为 K 的倍数，是两个前缀对 K 的余数相同；0 和 1 数量相等，是把 0 记为 -1 后两个前缀和相同；优美子数组，是奇数计数前缀相差 K。每道题看似不同，实际都在问“当前状态需要什么历史状态配对”。只要能写出这条等式，哈希表保存什么就自然了。

哈希表的复杂度也要说完整。标准库 `unordered_map` 平均查询、插入是 $O(1)$ ，但平均依赖哈希分布和负载因子。大量元素插入时可以 `reserve` 降低 rehash 次数；纯查询时不要用 `operator[]`，因为不存在的 key 会被插入默认值；自定义 key 要同时满足相等语义和哈希语义：相等对象必须得到相同哈希值。刷题面试通常不考哈希攻击，但工程表达中必须知道“平均常数”不是“绝对常数”。

哈希和前缀题的四问

第一，暴力方法在查什么历史信息。第二，这个历史信息能否用等值 key 表示。第三，value 应该是次数、下标、最早下标、最新下标、集合存在性还是一组对象。第四，查询发生在更新之前、之后，还是窗口增删之间。四问都回答完，再写 `unordered_map` 才是有根据的实现。

两数之和是哈希表入门，但真正要记住的是模型：当前元素需要查询历史中是否存在某个补数。只要查询对象可以作为 key，哈希表就能把线性查找降为平均常数查找。暴力方法枚举 *i* 和 *j*，判断 `nums[i] + nums[j] == target`，时间复杂度 $O(n^2)$ 。慢点不在加法，而在“为了每个数反复查找另一个数”。哈希表版本从左到右扫描，用 `value_to_index` 保存已经出现过的值及其下标；处理当前值时，只查 `target - nums[i]` 是否在历史中出现过。

Listing 5.1 LeetCode 1 两数之和：保存历史值到下标

```

1 std::vector<int> two_sum_hash(const std::vector<int>& nums, int target)
2 {
3     std::unordered_map<int, int> value_to_index;
4     value_to_index.reserve(nums.size());
5
6     for (int index = 0; index < static_cast<int>(nums.size()); ++index) {
7         const int need = target - nums[index];
8         const auto found = value_to_index.find(need);
9         if (found != value_to_index.end()) {
10             return {found->second, index};
11         }
12         value_to_index.emplace(nums[index], index);
13     }
14
15     return {};
16 }

```

这段代码必须先查再插入，避免同一个元素和自己配对。如果数组里有重复值，`emplace` 会保留第一次出现的下标；对力扣 1 这种保证唯一答案的题足够。如果题目要求所有答案，就不能这样提前返回，而要继续扫描并处理去重。面试时可以这样表达：哈希表保存历史值到下标的映射，查询 `key` 是当前值的补数，平均时间 $O(n)$ ，空间 $O(n)$ 。

前缀和的定义是：`prefix[i]` 表示前 `i` 个元素的和。那么子数组 `[left, right]` 的和是 `prefix[right + 1] - prefix[left]`。如果题目问和为 `k` 的子数组个数，就等价于对每个当前位置查找之前有多少个前缀和等于 `current_prefix - k`。

Listing 5.2 LeetCode 560 和为 K 的子数组

```

1 int subarray_sum(const std::vector<int>& nums, int k)
2 {
3     std::unordered_map<int, int> prefix_count;
4     prefix_count.reserve(nums.size() + 1);
5     prefix_count[0] = 1;
6
7     int prefix = 0;
8     int answer = 0;
9     for (int x : nums) {
10         prefix += x;
11         const int need = prefix - k;
12         if (const auto found = prefix_count.find(need); found != prefix_count.end()) ←
13             ↪ {
14             answer += found->second;
15         }
16         ++prefix_count[prefix];
17     }
18     return answer;
19 }

```

为什么先查再插入？因为当前前缀只能和之前的前缀组成非空子数组。如果先插入，在 `k == 0` 时可能把当前位置和自己配对。为什么保存次数而不是下标？因为题目问个数，一个当前前缀可以和多个历史前缀组成答案。

最长连续序列展示 `unordered_set` 的另一种用法。把所有数放进集合后，只从没有前驱 `x - 1` 的数开始扩展。这样每条连续链只从头部扫描一次，每个数最多被访问常数次数，平均时间 $O(n)$ 。

哈希题常见错误包括：用 `operator[]` 做纯查询导致插入多余 key；忘记处理重复值；依赖 `unordered_map` 遍历顺序；没有考虑 `long long` 前缀和溢出；把滑动窗口误用到含负数数组。

最长连续序列要求在未排序数组中找最长的连续整数链。排序可以做，复杂度 $O(n \log n)$ ；暴力方法从每个数开始向后找 $x + 1$ 、 $x + 2$ ，如果每次都线性查找，会退化到 $O(n^2)$ 。哈希集合优化了“某个数是否存在”的查询，但如果从每个数都向后扩展，连续链中间的数会被重复扫描。真正的关键是只从链头开始扩展：如果 $x - 1$ 存在，说明 x 不是开头，跳过。

Listing 5.3 LeetCode 128 最长连续序列：只从链头扩展

```
1 int longest_consecutive(const std::vector<int>& nums)
2 {
3     std::unordered_set<int> values;
4     values.reserve(nums.size());
5     for (const int x : nums) {
6         values.insert(x);
7     }
8
9     int best = 0;
10    for (const int x : values) {
11        if (values.contains(x - 1)) {
12            continue;
13        }
14
15        int current = x;
16        int length = 1;
17        while (values.contains(current + 1)) {
18            ++current;
19            ++length;
20        }
21        best = std::max(best, length);
22    }
23
24    return best;
25 }
```

这题的摊还分析要讲清楚：每条连续链只会从最小值开始扫描一次，链中每个元素只在这次扫描里被访问，因此平均时间是 $O(n)$ 。不能在遍历 `values` 时删除元素，除非你非常清楚迭代器失效规则；本书这里选择不删除，代码更直接。

四数相加 II 展示“拆半”的哈希思想。题目给四个数组，统计 $a + b + c + d == 0$ 的元组数量。暴力枚举四个下标是 $O(n^4)$ 。如果固定前三个再查第四个，仍然是 $O(n^3)$ 。更好的做法是把四个数组拆成两组：先统计所有 $a + b$ 的和出现次数，再枚举 $c + d$ ，查询 $-(c + d)$ 出现过多少次。

Listing 5.4 LeetCode 454 四数相加 II：两两拆分

```
1 int four_sum_count(const std::vector<int>& nums1,
2                   const std::vector<int>& nums2,
3                   const std::vector<int>& nums3,
4                   const std::vector<int>& nums4)
5 {
6     std::unordered_map<int, int> pair_sum_count;
7     pair_sum_count.reserve(nums1.size() * nums2.size());
8
9     for (const int a : nums1) {
```

```

10     for (const int b : nums2) {
11         ++pair_sum_count[a + b];
12     }
13 }
14
15 int answer = 0;
16 for (const int c : nums3) {
17     for (const int d : nums4) {
18         const auto found = pair_sum_count.find(-(c + d));
19         if (found != pair_sum_count.end()) {
20             answer += found->second;
21         }
22     }
23 }
24
25 return answer;
26 }

```

拆半的本质是把 n^4 的组合空间拆成两个 n^2 的集合,再用哈希表连接它们。空间复杂度是 $O(n^2)$ 。这类题的面试表达要说明为什么保存次数: 同一个两数和可能由多个下标对产生, 每一个都能和后半部分组成不同元组。

哈希表的底层原理也要理解。哈希函数把 key 映射到桶编号; 如果多个 key 映射到同一个桶, 就产生冲突。标准库实现细节不完全相同, 但复杂度承诺通常是平均 $O(1)$, 最坏可能退化。为了让平均复杂度可靠, 容器会在负载因子过高时 rehash, 增加桶数量并重新分布元素。刷题中常用 `reserve` 预留容量, 减少 rehash 次数; 工程中还要考虑自定义 key 的哈希质量、相等比较是否正确、是否可能遭遇恶意输入。

Listing 5.5 自定义 pair 哈希: 把结构化 key 放进 unordered_map

```

1 struct PairHash {
2     std::size_t operator()(const std::pair<int, int>& value) const noexcept
3     {
4         constexpr int HASH_SHIFT = 32;
5         const auto high = static_cast<std::uint64_t>(
6             static_cast<std::uint32_t>(value.first));
7         const auto low = static_cast<std::uint64_t>(
8             static_cast<std::uint32_t>(value.second));
9         return std::hash<std::uint64_t>{}((high << HASH_SHIFT) ^ low);
10    }
11 };
12
13 using PointCount = std::unordered_map<std::pair<int, int>, int, PairHash>;

```

这段代码说明 key 不一定只能是整数或字符串。只要能定义“相等”和“哈希”, 结构化对象也可以作为 key。要注意哈希相等的必要条件: 如果两个 key 相等, 它们的哈希值必须相同; 如果两个 key 哈希值相同, 它们不一定相等, 容器还会用相等比较区分。很多工程 bug 来自只写了哈希函数, 却忘记相等语义, 或者哈希组合方式让大量不同 key 落入相同桶。

字母异位词分组是 key 设计题。排序后的字符串可以作为 key, 简单直接; 字符计数也可以作为 key, 避免每个单词排序。若只含小写字母, 26 维计数就是一个规范表示。难点在于把计数数组变成无歧义 key。如果直接拼接数字, `1,11` 和 `11,1` 可能混淆; 加入分隔符或使用数组作为自定义 key 更安全。

Listing 5.6 LeetCode 49 字母异位词分组: 计数 key 带分隔符

```

1 std::vector<std::vector<std::string>> group_anagrams(
2     const std::vector<std::string>& words)
3 {
4     std::unordered_map<std::string, std::vector<std::string>> groups;
5     groups.reserve(words.size());
6
7     for (const std::string& word : words) {
8         std::array<int, 26> count{};
9         for (const char ch : word) {
10             ++count[ch - 'a'];
11         }
12
13         std::string key;
14         for (const int value : count) {
15             key += '#';
16             key += std::to_string(value);
17         }
18         groups[key].push_back(word);
19     }
20
21     std::vector<std::vector<std::string>> answer;
22     answer.reserve(groups.size());
23     for (auto& [key, group] : groups) {
24         (void)key;
25         answer.push_back(std::move(group));
26     }
27     return answer;
28 }

```

这个题的优化取舍要会说。排序 key 每个单词成本是 $O(L \log L)$ ，计数 key 是 $O(L + |\Sigma|)$ 。当字符集固定且较小，计数更好；当字符集复杂或实现时间紧，排序 key 更稳。面试中并不是所有题都追求理论最优，清晰、正确、符合约束的方案更重要。

连续的子数组和使用同余前缀。若两个前缀和对 k 的余数相同，它们之间的子数组和就是 k 的倍数。题目要求长度至少为 2，所以哈希表保存每个余数最早出现的位置。为什么保存最早位置？因为越早越容易满足长度条件；后面相同余数不应该覆盖它。初始化余数 0 的位置为 -1，表示从数组开头到当前位置的前缀本身就能被 k 整除。

Listing 5.7 LeetCode 523 连续的子数组和：余数最早位置

```

1 bool check_subarray_sum(const std::vector<int>& nums, int k)
2 {
3     std::unordered_map<int, int> first_index;
4     first_index.reserve(nums.size() + 1);
5     first_index[0] = -1;
6
7     int prefix = 0;
8     for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
9         prefix += nums[i];
10        const int remainder = prefix % k;
11        const auto found = first_index.find(remainder);
12        if (found != first_index.end()) {
13            if (i - found->second >= 2) {
14                return true;
15            }
16        }
17    }
18    return false;
19 }

```

```

15     }
16     } else {
17         first_index[remainder] = i;
18     }
19 }
20 return false;
21 }

```

这题的关键不是取模代码，而是“同余差值可整除”的数学关系。若题目允许 $k == 0$ ，取模会出错，需要单独处理连续至少两个 0 或直接用前缀和相等判断；力扣当前约束通常给出非零 k ，但面试中要先确认。前缀和如果可能很大，要用 `long long`；取模后余数范围有限，才适合放进哈希表。

连续数组把 0 和 1 数量相等转成前缀和相等。把 0 看成 -1，1 看成 +1，一个区间内 0 和 1 数量相等，就等价于区间转换后的和为 0。扫描前缀和时，若某个前缀和第二次出现，两次出现之间的区间和为 0。因为要求最长长度，所以保存每个前缀和第一次出现的位置。

Listing 5.8 LeetCode 525 连续数组：0 转成 -1 后找相同前缀和

```

1 int find_max_length_equal_zero_one(const std::vector<int>& nums)
2 {
3     std::unordered_map<int, int> first_index;
4     first_index.reserve(nums.size() * 2);
5     first_index[0] = -1;
6
7     int prefix = 0;
8     int answer = 0;
9     for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
10         prefix += nums[i] == 0 ? -1 : 1;
11         const auto found = first_index.find(prefix);
12         if (found != first_index.end()) {
13             answer = std::max(answer, i - found->second);
14         } else {
15             first_index[prefix] = i;
16         }
17     }
18     return answer;
19 }

```

这个转换技巧可以推广。两个类别数量相等，可以把一个类别记为 +1，另一个记为 -1；多个类别的平衡问题，则可能需要向量差作为 key。比如同时要求 A 和 B、A 和 C 数量差保持一致，可以保存二维差值。哈希表的 key 设计会更复杂，但核心仍然是：把区间约束转成两个前缀状态相等。

快乐数和环检测展示哈希集合的“历史状态去重”。每次把数字替换为各位平方和，如果出现 1 就成功；如果某个中间数字再次出现，就进入循环，不可能再到 1。哈希集合保存见过的数字，快慢指针也能做，因为转换函数把每个数字映射到唯一后继。哈希写法更容易解释，快慢指针空间更低。

Listing 5.9 LeetCode 202 快乐数：哈希集合检测循环

```

1 int digit_square_sum(int value)
2 {
3     int sum = 0;
4     while (value > 0) {
5         const int digit = value % 10;
6         sum += digit * digit;
7         value /= 10;

```

```

8     }
9     return sum;
10 }
11
12 bool is_happy(int n)
13 {
14     std::unordered_set<int> seen;
15     while (n != 1 && !seen.contains(n)) {
16         seen.insert(n);
17         n = digit_square_sum(n);
18     }
19     return n == 1;
20 }

```

这个题让我们看到哈希不只用于数组下标，也用于状态空间去重。图搜索中的 `visited` 集合、字符串 BFS 中的字典删除、回溯中的去重集合，都是同一思想。只要状态能被规范编码成 key，就可以避免重复访问。

哈希和前缀和也有边界。第一，哈希表不能提供有序关系；需要最小大于、最大不超过、排名、区间统计时，要考虑平衡树、树状数组、线段树。第二，前缀和适合可逆的区间聚合，例如加法；最大值、最小值不能用两个前缀值相减得到，通常要用稀疏表、线段树或单调队列。第三，哈希表平均常数不等于稳定常数，key 很大、字符串很长、自定义哈希很差时，实际成本会升高。

深入理解

从源码视角看，`unordered_map` 的一个元素通常不只是 key 和 value，还包含节点指针、哈希桶链接等额外信息。大量小对象会带来分配成本和缓存不友好。频次统计中，如果 key 是小范围整数，用数组计数通常比哈希表更快；如果 key 是稀疏大整数或字符串，哈希表才体现优势。刷题答案可以写 `unordered_map`，但工程判断要先看 key 的值域、数量、生命周期和访问模式。

哈希实验：第一，把两数之和分别用排序双指针和哈希表实现，比较是否需要保留原下标。第二，给字母异位词分组构造长单词，比较排序 key 和计数 key 的耗时。第三，给连续数组打印前缀和第一次出现位置，观察为什么不能覆盖旧下标。第四，用数组计数和 `unordered_map` 计数分别统计小写字母频次，比较常数差异。

哈希题复盘模板

哈希题不要只写“用 map”。复盘时必须写：key 是什么，value 是什么；查询发生在插入前还是插入后；重复 key 怎么处理；是否需要次数、最早位置、最新位置还是集合存在性；平均复杂度和最坏情况分别是什么。

本章力扣主线：1 两数之和、49 字母异位词分组、128 最长连续序列、560 和为 K 的子数组、523 连续子数组和、525 连续数组、454 四数相加 II。复盘时必须说明哈希表保存的是值、下标、次数还是最早位置。

hash-binary-stack 深度题卡

前缀和与哈希表

下面只在这里讲一次共性：先写暴力基线，定位最贵的重复工作，再选择能维护不变量的数据结构。每道题卡只分析本题自己的建模、瓶颈、边界和变体，复习时不要把家族共性当成答案。

图示：前缀和与哈希表推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	瓶颈在于当前右端需要寻找很多历史左端。若这些历史状态能被同一个键表示，就不该逐个扫描，而应把历史状态压进哈希表。
结构选择	前缀和把区间问题改写成两个前缀状态的差；哈希表把寻找匹配前缀改成查表。频次表、最早位置表、最近位置表对应不同目标：计数、最长区间、最短区间不能混用。
不变量	哈希表的不变量通常是当前下标之前的历史前缀信息。先查询还是先更新不是风格问题，而是决定是否允许空区间和是否重复使用当前前缀。
代码落地	实现时用 <code>find</code> 判断是否存在，用 <code>operator[]</code> 只在确认要插入或累加时使用。总和可能溢出时使用 <code>long long</code> ；模运算要处理负数余数归一化。

LeetCode 1 两数之和

题目模型。当前元素只需要寻找唯一补数，历史值到下标的映射就是最小状态。

推导重点。先查再插入，避免同一个下标被用两次；若数组有序才考虑双指针。

边界。重复数字、目标为偶数且补数等于当前值、没有答案时返回空结果。

变体追问。若改成返回所有不重复数对，要先排序或用频次表处理重复。

LeetCode 49 字母异位词分组

题目模型。异位词共享同一个规范表示，可以是排序字符串或字符计数。

推导重点。哈希表按规范键分桶，避免两两比较字符串。

边界。空字符串、重复单词、字符集不止小写字母、计数键必须无歧义。

变体追问。若字符串很长且字符集固定，计数键通常比排序键更稳定。

LeetCode 128 最长连续序列

题目模型。连续段只需要从没有前驱的数开始扩展。

推导重点。哈希集合判断 $x-1$ 是否存在，跳过非起点避免重复扫描。

边界。重复元素、负数、空数组、整数边界。

变体追问。若数据流动态插入，可以用并查集或区间合并维护长度。

LeetCode 202 快乐数

题目模型。反复变换数字会进入一或循环，状态空间最终有限。

推导重点。用集合检测重复状态，或用快慢指针看成函数图。

边界。输入为一、进入循环、位平方和函数、负数不在题意内。

变体追问。快慢指针版本能训练把数值迭代看成链表。

LeetCode 217 存在重复元素

题目模型。判断是否出现过是哈希集合最直接的职责。

推导重点。扫描时若插入前已经存在则立即返回真。

边界。空数组、全唯一、重复在末尾、值域很小。

变体追问。若允许修改输入，排序后看相邻可以降低额外空间。

LeetCode 560 和为 K 的子数组

题目模型。当前前缀和减去目标值，就是需要匹配的历史前缀。

推导重点。哈希表保存前缀和出现次数，先查询再更新。

边界。负数、目标为零、从下标零开始的区间、重复前缀。

变体追问。若要求最长区间，哈希表应保存最早位置而不是次数。

二分查找与答案空间

下面只在这里讲一次共性：先写暴力基线，定位最贵的重复工作，再选择能维护不变量的数据结构。每道题目只分析本题自己的建模、瓶颈、边界和变体，复习时不要把家族共性当成答案。

图示：二分查找与答案空间推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	慢点在于每次只排除一个候选。二分能成立，是因为一次比较可以排除一半下标，或者一次可行性检查可以排除一半答案范围。
结构选择	二分前必须先定义谓词：某个位置是否已经满足边界条件，或者某个答案是否可行。数组二分找的是边界，答案二分找的是最小可行值或最大可行值。
不变量	建议固定左闭右开区间。循环中始终维护答案在 <code>[left, right)</code> 内，左侧一定不可行或小于目标，右侧外部已经被排除。
代码落地	中点写成 <code>left + (right - left) / 2</code> 。判断平方、乘法、总量时避免溢出。用 <code>std::lower_bound</code> 能表达清楚时优先用它，但面试要能手写同等语义。

LeetCode 33 搜索旋转排序数组

题目模型。旋转数组每次二分至少有一半保持有序。

推导重点。判断哪一半有序，再看目标是否落在有序半边范围内。

边界。未旋转、长度一、目标不存在、边界等号、存在重复元素时退化。

变体追问。若重复元素很多，需要在无法判断有序半边时收缩边界。

LeetCode 34 排序数组中查找首尾位置

题目模型。首尾位置可以拆成两个边界：第一个大于等于和第一个大于。

推导重点。不要找到一个目标后向两边线性扩展，否则重复元素很多时退化。

边界。目标不存在、目标在开头或结尾、数组为空、全是目标。

变体追问。这个模型能迁移到计数某值出现次数和插入区间边界。

LeetCode 35 搜索插入位置

题目模型。题目等价于寻找第一个大于等于目标的位置。

推导重点。统一用 lower bound 语义，不在相等时提前返回也能保持边界一致。

边界。目标小于所有元素、目标大于所有元素、重复值、空数组。

变体追问。手写二分时用左闭右开可直接返回 left。

LeetCode 153 寻找旋转排序数组中的最小值

题目模型。最小值是旋转断点，右端点可帮助判断中点在哪一段。

推导重点。若 mid 大于 right，最小值在右侧；否则在左侧含 mid。

边界。未旋转、长度一、旋转一位、无重复条件。

变体追问。有重复时遇到相等只能收缩 right，最坏退化。

LeetCode 154 寻找旋转排序数组中的最小值 II

题目模型。重复元素会让 mid 和 right 相等时无法判断最小值方向。

推导重点。相等时只能安全地丢掉一个 right，因为它不是唯一必要候选。

边界。全相等、重复值跨断点、最小值出现多次。

变体追问。这题说明二分依赖的信息不足时会退化，不是所有旋转题都严格对数。

LeetCode 162 寻找峰值

题目模型。比较 mid 和 mid 加一的斜率能判断某侧一定存在峰值。

推导重点。上升则右侧有峰，下降则左侧含 mid 有峰。

边界。长度一、峰在边界、多个峰、相邻不相等假设。

变体追问。二维峰值需要按行列最大值进行更复杂二分。

LeetCode 240 搜索二维矩阵 II

题目模型。右上角同时具有向左变小、向下变大的排除能力。

推导重点。当前值大于目标就左移，小于目标就下移。

边界。空矩阵、单行、单列、目标在角落、重复值。

变体追问。从左上角无法一次排除一行或一列。

LeetCode 704 二分查找

题目模型。基础题的重点是固定区间语义。

推导重点。左闭右开写法能统一 lower bound 和存在性检查。

边界。空数组、长度一、目标在边界、目标不存在。

变体追问。所有复杂二分都应该能退回到这道题的边界不变量。

LeetCode 875 爱吃香蕉的珂珂

题目模型。速度越大，总耗时越小，答案具有单调可行性。

推导重点。二分速度，检查函数用向上取整累加小时数。

边界。速度下界、堆很大、h 等于堆数、乘法加法溢出。

变体追问。船运包裹和分割数组都是同类最小可行上限。

LeetCode 1011 在 D 天内送达包裹

题目模型。容量越大，需要天数越少，存在最小可行容量。

推导重点。下界是最大包裹，上界是总重量，检查函数按顺序贪心装船。

边界。D 为一、D 等于包裹数、单个包裹超过猜测容量。

变体追问。它和分割数组最大值都是连续分段最大和模型。

栈、单调栈与表达式

下面只在这里讲一次共性：先写暴力基线，定位最贵的重复工作，再选择能维护不变量的数据结构。每道题卡只分析本题自己的建模、瓶颈、边界和变体，复习时不要把家族共性当成答案。

图示：栈、单调栈与表达式推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	瓶颈是未解决元素没有被组织起来。单调栈保存的是仍在等待某个未来元素解决的候选；普通栈保存的是最近打开但尚未关闭的结构。
结构选择	当新元素到来时，它会一次性解决栈顶一串被它支配的旧元素。被弹出的元素以后再也不会参与比较，因此总体线性。
不变量	栈内下标对应的值要保持单调，或者栈顶必须是最近未匹配对象。弹出时要说清楚当前元素充当右边界、下一个栈顶充当左边界，还是当前运算符触发计算。
代码落地	栈里通常存下标而不是值，因为要计算距离、宽度和过期。可以用 <code>std::vector<int></code> 当栈，便于查看顶部和减少适配器限制。

LeetCode 42 接雨水

题目模型。每个位置的水由左侧最高和右侧最高的较小者决定。

推导重点。前后缀、双指针、单调栈都是不同状态维护方式；要能说明各自结算对象。

边界。单调递增、单调递减、平台高度、多个凹槽、数组长度不足三。

变体追问。若柱子变成二维网格，需要最小堆从边界向内扩展。

LeetCode 84 柱状图中最大的矩形

题目模型。每根柱子作为最矮高度时，需要知道左右第一个更矮位置。

推导重点。单调递增栈在遇到更矮柱时结算被弹出柱子的最大宽度。

边界。递增、递减、相等高度、哨兵零、宽度计算。

变体追问。最大矩形可以作为二维矩阵最大矩形的行压缩子问题。

LeetCode 150 逆波兰表达式求值

题目模型。后缀表达式把优先级和括号都编码进 token 顺序。

推导重点。遇到数字入栈，遇到运算符弹出右操作数和左操作数计算。

边界。负数 token、多位数、除法向零截断、减法顺序。

变体追问。中缀表达式求值需要另一个运算符栈处理优先级。

LeetCode 155 最小栈

题目模型。普通栈只能看到栈顶，不能常数时间知道历史最小值。

推导重点。辅助栈同步保存每个深度对应的最小值。

边界。重复最小值、弹出最小值、空栈操作约定。

变体追问。也可保存差值压缩空间，但可读性和溢出处理更复杂。

LeetCode 232 用栈实现队列

题目模型。两个栈分别负责输入顺序和输出顺序。

推导重点。只有输出栈为空时，才把输入栈整体倒过去，保证均摊常数。

边界。连续 peek、空队列、交替 push pop。

变体追问。用队列实现栈则要通过轮转保持最近元素在队首。

LeetCode 239 滑动窗口最大值

题目模型。每个窗口最大值来自仍未过期且没有被新元素支配的候选。

推导重点。单调队列保存下标，队尾小于等于新值的候选被弹出。

边界。k 为一、k 等于数组长度、重复最大值、队首过期。

变体追问。受限子序列和使用同样队列维护最近 k 个 DP 最大值。

LeetCode 394 字符串解码

题目模型。嵌套结构需要保存上一层字符串和重复次数。

推导重点。遇到左括号入栈当前状态，右括号弹出并拼接展开。

边界。多位数字、嵌套、空片段、数字后必须跟括号。

变体追问。递归下降解析更通用，但迭代栈更符合工程栈控制。

LeetCode 496 下一个更大元素 I

题目模型。第二个数组提供每个值右侧第一个更大值的全局关系。

推导重点。单调栈预处理值到下一个更大值映射，再回答第一个数组查询。

边界。不存在更大值、递减数组、值唯一假设。

变体追问。若值不唯一，应按下标而不是值建映射。

LeetCode 503 下一个更大元素 II

题目模型。环形数组可以通过遍历两倍下标模拟绕回。

推导重点。第一遍入栈，第二遍只帮助未解决下标找到答案。

边界。全递减、全相等、长度一、取模下标。

变体追问。不要在第二遍重复入栈，否则可能死循环或重复计算。

LeetCode 739 每日温度

题目模型。每一天等待右侧第一个更高温度。

推导重点。单调递减栈保存尚未找到答案的下标。

边界。没有更高温、相等温度、递增、递减。

变体追问。下一个更大元素与它同型，只是输出值或距离不同。

位运算与状态压缩

下面只在这里讲一次共性：先写暴力基线，定位最贵的重复工作，再选择能维护不变量的数据结构。每道题卡只分析本题自己的建模、瓶颈、边界和变体，复习时不要把家族共性当成答案。

图示：位运算与状态压缩推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	慢点在于状态复制和集合比较。位操作让包含、加入、删除、枚举子集变成常数或接近常数的机器指令。
结构选择	异或解决成对抵消，按位计数处理出现三次，掩码表示访问集合或可用集合，低位提取用于枚举候选。
不变量	掩码的每一位必须有固定含义。状态去重时，位置相同但掩码不同不能合并，因为未来能力不同。
代码落地	使用无符号整数能减少移位歧义。位数超过 31 时用 <code>long long</code> 或 <code>std::bitset</code> 。表达式加括号，避免位运算优先级误读。

LeetCode 136 只出现一次的数字

题目模型。成对元素可以用异或抵消，剩下单个元素。

推导重点。异或满足交换结合，扫描顺序不影响答案。

边界。零、负数、唯一值在开头或结尾、所有其他值恰好两次。

变体追问。若其他值出现三次，要改为按位计数取模。

LeetCode 287 寻找重复数

题目模型。数组值域使下标到值形成函数图，重复数是环入口。

推导重点。快慢指针不修改数组且常数空间；值域二分用抽屉原理。

边界。重复数出现多次、不能排序、不能改数组、长度为 n 加一。

变体追问。快慢指针要求值都能作为合法下标。

第 6 章

二分查找和答案空间

二分查找的本质不是在数组里找数，而是在单调结构中找边界。只要能定义判断函数 `check(x)`，并且它在答案空间上单调，就可以二分。

二分的对象不是中点，而是边界

很多二分错误来自把注意力放在 `mid` 怎么算，而不是放在“答案边界在哪里”。二分要解决的问题通常可以改写成：在一段有序或单调的空间里，找到第一个满足条件的位置，或最后一个满足条件的位置。普通查找目标值，只是边界问题的特例；搜索插入位置是第一个大于等于目标；求平方根是最后一个平方不超过目标的整数；爱吃香蕉是第一个能按时完成的速度。只要边界定义清楚，循环条件和返回值就不会靠猜。

二分空间可以是下标，也可以是答案值，还可以是浮点区间。下标二分的单调性来自数组有序；答案二分的单调性来自可行性随答案放宽而不变坏；浮点二分的单调性来自连续函数或可比较的大小关系。三者外形相似，但检查函数不同。下标二分直接读 `nums[mid]`；答案二分要运行一个 `check(mid)`；浮点二分不能依赖精确相等，只能用迭代次数或误差阈值收缩区间。

区间语义是二分代码的地基。左闭右闭 `[left, right]` 表示两端都可能是答案，循环常写 `left <= right`，排除中点时用 `mid + 1` 或 `mid - 1`。左闭右开 `[left, right)` 表示 `right` 是不包含的上界，循环常写 `left < right`，找左边界时若 `mid` 可能是答案就保留它，令 `right = mid`。两种语义都能写对，但最怕把“`mid` 可能是答案”和“`mid` 已经排除”混在一起。每一行赋值都应该能回答：新的区间是否仍然包含目标边界。

谓词单调性必须证明。对于“第一个 true”模型，答案空间应该呈现 false、false、true、true 的形状；对于“最后一个 true”模型，形状则是 true、true、false、false。分割数组最大值中，允许的最大段数和越大，越容易分成不超过 `k` 段，所以可行性从 false 变 true；制作花束中，天数越大，可开放的花越多，可行性也从 false 变 true。如果检查函数写错，即使二分外壳完全标准，也只是在错误谓词上找边界。

答案范围同样是算法的一部分。下界必须不大于真实答案，上界必须不小于真实答案。爱吃香蕉速度下界是 1，上界是最大堆；船运包裹容量下界是最大单件重量，上界是总重量；分割数组最大值下界也是最大元素，上界是总和。范围太窄会漏答案，范围太宽通常仍正确但会多几次检查。若检查函数本身成本很高，边界设置也会影响实际性能。

二分和标准库关系密切。`std::lower_bound` 找第一个不小于目标的位置，`std::upper_bound` 找第一个大于目标的位置。自己手写二分，应尽量让语义贴近这两个函数，因为它们经过长期检验，也能帮助你解释边界。范围查找、插入位置、计数某个值出现次数，都可以由 lower bound 和 upper bound 组合出来。若题目只需要标准库语义，面试中可以先说明标准库函数，再手写以展示边界能力。

二分题的证明清单

先说答案空间是什么，再说要找第一个可行还是最后一个可行；然后写出谓词，并证明谓词单调；接着给出左右边界为什么覆盖真实答案；最后说明循环中每次更新排除了哪些不可能区域，以及返回值为什么就是边界。没有这五步，代码看起来像二分，也可能只是碰巧通过样例。

基础二分在有序数组中查找目标值。写法看似简单，错误通常来自边界没有排除 `mid`、返回值不清楚或 `left + right` 溢出。

Listing 6.1 LeetCode 704 二分查找：基础写法

```
1 int binary_search_index(const std::vector<int>& nums, int target)
2 {
```

```

3   int left = 0;
4   int right = static_cast<int>(nums.size()) - 1;
5   while (left <= right) {
6       const int mid = left + (right - left) / 2;
7       if (nums[mid] == target) {
8           return mid;
9       }
10      if (nums[mid] < target) {
11          left = mid + 1;
12      } else {
13          right = mid - 1;
14      }
15  }
16  return -1;
17 }

```

找左边界时，标准库 `lower_bound` 是最好的参照：它找第一个不小于目标的位置。很多题不是找一个等于目标的位置，而是找第一个满足条件的位置。

搜索插入位置就是左边界模型。题目要求如果目标存在，返回下标；如果不存在，返回应该插入的位置。这个位置其实就是第一个 `nums[i] >= target` 的下标。如果所有数都小于目标，答案就是 `nums.size()`。暴力方法从左到右扫描，时间 $O(n)$ ；有序数组给了单调性，所以可以二分。

Listing 6.2 LeetCode 35 搜索插入位置: `lower_bound` 写法

```

1  int search_insert_position(const std::vector<int>& nums, int target)
2  {
3      int left = 0;
4      int right = static_cast<int>(nums.size());
5
6      while (left < right) {
7          const int mid = left + (right - left) / 2;
8          if (nums[mid] < target) {
9              left = mid + 1;
10         } else {
11             right = mid;
12         }
13     }
14
15     return left;
16 }

```

这里使用左闭右开区间 `[left, right)`。循环不变量是答案一定在这个区间里；当 `nums[mid] < target`，`mid` 以及左侧都不可能是第一个不小于目标的位置，所以令 `left = mid + 1`；否则 `mid` 可能就是答案，不能排除，只能令 `right = mid`。这个“保留 `mid`”是左边界二分的核心。

答案二分把“位置”换成“答案值”。爱吃香蕉的珂珂中，答案是速度。速度越大越容易在规定时间内吃完，所以 `check(speed)` 从 `false` 变成 `true`。我们要找第一个 `true`。分割数组最大值、运输包裹能力、制作花束最少天数也都是这个模型。

Listing 6.3 LeetCode 875 爱吃香蕉的珂珂：答案二分

```

1  int min_eating_speed(const std::vector<int>& piles, int hours)
2  {
3      int left = 1;
4      int right = *std::max_element(piles.begin(), piles.end());

```

```

5
6     auto can_finish = [&](int speed) {
7         long long used_hours = 0;
8         for (const int pile : piles) {
9             used_hours += (pile + speed - 1LL) / speed;
10        }
11        return used_hours <= hours;
12    };
13
14    while (left < right) {
15        const int mid = left + (right - left) / 2;
16        if (can_finish(mid)) {
17            right = mid;
18        } else {
19            left = mid + 1;
20        }
21    }
22
23    return left;
24 }

```

暴力方法可以从速度 1 一直试到最大堆大小，每次计算总小时数，复杂度 $O(mn)$ ，其中 m 是最大速度范围。二分优化来自单调性：速度越大，吃完所需小时数越少；如果某个速度可以完成，那么更大的速度也一定可以完成。我们要找最小可行速度，所以是“第一个 true”。计算向上取整时用 $(pile + speed - 1LL) / speed$ ，其中 $1LL$ 避免整数溢出。

搜索旋转排序数组是另一类二分：单调性不是全局存在，而是每一步至少有一半区间有序。暴力扫描是 $O(n)$ 。二分时先判断 $[left, mid]$ 是否有序；如果有序，再判断目标是否落在这一半里，落在里面就收缩到左半，否则去右半。若左半无序，右半一定有序，用同样逻辑判断。这个题不要死背条件，应该画出旋转数组的两段有序结构。

Listing 6.4 LeetCode 33 搜索旋转排序数组

```

1 int search_rotated_array(const std::vector<int>& nums, int target)
2 {
3     int left = 0;
4     int right = static_cast<int>(nums.size()) - 1;
5
6     while (left <= right) {
7         const int mid = left + (right - left) / 2;
8         if (nums[mid] == target) {
9             return mid;
10        }
11
12        if (nums[left] <= nums[mid]) {
13            if (nums[left] <= target && target < nums[mid]) {
14                right = mid - 1;
15            } else {
16                left = mid + 1;
17            }
18        } else {
19            if (nums[mid] < target && target <= nums[right]) {
20                left = mid + 1;
21            } else {

```

```

22         right = mid - 1;
23     }
24 }
25 }
26
27 return -1;
28 }

```

这题默认数组没有重复值。如果允许重复，例如 `[1, 0, 1, 1, 1]`，`nums[left] == nums[mid]` 时无法判断哪边有序，可能需要移动边界退化处理，最坏复杂度会变成 $O(n)$ 。面试时主动说明这个前提，会显得你不是机械套模板。

二分最重要的习惯是固定一种区间语义。左闭右闭 `[left, right]` 和左闭右开 `[left, right)` 都可以，但不能混用。本书在找边界时优先使用左闭右开：`left` 是可能答案，`right` 是不包含的上界，循环条件是 `left < right`。当 `check(mid)` 为真且 `mid` 可能是答案时，令 `right = mid`；当 `mid` 不可能是答案时，令 `left = mid + 1`。这个模板天然适合“找第一个满足条件的位置”。

查找目标的第一个和最后一个位置，可以拆成两个边界问题。第一个位置是 `lower_bound(target)`；最后一个位置可以用 `lower_bound(target + 1) - 1`，但 `target + 1` 可能溢出，也不适用于非整数类型。更通用的写法是写两个函数：第一个大于等于 `target` 的位置，以及第一个大于 `target` 的位置。若左边界越界或值不等于 `target`，说明目标不存在。

Listing 6.5 LeetCode 34 查找目标范围：两个边界

```

1 int lower_bound_index(const std::vector<int>& nums, int target)
2 {
3     int left = 0;
4     int right = static_cast<int>(nums.size());
5     while (left < right) {
6         const int mid = left + (right - left) / 2;
7         if (nums[mid] < target) {
8             left = mid + 1;
9         } else {
10             right = mid;
11         }
12     }
13     return left;
14 }
15
16 int upper_bound_index(const std::vector<int>& nums, int target)
17 {
18     int left = 0;
19     int right = static_cast<int>(nums.size());
20     while (left < right) {
21         const int mid = left + (right - left) / 2;
22         if (nums[mid] <= target) {
23             left = mid + 1;
24         } else {
25             right = mid;
26         }
27     }
28     return left;
29 }
30

```

```

31 std::vector<int> search_range(const std::vector<int>& nums, int target)
32 {
33     const int first = lower_bound_index(nums, target);
34     if (first == static_cast<int>(nums.size()) || nums[first] != target) {
35         return {-1, -1};
36     }
37     const int last = upper_bound_index(nums, target) - 1;
38     return {first, last};
39 }

```

这段代码比“找到一个 target 后向左右扩展”更稳定，因为后者在全数组都是 target 时会退化到 $O(n)$ 。边界二分始终是 $O(\log n)$ 。注意 `upper_bound_index` 的条件是 `nums[mid] <= target`，因为小于等于目标的位置都不可能是第一个大于目标的位置。

寻找峰值是“根据局部趋势排除一半”的二分。题目给出相邻元素不相等，峰值是大于左右邻居的位置，数组边界外可以视为负无穷。若 `nums[mid] < nums[mid + 1]`，说明从 `mid` 到 `mid + 1` 是上升趋势，右侧一定存在峰值；否则左侧包含 `mid` 一定存在峰值。这个证明来自连续上升最终会在边界或下降处形成峰。

Listing 6.6 LeetCode 162 寻找峰值：沿上坡方向保留峰值

```

1 int find_peak_element(const std::vector<int>& nums)
2 {
3     int left = 0;
4     int right = static_cast<int>(nums.size()) - 1;
5     while (left < right) {
6         const int mid = left + (right - left) / 2;
7         if (nums[mid] < nums[mid + 1]) {
8             left = mid + 1;
9         } else {
10            right = mid;
11        }
12    }
13    return left;
14 }

```

这里 `mid + 1` 不会越界，因为循环条件 `left < right` 保证 `mid < right`。这个细节说明二分边界和访问表达式要配套设计。若你用左闭右闭写法，也必须确保访问 `mid + 1` 安全。峰值题不要求找全局最大，只要任意峰值；如果题目要求最左峰值或最大峰值，模型就要改。

答案二分的检查函数往往比二分外壳更重要。分割数组最大值要求把数组分成 `k` 个非空连续子数组，使最大子数组和最小。暴力枚举切分点是组合爆炸，DP 可以做但复杂度较高。二分答案把最大段和上限设为 `limit`，检查能否在不超 `k` 段内完成。贪心检查从左到右尽量往当前段放，放不下才开新段。这个贪心正确，因为提前开新段不会减少段数，只会让当前段剩余容量浪费。

Listing 6.7 LeetCode 410 分割数组最大值：二分最大段和

```

1 int split_array_largest_sum(const std::vector<int>& nums, int k)
2 {
3     int left = *std::max_element(nums.begin(), nums.end());
4     int right = std::accumulate(nums.begin(), nums.end(), 0);
5
6     auto can_split = [&](int limit) {
7         int parts = 1;
8         int current = 0;
9         for (const int x : nums) {

```



```

10         if (current + x > limit) {
11             ++parts;
12             current = 0;
13         }
14         current += x;
15     }
16     return parts <= k;
17 };
18
19 while (left < right) {
20     const int mid = left + (right - left) / 2;
21     if (can_split(mid)) {
22         right = mid;
23     } else {
24         left = mid + 1;
25     }
26 }
27 return left;
28 }

```

这题的答案下界是最大元素，因为任何段都必须容纳它；上界是总和，因为把整个数组作为一段一定可行。若某个 `limit` 可行，更大的 `limit` 也可行，所以可行性单调。我们要找最小可行上限，所以是第一个 true。船运包裹、画家分板、磁带分组等题都可以用同一模型。

制作花束最少天数也是答案二分。给定每朵花开放日期，要求制作 `m` 束花，每束需要 `k` 朵相邻开放的花。天数越大，可用花越多，能做出的花束数不会减少。检查函数扫描数组，连续开放花计数达到 `k` 就做一束并清零；遇到未开放花则连续计数归零。若总花数小于 `m*k`，直接不可能。

Listing 6.8 LeetCode 1482 制作花束：二分天数，扫描相邻开放段

```

1 int min_days_make_bouquets(const std::vector<int>& bloom_day, int m, int k)
2 {
3     if (static_cast<long long>(m) * k > static_cast<long long>(bloom_day.size())) {
4         return -1;
5     }
6
7     int left = *std::min_element(bloom_day.begin(), bloom_day.end());
8     int right = *std::max_element(bloom_day.begin(), bloom_day.end());
9
10    auto can_make = [&](int day) {
11        int bouquets = 0;
12        int consecutive = 0;
13        for (const int bloom : bloom_day) {
14            if (bloom <= day) {
15                ++consecutive;
16                if (consecutive == k) {
17                    ++bouquets;
18                    consecutive = 0;
19                }
20            } else {
21                consecutive = 0;
22            }
23        }
24        return bouquets >= m;
25    };

```

```

26
27 while (left < right) {
28     const int mid = left + (right - left) / 2;
29     if (can_make(mid)) {
30         right = mid;
31     } else {
32         left = mid + 1;
33     }
34 }
35 return left;
36 }

```

这题容易错在“相邻”。不能只统计开放花总数，因为一束花要求连续位置。检查函数中的 `consecutive` 就是在维护当前连续开放段长度。每做出一束后清零，是因为同一朵花不能重复使用。这个例子说明答案二分不等于检查函数随便写，检查函数本身也必须严格贴合题意。

二分还可以处理浮点答案，例如求平方根或最大平均值，但终止条件不同。整数二分可以用 `left < right`，因为答案空间离散；浮点二分通常迭代固定次数或直到区间长度小于精度。浮点比较存在误差，不要写需要精确相等的条件。刷题中如果要求保留 `1e-5`，迭代 60 次通常足够覆盖 `double` 精度。

Listing 6.9 浮点二分：固定迭代次数

```

1 double sqrt_by_binary_search(double value)
2 {
3     double left = 0.0;
4     double right = std::max(1.0, value);
5     constexpr int ITERATIONS = 80;
6
7     for (int i = 0; i < ITERATIONS; ++i) {
8         const double mid = left + (right - left) / 2.0;
9         if (mid * mid <= value) {
10             left = mid;
11         } else {
12             right = mid;
13         }
14     }
15     return left;
16 }

```

浮点二分的思想和整数二分相同：判断某个值是否偏小或偏大，然后保留包含答案的一侧。区别在于没有“mid 加一”这种离散排除，区间靠不断缩小逼近答案。若 `value` 可能非常大，`mid * mid` 也可能溢出为无穷，需要按题目范围决定是否改写判断。

二分的反例同样重要。如果 `check(x)` 不是单调的，二分会得到毫无意义的答案。比如数组中某个值是否等于 `x`，在 `x` 的数值范围上不是从 `false` 到 `true` 再保持 `true` 的单调函数；再比如某个速度下恰好用完所有时间，这个“恰好”也不单调。二分要找的是边界，而不是猜测某个离散点。

常见误区

二分常见错误有五个。第一，`mid = (left + right) / 2` 溢出，应该写 `left + (right - left) / 2`。第二，`right = mid - 1` 和 `right = mid` 混用，导致漏掉答案。第三，答案范围不包含真实答案。第四，检查函数不是单调的。第五，返回 `left` 后不验证目标是否存在，尤其是普通查找和范围查找。

二分实验：第一，把搜索插入位置用左闭右闭和左闭右开各写一版，给空数组、单元素、目标小于所有值、目标大于所有值做测试。第二，把查找范围改成找到一个目标后向两边扩展，用全重复数组比较复杂度。第三，为分割数组最大值打印不同 `limit` 下的段数，观察可行性单调。第四，故意构造一个非单调检查函数，用二分跑出错误结果，理解为什么单调性是前提。

答案二分三步

第一，写出答案范围。第二，写出 `check(x)`。第三，证明单调性，并说明要找最小可行值还是最大可行值。没有单调性，不要二分。

本章力扣主线：704 二分查找、35 搜索插入位置、34 查找第一个和最后一个位置、33 搜索旋转排序数组、162 寻找峰值、875 爱吃香蕉的珂珂、1011 在 D 天内送达包裹的能力、410 分割数组的最大值。

二分查找、边界和答案单调性

二分查找最难的不是中点公式，而是把问题改写成边界问题。搜索插入位置找的是第一个大于等于目标的位置，查找首尾区间找的是两个边界，旋转数组找的是有序半边，答案二分找的是可行和不可行的分界。只要边界语义清楚，循环条件、左右更新和返回值就会自然统一；如果边界语义模糊，就会陷入各种加一减一的补丁。

推荐固定左闭右开区间 $[left, right)$ 。初始 `right` 等于候选数量，循环条件是 `left` 小于 `right`，中点落在合法候选中。当谓词在 `mid` 处为假，说明答案在 `mid` 右侧，于是 `left` 等于 `mid` 加一；当谓词为真，说明 `mid` 可能就是答案，于是 `right` 等于 `mid`。循环结束时 `left` 等于 `right`，是第一个满足谓词的位置。这个模板能覆盖 `lower_bound`、答案最小可行值和许多边界题。

答案二分的关键是谓词单调。爱吃香蕉中，速度越快越容易完成；船运包裹中，容量越大需要天数越少；分割数组中，允许的最大段和越大，所需段数越少；最小体力路径中，体力阈值越大，可走边越多。这些问题的原数组未必有序，但答案空间有序。若检查函数内部使用了不正确的贪心，外层二分再漂亮也没有意义。

写检查函数时要先说明贪心为什么正确。船运包裹按顺序装包，当前天能放就放，放不下再开新天；因为包裹顺序不能改变，提前开新天只会让剩余天数更紧，不会更优。分割数组最大值也是同理：在段和不超过上限的前提下尽量延长当前段，可以最小化段数。检查函数本身常常就是一个贪心证明题。

二分的溢出问题不能忽略。中点用 `left` 加 $(right-left)/2$ ，平方比较用 `mid <= x/mid`，求总耗时或总容量时用 `long long`。很多题的输入范围故意接近 `int` 上限，代码逻辑正确但乘法溢出会得到完全错误的判断。复杂度分析也要写成 $O(n \log M)$ ，其中 M 是答案范围，不要笼统写 $O(\log n)$ 。

旋转数组和峰值题展示了另一类二分：不是直接找单调谓词，而是利用局部比较保证某一侧存在答案。寻找峰值中，如果 `nums[mid]` 小于 `nums[mid+1]`，右侧一定有峰；否则左侧含 `mid` 一定有峰。这个证明依赖边界可视为负无穷和相邻不相等的条件。若题目条件改变，例如允许大量重复，就要重新证明是否还能排除一半。

实验建议：把同一道 `lower_bound` 用左闭右闭、左闭右开两种写法实现，再用长度从零到五的所有数组暴力枚举测试。答案二分实验中，先写一个很慢但显然正确的枚举版本，再和二分版本对比。对于每个失败用例，记录 `left`、`mid`、`right` 和谓词值，直到能指出哪条不变量被破坏。

本节复盘

阅读本节后，请回到相邻章节的例题，把暴力瓶颈、优化依据、状态不变量、C++ 容器成本和边界测试重新写一遍。能把同一思想迁移到变体题，才算真正掌握。

树、图、BFS 和 DFS

树和图题的第一难点是建模。题目未必直接给你图，但只要有状态和状态之间的一步转移，就可以看成图。网格题的格子是节点，相邻方向是边；课程表的课程是节点，先修关系是有向边；单词接龙的单词是节点，一次变化是边。

树是没有环的特殊图。二叉树节点通常只有左右孩子，父子关系天然给出层次；普通图没有固定根，可能有环，必须显式维护访问状态。二者的共同点是：算法不会凭空发生，所有遍历都在回答“当前节点能把什么信息交给相邻节点”。BFS 按层推进，适合无权最短步数、层序遍历和多源扩散；DFS 沿路径推进，适合连通块、路径搜索、状态检测和回溯枚举。本书会讲递归思想，但 C++ 实现优先使用显式队列、栈和循环，因为在线评测里极端深树和大图可能让递归调用栈失控。

从对象关系到状态图

树图题的第一步是把题面翻译成状态图，而不是选择 BFS 或 DFS。节点可以是一个对象，也可以是一个复合状态。岛屿数量中，节点是陆地格子；课程表中，节点是课程；单词接龙中，节点是单词；钥匙迷宫中，节点不能只写格子位置，还要加上已经拿到的钥匙集合；障碍消除最短路中，节点要包含剩余可消除次数。状态维度少了会把不同未来能力的状态错误合并，状态维度多了会让搜索空间膨胀，所以建模必须贴着题目约束来。

边表示一步合法转移。网格四方向移动是边，课程先修关系是有向边，字符串改变一个字符是边，树的父子指针也是边。边还可能带权：普通 BFS 默认每条边代价都是一；Dijkstra 需要非负边权；0-1 BFS 需要边权只有零和一；若边权可能为负，就不能直接套 Dijkstra。很多题表面不是图题，但只要存在“当前状态可以一步变成哪些状态”，就可以按图来分析。

访问标记的时机决定复杂度。无权 BFS 中，通常在入队时标记访问，而不是出队时标记。原因是同一层的多个父状态可能指向同一个子状态，如果等到出队才标记，子状态会被重复入队，队列规模可能明显膨胀。DFS 标记则要区分全局访问和路径访问：岛屿连通块用全局访问，访问过的陆地不会再属于新岛；单词搜索用路径访问，当前路径不能重复用格子，但另一条路径仍然可以使用同一格子。把这两类标记混淆，是树图题常见错误。

BFS 的最短性来自层次。队列中先入队的状态距离不大于后入队的状态；当所有边权相等时，第一次到达某个状态就是最短步数。层序遍历、腐烂橘子、多源扩散、转盘锁、无权网格最短路都依赖这个性质。若边权不同，先到达不代表总代价最小，必须改用带权最短路。多源 BFS 不是特殊技巧，它只是把所有起点距离设为零，同时入队，让扩散边界自然合并。

DFS 的价值在于沿路径推进和完成一个连通块。岛屿数量用 DFS 或栈把一个连通块完整标记；拓扑检测可以用三色状态区分未访问、当前路径中、已完成；树的后序问题让子节点信息汇总回父节点。递归版本很短，但 C++ 中深树或大图可能让调用栈溢出，所以显式栈和队列是更稳的工程表达。显式结构还有一个好处：状态里需要额外携带深度、父节点、路径信息或资源时，结构体比隐式调用栈更容易看清。

树题还要先判断它是不是搜索树。普通二叉树只有父子结构，没有值域顺序；BST 额外保证左子树小于根、右子树大于根。验证 BST、中序第 k 小、BST 最近公共祖先都可以使用有序性；普通树最近公共祖先、最大深度、路径和不能随便使用值大小剪枝。很多错误来自把 BST 性质套到普通树，或者只比较当前节点和左右孩子而忽略祖先边界。

树图题的建模清单

写代码前先回答：节点状态是什么，边是什么，边是否有权，访问标记按什么维度保存，访问标记何时设置，输出目标是连通块、最短路、拓扑顺序、路径枚举还是子树汇总。若同一位置携带不同资源会影响未来选择，就必须把资源放入状态；若边权不是统一的一，普通 BFS 的第一次到达最短性就不再成立。

为了让代码统一，本章二叉树示例使用下面的节点字段名。力扣原题多使用 `val`，实际提交时

把 `value` 改成 `val` 即可，核心逻辑不变。

Listing 7.1 二叉树节点示意

```
1 struct TreeNode {
2     int value = 0;
3     TreeNode* left = nullptr;
4     TreeNode* right = nullptr;
5 };
```

二叉树层序遍历是 BFS 的入门题，但它不是“把队列模板背下来”这么简单。题目要求按层返回节点值，因此队列里保存的是下一批待访问节点，而外层循环开始时，队列必须恰好包含当前层的所有节点。暴力做法可以先求树高，再对每一层从根节点重新扫描一次，把深度等于目标层的节点收集出来。这个思路正确，但一棵高度为 h 的树会被重复扫描很多次，最坏退化到 $O(nh)$ 。

优化来自队列的先进先出顺序。根节点先入队，处理当前层时，把它的孩子按从左到右的顺序放到队尾。当前层节点全部弹出后，队列中剩下的正好是下一层节点。这里的关键变量是 `level_size`：它把“当前层的边界”固定下来，避免新入队的下一层节点被提前处理。

Listing 7.2 LeetCode 102 二叉树层序遍历：固定当前层边界

```
1 std::vector<std::vector<int>> level_order(TreeNode* root)
2 {
3     std::vector<std::vector<int>> levels;
4     if (root == nullptr) {
5         return levels;
6     }
7
8     std::queue<TreeNode*> queue;
9     queue.push(root);
10
11     while (!queue.empty()) {
12         const int level_size = static_cast<int>(queue.size());
13         std::vector<int> level;
14         level.reserve(level_size);
15
16         for (int i = 0; i < level_size; ++i) {
17             TreeNode* node = queue.front();
18             queue.pop();
19             level.push_back(node->value);
20
21             if (node->left != nullptr) {
22                 queue.push(node->left);
23             }
24             if (node->right != nullptr) {
25                 queue.push(node->right);
26             }
27         }
28
29         levels.push_back(std::move(level));
30     }
31
32     return levels;
33 }
```


这题的复杂度是 $O(n)$ ，每个节点入队一次、出队一次。空间复杂度是 $O(w)$ ， w 是树的最大宽度；如果把输出也计入空间，则是 $O(n)$ 。常见错误有三个：空树忘记返回空数组；没有记录 `level_size`，导致层边界混在一起；把右孩子放到左孩子前面，破坏题目要求的从左到右顺序。面试时要把“不变量”说出来：每次进入外层循环，队列中保存的是同一层、尚未输出的全部节点。

最大深度看起来更像递归题，数学定义也很自然：空树深度为 0，非空树深度是左右子树最大深度加 1。递归写法短，但深度很大的链式树会把调用栈压得很深。用 BFS 写法时，最大深度就是层序遍历处理过的层数；用显式栈写法时，栈里保存节点和它所在深度。两种实现都能避免把深度交给语言调用栈管理。

Listing 7.3 LeetCode 104 二叉树最大深度：显式栈保存节点深度

```
1 int max_depth(TreeNode* root)
2 {
3     if (root == nullptr) {
4         return 0;
5     }
6
7     int answer = 0;
8     std::vector<std::pair<TreeNode*, int>> stack{{root, 1}};
9
10    while (!stack.empty()) {
11        const auto [node, depth] = stack.back();
12        stack.pop_back();
13        answer = std::max(answer, depth);
14
15        if (node->left != nullptr) {
16            stack.emplace_back(node->left, depth + 1);
17        }
18        if (node->right != nullptr) {
19            stack.emplace_back(node->right, depth + 1);
20        }
21    }
22
23    return answer;
24 }
```

这段代码的状态非常明确：栈里的每个元素都是一个尚未展开的节点，以及从根到它经过的节点数。暴力和优化的差别不在复杂度，递归和显式栈都是 $O(n)$ ；差别在工程边界。显式栈把潜在的系统调用栈风险变成可控的堆上容器，也更容易扩展成“最大深度路径”“带权深度”“同时记录父节点”等变体。易错点是深度定义：力扣 104 问节点数层数，根节点深度应是 1；如果题目问边数高度，根节点高度才可能记作 0。

验证二叉搜索树经常暴露“只看局部”的错误。很多人会写成：当前节点大于左孩子、小于右孩子，就继续检查左右子树。这个判断不够，因为 BST 的约束是全局的：左子树所有节点都必须小于当前节点，右子树所有节点都必须大于当前节点。暴力补救方法是对每个节点扫描整棵左子树最大值和右子树最小值，但会重复访问，最坏 $O(n^2)$ 。

更稳定的优化是利用 BST 的中序遍历性质。对合法 BST 做中序遍历，访问值必须严格递增。于是全局范围约束被转换成相邻访问值比较：只要当前值不大于前一个中序值，就一定非法。这里要强调“严格递增”，因为力扣 98 不允许重复值出现在左右子树。

Listing 7.4 LeetCode 98 验证二叉搜索树：显式栈中序遍历

```
1 bool is_valid_bst(TreeNode* root)
2 {
```

```

3     std::vector<TreeNode*> stack;
4     TreeNode* current = root;
5     std::optional<long long> previous;
6
7     while (current != nullptr || !stack.empty()) {
8         while (current != nullptr) {
9             stack.push_back(current);
10            current = current->left;
11        }
12
13        current = stack.back();
14        stack.pop_back();
15
16        if (previous.has_value() && current->value <= previous.value()) {
17            return false;
18        }
19        previous = current->value;
20        current = current->right;
21    }
22
23    return true;
24 }

```

这题的时间复杂度是 $O(n)$ ，空间复杂度是 $O(h)$ ， h 是树高。代码使用 `long long` 保存前一个值，是为了避免节点值接近 `int` 边界时用哨兵值出错；用 `std::optional` 比写一个“极小初值”更严谨。面试表达可以这样组织：先指出局部比较错误，再给出全局约束，最后说明中序遍历把全局约束转成单调序列检查。

最近公共祖先的题意是：给定二叉树中两个节点 p 和 q ，找出最深的那个节点，使它同时是 p 和 q 的祖先。暴力想法是对每个候选节点分别判断它的子树里是否包含 p 和 q ，这会产生大量重复搜索。经典递归解法会让子树向父节点汇报“是否找到了目标”，但同样存在深树调用栈问题。本书给出显式父指针版本：先遍历树，记录每个节点的父节点；再把 p 的所有祖先放入集合；最后从 q 不断向上走，第一次遇到的已访问祖先就是答案。

Listing 7.5 LeetCode 236 二叉树最近公共祖先：父指针和祖先集合

```

1  TreeNode* lowest_common_ancestor(TreeNode* root, TreeNode* p, TreeNode* q)
2  {
3      if (root == nullptr || p == nullptr || q == nullptr) {
4          return nullptr;
5      }
6
7      std::unordered_map<TreeNode*, TreeNode*> parent;
8      std::vector<TreeNode*> stack{root};
9      parent.emplace(root, nullptr);
10
11     while (!stack.empty() && (!parent.contains(p) || !parent.contains(q))) {
12         TreeNode* node = stack.back();
13         stack.pop_back();
14
15         if (node->left != nullptr) {
16             parent.emplace(node->left, node);
17             stack.push_back(node->left);
18         }

```

```

19     if (node->right != nullptr) {
20         parent.emplace(node->right, node);
21         stack.push_back(node->right);
22     }
23 }
24
25 std::unordered_set<TreeNode*> ancestors;
26 TreeNode* current = p;
27 while (current != nullptr) {
28     ancestors.insert(current);
29     current = parent[current];
30 }
31
32 current = q;
33 while (current != nullptr) {
34     if (ancestors.contains(current)) {
35         return current;
36     }
37     current = parent[current];
38 }
39
40 return nullptr;
41 }

```

为什么从 q 向上遇到的第一个公共祖先就是“最近”的？因为向上走的顺序是从深到浅，越早遇到越靠近 q ，同时它已经在 p 的祖先集合中。这个算法时间 $O(n)$ ，空间 $O(n)$ 。如果题目保证 p 和 q 一定在树中，前面的遍历可以在二者父指针都找到后提前停止；如果不保证，就需要完整遍历并检查二者是否出现。常见错误是把“节点值相同”当成“节点相同”，但力扣 236 给的是节点指针，应该比较节点地址。

岛屿数量把网格转成无向图。节点是值为 '1' 的格子，边是上下左右相邻的陆地。题目要求岛屿个数，就是要求连通块个数。暴力思路可能会对每个陆地格子都搜索一次，看它属于哪个岛，这会反复走过同一片陆地。正确优化是：扫描网格时，只要遇到一块尚未访问的陆地，就说明发现了一个新连通块；从它出发把整个连通块标记掉，答案加一。

Listing 7.6 LeetCode 200 岛屿数量：显式栈标记连通块

```

1 int num_islands(std::vector<std::vector<char>>& grid)
2 {
3     if (grid.empty() || grid[0].empty()) {
4         return 0;
5     }
6
7     constexpr char LAND = '1';
8     constexpr char WATER = '0';
9     constexpr std::array<std::pair<int, int>, 4> DIRECTIONS{
10         std::pair{1, 0}, std::pair{-1, 0},
11         std::pair{0, 1}, std::pair{0, -1}
12     };
13
14     const int rows = static_cast<int>(grid.size());
15     const int cols = static_cast<int>(grid[0].size());
16     int islands = 0;
17

```

```

18     for (int row = 0; row < rows; ++row) {
19         for (int col = 0; col < cols; ++col) {
20             if (grid[row][col] != LAND) {
21                 continue;
22             }
23
24             ++islands;
25             std::vector<std::pair<int, int>> stack{{row, col}};
26             grid[row][col] = WATER;
27
28             while (!stack.empty()) {
29                 const auto [r, c] = stack.back();
30                 stack.pop_back();
31
32                 for (const auto [dr, dc] : DIRECTIONS) {
33                     const int next_row = r + dr;
34                     const int next_col = c + dc;
35                     const bool out_of_bounds =
36                         next_row < 0 || next_row >= rows ||
37                         next_col < 0 || next_col >= cols;
38                     if (out_of_bounds || grid[next_row][next_col] != LAND) {
39                         continue;
40                     }
41
42                     grid[next_row][next_col] = WATER;
43                     stack.emplace_back(next_row, next_col);
44                 }
45             }
46         }
47     }
48
49     return islands;
50 }

```

这个实现把访问过的陆地直接改成水，因此不需要额外 **visited** 数组。如果题目不允许修改输入，就用同样大小的布尔矩阵保存访问状态。不变量是：扫描到任意位置时，所有已经发现过的岛都被完全标记为水，尚未被扫描到的陆地如果仍为 '1'，它一定属于一个尚未计数的新岛。时间复杂度 $O(mn)$ ，空间复杂度最坏 $O(mn)$ ，来自显式栈。易错点是访问标记时机：应该在入栈或入队时立刻标记，而不是弹出后再标记，否则同一格子可能被多个邻居重复加入。

腐烂橘子是多源 BFS。每个新鲜橘子变烂的时间，等于它到最近初始腐烂橘子的最短四联通距离。暴力想法是每一分钟扫描整张网格，看哪些新鲜橘子旁边有腐烂橘子。这种写法虽然直观，却会反复扫描空格、已经腐烂的格子和暂时不会变化的格子。优化方法是把所有初始腐烂橘子同时入队，只从当前腐烂边界向外扩散。

Listing 7.7 LeetCode 994 腐烂的橘子：多源 BFS 计算最短腐烂时间

```

1 int oranges_rotting(std::vector<std::vector<int>>& grid)
2 {
3     if (grid.empty() || grid[0].empty()) {
4         return 0;
5     }
6
7     constexpr int EMPTY = 0;

```

```

8   constexpr int FRESH = 1;
9   constexpr int ROTTEN = 2;
10  constexpr std::array<std::pair<int, int>, 4> DIRECTIONS{
11      std::pair{1, 0}, std::pair{-1, 0},
12      std::pair{0, 1}, std::pair{0, -1}
13  };
14
15  const int rows = static_cast<int>(grid.size());
16  const int cols = static_cast<int>(grid[0].size());
17  std::queue<std::pair<int, int>> queue;
18  int fresh_count = 0;
19
20  for (int row = 0; row < rows; ++row) {
21      for (int col = 0; col < cols; ++col) {
22          if (grid[row][col] == ROTTEN) {
23              queue.emplace(row, col);
24          } else if (grid[row][col] == FRESH) {
25              ++fresh_count;
26          }
27      }
28  }
29
30  int minutes = 0;
31  while (!queue.empty() && fresh_count > 0) {
32      const int level_size = static_cast<int>(queue.size());
33      for (int step = 0; step < level_size; ++step) {
34          const auto [row, col] = queue.front();
35          queue.pop();
36
37          for (const auto [dr, dc] : DIRECTIONS) {
38              const int next_row = row + dr;
39              const int next_col = col + dc;
40              const bool out_of_bounds =
41                  next_row < 0 || next_row >= rows ||
42                  next_col < 0 || next_col >= cols;
43              if (out_of_bounds || grid[next_row][next_col] != FRESH) {
44                  continue;
45              }
46
47              grid[next_row][next_col] = ROTTEN;
48              --fresh_count;
49              queue.emplace(next_row, next_col);
50          }
51      }
52      ++minutes;
53  }
54
55  return fresh_count == 0 ? minutes : -1;
56 }

```

这里 `level_size` 表示同一分钟内会继续扩散的腐烂橘子数量。处理完这一层后，才让 `minutes` 加一。初始没有新鲜橘子时，答案是 0；队列空了但仍有新鲜橘子时，说明有橘子被空格隔开，永远不会腐烂，答案是 -1。代码中 `EMPTY` 没有参与判断，但保留常量能让输入语义更清楚。

复杂度仍是 $O(mn)$ ，因为每个格子最多入队一次。

课程表是有向图判环。每门课是节点，先修关系 `[course, prerequisite]` 表示必须先学 `prerequisite`，再学 `course`，所以边的方向应是 `prerequisite -> course`。暴力想法是反复扫描所有课程，找当前没有未完成先修课的课程；每学一门课又重新检查所有依赖，容易写成 $O(nm)$ 。拓扑排序的优化点是入度数组：`indegree[x]` 表示课程 `x` 还有多少前置依赖没有被满足。

Listing 7.8 LeetCode 210 课程表 II：拓扑排序输出可行学习顺序

```

1 std::vector<int> find_order(
2     int num_courses,
3     const std::vector<std::vector<int>>& prerequisites)
4 {
5     std::vector<std::vector<int>> graph(num_courses);
6     std::vector<int> indegree(num_courses, 0);
7
8     for (const auto& edge : prerequisites) {
9         const int course = edge[0];
10        const int prerequisite = edge[1];
11        graph[prerequisite].push_back(course);
12        ++indegree[course];
13    }
14
15    std::queue<int> queue;
16    for (int course = 0; course < num_courses; ++course) {
17        if (indegree[course] == 0) {
18            queue.push(course);
19        }
20    }
21
22    std::vector<int> order;
23    order.reserve(num_courses);
24
25    while (!queue.empty()) {
26        const int course = queue.front();
27        queue.pop();
28        order.push_back(course);
29
30        for (const int next : graph[course]) {
31            --indegree[next];
32            if (indegree[next] == 0) {
33                queue.push(next);
34            }
35        }
36    }
37
38    if (static_cast<int>(order.size()) != num_courses) {
39        return {};
40    }
41    return order;
42 }

```

这段代码同时覆盖力扣 207 和 210。207 只问能否完成所有课程，可以返回 `order.size() == num_courses`；210 要返回一个具体顺序，若有环则返回空数组。正确性的核心是：入度为 0 的节

点当前没有任何未满足依赖，可以安全学习；学习它只会减少它指向的后继课程入度，不会影响其他边。每个节点最多入队一次，每条边处理一次，复杂度 $O(n + m)$ 。最常见错误是把边方向建反。边建反时，有时仍然能判断是否有环，但输出顺序会变成反向，面试中必须主动说清方向。

单词接龙是无权图最短路。节点是单词，两个单词只差一个字符就连一条边；答案是起点词到终点词的最短转换长度。直接建图的暴力方法是枚举所有单词对，逐字符比较是否只差一位，复杂度约为 $O(n^2L)$ ，其中 L 是单词长度。BFS 本身没有问题，慢在找邻居。

优化思路是用通配模式把邻居查询变成哈希查询。比如 **hot** 可以生成 ***ot**、**h*t**、**ho*** 三个模式；所有能匹配同一模式的单词，彼此之间只差一个字符。预处理时把模式映射到单词列表，BFS 时对当前单词生成模式，从桶里取候选邻居。这样避免了每一步都扫描全词表。

Listing 7.9 LeetCode 127 单词接龙：通配模式加 BFS

```

1 int ladder_length(const std::string& begin_word,
2                 const std::string& end_word,
3                 const std::vector<std::string>& word_list)
4 {
5     constexpr char WILDCARD = '*';
6     const int word_length = static_cast<int>(begin_word.size());
7
8     std::unordered_set<std::string> dictionary(
9         word_list.begin(), word_list.end());
10    if (!dictionary.contains(end_word)) {
11        return 0;
12    }
13
14    std::unordered_map<std::string, std::vector<std::string>> buckets;
15    for (const std::string& word : word_list) {
16        for (int i = 0; i < word_length; ++i) {
17            std::string pattern = word;
18            pattern[i] = WILDCARD;
19            buckets[pattern].push_back(word);
20        }
21    }
22
23    std::queue<std::pair<std::string, int>> queue;
24    std::unordered_set<std::string> visited;
25    queue.emplace(begin_word, 1);
26    visited.insert(begin_word);
27
28    while (!queue.empty()) {
29        const auto [word, distance] = queue.front();
30        queue.pop();
31        if (word == end_word) {
32            return distance;
33        }
34
35        for (int i = 0; i < word_length; ++i) {
36            std::string pattern = word;
37            pattern[i] = WILDCARD;
38            auto found = buckets.find(pattern);
39            if (found == buckets.end()) {
40                continue;
41            }
42        }
    }

```

```

43         for (const std::string& next : found->second) {
44             if (visited.contains(next)) {
45                 continue;
46             }
47             visited.insert(next);
48             queue.emplace(next, distance + 1);
49         }
50         found->second.clear();
51     }
52 }
53
54 return 0;
55 }

```

这题的 BFS 不变量是：队列中每个状态的 `distance` 都是从起点到该单词的最短转换长度；第一次到达 `endWord` 时，答案已经最短。为什么可以在访问后清空模式桶？因为一个模式桶只需要被展开一次。之后再遇到同一模式，桶里的所有邻居要么已经访问，要么已经入队；重复扫描只会增加时间。预处理复杂度约为 $O(nL^2)$ ，因为每次生成模式会复制长度为 `L` 的字符串；BFS 展开在清桶优化后也是同量级。若要继续优化，可以把单词编号化，模式只保存编号，但面试中这个版本已经足够清晰。

树图题复盘模板

每道树图题都先写四句话：节点是什么，边是什么，状态里保存什么，访问标记何时设置。二叉树题再补一句返回值语义或队列不变量；图题再补一句为什么 BFS 第一次到达就是最短，或者为什么 DFS 标记完整连通块。写完代码后，用空输入、单节点、链式树、完全树、孤立节点、有环依赖、不可达终点各测一遍。能把这些边界说清楚，才算真正掌握，而不是只背了遍历模板。

tree-graph 深度题卡

二叉树与树形状态

下面只在这里讲一次共性：先写暴力基线，定位最贵的重复工作，再选择能维护不变量的数据结构。每道题卡只分析本题自己的建模、瓶颈、边界和变体，复习时不要把家族共性当成答案。

图示：二叉树与树形状态推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	慢点在于同一棵子树被重复求高度、最大值、是否包含目标等信息。若每个节点能一次性向父节点汇报足够信息，就不应重复下钻。
结构选择	树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。
不变量	返回值或栈元素必须有语义：当前子树高度、当前路径和、当前节点合法上下界、父指针、层号。不要只说遍历。
代码落地	示例中可以用指针节点，但比较节点身份时比较指针。极端深树用 <code>std::vector<TreeNode*></code> 显式栈或队列。最大路径类题要用足够小的初值，不能默认零。

LeetCode 94 二叉树中序遍历

题目模型。中序访问顺序是左子树、根、右子树，BST 中会得到有序序列。

推导重点。用显式栈模拟一路向左，再回退访问根并转向右子树。

边界。空树、单节点、链式左树、链式右树。

变体追问。 Morris 遍历可做到常数额外空间，但会临时改树结构。

LeetCode 98 验证二叉搜索树

题目模型。 BST 约束是全局上下界，不是只比较父子节点。

推导重点。 中序严格递增或显式传递上下界都可以；中序版本要保存前一个访问值。

边界。 重复值、int 最小最大值、局部合法但全局非法。

变体追问。 若允许重复值，需要明确重复放左还是放右。

LeetCode 102 二叉树层序遍历

题目模型。 队列在每轮开始时保存当前层所有节点。

推导重点。 记录 level size 固定层边界，避免下一层节点混入当前层。

边界。 空树、只有根、最后一层不满、左右顺序。

变体追问。 锯齿形层序只是在每层输出顺序上增加方向控制。

LeetCode 103 二叉树锯齿形层序遍历

题目模型。 BFS 层边界不变，变化的是当前层写入结果的方向。

推导重点。 可以层内反转，也可以预分配数组按方向写入目标下标。

边界。 单层、偶数层、奇数层、空树。

变体追问。 若要求从底向上输出，收集后反转层列表即可。

LeetCode 104 二叉树最大深度

题目模型。 深度可以看成层数，也可以作为栈中携带的状态。

推导重点。 显式栈保存节点和深度，避免极端链式树递归栈过深。

边界。 空树返回零、根深度是一、链式树。

变体追问。 最小深度不能简单取左右最小，因为空子树不构成叶子路径。

LeetCode 105 前序中序构造二叉树

题目模型。 前序给根，中序把左右子树切开。

推导重点。 用哈希表记录中序下标，避免每次线性寻找根位置。

边界。 节点值唯一、空子树、左右子树长度计算、输入不合法。

变体追问。 后序中序构造类似，只是根来自后序末尾。

LeetCode 106 中序后序构造二叉树

题目模型。 后序末尾是根，中序位置确定左右子树规模。

推导重点。 构造时要小心后序左右区间边界，避免切分错位。

边界。 空树、单节点、全左链、全右链。

变体追问。 若给前序和后序，二叉树通常不能唯一确定，除非额外限制。

LeetCode 108 有序数组转二叉搜索树

题目模型。 有序数组中点作为根能让左右规模接近。

推导重点。 每次选择区间中点，左右子区间构造左右子树。

边界。 空区间、偶数长度选左中还是右中、高度平衡定义。

变体追问。 工程上递归可读，若严格避免递归可用队列保存区间和父指针。

LeetCode 110 平衡二叉树

题目模型。 每个节点需要知道子树高度以及是否已经失衡。

推导重点。 后序汇总时一旦子树失衡就向上返回失败标记，避免重复算高度。

边界。 空树、单节点、左右高度差正好一、链式树。

变体追问。 可以把返回值设计成 optional 高度，空表示不平衡。

LeetCode 111 二叉树最小深度

题目模型。 最小深度是到最近叶子的节点数，不是左右深度简单取最小。

推导重点。 BFS 第一次遇到叶子就是最小深度，因为按层扩展。

边界。 只有左子树、只有右子树、根为空、根是叶子。

变体追问。 DFS 写法必须特殊处理空子树不能作为最小路径。

LeetCode 112 路径总和

题目模型。 路径必须从根到叶子，状态是剩余目标值。

推导重点。 沿路径减去当前节点值，到叶子时检查剩余是否等于叶子值。

边界。 负数节点、目标为零、非叶子提前达到目标、空树。

变体追问。 若要求列出所有路径，需要保存路径数组并在回退时弹出。

LeetCode 113 路径总和 II

题目模型。 相比判定题，状态还要保存当前路径。

推导重点。 到叶子且和满足时复制路径；回溯时恢复路径。

边界。 多个答案、负数、空树、路径数组复制时机。

变体追问。 若路径不要求从根到叶，模型会变成前缀和加哈希。

LeetCode 114 二叉树展开为链表

题目模型。 目标是按前序顺序把树原地改成右指针链。

推导重点。 显式栈按右后左先入顺序处理，逐个接到前驱右侧。

边界。 空树、单节点、原有右子树不能丢、左指针置空。

变体追问。 也可用寻找左子树最右节点的原地方法。

LeetCode 124 二叉树最大路径和

题目模型。 路径可以从任意节点到任意节点，但不能分叉向父节点贡献两边。

推导重点。 每个节点向父亲贡献单边最大增益，全局答案可用左右两边加当前节点更新。

边界。 全负数、单节点、左右增益为负时取零、答案初始值。

变体追问。 若路径必须从根到叶，状态和答案位置完全不同。

LeetCode 199 二叉树右视图

题目模型。 每层从右侧能看到的的就是该层最后访问或最先访问的右侧节点。

推导重点。 BFS 固定层边界，取每层最后一个；或 DFS 按右优先首次到达深度。

边界。 空树、左链、右链、左右交错。

变体追问。 若求左视图，只改变层内取值方向。

LeetCode 208 实现 Trie

题目模型。 前缀树把字符串公共前缀共享成路径。

推导重点。 插入和查询都逐字符沿边移动，不存在的边按需要创建。

边界。 空字符串约定、单词结束标记、前缀存在不等于单词存在。

变体追问。 若字符集很大，用哈希子节点；小写字母用数组更快。

LeetCode 211 添加与搜索单词

题目模型。 点号通配符让查询可能分叉，但前缀树仍能剪掉不匹配前缀。

推导重点。 普通字符沿唯一边走，点号枚举当前节点所有孩子。

边界。 连续通配符、空结果、单词结束标记、字符集大小。

变体追问。 大量通配符会接近遍历整棵 Trie，需要规模约束。

LeetCode 226 翻转二叉树

题目模型。 每个节点交换左右孩子即可，遍历顺序不影响最终结构。

推导重点。 显式队列或栈逐个节点交换，避免递归深度。

边界。 空树、单节点、只有一侧子树、重复值。

变体追问。 这题简单但适合训练树节点指针修改。

LeetCode 230 二叉搜索树中第 K 小

题目模型。 BST 中序遍历按升序访问节点。

推导重点。 显式栈中序，访问第 k 个节点时返回。

边界。 k 为一、k 为节点数、树不平衡、是否有重复值。

变体追问。若频繁查询，可在节点维护子树大小成为顺序统计树。

LeetCode 235 二叉搜索树最近公共祖先

题目模型。BST 的值域关系能决定两个目标在当前节点的哪一侧。

推导重点。两个值都小去左，都大去右，否则当前节点就是分叉点。

边界。一个节点是另一个祖先、目标顺序、重复值约定。

变体追问。普通二叉树不能用值域剪枝，需要父指针或后序状态。

LeetCode 236 二叉树最近公共祖先

题目模型。普通树没有有序性，只能依靠祖先关系。

推导重点。父指针集合或后序返回目标命中状态都可行。

边界。目标不存在、p 等于 q、一个是另一个祖先、比较节点地址。

变体追问。多次 LCA 查询可预处理倍增祖先表。

LeetCode 337 打家劫舍 III

题目模型。树上相邻节点不能同时选，每个节点需要偷和不偷两个状态。

推导重点。后序汇总：偷当前则孩子不能偷，不偷当前则孩子可偷可不偷。

边界。空树、负值是否可能、链式树、递归深度。

变体追问。可用显式栈后序保存节点状态，避免调用栈。

LeetCode 450 删除二叉搜索树中的节点

题目模型。删除节点后仍要保持 BST 中序有序。

推导重点。若有两个孩子，用后继或前驱替换当前值，再删除那个后继节点。

边界。删除叶子、一个孩子、两个孩子、删除根节点。

变体追问。工程实现要清楚是移动值还是重连节点。

LeetCode 543 二叉树直径

题目模型。直径经过某节点时等于左高加右高，但向父亲只能贡献单边高度。

推导重点。后序遍历同时更新全局直径并返回高度。

边界。空树、单节点、链式树、边数还是节点数定义。

变体追问。最大路径和是同型题，但贡献值和全局更新有负数处理。

图建模、BFS 与 DFS

下面只在这里讲一次共性：先写暴力基线，定位最贵的重复工作，再选择能维护不变量的数据结构。每道题目只分析本题自己的建模、瓶颈、边界和变体，复习时不要把家族共性当成答案。

图示：图建模、BFS 与 DFS 推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	慢点来自重复访问和邻居查询过慢。若节点很多但边很少，用邻接矩阵会浪费空间；若每次找邻居都扫描全体节点，BFS 本身再正确也会超时。
结构选择	无权最短步数用 BFS，连通块和状态检测用 DFS 或显式栈，有向无环依赖用拓扑排序。多源问题把所有源同时入队，状态带资源的问题不能只按位置去重。
不变量	访问标记通常在入队或入栈时设置，保证一个状态只被加入一次。BFS 队列按距离递增；拓扑排序队列保存当前入度为零的节点。
代码落地	图节点连续时用 <code>std::vector<std::vector<int>></code> ；节点是字符串时先编号或用哈希表。网格方向数组用 <code>constexpr std::array</code> ，边界判断集中写。

LeetCode 127 单词接龙

题目模型。每个单词是节点，一次修改一个字符形成边。

推导重点。通配模式桶加速邻居查询，BFS 第一次到达终点就是最短转换长度。

边界。终点不在词表、起点和终点相同、桶重复扫描、单词长度一致。

变体追问。双向 BFS 可以进一步降低搜索空间。

LeetCode 130 被围绕的区域

题目模型。只有不连通到边界的 O 才会被翻转。

推导重点。从边界 O 出发标记安全区域，再翻转剩余 O。

边界。全边界、无 O、单行单列、内部岛屿。

变体追问。这是反向思维：找不能被翻转的区域比直接判断每块区域更简单。

LeetCode 133 克隆图

题目模型。每个原节点需要唯一对应一个新节点，边按原图连接。

推导重点。哈希表保存原节点到克隆节点的映射，BFS 或 DFS 遍历图。

边界。空图、自环、重边、环结构、不能按节点值唯一假设。

变体追问。若图节点值不唯一，必须用地址作为键。

LeetCode 200 岛屿数量

题目模型。陆地格子是节点，四方向相邻是边，岛屿是连通块。

推导重点。扫描遇到未访问陆地就启动搜索并标记整块。

边界。空网格、全水、全陆、斜对角不连通、是否允许修改输入。

变体追问。也可用并查集合并相邻陆地，适合动态岛屿扩展。

LeetCode 207 课程表

题目模型。课程是节点，先修关系是有向边，问题是判断是否有环。

推导重点。入度拓扑排序不断学习当前无依赖课程。

边界。边方向、孤立课程、环、自依赖、重复边。

变体追问。DFS 三色标记也能判环，但要明确访问状态。

LeetCode 210 课程表 II

题目模型。在判环基础上还要输出一个可行拓扑序。

推导重点。每弹出一个入度为零课程，就把它加入顺序并删除出边。

边界。有环返回空数组、多个可行顺序、边方向。

变体追问。若需要字典序最小拓扑序，把队列换成小顶堆。

LeetCode 417 太平洋大西洋水流问题

题目模型。从每个格子向海搜索很慢，反向从海边沿非递减高度搜索。

推导重点。分别标记能到太平洋和大西洋的格子，交集就是答案。

边界。单行单列、平台高度、边界重复入队、方向条件反向。

变体追问。反向搜索是很多可达性题的重要技巧。

LeetCode 695 岛屿的最大面积

题目模型。面积就是一个连通块中陆地格子的数量。

推导重点。每发现新陆地就搜索完整连通块并计数，取最大值。

边界。全水、全陆、细长岛、是否允许修改输入。

变体追问。与岛屿数量相比，只是连通块聚合值不同。

LeetCode 752 打开转盘锁

题目模型。四位密码是状态，每次转动一位形成边。

推导重点。BFS 从起点扩展，死亡状态不能入队，访问状态避免重复。

边界。起点死亡、目标就是起点、数字环绕、死亡集合很大。

变体追问。双向 BFS 能明显减少状态展开。

LeetCode 994 腐烂的橘子

题目模型。所有初始腐烂橘子同时作为 BFS 源点。

推导重点。按层扩散，分钟数等于最后一个新鲜橘子被首次访问的层数。

边界。没有新鲜橘子、没有腐烂源、隔离新鲜橘子、空格阻断。

变体追问。多源 BFS 也适用于最近零距离、地图扩散等题。

最短路与带权图

下面只在这里讲一次共性：先写暴力基线，定位最贵的重复工作，再选择能维护不变量的数据结构。每道题目只分析本题自己的建模、瓶颈、边界和变体，复习时不要把家族共性当成答案。

图示：最短路与带权图推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	路径数量可能指数级。最短路算法的核心是用当前已知最小代价组织扩展顺序，而不是盲目枚举路径。
结构选择	非负权图用 Dijkstra，小顶堆保存当前距离最小的候选；边权为一用 BFS；有负边要考虑 Bellman-Ford 或重新建模。路径代价若是最大边权，也可用 Dijkstra 或二分答案。
不变量	Dijkstra 的不变量是：每次弹出的未过期节点，其距离已经是最终最短距离。过期堆元素必须跳过，否则会重复扩展旧状态。
代码落地	由于 <code>priority_queue</code> 没有 <code>decrease-key</code> ，更新距离时压入新状态，弹出时比较距离数组。距离用 <code>long long</code> 更稳，图用邻接表保存 (<code>to, weight</code>)。

LeetCode 743 网络延迟时间

题目模型。有向正权图从起点到所有节点的最短路最大值。

推导重点。Dijkstra 求每个节点最短距离，最后取最大，若有无穷则不可达。

边界。节点编号从一开始、不可达、重边、过期堆元素。

变体追问。边权相同才可用普通 BFS。

LeetCode 1631 最小体力消耗路径

题目模型。路径代价是沿途最大高度差，不是边权和。

推导重点。Dijkstra 的松弛改成取当前代价和新边代价的最大值。

边界。单格、平坦网格、必须绕路、多个相同代价。

变体追问。也可二分体力上限后 BFS 检查可达。

并查集与动态连通性

下面只在这里讲一次共性：先写暴力基线，定位最贵的重复工作，再选择能维护不变量的数据结构。每道题目只分析本题自己的建模、瓶颈、边界和变体，复习时不要把家族共性当成答案。

图示：并查集与动态连通性推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	瓶颈是连通性查询和合并交替出现。若只需要知道是否同组，不需要路径本身，并查集正好匹配。
结构选择	路径压缩把查找路径上的节点直接挂到根，按大小或按秩合并控制树高。邮箱、等式、边、网格都可以映射成元素集合。
不变量	每个集合有一个代表元， <code>parent[root] == root</code> 。合并只改变一个根的父亲，非根节点的父亲可能不是最终根，但 <code>find</code> 会返回正确代表。
代码落地	类成员访问使用 <code>this-></code> 。迭代版 <code>find</code> 可以避免递归。若需要维护集合大小、边数、权值差或奇偶关系，要扩展额外数组并在合并时同步更新。

LeetCode 547 省份数量

题目模型。城市连接关系形成无向图，省份就是连通分量。

推导重点。并查集合并所有直接连接的城市，最后统计根数量。

边界。邻接矩阵对称、自连接、孤立城市、只扫描上三角。

变体追问。BFS 也可做；并查集更突出动态合并。

LeetCode 684 冗余连接

题目模型。树加一条边会形成唯一环。

推导重点。按顺序加边，若一条边两端已连通，则它就是冗余边。

边界。节点编号、最后一条成环边、重复边、并查集初始化大小。

变体追问。有向冗余连接需要同时处理入度为二和环，复杂度更高。

图搜索、最短路和状态建模

图题不是看到网格就写 DFS，也不是看到最短就写 BFS。第一步永远是建模：节点是什么，边是什么，边权是什么，状态是否还包含钥匙、剩余障碍次数、访问集合或时间。岛屿数量的节点是陆地格子，边是四方向相邻；课程表的节点是课程，边是先修依赖；开锁问题的节点是四位密码，边是转动一位；最小体力路径的节点是格子，边权是高度差，但路径代价是最大边权。

BFS 的正确性来自层次扩展。无权图中，从起点出发，队列先处理距离为零的状态，再处理距离为一的状态，然后是距离为二。只要访问标记在入队时设置，一个状态第一次入队时的距离就是最短距离。若等到出队才标记，同一状态可能被同层多个前驱重复加入，虽然有时答案仍对，复杂度却会膨胀。多源 BFS 只是把所有源点同时放进距离为零的层。

DFS 更适合连通块、路径搜索和状态检测。连通块题中，一次 DFS 或显式栈搜索会标记从起点可达的全部节点；拓扑判环中，三色标记能区分未访问、当前路径中、已完成；回溯搜索中，DFS 路径状态可以在进入和退出时维护。递归写法短，但工程上要认识深度风险，尤其是网格很大或树退化成链时。

拓扑排序是有向图依赖关系的核心工具。入度为零的节点表示当前没有未满足依赖，可以安全输出；输出它以后，只会减少它指向的后继入度。若最终输出节点数少于总数，剩余节点一定在环中或被环依赖。边方向必须讲清楚：课程表中，先修课指向后续课程，还是课程指向先修课，决定输出顺序是否正确。

Dijkstra 适用于非负边权。它维护当前已知最短距离，每次从堆中取出距离最小的未过期状态。非负边保证从这个状态再走出去不会产生一条更短路径回头修改它。C++ 的 `priority_queue` 不支持 `decrease-key`，所以常用做法是更新距离时压入新状态，弹出时若距离不是当前 `distance` 数组中的值，就说明它过期，直接跳过。

有些题看起来不像普通最短路，但仍满足 Dijkstra 的单调扩展。最小体力路径中，路径代价不是边权和，而是路径上最大高度差；扩展一条边后，新代价是当前代价和边代价的最大值。这个值不会随着路径延长而降低，因此仍可用堆按当前最小代价扩展。另一种解法是二分体力阈值，再用 BFS 检查是否可达，因为阈值越大，可走边越多。

状态压缩 BFS 要特别小心 `visited` 的维度。位置相同但钥匙集合不同，未来能力不同；位置相同但剩余障碍消除次数不同，未来能力也不同。此时 `visited` 不能只按位置记录，而要按照位置加资源状态记录。很多看似超时或错误的 BFS，根本原因是把不同状态错误合并，导致要么漏解，要么重复搜索。

实验建议：把腐烂橘子写成每分钟全图扫描和多源 BFS 两版，比较重复扫描次数。把网络延迟时间写成普通 BFS 和 Dijkstra 两版，用不同权重反例证明 BFS 错误。再把带钥匙迷宫的 `visited` 从二维改成三维，观察二维 `visited` 如何错误剪掉合法路径。

本节复盘

阅读本节后，请回到相邻章节的例题，把暴力瓶颈、优化依据、状态不变量、C++ 容器成本和边界测试重新写一遍。能把同一思想迁移到变体题，才算真正掌握。

回溯、枚举和搜索空间剪枝

回溯题表面上是在“试所有可能”，本质上是在一棵决策树上搜索。每一层做一个选择，每条从根到叶子的路径代表一个候选答案。真正要学会的不是某个递归模板，而是四个问题：当前层决定什么，已经选择了什么，下一步还能选什么，什么时候可以停止或剪枝。只要这四件事说不清，代码再短也只是碰巧通过样例。

暴力枚举和回溯的边界并不绝对。最朴素的暴力会把所有组合全部生成出来，再逐个检查是否合法；回溯则把“合法性判断”提前到生成过程中。一旦发现当前前缀已经不可能形成答案，就不再向下扩展。这个动作叫剪枝。剪枝不是为了让代码看起来高级，而是减少决策树里根本不需要访问的分支。

C++ 面试题常用递归表达回溯，因为调用栈天然保存了“走到第几层、下一步尝试哪个分支、回退后恢复什么状态”。但递归深度不受控时会带来工程风险。本章讲递归心智模型，代码尽量把栈帧显式写出来，让搜索过程、状态恢复和边界条件更可见。这样写会比递归略长，但它能训练你看清楚回溯真正维护的状态。

搜索树、约束和输出规模

回溯题要先画搜索树。树的第几层代表什么，取决于题目对象。全排列中，第 `depth` 层决定排列的第 `depth` 个位置；子集中，第 `index` 层决定当前元素选或不选；组合题中，一层选择下一个加入组合的候选；切分题中，一层选择下一个切分终点；网格路径中，一层选择下一步方向。层含义不同，代码里的 `start`、`used`、`index` 和访问标记就不同，不能统一套一个函数形状。

路径状态表示从根到当前节点已经做出的选择。它可以是一个数组、一段字符串、若干个切分片段、棋盘上已经放置的皇后位置，或者一个网格路径访问集合。候选集合表示当前层还能选什么。排列题的候选是所有未使用下标；组合题的候选是从 `start` 往后的下标；括号生成的候选是左括号或右括号，但受剩余数量限制；N 皇后的候选是当前行的各列，但受列和两条对角线限制。把路径和候选集合分清，回溯代码才有明确职责。

剪枝必须是必要条件或支配关系，不能是直觉。组合总和中，候选全为正数且已排序，当前值大于剩余目标时，后续更大值也不能使用，这是合法剪枝；括号生成中，任意前缀右括号数不能超过左括号数，否则后面无法修复；复原 IP 地址中，剩余字符数量必须落在剩余段数到三倍剩余段数之间；N 皇后中，同列或同对角线冲突一定非法。没有证明的剪枝会制造隐藏漏解，尤其是在输入有负数、重复值或目标条件变化时。

去重也要分层次。路径内去重解决“同一个元素不能在一条路径里重复使用”，常用 `used` 或访问标记；同层去重解决“相同值作为本层不同分支会生成重复答案”，常用排序后跳过相邻重复值。全排列 II 同时有路径内去重和同层去重；组合总和 II 通过 `start` 保证下标不重复，再通过同层跳过重复值保证答案不重复；子集 II 的去重只在同一层发生。把这些条件混在一起，会出现要么漏掉合法重复使用，要么产生重复答案。

输出规模决定复杂度下限。全排列有 $n!$ 个答案，子集有 2^n 个答案，括号生成有卡特兰数个答案，N 皇后答案数量也随 n 快速增长。如果题目要求输出所有方案，算法时间不可能低于输出总大小。复杂度分析要把“搜索访问的状态数”和“复制答案的成本”分开：生成一个长度为 n 的排列，光复制路径就是 $O(n)$ 。这不是实现低效，而是输出本身需要这么多信息。

递归和迭代只是保存栈帧的两种方式。递归函数参数通常包括层号、路径、候选起点、剩余目标和访问标记；显式栈版本则把这些字段放进结构体。递归写法短，适合表达；显式栈写法长，但能避免调用栈风险，也更容易看清状态复制成本。本书代码偏向显式栈，并不否认递归思维；恰恰相反，只有先理解递归栈帧保存了什么，才能把它可靠地改写成迭代。

回溯题的建模清单

先写搜索树层含义，再写路径状态和候选集合；然后写终止条件，说明何时把路径加入答案；接着写剪枝条件，并证明被剪掉的分支不可能产生答案；最后写去重条件，区分路径内去重和同层去重。若答案数量本身是指数级，复杂度表达必须包含输出规模。

全排列是回溯的第一道标准题。题目给定互不相同的数字，要求返回所有排列。暴力想法是枚举长度为 n 的所有序列，再检查是否每个数字恰好使用一次。若把每个位置都看成有 n 种选择，总分支数是 n^n ，绝大多数序列会因为重复使用元素而被丢弃。优化方法是在生成过程中维护 `used` 数组，保证同一个下标不会被选两次。这样搜索树只保留真正的排列分支，叶子数变成 $n!$ 。

下面的实现用显式栈模拟递归。每个栈帧保存当前层接下来要尝试的候选下标 `next_index`。进入下一层时，把选择加入 `path` 并标记 `used`；一层候选全部尝试完时，弹出栈帧，并撤销上一层做出的选择。

Listing 8.1 LeetCode 46 全排列：显式栈模拟回溯

```

1 std::vector<std::vector<int>> permute(const std::vector<int>& nums)
2 {
3     struct Frame {
4         int next_index = 0;
5     };
6
7     const int n = static_cast<int>(nums.size());
8     std::vector<std::vector<int>> answer;
9     std::vector<int> path;
10    std::vector<char> used(n, false);
11    std::vector<Frame> stack{{0}};
12
13    while (!stack.empty()) {
14        if (static_cast<int>(path.size()) == n) {
15            answer.push_back(path);
16            stack.pop_back();
17            if (!path.empty()) {
18                const int last_value = path.back();
19                path.pop_back();
20                const auto found = std::find(nums.begin(), nums.end(), last_value);
21                used[static_cast<int>(found - nums.begin())] = false;
22            }
23            continue;
24        }
25
26        Frame& frame = stack.back();
27        while (frame.next_index < n && used[frame.next_index]) {
28            ++frame.next_index;
29        }
30        if (frame.next_index == n) {
31            stack.pop_back();
32            if (!path.empty()) {
33                const int last_value = path.back();
34                path.pop_back();
35                const auto found = std::find(nums.begin(), nums.end(), last_value);
36                used[static_cast<int>(found - nums.begin())] = false;
37            }
38            continue;

```

```

39     }
40
41     const int chosen_index = frame.next_index;
42     ++frame.next_index;
43     used[chosen_index] = true;
44     path.push_back(nums[chosen_index]);
45     stack.push_back(Frame{0});
46 }
47
48 return answer;
49 }

```

这段代码为了突出栈帧，撤销选择时用值反查下标；在生产代码里可以把“本层选择的下标”也放进下一层栈帧，避免反查。它的时间复杂度必须按输出规模理解：一共有 $n!$ 个排列，每个排列复制长度为 n 的路径，所以输出成本是 $O(n \cdot n!)$ 。额外空间是路径、标记和栈，都是 $O(n)$ ，不计答案。复盘这题时要能说清楚：`used` 是路径合法性的约束，`path.size() == n` 是叶子条件，回退必须撤销最后一次选择。

如果输入含重复数字，全排列 II 的难点就变成去重。很多人会先生成所有排列，再放进 `set` 去重；这当然能做，但浪费大量重复分支。更好的方法是先排序，然后在同一层跳过“相同数值且前一个相同值还没有被本路径使用”的候选。这个条件的含义是：同一层里，相同值只允许最左边那个代表这一类选择，避免生成形如交换两个相同值位置的重复排列。

Listing 8.2 LeetCode 47 全排列 II：排序后按层去重

```

1 std::vector<std::vector<int>> permute_unique(std::vector<int> nums)
2 {
3     std::sort(nums.begin(), nums.end());
4
5     const int n = static_cast<int>(nums.size());
6     std::vector<std::vector<int>> answer;
7     std::vector<int> path;
8     std::vector<char> used(n, false);
9
10    struct State {
11        std::vector<int> path;
12        std::vector<char> used;
13    };
14
15    std::vector<State> stack{{path, used}};
16    while (!stack.empty()) {
17        State state = std::move(stack.back());
18        stack.pop_back();
19
20        if (static_cast<int>(state.path.size()) == n) {
21            answer.push_back(std::move(state.path));
22            continue;
23        }
24
25        for (int i = n - 1; i >= 0; --i) {
26            if (state.used[i]) {
27                continue;
28            }
29            if (i > 0 && nums[i] == nums[i - 1] && !state.used[i - 1]) {

```

```

30         continue;
31     }
32
33     State next = state;
34     next.used[i] = true;
35     next.path.push_back(nums[i]);
36     stack.push_back(std::move(next));
37 }
38 }
39
40 return answer;
41 }

```

这个版本为了让去重条件清楚，直接把状态复制进栈，适合教材解释；高性能版本会用一份路径和标记做原地回退。去重条件最容易写反。排序后，相同数字挨在一起，`!state.used[i - 1]` 表示前一个相同数字在当前层还没有被拿来作为代表，此时选择后一个会制造重复分支，所以跳过。时间复杂度仍然与唯一排列数量成正比，最坏不超过 $O(n \cdot n!)$ 。

子集题的决策树更简单。每个元素只有两种选择：选或者不选。暴力枚举可以用二进制掩码，从 0 到 $2^n - 1$ ，第 i 位表示是否选择第 i 个元素。回溯写法则是沿着数组位置向后决定。两种方法本质相同，都是遍历大小为 2^n 的搜索空间。

Listing 8.3 LeetCode 78 子集：迭代扩展已有答案

```

1 std::vector<std::vector<int>> subsets(const std::vector<int>& nums)
2 {
3     std::vector<std::vector<int>> answer{{}};
4
5     for (const int x : nums) {
6         const int old_size = static_cast<int>(answer.size());
7         for (int i = 0; i < old_size; ++i) {
8             std::vector<int> next = answer[i];
9             next.push_back(x);
10            answer.push_back(std::move(next));
11        }
12    }
13
14    return answer;
15 }

```

这段代码的不变量是：处理完前 i 个元素后，`answer` 包含这些元素的全部子集。加入新元素 x 时，旧子集保持“不选 x ”的版本；每个旧子集复制一份并加上 x ，得到“选 x ”的版本。这个算法没有剪枝，因为所有子集都是合法答案。时间复杂度 $O(n2^n)$ ，空间复杂度同样由输出规模主导。

组合总和比子集多了一个目标约束。题目给定候选数组和目标值，要求找出所有和为目标值的组合，同一个数字可以重复使用。暴力思路是不断选数字，直到长度达到某个上限再检查和；问题是上限不好定，而且大量分支会在总和已经超过目标后继续扩展。优化来自两个事实：候选数字都是正数；排序后，如果当前候选已经大于剩余目标，后面的更大候选也不可能使用。

Listing 8.4 LeetCode 39 组合总和：显式状态保存起点和剩余目标

```

1 std::vector<std::vector<int>> combination_sum(std::vector<int> candidates,
2                                             int target)
3 {
4     std::sort(candidates.begin(), candidates.end());
5

```

```

6     struct State {
7         int start = 0;
8         int remaining = 0;
9         std::vector<int> path;
10    };
11
12    std::vector<std::vector<int>> answer;
13    std::vector<State> stack{{0, target, {}}};
14
15    while (!stack.empty()) {
16        State state = std::move(stack.back());
17        stack.pop_back();
18
19        if (state.remaining == 0) {
20            answer.push_back(std::move(state.path));
21            continue;
22        }
23
24        for (int i = static_cast<int>(candidates.size()) - 1;
25             i >= state.start;
26             --i) {
27            if (candidates[i] > state.remaining) {
28                continue;
29            }
30            State next = state;
31            next.path.push_back(candidates[i]);
32            next.remaining -= candidates[i];
33            next.start = i;
34            stack.push_back(std::move(next));
35        }
36    }
37
38    return answer;
39 }

```

`start` 的语义很重要：组合不关心顺序，`[2,3,2]` 和 `[2,2,3]` 是同一个答案，因此后续只能从当前下标及其右侧继续选，避免同一组合以不同顺序重复出现。因为允许重复使用当前数字，下一层的 `start` 仍然是 `i`；如果题目变成每个数字只能用一次，就应设为 `i + 1`。这就是组合类题最核心的变体开关。复杂度取决于目标值和候选分布，不能简单写成多项式；面试里应说明它是指数级搜索，但剪枝能显著减少无效分支。

括号生成的关键不是枚举所有长度为 $2n$ 的括号串再检查。那样有 2^{2n} 个候选，大多数不合法。合法前缀必须满足两个计数约束：左括号数量不能超过 `n`；任何前缀中右括号数量不能超过左括号数量。把这两个约束提前放进生成过程，就不会产生明显非法的前缀。

Listing 8.5 LeetCode 22 括号生成：前缀合法性剪枝

```

1 std::vector<std::string> generate_parenthesis(int n)
2 {
3     struct State {
4         std::string text;
5         int left = 0;
6         int right = 0;
7     };

```

```

8
9     std::vector<std::string> answer;
10    std::vector<State> stack{{"", 0, 0}};
11
12    while (!stack.empty()) {
13        State state = std::move(stack.back());
14        stack.pop_back();
15
16        if (static_cast<int>(state.text.size()) == 2 * n) {
17            answer.push_back(std::move(state.text));
18            continue;
19        }
20
21        if (state.right < state.left) {
22            State next = state;
23            next.text.push_back(')');
24            ++next.right;
25            stack.push_back(std::move(next));
26        }
27        if (state.left < n) {
28            State next = state;
29            next.text.push_back('(');
30            ++next.left;
31            stack.push_back(std::move(next));
32        }
33    }
34
35    return answer;
36 }

```

这题的正确性可以用前缀不变量解释：任何时刻 $left \leq n$ 且 $right \leq left$ 。如果 $right > left$ ，说明某个右括号没有左括号匹配，后面再加字符也无法修复这个非法前缀；如果 $left > n$ ，也已经超过题目允许。输出数量是第 n 个卡特兰数，算法复杂度至少与答案数量成正比。易错点是终止条件：必须长度达到 $2n$ 才能加入答案，只看 $left == right$ 会把短前缀误加进去。

单词搜索把回溯放进网格图。节点是格子，边是上下左右相邻，路径不能重复使用同一个格子。暴力想法是从每个格子出发尝试所有方向，最后检查路径字符串；优化点是边走边比较目标单词的当前位置，一旦字符不匹配立即停止。访问标记也必须属于当前路径，而不是全局永久标记，因为同一个格子可以出现在不同起点或不同路径中。

Listing 8.6 LeetCode 79 单词搜索：显式栈保存路径访问状态

```

1 bool exist(std::vector<std::vector<char>>& board, const std::string& word)
2 {
3     if (word.empty()) {
4         return true;
5     }
6     if (board.empty() || board[0].empty()) {
7         return false;
8     }
9
10    constexpr std::array<std::pair<int, int>, 4> DIRECTIONS{
11        std::pair{1, 0}, std::pair{-1, 0},
12        std::pair{0, 1}, std::pair{0, -1}

```



```

13     };
14
15     const int rows = static_cast<int>(board.size());
16     const int cols = static_cast<int>(board[0].size());
17
18     struct State {
19         int row = 0;
20         int col = 0;
21         int index = 0;
22         std::vector<std::vector<char>> visited;
23     };
24
25     for (int row = 0; row < rows; ++row) {
26         for (int col = 0; col < cols; ++col) {
27             if (board[row][col] != word[0]) {
28                 continue;
29             }
30
31             std::vector<std::vector<char>> visited(rows, std::vector<char>(cols, ↵
↵ false));
32             visited[row][col] = true;
33             std::vector<State> stack{{row, col, 0, std::move(visited)}};
34
35             while (!stack.empty()) {
36                 State state = std::move(stack.back());
37                 stack.pop_back();
38                 if (state.index + 1 == static_cast<int>(word.size())) {
39                     return true;
40                 }
41
42                 for (const auto [dr, dc] : DIRECTIONS) {
43                     const int next_row = state.row + dr;
44                     const int next_col = state.col + dc;
45                     const int next_index = state.index + 1;
46                     const bool out_of_bounds =
47                         next_row < 0 || next_row >= rows ||
48                         next_col < 0 || next_col >= cols;
49                     if (out_of_bounds || state.visited[next_row][next_col] ||
50                         board[next_row][next_col] != word[next_index]) {
51                         continue;
52                     }
53
54                     State next = state;
55                     next.row = next_row;
56                     next.col = next_col;
57                     next.index = next_index;
58                     next.visited[next_row][next_col] = true;
59                     stack.push_back(std::move(next));
60                 }
61             }
62         }
63     }
64

```

```

65     return false;
66 }

```

这个教材版把 `visited` 放进状态里，语义最清楚，但会复制较多布尔矩阵；原地回退性能更好。本题最坏复杂度接近 $O(mn \cdot 3^L)$ ：第一步最多从每个格子出发，之后每一步通常不能回到上一个格子，所以分支近似 3， L 是单词长度。常见错误是把访问标记设成全局，导致某个格子被一条失败路径占用后，其他合法路径无法使用它。

N 皇后是回溯剪枝的代表。暴力方法在 $n \times n$ 棋盘上选 n 个格子，再检查行、列、对角线是否冲突，搜索空间巨大。优化方式是按行放皇后：第 `row` 层只决定第 `row` 行皇后放在哪一列。这样行冲突天然消失，只需要维护列、主对角线和副对角线是否被占用。

Listing 8.7 LeetCode 51 N 皇后：按行放置并维护三类占用

```

1  std::vector<std::vector<std::string>> solve_n_queens(int n)
2  {
3      struct State {
4          int row = 0;
5          std::vector<int> queens;
6          std::vector<char> cols;
7          std::vector<char> diag1;
8          std::vector<char> diag2;
9      };
10
11     std::vector<std::vector<std::string>> answer;
12     std::vector<State> stack{{0,
13         {},
14         std::vector<char>(n, false),
15         std::vector<char>(2 * n - 1, false),
16         std::vector<char>(2 * n - 1, false)}};
17
18     while (!stack.empty()) {
19         State state = std::move(stack.back());
20         stack.pop_back();
21
22         if (state.row == n) {
23             std::vector<std::string> board(n, std::string(n, '.'));
24             for (int row = 0; row < n; ++row) {
25                 board[row][state.queens[row]] = 'Q';
26             }
27             answer.push_back(std::move(board));
28             continue;
29         }
30
31         for (int col = n - 1; col >= 0; --col) {
32             const int main_diag = state.row - col + n - 1;
33             const int anti_diag = state.row + col;
34             if (state.cols[col] || state.diag1[main_diag] ||
35                 state.diag2[anti_diag]) {
36                 continue;
37             }
38
39             State next = state;
40             next.queens.push_back(col);

```

```

41         next.cols[col] = true;
42         next.diag1[main_diag] = true;
43         next.diag2[anti_diag] = true;
44         ++next.row;
45         stack.push_back(std::move(next));
46     }
47 }
48
49 return answer;
50 }

```

主对角线上的格子满足 $\text{row} - \text{col}$ 相同；为了把负数下标变成数组下标，代码加上 $n - 1$ 。副对角线满足 $\text{row} + \text{col}$ 相同，范围天然是 0 到 $2n - 2$ 。这题的面试重点不是把棋盘画出来，而是把冲突检测从扫描整张棋盘优化成三个数组的常数查询。复杂度仍然是指数级，但剪枝把不可行分支尽早挡在下一层之外。

回溯去重可以分成两类：路径内去重和同层去重。路径内去重解决“同一个元素不能被重复使用”，典型工具是 `used` 数组；同层去重解决“相同值作为同一层选择会生成重复答案”，典型工具是排序后跳过相邻重复值。全排列 II 同时需要这两类去重，组合总和 II 主要需要同层去重。只要把这两个概念混在一起，条件就很容易写反。

组合总和 II 中，每个候选数字只能使用一次，输入可能有重复值，答案组合不能重复。与组合总和 I 相比，下一层起点必须是 $i + 1$ ，因为当前下标不能再次使用；与全排列 II 相比，我们不需要 `used` 数组，因为 `start` 已经保证每个下标最多被使用一次。同层去重条件是：在同一个循环层里，如果当前值和前一个值相同，并且前一个值已经作为本层候选被跳过或处理过，那么当前值也不能再作为新的分支开头。

Listing 8.8 LeetCode 40 组合总和 II：每个下标最多使用一次，同层跳过重复值

```

1 std::vector<std::vector<int>> combination_sum2(std::vector<int> candidates,
2                                              int target)
3 {
4     std::sort(candidates.begin(), candidates.end());
5
6     struct State {
7         int start = 0;
8         int remaining = 0;
9         std::vector<int> path;
10    };
11
12    std::vector<std::vector<int>> answer;
13    std::vector<State> stack{{0, target, {}}};
14
15    while (!stack.empty()) {
16        State state = std::move(stack.back());
17        stack.pop_back();
18
19        if (state.remaining == 0) {
20            answer.push_back(std::move(state.path));
21            continue;
22        }
23
24        for (int i = static_cast<int>(candidates.size()) - 1;
25             i >= state.start;
26             --i) {

```

```

27         if (i + 1 < static_cast<int>(candidates.size()) &&
28             candidates[i] == candidates[i + 1] &&
29             i + 1 >= state.start) {
30             continue;
31         }
32         if (candidates[i] > state.remaining) {
33             continue;
34         }
35
36         State next = state;
37         next.path.push_back(candidates[i]);
38         next.remaining -= candidates[i];
39         next.start = i + 1;
40         stack.push_back(std::move(next));
41     }
42 }
43
44 return answer;
45 }

```

这段显式栈代码为了保持输出方向，用倒序枚举候选；倒序时同层去重要和相邻的后一个元素比较。递归版本通常正序枚举，条件写成 `i > start && candidates[i] == candidates[i - 1]`。两个条件本质相同：同一个递归层里，相同值只能由一个下标代表。面试中如果不确定倒序条件，先写递归式正序伪代码说明去重，再实现自己更熟悉的版本。最重要的是讲清楚：同层重复会制造相同组合，路径重复则是同一个下标重复使用，这两个问题不能用同一个条件糊过去。

字符串切分题也属于回溯。分割回文串要求把字符串切成若干段，每一段都是回文。暴力做法是枚举所有切分方式，再逐段检查是否回文；如果每次检查都扫描子串，会产生大量重复判断。优化方法是先用 DP 预处理 `is_palindrome[left][right]`，表示闭区间子串是否回文。之后回溯只需要常数时间判断某一段是否合法。

Listing 8.9 LeetCode 131 分割回文串：预处理回文表后搜索切分点

```

1 std::vector<std::vector<std::string>> partition_palindrome(const std::string& s)
2 {
3     const int n = static_cast<int>(s.size());
4     std::vector<std::vector<char>> is_palindrome(n, std::vector<char>(n, false));
5     for (int length = 1; length <= n; ++length) {
6         for (int left = 0; left + length - 1 < n; ++left) {
7             const int right = left + length - 1;
8             if (s[left] != s[right]) {
9                 continue;
10            }
11            is_palindrome[left][right] =
12                length <= 2 || is_palindrome[left + 1][right - 1];
13        }
14    }
15
16    struct State {
17        int start = 0;
18        std::vector<std::string> parts;
19    };
20
21    std::vector<std::vector<std::string>> answer;

```

```

22     std::vector<State> stack{{0, {}}};
23     while (!stack.empty()) {
24         State state = std::move(stack.back());
25         stack.pop_back();
26         if (state.start == n) {
27             answer.push_back(std::move(state.parts));
28             continue;
29         }
30
31         for (int end = n - 1; end >= state.start; --end) {
32             if (!is_palindrome[state.start][end]) {
33                 continue;
34             }
35             State next = state;
36             next.parts.push_back(s.substr(state.start, end - state.start + 1));
37             next.start = end + 1;
38             stack.push_back(std::move(next));
39         }
40     }
41     return answer;
42 }

```

这题的决策树层数不是固定的 n ，而是取决于切了多少段。每个状态的 `start` 表示下一个待切分位置，选择是从 `start` 开始切到哪个 `end`。预处理回文表的转移也很经典：两端字符相等，且内部子串是回文，整个子串才是回文；长度为 1 或 2 时内部为空或单字符，可以直接成立。搜索复杂度仍然可能指数级，因为合法切分数量本身可能很多，但预处理避免了在搜索树中反复扫描同一段字符串。

复原 IP 地址是切分题的约束版。字符串要切成四段，每段长度 1 到 3，数值 0 到 255，且不能有前导零。暴力枚举三个切分点就能做，因为段数固定；也可以用回溯统一表达。剪枝点很明确：剩余字符太少或太多时停止；当前段长度超过 3 停止；以 0 开头的多位段非法；数值超过 255 非法。

Listing 8.10 LeetCode 93 复原 IP 地址：段数和剩余长度剪枝

```

1  std::vector<std::string> restore_ip_addresses(const std::string& s)
2  {
3      constexpr int SEGMENT_COUNT = 4;
4      constexpr int MAX_SEGMENT_VALUE = 255;
5      std::vector<std::string> answer;
6
7      struct State {
8          int index = 0;
9          std::vector<std::string> segments;
10     };
11
12     std::vector<State> stack{{0, {}}};
13     while (!stack.empty()) {
14         State state = std::move(stack.back());
15         stack.pop_back();
16
17         const int used_segments = static_cast<int>(state.segments.size());
18         const int remaining_segments = SEGMENT_COUNT - used_segments;
19         const int remaining_chars = static_cast<int>(s.size()) - state.index;
20         if (remaining_chars < remaining_segments ||

```



```

21         remaining_chars > remaining_segments * 3) {
22             continue;
23         }
24
25         if (used_segments == SEGMENT_COUNT) {
26             if (state.index == static_cast<int>(s.size())) {
27                 answer.push_back(state.segments[0] + "." + state.segments[1] +
28                                 "." + state.segments[2] + "." +
29                                 state.segments[3]);
30             }
31             continue;
32         }
33
34         int value = 0;
35         for (int length = 1; length <= 3; ++length) {
36             if (state.index + length > static_cast<int>(s.size())) {
37                 break;
38             }
39             if (length > 1 && s[state.index] == '0') {
40                 break;
41             }
42             value = value * 10 + (s[state.index + length - 1] - '0');
43             if (value > MAX_SEGMENT_VALUE) {
44                 break;
45             }
46
47             State next = state;
48             next.segments.push_back(s.substr(state.index, length));
49             next.index += length;
50             stack.push_back(std::move(next));
51         }
52     }
53
54     return answer;
55 }

```

这题适合训练“剩余量剪枝”。如果剩余字符数少于剩余段数，每段至少一个字符都不够；如果剩余字符数大于剩余段数乘 3，每段最多三个字符也放不下。这种剪枝不依赖数据运气，是严格必要条件，因此不会漏解。很多组合题都可以做类似估算：剩余元素数量是否足够，剩余目标是否可能达到，剩余步数是否还能完成。

回溯题也可以用位掩码表示集合。子集枚举、访问状态、N 皇后列占用都可以压成整数位。位掩码的优势是状态复制便宜、哈希和比较方便；缺点是可读性下降，并且 n 不能太大。对于 $n \leq 20$ 的集合问题，位掩码常常是从回溯过渡到状态压缩 DP 的桥梁。

Listing 8.11 位掩码枚举子集：每一位表示选或不选

```

1 std::vector<std::vector<int>> subsets_by_mask(const std::vector<int>& nums)
2 {
3     const int n = static_cast<int>(nums.size());
4     const int state_count = 1 << n;
5     std::vector<std::vector<int>> answer;
6     answer.reserve(state_count);
7

```

```

8   for (int mask = 0; mask < state_count; ++mask) {
9       std::vector<int> subset;
10      for (int i = 0; i < n; ++i) {
11          if ((mask & (1 << i)) != 0) {
12              subset.push_back(nums[i]);
13          }
14      }
15      answer.push_back(std::move(subset));
16  }
17
18  return answer;
19 }

```

位掩码写法的复杂度是 $O(n2^n)$ ，因为每个集合要检查 n 位。它没有递归栈，也不需要回退，适合子集这种所有状态都合法的题；如果有复杂剪枝，例如组合总和或单词搜索，回溯更自然。选择表达方式时要看题目结构，不要为了使用位运算牺牲清晰性。

剪枝必须证明不会漏解。常见剪枝有四类。第一，合法性剪枝：括号前缀右括号不能超过左括号。第二，容量剪枝：组合总和中当前和超过目标立即停止。第三，剩余量剪枝：IP 地址剩余字符数量必须落在可行范围。第四，排序剪枝：候选已排序时，当前值超过剩余目标，后续更大值也不可能使用。没有证明的剪枝很危险，比如“看起来这个分支不太可能有答案”不能作为正确算法的一部分。

回溯实验：第一，把组合总和 II 的同层去重条件删除，观察重复答案如何出现。第二，把复原 IP 地址的剩余量剪枝删除，统计访问状态数量变化。第三，把分割回文串的回文预处理去掉，改成每次扫描判断，比较长字符串上的耗时。第四，用位掩码和迭代扩展两种方式生成子集，比较输出顺序和代码复杂度。

回溯题复盘模板

回溯题不要先背代码。先画出决策树的一层，写清本层选择、路径状态、可选集合、终止条件和剪枝条件。组合题重点看 **start** 是否允许重复使用当前元素；排列题重点看 **used** 和同层去重；网格题重点看访问标记是路径级还是全局级；括号和棋盘题重点把非法前缀提前剪掉。最后再问一句：答案数量本身有多大？如果答案就是指数级，复杂度不可能被写成线性或多项式。

动态规划：状态、转移和重复子问题

动态规划不是“看到最值就开数组”。它解决的是有重叠子问题、且大问题能由小问题稳定推出的问题。暴力搜索之所以慢，是因为同一个子问题被不同路径反复计算；动态规划的优化，就是给子问题命名、保存结果，并按照依赖顺序把结果推出来。学 DP 最重要的五件事是：状态表示什么，答案在哪里，初始状态是什么，转移从哪里来，遍历顺序为什么满足依赖。

从暴力搜索树到状态图

DP 的起点是暴力搜索，而不是公式。先把暴力搜索树画出来：每一层做什么选择，选择后哪些信息会影响未来。如果两条不同路径到达某个局面后，未来可选动作和最优结果完全相同，那么这个局面就可以被抽象成同一个状态。爬楼梯中，到达第 i 阶以后，之前怎么走来的不再影响之后的走法；零钱兑换中，剩余金额相同，之后需要解决的问题相同；背包中，只知道容量还不够，还要知道已经处理到第几个物品，因为前面的物品不能再选。状态定义就是在回答“哪些历史必须保留，哪些历史可以丢掉”。

状态必须足够，也必须克制。不足够的状态会把不同未来能力混在一起，例如带冷冻期股票题只保存当前利润，不保存是否持股或是否刚卖出，就无法判断今天能不能买；障碍消除网格只保存位置，不保存剩余消除次数，就会错误剪枝。过度的状态则会导致维度爆炸，例如本来只需要前缀长度，却把完整路径也放进状态。好的状态定义应该让转移只依赖状态本身，不再偷偷依赖未记录的历史。

转移来自最后一步选择。线性 DP 常问当前位置的最后一步从哪里来；网格 DP 问当前格子从上方还是左方来；背包 DP 问当前物品选或不选；区间 DP 问最后合并点或最后操作点在哪里；树形 DP 问子树向父节点提供什么贡献；状态机 DP 问动作如何让状态之间转移。所谓公式，其实是把最后一步选择翻译成代码。只背 $dp[i] = \max(\dots)$ 没有意义，必须说出每个候选项对应哪一种选择。

初始化要服从状态语义。 $dp[0] = 1$ 在计数题里通常表示空方案有一种； $dp[0] = 0$ 在最少代价题里表示空目标不需要成本；不可达状态要用无穷或特殊值，但特殊值不能参与错误转移。编辑距离中，空串到长度 j 的串需要 j 次插入；背包中，容量为零的可达性为真；股票状态中，第一天持股收益是负价格。初始化不是例行填表，而是最小子问题的答案。

遍历顺序必须满足依赖。二维 LCS 按行列递增，因为 $dp[i][j]$ 依赖上方、左方和左上；区间 DP 按区间长度递增，因为大区间依赖小区间；0-1 背包一维压缩时容量倒序，因为当前物品不能在同一轮被重复使用；完全背包容量正序，因为当前物品允许重复使用。空间优化后，遍历顺序更重要，因为同一个数组位置可能同时代表上一阶段和当前阶段。解释不了旧值来自哪里，就不要急着压缩空间。

DP 的复杂度常常是状态数乘以转移枚举数。状态数由维度决定，转移枚举数由每个状态需要尝试多少选择决定。LCS 有 mn 个状态，每个状态常数转移；戳气球有 n^2 个区间状态，每个状态枚举一个中点，所以是 $O(n^3)$ ；背包有 $n \cdot \text{capacity}$ 个逻辑状态，这是伪多项式复杂度，依赖容量数值而不是输入长度。面试中说复杂度时，要按真实状态维度说明，不要只说“开了一个数组”。

DP 题的推导清单

先写暴力搜索的选择，再找哪些局面会重复；定义状态时说明保留了哪些历史、丢掉了哪些历史；写出答案位置和初始化含义；从最后一步选择推导转移；最后证明遍历顺序满足依赖。如果要空间压缩，必须额外说明被覆盖的值是否还会被后续转移使用。

爬楼梯是最小的 DP 模型。题目问到达第 n 阶有多少种方法，每次可以爬 1 或 2 阶。暴力递归会写成 $\text{ways}(n) = \text{ways}(n - 1) + \text{ways}(n - 2)$ ，但 $\text{ways}(n - 2)$ 、 $\text{ways}(n - 3)$ 等子问题会被反复计算，形成指数级时间。优化方法是从小到大保存每个阶数的答案。

Listing 9.1 LeetCode 70 爬楼梯：滚动变量保存最近两个状态

```

1 int climb_stairs(int n)
2 {
3     if (n <= 2) {
4         return n;
5     }
6
7     int previous_two = 1;
8     int previous_one = 2;
9     for (int step = 3; step <= n; ++step) {
10         const int current = previous_one + previous_two;
11         previous_two = previous_one;
12         previous_one = current;
13     }
14
15     return previous_one;
16 }

```

这里状态可以写成 $dp[i]$ ：到达第 i 阶的方法数。最后一步要么从 $i - 1$ 走 1 阶，要么从 $i - 2$ 走 2 阶，因此转移是 $dp[i] = dp[i - 1] + dp[i - 2]$ 。由于每个状态只依赖前两个状态，可以把数组压成两个变量。时间复杂度 $O(n)$ ，空间复杂度 $O(1)$ 。这题的价值不在难度，而在于展示 DP 的完整骨架。

打家劫舍把“相邻不能同时选”的约束放进状态。暴力方法是对每间房决定偷或不偷，枚举 2^n 个子集，再过滤相邻冲突。更好的建模是让 $dp[i]$ 表示考虑前 i 间房能偷到的最大金额。第 i 间房只有两种结局：不偷它，答案是 $dp[i - 1]$ ；偷它，就不能偷第 $i - 1$ 间，答案是 $dp[i - 2] + \text{nums}[i - 1]$ 。

Listing 9.2 LeetCode 198 打家劫舍：选与不选的状态转移

```

1 int rob(const std::vector<int>& nums)
2 {
3     int previous_two = 0;
4     int previous_one = 0;
5
6     for (const int money : nums) {
7         const int current = std::max(previous_one, previous_two + money);
8         previous_two = previous_one;
9         previous_one = current;
10    }
11
12    return previous_one;
13 }

```

这段代码把 $dp[i - 2]$ 和 $dp[i - 1]$ 压成两个变量。遍历到当前房屋时，`previous_one` 是不偷当前房屋时可继承的最优值，`previous_two + money` 是偷当前房屋时的最优值。正确性来自最优子结构：当前决策之后，剩余可用的历史范围只与前一间或前两间有关，不需要知道更早的具体选择。易错点是下标：如果用数组，建议定义 $dp[0] = 0$ ， $dp[1] = \text{nums}[0]$ ，这样第 i 个状态表示前 i 间房，避免把房屋下标和状态下标混在一起。

零钱兑换展示“最少数量”型 DP。题目给硬币面额和目标金额，要求凑成目标金额所需的最少硬币数。暴力搜索会尝试所有硬币序列，许多不同顺序会到达同一个剩余金额，例如先选 1 再选 2，与先选 2 再选 1，都可能进入同一个子问题。状态应定义为 $dp[\text{amount}]$ ：凑出金额 `amount` 的最少硬币数。

Listing 9.3 LeetCode 322 零钱兑换：从可达金额推出新金额

```

1 int coin_change(const std::vector<int>& coins, int amount)
2 {
3     constexpr int INF = 1'000'000'000;
4     std::vector<int> dp(amount + 1, INF);
5     dp[0] = 0;
6
7     for (int value = 1; value <= amount; ++value) {
8         for (const int coin : coins) {
9             if (coin > value || dp[value - coin] == INF) {
10                 continue;
11             }
12             dp[value] = std::min(dp[value], dp[value - coin] + 1);
13         }
14     }
15
16     return dp[amount] == INF ? -1 : dp[amount];
17 }

```

转移的含义是：如果最后一枚硬币是 `coin`，那么前面必须先凑出 `value - coin`，所以候选答案是 `dp[value - coin] + 1`。这里用一个足够大的 `INF` 表示不可达，而不是用 `-1` 参与最小值计算，因为 `-1 + 1` 会污染转移。时间复杂度 $O(\text{amount} \cdot c)$ ， c 是硬币种类数。完全背包类题还要区分“组合数”和“排列数”：本题只问最少硬币数，内外循环顺序不会影响最优值；如果问方案数量，循环顺序就会改变计数语义。

最长递增子序列有两种经典解法。第一种 DP 定义 `dp[i]` 为以 `nums[i]` 结尾的最长递增子序列长度。为了求它，需要看所有 `j < i` 且 `nums[j] < nums[i]` 的位置，转移为 `dp[i] = max(dp[j] + 1)`。这比枚举所有子序列的 2^n 好很多，但仍是 $O(n^2)$ 。

Listing 9.4 LeetCode 300 最长递增子序列：二次 DP

```

1 int length_of_lis_dp(const std::vector<int>& nums)
2 {
3     if (nums.empty()) {
4         return 0;
5     }
6
7     std::vector<int> dp(nums.size(), 1);
8     int answer = 1;
9
10    for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
11        for (int j = 0; j < i; ++j) {
12            if (nums[j] < nums[i]) {
13                dp[i] = std::max(dp[i], dp[j] + 1);
14            }
15        }
16        answer = std::max(answer, dp[i]);
17    }
18
19    return answer;
20 }

```

第二种优化来自“只关心长度对应的最小结尾”。维护数组 `tails`，其中 `tails[len - 1]` 表示长度为 `len` 的递增子序列中，最小可能结尾值。结尾越小，后面越容易接上更大的数。遍历新数

x 时，用二分找到第一个大于等于 x 的位置并替换；如果找不到，说明 x 可以扩展出更长序列。

Listing 9.5 LeetCode 300 最长递增子序列：贪心尾值加二分

```
1 int length_of_lis_binary_search(const std::vector<int>& nums)
2 {
3     std::vector<int> tails;
4
5     for (const int x : nums) {
6         auto position = std::lower_bound(tails.begin(), tails.end(), x);
7         if (position == tails.end()) {
8             tails.push_back(x);
9         } else {
10            *position = x;
11        }
12    }
13
14    return static_cast<int>(tails.size());
15 }
```

这个算法经常被误解成 `tails` 本身就是最终的子序列。它不一定是。它保存的是每个长度下最有潜力的最小结尾。替换操作不会减少已经找到的长度，只会让同长度的结尾更小，从而给未来元素更多机会。时间复杂度 $O(n \log n)$ ，空间复杂度 $O(n)$ 。如果面试官追问为什么能替换，要回答：相同长度下，更小结尾不会比更大结尾更差，因为它能接纳的后续元素集合只会更多。

单词拆分把字符串前缀变成状态。题目给字符串和词典，问字符串能否被切成若干词典单词。暴力搜索会尝试每个切分点，遇到相同后缀时重复判断。状态 `dp[i]` 表示前 i 个字符是否可以被合法拆分。若存在 $j < i$ ，使得 `dp[j]` 为真且子串 `s[j..i]` 在词典中，则 `dp[i]` 为真。

Listing 9.6 LeetCode 139 单词拆分：前缀可达状态

```
1 bool word_break(const std::string& s, const std::vector<std::string>& word_dict)
2 {
3     std::unordered_set<std::string> words(word_dict.begin(), word_dict.end());
4     std::vector<char> dp(s.size() + 1, false);
5     dp[0] = true;
6
7     for (std::size_t right = 1; right <= s.size(); ++right) {
8         for (std::size_t left = 0; left < right; ++left) {
9             if (!dp[left]) {
10                continue;
11            }
12            if (words.contains(s.substr(left, right - left))) {
13                dp[right] = true;
14                break;
15            }
16        }
17    }
18
19    return dp[s.size()];
20 }
```

这里 `dp[0] = true` 代表空前缀可拆分，是所有从 0 开始的单词能够成立的基础。代码直接使用 `substr`，写法清晰，但会复制子串；如果输入规模更大，可以限制最大单词长度，或者用字符串视图和自定义哈希、字典树来减少复制。复杂度通常写成 $O(n^3)$ 的保守形式，因为有两层切分枚举，

子串构造和哈希也与长度有关；若把子串查询视作均摊常数，则是 $O(n^2)$ 。面试中最好主动说明这个实现的复制成本。

最长公共子序列是二维 DP 的代表。子序列可以删除字符但不能改变相对顺序。暴力方法会枚举两个字符串的所有子序列再比较，数量指数级。状态 $dp[i][j]$ 表示 $text1$ 的前 i 个字符和 $text2$ 的前 j 个字符的最长公共子序列长度。

Listing 9.7 LeetCode 1143 最长公共子序列：二维前缀 DP

```
1 int longest_common_subsequence(const std::string& text1,
2                               const std::string& text2)
3 {
4     const int rows = static_cast<int>(text1.size());
5     const int cols = static_cast<int>(text2.size());
6     std::vector<std::vector<int>> dp(rows + 1, std::vector<int>(cols + 1, 0));
7
8     for (int i = 1; i <= rows; ++i) {
9         for (int j = 1; j <= cols; ++j) {
10             if (text1[i - 1] == text2[j - 1]) {
11                 dp[i][j] = dp[i - 1][j - 1] + 1;
12             } else {
13                 dp[i][j] = std::max(dp[i - 1][j], dp[i][j - 1]);
14             }
15         }
16     }
17
18     return dp[rows][cols];
19 }
```

为什么相等时来自左上角？因为当前两个字符可以配成公共子序列的最后一个字符，剩下问题变成两个前缀各去掉这个字符。为什么不等时取上方和左方最大值？因为两个末尾字符不能同时作为同一个结尾，至少要丢掉其中一个。多出来的第 0 行和第 0 列代表空串，避免了边界分支。时间复杂度 $O(mn)$ ，空间复杂度 $O(mn)$ ；如果只要长度，可以滚动数组压到 $O(n)$ 。

编辑距离比 LCS 更综合。题目允许插入、删除、替换，把一个单词变成另一个单词，求最少操作数。状态 $dp[i][j]$ 表示把 $word1$ 的前 i 个字符变成 $word2$ 的前 j 个字符的最少操作数。这个定义必须精确，因为三种操作都围绕“前缀末尾”展开。

Listing 9.8 LeetCode 72 编辑距离：插入、删除、替换三种转移

```
1 int min_distance(const std::string& word1, const std::string& word2)
2 {
3     const int rows = static_cast<int>(word1.size());
4     const int cols = static_cast<int>(word2.size());
5     std::vector<std::vector<int>> dp(rows + 1, std::vector<int>(cols + 1, 0));
6
7     for (int i = 0; i <= rows; ++i) {
8         dp[i][0] = i;
9     }
10    for (int j = 0; j <= cols; ++j) {
11        dp[0][j] = j;
12    }
13
14    for (int i = 1; i <= rows; ++i) {
15        for (int j = 1; j <= cols; ++j) {
16            if (word1[i - 1] == word2[j - 1]) {
```

```

17         dp[i][j] = dp[i - 1][j - 1];
18         continue;
19     }
20
21     const int delete_cost = dp[i - 1][j] + 1;
22     const int insert_cost = dp[i][j - 1] + 1;
23     const int replace_cost = dp[i - 1][j - 1] + 1;
24     dp[i][j] = std::min({delete_cost, insert_cost, replace_cost});
25 }
26 }
27
28 return dp[rows][cols];
29 }

```

初始化含义是：把长度为 i 的前缀变成空串，需要删除 i 次；把空串变成长度为 j 的前缀，需要插入 j 次。若末尾字符相同，不需要新增操作，直接继承左上角；若不同，删除对应上方，插入对应左方，替换对应左上角。时间和空间都是 $O(mn)$ 。易错点是从哪个字符串视角解释插入和删除：只要状态定义固定， $dp[i - 1][j] + 1$ 就是删掉 `word1` 的末尾， $dp[i][j - 1] + 1$ 就是在 `word1` 末尾插入 `word2[j - 1]`。

动态规划题经常可以做空间优化，但不要为了炫技先写滚动数组。二维表能展示依赖关系，滚动数组则要求你非常清楚哪些旧值还要用、哪些可以覆盖。面试时，先给出清晰状态和二维实现，再说明可压缩空间，通常比一上来写难读的一维代码更稳。

从暴力搜索到 DP，通常要经历三步。第一步写出递归含义，例如 `solve(i, cap)` 表示处理到第 i 个物品、剩余容量为 `cap` 时的最优值。第二步观察重复子问题：不同选择路径可能来到同一个 (i, cap) 。第三步把这个状态保存起来，或者按依赖顺序填表。动态规划不是凭空发明公式，而是把搜索树压缩成状态图。

0-1 背包是最重要的状态压缩训练。每个物品只能选一次，状态可以定义为 $dp[i][cap]$ ：只考虑前 i 个物品、容量为 `cap` 时的最大价值。第 i 个物品要么不选，继承 $dp[i-1][cap]$ ；要么选，在容量足够时得到 $dp[i-1][cap-weight] + value$ 。二维表最清楚，一维表最高频。

Listing 9.9 0-1 背包：二维状态到一维滚动

```

1 int knapsack_01(const std::vector<int>& weights,
2               const std::vector<int>& values,
3               int capacity)
4 {
5     std::vector<int> dp(capacity + 1, 0);
6
7     for (int item = 0; item < static_cast<int>(weights.size()); ++item) {
8         for (int cap = capacity; cap >= weights[item]; --cap) {
9             dp[cap] = std::max(dp[cap],
10                               dp[cap - weights[item]] + values[item]);
11         }
12     }
13
14     return dp[capacity];
15 }

```

一维数组必须倒序遍历容量。原因是 $dp[cap - weights[item]]$ 应该来自上一轮物品集合，而不是当前物品已经更新后的状态。如果正序遍历，当前物品会在同一轮被重复使用，语义就变成完全背包。这个遍历方向是背包题的核心，不是实现细节。分割等和子集、目标和都可以看成 0-1 背包变体。

完全背包中，每个物品可以使用无限次。零钱兑换 II 问组合数，状态 $dp[value]$ 表示凑出金

额 `value` 的组合数量。外层遍历硬币，内层正序遍历金额。正序是为了允许当前硬币重复使用；硬币在外层是为了每种组合只按一种硬币顺序被统计，避免把排列当组合。

Listing 9.10 LeetCode 518 零钱兑换 II：完全背包组合数

```
1 int change(int amount, const std::vector<int>& coins)
2 {
3     std::vector<int> dp(amount + 1, 0);
4     dp[0] = 1;
5
6     for (const int coin : coins) {
7         for (int value = coin; value <= amount; ++value) {
8             dp[value] += dp[value - coin];
9         }
10    }
11
12    return dp[amount];
13 }
```

如果题目问排列数，例如“有多少种有序序列凑成目标”，循环顺序要反过来：外层金额，内层硬币。这样 `1+2` 和 `2+1` 会在不同转移路径中被分别统计。背包题最容易错的不是公式，而是没有分清“每个物品能用几次”和“答案是否区分顺序”。这两个问题决定循环方向。

目标和展示代数转换。给每个数加正号或负号，使结果为 `target`。设正号集合和为 `P`，负号集合和为 `N`，总和为 `S`，则 $P - N = \text{target}$ ， $P + N = S$ ，推出 $P = (S + \text{target}) / 2$ 。问题变成从数组中选若干数，使和为 `P` 的方案数。每个数只能选一次，所以是 0-1 背包计数。

Listing 9.11 LeetCode 494 目标和：0-1 背包计数

```
1 int find_target_sum_ways(const std::vector<int>& nums, int target)
2 {
3     const int total = std::accumulate(nums.begin(), nums.end(), 0);
4     if (std::abs(target) > total || (total + target) % 2 != 0) {
5         return 0;
6     }
7
8     const int positive = (total + target) / 2;
9     std::vector<int> dp(positive + 1, 0);
10    dp[0] = 1;
11
12    for (const int x : nums) {
13        for (int sum = positive; sum >= x; --sum) {
14            dp[sum] += dp[sum - x];
15        }
16    }
17
18    return dp[positive];
19 }
```

如果数组中有 0，这段代码仍然正确。当 `x == 0` 时，`dp[sum] += dp[sum]`，所有已有方案翻倍，正好对应给 0 加正号或负号都不改变总和但算作不同表达式。很多手写特判会在这里出错。计数 DP 还要关注结果范围；力扣题通常保证答案放进 `int`，工程中应考虑 `long long`。

股票问题是状态机 DP。只允许一次交易时，可以维护历史最低价；但加入冷冻期、手续费、交易次数限制后，最低价模型不够用。状态机方法把“每天结束时处于什么状态”作为 DP 状态。以含手续费的无限次交易为例，状态只有两个：`hold` 表示持有股票的最大收益，`cash` 表示不持有股

票的最大收益。买入让 **cash** 变成 **hold**，卖出让 **hold** 变成 **cash**。

Listing 9.12 LeetCode 714 股票含手续费：两状态 DP

```

1 int max_profit_with_fee(const std::vector<int>& prices, int fee)
2 {
3     if (prices.empty()) {
4         return 0;
5     }
6
7     int hold = -prices[0];
8     int cash = 0;
9
10    for (int i = 1; i < static_cast<int>(prices.size()); ++i) {
11        const int previous_hold = hold;
12        const int previous_cash = cash;
13        hold = std::max(previous_hold, previous_cash - prices[i]);
14        cash = std::max(previous_cash, previous_hold + prices[i] - fee);
15    }
16
17    return cash;
18 }

```

必须先保存昨天状态，否则同一天买入后又立刻卖出会污染转移。手续费可以在买入时扣，也可以在卖出时扣，只要保持一致。冷冻期则需要额外状态或引用更早的 **cash**。至多 K 次交易需要把交易次数加入状态维度。股票题统一复盘方式是：列出状态，列出动作，说明动作受哪些规则限制。

区间 DP 的思维与线性 DP 不同。线性 DP 通常按位置从左到右推进；区间 DP 处理一个连续区间，转移往往枚举最后一个操作点或切分点。戳气球的关键是反过来想“最后戳哪个气球”。如果先戳某个气球，它的邻居会变化，状态不稳定；如果最后戳 **mid**，那么它的左右邻居一定是区间边界，收益可确定。

Listing 9.13 LeetCode 312 戳气球：区间 DP 枚举最后操作点

```

1 int max_coins(const std::vector<int>& nums)
2 {
3     std::vector<int> values;
4     values.reserve(nums.size() + 2);
5     values.push_back(1);
6     values.insert(values.end(), nums.begin(), nums.end());
7     values.push_back(1);
8
9     const int n = static_cast<int>(values.size());
10    std::vector<std::vector<int>> dp(n, std::vector<int>(n, 0));
11
12    for (int length = 2; length < n; ++length) {
13        for (int left = 0; left + length < n; ++left) {
14            const int right = left + length;
15            for (int mid = left + 1; mid < right; ++mid) {
16                const int score = dp[left][mid] + dp[mid][right] +
17                    values[left] * values[mid] * values[right];
18                dp[left][right] = std::max(dp[left][right], score);
19            }
20        }
21    }
22 }

```



```

21     }
22
23     return dp[0][n - 1];
24 }

```

`dp[left][right]` 表示开区间 $(left, right)$ 内所有气球被戳掉的最大收益。按区间长度从小到大遍历，是因为大区间依赖小区间。复杂度 $O(n^3)$ ，空间 $O(n^2)$ 。区间 DP 不是所有题都能优化，除非存在决策单调性、四边形不等式等额外性质；面试中不要随便承诺能降复杂度。

状态压缩 DP 把集合表示成二进制位，适合 n 很小但状态与“已选集合”有关的问题。访问所有节点的最短路径可以看成 BFS，也可以看成状态压缩：状态是 $(node, mask)$ ，表示当前在 `node`，已经访问集合为 `mask`。同一个节点配不同访问集合，是不同状态，因为未来能力不同。

Listing 9.14 LeetCode 847 访问所有节点：状态是当前位置加访问集合

```

1 int shortest_path_length(const std::vector<std::vector<int>>& graph)
2 {
3     const int n = static_cast<int>(graph.size());
4     const int full = (1 << n) - 1;
5     std::queue<std::pair<int, int>> queue;
6     std::vector<std::vector<int>> distance(n, std::vector<int>(1 << n, -1));
7
8     for (int node = 0; node < n; ++node) {
9         const int mask = 1 << node;
10        distance[node][mask] = 0;
11        queue.emplace(node, mask);
12    }
13
14    while (!queue.empty()) {
15        const auto [node, mask] = queue.front();
16        queue.pop();
17        if (mask == full) {
18            return distance[node][mask];
19        }
20
21        for (const int next : graph[node]) {
22            const int next_mask = mask | (1 << next);
23            if (distance[next][next_mask] != -1) {
24                continue;
25            }
26            distance[next][next_mask] = distance[node][mask] + 1;
27            queue.emplace(next, next_mask);
28        }
29    }
30
31    return 0;
32 }

```

状态数量是 $O(n2^n)$ ，边扩展约为 $O(m2^n)$ 。这类算法只适合 n 较小的题。状态压缩的核心不是位运算技巧，而是意识到“访问集合”是状态的一部分，不能只用当前位置去重。很多网格题带钥匙、障碍消除、剩余资源，也有类似思想：位置相同但资源不同，未来能力不同，就不能合并成一个状态。

DP 和图最短路之间可以互相转化。若状态转移形成有向无环图，可以按拓扑顺序填 DP；若转移有环且边权为 1，就可能需要 BFS；若边权非负，就可能需要 Dijkstra；若有负边，就要考虑

Bellman-Ford。所谓状态，就是图上的节点；所谓转移，就是边。这个视角能帮助你判断一道题到底该填表、该 BFS，还是该用堆。

常见误区

DP 常见错误有四个。第一，状态定义含糊，例如只写 `dp[i]` 是“最优值”，却不说考虑哪些元素、必须不必须选第 i 个。第二，初始化不符合状态含义。第三，遍历顺序让当前阶段读到了已经被污染的状态。第四，空间优化后无法解释旧值来自哪里。解决办法是先写自然语言状态，再写转移，再写代码。

DP 实验：第一，把 0-1 背包容量循环改成正序，用一个物品被重复使用的用例验证错误。第二，把零钱兑换 II 的循环顺序反过来，观察组合数变成排列数。第三，给股票含手续费打印每天的 `hold` 和 `cash`，理解状态机转移。第四，用小数组手算戳气球区间 DP 表，观察为什么要按区间长度遍历。

动态规划题复盘模板

每道 DP 题按五步复盘：第一，状态是什么，必须能用自然语言解释；第二，答案在 `dp[n]`、`dp[m][n]` 还是某个最大值里；第三，空状态和最小状态如何初始化；第四，转移枚举的是最后一步、最后一个选择，还是最后一段区间；第五，遍历顺序是否保证依赖状态已经算完。若你只能背出公式，却说不出公式对应的最后一步选择，这题就还没有真正掌握。

dp-greedy 深度题卡

动态规划与状态定义

下面只在这里讲一次共性：先写暴力基线，定位最贵的重复工作，再选择能维护不变量的数据结构。每道题卡只分析本题自己的建模、瓶颈、边界和变体，复习时不要把家族共性当成答案。

图示：动态规划与状态定义推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	慢点不是递归写法，而是相同状态被反复计算。若一个状态的未来只取决于少量参数，就应保存结果并按依赖顺序计算。
结构选择	状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择，二维 DP 看两个前缀或网格位置，背包看物品阶段和容量，区间 DP 看最后合并点。
不变量	填表不变量是：计算当前状态时，所有依赖状态已经算完且语义一致。滚动数组必须解释当前格读到的是上一轮还是本轮。
代码落地	数组维度和初始化要服务于状态含义。不可达用大正数或负数哨兵时要避免溢出。方案数可能超过 <code>int</code> 时改用 <code>long long</code> 或按题目取模。

LeetCode 53 最大子数组和

题目模型。状态是必须以当前位置结尾的最大连续和。

推导重点。若前一个后缀为负，接上它只会拖累当前元素，因此可以重新开始。

边界。全负数组、单元素、和溢出、答案不能初始化为零。

变体追问。若要求返回区间下标，同步记录重新开始位置和最优左右端。

LeetCode 62 不同路径

题目模型。网格状态由上方和左方两种最后一步转移而来。

推导重点。二维表可压缩为一维，因为当前行只依赖上一行同列和当前行左侧。

边界。一行、一列、较大网格结果溢出、障碍物版本边界。

变体追问。带障碍物时遇到障碍状态清零，第一行第一列要按阻断处理。

LeetCode 63 不同路径 II

题目模型。障碍物让某些格子的路径数变为零。

推导重点。滚动数组中遇到障碍直接把当前位置清零，否则累加左侧。

边界。起点或终点是障碍、整行被截断、单行障碍。

变体追问。若允许向上或向左走，图会有环，普通填表不再成立。

LeetCode 64 最小路径和

题目模型。状态是到达当前格子的最小代价，最后一步来自上方或左方。

推导重点。先初始化第一行和第一列，再处理内部，主转移保持简单。

边界。单行、单列、权值为零、路径和溢出。

变体追问。若允许四方向移动，变成最短路而不是普通网格 DP。

LeetCode 70 爬楼梯

题目模型。最后一步来自前一阶或前两阶，是最小的重叠子问题模型。

推导重点。滚动变量保存最近两个状态即可，不需要完整数组。

边界。 n 为一、 n 为二、题目是否允许零阶、结果范围。

变体追问。若每次可走一到 k 阶，转移变成固定宽度窗口求和。

LeetCode 72 编辑距离

题目模型。二维状态表示两个前缀之间的最少编辑操作。

推导重点。末尾字符相同继承左上；不同则枚举插入、删除、替换。

边界。空串、完全相同、完全不同、操作代价不同。

变体追问。若只允许插入删除，问题退化到和最长公共子序列相关。

LeetCode 121 买卖股票的最佳时机

题目模型。卖出日固定时，最佳买入日是此前最低价格。

推导重点。扫描维护历史最低价和最大利润，一次交易无需复杂状态机。

边界。单日价格、一直下跌、一直上涨、价格相同。

变体追问。多次交易、冷冻期和手续费版本都要扩展成状态机。

LeetCode 123 买卖股票 III

题目模型。最多两次交易，状态需要包含交易次数和持有状态。

推导重点。维护 buy1、sell1、buy2、sell2 四个状态，按天更新。

边界。价格为空、只够一次交易、两次交易间不能重叠。

变体追问。最多 k 次交易是它的泛化， k 很大时退化为无限次交易。

LeetCode 139 单词拆分

题目模型。状态表示前缀能否由字典单词拼出。

推导重点。枚举最后一个单词的起点，左侧前缀可达且子串在字典中则当前可达。

边界。空串、重复单词、长单词、substr 复制成本。

变体追问。若要输出所有句子，需要 DFS 加记忆化或前驱列表。

LeetCode 152 乘积最大子数组

题目模型。负数会让最大乘积和最小乘积互换。

推导重点。同时维护以当前位置结尾的最大和最小乘积。

边界。零、负数个数奇偶、单元素、乘积溢出。

变体追问。如果要求返回区间，同步记录状态来源。

LeetCode 198 打家劫舍

题目模型。每间房只有偷和不偷两种结局，偷当前就不能偷前一间。

推导重点。滚动保存前一状态和前两状态。

边界。空数组、单间、全零、金额很大。

变体追问。环形房屋要拆成不含首和不含尾两个线性问题。

LeetCode 221 最大正方形

题目模型。以当前位置为右下角的最大正方形由上、左、左上三个状态限制。

推导重点。当前位置为一时取三者最小加一，否则为零。

边界。全零、全一、单行单列、字符一和数字一不要混淆。

变体追问。最大矩形则需要按行构造柱状图再用单调栈。

LeetCode 300 最长递增子序列

题目模型。二次 DP 固定结尾，二分优化维护每个长度的最小结尾。

推导重点。tails 不是实际答案序列，而是未来扩展潜力表。

边界。重复元素、严格递增和非递减区别、空数组。

变体追问。若要还原路径，需要额外前驱数组，不能只看 tails。

LeetCode 309 最佳买卖股票含冷冻期

题目模型。冷冻期让买入来源受昨天卖出状态限制。

推导重点。状态机维护持有、刚卖出、休息三种日终状态。

边界。空价格、一天、连续上涨、卖出后不能次日买入。

变体追问。手续费版本可以在买入或卖出转移中扣费。

LeetCode 714 买卖股票含手续费

题目模型。手续费改变交易收益，仍可用持有和不持有状态。

推导重点。卖出时扣费或买入时扣费都可以，但不要重复扣。

边界。手续费为零、价格震荡、小利润不足手续费。

变体追问。这是状态机比局部贪心更稳的例子。

LeetCode 1143 最长公共子序列

题目模型。状态表示两个前缀的最长公共子序列长度。

推导重点。末尾相同取左上加一，不同取上方和左方最大。

边界。空串、完全相同、无公共字符、子序列不是子串。

变体追问。若要求还原序列，需要从表右下角反向追踪选择。

背包模型

下面只在这里讲一次共性：先写暴力基线，定位最贵的重复工作，再选择能维护不变量的数据结构。每道题目只分析本题自己的建模、瓶颈、边界和变体，复习时不要把家族共性当成答案。

图示：背包模型推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	瓶颈是阶段相同、剩余容量相同的子问题反复出现。容量把未来选择压缩成一个数字，因此可以用表保存。
结构选择	0-1 背包每个物品只能用一次，一维压缩时容量倒序；完全背包物品可无限使用，容量正序。计数组合和计数排列的循环顺序不同。
不变量	处理到第 i 个物品后， $dp[cap]$ 表示只使用前 i 类物品能达到的最优值或方案数。倒序是为了不在同一轮重复使用当前物品。
代码落地	容量通常与数值大小有关，所以复杂度是伪多项式。实现时先判断总和奇偶、目标范围和负数是否存在。

LeetCode 279 完全平方数

题目模型。完全平方数可以看成无限使用的物品，目标是凑出 n 的最少数量。

推导重点。完全背包最少物品数，或把数字看成图节点做 BFS。

边界。 n 为零或一、完全平方数本身、较大 n 。

变体追问。数学四平方定理可给更强解法，但面试 DP 更通用。

LeetCode 322 零钱兑换

题目模型。金额是容量，硬币可无限使用，目标是最少硬币数。

推导重点。 $dp[value]$ 从 $value$ 减 $coin$ 的已知状态转移。

边界。 无法凑出、 $amount$ 为零、硬币面额大于目标、哨兵溢出。

变体追问。 零钱兑换 II 问组合数，循环语义不同。

LeetCode 416 分割等和子集

题目模型。 总和一半是背包容量，每个数只能使用一次。

推导重点。 0-1 背包可达性，一维容量必须倒序。

边界。 总和为奇数、目标为零、数值较大、复杂度依赖 $target$ 。

变体追问。 负数会破坏普通容量模型，需要重新建模。

LeetCode 494 目标和

题目模型。 正负号选择可代数转换为选若干数凑出特定正和。

推导重点。 若 S 加 $target$ 为奇数或目标绝对值过大，直接无解。

边界。 零元素会让方案数翻倍、目标为负、容量为零。

变体追问。 也可用哈希 DP 保存所有可能和，但背包转换更高效。

LeetCode 518 零钱兑换 II

题目模型。 硬币无限使用，问组合数而不是最少数量。

推导重点。 外层遍历硬币、内层金额正序，保证组合按固定硬币顺序统计。

边界。 $amount$ 为零、无硬币、组合数较大、循环顺序反例。

变体追问。 若外层金额内层硬币，会统计排列数。

贪心与交换证明

下面只在这里讲一次共性：先写暴力基线，定位最贵的重复工作，再选择能维护不变量的数据结构。每道题目只分析本题自己的建模、瓶颈、边界和变体，复习时不要把家族共性当成答案。

图示：贪心与交换证明推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	慢点是选择空间爆炸。若每一步都有很多候选，而某个排序规则或边界规则能安全丢掉大部分候选，就可能存在贪心。
结构选择	贪心需要找到可交换的局部最优：最早结束、最小右端、当前最远覆盖、当前最大收益或最小代价。排序只是手段，不是证明。
不变量	贪心不变量通常是当前方案在相同选择次数下不落后于任何其他方案，或者当前保留的资源最多、结束最早、右边界最小。
代码落地	实现时先把比较器写成明确的业务顺序。区问题注意起点相同时终点升序还是降序；覆盖题注意闭区间和半开区间的等号。

LeetCode 55 跳跃游戏

题目模型。 扫描过程中只关心当前能到达的最远位置。

推导重点。 如果下标超过最远可达，任何之前路径都不能到达它；否则用它扩展边界。

边界。 第一个元素为零、数组长度一、恰好到达最后、远超最后。

变体追问。 求最少跳数时把可达区间看成 BFS 层。

LeetCode 122 买卖股票 II

题目模型。 允许多次交易且不能同时持有，所有正收益差都可以收集。

推导重点。 把长上涨段拆成每天买卖收益相同，因此累加正差值最优。

边界。 下降段、平台价格、只有一天、手续费不存在。

变体追问。 有手续费时不能简单累加正差，需要状态机或调整贪心阈值。

LeetCode 169 多数元素

题目模型。 超过一半的元素可以和其他元素两两抵消后仍然剩下。

推导重点。 Boyer-Moore 投票维护候选和计数。

边界。 题目是否保证存在多数、计数归零、全相同。

变体追问。若找超过三分之一的元素，需要维护两个候选。

LeetCode 435 无重叠区间

题目模型。保留最多不重叠区间等价于删除最少区间。

推导重点。按终点升序选择，结束越早给后续留下空间越多。

边界。端点相接是否重叠、完全覆盖、相同终点、空列表。

变体追问。用交换论证说明最早结束的选择不劣于其他第一选择。

LeetCode 452 用最少数量的箭引爆气球

题目模型。一支箭的位置可以覆盖所有包含该位置的区间。

推导重点。按右端点排序，当前箭放在最早结束气球的右端。

边界。端点相同、负坐标、区间包含、闭区间边界。

变体追问。它和无重叠区间是同一类最早结束贪心。

LeetCode 763 划分字母区间

题目模型。每个片段必须覆盖其中所有字符的最后出现位置。

推导重点。扫描维护当前片段最远右端，到达右端即可切分。

边界。单字符、所有字符相同、字符多次跨段。

变体追问。这是最早合法切分贪心，能最大化片段数量。

区间、排序与合并

下面只在这里讲一次共性：先写暴力基线，定位最贵的重复工作，再选择能维护不变量的数据结构。每道题目只分析本题自己的建模、瓶颈、边界和变体，复习时不要把家族共性当成答案。

图示：区间、排序与合并推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	瓶颈是没有利用排序后相邻关系。区间按起点或终点排序后，很多远离的区间不可能再影响当前决策。
结构选择	合并按起点排序，调度按终点排序，覆盖题常用起点升序且终点降序。扫描时维护当前最远右端、结果尾区间或已选区间终点。
不变量	结果数组中的区间已经互不重叠且有序；当前扫描区间只可能影响结果最后一个区间或当前边界。
代码落地	比较器必须处理相同起点，否则覆盖和去重会错。区间端点可能溢出时用更大整数。闭区间和半开区间决定是否用小于等于。

LeetCode 56 合并区间

题目模型。排序后可能与当前区间相交的只会是结果尾部。

推导重点。按起点排序，扫描时维护结果最后区间的右端点。

边界。端点相接、完全覆盖、无重叠、空列表、输入未排序。

变体追问。若区间是半开区间，端点相等不一定合并。

LeetCode 57 插入区间

题目模型。新区间只会影响与它相交的一段连续区间。

推导重点。先追加左侧完全不相交区间，再合并重叠段，最后追加右侧。

边界。新区间在最前、最后、覆盖全部、刚好相邻、原列表为空。

变体追问。若输入不是有序且不重叠，必须先排序或换模型。

LeetCode 1288 删除被覆盖区间

题目模型。被覆盖要求另一区间起点不晚且终点不早。

推导重点。按起点升序、终点降序排序，扫描维护最大右端。

边界。同起点区间、完全覆盖、无覆盖、端点相等。

变体追问。终点降序是关键，否则同起点小区间会先出现造成误判。

回溯、枚举与剪枝

下面只在这里讲一次共性：先写暴力基线，定位最贵的重复工作，再选择能维护不变量的数据结构。每道题目只分析本题自己的建模、瓶颈、边界和变体，复习时不要把家族共性当成答案。

图示：回溯、枚举与剪枝推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	慢点在于大量分支在很早就已经不可能成功，仍然被继续展开。重复元素还会产生等价排列或组合。
结构选择	剪枝来自容量、剩余长度、排序后去重、合法性预检查和提前终止。组合题关注起点推进，排列题关注元素是否使用，分割题关注当前片段是否合法。
不变量	路径数组保存已经做出的选择；起点下标保证组合不回头；同层去重保证同一个位置层级不会选择相同值；已使用数组保证排列不重复使用同一元素。
代码落地	回溯代码虽然天然递归，但工程中要意识到深度。题目规模较小时递归可读性好；若要遵守严格工程栈限制，可以改成显式栈状态机。

LeetCode 78 子集

题目模型。每个元素都有选或不选两种选择，答案是所有路径节点。

推导重点。回溯按起点向后枚举，避免不同顺序生成同一集合。

边界。空数组、单元素、输出数量为二的 n 次方、顺序不要求。

变体追问。也可用位掩码枚举所有子集，适合 n 较小的场景。

LeetCode 79 单词搜索

题目模型。网格路径不能重复使用同一格，状态包含位置、匹配下标和访问标记。

推导重点。剪枝包括首字符不匹配、剩余长度不足、字符频次不足和越界访问。

边界。单字符单词、路径需要回退、重复字母、单元格不能复用。

变体追问。若要查很多单词，应使用 Trie 合并搜索前缀。

LeetCode 90 子集 II

题目模型。重复元素会让不同下标路径生成相同集合。

推导重点。排序后在同一层跳过相同值，保留不同层使用重复值的可能。

边界。全重复、空集、重复值相邻、去重条件写成同层而不是全局。

变体追问。组合总和 II 使用同样的同层去重思想。

LeetCode 131 分割回文串

题目模型。每次选择一个从当前位置开始的回文前缀。

推导重点。预处理回文 DP 表，让回溯合法性检查为常数。

边界。空串、单字符、全相同字符、重复分割路径。

变体追问。若只求最少切割次数，可改成 DP 最短切分。

LeetCode 216 组合总和 III

题目模型。从一到九中选 k 个数和为 n ，每个数最多用一次。

推导重点。按递增起点枚举，利用剩余个数和剩余和剪枝。

边界。 n 太小、 n 太大、 k 为零、路径长度已经超过 k 。

变体追问。可预估剩余最小和最大和进一步剪枝。

堆与动态极值

下面只在这里讲一次共性：先写暴力基线，定位最贵的重复工作，再选择能维护不变量的数据结构。每道题目只分析本题自己的建模、瓶颈、边界和变体，复习时不要把家族共性当成答案。

图示：堆与动态极值推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	慢点在于动态极值查询。我们只需要当前最极端元素，不需要整个集合完全有序。
结构选择	堆把插入和弹出极值控制在对数复杂度。Top K 用固定大小堆维护边界，合并多路序列用堆保存每一路当前头部，数据流中位数用双堆维护两半。
不变量	堆不变量不是全局有序，而是堆顶代表当前最优候选。若存在过期元素，需要在弹出时懒删除，保证真正使用堆顶前它仍然有效。
代码落地	C++ 的 <code>std::priority_queue</code> 默认大顶堆。小顶堆用 <code>std::greater<T></code> ；结构体比较器要按谁优先级更低来写。堆中保存大对象时尽量保存下标或指针。

LeetCode 215 数组中的第 K 个最大元素

题目模型。只关心第 K 大，不需要完整排序。

推导重点。大小为 K 的小顶堆保存当前前 K 大边界；快速选择可做到平均线性。

边界。K 为一、K 为 n、重复值、随机 pivot。

变体追问。若数据流持续到来，堆方案比一次性快速选择更自然。

LeetCode 295 数据流的中位数

题目模型。数据流需要动态维护较小一半和较大一半。

推导重点。大顶堆保存较小一半，小顶堆保存较大一半，大小差不超过一。

边界。偶数个元素、负数、重复值、平均值溢出。

变体追问。若还要删除任意元素，需要延迟删除哈希表。

LeetCode 347 前 K 个高频元素

题目模型。先统计频次，再只维护前 K 个候选。

推导重点。小顶堆大小超过 K 就弹出最低频；桶排序利用频次上界。

边界。K 等于不同元素数、频次相同、结果顺序不要求。

变体追问。若要按频次和字典序排序，比较器要同时处理两个键。

LeetCode 621 任务调度器

题目模型。相同任务之间有冷却间隔，核心受最高频任务骨架限制。

推导重点。可用大顶堆模拟，也可用最高频公式计算最短时间。

边界。冷却为零、多个最高频、空闲被其他任务填满。

变体追问。若任务有不同释放时间或执行时长，就变成更一般的调度问题。

LeetCode 973 最接近原点的 K 个点

题目模型。距离只需比较平方，避免开方误差和成本。

推导重点。大小为 K 的大顶堆保存当前最近 K 个点中最远者。

边界。K 为一、K 为全部、距离相同、坐标平方溢出。

变体追问。快速选择可平均线性，但堆更适合数据流。

链表与指针重连

下面只在这里讲一次共性：先写暴力基线，定位最贵的重复工作，再选择能维护不变量的数据结构。每道题卡只分析本题自己的建模、瓶颈、边界和变体，复习时不要把家族共性当成答案。

图示：链表与指针重连推导链

暴力基线	枚举候选、完整模拟或逐个检查，先保证覆盖全部可能答案。
瓶颈定位	链表慢点在于不能随机访问。要找倒数位置、分组边界或交点，只能通过指针间距离、快慢指针或哈希记录访问过的节点。
结构选择	优化来自哨兵节点、快慢指针、前驱指针和局部反转。链表题的核心不是值交换，而是保持断开和接回的顺序正确。
不变量	必须明确每个指针指向哪一段：已经处理好的前缀头尾、当前待处理节点、下一段头节点。每次改 <code>next</code> 前先保存后继。
代码落地	节点通常由评测平台管理，代码不要随意 <code>delete</code> 输入节点。若创建新节点，要说明所有权。比较交点时比较地址，不比较值。

LeetCode 2 两数相加

题目模型。链表按低位到高位保存数字，本质是竖式加法而不是整数转换。

推导重点。维护两个游标和进位，任一链未结束或进位非零都继续生成节点。

边界。两链长度不同、最后进位、输入为零、不能用内置整数承载超长数字。

变体追问。若链表高位在前，可以用栈反向处理或先反转链表。

LeetCode 141 环形链表

题目模型。快慢指针在有环时必然相遇，无环时快指针先到空。

推导重点。不用哈希表也能常数空间检测环。

边界。空链、单节点自环、两节点环、尾部无环。

变体追问。若要求环入口，用相遇后双指针同步走。

LeetCode 142 环形链表 II

题目模型。相遇点到入口的距离关系来自快慢指针速度差。

推导重点。相遇后一个指针回头，两个指针每次走一步，再次相遇就是入口。

边界。入口在头节点、长尾短环、无环、单节点环。

变体追问。也可用哈希集合，但空间更高。

LeetCode 146 LRU 缓存

题目模型。哈希表负责按 key 定位，双向链表负责维护新旧顺序。

推导重点。访问和更新都把节点移动到头部，容量满时删除尾部最旧节点。

边界。容量为一、重复 put、get 不存在、更新已有值。

变体追问。C++ 可用 `std::list` 加迭代器，也可手写双向链表理解所有权。

LeetCode 148 排序链表

题目模型。链表不能随机访问，适合归并排序而不是快速排序数组 partition。

推导重点。自底向上迭代归并能避免递归栈，同时保持 $O(n \log n)$ 。

边界。空链、单节点、重复值、奇数长度、切断子链。

变体追问。若把值复制到数组排序，会改变节点身份语义。

LeetCode 160 相交链表

题目模型。相交是节点地址相同，不是节点值相同。

推导重点。两个指针走完各自链表后切换到另一条链，走过总长度相同后相遇。

边界。不相交、头节点相交、尾节点相交、长度差很大。

变体追问。哈希集合更直观但空间更高。

LeetCode 206 反转链表

题目模型。每次把当前节点的 next 指向已经反转好的前缀头。

推导重点。先保存后继，再改指针，再推进 previous 和 current。

边界。空链、单节点、两个节点、不要丢失剩余链。

变体追问。K 组反转是在这段局部反转外增加分组边界。

动态规划证明、背包和区间 DP

动态规划不是数组技巧，而是对子问题图的压缩求解。暴力搜索中，一个节点表示当前处理到哪里、已经做了哪些选择、剩余资源是什么。若不同路径到达同一个状态后，未来可选集合完全相同，就出现重叠子问题。DP 把这个状态命名并保存结果，再按依赖顺序计算。状态定义越稳定，代码越短；状态缺失一项，转移就会偷偷依赖历史细节。

线性 DP 常按最后一步思考。最大子数组和问以当前位置结尾的最佳后缀，打家劫舍问考虑前 i 个房屋的最大收益，股票题问每天结束时持有或不持有的最大收益。最后一步方法的价值在于把全局最优拆成互斥候选：当前元素单独开始还是接在前面，当前房屋偷还是不偷，今天买、卖还是休息。每个候选都必须对应一个已经计算好的状态。

二维字符串 DP 需要精确定义前缀。最长公共子序列的 $dp[i][j]$ 表示两个前缀的答案，不是以某个字符结尾的答案。编辑距离的 $dp[i][j]$ 表示把 word1 前 i 个字符变成 word2 前 j 个字符的最少操作。多出第零行和第零列不是为了方便而已，它们代表空串，给插入和删除操作提供统一边界。

背包题最容易背错循环顺序。0-1 背包中每个物品只能用一次，一维压缩时容量必须倒序，确保读取的是上一轮物品阶段的状态。完全背包中物品可无限使用，容量正序允许当前物品在同一轮被再次使用。组合计数外层遍历物品，排列计数外层遍历容量。循环顺序不是模板差异，而是状态语义差异。

目标和、分割等和子集和零钱兑换 II 是背包三连。分割等和是可达性，目标容量为总和一半；目标和通过代数变换把正负号选择转成选若干数凑出正集合和；零钱兑换 II 是完全背包组合计数。三题放在一起能训练四个问题：物品是否只能用一次，状态保存布尔值还是方案数，容量遍历正序还是倒序，答案是否受零元素影响。

区间 DP 的关键是按长度枚举。戳气球中，如果最后戳破区间内某个气球，那么它左右两侧已经分别被处理完，边界气球仍在；矩阵链乘法中，最后一次乘法把区间切成左右两段；回文相关问题中，外层字符关系决定内部区间是否有效。区间 DP 的状态通常是 $dp[left][right]$ ，依赖更短区间，因此要先枚举区间长度。

空间压缩必须能解释旧值来自哪里。二维 LCS 压成一维时， $dp[j]$ 在更新前是上一行同列， $dp[j-1]$ 是当前行左侧，左上角需要额外变量保存。若解释不清这三个来源，空间压缩很容易写错。初学阶段先写二维表，确认状态和转移，再压缩空间。

实验建议：把 0-1 背包容量循环改成正序，用一个物品重量为一、价值为一、容量为二的用例观察它被重复使用。把零钱兑换 II 外层内层调换，观察组合数变成排列数。给 LCS 打印整个 DP 表，再手动从右下角回溯出一个公共子序列，理解表中每个值的来源。

本节复盘

阅读本节后，请回到相邻章节的例题，把暴力瓶颈、优化依据、状态不变量、C++ 容器成本和边界测试重新写一遍。能把同一思想迁移到变体题，才算真正掌握。

第零部分

面试专项：从题型到工程表达

第 10 章

线性结构专项：链表、栈、队列和单调结构

线性结构专项题看上去都很短，但它们是面试中最容易暴露基本功的一类。数组和字符串通常考察下标、窗口和局部状态；链表考察节点关系和指针顺序；栈考察最近未解决状态；队列考察层次推进；单调栈和单调队列考察候选淘汰。很多候选人的代码错在一个小边界，但真正的原因不是粗心，而是没有说清楚“当前结构里保存的对象到底代表什么”。

链表题首先要把“值”和“节点”分开。数组里交换两个元素，很多时候只要交换值；链表里更重要的是节点之间的连接关系。删除节点、反转链表、合并链表、K 个一组翻转，都不是在移动一段连续内存，而是在改变若干个 `next` 指针。写链表题之前，建议先在纸上画三段：已经处理好的部分、当前正在处理的部分、尚未处理的部分。每次改指针前都问一句：原来的后继是否已经保存？新的前驱是谁？最终头节点从哪里返回？

删除倒数第 N 个节点的暴力想法是先遍历一遍得到长度 `len`，再删除第 `len - n + 1` 个节点。这个做法完全正确，复杂度也是 $O(n)$ ，但需要两次遍历。面试更常希望你展示快慢指针：让快指针先走 `n` 步，然后快慢一起走；当快指针到达尾部时，慢指针正好停在待删除节点的前驱。优化的本质不是少了一行代码，而是把“倒数位置”转换成“两个指针之间保持固定距离”的不变量。

Listing 10.1 LeetCode 19 删除倒数第 N 个节点：虚拟头节点和快慢指针

```
1 struct ListNode {
2     int value = 0;
3     ListNode* next = nullptr;
4 };
5
6 ListNode* remove_nth_from_end(ListNode* head, int n)
7 {
8     ListNode dummy;
9     dummy.next = head;
10
11     ListNode* fast = &dummy;
12     ListNode* slow = &dummy;
13
14     for (int step = 0; step < n; ++step) {
15         fast = fast->next;
16     }
17     while (fast->next != nullptr) {
18         fast = fast->next;
19         slow = slow->next;
20     }
21
22     ListNode* removed = slow->next;
23     slow->next = removed->next;
24     return dummy.next;
25 }
```

虚拟头节点的价值在这里非常明显。如果删除的是原来的头节点，没有虚拟头，就要单独写一段分支；有了虚拟头，删除头节点和删除中间节点统一成“修改某个前驱节点的 `next`”。这类题的面试表达要明确：快慢指针之间相隔 `n` 个真实节点，循环结束时快指针停在最后一个真实节点，慢指针停在待删节点前驱。边界要测三个：只有一个节点，删除头节点，删除尾节点。

合并两个有序链表是链表和双指针结合的题。暴力方法可以把两个链表的值全部放进数组，排序后重新建链表；这样会丢掉链表题真正考察的节点重连能力，也多用了 $O(n)$ 额外空间。更自然的做法是维护一个结果链表尾指针，每次从两个链表头部取较小节点接到尾部。由于两个输入链表本来有序，每一步的局部最小节点就是全局下一个节点。

Listing 10.2 LeetCode 21 合并两个有序链表：尾指针接入节点

```

1 ListNode* merge_two_lists(ListNode* first, ListNode* second)
2 {
3     ListNode dummy;
4     ListNode* tail = &dummy;
5
6     while (first != nullptr && second != nullptr) {
7         if (first->value <= second->value) {
8             tail->next = first;
9             first = first->next;
10        } else {
11            tail->next = second;
12            second = second->next;
13        }
14        tail = tail->next;
15    }
16
17    tail->next = first != nullptr ? first : second;
18    return dummy.next;
19 }
```

这段代码没有创建新节点，只是复用原链表节点。因此如果题目要求不能修改输入，就要改成新建节点版本。稳定性也值得说明：当两个节点值相等时，代码优先取 **first**，这会保持第一条链表中相等元素的相对顺序。复杂度是 $O(m+n)$ ，空间是 $O(1)$ 。错误高发点是忘记移动 **tail**，或者循环结束后忘记把剩余链表整体接上。

K 个一组翻转链表比普通反转多了“分组边界”。暴力想法是把所有节点放进数组，按 K 个一组交换指针或值，再重新连接；这样虽然容易写，但牺牲了链表题的原地处理能力。原地做法要先检查当前组是否有 K 个节点，不足 K 个就保持原样；足够 K 个时，只翻转这一段，并把上一组的尾部接到新头，把新尾接到下一组开头。

Listing 10.3 LeetCode 25 K 个一组翻转链表：先找边界再局部反转

```

1 ListNode* reverse_k_group(ListNode* head, int k)
2 {
3     if (k <= 1 || head == nullptr) {
4         return head;
5     }
6
7     ListNode dummy;
8     dummy.next = head;
9     ListNode* group_previous = &dummy;
10
11    while (true) {
12        ListNode* kth = group_previous;
13        for (int step = 0; step < k && kth != nullptr; ++step) {
14            kth = kth->next;
15        }
16        if (kth == nullptr) {
```

```

17         break;
18     }
19
20     ListNode* group_next = kth->next;
21     ListNode* previous = group_next;
22     ListNode* current = group_previous->next;
23
24     while (current != group_next) {
25         ListNode* next = current->next;
26         current->next = previous;
27         previous = current;
28         current = next;
29     }
30
31     ListNode* new_group_tail = group_previous->next;
32     group_previous->next = kth;
33     group_previous = new_group_tail;
34 }
35
36 return dummy.next;
37 }

```

这题最重要的是命名。没有 `group_previous`、`kth`、`group_next` 这三个边界名，代码会变成一堆难以验证的指针操作。反转循环里让 `previous` 初始等于 `group_next`，是为了反转完成后原组头节点自然指向下一组开头。这个技巧能减少额外连接分支。面试时不要急着写代码，先画出一组翻转前后：上一组尾、当前组头、当前组尾、下一组头，四个位置说清楚后，代码才有依据。

栈题的核心是“最近未解决”。有效括号中，最近出现的左括号必须最先被匹配；路径简化中，最近进入的目录会被 `..` 回退；表达式求值中，最近的操作数和运算符构成局部归约。栈的优化不是为了降低所有问题的复杂度，而是把嵌套结构或最近依赖关系变成线性扫描。

最小栈是一个很好的设计题。只用一个普通栈保存元素，`top` 和 `pop` 都是常数时间，但 `getMin` 需要扫描所有元素。优化方法是额外维护一个同步最小值栈。每压入一个值，同时压入“截至当前栈顶的最小值”；弹出时两个栈一起弹。这样任意时刻最小值栈栈顶就是当前全部元素的最小值。

Listing 10.4 LeetCode 155 最小栈：数据栈和最小值栈同步

```

1 class MinStack {
2 public:
3     void push(int value)
4     {
5         this->values_.push_back(value);
6         if (this->minimums_.empty()) {
7             this->minimums_.push_back(value);
8         } else {
9             this->minimums_.push_back(std::min(value, this->minimums_.back()));
10        }
11    }
12
13    void pop()
14    {
15        this->values_.pop_back();
16        this->minimums_.pop_back();
17    }
18 }

```

```

19     int top() const
20     {
21         return this->values_.back();
22     }
23
24     int get_min() const
25     {
26         return this->minimums_.back();
27     }
28
29 private:
30     std::vector<int> values_;
31     std::vector<int> minimums_;
32 };

```

这段代码故意使用 `std::vector` 作为栈，因为 `vector` 支持连续存储、`back`、`push_back` 和 `pop_back`，足够表达栈语义。类成员访问使用 `this->`，避免成员和局部变量混淆。常见错误是只在新值更小时才压入最小值栈，但弹出重复最小值时就会错。例如依次压入 2, 2, 3，如果只记录一个 2，弹出一个 2 后最小值仍应是 2。同步栈写法用空间换清晰和可靠。

单调栈是“最近未解决”的强化版：栈里保存的不是所有历史元素，而是一组仍有可能成为答案的候选。每日温度中，栈保存还没找到更高温度的下标，且栈中温度从底到顶单调不增。新温度到来时，如果高于栈顶，就说明栈顶那一天的答案已经确定，因为当前天是它右侧第一个更高温度的位置。

柱状图中最大的矩形更能体现单调栈的威力。暴力做法枚举每根柱子作为矩形高度，再向左右扩展找到第一个更矮柱子，最坏 $O(n^2)$ 。优化的关键是：当我们知道某根柱子左右两侧第一个更矮的位置时，以它为最低高度的最大矩形宽度就确定了。单调递增栈可以在一次扫描中确定“右侧第一个更矮”。

Listing 10.5 LeetCode 84 柱状图最大矩形：单调递增栈

```

1  int largest_rectangle_area(const std::vector<int>& heights)
2  {
3      std::vector<int> stack;
4      stack.reserve(heights.size() + 1);
5      int answer = 0;
6
7      for (int i = 0; i <= static_cast<int>(heights.size()); ++i) {
8          const int current_height =
9              i == static_cast<int>(heights.size()) ? 0 : heights[i];
10
11          while (!stack.empty() && heights[stack.back()] > current_height) {
12              const int height = heights[stack.back()];
13              stack.pop_back();
14              const int left_less_index = stack.empty() ? -1 : stack.back();
15              const int width = i - left_less_index - 1;
16              answer = std::max(answer, height * width);
17          }
18          stack.push_back(i);
19      }
20
21      return answer;
22 }

```

这里末尾人为加入高度 0 的哨兵，目的是把栈里剩余柱子全部弹出结算。弹出某个下标时，当

前下标 i 是它右侧第一个更矮的位置；弹出后新的栈顶是它左侧第一个更矮的位置。因此宽度是两者之间的柱子数量。这个解释比“套单调栈模板”重要得多。复杂度是 $O(n)$ ，因为每个下标最多入栈一次、出栈一次。易错点是比较符号：使用 $>$ 或 $>=$ 都能做，但对重复高度的归属不同，必须和宽度计算保持一致。

队列和双端队列负责维护“按时间进入、按窗口过期”的候选。滑动窗口最大值的暴力解对每个窗口扫描一遍最大值，复杂度 $O(nk)$ 。堆也能做，但需要处理过期下标。单调队列更贴合题目：队头永远是当前窗口最大值下标，队尾负责删除不可能再成为最大值的候选。

Listing 10.6 LeetCode 239 滑动窗口最大值：队头保存当前最大值下标

```
1 std::vector<int> max_sliding_window(const std::vector<int>& nums, int k)
2 {
3     std::vector<int> answer;
4     if (nums.empty() || k <= 0) {
5         return answer;
6     }
7
8     std::deque<int> deque;
9     for (int right = 0; right < static_cast<int>(nums.size()); ++right) {
10        while (!deque.empty() && nums[deque.back()] <= nums[right]) {
11            deque.pop_back();
12        }
13        deque.push_back(right);
14
15        const int left = right - k + 1;
16        if (deque.front() < left) {
17            deque.pop_front();
18        }
19        if (left >= 0) {
20            answer.push_back(nums[deque.front()]);
21        }
22    }
23    return answer;
24 }
```

为什么队尾较小元素可以删除？设队尾下标为 j ，新下标为 $right$ ，如果 $nums[j] \leq nums[right]$ ，那么在未来任何同时包含 j 和 $right$ 的窗口里， j 都不可能比 $right$ 更优；而且 j 更早过期。因此删除 j 不会影响任何未来答案。这就是候选淘汰的证明。单调队列的复盘要说清两个动作：入队前从队尾删掉被新元素支配的候选；出答案前从队头删掉已过期的候选。

环形链表是快慢指针的另一条主线。暴力方法可以把走过的节点地址放进哈希集合，每到一个节点就检查是否见过；如果见过，说明有环；如果走到空指针，说明无环。这个方法直观，时间 $O(n)$ ，空间 $O(n)$ 。快慢指针把空间降到常数：慢指针每次走一步，快指针每次走两步。如果没有环，快指针会先到空；如果有环，快指针进入环后会不断追赶慢指针，最终相遇。

Listing 10.7 LeetCode 141 环形链表：快慢指针检测相遇

```
1 bool has_cycle(ListNode* head)
2 {
3     ListNode* slow = head;
4     ListNode* fast = head;
5
6     while (fast != nullptr && fast->next != nullptr) {
7         slow = slow->next;
8         fast = fast->next->next;
```

```

9         if (slow == fast) {
10             return true;
11         }
12     }
13
14     return false;
15 }

```

这题的正确性可以用相对速度解释。进入环之后，快指针相对慢指针每轮多走一步；环长度有限，因此相对距离会按环长度取模不断变化，必然出现 0，也就是相遇。不要把这题说成“快指针总会追上慢指针”就结束，最好补一句“因为在环内相对距离每次加一，对环长取模”。如果题目进一步要求找到入环节点，做法是在相遇后让一个指针回到头节点，两个指针都每次走一步，再次相遇的位置就是入口。这来自公式：头到入口距离加上入口到相遇点距离，与快慢指针走过路程差之间存在整环倍数关系。面试里不一定要完整推公式，但要知道这不是经验结论。

LRU 缓存是线性结构和哈希表组合的设计题。题目要求 `get` 和 `put` 都是 $O(1)$ ，并且容量满时淘汰最近最少使用的键。单独用数组或链表，可以维护使用顺序，但查找 `key` 是 $O(n)$ ；单独用哈希表可以常数查找，但无法常数时间移动“最近使用”的位置。正确组合是：双向链表维护从最近到最久的顺序，哈希表从 `key` 映射到链表节点。

Listing 10.8 LeetCode 146 LRU 缓存：哈希表定位节点，双向链表维护顺序

```

1 class LRUCache {
2 public:
3     explicit LRUCache(int capacity)
4         : capacity_(capacity)
5     {
6     }
7
8     int get(int key)
9     {
10         const auto found = this->items_.find(key);
11         if (found == this->items_.end()) {
12             return -1;
13         }
14         this->touch(found->second);
15         return found->second->second;
16     }
17
18     void put(int key, int value)
19     {
20         const auto found = this->items_.find(key);
21         if (found != this->items_.end()) {
22             found->second->second = value;
23             this->touch(found->second);
24             return;
25         }
26
27         this->entries_.emplace_front(key, value);
28         this->items_[key] = this->entries_.begin();
29
30         if (static_cast<int>(this->entries_.size()) > this->capacity_) {
31             const int removed_key = this->entries_.back().first;
32             this->items_.erase(removed_key);

```

```

33         this->entries_.pop_back();
34     }
35 }
36
37 private:
38     using Entry = std::pair<int, int>;
39     using Iterator = std::list<Entry>::iterator;
40
41     void touch(Iterator iterator)
42     {
43         this->entries_.splice(this->entries_.begin(), this->entries_, iterator);
44     }
45
46     int capacity_ = 0;
47     std::list<Entry> entries_;
48     std::unordered_map<int, Iterator> items_;
49 };

```

这段代码使用 `std::list`，不是因为链表在刷题中通常更快，而是因为 LRU 需要在已知节点迭代器的情况下常数时间移动节点到表头。`splice` 不复制节点，只改变链表连接关系；哈希表保存的迭代器仍指向同一节点。这里 `std::list` 的节点稳定性正好匹配需求。反过来，如果用 `vector` 保存顺序，移动到表头会导致大量元素移动，还可能让保存的下标或迭代器失效。LRU 的复盘重点是“为什么必须两个结构一起用”：哈希表负责定位，双向链表负责顺序，任何一个单独使用都达不到题目复杂度。

接雨水可以用多个思路解决。暴力方法枚举每个位置，向左找最高柱子，向右找最高柱子，该位置能接的水是左右最高的较小值减当前高度；每个位置都向两边扫描，复杂度 $O(n^2)$ 。前后缀数组可以把左右最高值预处理出来，时间降到 $O(n)$ ，空间 $O(n)$ 。双指针进一步把空间降到 $O(1)$ ：左右两侧各维护当前最高柱子，哪边较低，就结算哪边。

Listing 10.9 LeetCode 42 接雨水：双指针维护两侧最高柱

```

1 int trap(const std::vector<int>& height)
2 {
3     int left = 0;
4     int right = static_cast<int>(height.size()) - 1;
5     int left_max = 0;
6     int right_max = 0;
7     int water = 0;
8
9     while (left < right) {
10         if (height[left] < height[right]) {
11             left_max = std::max(left_max, height[left]);
12             water += left_max - height[left];
13             ++left;
14         } else {
15             right_max = std::max(right_max, height[right]);
16             water += right_max - height[right];
17             --right;
18         }
19     }
20
21     return water;
22 }

```

为什么可以结算较低的一侧？假设 `height[left] < height[right]`，右侧至少存在一个高度大于左侧当前位置的柱子，因此当前位置水位的右边界不会低于 `height[left]`；它能接多少水只取决于左侧最高柱 `left_max`。如果左侧最高还不够高，就接不到水；如果够高，就能立刻计算。单调栈也能做接雨水，它在遇到右边界时弹出低洼底部并结算横向宽度。双指针更节省空间，单调栈更接近“找左右边界”的模型，面试中可以比较两者。

表达式求值考察栈对局部归约的建模。最简单的四则表达式如果没有括号，但有乘除优先级，可以扫描字符串，用栈保存已经归约到加减层面的项。遇到数字时，根据前一个运算符决定：加号就压入正数，减号压入负数，乘除就和栈顶立即合并。这样乘除不会等到最后才处理，加减则通过最后求和完成。

Listing 10.10 LeetCode 227 基本计算器 II：栈处理乘除优先级

```
1 int calculate(const std::string& s)
2 {
3     std::vector<int> stack;
4     char operation = '+';
5     int number = 0;
6
7     for (int i = 0; i <= static_cast<int>(s.size()); ++i) {
8         const char ch = i == static_cast<int>(s.size()) ? '+' : s[i];
9         if (std::isdigit(static_cast<unsigned char>(ch))) {
10             number = number * 10 + (ch - '0');
11             continue;
12         }
13         if (ch == ' ') {
14             continue;
15         }
16
17         if (operation == '+') {
18             stack.push_back(number);
19         } else if (operation == '-') {
20             stack.push_back(-number);
21         } else if (operation == '*') {
22             stack.back() *= number;
23         } else if (operation == '/') {
24             stack.back() /= number;
25         }
26
27         operation = ch;
28         number = 0;
29     }
30
31     return std::accumulate(stack.begin(), stack.end(), 0);
32 }
```

这里在循环末尾人为补一个加号，是为了触发最后一个数字的归约。表达式题的易错点非常多：多位数要持续累积；空格要跳过；除法在 C++ 中对整数向 0 截断，和题目语义是否一致要确认；如果有括号，就需要另一个层次的栈或显式状态机。不要一开始就写复杂解析器，先按题目支持的语法范围选择最小可行模型。面试表达时可以说：栈里保存的是已经处理完乘除优先级、等待最终加总的项。

线性结构题的实验训练不应只跑样例。链表题要构造空链表、单节点、删除头、删除尾、K 不整除长度；栈题要构造重复最小值、空弹出是否题目保证、嵌套和交叉括号；单调结构要构造严格递增、严格递减、全相等、窗口为 1、窗口等于数组长度。面试中如果能主动说出这些测试，说明你

不仅知道算法，也知道边界来自哪里。

相交链表是链表“节点身份”意识的经典题。两个链表如果相交，意思不是某个节点的值相等，而是从某个节点开始共享同一批节点对象。暴力方法可以把第一条链表所有节点地址放入哈希集合，再扫描第二条链表，第一个出现在集合中的节点就是交点，复杂度 $O(m+n)$ ，空间 $O(m)$ 。双指针优化的思路更巧：指针 A 走完第一条链后转到第二条链，指针 B 走完第二条链后转到第一条链。若两链相交，它们会在交点相遇；若不相交，最终同时到达空指针。

Listing 10.11 LeetCode 160 相交链表：双指针走完两条路径

```

1 ListNode* get_intersection_node(ListNode* first, ListNode* second)
2 {
3     ListNode* a = first;
4     ListNode* b = second;
5
6     while (a != b) {
7         a = a == nullptr ? second : a->next;
8         b = b == nullptr ? first : b->next;
9     }
10
11     return a;
12 }

```

为什么这段代码成立？设第一条链表独有长度为 x ，第二条独有长度为 y ，共享尾部长度为 z 。指针 A 走过的路径是 $x + z + y$ 后到达交点，指针 B 走过的路径是 $y + z + x$ 后到达交点，总长度相同。若没有共享尾部，两个指针都会在走完 $m+n$ 后变成空指针。这里不能比较节点值，因为不同节点可以有相同值；也不能修改链表结构，除非题目允许。

回文链表可以用数组保存所有值后双指针判断，时间 $O(n)$ 、空间 $O(n)$ 。如果要求 $O(1)$ 额外空间，就要找到中点、反转后半段、逐个比较，最后最好把链表恢复。这个题综合考察快慢指针和链表反转。快指针每次走两步，慢指针每次走一步；当快指针到达尾部时，慢指针在中点附近。奇数长度时，中间节点不影响回文判断，可以让后半段从慢指针之后开始。

Listing 10.12 LeetCode 234 回文链表：反转后半段后比较

```

1 ListNode* reverse_list(ListNode* head)
2 {
3     ListNode* previous = nullptr;
4     ListNode* current = head;
5     while (current != nullptr) {
6         ListNode* next = current->next;
7         current->next = previous;
8         previous = current;
9         current = next;
10    }
11    return previous;
12 }
13
14 bool is_palindrome_list(ListNode* head)
15 {
16     if (head == nullptr || head->next == nullptr) {
17         return true;
18     }
19
20     ListNode* slow = head;
21     ListNode* fast = head;

```



```

22     while (fast->next != nullptr && fast->next->next != nullptr) {
23         slow = slow->next;
24         fast = fast->next->next;
25     }
26
27     ListNode* second_half = reverse_list(slow->next);
28     ListNode* restore_head = second_half;
29     ListNode* first_half = head;
30     bool ok = true;
31
32     while (second_half != nullptr) {
33         if (first_half->value != second_half->value) {
34             ok = false;
35             break;
36         }
37         first_half = first_half->next;
38         second_half = second_half->next;
39     }
40
41     slow->next = reverse_list(restore_head);
42     return ok;
43 }

```

这段代码刻意恢复链表，因为真实工程里“检查是否回文”不应该悄悄改变输入结构。力扣题可能不检查恢复，但教材要训练副作用意识。中点循环使用 `fast->next != nullptr && fast->next->next != nullptr`，能让 `slow` 停在前半段尾部。比较时只遍历后半段长度即可。易错点是反转后半段之前没有保存头节点，导致无法恢复。

链表排序可以用归并排序。数组排序可以随机访问中点，链表不行；链表找中点要用快慢指针，并从中点断开。暴力做法是把节点值放进数组排序再写回链表，这改变的是值，不是节点顺序；如果节点携带更多字段或外部引用，这种做法就不等价。链表归并排序的优势是稳定、时间 $O(n \log n)$ ，额外空间如果用递归会有栈成本；为了符合工程中避免深递归的习惯，可以使用自底向上的迭代归并。

Listing 10.13 LeetCode 148 链表排序：自底向上迭代归并

```

1  ListNode* split_list(ListNode* head, int size)
2  {
3      for (int i = 1; head != nullptr && i < size; ++i) {
4          head = head->next;
5      }
6      if (head == nullptr) {
7          return nullptr;
8      }
9      ListNode* second = head->next;
10     head->next = nullptr;
11     return second;
12 }
13
14 ListNode* merge_sorted_lists(ListNode* first, ListNode* second, ListNode** tail)
15 {
16     ListNode dummy;
17     ListNode* current = &dummy;
18     while (first != nullptr && second != nullptr) {

```

```

19     if (first->value <= second->value) {
20         current->next = first;
21         first = first->next;
22     } else {
23         current->next = second;
24         second = second->next;
25     }
26     current = current->next;
27 }
28 current->next = first != nullptr ? first : second;
29 while (current->next != nullptr) {
30     current = current->next;
31 }
32 *tail = current;
33 return dummy.next;
34 }
35
36 ListNode* sort_list(ListNode* head)
37 {
38     int length = 0;
39     for (ListNode* node = head; node != nullptr; node = node->next) {
40         ++length;
41     }
42
43     ListNode dummy;
44     dummy.next = head;
45     for (int size = 1; size < length; size *= 2) {
46         ListNode* previous = &dummy;
47         ListNode* current = dummy.next;
48         while (current != nullptr) {
49             ListNode* first = current;
50             ListNode* second = split_list(first, size);
51             current = split_list(second, size);
52             ListNode* merged_tail = nullptr;
53             previous->next = merge_sorted_lists(first, second, &merged_tail);
54             previous = merged_tail;
55         }
56     }
57     return dummy.next;
58 }

```

自底向上归并的思想是先合并长度为 1 的有序段，再合并长度为 2、4、8 的段，直到段长覆盖整个链表。每次 `split_list` 都把一段切断，避免合并时越界串到后续段。代码较长，但它把递归栈换成了显式循环。面试中如果时间有限，可以先讲递归归并，再说明生产环境可改成迭代版本。这里的质量点不是炫技，而是知道链表排序的节点重连语义和递归深度风险。

删除排序链表中的重复元素也有两个版本。简单版保留一个重复值，只要当前节点和下一个节点值相等，就跳过下一个节点。进阶版删除所有重复值，只保留原链表中没有重复出现的值；这时必须使用虚拟头节点和前驱指针，因为重复段可能从头节点开始。先识别一整段相等值，再决定是保留还是跳过整段。

Listing 10.14 LeetCode 82 删除排序链表中的重复元素 II：识别重复段后整体跳过

```

1 ListNode* delete_duplicates_all(ListNode* head)

```

```

2 {
3     ListNode dummy;
4     dummy.next = head;
5     ListNode* previous = &dummy;
6     ListNode* current = head;
7
8     while (current != nullptr) {
9         bool duplicated = false;
10        while (current->next != nullptr &&
11               current->value == current->next->value) {
12            duplicated = true;
13            current = current->next;
14        }
15        if (duplicated) {
16            previous->next = current->next;
17        } else {
18            previous = previous->next;
19        }
20        current = current->next;
21    }
22
23    return dummy.next;
24 }

```

这题的重点是 `previous` 的含义：它始终指向结果链表的尾部，也就是最后一个确认保留的节点。遇到重复段时，`previous` 不动，只修改它的后继；遇到非重复节点时，`previous` 才前进。边界包括全是重复值、头部重复、尾部重复、没有重复。链表题里，前驱指针比当前指针更重要，因为删除动作发生在前驱的 `next` 上。

路径简化是栈在字符串解析中的应用。Unix 路径中，多个斜杠等价于一个斜杠，`.` 表示当前目录，`..` 表示回到上一级目录。暴力按字符处理容易被连续斜杠和边界搞乱。更清晰的做法是把路径按斜杠切分成段，栈保存有效目录名：普通目录入栈，`..` 弹出一个目录，空段和 `.` 忽略。最后用栈中目录重建规范路径。

Listing 10.15 LeetCode 71 简化路径：栈保存有效目录段

```

1 std::string simplify_path(const std::string& path)
2 {
3     std::vector<std::string> stack;
4     int index = 0;
5     while (index < static_cast<int>(path.size())) {
6         while (index < static_cast<int>(path.size()) && path[index] == '/') {
7             ++index;
8         }
9         int start = index;
10        while (index < static_cast<int>(path.size()) && path[index] != '/') {
11            ++index;
12        }
13        const std::string part = path.substr(start, index - start);
14        if (part.empty() || part == ".") {
15            continue;
16        }
17        if (part == "..") {
18            if (!stack.empty()) {

```

```

19         stack.pop_back();
20     }
21     } else {
22         stack.push_back(part);
23     }
24 }
25
26 if (stack.empty()) {
27     return "/";
28 }
29 std::string result;
30 for (const std::string& part : stack) {
31     result += "/";
32     result += part;
33 }
34 return result;
35 }

```

这里的“栈”不是必须用 `std::stack`，用 `vector` 更方便遍历重建路径。容器选择要服务于操作：我们需要尾部压入弹出，也需要最后从头到尾遍历，所以 `vector` 很合适。若路径很长，频繁字符串拼接可以用 `ostringstream` 或先估算长度，但刷题中清晰优先。易错点是根目录下遇到 `..` 不能继续向上，只能保持根。

括号相关题还会扩展为“最长有效括号”。暴力枚举所有子串再判断是否合法，复杂度高。栈做法保存下标，而不是保存字符。栈底保存最近一个无法匹配的位置，作为当前有效段的左边界。遇到左括号入栈；遇到右括号先弹出一个左括号，若弹出后栈空，说明当前右括号无法匹配，它的下标作为新边界；否则当前有效长度是当前下标减栈顶下标。

Listing 10.16 LeetCode 32 最长有效括号：栈保存边界下标

```

1 int longest_valid_parentheses(const std::string& s)
2 {
3     std::vector<int> stack;
4     stack.push_back(-1);
5     int answer = 0;
6
7     for (int i = 0; i < static_cast<int>(s.size()); ++i) {
8         if (s[i] == '(') {
9             stack.push_back(i);
10            continue;
11        }
12
13        stack.pop_back();
14        if (stack.empty()) {
15            stack.push_back(i);
16        } else {
17            answer = std::max(answer, i - stack.back());
18        }
19    }
20
21    return answer;
22 }

```

这个题中 `-1` 是哨兵边界，表示在字符串开始之前有一个不可跨越位置。栈顶在计算长度时不

是当前有效段的第一个字符，而是当前有效段左侧的边界。这个语义非常重要。若只保存括号字符，就无法计算长度；若只保存左括号数量，就无法处理中间断裂。单调栈、括号栈、路径栈都在提醒同一件事：栈里保存的不是“看起来像栈的东西”，而是后续计算真正需要的历史状态。

队列题的进阶点是层次和边界。普通 BFS 中队列保存待扩展节点；若需要输出每层，就在每轮开始记录队列大小；若需要最短步数，就把层数和状态一起维护，或每处理一层步数加一。双端队列出现在两类场景：一类是单调队列维护窗口候选，另一类是 0-1 BFS 中根据边权把状态放队首或队尾。它们都不是“队列模板”，而是对候选优先级的精确表达。

线性结构实验：第一，用同一组链表节点分别测试删除倒数节点、回文判断、排序，检查函数是否意外破坏输入。第二，把最长有效括号的栈内容逐步打印出来，观察栈顶为何是边界下标。第三，分别用数组扫描、单调队列解决滑动窗口最大值，统计窗口大小增大时的运行差异。第四，给路径简化加入连续斜杠、根目录回退、隐藏文件名等测试，确认规则没有被字符串细节干扰。

线性结构题的统一问题

链表问“哪条边会断，断之前有没有保存”；栈问“哪个历史状态最近且必须先解决”；队列问“当前层或当前窗口的边界在哪里”；单调结构问“哪个候选被新元素支配，为什么永远不可能再成为答案”。只要这四个问题能回答，线性结构题就不再是零散模板。

第 11 章

排序、选择和区间问题

排序不是为了把数组排好看，而是为了制造结构。无序输入里，两个元素之间没有稳定关系；排序之后，大小关系变成单调关系，重复元素挨在一起，区间端点可以按起点或终点扫描，双指针和二分才有施展空间。面试中很多题的关键不是“调用 `std::sort`”，而是说明排序之后哪个约束变简单了，排序代价是否值得，以及排序是否会破坏题目要求的原始下标或稳定性。

C++ 的 `std::sort` 通常是内省排序，综合快速排序、堆排序和插入排序思想，平均和最坏复杂度都能控制在 $O(n \log n)$ 级别。刷题时可以放心使用它，但要理解比较器必须满足严格弱序。比较器不能写成 `a <= b`，也不能依赖会变化的外部状态，否则排序行为未定义。对结构体排序时，比较器应该只表达排序键，例如先按起点升序，再按终点升序。

Listing 11.1 区间排序比较器：严格弱序

```
1 struct Interval {
2     int start = 0;
3     int end = 0;
4 };
5
6 void sort_intervals(std::vector<Interval>& intervals)
7 {
8     std::sort(intervals.begin(), intervals.end(),
9               [](const Interval& lhs, const Interval& rhs) {
10                 if (lhs.start != rhs.start) {
11                     return lhs.start < rhs.start;
12                 }
13                 return lhs.end < rhs.end;
14             });
15 }
```

这个比较器在 `lhs.start == rhs.start && lhs.end == rhs.end` 时返回 `false`，满足严格弱序要求。很多线上题数据小，错误比较器也可能侥幸通过，但这是 C++ 基本功问题。排序后，如果题目仍需要原始下标，就不能只排序值；要把值和下标一起保存成 `pair` 或结构体。

颜色分类是荷兰国旗问题。题目只包含 0、1、2 三种值，要求原地排序。直接调用排序当然能过，复杂度 $O(n \log n)$ ；计数排序也能 $O(n)$ ，但需要两次扫描。更经典的一次扫描做法是维护三个区域：左侧全是 0，中间是已经确认的 1，右侧全是 2，未知区域在中间游动。指针 `low` 指向下一个 0 应放的位置，`mid` 扫描未知区域，`high` 指向下一个 2 应放的位置。

Listing 11.2 LeetCode 75 颜色分类：三指针维护四段区域

```
1 void sort_colors(std::vector<int>& nums)
2 {
3     constexpr int RED = 0;
4     constexpr int WHITE = 1;
5     constexpr int BLUE = 2;
6
7     int low = 0;
8     int mid = 0;
9     int high = static_cast<int>(nums.size()) - 1;
10 }
```

```

11 while (mid <= high) {
12     if (nums[mid] == RED) {
13         std::swap(nums[low], nums[mid]);
14         ++low;
15         ++mid;
16     } else if (nums[mid] == WHITE) {
17         ++mid;
18     } else if (nums[mid] == BLUE) {
19         std::swap(nums[mid], nums[high]);
20         --high;
21     }
22 }
23 }

```

这题最容易错在交换 2 之后是否移动 `mid`。答案是不能移动，因为从 `high` 换过来的元素还没有检查，可能是 0、1 或 2。交换 0 后可以移动 `mid`，因为 `low` 左边已经处理完，换到 `mid` 的元素来自已确认的 1 区或当前位置本身。这个差别正是不变量的体现：`[0, low)` 是 0，`[low, mid)` 是 1，`(high, n)` 是 2，`[mid, high]` 未知。

合并区间展示排序如何把二维关系变成线性扫描。暴力方法是反复找任意两个相交区间合并，直到不能合并为止；每次合并后都可能重新扫描全部区间，复杂度很高。按起点排序后，相交关系只需要和当前结果的最后一个区间比较。因为后续区间起点只会越来越大，如果它不能和最后区间相交，就不可能和更早已经结束的区间相交。

Listing 11.3 LeetCode 56 合并区间：排序后只看结果尾部

```

1 std::vector<std::vector<int>> merge_intervals(
2     std::vector<std::vector<int>> intervals)
3 {
4     if (intervals.empty()) {
5         return {};
6     }
7
8     std::sort(intervals.begin(), intervals.end(),
9         [](const std::vector<int>& lhs, const std::vector<int>& rhs) {
10             return lhs[0] < rhs[0];
11         });
12
13     std::vector<std::vector<int>> merged;
14     merged.push_back(intervals.front());
15
16     for (int i = 1; i < static_cast<int>(intervals.size()); ++i) {
17         std::vector<int>& last = merged.back();
18         const std::vector<int>& current = intervals[i];
19         if (current[0] <= last[1]) {
20             last[1] = std::max(last[1], current[1]);
21         } else {
22             merged.push_back(current);
23         }
24     }
25
26     return merged;
27 }

```

这里参数按值传入，是因为函数内部排序会改变区间顺序。如果调用者不需要保留原顺序，传引用也可以。复杂度是排序 $O(n \log n)$ 加扫描 $O(n)$ 。易错点是相交条件：如果区间是闭区间，`current[0] <= last[1]` 表示相交或接触；如果题目定义半开区间，就要重新审视等号是否成立。区间问题一定要先明确端点语义。

插入区间与合并区间类似，但输入已经有序且不重叠。暴力方法可以把新区间放进去重新排序再合并，复杂度仍可接受，但没有利用已有条件。线性做法分三段：先加入所有结束在新区间之前的区间；再把所有和新区间相交的区间合并进新区间；最后加入剩余区间。这种写法体现了区间扫描的分段思维。

Listing 11.4 LeetCode 57 插入区间：前段、中段、后段

```

1 std::vector<std::vector<int>> insert_interval(
2     const std::vector<std::vector<int>>& intervals,
3     std::vector<int> new_interval)
4 {
5     std::vector<std::vector<int>> answer;
6     int index = 0;
7     const int n = static_cast<int>(intervals.size());
8
9     while (index < n && intervals[index][1] < new_interval[0]) {
10         answer.push_back(intervals[index]);
11         ++index;
12     }
13
14     while (index < n && intervals[index][0] <= new_interval[1]) {
15         new_interval[0] = std::min(new_interval[0], intervals[index][0]);
16         new_interval[1] = std::max(new_interval[1], intervals[index][1]);
17         ++index;
18     }
19     answer.push_back(new_interval);
20
21     while (index < n) {
22         answer.push_back(intervals[index]);
23         ++index;
24     }
25
26     return answer;
27 }

```

这题的优化条件是“原区间已经按起点排序且互不重叠”。如果这个条件不存在，就必须先排序或换模型。第一段的判断是 `intervals[index][1] < new_interval[0]`，说明当前区间完全在新区间左边；第二段的判断是 `intervals[index][0] <= new_interval[1]`，说明存在重叠。两个等号同样取决于闭区间语义。面试中把三段说清楚，代码通常很自然。

无重叠区间是区间贪心题。题目要求删除最少区间，使剩余区间互不重叠。等价地，可以保留最多数量的不重叠区间。暴力方法是枚举所有区间子集，检查是否互不重叠，复杂度指数级。贪心优化依据是：每次选择结束最早的区间，会给后面留下最大的空间。按结束时间排序，能选就选，不能选就跳过。

Listing 11.5 LeetCode 435 无重叠区间：按结束时间贪心保留最多区间

```

1 int erase_overlap_intervals(std::vector<std::vector<int>> intervals)
2 {
3     if (intervals.empty()) {
4         return 0;

```

```

5     }
6
7     std::sort(intervals.begin(), intervals.end(),
8             [](const std::vector<int>& lhs, const std::vector<int>& rhs) {
9                 return lhs[1] < rhs[1];
10            });
11
12     int kept = 0;
13     int current_end = std::numeric_limits<int>::min();
14     for (const std::vector<int>& interval : intervals) {
15         if (interval[0] >= current_end) {
16             ++kept;
17             current_end = interval[1];
18         }
19     }
20
21     return static_cast<int>(intervals.size()) - kept;
22 }

```

这个贪心的交换论证是：假设某个最优解第一步选择了一个结束更晚的区间，而存在另一个兼容区间结束更早。把最优解里的第一个区间替换成结束更早的区间，不会减少后续可选空间，因此不会让答案变差。反复替换，就得到按结束时间贪心的解。区间贪心题不要只说“排序后贪心”，必须说明排序键为什么是结束时间而不是开始时间。

数组中的第 K 大元素可以排序解决，时间 $O(n \log n)$ 。如果只需要第 K 大，不需要完整有序，快速选择更合适。它借鉴快速排序的分区思想：选择一个枢轴，把大于枢轴的放左边，小于枢轴的放右边；如果枢轴最终位置正好是 $k - 1$ ，答案找到；如果位置偏左或偏右，只在一侧继续。平均复杂度 $O(n)$ 。

Listing 11.6 LeetCode 215 数组中的第 K 个最大元素：迭代快速选择

```

1 int find_kth_largest_quickselect(std::vector<int> nums, int k)
2 {
3     const int target = k - 1;
4     int left = 0;
5     int right = static_cast<int>(nums.size()) - 1;
6
7     while (left <= right) {
8         const int pivot = nums[right];
9         int store = left;
10        for (int i = left; i < right; ++i) {
11            if (nums[i] > pivot) {
12                std::swap(nums[store], nums[i]);
13                ++store;
14            }
15        }
16        std::swap(nums[store], nums[right]);
17
18        if (store == target) {
19            return nums[store];
20        }
21        if (store < target) {
22            left = store + 1;
23        } else {

```

```

24         right = store - 1;
25     }
26 }
27
28 return nums[target];
29 }

```

这里按值传参是刻意的，因为快速选择会重排数组。如果题目允许修改输入，可以传引用减少一次复制。代码使用最后一个元素作枢轴，最坏情况下会退化到 $O(n^2)$ ；工业实现通常会随机化枢轴或使用更稳健的选择算法。面试中要主动说出这个风险。快速选择的正确性来自分区后的位置含义：左侧元素都比枢轴大，因此枢轴排名已经确定，不需要管左右两侧内部是否有序。

排序和选择题经常和去重绑定。三数之和先排序，是为了让固定一个数后，剩余两数可以用双指针线性查找；合并区间先排序，是为了让所有可能相交的区间在扫描时相邻；第 K 大使用分区，是为了只确定目标排名而不完整排序。三者共同点是：排序或分区不是目的，而是让后续操作只看局部信息。

归并排序在刷题里经常不是为了排序本身，而是为了在“合并两个有序段”的过程中统计跨区间关系。逆序对就是典型例子。暴力方法枚举所有 $i < j$ ，检查 $\text{nums}[i] > \text{nums}[j]$ ，复杂度 $O(n^2)$ 。归并排序把数组分成左右两半，递归意义上左右内部逆序对已经统计完；合并时，只需要统计左半元素和右半元素形成的跨区间逆序对。由于左右两边已经有序，若 $\text{left}[i] > \text{right}[j]$ ，那么从 i 到左半末尾的所有元素都大于 $\text{right}[j]$ 。

Listing 11.7 逆序对：归并过程中统计跨区间关系

```

1 long long count_reverse_pairs(std::vector<int> nums)
2 {
3     std::vector<int> buffer(nums.size());
4     long long answer = 0;
5
6     for (int width = 1; width < static_cast<int>(nums.size()); width *= 2) {
7         for (int left = 0; left < static_cast<int>(nums.size()); left += 2 * width) {
8             const int mid = std::min(left + width, static_cast<int>(nums.size()));
9             const int right = std::min(left + 2 * width, static_cast<int>(nums.size())↵
↵ ));
10
11             int i = left;
12             int j = mid;
13             int out = left;
14             while (i < mid && j < right) {
15                 if (nums[i] <= nums[j]) {
16                     buffer[out++] = nums[i++];
17                 } else {
18                     answer += mid - i;
19                     buffer[out++] = nums[j++];
20                 }
21             }
22             while (i < mid) {
23                 buffer[out++] = nums[i++];
24             }
25             while (j < right) {
26                 buffer[out++] = nums[j++];
27             }
28             for (int k = left; k < right; ++k) {
29                 nums[k] = buffer[k];

```



```

30     }
31 }
32 }
33
34 return answer;
35 }

```

这里使用自底向上的迭代归并，避免递归。`width` 表示当前已经有序的段长度，每轮把相邻两段合并成更长有序段。统计逆序对的关键行是 `answer += mid - i`：当右侧当前元素更小，左侧从 `i` 到 `mid - 1` 的所有元素都会和它形成逆序对。这个推理依赖左半段已经有序。归并类统计题的复盘要问：跨区间关系能不能在两个有序段合并时一次性数出来？如果能，通常可以从平方复杂度降到 $O(n \log n)$ 。

会议室问题把区间排序和堆结合起来。会议室 I 问一个人能否参加所有会议，只要按开始时间排序，检查相邻会议是否重叠即可。会议室 II 问至少需要多少会议室，就不能只看相邻会议，因为多个会议可能同时进行。暴力方法对每个会议扫描当前所有房间，找能复用的房间；优化方法用小顶堆保存每个房间当前会议的结束时间。新会议开始时，如果最早结束的房间已经空出来，就复用它；否则需要新房间。

Listing 11.8 LeetCode 253 会议室 II：小顶堆保存房间结束时间

```

1 int min_meeting_rooms(std::vector<std::vector<int>> intervals)
2 {
3     if (intervals.empty()) {
4         return 0;
5     }
6
7     std::sort(intervals.begin(), intervals.end(),
8               [](const std::vector<int>& lhs, const std::vector<int>& rhs) {
9                 return lhs[0] < rhs[0];
10            });
11
12     std::priority_queue<int, std::vector<int>, std::greater<int>> endings;
13     for (const std::vector<int>& meeting : intervals) {
14         if (!endings.empty() && endings.top() <= meeting[0]) {
15             endings.pop();
16         }
17         endings.push(meeting[1]);
18     }
19
20     return static_cast<int>(endings.size());
21 }

```

堆顶表示当前所有占用房间中最早结束的会议。如果它都晚于新会议开始，那么其他房间结束更晚，也不能复用；如果它不晚于新会议开始，就弹出并把该房间的新结束时间压入。复杂度是排序 $O(n \log n)$ 加每个会议一次堆操作。注意这里一次只弹出一个结束时间就够了，因为每个新会议只需要一个房间；如果题目问某个时间点同时进行会议数量，可能会弹出所有已结束会议。

差分数组是区间更新题的另一种排序替代思路。如果题目反复对区间加值，再询问每个位置最终值，暴力对每次更新逐个位置加，复杂度 $O(qn)$ 。差分数组保存相邻元素变化量：对区间 `[left, right]` 加 `value`，只需要 `diff[left] += value`, `diff[right + 1] -= value`。最后做一次前缀和还原所有位置。

Listing 11.9 LeetCode 1109 航班预订统计：差分数组处理区间加法

```

1 std::vector<int> corp_flight_bookings(const std::vector<std::vector<int>>& bookings,
2                                     int n)
3 {
4     std::vector<int> diff(n + 1, 0);
5     for (const std::vector<int>& booking : bookings) {
6         const int left = booking[0] - 1;
7         const int right = booking[1] - 1;
8         const int seats = booking[2];
9         diff[left] += seats;
10        if (right + 1 < n) {
11            diff[right + 1] -= seats;
12        }
13    }
14
15    std::vector<int> answer(n, 0);
16    int current = 0;
17    for (int i = 0; i < n; ++i) {
18        current += diff[i];
19        answer[i] = current;
20    }
21    return answer;
22 }

```

差分正确性来自“影响从 left 开始，在 right 之后结束”。它把区间内每个点都加的操作，变成两个边界事件。扫描线也有类似思想：把区间起点看成 +1 事件，终点看成 -1 事件，排序后扫描事件就能得到当前重叠数量。差分适合坐标连续且范围可开数组的场景；扫描线适合坐标很大或稀疏，需要排序事件的场景。

计数排序和桶排序利用值域信息。若数组长度很大但值域很小，比较排序的 $O(n \log n)$ 下界不再是必须承受的，因为我们不靠两两比较区分所有排列，而是统计每个值出现次数。颜色分类可以看成值域只有 3 的计数排序变体；前 K 高频元素也可以在统计频次后按频次分桶。桶排序的风险是空间由值域或频次范围决定，不能在值域巨大时盲目开数组。

Listing 11.10 LeetCode 347 前 K 高频元素：桶按频次收集

```

1 std::vector<int> top_k_frequent_bucket(const std::vector<int>& nums, int k)
2 {
3     std::unordered_map<int, int> frequency;
4     frequency.reserve(nums.size());
5     for (const int x : nums) {
6         ++frequency[x];
7     }
8
9     std::vector<std::vector<int>> buckets(nums.size() + 1);
10    for (const auto& [value, count] : frequency) {
11        buckets[count].push_back(value);
12    }
13
14    std::vector<int> answer;
15    for (int count = static_cast<int>(buckets.size()) - 1;
16         count >= 0 && static_cast<int>(answer.size()) < k;
17         --count) {
18        for (const int value : buckets[count]) {
19            answer.push_back(value);

```

```

20         if (static_cast<int>(answer.size()) == k) {
21             break;
22         }
23     }
24 }
25 return answer;
26 }

```

这个版本的时间复杂度是 $O(n)$ ，空间复杂度也是 $O(n)$ ，因为最大频次不超过数组长度。它比堆更快吗？理论上是线性的，但常数和内存占用更高，且输出顺序不固定。实际面试可以先讲堆，因为堆适用范围更广；再补桶排序作为值域或频次范围明确时的优化。高质量答案不是只追求最低复杂度，而是解释方案适用条件。

相对排序数组展示“值域有限时不一定要比较排序”。题目给两个数组，第二个数组给出部分元素的相对顺序，要求第一个数组中这些元素按第二个数组顺序排列，其余元素升序排列。暴力做法可以写自定义比较器：若两个元素都在第二个数组中，就比较它们的排名；若只有一个在第二个数组中，它排前面；否则按值升序。这个做法清晰，但每次比较都要查询排名，复杂度仍是排序的 $O(n \log n)$ 。如果值域较小，可以计数后按规则输出，复杂度接近线性。

Listing 11.11 LeetCode 1122 数组的相对排序：计数后按给定顺序输出

```

1 std::vector<int> relative_sort_array(const std::vector<int>& arr1,
2                                     const std::vector<int>& arr2)
3 {
4     constexpr int MAX_VALUE = 1000;
5     std::array<int, MAX_VALUE + 1> count{};
6     for (const int x : arr1) {
7         ++count[x];
8     }
9
10    std::vector<int> answer;
11    answer.reserve(arr1.size());
12    for (const int x : arr2) {
13        while (count[x] > 0) {
14            answer.push_back(x);
15            --count[x];
16        }
17    }
18    for (int value = 0; value <= MAX_VALUE; ++value) {
19        while (count[value] > 0) {
20            answer.push_back(value);
21            --count[value];
22        }
23    }
24    return answer;
25 }

```

这段代码的前提是题目给定值域不超过 1000，所以 `MAX_VALUE` 是题目约束，不是随便拍脑袋。若值域很大，就不能开这么大的数组，应改用哈希表统计，再对剩余元素排序。这个题训练的是“比较排序下界的适用范围”：当元素值域和输出顺序被额外约束时，我们可以不通过比较确定所有相对顺序。

按奇偶排序、按自定义规则排序、把负数放前面等题，看起来简单，其实都在考“稳定性”。如果题目要求保持同类元素的原始相对顺序，就需要稳定划分，可以新开数组，或者用 `stable_partition`。如果不要稳定，就可以用左右指针原地交换。原地不稳定划分的复杂度低、空间少，但会改变相

对顺序；稳定划分保留顺序，但通常需要额外空间或更复杂的移动。面试中主动问清稳定性，是比直接写代码更成熟的表现。

Listing 11.12 LeetCode 283 移动零：稳定划分保持相对顺序

```

1 std::vector<int> stable_partition_by_sign(const std::vector<int>& nums)
2 {
3     std::vector<int> answer;
4     answer.reserve(nums.size());
5     for (const int x : nums) {
6         if (x < 0) {
7             answer.push_back(x);
8         }
9     }
10    for (const int x : nums) {
11        if (x >= 0) {
12            answer.push_back(x);
13        }
14    }
15    return answer;
16 }

```

这不是最省空间的写法，但语义非常清晰。若题目必须原地且稳定，问题会变难，因为把一个负数移动到前面可能要整体搬移中间元素。很多面试题默认不要求稳定，允许左右指针交换；但如果业务语义中原始顺序有意义，例如日志时间顺序、用户输入顺序，就不能随便破坏。排序和划分题一定要把“允许改变什么”说清楚。

数组中的逆序对除了归并计数，还可以用树状数组。归并排序从“跨左右两半的逆序对”出发；树状数组从“扫描到当前数时，之前有多少数比它大”出发。由于数值可能很大或为负，需要先离散化，把值映射到排名。扫描数组时，查询已经出现过的元素总数减去小于等于当前排名的数量，就是以当前元素为右端的逆序对数量；然后把当前排名出现次数加一。

Listing 11.13 逆序对：离散化加树状数组

```

1 class FenwickTree {
2 public:
3     explicit FenwickTree(int size) : tree_(size + 1, 0) {}
4
5     void add(int index, int delta)
6     {
7         for (int i = index; i < static_cast<int>(this->tree_.size());
8             i += i & -i) {
9             this->tree_[i] += delta;
10        }
11    }
12
13    int sum(int index) const
14    {
15        int result = 0;
16        for (int i = index; i > 0; i -= i & -i) {
17            result += this->tree_[i];
18        }
19        return result;
20    }
21 }

```

```

22 private:
23     std::vector<int> tree_;
24 };
25
26 long long reverse_pairs_by_fenwick(const std::vector<int>& nums)
27 {
28     std::vector<int> values = nums;
29     std::sort(values.begin(), values.end());
30     values.erase(std::unique(values.begin(), values.end()), values.end());
31
32     FenwickTree tree(static_cast<int>(values.size()));
33     long long answer = 0;
34     for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
35         const int rank = static_cast<int>(
36             std::lower_bound(values.begin(), values.end(), nums[i]) -
37             values.begin()) + 1;
38         const int seen = i;
39         const int not_greater = tree.sum(rank);
40         answer += seen - not_greater;
41         tree.add(rank, 1);
42     }
43     return answer;
44 }

```

树状数组适合动态维护前缀计数。它的下标从 1 开始， i & $-i$ 表示当前节点覆盖区间的长度。这个结构比归并计数更抽象，但能迁移到很多“动态排名、区间频次、前缀和修改查询”问题。面试中如果只考逆序对，归并排序更容易解释；如果题目有多次更新或在线查询，树状数组更有优势。这里再次体现：排序不是唯一制造顺序的方法，离散化加索引树也能制造排名结构。

扫描线是区间问题的重要模型。区间重叠数量、天际线、会议室、航班预订，都可以看成一系列事件按坐标发生。区间开始是一个加事件，区间结束是一个减事件。排序事件后从左到右扫描，当前累计值就是当前位置的活跃区间数量。对于闭区间和半开区间，开始和结束在同一坐标时的处理顺序不同；例如会议室通常认为一个会议在时间 t 结束，另一个会议在时间 t 开始可以复用同一房间，所以结束事件应该先于开始事件处理。

Listing 11.14 会议室数量：扫描线事件排序

```

1 int min_meeting_rooms_by_events(const std::vector<std::vector<int>>& intervals)
2 {
3     std::vector<std::pair<int, int>> events;
4     events.reserve(intervals.size() * 2);
5     for (const std::vector<int>& interval : intervals) {
6         events.emplace_back(interval[0], 1);
7         events.emplace_back(interval[1], -1);
8     }
9
10    std::sort(events.begin(), events.end(),
11              [](const auto& lhs, const auto& rhs) {
12                  if (lhs.first != rhs.first) {
13                      return lhs.first < rhs.first;
14                  }
15                  return lhs.second < rhs.second;
16              });
17

```



```

18     int active = 0;
19     int answer = 0;
20     for (const auto [time, delta] : events) {
21         (void)time;
22         active += delta;
23         answer = std::max(answer, active);
24     }
25     return answer;
26 }

```

这里 `-1` 事件在同一时间排在 `+1` 前面，所以会议结束先释放房间。若题目是闭区间覆盖点，结束和开始的同点处理可能要反过来。扫描线的难点不是代码，而是事件语义：一个事件表示影响从哪里开始，在哪里结束，边界是否包含。差分数组是坐标连续且范围可开数组时的扫描线；事件排序是坐标稀疏或很大时的扫描线。

用最少数引爆气球是区间贪心。每个气球是一个水平区间，一支箭射在某个坐标可以引爆所有覆盖该坐标的气球。暴力想法是尝试每个可能射击位置，但坐标范围巨大。排序后贪心：按右端点升序排列，第一支箭射在第一个气球的右端点，这个选择最不容易影响后续，因为它尽可能靠右，同时仍能引爆当前气球。之后扫描气球，若当前气球起点大于当前箭位置，说明必须新射一箭。

Listing 11.15 LeetCode 452 最少箭引爆气球：按右端点贪心

```

1 int find_min_arrow_shots(std::vector<std::vector<int>> points)
2 {
3     if (points.empty()) {
4         return 0;
5     }
6     std::sort(points.begin(), points.end(),
7               [](const std::vector<int>& lhs, const std::vector<int>& rhs) {
8                 return lhs[1] < rhs[1];
9             });
10
11     int arrows = 1;
12     int arrow_position = points[0][1];
13     for (int i = 1; i < static_cast<int>(points.size()); ++i) {
14         if (points[i][0] <= arrow_position) {
15             continue;
16         }
17         ++arrows;
18         arrow_position = points[i][1];
19     }
20     return arrows;
21 }

```

正确性可以用交换论证说明：对于最早结束的气球，任何可行解都必须有一支箭射在它的区间内。把这支箭移动到该气球右端点，不会让它失去对已覆盖气球的覆盖能力，反而给后续区间留下更大机会。因此选择最早右端点是安全的。这个思路也适用于无重叠区间中“保留最多区间”：按结束时间选择最早结束的区间，给后面留下空间。

区间列表交集是双指针题。两个区间列表各自有序且互不重叠。暴力两两比较复杂度 $O(mn)$ 。双指针优化来自有序性：当前两个区间的交集左端是较大起点，右端是较小终点；若左端不大于右端，就有交集。然后谁的终点更早，谁就不可能再和对方后续区间相交，移动谁的指针。

Listing 11.16 LeetCode 986 区间列表交集：移动较早结束的一方

```

1 std::vector<std::vector<int>> interval_intersection(

```

```

2  const std::vector<std::vector<int>>& first,
3  const std::vector<std::vector<int>>& second)
4  {
5      std::vector<std::vector<int>> answer;
6      int i = 0;
7      int j = 0;
8      while (i < static_cast<int>(first.size()) &&
9              j < static_cast<int>(second.size())) {
10         const int left = std::max(first[i][0], second[j][0]);
11         const int right = std::min(first[i][1], second[j][1]);
12         if (left <= right) {
13             answer.push_back({left, right});
14         }
15
16         if (first[i][1] < second[j][1]) {
17             ++i;
18         } else {
19             ++j;
20         }
21     }
22     return answer;
23 }

```

这题的推理比代码重要：终点较早的区间在当前比较后已经没有机会和另一个当前区间或它后面的区间形成新的交集，因为后续区间起点只会更大。因此可以安全丢弃它。这个“丢弃谁”的逻辑和合并区间、插入区间、会议室复用都属于同一类端点单调推理。

日志重排和版本号比较这类题看起来不是传统算法，但同样考排序规则。日志重排要求字母日志排在数字日志前，字母日志按内容排序，内容相同按标识符排序，数字日志保持原始相对顺序。若直接写比较器处理数字日志之间返回 `false`，还需要使用稳定排序才能保持它们原顺序；或者把字母日志单独取出排序，再追加数字日志。这里的关键是理解 `std::sort` 不稳定，不能指望相等元素原顺序不变。

常见误区

排序比较器有一个底线：不能让两个元素互相都“小于”等关系成立，也不能让相等元素比较返回真。比较器依赖外部可变状态、随机数、递增计数器，都会破坏排序算法假设。若题目要求保持相对顺序，请使用稳定排序或显式分组，不要假设普通排序“碰巧没改顺序”。

区间和排序题的测试要覆盖边界语义。合并区间要测相邻是否合并、完全包含、乱序输入；会议室要测一个会议结束另一个会议开始；最少箭要测端点相同的大坐标；快速选择要测重复值、K 为 1、K 为数组长度；计数排序要测值域边界。很多排序题代码看起来一行 `sort` 很短，但真正的错误都藏在比较器和边界定义里。

排序区间实验：第一，把会议室问题分别用堆和扫描线写一遍，用同一组边界用例比较答案。第二，把无重叠区间、最少箭、会议安排放在一起复盘，写出它们为什么都偏好按右端点排序。第三，给相对排序构造值域很大但元素很少的用例，比较计数数组和哈希统计的空间差异。第四，故意写一个 `a <= b` 的错误比较器，观察小数据可能不报错但语义已经未定义。

排序题复盘模板

先写清楚排序键，再说明排序后获得了什么性质：相同值相邻、端点单调、排名确定、候选可淘汰，还是可以双指针。然后说明是否需要保留原下标，比较器是否满足严格弱序，

排序是否改变输入。最后比较完整排序、堆、快速选择、计数排序等方案的适用范围。能说清这些，排序题才不是“先 sort 一下试试”。

排序、选择和分治的系统讲解

排序题表面是在排列元素，实际是在制造顺序信息。没有顺序时，两数比较、区间合并、去重、前驱后继都可能需要线性寻找；排序以后，许多关系会变成相邻关系或单调关系。三数之和能够从三层枚举降到固定一数加双指针，是因为排序把剩余两数的和变成可单调调整的量。合并区间能够线性扫描，是因为按起点排序后，所有可能与当前结果尾部相交的区间都会连续出现。排序不是免费的，复杂度通常是 $O(n \log n)$ ，所以必须说明排序换来了什么查询能力。

比较排序的下界来自信息论： n 个不同元素有 $n!$ 种可能顺序，比较一次最多提供一位左右的信息，因此最坏至少需要数量级为 $n \log n$ 的比较。这个结论告诉我们，想突破 $n \log n$ ，必须利用额外条件，例如值域有限、元素是整数、只需要第 K 个而不是完整顺序、或者允许随机化平均复杂度。计数排序利用小值域，桶排序利用分布，基数排序利用数字位，快速选择利用只关心目标排名。这些算法不是比标准排序更高级，而是使用了更强前提。

快速排序的核心是 partition：选择一个枢轴，把小于它的元素放一边，大于它的元素放另一边。partition 完成后，枢轴位置已经确定，左右两边再分别排序。快速选择只进入目标排名所在的一边，因此平均时间为线性。这里的平均性依赖枢轴足够随机；若每次都选到极端枢轴，最坏会退化到平方。面试中写快速选择必须主动说明随机化或三数取中，不然复杂度承诺不完整。

归并排序和快速排序的思维不同。归并排序先分再合，合并两个有序段时保证稳定性，最坏复杂度稳定为 $O(n \log n)$ ，但需要额外空间。链表排序常用归并，因为链表合并不需要随机访问，节点重连成本低；数组排序常用 introsort 一类混合策略，在快速排序、堆排序、插入排序之间切换，兼顾平均性能和最坏情况。理解这些取舍后，才能解释为什么 C++ 的 `std::sort` 不承诺稳定，而 `std::stable_sort` 需要更多空间。

选择第 K 大时有三种主线。完整排序最简单，适合数据量不大或后续还需要有序结果；大小为 K 的堆适合 K 远小于 n 或数据流持续到来；快速选择适合一次性数组、需要平均线性。三种方法没有绝对优劣，差别在输出需求、数据是否在线、实现风险和最坏复杂度。高质量题解应该比较它们，而不是只写一种。

排序题的边界往往出在比较器。比较器必须满足严格弱序，不能写出自相矛盾的关系。区间覆盖题中同起点要按终点降序，合并区间按起点升序即可，会议室按开始时间或结束时间有不同含义。比较器写错时，代码看似能跑，但排序结果不满足算法不变量。每次写比较器都要用三组数据验证：相同主键、完全相同元素、主键相反顺序。

实验建议：实现完整排序、大小为 K 的堆、快速选择三种第 K 大算法，用随机数组、几乎有序数组、逆序数组和大量重复数组测试。记录比较次数或运行时间，观察 K 很小时堆的优势、需要完整有序时排序的优势、随机化快速选择的波动。再把快速选择枢轴固定为首元素，用有序数组制造最坏情况，理解为什么工程库不会使用裸快速排序。

本节复盘

阅读本节后，请回到相邻章节的例题，把暴力瓶颈、优化依据、状态不变量、C++ 容器成本和边界测试重新写一遍。能把同一思想迁移到变体题，才算真正掌握。

第 12 章

堆、Top K 和贪心证明

堆和贪心经常一起出现，但它们不是同一个概念。堆是一种数据结构，负责快速取出当前最大或最小候选；贪心是一种算法策略，声称每一步做局部最优选择不会破坏全局最优。堆题的关键是“当前最该处理的候选是谁”；贪心题的关键是“为什么现在做这个选择以后不会后悔”。面试中只写出 `priority_queue` 远远不够，还要讲清候选含义、过期规则、比较器方向和正确性理由。

C++ 的 `std::priority_queue` 默认是大顶堆。要写小顶堆，需要使用 `std::greater<T>`。如果元素是结构体，需要自定义比较器。比较器的方向经常让人困惑：`priority_queue` 会把“优先级最大”的元素放在 `top`，比较器返回 `true` 表示左侧优先级低于右侧。因此写自定义比较器时，最好用“谁应该排后面”来理解。

Listing 12.1 `priority_queue` 的比较器方向

```
1 struct Entry {
2     int value = 0;
3     int frequency = 0;
4 };
5
6 struct LowerFrequencyFirst {
7     bool operator()(const Entry& lhs, const Entry& rhs) const
8     {
9         return lhs.frequency > rhs.frequency;
10    }
11 };
12
13 using MinHeap = std::priority_queue<Entry,
14                                     std::vector<Entry>,
15                                     LowerFrequencyFirst>;
```

这段比较器让频次更小的元素优先出现在堆顶，因为当 `lhs.frequency > rhs.frequency` 时，`lhs` 被认为优先级更低。很多人会在这里写反，导致小顶堆变成大顶堆。简单类型能用 `std::greater<int>` 时就用标准库；复杂类型必须写清比较器语义，并用小样例验证堆顶是谁。

前 K 个高频元素是哈希表和堆的组合题。暴力做法是统计频次后把所有不同元素按频次排序，复杂度 $O(m \log m)$ ， m 是不同元素个数。如果只要前 K 个，不需要完整排序，可以维护一个大小为 K 的小顶堆。堆顶是当前前 K 高频候选里频次最低的元素；新元素频次如果不超过堆顶，就不可能进入前 K；如果超过，就替换堆顶。

Listing 12.2 LeetCode 347 前 K 个高频元素：频次表加大小为 K 的小顶堆

```
1 std::vector<int> top_k_frequent(const std::vector<int>& nums, int k)
2 {
3     std::unordered_map<int, int> frequency;
4     frequency.reserve(nums.size());
5     for (const int x : nums) {
6         ++frequency[x];
7     }
8
9     using Entry = std::pair<int, int>; // frequency, value
10    std::priority_queue<Entry, std::vector<Entry>, std::greater<Entry>> heap;
```

```

11
12     for (const auto& [value, count] : frequency) {
13         heap.emplace(count, value);
14         if (static_cast<int>(heap.size()) > k) {
15             heap.pop();
16         }
17     }
18
19     std::vector<int> answer;
20     answer.reserve(k);
21     while (!heap.empty()) {
22         answer.push_back(heap.top().second);
23         heap.pop();
24     }
25     return answer;
26 }

```

这里用 `pair<count, value>` 是为了让 `std::greater` 按频次小顶堆工作；如果频次相同，会按值比较，但题目通常不要求固定顺序。复杂度是统计 $O(n)$ ，堆维护 $O(m \log k)$ 。如果 K 接近 m ，完整排序可能更简单；如果值域有限，也可以用桶排序按频次分桶，达到线性复杂度。面试时要比较这三种方案，而不是只给一种。

数据流中位数展示双堆模型。一个大顶堆保存较小一半，一个小顶堆保存较大一半。两个堆的大小差不超过 1，并且大顶堆堆顶不大于小顶堆堆顶。这样总数为奇数时，中位数是元素更多那一侧的堆顶；总数为偶数时，中位数是两个堆顶平均值。暴力方法每次插入后排序或线性插入，查询中位数容易但插入代价高。

Listing 12.3 LeetCode 295 数据流中位数：双堆维护两半

```

1 class MedianFinder {
2 public:
3     void add_num(int num)
4     {
5         if (this->lower_.empty() || num <= this->lower_.top()) {
6             this->lower_.push(num);
7         } else {
8             this->upper_.push(num);
9         }
10        this->rebalance();
11    }
12
13    double find_median() const
14    {
15        if (this->lower_.size() == this->upper_.size()) {
16            return (static_cast<double>(this->lower_.top()) +
17                    static_cast<double>(this->upper_.top())) / 2.0;
18        }
19        return this->lower_.size() > this->upper_.size()
20               ? static_cast<double>(this->lower_.top())
21               : static_cast<double>(this->upper_.top());
22    }
23
24 private:
25     void rebalance()

```



```

26     {
27         if (this->lower_.size() > this->upper_.size() + 1) {
28             this->upper_.push(this->lower_.top());
29             this->lower_.pop();
30         } else if (this->upper_.size() > this->lower_.size() + 1) {
31             this->lower_.push(this->upper_.top());
32             this->upper_.pop();
33         }
34     }
35
36     std::priority_queue<int> lower_;
37     std::priority_queue<int, std::vector<int>, std::greater<int>> upper_;
38 };

```

这段代码把重平衡逻辑拆成私有函数，避免 `add_num` 过长。成员访问使用 `this->`，符合 C++ 代码规范。复杂度是插入 $O(\log n)$ ，查询 $O(1)$ 。易错点有两个：偶数个元素时要转成 `double` 避免整数溢出和整数除法；重平衡只保证大小差，不要忘记插入时先按堆顶决定放入哪一侧，否则大小平衡但顺序不平衡。

合并 K 个升序链表是堆在链表上的应用。暴力方法可以每次扫描 K 个链表头，找最小节点接入结果，若总节点数为 N ，复杂度 $O(NK)$ 。堆优化是把每条链表当前头节点放入小顶堆，每次弹出最小节点，再把它后继放入堆。堆里最多 K 个节点，所以复杂度 $O(N \log K)$ 。

Listing 12.4 LeetCode 23 合并 K 个升序链表：堆保存每条链当前头节点

```

1 ListNode* merge_k_lists(std::vector<ListNode*> lists)
2 {
3     struct CompareNode {
4         bool operator()(const ListNode* lhs, const ListNode* rhs) const
5         {
6             return lhs->value > rhs->value;
7         }
8     };
9
10    std::priority_queue<ListNode*, std::vector<ListNode*>, CompareNode> heap;
11    for (ListNode* node : lists) {
12        if (node != nullptr) {
13            heap.push(node);
14        }
15    }
16
17    ListNode dummy;
18    ListNode* tail = &dummy;
19    while (!heap.empty()) {
20        ListNode* node = heap.top();
21        heap.pop();
22        if (node->next != nullptr) {
23            heap.push(node->next);
24        }
25        tail->next = node;
26        tail = tail->next;
27    }
28    tail->next = nullptr;
29    return dummy.next;

```

```
30 }
```

这里比较器把值更大的节点放到后面，因此堆顶是值最小节点。弹出节点后先把后继放入堆，再接入结果，两个顺序都可以，但最后要确保 `tail->next = nullptr`，避免结果尾部还连着旧链表的残余路径产生误解。这个算法没有创建新节点，复用输入节点；如果不允许修改输入，需要新建节点。面试表达要强调堆里保存的是“每条链表尚未合并部分的最小候选”。

跳跃游戏是贪心的入门题。题目问能否从下标 0 跳到最后。暴力搜索会从每个位置尝试所有可跳长度，存在大量重复状态。DP 可以记录每个位置是否可达，复杂度 $O(n^2)$ 。贪心更简单：扫描过程中维护当前能到达的最远下标 `farthest`。如果扫描到的位置已经超过 `farthest`，说明这个位置不可达；否则用它继续更新最远范围。

Listing 12.5 LeetCode 55 跳跃游戏：维护当前可达最远位置

```
1 bool can_jump(const std::vector<int>& nums)
2 {
3     int farthest = 0;
4     for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
5         if (i > farthest) {
6             return false;
7         }
8         farthest = std::max(farthest, i + nums[i]);
9         if (farthest >= static_cast<int>(nums.size()) - 1) {
10             return true;
11         }
12     }
13     return true;
14 }
```

这个贪心的正确性来自覆盖区间：只要当前位置 `i` 可达，那么从它出发能扩展的所有位置都被 `farthest` 覆盖；我们不关心具体从哪条路径到达，只关心可达区域的右边界。每次扫描到边界内的新位置，都可能把边界向右推。若某个位置在边界之外，任何之前位置都无法到达它，后面的位置更不可能被访问。

跳跃游戏 II 要求最少跳跃次数。暴力 BFS 可以把每个下标当节点，每条可跳边连接到后续位置，求无权最短路，但边数最坏 $O(n^2)$ 。贪心做法把一跳能到达的范围看成 BFS 的一层：`current_end` 是当前跳数能覆盖的最右位置，扫描这一层内所有点，计算下一跳能到达的最远位置 `farthest`；当扫描到 `current_end`，说明这一层结束，必须增加一次跳跃。

Listing 12.6 LeetCode 45 跳跃游戏 II：按层扩展可达区间

```
1 int jump(const std::vector<int>& nums)
2 {
3     if (nums.size() <= 1) {
4         return 0;
5     }
6
7     int jumps = 0;
8     int current_end = 0;
9     int farthest = 0;
10
11     for (int i = 0; i < static_cast<int>(nums.size()) - 1; ++i) {
12         farthest = std::max(farthest, i + nums[i]);
13         if (i == current_end) {
14             ++jumps;
15             current_end = farthest;
16         }
17     }
```

```

16     }
17 }
18
19 return jumps;
20 }

```

这段代码本质上是压缩版 BFS。每次增加 `jumps`，代表进入下一层；`current_end` 以内的位置都属于当前层。为什么循环只到倒数第二个位置？因为一旦已经能到达最后，就不需要从最后一个位置再跳。常见错误是在每个位置都加跳数，或者没有区分当前层边界和下一层最远边界。

贪心证明通常有两种表达：交换论证和领先性论证。区间调度用交换论证：把最优解中结束更晚的选择换成结束更早的选择，不会变差。跳跃游戏用领先性论证：贪心维护的可达最远边界始终不落后于任何其他方案在相同跳数下能到达的边界。面试中给出哪种证明不重要，重要的是不要把“看起来对”当成证明。

任务调度器是堆、计数和构造思维的交叉题。题目给一组任务和冷却时间 `n`，相同任务之间必须至少间隔 `n` 个单位时间，问完成所有任务的最短时间。暴力模拟可以每个时间点扫描所有任务，选择当前可执行且剩余次数最多的任务；如果任务种类多，扫描会重复。堆优化是把可执行任务按剩余次数放进大顶堆，把冷却中的任务放进队列，时间推进时把冷却结束的任务放回堆。

Listing 12.7 LeetCode 621 任务调度器：堆处理可执行任务，队列处理冷却

```

1 int least_interval(const std::vector<char>& tasks, int n)
2 {
3     std::array<int, 26> count{};
4     for (const char task : tasks) {
5         ++count[task - 'A'];
6     }
7
8     std::priority_queue<int> available;
9     for (const int c : count) {
10         if (c > 0) {
11             available.push(c);
12         }
13     }
14
15     std::queue<std::pair<int, int>> cooling; // ready_time, remaining
16     int time = 0;
17     while (!available.empty() || !cooling.empty()) {
18         ++time;
19         if (!available.empty()) {
20             int remaining = available.top();
21             available.pop();
22             --remaining;
23             if (remaining > 0) {
24                 cooling.emplace(time + n, remaining);
25             }
26         }
27
28         if (!cooling.empty() && cooling.front().first == time) {
29             available.push(cooling.front().second);
30             cooling.pop();
31         }
32     }
33 }

```

```

34     return time;
35 }

```

这个模拟版本容易解释，但还有数学构造解。设最高频任务出现 `max_count` 次，有 `max_kinds` 种任务都达到最高频。最短时间至少是 $(\text{max_count} - 1) * (n + 1) + \text{max_kinds}$ ，因为最高频任务形成 `max_count - 1` 个完整间隔块，最后一组放所有最高频任务。答案还不能小于任务总数，所以取二者最大。模拟适理解过程，公式适合直接求最短长度。面试中给出一种后，最好能说明另一种的直觉。

加油站是贪心的区间累积题。题目给每个站的油量和到下一站的消耗，问是否存在起点能绕一圈。暴力方法从每个站出发模拟一圈，复杂度 $O(n^2)$ 。贪心依据是：如果从 `start` 出发到 `i` 之间某处油量变成负数，那么 `start` 到 `i` 之间任何站都不可能作为合法起点。因为从这些中间站出发，只会少掉 `start` 到该站之前积累的油量，不会更好。

Listing 12.8 LeetCode 134 加油站：失败区间整体跳过

```

1 int can_complete_circuit(const std::vector<int>& gas,
2                           const std::vector<int>& cost)
3 {
4     int total = 0;
5     int tank = 0;
6     int start = 0;
7
8     for (int i = 0; i < static_cast<int>(gas.size()); ++i) {
9         const int delta = gas[i] - cost[i];
10        total += delta;
11        tank += delta;
12        if (tank < 0) {
13            start = i + 1;
14            tank = 0;
15        }
16    }
17
18    return total >= 0 ? start : -1;
19 }

```

`total < 0` 表示全局油量不足，任何起点都不可能成功。若全局油量足够，局部失败时把起点跳到 `i + 1` 是安全的。这个跳跃是贪心正确性的核心，不是经验优化。加油站题的复盘要把“失败区间整体排除”说清楚，否则代码看起来像玄学。

划分字母区间展示“最后出现位置”如何制造贪心边界。题目要求把字符串划分成尽可能多的片段，使每个字母最多出现在一个片段中。暴力方法枚举所有切分并检查字符集合是否交叉，复杂度高。优化思路是先记录每个字符最后出现的位置。扫描时，当前片段右边界必须至少覆盖已经遇到的所有字符的最后位置；当扫描下标到达这个右边界时，当前片段可以安全切开。

Listing 12.9 LeetCode 763 划分字母区间：扫描当前片段最远边界

```

1 std::vector<int> partition_labels(const std::string& s)
2 {
3     std::array<int, 26> last{};
4     for (int i = 0; i < static_cast<int>(s.size()); ++i) {
5         last[s[i] - 'a'] = i;
6     }
7
8     std::vector<int> answer;
9     int start = 0;

```

```

10     int end = 0;
11     for (int i = 0; i < static_cast<int>(s.size()); ++i) {
12         end = std::max(end, last[s[i] - 'a']);
13         if (i == end) {
14             answer.push_back(end - start + 1);
15             start = i + 1;
16         }
17     }
18     return answer;
19 }

```

这个贪心的边界语义非常清楚：当前片段中出现过的所有字符，它们的最后出现位置都不超过 `end`。当 `i == end`，说明这些字符不会在后面再次出现，因此切开不会违反题意；如果提前切开，至少有一个字符会跨片段。类似思想也出现在字符串覆盖、最小区间和扫描线问题中：先知道每个对象的最后影响范围，再扫描形成最小合法边界。

IPO 项目选择题把贪心和双堆结合起来。给定初始资本、最多做 k 个项目、每个项目需要资本和带来利润，问最大资本。暴力方法每轮扫描所有项目，找当前资本能做且利润最高的项目，复杂度 $O(kn)$ 。优化方法是先按资本要求排序，用指针把当前资本可负担的项目加入利润大顶堆，每轮从堆顶选择利润最高项目。

Listing 12.10 LeetCode 502 IPO：按资本解锁项目，用利润堆选择

```

1  int find_maximized_capital(int k,
2                             int capital,
3                             std::vector<int> profits,
4                             std::vector<int> required)
5  {
6      std::vector<std::pair<int, int>> projects;
7      projects.reserve(profits.size());
8      for (int i = 0; i < static_cast<int>(profits.size()); ++i) {
9          projects.emplace_back(required[i], profits[i]);
10     }
11     std::sort(projects.begin(), projects.end());
12
13     std::priority_queue<int> available_profits;
14     int index = 0;
15     for (int round = 0; round < k; ++round) {
16         while (index < static_cast<int>(projects.size()) &&
17             projects[index].first <= capital) {
18             available_profits.push(projects[index].second);
19             ++index;
20         }
21         if (available_profits.empty()) {
22             break;
23         }
24         capital += available_profits.top();
25         available_profits.pop();
26     }
27
28     return capital;
29 }

```

这个问题的局部最优选择是：在当前能做的项目中，选择利润最高的。为什么不会后悔？因为

做项目只会增加资本，不会消耗资本；利润越高，下一轮可选项目集合只会更大，不会更小。这个性质非常关键。如果项目会消耗资源，或者项目之间有依赖，简单贪心就可能失败。堆这里只是让“当前可选项目中利润最高”查询更快。

贪心失败的例子同样重要。零钱兑换若面额是 1, 3, 4，目标 6，贪心先选 4，再选 1、1，共 3 枚；最优是 3、3，共 2 枚。这个反例说明“每次选最大面额”不一定全局最优。判断贪心是否成立，不能只看样例，要找交换论证、单调边界、领先性或 matroid 这类结构。没有证明时，优先退回 DP 或搜索。

重构字符串是堆处理“当前最紧急约束”的例子。题目要求重新排列字符串，使相邻字符不同。暴力回溯可以尝试所有排列，但重复字符多时搜索空间巨大。贪心想法是每次优先使用剩余次数最多的字符，因为它最难安排；但不能连续使用同一个字符，所以每轮从大顶堆取出两个频次最高的不同字符，依次放入答案，再把剩余频次放回堆。若某个字符出现次数超过 $(n+1)/2$ ，必然无法重构。

Listing 12.11 LeetCode 767 重构字符串：每次取两个最高频字符

```

1 std::string reorganize_string(const std::string& s)
2 {
3     std::array<int, 26> count{};
4     for (const char ch : s) {
5         ++count[ch - 'a'];
6     }
7
8     using Entry = std::pair<int, char>;
9     std::priority_queue<Entry> heap;
10    for (int i = 0; i < static_cast<int>(count.size()); ++i) {
11        if (count[i] > 0) {
12            heap.emplace(count[i], static_cast<char>('a' + i));
13        }
14    }
15
16    std::string answer;
17    answer.reserve(s.size());
18    while (heap.size() >= 2) {
19        auto [first_count, first_char] = heap.top();
20        heap.pop();
21        auto [second_count, second_char] = heap.top();
22        heap.pop();
23
24        answer.push_back(first_char);
25        answer.push_back(second_char);
26        if (--first_count > 0) {
27            heap.emplace(first_count, first_char);
28        }
29        if (--second_count > 0) {
30            heap.emplace(second_count, second_char);
31        }
32    }
33
34    if (!heap.empty()) {
35        auto [remaining_count, remaining_char] = heap.top();
36        if (remaining_count > 1) {
37            return "";
38        }
    }

```

```

39     if (!answer.empty() && answer.back() == remaining_char) {
40         return "";
41     }
42     answer.push_back(remaining_char);
43 }
44
45 return answer;
46 }

```

这个贪心的直觉是“高频字符最危险，越晚处理越可能无处可放”。每次取两个不同字符，可以保证刚放入的两个字符相邻不同，也避免同一个最高频字符连续出现。更简洁的写法是每次取一个字符，并把上一轮用过但还剩次数的字符延迟一轮再放回堆；两种写法本质相同。正确性还可以用必要条件解释：最高频字符需要插入到其他字符形成的空隙里，空隙数量最多是其他字符数加一，因此最高频不能超过 $(n+1)/2$ 。

任务调度器和重构字符串类似，但它要求相同任务之间至少间隔 n 个单位时间，可以插入空闲。模拟做法是每轮取最多 $n+1$ 个不同任务执行，剩余任务放回堆；如果任务还没做完而本轮不足 $n+1$ ，就需要补空闲。数学做法更快：设最高频为 maxFreq ，拥有最高频的任务种类数为 maxCount ，最短时间至少是 $(\text{maxFreq} - 1) * (n + 1) + \text{maxCount}$ ，再和任务总数取最大。

Listing 12.12 LeetCode 621 任务调度器：按最高频任务计算骨架长度

```

1 int least_interval(const std::vector<char>& tasks, int cooldown)
2 {
3     std::array<int, 26> count{};
4     for (const char task : tasks) {
5         ++count[task - 'A'];
6     }
7
8     int max_frequency = 0;
9     int max_count = 0;
10    for (const int frequency : count) {
11        if (frequency > max_frequency) {
12            max_frequency = frequency;
13            max_count = 1;
14        } else if (frequency == max_frequency) {
15            ++max_count;
16        }
17    }
18
19    const int skeleton =
20        (max_frequency - 1) * (cooldown + 1) + max_count;
21    return std::max(static_cast<int>(tasks.size()), skeleton);
22 }

```

公式的含义是：最高频任务形成 $\text{max_frequency} - 1$ 个完整间隔块，每块长度至少 $\text{cooldown} + 1$ ，最后再放所有最高频任务的尾部。若其他任务足够多，它们可以填满空隙，答案就是任务总数；若不够，就必须空闲，答案由骨架决定。这个题很好地区分了“堆模拟”和“数学贪心”：堆模拟更通用，公式更简洁但依赖对结构的深理解。

课程表 III 是“按截止时间排序 + 最大堆反悔”的经典题。每门课有持续时间和截止时间，要求最多能上多少门。暴力搜索选课集合是指数级。贪心先按截止时间升序处理课程，尽量把当前课程加入计划；如果加入后总时间超过当前截止时间，就从已选课程中删除耗时最长的一门。为什么删最长？因为课程数量减少一门时，删除耗时最长能让总时间降得最多，给后续课程留下最大空间。

Listing 12.13 LeetCode 630 课程表 III: 超过截止时删除最长课程

```

1 int schedule_course(std::vector<std::vector<int>> courses)
2 {
3     std::sort(courses.begin(), courses.end(),
4               [](const std::vector<int>& lhs, const std::vector<int>& rhs) {
5                 return lhs[1] < rhs[1];
6             });
7
8     std::priority_queue<int> selected_durations;
9     int total_time = 0;
10    for (const std::vector<int>& course : courses) {
11        const int duration = course[0];
12        const int deadline = course[1];
13        selected_durations.push(duration);
14        total_time += duration;
15
16        if (total_time > deadline) {
17            total_time -= selected_durations.top();
18            selected_durations.pop();
19        }
20    }
21
22    return static_cast<int>(selected_durations.size());
23 }

```

这个算法的堆不是为了取下一门课，而是为了在当前已选集合中执行“反悔”。按截止时间排序保证当处理当前课程时，只需要满足当前及更早截止时间；如果超时，必须删掉一门已选课程。删最长课程不会减少已选课程数量以外的可行性，且让总耗时最小。这个证明属于交换论证：如果某个可行方案删的是较短课程而保留较长课程，交换为保留较短、删除较长，课程数量不变，总时间不增。

无重叠区间也能从“反悔”角度理解。按起点排序扫描时，若当前区间和上一个保留区间重叠，就必须删除其中一个。为了给后续留下更多空间，应保留结束更早的那个，删除结束更晚的那个。按终点排序的版本更直接：每次选择结束最早且与已选区间不重叠的区间，最大化保留数量；答案也可以用总数减去最多保留数量。

Listing 12.14 LeetCode 435 无重叠区间：保留结束更早的区间

```

1 int erase_overlap_intervals(std::vector<std::vector<int>> intervals)
2 {
3     if (intervals.empty()) {
4         return 0;
5     }
6     std::sort(intervals.begin(), intervals.end(),
7               [](const std::vector<int>& lhs, const std::vector<int>& rhs) {
8                 return lhs[0] < rhs[0];
9             });
10
11    int removed = 0;
12    int current_end = intervals[0][1];
13    for (int i = 1; i < static_cast<int>(intervals.size()); ++i) {
14        if (intervals[i][0] >= current_end) {
15            current_end = intervals[i][1];
16            continue;
17        }
18        removed++;
19    }
20    return intervals.size() - removed;
21 }

```

```

17     }
18     ++removed;
19     current_end = std::min(current_end, intervals[i][1]);
20 }
21 return removed;
22 }

```

当发生重叠时, `current_end = min(current_end, intervals[i][1])` 表示保留结束更早的区间。这个写法不显式记录保留哪个区间, 但保留了对后续唯一重要的信息: 当前已保留区间的最小结束时间。贪心题经常不需要保留完整历史, 只保留能决定未来可行性的摘要状态。若题目要求输出具体删除哪些区间, 就需要额外记录选择。

根据身高重建队列是排序和贪心的组合。每个人由 `height` 和 `k` 表示, `k` 是前面身高大于等于自己的人数。若从矮到高放, 后面更高的人插入会影响矮个子的 `k`; 若从高到矮放, 已经放入的人都不矮于当前人, 当前人的 `k` 就等于插入位置。排序规则是身高降序, 身高相同则 `k` 升序, 然后按 `k` 插入结果数组。

Listing 12.15 LeetCode 406 根据身高重建队列: 高个先放

```

1 std::vector<std::vector<int>> reconstruct_queue(
2     std::vector<std::vector<int>> people)
3 {
4     std::sort(people.begin(), people.end(),
5         [](const std::vector<int>& lhs, const std::vector<int>& rhs) {
6             if (lhs[0] != rhs[0]) {
7                 return lhs[0] > rhs[0];
8             }
9             return lhs[1] < rhs[1];
10        });
11
12     std::vector<std::vector<int>> queue;
13     queue.reserve(people.size());
14     for (const std::vector<int>& person : people) {
15         queue.insert(queue.begin() + person[1], person);
16     }
17     return queue;
18 }

```

这段代码的插入是 $O(n)$, 整体 $O(n^2)$, 但力扣规模通常可接受。若规模很大, 可以用树状数组或线段树找第 `k` 个空位。正确性来自处理顺序: 当插入当前人时, 队列中已有的人都比他高或等高, 因此插入下标正好决定了他前面符合条件的人数; 后续更矮的人插入不会影响他的 `k`。同高的人按 `k` 升序处理, 避免同高较大 `k` 先插入造成位置不稳定。

合并石头、戳气球、最优二叉搜索树这类题看上去也有“每次选局部最小或最大”的诱惑, 但通常不能简单贪心。以合并石头为例, 如果每次合并当前最小的相邻两堆, 不一定得到全局最优, 因为相邻约束会改变未来可合并结构。没有相邻约束的哈夫曼合并可以每次取最小两项, 这是因为任意两项都可合并, 且最优前缀树有明确交换性质; 有相邻约束后, 问题变成区间 DP。这个对比非常重要: 同样是“合并代价”, 约束不同, 算法类型就不同。

柠檬水找零是简单贪心, 但很适合训练“优先保留灵活资源”。顾客支付 5、10、20, 饮料价格 5。收到 10 需要找一个 5; 收到 20 时, 优先找 10+5, 而不是三个 5, 因为 10 只能在找 20 时使用, 5 更灵活, 后续找 10 和 20 都需要。这个证明不是复杂数学, 而是资源灵活性: 越通用的资源越应该保留。若无法找零则返回 `false`。

Listing 12.16 LeetCode 860 柠檬水找零: 优先使用较不灵活的 10 元

```

1 bool lemonade_change(const std::vector<int>& bills)
2 {
3     int five = 0;
4     int ten = 0;
5     for (const int bill : bills) {
6         if (bill == 5) {
7             ++five;
8         } else if (bill == 10) {
9             if (five == 0) {
10                return false;
11            }
12            --five;
13            ++ten;
14        } else {
15            if (ten > 0 && five > 0) {
16                --ten;
17                --five;
18            } else if (five >= 3) {
19                five -= 3;
20            } else {
21                return false;
22            }
23        }
24    }
25    return true;
26 }

```

这个题也能说明贪心证明可以很朴素。不是所有贪心都需要长篇交换论证，有时只要明确资源的支配关系：一张 5 元可以参与两类找零，一张 10 元只能参与 20 元找零；在能完成当前找零的前提下，消耗更不灵活的资源不会让未来更差。若题目币值变化，这个策略可能失效，因此不能把它泛化成“永远优先用大面额”。

最小加油次数是堆和贪心的又一个“反悔”模型。汽车沿路经过加油站，每个站有位置和油量，目标是用最少加油次数到达终点。暴力搜索在哪些站加油是指数级。贪心扫描所有可达加油站，把它们的油量放入大顶堆；当当前油量无法到达下一个站或终点时，就从已经路过且可选择的站中取油量最大的一个加油。虽然看起来像回头加油，但等价于在之前经过时选择了那个站。

Listing 12.17 LeetCode 871 最小加油次数：不可达时从已过站点中选最大油量

```

1 int min_refuel_stops(int target,
2                     int start_fuel,
3                     std::vector<std::vector<int>> stations)
4 {
5     stations.push_back({target, 0});
6     std::priority_queue<int> fuels;
7     int fuel = start_fuel;
8     int previous_position = 0;
9     int stops = 0;
10
11     for (const std::vector<int>& station : stations) {
12         const int position = station[0];
13         const int available_fuel = station[1];
14         fuel -= position - previous_position;
15         while (fuel < 0 && !fuels.empty()) {

```



```

16         fuel += fuels.top();
17         fuels.pop();
18         ++stops;
19     }
20     if (fuel < 0) {
21         return -1;
22     }
23     fuels.push(available_fuel);
24     previous_position = position;
25 }
26
27 return stops;
28 }

```

这个算法的正确性来自延迟决策：在还没被迫加油之前，不决定具体在哪个已过站点加；一旦必须加油，为了让未来最容易可达，就选择已过站点中油量最大的。选择更小油量不会减少加油次数，反而可能更早再次陷入不可达。堆中保存的是已经经过但尚未使用的加油机会。这个模型和 IPO、课程表 III 都有相似结构：先收集可用候选，必要时选最有利的一个。

贪心题的讲解应该包含失败测试。跳跃游戏 II 如果每一步只跳到当前能跳到的最远下标，可能不是最少跳数；正确做法是按当前层覆盖范围推进，选择下一层最远覆盖。课程表 III 如果只按持续时间短排序，会错过截止时间约束；如果只按截止时间排序但不反悔，也会被长课程拖垮。重构字符串如果每次只取最高频字符但不延迟上一字符，就可能产生相邻重复。把这些失败原因讲出来，答案才可信。

堆与贪心实验：第一，为零钱兑换构造 **1,3,4** 和目标 6，写出贪心失败过程，再用 DP 修正。第二，给课程表 III 构造一门很长但截止早的课程，观察最大堆如何反悔。第三，用任务调度器分别实现堆模拟和公式法，比较输出一致性。第四，把最小加油次数中的大顶堆改成小顶堆，找一个用例说明为什么会增加加油次数或失败。

堆和贪心题复盘模板

堆题先说明堆里保存什么候选，堆顶为什么是下一步最该处理的对象，候选何时入堆、何时出堆、何时过期。贪心题先给暴力或 DP，再说明局部选择为什么不会影响全局最优，最好用交换论证或领先性论证。遇到 Top K，主动比较排序、大小为 K 的堆、快速选择和桶排序；遇到区间覆盖，主动说明排序键和边界含义。

第 13 章

图论进阶：并查集、最短路和状态图

图论进阶题的难点通常不在 BFS 或 DFS 的基本写法，而在选择正确的图模型。无向连通性适合并查集；无权最短路适合 BFS；边权非负的最短路适合 Dijkstra；存在状态压缩或多维状态时，要先定义状态节点，再决定边和代价。很多题目表面不是图，例如密码锁、单词变化、网格体力消耗、课程安排、账户合并，只要存在“状态”和“一步转移”，就可以用图论语言重写。

并查集解决的是动态连通性。它不回答路径是什么，也不回答最短距离是多少；它只维护“两个元素当前是否属于同一集合”。暴力做法在每次合并后用 DFS 或 BFS 检查连通性，会重复扫描大量边。并查集把每个集合表示成一棵树，用根节点作为代表。路径压缩让查询时把沿途节点直接挂到根上，按大小合并让小树挂到大树下，两者结合后，均摊复杂度接近常数。

Listing 13.1 并查集：迭代 find、路径压缩和按大小合并

```
1 class DisjointSetUnion {
2 public:
3     explicit DisjointSetUnion(int n)
4         : parent_(n), size_(n, 1)
5     {
6         std::iota(this->parent_.begin(), this->parent_.end(), 0);
7     }
8
9     int find(int x)
10    {
11        int root = x;
12        while (this->parent_[root] != root) {
13            root = this->parent_[root];
14        }
15        while (this->parent_[x] != x) {
16            const int next = this->parent_[x];
17            this->parent_[x] = root;
18            x = next;
19        }
20        return root;
21    }
22
23    bool unite(int a, int b)
24    {
25        int root_a = this->find(a);
26        int root_b = this->find(b);
27        if (root_a == root_b) {
28            return false;
29        }
30        if (this->size_[root_a] < this->size_[root_b]) {
31            std::swap(root_a, root_b);
32        }
33        this->parent_[root_b] = root_a;
34        this->size_[root_a] += this->size_[root_b];
35        return true;
36    }
```

```

37
38 private:
39     std::vector<int> parent_;
40     std::vector<int> size_;
41 };

```

这里的 `find` 是迭代写法，避免递归深度问题。第一次循环找根，第二次循环压缩路径。`unite` 返回 `false` 表示两个元素本来已经连通，合并会形成环。这个返回值在很多题里正好就是答案判断条件。成员访问使用 `this->`，让读者清楚哪些状态属于并查集对象。

冗余连接是并查集的标准例题。题目给一棵树额外加一条边，要求找出导致环的边。暴力方法是每加入一条边，就在已有图中检查这两个端点是否已经连通；如果已经连通，再加边就会成环。并查集正好维护这个关系：扫描边时，如果 `unite(u, v)` 返回 `false`，说明这条边连接了同一集合内的两个点，它就是冗余边。

Listing 13.2 LeetCode 684 冗余连接：合并失败的边就是成环边

```

1 std::vector<int> find_redundant_connection(
2     const std::vector<std::vector<int>>& edges)
3 {
4     DisjointSetUnion dsu(static_cast<int>(edges.size()) + 1);
5     for (const std::vector<int>& edge : edges) {
6         const int u = edge[0];
7         const int v = edge[1];
8         if (!dsu.unite(u, v)) {
9             return edge;
10        }
11    }
12    return {};
13 }

```

这题要注意节点编号通常从 1 开始，所以并查集大小开到 `edges.size() + 1`。如果题目节点编号不连续，就需要哈希映射到连续编号。复杂度近似 $O(n)$ 。面试表达中要把树的性质说出来：一棵 n 个节点的树有 $n-1$ 条边且无环；额外加入一条边后，第一次发现两个端点已经连通的边，就是让无环结构变成有环结构的边。

省份数量也是连通块问题。输入是邻接矩阵，`isConnected[i][j] == 1` 表示城市 i 和 j 直接连接。DFS/BFS 可以做，并查集也可以做。暴力思路可能是从每个城市出发搜索所有可达城市并统计，但会重复访问。并查集做法扫描矩阵上三角，把直接连接的城市合并，最后统计根节点个数。

Listing 13.3 LeetCode 547 省份数量：邻接矩阵合并连通城市

```

1 int find_circle_num(const std::vector<std::vector<int>>& is_connected)
2 {
3     const int n = static_cast<int>(is_connected.size());
4     DisjointSetUnion dsu(n);
5
6     for (int i = 0; i < n; ++i) {
7         for (int j = i + 1; j < n; ++j) {
8             if (is_connected[i][j] == 1) {
9                 dsu.unite(i, j);
10            }
11        }
12    }
13
14    int provinces = 0;

```

```

15     for (int city = 0; city < n; ++city) {
16         if (dsu.find(city) == city) {
17             ++provinces;
18         }
19     }
20     return provinces;
21 }

```

邻接矩阵是对称的，因此只扫描上三角即可。这里并查集的优势不是绝对性能，而是模型清晰：省份就是连通分量，直接连接就是合并操作。若题目还要求列出每个省份包含哪些城市，则并查集统计完根后还要按根分组；若题目要求最短路径，并查集就不够，因为它丢掉了路径长度信息。

Dijkstra 处理非负边权最短路。无权图 BFS 成立，是因为每条边代价相同，按层访问保证第一次到达最短；带权图中，一条边可能代价很大，普通 BFS 不再可靠。Dijkstra 的贪心点是：每次从尚未确定的节点中取当前距离最小者，它的最短距离已经确定。这个结论依赖所有边权非负，因为后续绕路不可能通过负边把距离降得更低。

网络延迟时间是 Dijkstra 的典型题。节点是服务器，有向边带传播时间。题目问从起点发信号，到所有节点都收到信号需要多久，也就是起点到所有节点最短路的最大值。暴力方法可以反复松弛所有边，类似 Bellman-Ford，复杂度 $O(nm)$ 。由于边权非负，用堆优化 Dijkstra 更合适。

Listing 13.4 LeetCode 743 网络延迟时间：堆优化 Dijkstra

```

1  int network_delay_time(const std::vector<std::vector<int>>& times,
2                          int n,
3                          int source)
4  {
5      using Edge = std::pair<int, int>; // neighbor, weight
6      std::vector<std::vector<Edge>> graph(n + 1);
7      for (const std::vector<int>& edge : times) {
8          graph[edge[0]].emplace_back(edge[1], edge[2]);
9      }
10
11     constexpr int INF = 1'000'000'000;
12     std::vector<int> distance(n + 1, INF);
13     distance[source] = 0;
14
15     using State = std::pair<int, int>; // distance, node
16     std::priority_queue<State, std::vector<State>, std::greater<State>> heap;
17     heap.emplace(0, source);
18
19     while (!heap.empty()) {
20         const auto [dist, node] = heap.top();
21         heap.pop();
22         if (dist != distance[node]) {
23             continue;
24         }
25
26         for (const auto [next, weight] : graph[node]) {
27             if (dist + weight < distance[next]) {
28                 distance[next] = dist + weight;
29                 heap.emplace(distance[next], next);
30             }
31         }
32     }

```

```

33
34     int answer = 0;
35     for (int node = 1; node <= n; ++node) {
36         if (distance[node] == INF) {
37             return -1;
38         }
39         answer = std::max(answer, distance[node]);
40     }
41     return answer;
42 }

```

这段代码没有使用显式 `visited`，而是用 `dist != distance[node]` 跳过堆中的过期状态。因为 C++ 标准堆不支持 `decrease-key`，更新距离时直接把新状态再压入堆，旧状态留在堆里，弹出时识别并跳过。复杂度是 $O((n+m)\log m)$ 或常写为 $O(m\log n)$ 。如果存在负边，Dijkstra 不适用，应考虑 Bellman-Ford 或 SPFA 的风险分析。

最小体力消耗路径看似网格题，实质是最短路变体。路径代价不是边权之和，而是路径上最大高度差的最小可能值。状态是格子，边是上下左右相邻格子，走一条边的代价是高度差。到达某个格子的最优体力定义为：从起点到它的路径中，最大边权尽可能小。转移时，新代价是 **max(当前路径体力, 新边高度差)**。这个代价仍满足 Dijkstra 的非负和单调扩展性质。

Listing 13.5 LeetCode 1631 最小体力路径：以最大边权作为路径代价

```

1  int minimum_effort_path(const std::vector<std::vector<int>>& heights)
2  {
3      constexpr std::array<std::pair<int, int>, 4> DIRECTIONS{
4          std::pair{1, 0}, std::pair{-1, 0},
5          std::pair{0, 1}, std::pair{0, -1}
6      };
7      const int rows = static_cast<int>(heights.size());
8      const int cols = static_cast<int>(heights[0].size());
9      constexpr int INF = 1'000'000'000;
10
11     std::vector<std::vector<int>> effort(rows, std::vector<int>(cols, INF));
12     effort[0][0] = 0;
13
14     using State = std::tuple<int, int, int>; // effort, row, col
15     std::priority_queue<State, std::vector<State>, std::greater<State>> heap;
16     heap.emplace(0, 0, 0);
17
18     while (!heap.empty()) {
19         const auto [cost, row, col] = heap.top();
20         heap.pop();
21         if (row == rows - 1 && col == cols - 1) {
22             return cost;
23         }
24         if (cost != effort[row][col]) {
25             continue;
26         }
27
28         for (const auto [dr, dc] : DIRECTIONS) {
29             const int next_row = row + dr;
30             const int next_col = col + dc;
31             const bool out_of_bounds =

```



```

32         next_row < 0 || next_row >= rows ||
33         next_col < 0 || next_col >= cols;
34     if (out_of_bounds) {
35         continue;
36     }
37
38     const int edge_cost =
39         std::abs(heights[row][col] - heights[next_row][next_col]);
40     const int next_cost = std::max(cost, edge_cost);
41     if (next_cost < effort[next_row][next_col]) {
42         effort[next_row][next_col] = next_cost;
43         heap.emplace(next_cost, next_row, next_col);
44     }
45 }
46 }
47
48 return 0;
49 }

```

这题也可以二分答案加 BFS：猜一个最大允许体力，看能否只走高度差不超过该值的边到终点。二分答案复杂度是 $O(mn \log H)$ ，Dijkstra 是 $O(mn \log(mn))$ 。两种方法都合理，面试时可以先给更统一的 Dijkstra，再说明二分可行的原因是“体力上限越大，可达性越强”，存在单调性。

状态图题还包括打开转盘锁、单词接龙、蛇梯棋等。共同点是输入没有直接给边，需要你根据规则生成邻居。生成邻居时要注意两件事：第一，状态编码要能高效去重；第二，访问标记要在入队时设置，避免同一状态被多个父状态重复加入。无权最短路使用 BFS，第一次到达目标即为最短；带权状态图则要换 Dijkstra 或 A*，不能强行 BFS。

真正写图论题之前，还要先决定图如何存储。邻接矩阵适合节点数量较小、需要频繁判断任意两点是否有边的场景，空间是 $O(n^2)$ ；邻接表适合稀疏图，空间是 $O(n + m)$ ；边数组适合 Kruskal、Bellman-Ford 这类按边扫描的算法；网格图通常不需要显式建图，用坐标和方向数组现场生成邻居即可。很多低质量答案只说“建图”，却没有说明为什么这样建。面试中应该先看输入规模：如果 n 可以到十万，邻接矩阵基本不可能；如果 m 远小于 n^2 ，邻接表更自然；如果题目要求按边权排序，边数组会比邻接表更直接。

邻接表也有工程差异。`vector<vector<pair<int,int>>>` 写法直观，但每个内层 `vector` 都是独立对象，极端情况下会产生很多小分配；竞赛中常见的前向星把所有边放进几个连续数组，通过 `head` 和 `next` 串起来，局部性更好，但代码可读性低。刷题面试通常以正确性和表达清晰为第一优先，选 `vector` 邻接表即可；当题目数据极大或需要讲性能时，可以补充连续边数组的思路。这个判断体现的是工程能力：知道什么时候写简单代码，什么时候需要控制内存布局。

Listing 13.6 边数组转邻接表：保留输入语义并便于最短路

```

1 struct WeightedEdge {
2     int from;
3     int to;
4     int weight;
5 };
6
7 std::vector<std::vector<std::pair<int, int>>> build_directed_graph(
8     int node_count,
9     const std::vector<WeightedEdge>& edges)
10 {
11     std::vector<std::vector<std::pair<int, int>>> graph(node_count);
12     for (const WeightedEdge& edge : edges) {
13         graph[edge.from].emplace_back(edge.to, edge.weight);

```

```

14     }
15     return graph;
16 }

```

这段代码看似简单，但它体现了一个重要习惯：输入边和算法图结构分开。`WeightedEdge` 保留题目给出的边语义，邻接表服务于后续算法。若后面同时要跑 Kruskal 和 Dijkstra，就不必从邻接表反推出边数组。很多工程 bug 来自过早把所有信息塞进一种结构，后面算法一换就需要反复转换。

BFS 的本质是按距离层扩展。普通队列能保证这一点，因为所有边权都是 1。访问标记必须在入队时设置，而不是出队时设置；如果出队时才标记，一个节点可能被同一层的多个父节点重复入队，轻则浪费时间，重则在记录父节点或计数时产生错误。多源 BFS 只是把多个起点同时放入队列，把它们的初始距离都设为 0。腐烂的橘子、地图分析、离所有建筑最近的空地等题，都可以用这个模型理解。

Listing 13.7 多源 BFS：所有起点同时入队

```

1  int shortest_distance_from_sources(
2      const std::vector<std::vector<int>>& grid,
3      const std::vector<std::pair<int, int>>& sources)
4  {
5      constexpr std::array<std::pair<int, int>, 4> DIRECTIONS{
6          std::pair{1, 0}, std::pair{-1, 0},
7          std::pair{0, 1}, std::pair{0, -1}
8      };
9      const int rows = static_cast<int>(grid.size());
10     const int cols = static_cast<int>(grid[0].size());
11
12     std::vector<std::vector<int>> distance(rows, std::vector<int>(cols, -1));
13     std::queue<std::pair<int, int>> queue;
14     for (const auto [row, col] : sources) {
15         distance[row][col] = 0;
16         queue.emplace(row, col);
17     }
18
19     int farthest = 0;
20     while (!queue.empty()) {
21         const auto [row, col] = queue.front();
22         queue.pop();
23         farthest = std::max(farthest, distance[row][col]);
24
25         for (const auto [dr, dc] : DIRECTIONS) {
26             const int next_row = row + dr;
27             const int next_col = col + dc;
28             const bool out_of_bounds =
29                 next_row < 0 || next_row >= rows ||
30                 next_col < 0 || next_col >= cols;
31             if (out_of_bounds || grid[next_row][next_col] == 1 ||
32                 distance[next_row][next_col] != -1) {
33                 continue;
34             }
35             distance[next_row][next_col] = distance[row][col] + 1;
36             queue.emplace(next_row, next_col);
37         }

```

```

38     }
39
40     return farthest;
41 }

```

这段模板里，`distance == -1` 同时承担“尚未访问”和“距离未知”的含义。若题目需要允许障碍、空地、起点不同类型，就不要直接修改原数组，单独开 `distance` 会更安全。复杂度是 $O(mn)$ ，因为每个格子最多入队一次，每条网格边最多检查常数次数。面试时要强调：多源 BFS 不是从每个源分别 BFS，而是把所有源放在第 0 层同时扩散；这样第一次到达某个点的源，就是离它最近的一批源之一。

0-1 BFS 是普通 BFS 和 Dijkstra 之间的桥梁。当边权只可能是 0 或 1 时，使用优先队列当然正确，但 `deque` 可以把复杂度降到线性。权重为 0 的转移放到队首，权重为 1 的转移放到队尾。这样队列中的状态仍然按照非降距离处理，只是用双端队列代替了堆。最典型的题是网格中每个格子给出推荐方向，顺着方向走代价为 0，改方向代价为 1，求从左上到右下的最小修改次数。

Listing 13.8 LeetCode 1368 使网格图至少有一条有效路径的最小代价：0-1 BFS

```

1 int min_cost_valid_path(const std::vector<std::vector<int>>& grid)
2 {
3     constexpr std::array<std::pair<int, int>, 4> DIRECTIONS{
4         std::pair{0, 1}, std::pair{0, -1},
5         std::pair{1, 0}, std::pair{-1, 0}
6     };
7     constexpr int INF = 1'000'000'000;
8     const int rows = static_cast<int>(grid.size());
9     const int cols = static_cast<int>(grid[0].size());
10
11     std::vector<std::vector<int>> distance(rows, std::vector<int>(cols, INF));
12     std::deque<std::pair<int, int>> deque;
13     distance[0][0] = 0;
14     deque.emplace_front(0, 0);
15
16     while (!deque.empty()) {
17         const auto [row, col] = deque.front();
18         deque.pop_front();
19
20         for (int dir = 0; dir < static_cast<int>(DIRECTIONS.size()); ++dir) {
21             const auto [dr, dc] = DIRECTIONS[dir];
22             const int next_row = row + dr;
23             const int next_col = col + dc;
24             const bool out_of_bounds =
25                 next_row < 0 || next_row >= rows ||
26                 next_col < 0 || next_col >= cols;
27             if (out_of_bounds) {
28                 continue;
29             }
30
31             const int extra_cost = (grid[row][col] == dir + 1) ? 0 : 1;
32             const int next_distance = distance[row][col] + extra_cost;
33             if (next_distance >= distance[next_row][next_col]) {
34                 continue;
35             }
36

```

```

37         distance[next_row][next_col] = next_distance;
38         if (extra_cost == 0) {
39             deque.emplace_front(next_row, next_col);
40         } else {
41             deque.emplace_back(next_row, next_col);
42         }
43     }
44 }
45
46 return distance[rows - 1][cols - 1];
47 }

```

0-1 BFS 的正确性可以这样解释：普通 BFS 每次扩展当前最小层；Dijkstra 每次从堆里取最小距离；当新增边代价只有 0 或 1 时，当前状态的后继要么仍在当前距离层，要么进入下一距离层。放队首和队尾正好维护了这个层次。易错点有三个：第一，不能用普通 `queue`，因为 0 代价边应该优先处理；第二，不能只用 `visited`，因为某个状态可能先以较大代价到达，后来被较小代价改进；第三，方向编号要和题目约定对齐，示例里题目方向从 1 开始，所以比较时使用 `dir + 1`。

拓扑排序处理有向无环图。它回答的不是最短路，而是依赖顺序：如果一件事必须在另一件事之前完成，就可以画一条有向边。课程表就是标准题。暴力想法是不断找没有前置依赖的课程，删除它，再更新其他课程；拓扑排序把这个过程用入度数组和队列高效实现。若最后处理课程数小于总课程数，说明存在环，环内课程互相等待，无法完成。

Listing 13.9 LeetCode 207 课程表：Kahn 拓扑排序判断是否有环

```

1 bool can_finish(int course_count,
2                 const std::vector<std::vector<int>>& prerequisites)
3 {
4     std::vector<std::vector<int>> graph(course_count);
5     std::vector<int> indegree(course_count, 0);
6
7     for (const std::vector<int>& relation : prerequisites) {
8         const int course = relation[0];
9         const int dependency = relation[1];
10        graph[dependency].push_back(course);
11        ++indegree[course];
12    }
13
14    std::queue<int> ready;
15    for (int course = 0; course < course_count; ++course) {
16        if (indegree[course] == 0) {
17            ready.push(course);
18        }
19    }
20
21    int processed = 0;
22    while (!ready.empty()) {
23        const int course = ready.front();
24        ready.pop();
25        ++processed;
26
27        for (const int next : graph[course]) {
28            --indegree[next];
29            if (indegree[next] == 0) {

```

```

30         ready.push(next);
31     }
32 }
33 }
34
35 return processed == course_count;
36 }

```

这题最容易写反边。题目常写成 $[a, b]$ 表示想学 a 必须先学 b ，所以边应该是 $b \rightarrow a$ 。入度表示还有多少依赖没有满足。队列中放的是当前已经没有前置依赖的课程。正确性来自不变量：队列中的每门课入度为 0，已经可以安排；处理它等价于完成这门课，因此它指向的后继少一个未满足依赖。若图中有环，环上每个点至少依赖环内另一个点，永远不会全部入队。

拓扑排序不只判断能不能完成，还能在 DAG 上做动态规划。只要图无环，就可以按拓扑顺序保证前驱状态已经计算完，再更新后继。比如有向无环图中求从起点到每个点的最长路径，不能用 Dijkstra，因为目标是最长而不是最短；也不能在有环图上直接做，因为正权环会让最长路径没有上界。DAG 的拓扑顺序让“阶段”变得明确。

Listing 13.10 DAG 最长路径：拓扑序上的动态规划

```

1 std::vector<int> longest_path_in_dag(
2     int node_count,
3     const std::vector<std::tuple<int, int, int>>& edges,
4     int source)
5 {
6     std::vector<std::vector<std::pair<int, int>>> graph(node_count);
7     std::vector<int> indegree(node_count, 0);
8     for (const auto [from, to, weight] : edges) {
9         graph[from].emplace_back(to, weight);
10        ++indegree[to];
11    }
12
13    std::queue<int> ready;
14    for (int node = 0; node < node_count; ++node) {
15        if (indegree[node] == 0) {
16            ready.push(node);
17        }
18    }
19
20    constexpr int NEG_INF = -1'000'000'000;
21    std::vector<int> distance(node_count, NEG_INF);
22    distance[source] = 0;
23
24    while (!ready.empty()) {
25        const int node = ready.front();
26        ready.pop();
27        if (distance[node] != NEG_INF) {
28            for (const auto [next, weight] : graph[node]) {
29                distance[next] = std::max(distance[next], distance[node] + weight);
30            }
31        }
32        for (const auto [next, weight] : graph[node]) {
33            (void)weight;
34            --indegree[next];

```



```

35         if (indegree[next] == 0) {
36             ready.push(next);
37         }
38     }
39 }
40
41 return distance;
42 }

```

上面代码故意把“松弛后继”和“减少入度”分开写，便于读者看清两个动作。实际工程中可以合并循环，但教材里先强调概念更重要。DAG DP 的状态是 `distance[v]`，含义是从 `source` 到 `v` 的最大路径和；转移来自所有前驱。拓扑序保证当某个节点出队时，它的所有前驱已经处理完，状态不再被未来节点回头修改。这个结构和一维 DP 很像：所谓“阶段顺序”，在图上就是拓扑顺序。

如果图中存在负边，Dijkstra 的贪心基础会崩塌。因为一个当前距离较大的节点，未来可能通过负边把另一个已经确定的节点拉得更小。Bellman-Ford 的想法更朴素：一条最短路径如果最多包含 `k` 条边，那么做 `k` 轮按边松弛就能得到它。经典最短路最多需要 `n - 1` 条边，因为无负环的最短简单路径不会重复经过节点。力扣中“最多经过 `K` 站中转”的机票题，本质是限制边数的 Bellman-Ford。

Listing 13.11 LeetCode 787 K 站中转内最便宜航班：按边数分层松弛

```

1 int find_cheapest_price(int n,
2                         const std::vector<std::vector<int>>& flights,
3                         int source,
4                         int target,
5                         int max_stops)
6 {
7     constexpr int INF = 1'000'000'000;
8     std::vector<int> previous(n, INF);
9     previous[source] = 0;
10
11     for (int step = 0; step <= max_stops; ++step) {
12         std::vector<int> current = previous;
13         for (const std::vector<int>& flight : flights) {
14             const int from = flight[0];
15             const int to = flight[1];
16             const int price = flight[2];
17             if (previous[from] == INF) {
18                 continue;
19             }
20             current[to] = std::min(current[to], previous[from] + price);
21         }
22         previous = std::move(current);
23     }
24
25     return previous[target] == INF ? -1 : previous[target];
26 }

```

这里必须使用 `previous` 和 `current` 两层数组，不能在同一个数组里原地更新。原因是第 `step` 轮只允许使用最多 `step + 1` 条边，如果原地更新，一轮内刚更新出来的状态又被后面的边继续使用，就会偷偷使用更多边。这个问题和 0-1 背包的倒序遍历本质类似：转移要读取上一阶段，不能让当前阶段污染自己。复杂度是 $O((K + 1)m)$ ，空间 $O(n)$ 。

最小生成树回答的是“把所有点连起来的最低总成本”，它不是最短路。最短路关心从某个起点到其他点的路径长度；最小生成树关心选出一组边，让全图连通且总权重最小。Kruskal 的策略是按

边权从小到大扫描，能连接两个不同连通块就选，否则跳过。并查集用来判断一条边是否会成环。这个贪心正确性来自割性质：任意一个割上，权重最小的跨割边一定存在某棵最小生成树包含它。

Listing 13.12 Kruskal：按边权连接不同连通块

```

1 struct UndirectedEdge {
2     int u;
3     int v;
4     int weight;
5 };
6
7 int minimum_spanning_tree_cost(int node_count,
8                                 std::vector<UndirectedEdge> edges)
9 {
10     std::sort(edges.begin(), edges.end(),
11               [](const UndirectedEdge& lhs, const UndirectedEdge& rhs) {
12                 return lhs.weight < rhs.weight;
13             });
14
15     DisjointSetUnion dsu(node_count);
16     int total_cost = 0;
17     int used_edges = 0;
18
19     for (const UndirectedEdge& edge : edges) {
20         if (!dsu.unite(edge.u, edge.v)) {
21             continue;
22         }
23         total_cost += edge.weight;
24         ++used_edges;
25         if (used_edges == node_count - 1) {
26             return total_cost;
27         }
28     }
29
30     return -1;
31 }

```

面试中区分 Kruskal 和 Prim 的方式很简单：Kruskal 站在“边”的视角，排序后用并查集选边；Prim 站在“点”的视角，从一个连通块开始，每次选连接外部点的最小边。边比较少、已经容易生成边数组时，Kruskal 很顺；完全图且边很多时，Prim 可以避免显式存储所有边。连接所有点的最小费用中，点与点之间距离是曼哈顿距离，朴素建完全图有 $O(n^2)$ 条边， n 不大时可以接受；如果规模上去，就要研究几何图优化，而不是盲目套模板。

并查集还能和二分答案结合。最小体力路径除了 Dijkstra，也可以二分一个体力上限 `limit`，只把高度差不超过 `limit` 的相邻格子合并，最后看起点和终点是否连通。若某个 `limit` 可行，那么更大的上限一定可行；若不可行，更小的上限一定不可行。这就是二分的单调性。这个方法的优点是解释直观，缺点是每次检查都要重新扫描网格。

Listing 13.13 LeetCode 1631 最小体力路径：二分答案加并查集

```

1 bool can_reach_with_effort(const std::vector<std::vector<int>>& heights,
2                             int limit)
3 {
4     const int rows = static_cast<int>(heights.size());
5     const int cols = static_cast<int>(heights[0].size());

```

```

6   DisjointSetUnion dsu(rows * cols);
7
8   auto id = [cols](int row, int col) {
9       return row * cols + col;
10  };
11
12  for (int row = 0; row < rows; ++row) {
13      for (int col = 0; col < cols; ++col) {
14          if (row + 1 < rows &&
15              std::abs(heights[row][col] - heights[row + 1][col]) <= limit) {
16              dsu.unite(id(row, col), id(row + 1, col));
17          }
18          if (col + 1 < cols &&
19              std::abs(heights[row][col] - heights[row][col + 1]) <= limit) {
20              dsu.unite(id(row, col), id(row, col + 1));
21          }
22      }
23  }
24
25  return dsu.find(0) == dsu.find(rows * cols - 1);
26 }
27
28 int minimum_effort_path_by_binary_search(
29     const std::vector<std::vector<int>>& heights)
30 {
31     int low = 0;
32     int high = 1'000'000;
33     while (low < high) {
34         const int mid = low + (high - low) / 2;
35         if (can_reach_with_effort(heights, mid)) {
36             high = mid;
37         } else {
38             low = mid + 1;
39         }
40     }
41     return low;
42 }

```

这段代码只合并向下和向右的边，是因为无向网格中每条相邻边检查一次即可。复杂度是 $O(mn \log H \cdot \alpha(mn))$ ，其中 H 是高度差范围。和 Dijkstra 相比，二分并查集依赖答案单调性；Dijkstra 依赖路径代价的单调扩展性质。两种方法各有适用场景。面试时能同时说出这两种建模，说明你不是在背题，而是在识别“最小化最大值”的结构。

账户合并是并查集的另一个高频场景。题目给出若干账户，每个账户包含姓名和邮箱；只要两个账户共享邮箱，就属于同一个人。暴力方法是两两比较账户邮箱集合，复杂度可能达到 $O(n^2s)$ ，其中 s 是邮箱数量。更好的模型是：邮箱是节点，同一个账户里的所有邮箱属于同一集合。扫描账户时，把第一个邮箱和后续邮箱合并；最后按根收集邮箱，再排序输出。

Listing 13.14 LeetCode 721 账户合并：把邮箱映射为并查集节点

```

1  std::vector<std::vector<std::string>> accounts_merge(
2      const std::vector<std::vector<std::string>>& accounts)
3  {
4      std::unordered_map<std::string, int> email_id;

```

```

5     std::unordered_map<std::string, std::string> email_name;
6
7     for (const std::vector<std::string>& account : accounts) {
8         const std::string& name = account[0];
9         for (int i = 1; i < static_cast<int>(account.size()); ++i) {
10            const std::string& email = account[i];
11            if (!email_id.contains(email)) {
12                const int id = static_cast<int>(email_id.size());
13                email_id.emplace(email, id);
14            }
15            email_name[email] = name;
16        }
17    }
18
19    DisjointSetUnion dsu(static_cast<int>(email_id.size()));
20    for (const std::vector<std::string>& account : accounts) {
21        const int first_id = email_id[account[1]];
22        for (int i = 2; i < static_cast<int>(account.size()); ++i) {
23            dsu.unite(first_id, email_id[account[i]]);
24        }
25    }
26
27    std::unordered_map<int, std::vector<std::string>> groups;
28    for (const auto& [email, id] : email_id) {
29        groups[dsu.find(id)].push_back(email);
30    }
31
32    std::vector<std::vector<std::string>> answer;
33    answer.reserve(groups.size());
34    for (auto& [root, emails] : groups) {
35        (void)root;
36        std::sort(emails.begin(), emails.end());
37        std::vector<std::string> merged;
38        merged.reserve(emails.size() + 1);
39        merged.push_back(email_name[emails.front()]);
40        merged.insert(merged.end(), emails.begin(), emails.end());
41        answer.push_back(std::move(merged));
42    }
43    return answer;
44 }

```

这里的关键不是并查集代码，而是节点选择。若把账户编号作为节点，也能合并共享邮箱的账户，但需要用邮箱反查已经出现过的账户编号；若把邮箱作为节点，则最后天然得到邮箱分组。由于输出要求邮箱有序，每个组还要排序。姓名通常不用于判断合并，只用于输出；如果真实业务中同一邮箱对应多个姓名，就要定义冲突规则，不能默认题目条件成立。刷题题解可以借题目保证，工程代码必须把数据一致性说清楚。

单词接龙是状态图建模的代表。每个单词是节点，若两个单词只差一个字符，就有一条无权边。暴力建图需要两两比较单词，复杂度高。更好的做法是用通配模式建立中间索引，例如 **hot** 可以生成 ***ot**、**h*t**、**ho***。所有拥有同一通配模式的单词互为相邻候选。搜索时从当前单词生成模式，再找到候选单词。由于边权相同，用 BFS，第一次到达目标就是最短转换长度。

Listing 13.15 LeetCode 127 单词接龙：通配模式建索引后 BFS

```

1 int ladder_length(const std::string& begin_word,
2                  const std::string& end_word,
3                  const std::vector<std::string>& word_list)
4 {
5     std::unordered_set<std::string> dictionary(word_list.begin(), word_list.end());
6     if (!dictionary.contains(end_word)) {
7         return 0;
8     }
9     dictionary.insert(begin_word);
10
11     std::unordered_map<std::string, std::vector<std::string>> buckets;
12     for (const std::string& word : dictionary) {
13         for (int i = 0; i < static_cast<int>(word.size()); ++i) {
14             std::string pattern = word;
15             pattern[i] = '*';
16             buckets[pattern].push_back(word);
17         }
18     }
19
20     std::queue<std::string> queue;
21     std::unordered_map<std::string, int> distance;
22     queue.push(begin_word);
23     distance[begin_word] = 1;
24
25     while (!queue.empty()) {
26         const std::string word = queue.front();
27         queue.pop();
28         if (word == end_word) {
29             return distance[word];
30         }
31
32         for (int i = 0; i < static_cast<int>(word.size()); ++i) {
33             std::string pattern = word;
34             pattern[i] = '*';
35             for (const std::string& next : buckets[pattern]) {
36                 if (distance.contains(next)) {
37                     continue;
38                 }
39                 distance[next] = distance[word] + 1;
40                 queue.push(next);
41             }
42         }
43     }
44
45     return 0;
46 }

```

这段代码为了表达清楚保留了字符串拷贝。面试里可以进一步优化：每个模式桶被访问后清空，避免不同路径反复扫描同一桶；也可以使用双向 BFS，从起点集合和终点集合中较小的一侧扩展，降低搜索宽度。暴力为什么慢？因为单词数量为 n 、长度为 L 时，两两比较是 $O(n^2L)$ ；通配索引构建约为 $O(nL^2)$ ，搜索时主要消耗在模式生成和桶扫描。若使用 `string_view` 或预编码单词，工程上还能减少临时字符串分配，但面试中要先保证模型清楚。

双向 BFS 的优化依据是搜索树宽度。单向 BFS 深度为 d 、平均分支因子为 b 时，可能访问约

b^d 个状态；双向 BFS 从两端各扩展约 $d/2$ 层，理想情况下访问量接近 $2b^{d/2}$ 。这不是改变最坏复杂度的魔法，而是利用了许多状态图的指数扩展特征。写双向 BFS 时，永远扩展当前较小的一侧，并在生成新状态时检查它是否已经出现在另一侧。如果相遇，当前步数就是答案。

Listing 13.16 LeetCode 752 打开转盘锁：双向 BFS 的状态集合扩展

```

1 int open_lock(const std::vector<std::string>& deadends,
2               const std::string& target)
3 {
4     const std::unordered_set<std::string> dead(deadends.begin(), deadends.end());
5     const std::string start = "0000";
6     if (dead.contains(start)) {
7         return -1;
8     }
9     if (target == start) {
10        return 0;
11    }
12
13    auto neighbors = [](const std::string& state) {
14        std::vector<std::string> result;
15        result.reserve(8);
16        for (int i = 0; i < static_cast<int>(state.size()); ++i) {
17            for (const int delta : {-1, 1}) {
18                std::string next = state;
19                const int digit = state[i] - '0';
20                next[i] = static_cast<char>('0' + (digit + delta + 10) % 10);
21                result.push_back(std::move(next));
22            }
23        }
24        return result;
25    };
26
27    std::unordered_set<std::string> front{start};
28    std::unordered_set<std::string> back{target};
29    std::unordered_set<std::string> visited{start, target};
30    int steps = 0;
31
32    while (!front.empty() && !back.empty()) {
33        if (front.size() > back.size()) {
34            std::swap(front, back);
35        }
36        std::unordered_set<std::string> next_front;
37        ++steps;
38
39        for (const std::string& state : front) {
40            for (const std::string& next : neighbors(state)) {
41                if (dead.contains(next) || visited.contains(next)) {
42                    continue;
43                }
44                if (back.contains(next)) {
45                    return steps;
46                }
47                visited.insert(next);
48                next_front.insert(next);

```



```

49     }
50     }
51     front = std::move(next_front);
52 }
53
54 return -1;
55 }

```

转盘锁的状态空间最多一万个，因此普通 BFS 也足够；这里展示双向 BFS，是为了训练状态集合扩展的写法。注意 `visited` 同时包含两端访问过的状态，避免重复回头。若先检查 `visited` 再检查 `back`，可能会错过相遇状态，所以上面代码对新状态的处理顺序是：死点跳过，已访问跳过，相遇返回，再加入下一层。也可以把相遇检查放在已访问之前，并单独维护两侧访问集；关键是语义一致。

图题中还有一类“边不是原题里的动作，而是你定义出来的关系”。比如岛屿数量把相邻陆地视为同一连通块；方程除法把变量视为节点、比例视为带权有向边；除法求值时， $a / b = 2$ 可以建立 $a \rightarrow b$ 权重 2 和 $b \rightarrow a$ 权重 $1/2$ ，查询 x / y 就是从 x 到 y 的路径乘积。这个模型不适合普通最短路，因为路径权重不是相加，而是相乘；但 DFS/BFS 沿路径累计乘积即可。若查询很多，还可以用带权并查集维护每个节点到根的比例。

带权并查集比普通并查集难，因为集合关系不仅是“是否连通”，还要维护相对权值。以方程除法为例，`weight[x]` 可以表示 $x / \text{parent}[x]$ 。路径压缩时，需要同时更新 x / root 。合并 x 和 y 且已知 $x / y = \text{value}$ 时，若令 `root_x` 挂到 `root_y`，就要计算 $\text{root}_x / \text{root}_y$ 的值。这个推导容易错，因此面试中如果时间紧，先给 BFS 图解法更稳；只有查询量大或题目明确动态合并时，再写带权并查集。

网格图的另一个高频变化是“状态不只包含位置”。最短路径消除障碍物中，状态应为 `(row, col, remaining)`，表示走到某个格子时还剩多少次消除机会。若只用 `visited[row][col]`，会错误地剪掉更优状态：同样到达一个格子，剩余消除次数越多，后续能力越强。正确的访问标记可以是三维布尔数组，也可以记录到达每个格子时见过的最大剩余次数；当新状态剩余次数不超过历史最好时，才可以剪掉。

Listing 13.17 LeetCode 1293 网格中的最短路径：位置加资源才是完整状态

```

1 int shortest_path_with_obstacle_elimination(
2     const std::vector<std::vector<int>>& grid,
3     int eliminations)
4 {
5     constexpr std::array<std::pair<int, int>, 4> DIRECTIONS{
6         std::pair{1, 0}, std::pair{-1, 0},
7         std::pair{0, 1}, std::pair{0, -1}
8     };
9     const int rows = static_cast<int>(grid.size());
10    const int cols = static_cast<int>(grid[0].size());
11
12    struct State {
13        int row;
14        int col;
15        int remaining;
16        int steps;
17    };
18
19    std::vector<std::vector<int>> best_remaining(
20        rows, std::vector<int>(cols, -1));
21    std::queue<State> queue;
22    queue.push(State{0, 0, eliminations, 0});

```

```

23     best_remaining[0][0] = eliminations;
24
25     while (!queue.empty()) {
26         const State state = queue.front();
27         queue.pop();
28         if (state.row == rows - 1 && state.col == cols - 1) {
29             return state.steps;
30         }
31
32         for (const auto [dr, dc] : DIRECTIONS) {
33             const int next_row = state.row + dr;
34             const int next_col = state.col + dc;
35             const bool out_of_bounds =
36                 next_row < 0 || next_row >= rows ||
37                 next_col < 0 || next_col >= cols;
38             if (out_of_bounds) {
39                 continue;
40             }
41             const int next_remaining =
42                 state.remaining - grid[next_row][next_col];
43             if (next_remaining < 0 ||
44                 next_remaining <= best_remaining[next_row][next_col]) {
45                 continue;
46             }
47             best_remaining[next_row][next_col] = next_remaining;
48             queue.push(State{next_row, next_col, next_remaining, state.steps + 1});
49         }
50     }
51
52     return -1;
53 }

```

这个题非常适合训练“状态支配”思想。到达同一格子时，步数由 BFS 层保证是非降的；如果某个新状态剩余消除次数更少或相等，那么它在未来不会比历史状态更好，可以剪掉。若新状态剩余次数更多，即使步数不更短，也可能穿过更多障碍，不能剪。状态支配比简单 `visited` 更细，是很多高阶 BFS 的核心。

深入理解

从源码和工程角度看，图算法的性能经常被内存访问模式决定。邻接表遍历时，外层 `vector` 的每个桶指向独立存储；节点度数分布很不均匀时，热点节点的邻接边会形成大块连续扫描，而大量低度节点会带来分散访问。前向星或 CSR 风格结构把所有边压进连续数组，遍历时更接近顺序读。Linux 内核大量使用侵入式链表、红黑树、基数树或 XArray 这类结构，是因为内核对象本身已经存在，数据结构常常不拥有对象，只负责把对象链接进某种索引。刷题不需要照搬内核写法，但要理解背后的原则：结构选择服务于访问模式、生命周期和更新频率。

实验建议：选三道题做横向对比。第一道用 Dijkstra 和二分并查集分别解决最小体力路径，记录两种建模的状态、转移和复杂度。第二道用单向 BFS 与双向 BFS 解决打开转盘锁，统计访问状态数量。第三道把课程表从“能否完成”改成“给出一种学习顺序”，再改成“每学期最多学 k 门课时最少需要几学期”，观察拓扑排序、BFS 和状态压缩之间的边界。实验报告不要求长，但必须写清楚为什么一种模型能替代另一种模型，什么时候不能替代。

图论进阶选型

只问是否连通或连通块数量，优先考虑并查集或 BFS/DFS；问无权最短步数，优先 BFS；问非负权最短距离，优先 Dijkstra；问能否在某个阈值下连通，可以考虑二分答案加 BFS/并查集；问所有边反复松弛且可能有负边，才考虑 Bellman-Ford。先选模型，再写代码。

第 14 章

动态规划进阶、容器原理和源码阅读

刷题到中后期，很多错误不再来自不会写循环，而来自状态定义不稳定、容器成本估计错误、以及对标准库行为理解不够。动态规划进阶要把问题拆成“选择、容量、阶段、状态”；C++ 容器原理要知道数据如何存储、什么时候扩容、迭代器何时失效；源码阅读要能从接口行为反推实现约束。面试中如果能同时讲算法和容器成本，答案会明显更可靠。

0-1 背包是 DP 进阶的基础模型。给定若干物品，每个物品只能选一次，有重量和价值，背包容量有限，求最大价值。暴力方法是枚举每个物品选或不选，复杂度 $O(2^n)$ 。状态可以定义为 `dp[i][cap]`：只考虑前 `i` 个物品、容量为 `cap` 时的最大价值。第 `i` 个物品要么不选，要么在容量足够时选择。

Listing 14.1 0-1 背包：一维滚动数组

```
1 int knapsack_01(const std::vector<int>& weights,
2               const std::vector<int>& values,
3               int capacity)
4 {
5     std::vector<int> dp(capacity + 1, 0);
6
7     for (int i = 0; i < static_cast<int>(weights.size()); ++i) {
8         for (int cap = capacity; cap >= weights[i]; --cap) {
9             dp[cap] = std::max(dp[cap], dp[cap - weights[i]] + values[i]);
10        }
11    }
12
13    return dp[capacity];
14 }
```

一维滚动数组必须倒序遍历容量，因为每个物品只能使用一次。若正序遍历，`dp[cap - weights[i]]` 可能已经包含当前物品，等价于重复选择。这个遍历方向就是 0-1 背包和完全背包最重要的区别。面试中不要只背公式，要说清楚倒序是为了让当前物品的转移读取上一轮状态。

分割等和子集是 0-1 背包的判定版本。题目问数组能否分成两个和相等的子集。总和若为奇数，直接不可能；否则问题变成能否从数组中选一些数，使和为 `sum / 2`。每个数只能选一次，所以是 0-1 背包。状态 `dp[cap]` 表示是否能凑出容量 `cap`。

Listing 14.2 LeetCode 416 分割等和子集：0-1 背包可达性

```
1 bool can_partition(const std::vector<int>& nums)
2 {
3     const int total = std::accumulate(nums.begin(), nums.end(), 0);
4     if (total % 2 != 0) {
5         return false;
6     }
7
8     const int target = total / 2;
9     std::vector<bool> dp(target + 1, false);
10    dp[0] = true;
11
12    for (const int x : nums) {
13        for (int cap = target; cap >= x; --cap) {
```

```

14         dp[cap] = dp[cap] || dp[cap - x];
15     }
16 }
17
18 return dp[target];
19 }

```

这题的关键不是代码，而是把“分成两个相等子集”转成“选择若干元素凑出总和一半”。如果数组元素都是正数，背包容量有限，DP 很自然；如果存在负数，容量模型就不直接适用，需要偏移或换思路。复杂度是 $O(n \cdot \text{target})$ ，其中 **target** 与数值大小有关，所以这是伪多项式复杂度，不是严格输入长度多项式。

目标和可以转成背包计数。给每个数加正号或负号，使表达式等于 target。设正数集合和为 P，负数集合和为 N，总和 S。则 $P - N = \text{target}$ ， $P + N = S$ ，推出 $P = (S + \text{target}) / 2$ 。问题变成从数组中选若干数，使和为 P 的方案数。暴力枚举符号是 $O(2^n)$ ；DP 保存每个和的方案数。

Listing 14.3 LeetCode 494 目标和：转换为 0-1 背包计数

```

1 int find_target_sum_ways(const std::vector<int>& nums, int target)
2 {
3     const int total = std::accumulate(nums.begin(), nums.end(), 0);
4     if (std::abs(target) > total || (total + target) % 2 != 0) {
5         return 0;
6     }
7
8     const int positive_sum = (total + target) / 2;
9     std::vector<int> dp(positive_sum + 1, 0);
10    dp[0] = 1;
11
12    for (const int x : nums) {
13        for (int sum = positive_sum; sum >= x; --sum) {
14            dp[sum] += dp[sum - x];
15        }
16    }
17
18    return dp[positive_sum];
19 }

```

这题要特别注意 0。若数组里有 0，给它加正号或负号都不改变和，但方案数应翻倍。上述 DP 能自然处理：当 $x == 0$ 时， $\text{dp}[\text{sum}] += \text{dp}[\text{sum}]$ ，所有已有方案翻倍。很多手写特殊逻辑反而容易错。计数型 DP 要关注整型范围，力扣原题结果通常能放进 int，但工程中应考虑 long long。

零钱兑换 II 是完全背包计数。硬币可以无限使用，问凑成金额的组合数。注意是组合数，不是排列数。若外层遍历金额、内层遍历硬币，会把不同顺序当作不同方案；若外层遍历硬币、内层正序遍历金额，每种硬币按固定顺序加入，就只统计组合。

Listing 14.4 LeetCode 518 零钱兑换 II：完全背包组合计数

```

1 int change(int amount, const std::vector<int>& coins)
2 {
3     std::vector<int> dp(amount + 1, 0);
4     dp[0] = 1;
5
6     for (const int coin : coins) {
7         for (int value = coin; value <= amount; ++value) {
8             dp[value] += dp[value - coin];
9         }
10    }
11
12    return dp[amount];
13 }

```

```

9     }
10    }
11
12    return dp[amount];
13 }

```

为什么容量正序？因为当前硬币可以重复使用，`dp[value - coin]` 应该允许已经使用当前硬币。为什么硬币在外层？因为组合不关心顺序，我们规定按硬币种类逐步开放选择。这个题非常适合区分 0-1 背包、完全背包、组合计数和排列计数。只要循环顺序解释不清，背包题就还没有真正掌握。

股票题是状态机 DP。买卖股票的最佳时机 I 只允许一次交易，最简单做法是扫描时维护历史最低买入价和当前最大利润。暴力枚举买入日和卖出日是 $O(n^2)$ ；优化来自“卖出日固定时，最优买入日一定是它之前价格最低的一天”。

Listing 14.5 LeetCode 121 买卖股票 I：维护历史最低价格

```

1 int max_profit_one_transaction(const std::vector<int>& prices)
2 {
3     int min_price = std::numeric_limits<int>::max();
4     int answer = 0;
5
6     for (const int price : prices) {
7         min_price = std::min(min_price, price);
8         answer = std::max(answer, price - min_price);
9     }
10
11    return answer;
12 }

```

含冷冻期的股票题不能只维护最低价，因为今天能不能买取决于前一天是否卖出。状态机更清楚：`hold` 表示今天结束时持有股票的最大收益；`sold` 表示今天刚卖出；`rest` 表示今天结束时不持有且不在刚卖出的状态。转移时，买入只能从 `rest` 来，卖出只能从 `hold` 来，休息可以从昨天的 `rest` 或 `sold` 来。

Listing 14.6 LeetCode 309 含冷冻期股票：三状态 DP

```

1 int max_profit_with_cooldown(const std::vector<int>& prices)
2 {
3     if (prices.empty()) {
4         return 0;
5     }
6
7     int hold = -prices[0];
8     int sold = 0;
9     int rest = 0;
10
11    for (int i = 1; i < static_cast<int>(prices.size()); ++i) {
12        const int previous_hold = hold;
13        const int previous_sold = sold;
14        const int previous_rest = rest;
15
16        hold = std::max(previous_hold, previous_rest - prices[i]);
17        sold = previous_hold + prices[i];
18        rest = std::max(previous_rest, previous_sold);

```



```

19     }
20
21     return std::max(sold, rest);
22 }

```

这段代码的关键是先保存昨天状态，避免同一天内状态互相污染。返回值不能是 `hold`，因为结束时仍持有股票不代表已经实现收益。股票 DP 的复盘模板是：每天结束时有哪些状态，状态之间允许哪些动作，动作是否受冷冻期、手续费、交易次数限制。把限制放进状态机，公式就会自然出现。

动态规划最容易被学成公式库，但公式不是起点。起点永远是暴力搜索：有哪些选择，搜索树的节点表示什么，哪些节点会重复出现。以最长递增子序列为例，暴力做法是对每个元素选择“选或不选”，同时维护上一个被选元素，复杂度指数级。重复子问题来自“处理到某个位置、上一个选择约束相同”时，后续选择空间完全一样。第一种 DP 定义可以是 `dp[i]`：以 `nums[i]` 结尾的最长递增子序列长度。这样每个状态都问一个局部稳定的问题：最后一个元素固定为 `i`，前一个元素只能从 `0..i-1` 中找，并且值要更小。

Listing 14.7 LeetCode 300 最长递增子序列：二次 DP 和路径还原

```

1 std::vector<int> reconstruct_lis(const std::vector<int>& nums)
2 {
3     const int n = static_cast<int>(nums.size());
4     if (n == 0) {
5         return {};
6     }
7
8     std::vector<int> dp(n, 1);
9     std::vector<int> parent(n, -1);
10    int best_index = 0;
11
12    for (int i = 0; i < n; ++i) {
13        for (int j = 0; j < i; ++j) {
14            if (nums[j] >= nums[i] || dp[j] + 1 <= dp[i]) {
15                continue;
16            }
17            dp[i] = dp[j] + 1;
18            parent[i] = j;
19        }
20        if (dp[i] > dp[best_index]) {
21            best_index = i;
22        }
23    }
24
25    std::vector<int> sequence;
26    for (int index = best_index; index != -1; index = parent[index]) {
27        sequence.push_back(nums[index]);
28    }
29    std::reverse(sequence.begin(), sequence.end());
30    return sequence;
31 }

```

这段代码不是最优复杂度，但它最适合讲清楚状态。`parent[i]` 记录让 `dp[i]` 变大的前驱下标，所以最后可以从最优结尾倒着还原路径。正确性来自一个简单不变量：外层处理到 `i` 时，所有 `j < i` 的 `dp[j]` 已经是以前 `j` 结尾的最优答案；若 `nums[j] < nums[i]`，把 `i` 接到以前 `j` 结尾的最

优序列后面，一定得到一个合法递增序列。枚举所有可能前驱后取最大值，就不会漏掉以 i 结尾的最优解。复杂度 $O(n^2)$ ，空间 $O(n)$ 。

最长递增子序列还可以优化到 $O(n \log n)$ 。优化依据不是简单二分，而是维护一个更抽象的数组 **tails**：长度为 $\text{len} + 1$ 的递增子序列，结尾最小值是多少。结尾越小，未来接上新元素的机会越大。扫描每个数 x ，用二分找到第一个大于等于 x 的位置并替换它；如果找不到，就把 x 追加到末尾。**tails** 不一定是一条真实子序列，但它的长度等于 LIS 长度。这个区别必须讲清楚，否则读者会误以为 **tails** 本身就是答案路径。

Listing 14.8 LeetCode 300 最长递增子序列：tails 数组和二分优化

```
1 int length_of_lis(const std::vector<int>& nums)
2 {
3     std::vector<int> tails;
4     tails.reserve(nums.size());
5
6     for (const int x : nums) {
7         auto it = std::lower_bound(tails.begin(), tails.end(), x);
8         if (it == tails.end()) {
9             tails.push_back(x);
10        } else {
11            *it = x;
12        }
13    }
14
15    return static_cast<int>(tails.size());
16 }
```

为什么替换不会破坏答案？因为替换位置 **pos** 的含义是：存在长度为 $\text{pos} + 1$ 的递增子序列以 x 结尾，并且 x 不大于原来的结尾。更小的结尾不会让未来变差，只会让未来更容易扩展。若题目要求严格递增，使用 **lower_bound**；若要求非递减，使用 **upper_bound**。这是面试里常见追问：二分函数的选择取决于相等元素是否允许接在后面。

网格路径 DP 是二维状态的入门。不同路径问从左上到右下只能向右或向下走有多少条路径。暴力递归会从每个格子继续向右和向下，重复计算大量子矩形。状态 $\text{dp}[\text{row}][\text{col}]$ 表示到达当前格子的路径数，它只依赖上方和左方。由于每个状态只依赖上一行和当前行左侧，可以压缩成一维数组。这里的遍历顺序不是随便的：从左到右更新时， $\text{dp}[\text{col}]$ 仍表示上一行同列， $\text{dp}[\text{col} - 1]$ 已经表示当前行左侧。

Listing 14.9 LeetCode 62 不同路径：一维滚动数组

```
1 int unique_paths(int rows, int cols)
2 {
3     std::vector<int> dp(cols, 1);
4     for (int row = 1; row < rows; ++row) {
5         for (int col = 1; col < cols; ++col) {
6             dp[col] += dp[col - 1];
7         }
8     }
9     return dp[cols - 1];
10 }
```

最小路径和与不同路径结构相同，只是“方案数相加”变成“代价取最小”。状态 $\text{dp}[\text{col}]$ 表示处理到当前行时，到达该列的最小路径和。第一行只能从左边来，第一列只能从上边来，其余位置取上方和左方较小者。边界处理是这类题的主要错误来源。一个稳妥写法是先初始化第一行，再

逐行处理第一列和内部格子；不要在双循环里塞很多特殊判断，让主转移被边界噪声淹没。

Listing 14.10 LeetCode 64 最小路径和：边界初始化后主转移

```
1 int min_path_sum(const std::vector<std::vector<int>>& grid)
2 {
3     const int rows = static_cast<int>(grid.size());
4     const int cols = static_cast<int>(grid[0].size());
5     std::vector<int> dp(cols, 0);
6
7     dp[0] = grid[0][0];
8     for (int col = 1; col < cols; ++col) {
9         dp[col] = dp[col - 1] + grid[0][col];
10    }
11
12    for (int row = 1; row < rows; ++row) {
13        dp[0] += grid[row][0];
14        for (int col = 1; col < cols; ++col) {
15            dp[col] = std::min(dp[col], dp[col - 1]) + grid[row][col];
16        }
17    }
18
19    return dp[cols - 1];
20 }
```

编辑距离是字符串 DP 的核心题。给定两个单词，允许插入、删除、替换，求把一个单词变成另一个单词的最少操作数。暴力递归从两个字符串末尾比较：若字符相同，两个指针都左移；若不同，可以删除一个字符、插入一个字符、替换一个字符。暴力慢是因为同一对前缀长度会被多条路径反复访问。状态 $dp[i][j]$ 表示 `word1` 的前 i 个字符变成 `word2` 的前 j 个字符的最少操作数。空串边界很重要： $dp[i][0] = i$, $dp[0][j] = j$ 。

Listing 14.11 LeetCode 72 编辑距离：两行滚动数组

```
1 int min_distance(const std::string& word1, const std::string& word2)
2 {
3     const int n = static_cast<int>(word1.size());
4     const int m = static_cast<int>(word2.size());
5
6     std::vector<int> previous(m + 1, 0);
7     std::vector<int> current(m + 1, 0);
8     for (int j = 0; j <= m; ++j) {
9         previous[j] = j;
10    }
11
12    for (int i = 1; i <= n; ++i) {
13        current[0] = i;
14        for (int j = 1; j <= m; ++j) {
15            if (word1[i - 1] == word2[j - 1]) {
16                current[j] = previous[j - 1];
17                continue;
18            }
19            const int remove_cost = previous[j] + 1;
20            const int insert_cost = current[j - 1] + 1;
21            const int replace_cost = previous[j - 1] + 1;
```

```

22         current[j] = std::min({remove_cost, insert_cost, replace_cost});
23     }
24     std::swap(previous, current);
25 }
26
27 return previous[m];
28 }

```

三个操作的含义要能对上表格位置。删除 `word1[i-1]` 后，问题变成 `word1` 前 `i-1` 个字符到 `word2` 前 `j` 个字符，所以来自 `previous[j]`；插入 `word2[j-1]` 后，等价于先把 `word1` 前 `i` 个字符变成 `word2` 前 `j-1` 个字符，所以来自 `current[j-1]`；替换则来自 `previous[j-1]`。两行滚动数组的空间是 $O(m)$ ，时间是 $O(nm)$ 。如果题目要求还原操作序列，就不能只保留两行，需要记录决策或重新回溯。

最长公共子序列和编辑距离很接近。状态 `dp[i][j]` 表示两个前缀的 LCS 长度。若末尾字符相同，就从 `dp[i-1][j-1] + 1` 转移；否则取删掉任一末尾后的最大值。它和最长公共子串不同：子序列允许不连续，子串必须连续。这个概念差异决定了转移差异。最长公共子串在字符不同时必须归零，LCS 在字符不同时取上方或左方最大值。面试里一旦题目出现“可以删除若干字符但保持相对顺序”，就要想到子序列。

Listing 14.12 LeetCode 1143 最长公共子序列：一维数组和左上角缓存

```

1 int longest_common_subsequence(const std::string& text1,
2                               const std::string& text2)
3 {
4     const int n = static_cast<int>(text1.size());
5     const int m = static_cast<int>(text2.size());
6     std::vector<int> dp(m + 1, 0);
7
8     for (int i = 1; i <= n; ++i) {
9         int diagonal = 0;
10        for (int j = 1; j <= m; ++j) {
11            const int old_up = dp[j];
12            if (text1[i - 1] == text2[j - 1]) {
13                dp[j] = diagonal + 1;
14            } else {
15                dp[j] = std::max(dp[j], dp[j - 1]);
16            }
17            diagonal = old_up;
18        }
19    }
20
21    return dp[m];
22 }

```

一维 LCS 的难点在于保存左上角。更新前的 `dp[j]` 是上一行同列，`dp[j-1]` 是当前行左侧，`diagonal` 是上一行左上角。每次循环结束前把旧的 `dp[j]` 放进 `diagonal`，供下一列使用。若你觉得这个写法难以保证正确，可以先写二维表；面试时清楚解释二维转移比写一个容易错的一维优化更重要。空间优化应该建立在状态依赖已经完全理解之后。

单词拆分是“前缀可达性”DP。题目问字符串能否被字典中的单词拼接出来。暴力方法枚举第一段单词，然后递归处理剩余后缀；同一个起点会被反复递归。状态 `dp[i]` 表示前 `i` 个字符能否被拆分。转移枚举最后一个单词的起点 `j`：若 `dp[j]` 为真，且 `s[j..i)` 在字典中，则 `dp[i]` 为真。这里要注意 `substr` 的成本，它会创建新字符串。数据量不大时可接受；数据量大时，应限制最大单词长度，或用字典树、字符串视图、滚动哈希减少复制。

Listing 14.13 LeetCode 139 单词拆分：限制最大单词长度降低无效枚举

```

1 bool word_break(const std::string& s,
2               const std::vector<std::string>& word_dict)
3 {
4     std::unordered_set<std::string> words(word_dict.begin(), word_dict.end());
5     int max_word_length = 0;
6     for (const std::string& word : word_dict) {
7         max_word_length = std::max(max_word_length,
8                                   static_cast<int>(word.size()));
9     }
10
11     const int n = static_cast<int>(s.size());
12     std::vector<char> dp(n + 1, false);
13     dp[0] = true;
14
15     for (int end = 1; end <= n; ++end) {
16         const int begin_limit = std::max(0, end - max_word_length);
17         for (int begin = end - 1; begin >= begin_limit; --begin) {
18             if (!dp[begin]) {
19                 continue;
20             }
21             if (words.contains(s.substr(begin, end - begin))) {
22                 dp[end] = true;
23                 break;
24             }
25         }
26     }
27
28     return dp[n];
29 }

```

这个优化不是改变问题模型，而是减少不可能成为单词的长度。若字典中最长单词长度为 L ，枚举起点时只需要看最近 L 个位置。复杂度粗略为 $O(nL)$ 次查询，但每次 `substr` 仍有拷贝成本。面试表达可以分层：先给清楚的 DP，再说明字符串切片成本，最后提出可选优化。不要一上来写复杂字典树，把主线埋掉。

区间 DP 的状态通常是“处理一个连续区间的最优值”。它与普通线性 DP 的区别是：最后一步可能把区间切成两半，或者选择区间内某个点作为最后处理对象。戳气球是代表题。直接思考“先戳哪个气球”很混乱，因为戳掉一个气球会改变邻居；换成“最后戳哪个气球”就稳定了。设 `dp[left][right]` 表示开区间 $(left, right)$ 内的气球全部戳掉能获得的最大硬币数。若最后戳 `mid`，此时它左右邻居一定是 `left` 和 `right`，收益为 `nums[left] * nums[mid] * nums[right]`，左右两边互不影响。

Listing 14.14 LeetCode 312 戳气球：选择最后一个被戳的气球

```

1 int max_coins(const std::vector<int>& nums)
2 {
3     std::vector<int> values;
4     values.reserve(nums.size() + 2);
5     values.push_back(1);
6     values.insert(values.end(), nums.begin(), nums.end());
7     values.push_back(1);
8
9     const int n = static_cast<int>(values.size());

```

```

10     std::vector<std::vector<int>>> dp(n, std::vector<int>(n, 0));
11
12     for (int length = 2; length < n; ++length) {
13         for (int left = 0; left + length < n; ++left) {
14             const int right = left + length;
15             for (int mid = left + 1; mid < right; ++mid) {
16                 const int score = dp[left][mid] + dp[mid][right] +
17                     values[left] * values[mid] * values[right];
18                 dp[left][right] = std::max(dp[left][right], score);
19             }
20         }
21     }
22
23     return dp[0][n - 1];
24 }

```

区间 DP 的遍历顺序按区间长度从小到大，因为大区间依赖小区间。这里的边界 1 是虚拟气球，不会被戳，只是让左右边界收益统一。正确性要抓住“最后一步”：如果 `mid` 是最后被戳的气球，那么 `left..mid` 和 `mid..right` 两个子问题已经在它之前完成，且互相独立；枚举所有可能的最后气球，取最大值。复杂度是 $O(n^3)$ ，空间 $O(n^2)$ 。若面试官追问能否优化，要先说明一般区间 DP 不一定有四边形不等式或决策单调性，不能乱套优化。

状态压缩 DP 把集合放进整数二进制位。适用条件通常是元素数量不大，例如 $n \leq 20$ 。每一位表示某个任务、节点或物品是否已经被选择。经典问题是旅行商或访问所有节点的最短路径。若图无权，访问所有节点的最短路径可以用 BFS，把状态定义为 `(node, mask)`：当前在 `node`，已经访问的节点集合是 `mask`。这其实是状态压缩和图 BFS 的结合，而不是传统表格 DP。

Listing 14.15 LeetCode 847 访问所有节点的最短路径：节点加集合状态 BFS

```

1 int shortest_path_length(const std::vector<std::vector<int>>& graph)
2 {
3     const int n = static_cast<int>(graph.size());
4     const int full_mask = (1 << n) - 1;
5
6     std::queue<std::pair<int, int>> queue;
7     std::vector<std::vector<int>>> distance(n, std::vector<int>(1 << n, -1));
8
9     for (int node = 0; node < n; ++node) {
10         const int mask = 1 << node;
11         distance[node][mask] = 0;
12         queue.emplace(node, mask);
13     }
14
15     while (!queue.empty()) {
16         const auto [node, mask] = queue.front();
17         queue.pop();
18         if (mask == full_mask) {
19             return distance[node][mask];
20         }
21
22         for (const int next : graph[node]) {
23             const int next_mask = mask | (1 << next);
24             if (distance[next][next_mask] != -1) {
25                 continue;

```



```

26         }
27         distance[next][next_mask] = distance[node][mask] + 1;
28         queue.emplace(next, next_mask);
29     }
30 }
31
32 return 0;
33 }

```

这题如果只把节点作为访问标记，会出错。到达同一个节点时，已访问集合不同，未来能力不同。**(node, mask)** 才是完整状态。多源初始化也很关键：路径可以从任意节点开始，所以所有单节点状态都在第 0 层。复杂度是 $O(n2^n + m2^n)$ ，只适合小 n 。状态压缩题要先估算 2^n 是否可承受，再决定是否使用；看到集合并不意味着一定能压缩，如果 n 是一百，二进制集合就不现实。

另一类状态压缩是子集枚举。比如把数组分成若干组、每组和相等。可以用 **dp[mask]** 表示选择集合 **mask** 后，当前桶中已经放入的对目标桶容量取模是多少；若 **dp[mask] == -1** 表示不可达。每次尝试加入一个未选元素，只要当前桶不超容量，就更新新集合。这个写法比回溯更像 DP，因为每个集合状态只计算一次。

Listing 14.16 LeetCode 698 划分为 K 个等和子集：集合状态 DP

```

1 bool can_partition_k_subsets(std::vector<int> nums, int k)
2 {
3     const int total = std::accumulate(nums.begin(), nums.end(), 0);
4     if (total % k != 0) {
5         return false;
6     }
7     const int target = total / k;
8     std::sort(nums.begin(), nums.end(), std::greater<int>());
9     if (!nums.empty() && nums.front() > target) {
10         return false;
11     }
12
13     const int n = static_cast<int>(nums.size());
14     const int state_count = 1 << n;
15     std::vector<int> dp(state_count, -1);
16     dp[0] = 0;
17
18     for (int mask = 0; mask < state_count; ++mask) {
19         if (dp[mask] == -1) {
20             continue;
21         }
22         for (int i = 0; i < n; ++i) {
23             if ((mask & (1 << i)) != 0) {
24                 continue;
25             }
26             const int next_sum = dp[mask] + nums[i];
27             if (next_sum > target) {
28                 continue;
29             }
30             const int next_mask = mask | (1 << i);
31             dp[next_mask] = next_sum % target;
32         }
33     }

```

```

34
35     return dp[state_count - 1] == 0;
36 }

```

这个 DP 的状态值不是总和，而是当前桶的剩余进度。每当一个桶刚好装满，对 `target` 取模后的进度回到 0，表示开始装下一个桶。排序不是正确性必需，但大的数先处理能更早剪掉不可能状态。复杂度是 $O(n2^n)$ 。与回溯相比，状态 DP 的优势是避免不同选择顺序到达同一集合时重复搜索；缺点是内存随 2^n 增长。

单调队列优化 DP 是从“转移枚举范围”中挤出复杂度。以带限制的最大子序列和为例，题目要求选择一个非空子序列，相邻被选元素下标差最多为 k ，求最大和。朴素 DP 定义 `dp[i]` 为必须选择 `nums[i]` 且以它结尾的最大和，则

$$dp[i] = nums[i] + \max(0, dp[j]) \quad i - k \leq j < i.$$

如果每个 i 都扫描前 k 个位置，复杂度 $O(nk)$ 。优化来自窗口最大值：我们只需要最近 k 个 `dp[j]` 的最大值，用单调队列维护即可。

Listing 14.17 LeetCode 1425 受限子序列和：单调队列优化转移最大值

```

1 int constrained_subset_sum(const std::vector<int>& nums, int k)
2 {
3     std::deque<int> deque;
4     std::vector<int> dp(nums.size(), 0);
5     int answer = std::numeric_limits<int>::min();
6
7     for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
8         while (!deque.empty() && deque.front() < i - k) {
9             deque.pop_front();
10        }
11
12        dp[i] = nums[i];
13        if (!deque.empty()) {
14            dp[i] = std::max(dp[i], nums[i] + dp[deque.front()]);
15        }
16        answer = std::max(answer, dp[i]);
17
18        while (!deque.empty() && dp[deque.back()] <= dp[i]) {
19            deque.pop_back();
20        }
21        if (dp[i] > 0) {
22            deque.push_back(i);
23        }
24    }
25
26    return answer;
27 }

```

队列中保存下标，而不是保存值，因为要判断是否过期。队首永远是当前窗口内 `dp` 最大的位置。只把正的 `dp[i]` 放入队列，是因为负贡献不如从当前元素重新开始。正确性来自两个不变量：队列下标从前到后递增，保证能判断窗口；对应 `dp` 值从前到后递减，保证队首最大。每个下标最多进队一次、出队一次，所以总复杂度 $O(n)$ 。

动态规划和最短路之间有很深的联系。许多 DP 可以看成在有向无环图上找路径：状态是节点，转移是边，遍历顺序是拓扑序。若状态图有环但边权非负，可能变成 Dijkstra；若边权全为 1，可能变成 BFS；若存在负边但没有负环，可能变成 Bellman-Ford。这样理解后，题型边界会更清楚：不

是所有“求最优”都叫 DP，也不是所有“状态转移”都要填表。你要先问状态图是否有自然阶段顺序，是否允许回头，转移代价是否单调。

常见误区

DP 常见误区有四个。第一，只背公式，不说明状态含义，导致换一个题面就失效。第二，滚动数组时忘记转移读的是上一阶段还是当前阶段。第三，把组合数和排列数混在一起，循环顺序写反。第四，看到最值就写 DP，却没有检查状态数量是否可承受。每次写完 DP，都要用一句话解释 **dp** 的含义，用一句话解释遍历顺序，用一句话解释为什么不会漏解或重复计数。

C++ 容器原理是刷题代码质量的底座。**std::vector** 是连续数组，支持随机访问，扩容时会搬迁元素，导致指针、引用和迭代器失效。**std::deque** 分段连续，支持两端高效插入删除，但随机访问局部性通常不如 **vector**。**std::list** 节点稳定，但缓存局部性差，刷题中很少是首选。**std::unordered_map** 平均常数查询，但 rehash 会让迭代器失效，且不维护顺序。**std::map** 有序但每次操作是 $O(\log n)$ 。

阅读标准库源码时，不需要一开始钻进所有模板细节。先看三层：公开接口、核心数据成员、关键操作路径。以 **vector** 为例，大多数实现内部会维护三个指针或等价状态：开始位置、当前结束位置、容量结束位置。**size()** 是当前结束减开始，**capacity()** 是容量结束减开始，**push_back** 在容量足够时原地构造，在容量不足时申请更大内存并移动旧元素。理解这三根指针，就能解释很多迭代器失效规则。

Listing 14.18 vector 使用习惯：reserve、resize 和引用失效

```
1 void collect_values(const std::vector<int>& input)
2 {
3     std::vector<int> values;
4     values.reserve(input.size());
5
6     for (const int x : input) {
7         values.push_back(x * 2);
8     }
9
10    const int* data = values.data();
11    values.push_back(42);
12    // 如果 push_back 触发扩容，data 已经失效，不能继续解引用。
13 }
```

reserve 和 **resize** 的区别必须非常熟。**reserve** 只保证容量，不改变大小；**resize** 改变大小并构造元素。DP 数组需要可下标访问，应使用 **vector<int> dp(n + 1)** 或 **resize**；收集答案时知道最多数量的，可以先 **reserve** 再 **push_back**。把二者混用，是 C++ 刷题里非常常见的越界来源。

unordered_map 的源码阅读重点是桶数组、哈希函数、负载因子和 rehash。插入元素时，容器通过哈希值定位桶，再在桶内查找等价 key。平均常数复杂度依赖哈希分布；当元素数量相对桶数过多，会触发 rehash，重新分配桶并把元素放到新桶中。刷题时使用 **reserve** 可以减少 rehash 次数，尤其是两数之和、前缀和计数、频次统计这类一次性插入较多 key 的题。

Listing 14.19 unordered_map 查询：contains 与 find 的取舍

```
1 int count_frequency(const std::vector<int>& nums, int target)
2 {
3     std::unordered_map<int, int> frequency;
4     frequency.reserve(nums.size());
5
6     for (const int x : nums) {
```

```

7     ++frequency[x];
8 }
9
10    const auto found = frequency.find(target);
11    if (found == frequency.end()) {
12        return 0;
13    }
14    return found->second;
15 }

```

如果只判断存在，C++20 的 `contains` 很清楚；如果判断后还要取值，用 `find` 避免查两次。不要用 `operator[]` 做纯查询，因为 key 不存在时会插入默认值，改变容器状态。这个细节在前缀和计数里尤其重要：有些查询应该只读历史状态，不能悄悄插入一个 0。

源码阅读也能帮助理解算法复杂度的真实成本。字符串 `substr` 会创建新字符串，放在双层循环里可能把 $O(n^2)$ 变成接近 $O(n^3)$ 的实际成本；`vector<vector<int>>` 的每一行是独立分配，矩阵遍历仍然可用，但内存不一定完全连续；`priority_queue` 没有删除任意元素和 `decrease-key`，所以 Dijkstra 常用“压入新状态，弹出时跳过过期状态”的写法。理解容器能力边界，才能解释为什么代码这样写。

`std::deque` 的名字容易误导初学者。它不是双向链表，而是分段连续数组。典型实现会维护一个“map”，其中每个槽指向一段固定大小的缓冲区；首尾插入删除大多只移动段内指针，必要时分配新段。它支持随机访问，但随机访问需要先定位段再定位段内偏移，局部性通常不如 `vector`。这解释了为什么滑动窗口最大值适合用 `deque` 存下标，而普通顺序 DP 数组不应该为了“可能两端操作”随手换成 `deque`。

Listing 14.20 deque 在单调队列中的正确用法

```

1  std::vector<int> max_sliding_window(const std::vector<int>& nums, int k)
2  {
3      std::deque<int> deque;
4      std::vector<int> answer;
5      answer.reserve(nums.size());
6
7      for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
8          while (!deque.empty() && deque.front() <= i - k) {
9              deque.pop_front();
10         }
11         while (!deque.empty() && nums[deque.back()] <= nums[i]) {
12             deque.pop_back();
13         }
14         deque.push_back(i);
15         if (i + 1 >= k) {
16             answer.push_back(nums[deque.front()]);
17         }
18     }
19
20     return answer;
21 }

```

这里 `deque` 保存下标而不是值，原因和单调队列 DP 一样：需要判断过期位置。下标还能处理重复值，若只存值，窗口移出时很难知道队首值是否对应离开的元素。源码层面看，`pop_front` 和 `push_back` 都是 `deque` 的强项；若用 `vector` 在头部删除，就会移动大量元素；若用 `list`，虽然删除快，但无法高效访问队尾并且缓存局部性差。容器选择和算法不变量应该匹配。

`std::string` 在现代实现中通常具有小字符串优化。短字符串可以直接存放在对象内部，不必

单独堆分配；长字符串才指向堆内存。这个事实能解释为什么偶尔创建短字符串不一定灾难，但不能因此在热循环里无限 `substr`。标准并不要求具体的小字符串容量，不同标准库实现也不同，因此工程代码不能依赖“小于多少字符一定不分配”。源码阅读要区分“标准保证的语义”和“某个实现的优化”。

Listing 14.21 字符串扫描：用下标边界避免大量 `substr`

```
1 bool has_word_at(const std::string& text,
2                 int begin,
3                 const std::string& word)
4 {
5     if (begin + static_cast<int>(word.size()) >
6         static_cast<int>(text.size())) {
7         return false;
8     }
9     for (int i = 0; i < static_cast<int>(word.size()); ++i) {
10        if (text[begin + i] != word[i]) {
11            return false;
12        }
13    }
14    return true;
15 }
```

这段代码展示的是思想：当你只需要比较一段文本时，不一定要构造新字符串。C++20 可以使用 `std::string_view` 表示非拥有切片，但要注意生命周期；视图不能比原字符串活得更久。刷题中 `substr` 更短、更不易错，数据量允许时可以使用；工程中如果它处在双层或三层循环内，就应该估算分配和拷贝成本。

`std::priority_queue` 是容器适配器，默认底层容器通常是 `vector`，并用堆算法维护最大堆。它只暴露 `top`、`push`、`pop`，不支持遍历、不支持删除任意元素、不支持 `decrease-key`。这些限制不是缺陷，而是接口选择：标准库给的是简单、稳定、低成本的堆抽象。Dijkstra 中距离变小后重新入堆，是顺应这个接口；双堆中位数中延迟删除，也是因为堆不能直接删中间元素。

Listing 14.22 LeetCode 480 滑动窗口中位数：双堆延迟删除

```
1 class DualHeap {
2 public:
3     explicit DualHeap(int window_size) : window_size_(window_size) {}
4
5     void insert(int value)
6     {
7         if (this->small_.empty() || value <= this->small_.top()) {
8             this->small_.push(value);
9             ++this->small_size_;
10        } else {
11            this->large_.push(value);
12            ++this->large_size_;
13        }
14        this->rebalance();
15    }
16
17     void erase(int value)
18     {
19         ++this->delayed_[value];
20         if (value <= this->small_.top()) {
```

```

21         --this->small_size_;
22         if (value == this->small_.top()) {
23             this->prune_small();
24         }
25     } else {
26         --this->large_size_;
27         if (!this->large_.empty() && value == this->large_.top()) {
28             this->prune_large();
29         }
30     }
31     this->rebalance();
32 }
33
34 double median() const
35 {
36     if (this->window_size_ % 2 == 1) {
37         return this->small_.top();
38     }
39     return (static_cast<double>(this->small_.top()) +
40             static_cast<double>(this->large_.top())) / 2.0;
41 }
42
43 private:
44     void rebalance()
45     {
46         if (this->small_size_ > this->large_size_ + 1) {
47             this->large_.push(this->small_.top());
48             this->small_.pop();
49             --this->small_size_;
50             ++this->large_size_;
51             this->prune_small();
52         } else if (this->small_size_ < this->large_size_) {
53             this->small_.push(this->large_.top());
54             this->large_.pop();
55             ++this->small_size_;
56             --this->large_size_;
57             this->prune_large();
58         }
59     }
60
61     void prune_small()
62     {
63         while (!this->small_.empty()) {
64             const int value = this->small_.top();
65             auto found = this->delayed_.find(value);
66             if (found == this->delayed_.end()) {
67                 return;
68             }
69             if (--found->second == 0) {
70                 this->delayed_.erase(found);
71             }
72             this->small_.pop();
73         }

```



```

74     }
75
76     void prune_large()
77     {
78         while (!this->large_.empty()) {
79             const int value = this->large_.top();
80             auto found = this->delayed_.find(value);
81             if (found == this->delayed_.end()) {
82                 return;
83             }
84             if (--found->second == 0) {
85                 this->delayed_.erase(found);
86             }
87             this->large_.pop();
88         }
89     }
90
91     int window_size_;
92     std::priority_queue<int> small_;
93     std::priority_queue<int, std::vector<int>, std::greater<int>> large_;
94     std::unordered_map<int, int> delayed_;
95     int small_size_ = 0;
96     int large_size_ = 0;
97 };

```

这段结构比普通数据流中位数复杂，因为窗口滑动时需要删除离开的元素。既然堆不能删除任意位置，就把待删除值记在 `delayed_` 中；只有当它出现在堆顶时才真正弹出。`small_size_` 和 `large_size_` 记录的是有效元素数量，不是堆内部实际元素数量。成员访问使用 `this->` 是为了明确哪些变量属于对象状态。复杂度均摊 $O(\log k)$ 。如果用 `multiset`，删除任意元素更直接，但常数和迭代器维护方式不同；两种方案都要能解释取舍。

`std::map` 和 `std::set` 通常由红黑树实现，保证有序遍历和 $O(\log n)$ 插入、删除、查找。红黑树是一种近似平衡二叉搜索树，插入删除后通过旋转和染色维持高度对数级。面试刷题里，使用 `map` 的信号通常是需有序键、前驱后继、区间边界或按顺序输出。若只需要存在性或计数，`unordered_map` 平均更快；若哈希可能被攻击、需要稳定顺序或需要 `lower_bound`，就选有序树。

Listing 14.23 有序映射：用 `upper_bound` 找右侧区间

```

1 int count_covered_points(const std::map<int, int>& intervals, int point)
2 {
3     const auto right = intervals.upper_bound(point);
4     if (right == intervals.begin()) {
5         return 0;
6     }
7     const auto candidate = std::prev(right);
8     return candidate->second >= point ? 1 : 0;
9 }

```

`upper_bound(point)` 返回第一个起点大于 `point` 的区间，前一个区间才可能覆盖 `point`。这类写法是有序容器的典型优势。注意 `std::prev(right)` 之前必须判断 `right != begin()`，否则会越界。源码阅读中看到树迭代器时，要记住它不是连续内存指针，自增自减会沿树结构寻找后继前驱，复杂度通常是均摊常数或对数级细节由实现保证，但缓存局部性不如数组。

`unordered_map` 的平均常数复杂度不是无条件的。它依赖哈希函数把键分散到桶里，也依赖

负载因子保持在合理范围。若大量键落入同一桶，查找会退化。C++ 标准没有要求具体冲突处理方式必须是链表还是开放寻址，常见实现多使用桶加节点。节点式实现的好处是 rehash 时可以重挂节点，坏处是每个元素单独分配，局部性不如连续表。刷题时使用 `reserve`，不只是为了速度，也是为了减少 rehash 导致的迭代器失效风险。

Listing 14.24 LeetCode 560 和为 K 的子数组：查询和更新顺序不能反

```
1 int subarray_sum(const std::vector<int>& nums, int target)
2 {
3     std::unordered_map<int, int> count;
4     count.reserve(nums.size() * 2);
5     count[0] = 1;
6
7     int prefix = 0;
8     int answer = 0;
9     for (const int x : nums) {
10         prefix += x;
11         const auto found = count.find(prefix - target);
12         if (found != count.end()) {
13             answer += found->second;
14         }
15         ++count[prefix];
16     }
17     return answer;
18 }
```

这里必须先查询历史前缀，再加入当前前缀。若先更新，`target == 0` 时会把空子数组错误计入。这个例子说明容器 API 和算法不变量是绑在一起的：`count` 的语义是“当前下标之前出现过的前缀和次数”，不是所有前缀和次数。`find` 用于查询，`operator[]` 用于确认要插入或累加的位置；二者角色不同。

源码阅读可以按固定路线进行。第一步读标准语义：这个容器承诺什么复杂度，哪些操作会让迭代器失效。第二步看实现的核心成员：`vector` 的三个指针，`string` 的大小容量和小字符串缓冲，`deque` 的段表，`unordered_map` 的桶数组和节点，`map` 的树根和节点颜色。第三步跟一条热路径：`push_back` 如何判断容量，`rehash` 如何搬迁，`erase` 如何维护链接，`lower_bound` 如何沿树下降。第四步回到题目，解释为什么某个 API 成本可接受或不可接受。

深入理解

阅读 `libstdc++` 或 `libc++` 源码时，不要被模板细节吓住。可以先从头文件里的 `public` 函数名定位到内部辅助函数，再沿着分配、构造、移动、销毁四类操作看。比如 `vector::push_back` 最核心的问题只有两个：容量够不够，元素如何构造；容量不够时，才进入重新分配和移动旧元素的路径。再比如 `unordered_map::operator[]` 的关键语义是“没有 key 就插入默认值”，源码里会表现为查找失败后构造节点。把这些行为和题目中的不变量联系起来，源码阅读才会变成能力，而不是机械翻文件。

实验建议：第一，自己写一个小程序，创建 `vector<int>`，连续 `push_back` 并打印 `size`、`capacity` 和 `data()` 地址，观察扩容时机和地址变化。第二，写前缀和计数题，把“先查询后更新”故意改反，用 `target = 0` 的用例验证错误。第三，比较 `substr` 和下标比较在长字符串 DP 中的运行时间。第四，阅读本机标准库中 `vector`、`deque`、`unordered_map` 的头文件，只记录核心数据成员和迭代器失效规则，不要求理解每个模板分支。

DP 和容器源码阅读复盘

DP 先问状态是否稳定，转移是否只依赖已计算状态，遍历顺序是否正确；容器先问数据是否连续、插入删除位置在哪里、查询是否需要顺序、迭代器何时失效。源码阅读不追求背实现细节，而是用实现模型解释复杂度、失效规则和 API 选择。

代表性 C++20 代码骨架

下面这些代码骨架不是为了替代题解，而是给读者提供可复用的实现基准。每段代码都尽量把状态命名清楚，把边界放在显式位置，把容器成本控制题目需要的范围内。真正刷题时不要机械复制，而要先说清楚题目的不变量，再决定是否使用这些骨架。

Listing 14.25 并查集：迭代 find、路径压缩和按大小合并

```

1 class DisjointSetUnion {
2 public:
3     explicit DisjointSetUnion(int n)
4         : parent_(n), size_(n, 1)
5     {
6         std::iota(this->parent_.begin(), this->parent_.end(), 0);
7     }
8
9     int find(int x)
10    {
11        int root = x;
12        while (this->parent_[root] != root) {
13            root = this->parent_[root];
14        }
15        while (this->parent_[x] != x) {
16            const int next = this->parent_[x];
17            this->parent_[x] = root;
18            x = next;
19        }
20        return root;
21    }
22
23    bool unite(int lhs, int rhs)
24    {
25        int root_lhs = this->find(lhs);
26        int root_rhs = this->find(rhs);
27        if (root_lhs == root_rhs) {
28            return false;
29        }
30        if (this->size_[root_lhs] < this->size_[root_rhs]) {
31            std::swap(root_lhs, root_rhs);
32        }
33        this->parent_[root_rhs] = root_lhs;
34        this->size_[root_lhs] += this->size_[root_rhs];
35        return true;
36    }
37
38 private:
39     std::vector<int> parent_;
40     std::vector<int> size_;
41 };

```

Listing 14.26 Dijkstra: 跳过 priority queue 中的过期状态

```

1 std::vector<long long> shortest_paths(
2     int n,
3     const std::vector<std::vector<std::pair<int, int>>>& graph,
4     int source)
5 {
6     constexpr long long INF = 4'000'000'000'000'000'000LL;
7     using State = std::pair<long long, int>;
8
9     std::vector<long long> distance(n, INF);
10    std::priority_queue<State, std::vector<State>, std::greater<State>> heap;
11    distance[source] = 0;
12    heap.emplace(0, source);
13
14    while (!heap.empty()) {
15        const auto [current_distance, node] = heap.top();
16        heap.pop();
17        if (current_distance != distance[node]) {
18            continue;
19        }
20
21        for (const auto& [next, weight] : graph[node]) {
22            const long long candidate = current_distance + weight;
23            if (candidate >= distance[next]) {
24                continue;
25            }
26            distance[next] = candidate;
27            heap.emplace(candidate, next);
28        }
29    }
30
31    return distance;
32 }

```

Listing 14.27 滑动窗口最大值: 单调队列保存下标

```

1 std::vector<int> max_sliding_window(const std::vector<int>& nums, int k)
2 {
3     std::deque<int> candidates;
4     std::vector<int> answer;
5     answer.reserve(nums.size());
6
7     for (int right = 0; right < static_cast<int>(nums.size()); ++right) {
8         while (!candidates.empty() && nums[candidates.back()] <= nums[right]) {
9             candidates.pop_back();
10        }
11        candidates.push_back(right);
12
13        const int expired = right - k;
14        if (!candidates.empty() && candidates.front() <= expired) {

```

```

15         candidates.pop_front();
16     }
17     if (right + 1 >= k) {
18         answer.push_back(nums[candidates.front()]);
19     }
20 }
21
22 return answer;
23 }

```

Listing 14.28 前缀和计数：历史状态先查询后更新

```

1 int subarray_sum_equals_k(const std::vector<int>& nums, int target)
2 {
3     std::unordered_map<int, int> prefix_count;
4     prefix_count.reserve(nums.size() * 2 + 1);
5     prefix_count[0] = 1;
6
7     int prefix = 0;
8     int answer = 0;
9     for (const int x : nums) {
10         prefix += x;
11         const auto found = prefix_count.find(prefix - target);
12         if (found != prefix_count.end()) {
13             answer += found->second;
14         }
15         ++prefix_count[prefix];
16     }
17     return answer;
18 }

```

Listing 14.29 迭代二叉树中序遍历：验证 BST 的基础骨架

```

1 bool is_valid_bst(TreeNode* root)
2 {
3     std::vector<TreeNode*> stack;
4     TreeNode* current = root;
5     std::optional<long long> previous;
6
7     while (current != nullptr || !stack.empty()) {
8         while (current != nullptr) {
9             stack.push_back(current);
10            current = current->left;
11        }
12
13        current = stack.back();
14        stack.pop_back();
15        if (previous.has_value() && current->value <= previous.value()) {
16            return false;
17        }
18        previous = current->value;
19        current = current->right;
20    }
21    return true;
22 }

```

```

20     }
21
22     return true;
23 }

```

C++ 面试代码质量和容器工程细节

算法面试中的 C++ 代码不只是能 AC，还要让状态、所有权和复杂度可读。函数参数只读大对象应使用 const 引用，返回结果时依赖移动优化即可；循环中遍历复杂对象使用 const auto&，不要无意复制字符串、向量或节点记录。局部常量要有名字，尤其是方向数组、无穷大、字符状态、取模常数和哨兵值。

整数类型选择要主动。数组下标和 size 返回 size_t，但很多题需要与 -1 哨兵或差值比较，直接混用无符号可能产生隐蔽 bug。常见做法是在函数开始把 size 转成 int，并保证输入规模在 int 范围内；涉及总和、乘积、面积、距离时使用 long long。二分平方、堆距离、图最短路都要优先防溢出。

容器 API 语义要讲清。unordered_map 的 operator[] 在 key 不存在时会插入默认值，适合计数累加，不适合只检查存在。find 返回迭代器，适合查询后读取值。contains 只判断存在，如果随后还要取值会导致二次查找。vector 的 reserve 只改容量，resize 才改大小。priority_queue 的 top 只保证极值，不能遍历出有序序列。

迭代器失效是工程题常见追问。vector 扩容后所有指针和迭代器失效；erase 中间元素会让后续位置移动；unordered_map rehash 会让迭代器失效；list 的 splice 能移动节点且保持其他迭代器稳定。LRU 缓存为什么用 list 加哈希表，核心就是哈希定位和链表节点稳定的组合。

代码组织要让不变量有位置。二分答案把检查函数单独写出来，图最短路把松弛逻辑集中，回溯把 push、搜索、pop 成对出现，DP 把初始化和主转移分开。这样做能让读者看到状态生命周期。把所有逻辑塞进一个巨大循环，短期可能省行数，长期会让边界和证明都变模糊。

面试时可以主动说明替代方案。第 K 大可以排序、堆、快速选择；岛屿数量可以 DFS、BFS、并查集；最小体力路径可以 Dijkstra、二分加 BFS、并查集排序边；单词接龙可以普通 BFS、通配桶、双向 BFS。比较方案不是炫耀，而是说明你知道当前选择依赖什么约束。

Linux 源码阅读放到后面的独立专题中集中学习。面试代码部分只需要先记住一个工程判断：容器选择不只看复杂度，还要看对象生命周期、内存分配、并发保护和数据是否需要被多个索引同时组织。

实验建议：写一个前缀和计数题，分别用 contains 加 operator[]、find、直接 operator[] 三种方式实现，观察语义差异。写一个 vector 扩容实验，保存元素地址后继续 push_back，验证地址失效。写一个 LRU，先用 vector 保存顺序，再用 list 加 unordered_map，比较移动到头部的成本。

本节复盘

阅读本节后，请回到相邻章节的例题，把暴力瓶颈、优化依据、状态不变量、C++ 容器成本和边界测试重新写一遍。能把同一思想迁移到变体题，才算真正掌握。

进阶综合题卡

4 寻找两个正序数组的中位数。暴力可以合并两个数组再取中位数，复杂度 $O(m+n)$ 。优化做法是二分较短数组的切分位置，使左半部分元素数量等于右半部分，且左半最大值不大于右半最小值。难点在边界哨兵和奇偶长度。这个题训练的是二分在答案结构上的应用，不是普通下标查找。

10 正则表达式匹配。暴力递归尝试星号匹配零次或多次，会重复处理同一对下标。DP 状态 $dp[i][j]$ 表示字符串前 i 个字符能否匹配模式前 j 个字符。星号转移最容易错：它修饰前一个字符，可以表示出现零次，也可以在当前字符匹配时消耗一个字符。边界初始化要处理 a 星号 b 星号这种能匹配空串的模式。

23 合并 K 个升序链表。暴力每轮扫描 K 个链表头，总代价 $O(NK)$ 。小顶堆保存每条链表当前头节点，每次弹出最小节点并压入它的后继，复杂度 $O(N \log K)$ 。分治两两合并也是 $O(N \log K)$ ，常数和空间不同。面试中应说明堆里最多 K 个节点，且复用节点时要处理尾指针。

25 K 个一组翻转链表。关键不是反转，而是找到每组边界。每轮从上一组尾部向后找第 K 个节点，不足则停止；保存下一组头，反转当前组，再把上一组尾接到新头、旧头接到下一组。必须先

保存 next，否则反转中会丢失后续链。虚拟头节点可以统一处理第一组。

32 最长有效括号。栈保存尚未匹配的左括号下标和最后一个非法右括号位置。遇到右括号匹配成功后，当前有效长度由当前下标减去栈顶或非法边界得到。DP 也可做，状态表示以当前位置结尾的最长有效长度。这个题训练的是括号结构和连续区间长度的结合。

41 缺失的第一个正数。排序是 $O(n \log n)$ ，哈希是 $O(n)$ 空间。原地做法利用数组本身作为哈希表，把值 x 放到下标 $x-1$ 的位置。扫描时不断交换到正确位置，最后第一个 $nums[i]$ 不等于 $i+1$ 的位置就是答案。边界是只处理一到 n 的正数，重复值不能无限交换。

44 通配符匹配。问号匹配单个字符，星号匹配任意长度。DP 状态与正则匹配类似，但星号语义不同：它可以匹配空串，也可以消耗当前字符后继续保持星号。贪心双指针也能做，记录最近星号位置和当时字符串位置，失配时让星号多吞一个字符。两种解法都要讲清星号回退机制。

51 N 皇后。回溯逐行放皇后，列、主对角线、副对角线三个集合判断冲突。暴力在棋盘每格尝试会产生大量无效状态；逐行保证每行一个皇后，把选择空间降为每行选列。对角线可用 $row-col$ 和 $row+col$ 编码。剪枝正确性来自皇后攻击规则完全由这三个集合覆盖。

72 编辑距离。这个题必须能从操作反推转移：删除 word1 末尾，对应 $dp[i-1][j]$ 加一；插入 word2 末尾，对应 $dp[i][j-1]$ 加一；替换末尾，对应 $dp[i-1][j-1]$ 加一。字符相同则不需要操作。空串边界是连续插入或删除。它是二维 DP 状态定义最经典的训练题。

85 最大矩形。把每一行当作柱状图底部，连续一的高度逐行累积，然后对每行运行柱状图最大矩形单调栈算法。暴力枚举矩形会很慢，优化来自把二维问题转成多次一维单调栈。边界是字符零会把高度清零，哨兵零帮助结算所有柱子。

97 交错字符串。状态 $dp[i][j]$ 表示 $s1$ 前 i 个字符和 $s2$ 前 j 个字符能否交错组成 $s3$ 前 $i+j$ 个字符。转移来自最后一个字符取自 $s1$ 或 $s2$ 。暴力递归会在相同 i, j 上重复。边界先判断长度和，再初始化第一行第一列。这个题训练两个来源序列的前缀状态。

115 不同的子序列。状态 $dp[i][j]$ 表示 s 前 i 个字符中有多少个子序列等于 t 前 j 个字符。若当前字符相等，可以用它匹配 t 的末尾，也可以不用它；若不等，只能不用它。计数可能很大，工程上要考虑 long long。它和 LCS 都是双字符串 DP，但一个求长度，一个求数量。

126 单词接龙 II。相比单词接龙 I，不仅要最短长度，还要输出所有最短路径。做法是 BFS 建立最短层级和前驱关系，不在 BFS 中立即深搜所有路径；BFS 完成后从终点按前驱回溯。关键是同一层的多个前驱都要保留，不能因为某个单词已经被本层访问就丢掉另一条同长路径。

135 分发糖果。每个孩子至少一颗，评分更高者比相邻低评分者多。左到右保证比左邻高的约束，右到左保证比右邻高的约束，最终取两次约束最大值。暴力反复调整会来回传播。这个题体现局部相邻约束可以分两次单向扫描合并。

140 单词拆分 II。先用 DP 判断后缀是否可拆，避免在无解分支上回溯；再 DFS 枚举句子。若直接暴力切分，会在大量不可达后缀上浪费时间。输出规模可能指数级，所以复杂度必须包含输出。实现时要注意路径拼接成本，可以保存单词列表最后 join。

188 买卖股票 IV。状态包含交易次数和持有状态。若 k 大于等于 $n/2$ ，退化为无限次交易；否则维护 $buy[t]$ 和 $sell[t]$ 。更新顺序要避免同一天状态污染。这个题说明状态维度来自约束，交易次数不是后处理能补上的信息。

212 单词搜索 II。对每个单词单独 DFS 会重复遍历棋盘。Trie 把所有单词前缀合并，网格 DFS 时沿 Trie 前进，不存在前缀立即剪枝。找到单词后可标记避免重复加入答案，也可删除叶子做进一步剪枝。边界是访问标记回退和 Trie 节点生命周期。

224 基本计算器。表达式有括号和加减号。可以用栈保存进入括号前的结果和符号，也可以写递归下降解析。扫描时数字要累积多位，遇到符号结算前一个数字。这个题不是数学题，而是状态机和栈处理嵌套上下文。

227 基本计算器 II。乘除优先级高于加减。常见做法是扫描数字和运算符，遇到乘除立即和栈顶结合，遇到加减把带符号数字入栈，最后求和。除法要按题目要求向零截断。它训练的是延迟低优先级、立即处理高优先级。

295 数据流中位数。双堆的顺序不变量和大小不变量缺一不可。大顶堆所有元素不大于小顶堆所有元素，两个堆大小差不超过一。插入时先按堆顶分配，再重平衡。若只维护大小，不维护左右顺序，中位数会错。若要支持删除，需要懒删除机制。

312 戳气球。若按第一个戳的气球思考，左右邻居会随操作变化，状态不稳定。改成最后戳某个气球，左右边界在最后一刻固定，区间就能独立。 $dp[left][right]$ 表示开区间内气球全部戳完的最大

收益，枚举最后戳的 `mid`。这个题是区间 DP 状态设计的代表。

329 矩阵中的最长递增路径。从每个格子 DFS 找递增路径会重复计算后续格子。记忆化搜索把每个格子的最长路径保存下来；也可以把格子按值排序做 DAG DP。由于路径必须严格递增，状态图无环。若允许不下降，就可能出现环，需要重新处理。

394 字符串解码。栈保存上一层字符串和重复次数。遇到左括号入栈，遇到右括号弹出并展开。多位数字、嵌套括号和当前字符串重置是主要边界。这个题适合训练解析类问题的上下文保存，而不是简单字符串拼接。

460 LFU 缓存。LFU 不只要按频次淘汰，还要在相同频次下按最久未使用淘汰。常见结构是 key 到节点信息的哈希表、频次到双向链表的哈希表、当前最小频次。访问时节点频次加一，从旧频次链表移到新频次链表。它比 LRU 多了频次维度，能训练多个索引结构协同维护。

560 和为 K 的子数组。它在很多变体中出现：若问数量，哈希表保存前缀和频次；若问最长，保存最早下标；若问最短且有负数和至少 K，用前缀和加单调队列；若元素全正，可以滑动窗口。复盘时要根据目标选择历史状态，不能只记一个模板。

632 最小覆盖子串。每个列表当前取一个元素，区间由当前最小和最大决定。小顶堆保存各列表当前元素，同时维护当前最大值；弹出最小元素并推进它所在列表。若某个列表耗尽，就无法再覆盖所有列表。这个题是多路归并和滑动覆盖思想的结合。

678 有效的括号字符串。星号可以当左括号、右括号或空。贪心维护可能未匹配左括号数量的最小值和最大值区间。遇到左括号区间加一，右括号区间减一，星号让下界减一、上界加一，并把下界截到零。若上界为负则不可能。这个题展示状态集合可被区间压缩。

721 账户合并。邮箱是连接账户的证据。可以把邮箱映射到编号并查集合并，也可以把账户编号合并。最后按根收集邮箱并排序。姓名通常只用于输出，不能作为合并依据。这个题训练字符串哈希、并查集和输出排序的组合。

815 公交线路。站点很多，路线也很多。若把站点作为 BFS 节点，每次找经过该站的路线；更高效的建模是路线作为节点，两个路线若共享站点则可换乘。实际实现常用站点到路线列表的哈希表，从起点站可乘路线开始 BFS。访问路线比访问站点更能控制重复。

862 和至少为 K 的最短子数组。数组含负数，普通滑动窗口失效。前缀和数组上维护单调递增队列：队首若和当前前缀差至少 K，就更新答案并弹出；队尾若前缀和不少于当前前缀，就被当前下标支配。这个题是理解单调队列支配关系的关键。

895 最大频率栈。需要弹出频率最高且最近入栈的元素。用 value 到 frequency 的哈希表记录频次，用 frequency 到栈的哈希表保存达到该频次的元素顺序，再维护当前最大频次。push 时频次加一并入对应栈，pop 时从最大频次栈弹出。它是哈希表和栈按维度组合的例子。

981 基于时间的键值存储。每个 key 对应按时间递增的值列表，查询某个时间戳不超过目标的新值。哈希表定位 key，列表中二分 upper_bound 找第一个大于目标的位置再退一。它训练按 key 分组后在组内二分，不要把所有记录放进一个全局结构里。

1091 二进制矩阵中的最短路径。八方向无权网格最短路，BFS 即可。起点或终点为阻塞直接返回 -1。访问标记在入队时设置，距离可以放在队列状态里，也可以直接把矩阵改成距离。它和岛屿题的区别是要求最短路径，所以 DFS 不合适。

1235 规划兼职工作。按结束时间排序，每个工作选或不选。选择当前工作时，需要找到上一个结束时间不超过当前开始时间的工作，用二分定位。dp[i] 表示考虑前 i 个工作最大收益。它是排序、二分和 DP 的组合题，常见错误是只按收益贪心。

1248 统计优美子数组。恰好 K 个奇数可以转成前缀奇数计数差为 K。哈希表保存某个奇数计数出现次数；扫描到当前计数 c 时，历史 c-K 的次数就是新增答案。也可以用至多 K 减至多 K-1 的滑动窗口。两个解法对应前缀计数和窗口计数两种思维。

1438 绝对差不超过限制的最长连续子数组。窗口合法性取决于最大值和最小值差。用两个单调队列分别维护窗口最大和最小，右端扩展后若差超过限制就移动左端并清理过期下标。这个题说明窗口状态不一定是和或频次，也可以是动态极值。

1462 课程表 IV。如果查询很多，不能每次为一对课程单独搜索。可以用 Floyd 传递闭包、每个点 BFS/DFS 可达集合，或者拓扑 DP 合并先修集合。选择取决于课程数和查询数。它训练预处理和多查询之间的权衡。

1514 概率最大的路径。路径权重是概率乘积，目标是最大乘积。可以把 Dijkstra 的距离改成最大概率，用大顶堆每次扩展当前概率最高节点；概率乘以边概率不会变大，满足单调性。也可以取

负对数变成最短路。这个题说明最短路框架可以迁移到其他单调代价。

1675 数组的最小偏移量。每个奇数可以先乘二变偶数，之后所有偶数都可以不断除二。把所有数变成可减少的最大形态，用大顶堆维护当前最大值，同时维护当前最小值；每次尝试降低最大偶数并更新答案。若最大值变奇数就无法继续。这个题是堆和状态空间规范化的结合。

本节复盘

阅读本节后，请回到相邻章节的例题，把暴力瓶颈、优化依据、状态不变量、C++ 容器成本和边界测试重新写一遍。能把同一思想迁移到变体题，才算真正掌握。

二刷深度复盘卡

这一组卡片用于第二遍和第三遍复习。第一遍题卡帮助你建立解法，二刷卡片要求你把证明、追问、调试和工程成本讲清楚。每道题都要准备一个反例、一个边界用例和一个能说明优化价值的规模用例。

LeetCode 1 两数之和。模型：当前元素只需要寻找唯一补数，历史值到下标的映射就是最小状态。推导重点：先查再插入，避免同一个下标被用两次；若数组有序才考虑双指针。边界：重复数字、目标为偶数且补数等于当前值、没有答案时返回空结果。变体：若改成返回所有不重复数对，要先排序或用频次表处理重复。

LeetCode 2 两数相加。模型：链表按低位到高位保存数字，本质是竖式加法而不是整数转换。推导重点：维护两个游标和进位，任一链未结束或进位非零都继续生成节点。边界：两链长度不同、最后进位、输入为零、不能用内置整数承载超长数字。变体：若链表高位在前，可以用栈反向处理或先反转链表。

LeetCode 3 无重复字符的最长子串。模型：窗口保存当前无重复子串，右端每次加入一个字符。推导重点：用上次出现位置直接跳过无效左端，左边界只能向右不能回退。边界：空串、全相同字符、重复字符出现在窗口左侧之外、扩展字符集。变体：若要求恰好包含 k 种字符，窗口条件从无重复变成种类计数。

LeetCode 5 最长回文子串。模型：回文围绕中心对称，中心可以是字符也可以是两个字符之间。推导重点：中心扩展避免枚举子串后再逐个检查；每次扩展只比较新增左右边界。边界：偶数长度回文、单字符、全相同字符、多个同长答案。变体：若要求回文子序列，应转成区间 DP，不能用中心扩展。

LeetCode 11 盛最多水的容器。模型：左右指针代表当前选取的两条边，面积由短板和宽度共同决定。推导重点：每次移动短板，因为移动长板只会缩小宽度且不会提高短板。边界：两端高度相等、数组长度为二、面积乘法可能溢出。变体：接雨水计算每个位置贡献，不能把本题双指针证明直接搬过去。

LeetCode 15 三数之和。模型：排序后固定第一个数，剩余两数转成有序双指针。推导重点：和太小移动左指针，和太大移动右指针；固定值和左右值都要跳过重复。边界：全零、重复元素很多、没有答案、结果顺序不要求但组合不能重复。变体：四数之和再外层固定一个数，并用 long long 防止目标和溢出。

LeetCode 16 最接近的三数之和。模型：排序后固定一数，双指针寻找最接近目标的两数和。推导重点：每次根据当前和与目标的大小移动指针，同时更新最小绝对差。边界：差值相等、负数、目标很大、三数和溢出。变体：若要求返回所有最接近组合，需要记录同差值并去重。

LeetCode 18 四数之和。模型：两层固定加一层双指针，关键是剪枝和去重。推导重点：排序提供单调性，外层可用最小可能和和最大可能和提前跳过。边界：重复元素、目标超出 int、答案很多导致输出空间主导复杂度。变体： k 数之和可以递归降维，但工程实现要控制重复和边界。

LeetCode 26 删除有序数组重复项。模型：有序数组让重复元素相邻，慢指针表示下一个唯一值写入位置。推导重点：快指针扫描，只有发现新值才写入慢指针并推进。边界：空数组、全重复、无重复、返回长度不等于物理容量。变体：若允许每个元素最多保留两次，只需改变写入条件为和前两个比较。

LeetCode 27 移除元素。模型：原地过滤，慢指针左侧始终是不等于目标值的稳定前缀。推导重点：保持顺序用快慢指针；不保持顺序可用左右交换减少写入。边界：所有元素都被移除、没有元素被移除、目标值在尾部密集。变体：工程里要先确认是否要求稳定顺序，不同要求对应不同写法。

LeetCode 31 下一个排列。模型：字典序结构由最长非递增后缀决定，枢轴是后缀前一个位置。

推导重点：从右找第一个上升点，再从右找刚好更大的元素交换，最后反转后缀。边界：完全降序、重复数字、只有一个元素、交换后后缀必须最小化。变体：若要求上一个排列，比较方向和后缀处理对称变化。

LeetCode 33 搜索旋转排序数组。模型：旋转数组每次二分至少有一半保持有序。推导重点：判断哪一半有序，再看目标是否落在有序半边范围内。边界：未旋转、长度一、目标不存在、边界等号、存在重复元素时退化。变体：若重复元素很多，需要在无法判断有序半边时收缩边界。

LeetCode 34 排序数组中查找首尾位置。模型：首尾位置可以拆成两个边界：第一个大于等于和第一个大于。推导重点：不要找到一个目标后向两边线性扩展，否则重复元素很多时退化。边界：目标不存在、目标在开头或结尾、数组为空、全是目标。变体：这个模型能迁移到计数某值出现次数和插入区间边界。

LeetCode 35 搜索插入位置。模型：题目等价于寻找第一个大于等于目标的位置。推导重点：统一用 lower bound 语义，不在相等时提前返回也能保持边界一致。边界：目标小于所有元素、目标大于所有元素、重复值、空数组。变体：手写二分时用左闭右开可直接返回 left。

LeetCode 42 接雨水。模型：每个位置的水由左侧最高和右侧最高的较小者决定。推导重点：前后缀、双指针、单调栈都是不同状态维护方式；要能说明各自结算对象。边界：单调递增、单调递减、平台高度、多个凹槽、数组长度不足三。变体：若柱子变成二维网格，需要最小堆从边界向内扩展。

LeetCode 49 字母异位词分组。模型：异位词共享同一个规范表示，可以是排序字符串或字符计数。推导重点：哈希表按规范键分桶，避免两两比较字符串。边界：空字符串、重复单词、字符集不止小写字母、计数键必须无歧义。变体：若字符串很长且字符集固定，计数键通常比排序键更稳定。

LeetCode 53 最大子数组和。模型：状态是必须以当前位置结尾的最大连续和。推导重点：若前一个后缀为负，接上它只会拖累当前元素，因此可以重新开始。边界：全负数组、单元素、和溢出、答案不能初始化为零。变体：若要求返回区间下标，同步记录重新开始位置和最优左右端。

LeetCode 54 螺旋矩阵。模型：四个边界收缩模拟一圈一圈访问。推导重点：每走完一条边就收缩对应边界，并在走反方向前检查是否仍合法。边界：单行、单列、空矩阵、最后一圈只剩一个元素。变体：若改成生成矩阵，边界逻辑相同，只是读变成写。

LeetCode 55 跳跃游戏。模型：扫描过程中只关心当前能到达的最远位置。推导重点：如果下标超过最远可达，任何之前路径都不能到达它；否则用它扩展边界。边界：第一个元素为零、数组长度一、恰好到达最后、远超最后。变体：求最少跳数时把可达区间看成 BFS 层。

LeetCode 56 合并区间。模型：排序后可能与当前区间相交的只会是结果尾部。推导重点：按起点排序，扫描时维护结果最后区间的右端点。边界：端点相接、完全覆盖、无重叠、空列表、输入未排序。变体：若区间是半开区间，端点相等不一定合并。

LeetCode 57 插入区间。模型：新区间只会影响与它相交的一段连续区间。推导重点：先追加左侧完全不相交区间，再合并重叠段，最后追加右侧。边界：新区间在最前、最后、覆盖全部、刚好相邻、原列表为空。变体：若输入不是有序且不重叠，必须先排序或换模型。

LeetCode 62 不同路径。模型：网格状态由上方和左方两种最后一步转移而来。推导重点：二维表可压缩为一维，因为当前行只依赖上一行同列和当前行左侧。边界：一行、一列、较大网格结果溢出、障碍物版本边界。变体：带障碍物时遇到障碍状态清零，第一行第一列要按阻断处理。

LeetCode 63 不同路径 II。模型：障碍物让某些格子的路径数变为零。推导重点：滚动数组中遇到障碍直接把当前位置清零，否则累加左侧。边界：起点或终点是障碍、整行被截断、单行障碍。变体：若允许向上或向左走，图会有环，普通填表不再成立。

LeetCode 64 最小路径和。模型：状态是到达当前格子的最小代价，最后一步来自上方或左方。推导重点：先初始化第一行和第一列，再处理内部，主转移保持简单。边界：单行、单列、权值为零、路径和溢出。变体：若允许四方向移动，变成最短路而不是普通网格 DP。

LeetCode 70 爬楼梯。模型：最后一步来自前一阶或前两阶，是最小的重叠子问题模型。推导重点：滚动变量保存最近两个状态即可，不需要完整数组。边界：n 为一、n 为二、题目是否允许零阶、结果范围。变体：若每次可走一到 k 阶，转移变成固定宽度窗口求和。

LeetCode 72 编辑距离。模型：二维状态表示两个前缀之间的最少编辑操作。推导重点：末尾字符相同继承左上；不同则枚举插入、删除、替换。边界：空串、完全相同、完全不同、操作代价不同。变体：若只允许插入删除，问题退化到和最长公共子序列相关。

LeetCode 75 颜色分类。模型：三指针维护零区、一号区、未知区和二区。推导重点：遇到二时只移动右边界，不移动扫描指针，因为换回来的元素未知。边界：全零、全二、已经有序、逆序、交换后漏检查。变体：这是荷兰国旗问题，能推广到三类分区的原地维护。

LeetCode 76 最小覆盖子串。模型：窗口内字符计数必须覆盖目标计数，满足后尽量收缩左端。推导重点：用 `formed` 记录满足需求的字符种类，避免每次全量比较。边界：目标有重复字符、源串缺字符、最小窗口在开头或结尾、大小写。变体：若字符集固定，数组计数更快；若是 Unicode，哈希表更通用。

LeetCode 78 子集。模型：每个元素都有选或不选两种选择，答案是所有路径节点。推导重点：回溯按起点向后枚举，避免不同顺序生成同一集合。边界：空数组、单元素、输出数量为二的 n 次方、顺序不要求。变体：也可用位掩码枚举所有子集，适合 n 较小的场景。

LeetCode 79 单词搜索。模型：网格路径不能重复使用同一格，状态包含位置、匹配下标和访问标记。推导重点：剪枝包括首字符不匹配、剩余长度不足、字符频次不足和越界访问。边界：单字符单词、路径需要回退、重复字母、单元格不能复用。变体：若要查很多单词，应使用 Trie 合并搜索前缀。

LeetCode 84 柱状图中最大的矩形。模型：每根柱子作为最矮高度时，需要知道左右第一个更矮位置。推导重点：单调递增栈在遇到更矮柱时结算被弹出柱子的最大宽度。边界：递增、递减、相等高度、哨兵零、宽度计算。变体：最大矩形可以作为二维矩阵最大矩形的行压缩子问题。

LeetCode 88 合并两个有序数组。模型：第一个数组尾部有空位，适合从后往前写入最终最大值。推导重点：逆向填充避免覆盖尚未读取的 `nums1` 元素。边界：第二个数组为空、第一个有效元素为空、重复值、尾部剩余。变体：若没有额外空位，只能新开数组或使用更复杂原地归并。

LeetCode 90 子集 II。模型：重复元素会让不同下标路径生成相同集合。推导重点：排序后在同一层跳过相同值，保留不同层使用重复值的可能。边界：全重复、空集、重复值相邻、去重条件写成同层而不是全局。变体：组合总和 II 使用同样的同层去重思想。

LeetCode 94 二叉树中序遍历。模型：中序访问顺序是左子树、根、右子树，BST 中会得到有序序列。推导重点：用显式栈模拟一路向左，再回退访问根并转向右子树。边界：空树、单节点、链式左树、链式右树。变体：Morris 遍历可做到常数额外空间，但会临时改树结构。

LeetCode 98 验证二叉搜索树。模型：BST 约束是全局上下界，不是只比较父子节点。推导重点：中序严格递增或显式传递上下界都可以；中序版本要保存前一个访问值。边界：重复值、`int` 最小最大值、局部合法但全局非法。变体：若允许重复值，需要明确重复放左还是放右。

LeetCode 102 二叉树层序遍历。模型：队列在每轮开始时保存当前层所有节点。推导重点：记录 `level size` 固定层边界，避免下一层节点混入当前层。边界：空树、只有根、最后一层不满、左右顺序。变体：锯齿形层序只是在每层输出顺序上增加方向控制。

LeetCode 103 二叉树锯齿形层序遍历。模型：BFS 层边界不变，变化的是当前层写入结果的方向。推导重点：可以层内反转，也可以预分配数组按方向写入目标下标。边界：单层、偶数层、奇数层、空树。变体：若要求从底向上输出，收集后反转层列表即可。

LeetCode 104 二叉树最大深度。模型：深度可以看成层数，也可以作为栈中携带的状态。推导重点：显式栈保存节点和深度，避免极端链式树递归栈过深。边界：空树返回零、根深度是一、链式树。变体：最小深度不能简单取左右最小，因为空子树不构成叶子路径。

LeetCode 105 前序中序构造二叉树。模型：前序给根，中序把左右子树切开。推导重点：用哈希表记录中序下标，避免每次线性寻找根位置。边界：节点值唯一、空子树、左右子树长度计算、输入不合法。变体：后序中序构造类似，只是根来自后序末尾。

LeetCode 106 中序后序构造二叉树。模型：后序末尾是根，中序位置确定左右子树规模。推导重点：构造时要小心后序左右区间边界，避免切分错位。边界：空树、单节点、全左链、全右链。变体：若给前序和后序，二叉树通常不能唯一确定，除非额外限制。

LeetCode 108 有序数组转二叉搜索树。模型：有序数组中点作为根能让左右规模接近。推导重点：每次选择区间中点，左右子区间构造左右子树。边界：空区间、偶数长度选左中还是右中、高度平衡定义。变体：工程上递归可读，若严格避免递归可用队列保存区间和父指针。

LeetCode 110 平衡二叉树。模型：每个节点需要知道子树高度以及是否已经失衡。推导重点：后序汇总时一旦子树失衡就向上返回失败标记，避免重复算高度。边界：空树、单节点、左右高度差正好一、链式树。变体：可以把返回值设计成 `optional` 高度，空表示不平衡。

LeetCode 111 二叉树最小深度。模型：最小深度是到最近叶子的节点数，不是左右深度简单

取最小。推导重点：BFS 第一次遇到叶子就是最小深度，因为按层扩展。边界：只有左子树、只有右子树、根为空、根是叶子。变体：DFS 写法必须特殊处理空子树不能作为最小路径。

LeetCode 112 路径总和。模型：路径必须从根到叶子，状态是剩余目标值。推导重点：沿路径减去当前节点值，到叶子时检查剩余是否等于叶子值。边界：负数节点、目标为零、非叶子提前达到目标、空树。变体：若要求列出所有路径，需要保存路径数组并在回退时弹出。

LeetCode 113 路径总和 II。模型：相比判定题，状态还要保存当前路径。推导重点：到叶子且和满足时复制路径；回溯时恢复路径。边界：多个答案、负数、空树、路径数组复制时机。变体：若路径不要求从根到叶，模型会变成前缀和加哈希。

LeetCode 114 二叉树展开为链表。模型：目标是按前序顺序把树原地改成右指针链。推导重点：显式栈按右后左先入顺序处理，逐个接到前驱右侧。边界：空树、单节点、原有右子树不能丢、左指针置空。变体：也可用寻找左子树最右节点的原地方法。

LeetCode 121 买卖股票的最佳时机。模型：卖出日固定时，最佳买入日是此前最低价格。推导重点：扫描维护历史最低价和最大利润，一次交易无需复杂状态机。边界：单日价格、一直下跌、一直上涨、价格相同。变体：多次交易、冷冻期和手续费版本都要扩展成状态机。

LeetCode 122 买卖股票 II。模型：允许多次交易且不能同时持有，所有正收益差都可以收集。推导重点：把长上涨段拆成每天买卖收益相同，因此累加正差值最优。边界：下降段、平台价格、只有一天、手续费不存在。变体：有手续费时不能简单累加正差，需要状态机或调整贪心阈值。

LeetCode 123 买卖股票 III。模型：最多两次交易，状态需要包含交易次数和持有状态。推导重点：维护 buy1、sell1、buy2、sell2 四个状态，按天更新。边界：价格为空、只够一次交易、两次交易间不能重叠。变体：最多 k 次交易是它的泛化，k 很大时退化为无限次交易。

LeetCode 124 二叉树最大路径和。模型：路径可以从任意节点到任意节点，但不能分叉向父节点贡献两边。推导重点：每个节点向父亲贡献单边最大增益，全局答案可用左右两边加当前节点更新。边界：全负数、单节点、左右增益为负时取零、答案初始值。变体：若路径必须从根到叶，状态和答案位置完全不同。

LeetCode 127 单词接龙。模型：每个单词是节点，一次修改一个字符形成边。推导重点：通配模式桶加速邻居查询，BFS 第一次到达终点就是最短转换长度。边界：终点不在词表、起点和终点相同、桶重复扫描、单词长度一致。变体：双向 BFS 可以进一步降低搜索空间。

LeetCode 128 最长连续序列。模型：连续段只需要从没有前驱的数开始扩展。推导重点：哈希集合判断 x 减一是否存在，跳过非起点避免重复扫描。边界：重复元素、负数、空数组、整数边界。变体：若数据流动态插入，可以用并查集或区间合并维护长度。

LeetCode 130 被围绕的区域。模型：只有不连通到边界的 O 才会被翻转。推导重点：从边界 O 出发标记安全区域，再翻转剩余 O。边界：全边界、无 O、单行单列、内部岛屿。变体：这是反向思维：找不能被翻转的区域比直接判断每块区域更简单。

LeetCode 131 分割回文串。模型：每次选择一个从当前位置开始的回文前缀。推导重点：预处理回文 DP 表，让回溯合法性检查为常数。边界：空串、单字符、全相同字符、重复分割路径。变体：若只求最少切割次数，可改成 DP 最短切分。

LeetCode 133 克隆图。模型：每个原节点需要唯一对应一个新节点，边按原图连接。推导重点：哈希表保存原节点到克隆节点的映射，BFS 或 DFS 遍历图。边界：空图、自环、重边、环结构、不能按节点值唯一假设。变体：若图节点值不唯一，必须用地址作为键。

LeetCode 136 只出现一次的数字。模型：成对元素可以用异或抵消，剩下单个元素。推导重点：异或满足交换结合，扫描顺序不影响答案。边界：零、负数、唯一值在开头或结尾、所有其他值恰好两次。变体：若其他值出现三次，要改为按位计数取模。

LeetCode 139 单词拆分。模型：状态表示前缀能否由字典单词拼出。推导重点：枚举最后一个单词的起点，左侧前缀可达且子串在字典中则当前可达。边界：空串、重复单词、长单词、substr 复制成本。变体：若要输出所有句子，需要 DFS 加记忆化或前驱列表。

LeetCode 141 环形链表。模型：快慢指针在有环时必然相遇，无环时快指针先到空。推导重点：不用哈希表也能常数空间检测环。边界：空链、单节点自环、两节点环、尾部无环。变体：若要求环入口，用相遇后双指针同步走。

LeetCode 142 环形链表 II。模型：相遇点到入口的距离关系来自快慢指针速度差。推导重点：相遇后一个指针回头，两个指针每次走一步，再次相遇就是入口。边界：入口在头节点、长尾短环、无环、单节点环。变体：也可用哈希集合，但空间更高。

LeetCode 146 LRU 缓存。模型：哈希表负责按 key 定位，双向链表负责维护新旧顺序。推导重点：访问和更新都把节点移动到头部，容量满时删除尾部最旧节点。边界：容量为一、重复 put、get 不存在、更新已有值。变体：C++ 可用 `std::list` 加迭代器，也可手写双向链表理解所有权。

LeetCode 148 排序链表。模型：链表不能随机访问，适合归并排序而不是快速排序数组 partition。推导重点：自底向上迭代归并能避免递归栈，同时保持 $O(n \log n)$ 。边界：空链、单节点、重复值、奇数长度、切断子链。变体：若把值复制到数组排序，会改变节点身份语义。

LeetCode 150 逆波兰表达式求值。模型：后缀表达式把优先级和括号都编码进 token 顺序。推导重点：遇到数字入栈，遇到运算符弹出右操作数和左操作数计算。边界：负数 token、多位数、除法向零截断、减法顺序。变体：中缀表达式求值需要另一个运算符栈处理优先级。

LeetCode 152 乘积最大子数组。模型：负数会让最大乘积和最小乘积互换。推导重点：同时维护以当前位置结尾的最大和最小乘积。边界：零、负数个数奇偶、单元素、乘积溢出。变体：如果要求返回区间，同步记录状态来源。

LeetCode 153 寻找旋转排序数组中的最小值。模型：最小值是旋转断点，右端点可帮助判断中点在哪一段。推导重点：若 mid 大于 right，最小值在右侧；否则在左侧含 mid。边界：未旋转、长度一、旋转一位、无重复条件。变体：有重复时遇到相等只能收缩 right，最坏退化。

LeetCode 154 寻找旋转排序数组中的最小值 II。模型：重复元素会让 mid 和 right 相等时无法判断最小值方向。推导重点：相等时只能安全地丢掉一个 right，因为它不是唯一必要候选。边界：全相等、重复值跨断点、最小值出现多次。变体：这题说明二分依赖的信息不足时会退化，不是所有旋转题都严格对数。

LeetCode 155 最小栈。模型：普通栈只能看到栈顶，不能常数时间知道历史最小值。推导重点：辅助栈同步保存每个深度对应的最小值。边界：重复最小值、弹出最小值、空栈操作约定。变体：也可保存差值压缩空间，但可读性和溢出处理更复杂。

LeetCode 160 相交链表。模型：相交是节点地址相同，不是节点值相同。推导重点：两个指针走完各自链表后切换到另一条链，走过总长度相同后相遇。边界：不相交、头节点相交、尾节点相交、长度差很大。变体：哈希集合更直观但空间更高。

LeetCode 162 寻找峰值。模型：比较 mid 和 mid 加一的斜率能判断某侧一定存在峰值。推导重点：上升则右侧有峰，下降则左侧含 mid 有峰。边界：长度一、峰在边界、多个峰、相邻不相等假设。变体：二维峰值需要按行列最大值进行更复杂二分。

LeetCode 169 多数元素。模型：超过一半的元素可以和其他元素两两抵消后仍然剩下。推导重点：Boyer-Moore 投票维护候选和计数。边界：题目是否保证存在多数、计数归零、全相同。变体：若找超过三分之一的元素，需要维护两个候选。

LeetCode 189 轮转数组。模型：右移 k 位等价于把数组切成两段后换序。推导重点：三次反转能原地完成：整体、前 k、后 n 减 k。边界：k 为零、k 大于 n、空数组、单元素。变体：环状替换也可做，但要处理最大公约数个循环。

LeetCode 198 打家劫舍。模型：每间房只有偷和不偷两种结局，偷当前就不能偷前一间。推导重点：滚动保存前一状态和前两状态。边界：空数组、单间、全零、金额很大。变体：环形房屋要拆成不含首和不含尾两个线性问题。

LeetCode 199 二叉树右视图。模型：每层从右侧能看到的就是该层最后访问或最先访问的右侧节点。推导重点：BFS 固定层边界，取每层最后一个；或 DFS 按右优先首次到达深度。边界：空树、左链、右链、左右交错。变体：若求左视图，只改变层内取值方向。

LeetCode 200 岛屿数量。模型：陆地格子是节点，四方向相邻是边，岛屿是连通块。推导重点：扫描遇到未访问陆地就启动搜索并标记整块。边界：空网格、全水、全陆、斜对角不连通、是否允许修改输入。变体：也可用并查集合并相邻陆地，适合动态岛屿扩展。

LeetCode 202 快乐数。模型：反复变换数字会进入一或循环，状态空间最终有限。推导重点：用集合检测重复状态，或用快慢指针看成函数图。边界：输入为一、进入循环、位平方和函数、负数不在题意内。变体：快慢指针版本能训练把数值迭代看成链表。

LeetCode 206 反转链表。模型：每次把当前节点的 next 指向已经反转好的前缀头。推导重点：先保存后继，再改指针，再推进 previous 和 current。边界：空链、单节点、两个节点、不要丢失剩余链。变体：K 组反转是在这段局部反转外增加分组边界。

LeetCode 207 课程表。模型：课程是节点，先修关系是有向边，问题是判断是否有环。推导重点：入度拓扑排序不断学习当前无依赖课程。边界：边方向、孤立课程、环、自依赖、重复边。变

体：DFS 三色标记也能判环，但要明确访问状态。

LeetCode 208 实现 Trie。模型：前缀树把字符串公共前缀共享成路径。推导重点：插入和查询都逐字符沿边移动，不存在的边按需要创建。边界：空字符串约定、单词结束标记、前缀存在不等于单词存在。变体：若字符集很大，用哈希子节点；小写字母用数组更快。

LeetCode 209 长度最小的子数组。模型：正数数组让窗口和随右端增加、随左端减少。推导重点：右端扩展到满足目标后，不断左移缩短并更新答案。边界：不存在答案、单元素达到目标、全正是必要条件。变体：若含负数，需要前缀和加单调队列。

LeetCode 210 课程表 II。模型：在判环基础上还要输出一个可行拓扑序。推导重点：每弹出一个入度为零课程，就把它加入顺序并删除出边。边界：有环返回空数组、多个可行顺序、边方向。变体：若需要字典序最小拓扑序，把队列换成小顶堆。

LeetCode 211 添加与搜索单词。模型：点号通配符让查询可能分叉，但前缀树仍能剪掉不匹配前缀。推导重点：普通字符沿唯一边走，点号枚举当前节点所有孩子。边界：连续通配符、空结果、单词结束标记、字符集大小。变体：大量通配符会接近遍历整棵 Trie，需要规模约束。

LeetCode 215 数组中的第 K 个最大元素。模型：只关心第 K 大，不需要完整排序。推导重点：大小为 K 的小顶堆保存当前前 K 大边界；快速选择可做到平均线性。边界：K 为一、K 为 n、重复值、随机 pivot。变体：若数据流持续到来，堆方案比一次性快速选择更自然。

LeetCode 216 组合总和 III。模型：从一到九中选 k 个数和为 n，每个数最多用一次。推导重点：按递增起点枚举，利用剩余个数和剩余和剪枝。边界：n 太小、n 太大、k 为零、路径长度已经超过 k。变体：可预估剩余最小和最大和进一步剪枝。

LeetCode 217 存在重复元素。模型：判断是否出现过是哈希集合最直接的职责。推导重点：扫描时若插入前已经存在则立即返回真。边界：空数组、全唯一、重复在末尾、值域很小。变体：若允许修改输入，排序后看相邻可以降低额外空间。

LeetCode 221 最大正方形。模型：以当前位置为右下角的最大正方形由上、左、左上三个状态限制。推导重点：当前位置为一时取三者最小加一，否则为零。边界：全零、全一、单行单列、字符一和数字一不要混淆。变体：最大矩形则需要按行构造柱状图再用单调栈。

LeetCode 226 翻转二叉树。模型：每个节点交换左右孩子即可，遍历顺序不影响最终结构。推导重点：显式队列或栈逐个节点交换，避免递归深度。边界：空树、单节点、只有一侧子树、重复值。变体：这题简单但适合训练树节点指针修改。

LeetCode 230 二叉搜索树中第 K 小。模型：BST 中序遍历按升序访问节点。推导重点：显式栈中序，访问第 k 个节点时返回。边界：k 为一、k 为节点数、树不平衡、是否有重复值。变体：若频繁查询，可在节点维护子树大小成为顺序统计树。

LeetCode 232 用栈实现队列。模型：两个栈分别负责输入顺序和输出顺序。推导重点：只有输出栈为空时，才把输入栈整体倒过去，保证均摊常数。边界：连续 peek、空队列、交替 push pop。变体：用队列实现栈则要通过轮转保持最近元素在队首。

LeetCode 235 二叉搜索树最近公共祖先。模型：BST 的值域关系能决定两个目标在当前节点的哪一侧。推导重点：两个值都小去左，都大去右，否则当前节点就是分叉点。边界：一个节点是另一个祖先、目标顺序、重复值约定。变体：普通二叉树不能用值域剪枝，需要父指针或后序状态。

LeetCode 236 二叉树最近公共祖先。模型：普通树没有有序性，只能依靠祖先关系。推导重点：父指针集合或后序返回目标命中状态都可行。边界：目标不存在、p 等于 q、一个是另一个祖先、比较节点地址。变体：多次 LCA 查询可预处理倍增祖先表。

LeetCode 238 除自身以外数组的乘积。模型：不能用除法时，答案由左侧乘积和右侧乘积组成。推导重点：先写入左侧前缀积，再从右向左乘右侧后缀积。边界：一个零、多个零、负数、乘积溢出。变体：这个题展示输出数组可作为状态存储的一部分。

LeetCode 239 滑动窗口最大值。模型：每个窗口最大值来自仍未过期且没有被新元素支配的候选。推导重点：单调队列保存下标，队尾小于等于新值的候选被弹出。边界：k 为一、k 等于数组长度、重复最大值、队首过期。变体：受限子序列和使用同样队列维护最近 k 个 DP 最大值。

LeetCode 240 搜索二维矩阵 II。模型：右上角同时具有向左变小、向下变大的排除能力。推导重点：当前值大于目标就左移，小于目标就下移。边界：空矩阵、单行、单列、目标在角落、重复值。变体：从左上角无法一次排除一行或一列。

LeetCode 279 完全平方数。模型：完全平方数可以看成无限使用的物品，目标是凑出 n 的最少数量。推导重点：完全背包最少物品数，或把数字看成图节点做 BFS。边界：n 为零或一、完全

平方数本身、较大 n 。变体：数学四平方定理可给更强解法，但面试 DP 更通用。

LeetCode 283 移动零。模型：稳定保留非零元素顺序，把零集中到尾部。推导重点：慢指针写入下一个非零位置，最后补零或用交换。边界：全零、无零、零在开头结尾、稳定顺序要求。变体：若目标是移动指定值，模型与移除元素相同。

LeetCode 287 寻找重复数。模型：数组值域使下标到值形成函数图，重复数是环入口。推导重点：快慢指针不修改数组且常数空间；值域二分用抽屉原理。边界：重复数出现多次、不能排序、不能改数组、长度为 n 加一。变体：快慢指针要求值都能作为合法下标。

LeetCode 295 数据流的中位数。模型：数据流需要动态维护较小一半和较大一半。推导重点：大顶堆保存较小一半，小顶堆保存较大一半，大小差不超过一。边界：偶数个元素、负数、重复值、平均值溢出。变体：若还要删除任意元素，需要延迟删除哈希表。

LeetCode 300 最长递增子序列。模型：二次 DP 固定结尾，二分优化维护每个长度的最小结尾。推导重点：tails 不是实际答案序列，而是未来扩展潜力表。边界：重复元素、严格递增和非递减区别、空数组。变体：若要还原路径，需要额外前驱数组，不能只看 tails。

LeetCode 309 最佳买卖股票含冷冻期。模型：冷冻期让买入来源受昨天卖出状态限制。推导重点：状态机维护持有、刚卖出、休息三种日终状态。边界：空价格、一天、连续上涨、卖出后不能次日买入。变体：手续费版本可以在买入或卖出转移中扣费。

LeetCode 322 零钱兑换。模型：金额是容量，硬币可无限使用，目标是最少硬币数。推导重点：dp[value] 从 value 减 coin 的已知状态转移。边界：无法凑出、amount 为零、硬币面额大于目标、哨兵溢出。变体：零钱兑换 II 问组合数，循环语义不同。

LeetCode 337 打家劫舍 III。模型：树上相邻节点不能同时选，每个节点需要偷和不偷两个状态。推导重点：后序汇总：偷当前则孩子不能偷，不偷当前则孩子可偷可不偷。边界：空树、负值是否可能、链式树、递归深度。变体：可用显式栈后序保存节点状态，避免调用栈。

LeetCode 347 前 K 个高频元素。模型：先统计频次，再只维护前 K 个候选。推导重点：小顶堆大小超过 K 就弹出最低频；桶排序利用频次上界。边界：K 等于不同元素数、频次相同、结果顺序不要求。变体：若要按频次和字典序排序，比较器要同时处理两个键。

LeetCode 394 字符串解码。模型：嵌套结构需要保存上一层字符串和重复次数。推导重点：遇到左括号入栈当前状态，右括号弹出并拼接展开。边界：多位数字、嵌套、空片段、数字后必须跟括号。变体：递归下降解析更通用，但迭代栈更符合工程栈控制。

LeetCode 416 分割等和子集。模型：总和一半是背包容量，每个数只能使用一次。推导重点：0-1 背包可达性，一维容量必须倒序。边界：总和为奇数、目标为零、数值较大、复杂度依赖 target。变体：负数会破坏普通容量模型，需要重新建模。

LeetCode 417 太平洋大西洋水流问题。模型：从每个格子向海搜索很慢，反向从海边沿非递减高度搜索。推导重点：分别标记能到太平洋和大西洋的格子，交集就是答案。边界：单行单列、平台高度、边界重复入队、方向条件反向。变体：反向搜索是很多可达性题的重要技巧。

LeetCode 435 无重叠区间。模型：保留最多不重叠区间等价于删除最少区间。推导重点：按终点升序选择，结束越早给后续留下空间越多。边界：端点相接是否重叠、完全覆盖、相同终点、空列表。变体：用交换论证说明最早结束的选择不劣于其他第一选择。

LeetCode 438 找到字符串中所有字母异位词。模型：固定长度窗口内字符频次等于目标频次即为答案。推导重点：右进左出维护计数，窗口长度达到目标长度后检查差异。边界：目标比源串长、重复字符、字符集固定、答案重叠。变体：维护差异计数可以避免每次比较整个数组。

LeetCode 450 删除二叉搜索树中的节点。模型：删除节点后仍要保持 BST 中序有序。推导重点：若有两个孩子，用后继或前驱替换当前值，再删除那个后继节点。边界：删除叶子、一个孩子、两个孩子、删除根节点。变体：工程实现要清楚是移动值还是重连节点。

LeetCode 452 用最少数量的箭引爆气球。模型：一支箭的位置可以覆盖所有包含该位置的区间。推导重点：按右端点排序，当前箭放在最早结束气球的右端。边界：端点相同、负坐标、区间包含、闭区间边界。变体：它和无重叠区间是同一类最早结束贪心。

LeetCode 494 目标和。模型：正负号选择可代数转换为选若干数凑出特定正和。推导重点：若 S 加 target 为奇数或目标绝对值过大，直接无解。边界：零元素会让方案数翻倍、目标为负、容量为零。变体：也可用哈希 DP 保存所有可能和，但背包转换更高效。

LeetCode 496 下一个更大元素 I。模型：第二个数组提供每个值右侧第一个更大值的全局关系。推导重点：单调栈预处理值到下一个更大值映射，再回答第一个数组查询。边界：不存在更大

值、递减数组、值唯一假设。变体：若值不唯一，应按下标而不是值建映射。

LeetCode 503 下一个更大元素 II。模型：环形数组可以通过遍历两倍下标模拟绕回。推导重点：第一遍入栈，第二遍只帮助未解决下标找到答案。边界：全递减、全相等、长度一、取模下标。变体：不要在第二遍重复入栈，否则可能死循环或重复计算。

LeetCode 518 零钱兑换 II。模型：硬币无限使用，问组合数而不是最少数量。推导重点：外层遍历硬币、内层金额正序，保证组合按固定硬币顺序统计。边界：amount 为零、无硬币、组合数较大、循环顺序反例。变体：若外层金额内层硬币，会统计排列数。

LeetCode 543 二叉树直径。模型：直径经过某节点时等于左高加右高，但向父亲只能贡献单边高度。推导重点：后序遍历同时更新全局直径并返回高度。边界：空树、单节点、链式树、边数还是节点数定义。变体：最大路径和是同型题，但贡献值和全局更新有负数处理。

LeetCode 547 省份数量。模型：城市连接关系形成无向图，省份就是连通分量。推导重点：并查集合并所有直接连接的城市，最后统计根数量。边界：邻接矩阵对称、自连接、孤立城市、只扫描上三角。变体：BFS 也可做；并查集更突出动态合并。

LeetCode 560 和为 K 的子数组。模型：当前前缀和减去目标值，就是需要匹配的历史前缀。推导重点：哈希表保存前缀和出现次数，先查询再更新。边界：负数、目标为零、从下标零开始的区间、重复前缀。变体：若要求最长区间，哈希表应保存最早位置而不是次数。

LeetCode 581 最短无序连续子数组。模型：要找破坏全局有序性的最左和最右位置。推导重点：从左维护历史最大值确定右边界，从右维护历史最小值确定左边界。边界：已经有序、逆序、重复值、局部乱序。变体：排序比较法更直观但复杂度更高。

LeetCode 621 任务调度器。模型：相同任务之间有冷却间隔，核心受最高频任务骨架限制。推导重点：可用大顶堆模拟，也可用最高频公式计算最短时间。边界：冷却为零、多个最高频、空闲被其他任务填满。变体：若任务有不同释放时间或执行时长，就变成更一般的调度问题。

LeetCode 647 回文子串。模型：每个回文都有唯一中心或中心间隙。推导重点：对每个中心向两边扩展，扩展成功一次就计数一次。边界：空串、全相同字符、偶数回文、奇数回文。变体：Manacher 算法可线性解决，但中心扩展更适合面试基础。

LeetCode 684 冗余连接。模型：树加一条边会形成唯一环。推导重点：按顺序加边，若一条边两端已连通，则它就是冗余边。边界：节点编号、最后一条成环边、重复边、并查集初始化大小。变体：有向冗余连接需要同时处理入度为二和环，复杂度更高。

LeetCode 695 岛屿的最大面积。模型：面积就是一个连通块中陆地格子的数量。推导重点：每发现新陆地就搜索完整连通块并计数，取最大值。边界：全水、全陆、细长岛、是否允许修改输入。变体：与岛屿数量相比，只是连通块聚合值不同。

LeetCode 704 二分查找。模型：基础题的重点是固定区间语义。推导重点：左闭右开写法能统一 lower bound 和存在性检查。边界：空数组、长度一、目标在边界、目标不存在。变体：所有复杂二分都应该能退回到这道题的边界不变量。

LeetCode 714 买卖股票含手续费。模型：手续费改变交易收益，仍可用持有和不持有状态。推导重点：卖出时扣费或买入时扣费都可以，但不要重复扣。边界：手续费为零、价格震荡、小利润不足手续费。变体：这是状态机比局部贪心更稳的例子。

LeetCode 739 每日温度。模型：每一天等待右侧第一个更高温度。推导重点：单调递减栈保存尚未找到答案的下标。边界：没有更高温、相等温度、递增、递减。变体：下一个更大元素与它同型，只是输出值或距离不同。

LeetCode 743 网络延迟时间。模型：有向正权图从起点到所有节点的最短路最大值。推导重点：Dijkstra 求每个节点最短距离，最后取最大，若有无穷则不可达。边界：节点编号从一开始、不可达、重边、过期堆元素。变体：边权相同才可用普通 BFS。

LeetCode 752 打开转盘锁。模型：四位密码是状态，每次转动一位形成边。推导重点：BFS 从起点扩展，死亡状态不能入队，访问状态避免重复。边界：起点死亡、目标就是起点、数字环绕、死亡集合很大。变体：双向 BFS 能明显减少状态展开。

LeetCode 763 划分字母区间。模型：每个片段必须覆盖其中所有字符的最后出现位置。推导重点：扫描维护当前片段最远右端，到达右端即可切分。边界：单字符、所有字符相同、字符多次跨段。变体：这是最早合法切分贪心，能最大化片段数量。

LeetCode 875 爱吃香蕉的珂珂。模型：速度越大，总耗时越小，答案具有单调可行性。推导重点：二分速度，检查函数用向上取整累加小时数。边界：速度下界、堆很大、h 等于堆数、乘法加

法溢出。变体：船运包裹和分割数组都是同类最小可行上限。

LeetCode 973 最接近原点的 K 个点。模型：距离只需比较平方，避免开方误差和成本。推导重点：大小为 K 的大顶堆保存当前最近 K 个点中最远者。边界：K 为一、K 为全部、距离相同、坐标平方溢出。变体：快速选择可平均线性，但堆更适合数据流。

LeetCode 994 腐烂的橘子。模型：所有初始腐烂橘子同时作为 BFS 源点。推导重点：按层扩散，分钟数等于最后一个新鲜橘子被首次访问的层数。边界：没有新鲜橘子、没有腐烂源、隔离新鲜橘子、空格阻断。变体：多源 BFS 也适用于最近零距离、地图扩散等题。

LeetCode 1011 在 D 天内送达包裹。模型：容量越大，需要天数越少，存在最小可行容量。推导重点：下界是最大包裹，上界是总重量，检查函数按顺序贪心装船。边界：D 为一、D 等于包裹数、单个包裹超过猜测容量。变体：它和分割数组最大值都是连续分段最大和模型。

LeetCode 1143 最长公共子序列。模型：状态表示两个前缀的最长公共子序列长度。推导重点：末尾相同取左上加一，不同取上方和左方最大。边界：空串、完全相同、无公共字符、子序列不是子串。变体：若要求还原序列，需要从表右下角反向追踪选择。

LeetCode 1288 删除被覆盖区间。模型：被覆盖要求另一区间起点不晚且终点不早。推导重点：按起点升序、终点降序排序，扫描维护最大右端。边界：同起点区间、完全覆盖、无覆盖、端点相等。变体：终点降序是关键，否则同起点小区间会先出现造成误判。

LeetCode 1631 最小体力消耗路径。模型：路径代价是沿途最大高度差，不是边权和。推导重点：Dijkstra 的松弛改成取当前代价和新边代价的最大值。边界：单格、平坦网格、必须绕路、多个相同代价。变体：也可二分体力上限后 BFS 检查可达。

二刷标准

二刷不是重新看一遍答案，而是把题目变成可讲、可证、可调试、可迁移的知识块。每道题至少回答：暴力瓶颈、优化依据、不变量、复杂度、边界、C++ 成本和一个变体。

逐题单点推导手册

这一节专门解决“每一题思路都要讲清楚”的问题。它不重复完整代码，而是要求每道题都从自己的题面出发，说明暴力基线、优化线索、状态不变量、边界和变体。复习时可以把这一节当作口述题解提纲。

LeetCode 1 两数之和

题面模型。当前元素只需要寻找唯一补数，历史值到下标的映射就是最小状态。

暴力瓶颈。先查再插入，避免同一个下标被用两次；若数组有序才考虑双指针。

边界反例。重复数字、目标为偶数且补数等于当前值、没有答案时返回空结果。

变体迁移。若改成返回所有不重复数对，要先排序或用频次表处理重复。

LeetCode 2 两数相加

题面模型。链表按低位到高位保存数字，本质是竖式加法而不是整数转换。

暴力瓶颈。维护两个游标和进位，任一链未结束或进位非零都继续生成节点。

边界反例。两链长度不同、最后进位、输入为零、不能用内置整数承载超长数字。

变体迁移。若链表高位在前，可以用栈反向处理或先反转链表。

LeetCode 3 无重复字符的最长子串

题面模型。窗口保存当前无重复子串，右端每次加入一个字符。

暴力瓶颈。用上次出现位置直接跳过无效左端，左边界只能向右不能回退。

边界反例。空串、全相同字符、重复字符出现在窗口左侧之外、扩展字符集。

变体迁移。若要求恰好包含 k 种字符，窗口条件从无重复变成种类计数。

LeetCode 5 最长回文子串

题面模型。回文围绕中心对称，中心可以是字符也可以是两个字符之间。

暴力瓶颈。中心扩展避免枚举子串后再逐个检查；每次扩展只比较新增左右边界。

边界反例。偶数长度回文、单字符、全相同字符、多个同长答案。

变体迁移。若要求回文子序列，应转成区间 DP，不能用中心扩展。

LeetCode 11 盛最多水的容器

题面模型。左右指针代表当前选取的两条边，面积由短板和宽度共同决定。

暴力瓶颈。每次移动短板，因为移动长板只会缩小宽度且不会提高短板。

边界反例。两端高度相等、数组长度为二、面积乘法可能溢出。

变体迁移。接雨水计算每个位置贡献，不能把本题双指针证明直接搬过去。

LeetCode 15 三数之和

题面模型。排序后固定第一个数，剩余两数转成有序双指针。

暴力瓶颈。和太小移动左指针，和太大移动右指针；固定值和左右值都要跳过重复。

边界反例。全零、重复元素很多、没有答案、结果顺序不要求但组合不能重复。

变体迁移。四数之和再外层固定一个数，并用 long long 防止目标和溢出。

LeetCode 16 最接近的三数之和

题面模型。排序后固定一数，双指针寻找最接近目标的两数和。

暴力瓶颈。每次根据当前和与目标的大小移动指针，同时更新最小绝对差。

边界反例。差值相等、负数、目标很大、三数和溢出。

变体迁移。若要求返回所有最接近组合，需要记录同差值并去重。

LeetCode 18 四数之和

题面模型。两层固定加一层双指针，关键是剪枝和去重。

暴力瓶颈。排序提供单调性，外层可用最小可能和和最大可能和提前跳过。

边界反例。重复元素、目标超出 int、答案很多导致输出空间主导复杂度。

变体迁移。k 数之和可以递归降维，但工程实现要控制重复和边界。

LeetCode 26 删除有序数组重复项

题面模型。有序数组让重复元素相邻，慢指针表示下一个唯一值写入位置。

暴力瓶颈。快指针扫描，只有发现新值才写入慢指针并推进。

边界反例。空数组、全重复、无重复、返回长度不等于物理容量。

变体迁移。若允许每个元素最多保留两次，只需改变写入条件为和前两个比较。

LeetCode 27 移除元素

题面模型。原地过滤，慢指针左侧始终是不等于目标值的稳定前缀。

暴力瓶颈。保持顺序用快慢指针；不保持顺序可用左右交换减少写入。

边界反例。所有元素都被移除、没有元素被移除、目标值在尾部密集。

变体迁移。工程里要先确认是否要求稳定顺序，不同要求对应不同写法。

LeetCode 31 下一个排列

题面模型。字典序结构由最长非递增后缀决定，枢轴是后缀前一个位置。

暴力瓶颈。从右找第一个上升点，再从右找刚好更大的元素交换，最后反转后缀。

边界反例。完全降序、重复数字、只有一个元素、交换后后缀必须最小化。

变体迁移。若要求上一个排列，比较方向和后缀处理对称变化。

LeetCode 33 搜索旋转排序数组

题面模型。旋转数组每次二分至少有一半保持有序。

暴力瓶颈。判断哪一半有序，再看目标是否落在有序半边范围内。

边界反例。未旋转、长度一、目标不存在、边界等号、存在重复元素时退化。

变体迁移。若重复元素很多，需要在无法判断有序半边时收缩边界。

LeetCode 34 排序数组中查找首尾位置

题面模型。首尾位置可以拆成两个边界：第一个大于等于和第一个大于。

暴力瓶颈。不要找到一个目标后向两边线性扩展，否则重复元素很多时退化。

边界反例。目标不存在、目标在开头或结尾、数组为空、全是目标。

变体迁移。这个模型能迁移到计数某值出现次数和插入区间边界。

LeetCode 35 搜索插入位置

题面模型。题目等价于寻找第一个大于等于目标的位置。

暴力瓶颈。统一用 lower bound 语义，不在相等时提前返回也能保持边界一致。

边界反例。目标小于所有元素、目标大于所有元素、重复值、空数组。

变体迁移。手写二分时用左闭右开可直接返回 left。

LeetCode 42 接雨水

题面模型。每个位置的水由左侧最高和右侧最高的较小者决定。

暴力瓶颈。前后缀、双指针、单调栈都是不同状态维护方式；要能说明各自结算对象。

边界反例。单调递增、单调递减、平台高度、多个凹槽、数组长度不足三。

变体迁移。若柱子变成二维网格，需要最小堆从边界向内扩展。

LeetCode 49 字母异位词分组

题面模型。异位词共享同一个规范表示，可以是排序字符串或字符计数。

暴力瓶颈。哈希表按规范键分桶，避免两两比较字符串。

边界反例。空字符串、重复单词、字符集不止小写字母、计数键必须无歧义。

变体迁移。若字符串很长且字符集固定，计数键通常比排序键更稳定。

LeetCode 53 最大子数组和

题面模型。状态是必须以当前位置结尾的最大连续和。

暴力瓶颈。若前一个后缀为负，接上它只会拖累当前元素，因此可以重新开始。

边界反例。全负数组、单元素、和溢出、答案不能初始化为零。

变体迁移。若要求返回区间下标，同步记录重新开始位置和最优左右端。

LeetCode 54 螺旋矩阵

题面模型。四个边界收缩模拟一圈一圈访问。

暴力瓶颈。每走完一条边就收缩对应边界，并在走反方向前检查是否仍合法。

边界反例。单行、单列、空矩阵、最后一圈只剩一个元素。

变体迁移。若改成生成矩阵，边界逻辑相同，只是读变成写。

LeetCode 55 跳跃游戏

题面模型。扫描过程中只关心当前能到达的最远位置。

暴力瓶颈。如果下标超过最远可达，任何之前路径都不能到达它；否则用它扩展边界。

边界反例。第一个元素为零、数组长度一、恰好到达最后、远超最后。

变体迁移。求最少跳数时把可达区间看成 BFS 层。

LeetCode 56 合并区间

题面模型。排序后可能与当前区间相交的只会是结果尾部。

暴力瓶颈。按起点排序，扫描时维护结果最后区间的右端点。

边界反例。端点相接、完全覆盖、无重叠、空列表、输入未排序。

变体迁移。若区间是半开区间，端点相等不一定合并。

LeetCode 57 插入区间

题面模型。新区间只会影响与它相交的一段连续区间。

暴力瓶颈。先追加左侧完全不相交区间，再合并重叠段，最后追加右侧。

边界反例。新区间在最前、最后、覆盖全部、刚好相邻、原列表为空。

变体迁移。若输入不是有序且不重叠，必须先排序或换模型。

LeetCode 62 不同路径

题面模型。网格状态由上方和左方两种最后一步转移而来。

暴力瓶颈。二维表可压缩为一维，因为当前行只依赖上一行同列和当前行左侧。

边界反例。一行、一列、较大网格结果溢出、障碍物版本边界。

变体迁移。带障碍物时遇到障碍状态清零，第一行第一列要按阻断处理。

LeetCode 63 不同路径 II

题面模型。障碍物让某些格子的路径数变为零。

暴力瓶颈。滚动数组中遇到障碍直接把当前位置清零，否则累加左侧。

边界反例。起点或终点是障碍、整行被截断、单行障碍。

变体迁移。若允许向上或向左走，图会有环，普通填表不再成立。

LeetCode 64 最小路径和

题面模型。状态是到达当前格子的最小代价，最后一步来自上方或左方。

暴力瓶颈。先初始化第一行和第一列，再处理内部，主转移保持简单。

边界反例。单行、单列、权值为零、路径和溢出。

变体迁移。若允许四方向移动，变成最短路而不是普通网格 DP。

LeetCode 70 爬楼梯

题面模型。最后一步来自前一阶或前两阶，是最小的重叠子问题模型。

暴力瓶颈。滚动变量保存最近两个状态即可，不需要完整数组。

边界反例。 n 为一、 n 为二、题目是否允许零阶、结果范围。

变体迁移。若每次可走一到 k 阶，转移变成固定宽度窗口求和。

LeetCode 72 编辑距离

题面模型。二维状态表示两个前缀之间的最少编辑操作。

暴力瓶颈。末尾字符相同继承左上；不同则枚举插入、删除、替换。

边界反例。空串、完全相同、完全不同、操作代价不同。

变体迁移。若只允许插入删除，问题退化到和最长公共子序列相关。

LeetCode 75 颜色分类

题面模型。三指针维护零区、一号区、未知区和二区。

暴力瓶颈。遇到二时只移动右边界，不移动扫描指针，因为换回来的元素未知。

边界反例。全零、全二、已经有序、逆序、交换后漏检查。

变体迁移。这是荷兰国旗问题，能推广到三类分区的原地维护。

LeetCode 76 最小覆盖子串

题面模型。窗口内字符计数必须覆盖目标计数，满足后尽量收缩左端。

暴力瓶颈。用 `formed` 记录满足需求的字符种类，避免每次全量比较。

边界反例。目标有重复字符、源串缺字符、最小窗口在开头或结尾、大小写。

变体迁移。若字符集固定，数组计数更快；若是 Unicode，哈希表更通用。

LeetCode 78 子集

题面模型。每个元素都有选或不选两种选择，答案是所有路径节点。

暴力瓶颈。回溯按起点向后枚举，避免不同顺序生成同一集合。

边界反例。空数组、单元素、输出数量为二的 n 次方、顺序不要求。

变体迁移。也可用位掩码枚举所有子集，适合 n 较小的场景。

LeetCode 79 单词搜索

题面模型。网格路径不能重复使用同一格，状态包含位置、匹配下标和访问标记。

暴力瓶颈。剪枝包括首字符不匹配、剩余长度不足、字符频次不足和越界访问。

边界反例。单字符单词、路径需要回退、重复字母、单元格不能复用。

变体迁移。若要查很多单词，应使用 Trie 合并搜索前缀。

LeetCode 84 柱状图中最大的矩形

题面模型。每根柱子作为最矮高度时，需要知道左右第一个更矮位置。

暴力瓶颈。单调递增栈在遇到更矮柱时结算被弹出柱子的最大宽度。

边界反例。递增、递减、相等高度、哨兵零、宽度计算。

变体迁移。最大矩形可以作为二维矩阵最大矩形的行压缩子问题。

LeetCode 88 合并两个有序数组

题面模型。第一个数组尾部有空位，适合从后往前写入最终最大值。

暴力瓶颈。逆向填充避免覆盖尚未读取的 `nums1` 元素。

边界反例。第二个数组为空、第一个有效元素为空、重复值、尾部剩余。

变体迁移。若没有额外空位，只能新开数组或使用更复杂原地归并。

LeetCode 90 子集 II

题面模型。重复元素会让不同下标路径生成相同集合。

暴力瓶颈。排序后在同一层跳过相同值，保留不同层使用重复值的可能。

边界反例。全重复、空集、重复值相邻、去重条件写成同层而不是全局。

变体迁移。组合总和 II 使用同样的同层去重思想。

LeetCode 94 二叉树中序遍历

题面模型。中序访问顺序是左子树、根、右子树，BST 中会得到有序序列。

暴力瓶颈。用显式栈模拟一路向左，再回退访问根并转向右子树。

边界反例。空树、单节点、链式左树、链式右树。

变体迁移。Morris 遍历可做到常数额外空间，但会临时改树结构。

LeetCode 98 验证二叉搜索树

题面模型。BST 约束是全局上下界，不是只比较父子节点。

暴力瓶颈。中序严格递增或显式传递上下界都可以；中序版本要保存前一个访问值。

边界反例。重复值、`int` 最小最大值、局部合法但全局非法。

变体迁移。若允许重复值，需要明确重复放左还是放右。

LeetCode 102 二叉树层序遍历

题面模型。队列在每轮开始时保存当前层所有节点。

暴力瓶颈。记录 level size 固定层边界，避免下一层节点混入当前层。

边界反例。空树、只有根、最后一层不满、左右顺序。

变体迁移。锯齿形层序只是在每层输出顺序上增加方向控制。

LeetCode 103 二叉树锯齿形层序遍历

题面模型。BFS 层边界不变，变化的是当前层写入结果的方向。

暴力瓶颈。可以层内反转，也可以预分配数组按方向写入目标下标。

边界反例。单层、偶数层、奇数层、空树。

变体迁移。若要求从底向上输出，收集后反转层列表即可。

LeetCode 104 二叉树最大深度

题面模型。深度可以看成层数，也可以作为栈中携带的状态。

暴力瓶颈。显式栈保存节点和深度，避免极端链式树递归栈过深。

边界反例。空树返回零、根深度是一、链式树。

变体迁移。最小深度不能简单取左右最小，因为空子树不构成叶子路径。

LeetCode 105 前序中序构造二叉树

题面模型。前序给根，中序把左右子树切开。

暴力瓶颈。用哈希表记录中序下标，避免每次线性寻找根位置。

边界反例。节点值唯一、空子树、左右子树长度计算、输入不合法。

变体迁移。后序中序构造类似，只是根来自后序末尾。

LeetCode 106 中序后序构造二叉树

题面模型。后序末尾是根，中序位置确定左右子树规模。

暴力瓶颈。构造时要小心后序左右区间边界，避免切分错位。

边界反例。空树、单节点、全左链、全右链。

变体迁移。若给前序和后序，二叉树通常不能唯一确定，除非额外限制。

LeetCode 108 有序数组转二叉搜索树

题面模型。有序数组中点作为根能让左右规模接近。

暴力瓶颈。每次选择区间中点，左右子区间构造左右子树。

边界反例。空区间、偶数长度选左中还是右中、高度平衡定义。

变体迁移。工程上递归可读，若严格避免递归可用队列保存区间和父指针。

LeetCode 110 平衡二叉树

题面模型。每个节点需要知道子树高度以及是否已经失衡。

暴力瓶颈。后序汇总时一旦子树失衡就向上返回失败标记，避免重复算高度。

边界反例。空树、单节点、左右高度差正好一、链式树。

变体迁移。可以把返回值设计成 optional 高度，空表示不平衡。

LeetCode 111 二叉树最小深度

题面模型。最小深度是到最近叶子的节点数，不是左右深度简单取最小。

暴力瓶颈。BFS 第一次遇到叶子就是最小深度，因为按层扩展。

边界反例。只有左子树、只有右子树、根为空、根是叶子。

变体迁移。DFS 写法必须特殊处理空子树不能作为最小路径。

LeetCode 112 路径总和

题面模型。路径必须从根到叶子，状态是剩余目标值。

暴力瓶颈。沿路径减去当前节点值，到叶子时检查剩余是否等于叶子值。

边界反例。负数节点、目标为零、非叶子提前达到目标、空树。

变体迁移。若要求列出所有路径，需要保存路径数组并在回退时弹出。

LeetCode 113 路径总和 II

题面模型。相比判定题，状态还要保存当前路径。

暴力瓶颈。到叶子且和满足时复制路径；回溯时恢复路径。

边界反例。多个答案、负数、空树、路径数组复制时机。

变体迁移。若路径不要求从根到叶，模型会变成前缀和加哈希。

LeetCode 114 二叉树展开为链表

题面模型。目标是按前序顺序把树原地改成右指针链。

暴力瓶颈。显式栈按右后左先入顺序处理，逐个接到前驱右侧。

边界反例。空树、单节点、原有右子树不能丢、左指针置空。

变体迁移。也可用寻找左子树最右节点的原地方法。

LeetCode 121 买卖股票的最佳时机

题面模型。卖出日固定时，最佳买入日是此前最低价格。

暴力瓶颈。扫描维护历史最低价和最大利润，一次交易无需复杂状态机。

边界反例。单日价格、一直下跌、一直上涨、价格相同。

变体迁移。多次交易、冷冻期和手续费版本都要扩展成状态机。

LeetCode 122 买卖股票 II

题面模型。允许多次交易且不能同时持有，所有正收益差都可以收集。

暴力瓶颈。把长上涨段拆成每天买卖收益相同，因此累加正差值最优。

边界反例。下降段、平台价格、只有一天、手续费不存在。

变体迁移。有手续费时不能简单累加正差，需要状态机或调整贪心阈值。

LeetCode 123 买卖股票 III

题面模型。最多两次交易，状态需要包含交易次数和持有状态。

暴力瓶颈。维护 buy1、sell1、buy2、sell2 四个状态，按天更新。

边界反例。价格为空、只够一次交易、两次交易间不能重叠。

变体迁移。最多 k 次交易是它的泛化，k 很大时退化为无限次交易。

LeetCode 124 二叉树最大路径和

题面模型。路径可以从任意节点到任意节点，但不能分叉向父节点贡献两边。

暴力瓶颈。每个节点向父亲贡献单边最大增益，全局答案可用左右两边加当前节点更新。

边界反例。全负数、单节点、左右增益为负时取零、答案初始值。

变体迁移。若路径必须从根到叶，状态和答案位置完全不同。

LeetCode 127 单词接龙

题面模型。每个单词是节点，一次修改一个字符形成边。

暴力瓶颈。通配模式桶加速邻居查询，BFS 第一次到达终点就是最短转换长度。

边界反例。终点不在词表、起点和终点相同、桶重复扫描、单词长度一致。

变体迁移。双向 BFS 可以进一步降低搜索空间。

LeetCode 128 最长连续序列

题面模型。连续段只需要从没有前驱的数开始扩展。

暴力瓶颈。哈希集合判断 x 减一是否存在，跳过非起点避免重复扫描。

边界反例。重复元素、负数、空数组、整数边界。

变体迁移。若数据流动态插入，可以用并查集或区间合并维护长度。

LeetCode 130 被围绕的区域

题面模型。只有不连通到边界的 O 才会被翻转。

暴力瓶颈。从边界 O 出发标记安全区域，再翻转剩余 O。

边界反例。全边界、无 O、单行单列、内部岛屿。

变体迁移。这是反向思维：找不能被翻转的区域比直接判断每块区域更简单。

LeetCode 131 分割回文串

题面模型。每次选择一个从当前位置开始的回文前缀。

暴力瓶颈。预处理回文 DP 表，让回溯合法性检查为常数。

边界反例。空串、单字符、全相同字符、重复分割路径。

变体迁移。若只求最少切割次数，可改成 DP 最短切分。

LeetCode 133 克隆图

题面模型。每个原节点需要唯一对应一个新节点，边按原图连接。

暴力瓶颈。哈希表保存原节点到克隆节点的映射，BFS 或 DFS 遍历图。

边界反例。空图、自环、重边、环结构、不能按节点值唯一假设。

变体迁移。若图节点值不唯一，必须用地址作为键。

LeetCode 136 只出现一次的数字

题面模型。成对元素可以用异或抵消，剩下单个元素。

暴力瓶颈。异或满足交换结合，扫描顺序不影响答案。

边界反例。零、负数、唯一值在开头或结尾、所有其他值恰好两次。

变体迁移。若其他值出现三次，要改为按位计数取模。

LeetCode 139 单词拆分

题面模型。状态表示前缀能否由字典单词拼出。

暴力瓶颈。枚举最后一个单词的起点，左侧前缀可达且子串在字典中则当前可达。

边界反例。空串、重复单词、长单词、substr 复制成本。

变体迁移。若要输出所有句子，需要 DFS 加记忆化或前驱列表。

LeetCode 141 环形链表

题面模型。快慢指针在有环时必然相遇，无环时快指针先到空。

暴力瓶颈。不用哈希表也能常数空间检测环。

边界反例。空链、单节点自环、两节点环、尾部无环。

变体迁移。若要求环入口，用相遇后双指针同步走。

LeetCode 142 环形链表 II

题面模型。相遇点到入口的距离关系来自快慢指针速度差。

暴力瓶颈。相遇后一个指针回头，两个指针每次走一步，再次相遇就是入口。

边界反例。入口在头节点、长尾短环、无环、单节点环。

变体迁移。也可用哈希集合，但空间更高。

LeetCode 146 LRU 缓存

题面模型。哈希表负责按 key 定位，双向链表负责维护新旧顺序。

暴力瓶颈。访问和更新都把节点移动到头部，容量满时删除尾部最旧节点。

边界反例。容量为一、重复 put、get 不存在、更新已有值。

变体迁移。C++ 可用 `std::list` 加迭代器，也可手写双向链表理解所有权。

LeetCode 148 排序链表

题面模型。链表不能随机访问，适合归并排序而不是快速排序数组 partition。

暴力瓶颈。自底向上迭代归并能避免递归栈，同时保持 $O(n \log n)$ 。

边界反例。空链、单节点、重复值、奇数长度、切断子链。

变体迁移。若把值复制到数组排序，会改变节点身份语义。

LeetCode 150 逆波兰表达式求值

题面模型。后缀表达式把优先级和括号都编码进 token 顺序。

暴力瓶颈。遇到数字入栈，遇到运算符弹出右操作数和左操作数计算。

边界反例。负数 token、多位数、除法向零截断、减法顺序。

变体迁移。中缀表达式求值需要另一个运算符栈处理优先级。

LeetCode 152 乘积最大子数组

题面模型。负数会让最大乘积和最小乘积互换。

暴力瓶颈。同时维护以当前位置结尾的最大和最小乘积。

边界反例。零、负数个数奇偶、单元素、乘积溢出。

变体迁移。如果要求返回区间，同步记录状态来源。

LeetCode 153 寻找旋转排序数组中的最小值

题面模型。最小值是旋转断点，右端点可帮助判断中点在哪一段。

暴力瓶颈。若 mid 大于 right，最小值在右侧；否则在左侧含 mid。

边界反例。未旋转、长度一、旋转一位、无重复条件。

变体迁移。有重复时遇到相等只能收缩 right，最坏退化。

LeetCode 154 寻找旋转排序数组中的最小值 II

题面模型。重复元素会让 mid 和 right 相等时无法判断最小值方向。

暴力瓶颈。相等时只能安全地丢掉一个 right，因为它不是唯一必要候选。

边界反例。全相等、重复值跨断点、最小值出现多次。

变体迁移。这题说明二分依赖的信息不足时会退化，不是所有旋转题都严格对数。

LeetCode 155 最小栈

题面模型。普通栈只能看到栈顶，不能常数时间知道历史最小值。

暴力瓶颈。辅助栈同步保存每个深度对应的最小值。

边界反例。重复最小值、弹出最小值、空栈操作约定。

变体迁移。也可保存差值压缩空间，但可读性和溢出处理更复杂。

LeetCode 160 相交链表

题面模型。相交是节点地址相同，不是节点值相同。

暴力瓶颈。两个指针走完各自链表后切换到另一条链，走过总长度相同后相遇。

边界反例。不相交、头节点相交、尾节点相交、长度差很大。

变体迁移。哈希集合更直观但空间更高。

LeetCode 162 寻找峰值

题面模型。比较 mid 和 mid 加一的斜率能判断某侧一定存在峰值。

暴力瓶颈。上升则右侧有峰，下降则左侧含 mid 有峰。

边界反例。长度一、峰在边界、多个峰、相邻不相等假设。

变体迁移。二维峰值需要按行列最大值进行更复杂二分。

LeetCode 169 多数元素

题面模型。超过一半的元素可以和其他元素两两抵消后仍然剩下。

暴力瓶颈。Boyer-Moore 投票维护候选和计数。

边界反例。题目是否保证存在多数、计数归零、全相同。

变体迁移。若找超过三分之一的元素，需要维护两个候选。

LeetCode 189 轮转数组

题面模型。右移 k 位等价于把数组切成两段后换序。

暴力瓶颈。三次反转能原地完成：整体、前 k、后 n 减 k。

边界反例。k 为零、k 大于 n、空数组、单元素。

变体迁移。环状替换也可做，但要处理最大公约数个循环。

LeetCode 198 打家劫舍

题面模型。每间房只有偷和不偷两种结局，偷当前就不能偷前一间。

暴力瓶颈。滚动保存前一状态和前两状态。

边界反例。空数组、单间、全零、金额很大。

变体迁移。环形房屋要拆成不含首和不含尾两个线性问题。

LeetCode 199 二叉树右视图

题面模型。每层从右侧能看到的的就是该层最后访问或最先访问的右侧节点。

暴力瓶颈。BFS 固定层边界，取每层最后一个；或 DFS 按右优先首次到达深度。

边界反例。空树、左链、右链、左右交错。

变体迁移。若求左视图，只改变层内取值方向。

LeetCode 200 岛屿数量

题面模型。陆地格子是节点，四方向相邻是边，岛屿是连通块。

暴力瓶颈。扫描遇到未访问陆地就启动搜索并标记整块。

边界反例。空网格、全水、全陆、斜对角不连通、是否允许修改输入。

变体迁移。也可用并查集合并相邻陆地，适合动态岛屿扩展。

LeetCode 202 快乐数

题面模型。反复变换数字会进入一或循环，状态空间最终有限。

暴力瓶颈。用集合检测重复状态，或用快慢指针看成函数图。

边界反例。输入为一、进入循环、位平方和函数、负数不在题意内。

变体迁移。快慢指针版本能训练把数值迭代看成链表。

LeetCode 206 反转链表

题面模型。每次把当前节点的 next 指向已经反转好的前缀头。

暴力瓶颈。先保存后继，再改指针，再推进 previous 和 current。

边界反例。空链、单节点、两个节点、不要丢失剩余链。

变体迁移。K 组反转是在这段局部反转外增加分组边界。

LeetCode 207 课程表

题面模型。课程是节点，先修关系是有向边，问题是判断是否有环。

暴力瓶颈。入度拓扑排序不断学习当前无依赖课程。

边界反例。边方向、孤立课程、环、自依赖、重复边。

变体迁移。DFS 三色标记也能判环，但要明确访问状态。

LeetCode 208 实现 Trie

题面模型。前缀树把字符串公共前缀共享成路径。

暴力瓶颈。插入和查询都逐字符沿边移动，不存在的边按需要创建。

边界反例。空字符串约定、单词结束标记、前缀存在不等于单词存在。

变体迁移。若字符集很大，用哈希子节点；小写字母用数组更快。

LeetCode 209 长度最小的子数组

题面模型。正数数组让窗口和随右端增加、随左端减少。

暴力瓶颈。右端扩展到满足目标后，不断左移缩短并更新答案。

边界反例。不存在答案、单元素达到目标、全正是必要条件。

变体迁移。若含负数，需要前缀和加单调队列。

LeetCode 210 课程表 II

题面模型。在判环基础上还要输出一个可行拓扑序。

暴力瓶颈。每弹出一个入度为零课程，就把它加入顺序并删除出边。

边界反例。有环返回空数组、多个可行顺序、边方向。

变体迁移。若需要字典序最小拓扑序，把队列换成小顶堆。

LeetCode 211 添加与搜索单词

题面模型。点号通配符让查询可能分叉，但前缀树仍能剪掉不匹配前缀。

暴力瓶颈。普通字符沿唯一边走，点号枚举当前节点所有孩子。

边界反例。连续通配符、空结果、单词结束标记、字符集大小。

变体迁移。大量通配符会接近遍历整棵 Trie，需要规模约束。

LeetCode 215 数组中的第 K 个最大元素

题面模型。只关心第 K 大，不需要完整排序。

暴力瓶颈。大小为 K 的小顶堆保存当前前 K 大边界；快速选择可做到平均线性。

边界反例。K 为一、K 为 n、重复值、随机 pivot。

变体迁移。若数据流持续到来，堆方案比一次性快速选择更自然。

LeetCode 216 组合总和 III

题面模型。从一到九中选 k 个数和为 n，每个数最多用一次。

暴力瓶颈。按递增起点枚举，利用剩余个数和剩余和剪枝。

边界反例。n 太小、n 太大、k 为零、路径长度已经超过 k。

变体迁移。可预估剩余最小和最大和进一步剪枝。

LeetCode 217 存在重复元素

题面模型。判断是否出现过是哈希集合最直接的职责。

暴力瓶颈。扫描时若插入前已经存在则立即返回真。

边界反例。空数组、全唯一、重复在末尾、值域很小。

变体迁移。若允许修改输入，排序后看相邻可以降低额外空间。

LeetCode 221 最大正方形

题面模型。以当前位置为右下角的最大正方形由上、左、左上三个状态限制。

暴力瓶颈。当前位置为一时取三者最小加一，否则为零。

边界反例。全零、全一、单行单列、字符一和数字一不要混淆。

变体迁移。最大矩形则需要按行构造柱状图再用单调栈。

LeetCode 226 翻转二叉树

题面模型。每个节点交换左右孩子即可，遍历顺序不影响最终结构。

暴力瓶颈。显式队列或栈逐个节点交换，避免递归深度。

边界反例。空树、单节点、只有一侧子树、重复值。

变体迁移。这题简单但适合训练树节点指针修改。

LeetCode 230 二叉搜索树中第 K 小

题面模型。 BST 中序遍历按升序访问节点。

暴力瓶颈。 显式栈中序，访问第 k 个节点时返回。

边界反例。 k 为一、k 为节点数、树不平衡、是否有重复值。

变体迁移。 若频繁查询，可在节点维护子树大小成为顺序统计树。

LeetCode 232 用栈实现队列

题面模型。 两个栈分别负责输入顺序和输出顺序。

暴力瓶颈。 只有输出栈为空时，才把输入栈整体倒过去，保证均摊常数。

边界反例。 连续 peek、空队列、交替 push pop。

变体迁移。 用队列实现栈则要通过轮转保持最近元素在队首。

LeetCode 235 二叉搜索树最近公共祖先

题面模型。 BST 的值域关系能决定两个目标在当前节点的哪一侧。

暴力瓶颈。 两个值都小去左，都大去右，否则当前节点就是分叉点。

边界反例。 一个节点是另一个祖先、目标顺序、重复值约定。

变体迁移。 普通二叉树不能用值域剪枝，需要父指针或后序状态。

LeetCode 236 二叉树最近公共祖先

题面模型。 普通树没有有序性，只能依靠祖先关系。

暴力瓶颈。 父指针集合或后序返回目标命中状态都可行。

边界反例。 目标不存在、p 等于 q、一个是另一个祖先、比较节点地址。

变体迁移。 多次 LCA 查询可预处理倍增祖先表。

LeetCode 238 除自身以外数组的乘积

题面模型。 不能用除法时，答案由左侧乘积和右侧乘积组成。

暴力瓶颈。 先写入左侧前缀积，再从右向左乘右侧后缀积。

边界反例。 一个零、多个零、负数、乘积溢出。

变体迁移。 这个题展示输出数组可作为状态存储的一部分。

LeetCode 239 滑动窗口最大值

题面模型。 每个窗口最大值来自仍未过期且没有被新元素支配的候选。

暴力瓶颈。 单调队列保存下标，队尾小于等于新值的候选被弹出。

边界反例。 k 为一、k 等于数组长度、重复最大值、队首过期。

变体迁移。 受限子序列和使用同样队列维护最近 k 个 DP 最大值。

LeetCode 240 搜索二维矩阵 II

题面模型。 右上角同时具有向左变小、向下变大的排除能力。

暴力瓶颈。 当前值大于目标就左移，小于目标就下移。

边界反例。 空矩阵、单行、单列、目标在角落、重复值。

变体迁移。 从左上角无法一次排除一行或一列。

LeetCode 279 完全平方数

题面模型。 完全平方数可以看成无限使用的物品，目标是凑出 n 的最少数量。

暴力瓶颈。 完全背包最少物品数，或把数字看成图节点做 BFS。

边界反例。 n 为零或一、完全平方数本身、较大 n。

变体迁移。 数学四平方定理可给更强解法，但面试 DP 更通用。

LeetCode 283 移动零

题面模型。 稳定保留非零元素顺序，把零集中到尾部。

暴力瓶颈。 慢指针写入下一个非零位置，最后补零或用交换。

边界反例。 全零、无零、零在开头结尾、稳定顺序要求。

变体迁移。 若目标是移动指定值，模型与移除元素相同。

LeetCode 287 寻找重复数

题面模型。数组值域使下标到值形成函数图，重复数是环入口。

暴力瓶颈。快慢指针不修改数组且常数空间；值域二分用抽屉原理。

边界反例。重复数出现多次、不能排序、不能改数组、长度为 n 加一。

变体迁移。快慢指针要求值都能作为合法下标。

LeetCode 295 数据流的中位数

题面模型。数据流需要动态维护较小一半和较大一半。

暴力瓶颈。大顶堆保存较小一半，小顶堆保存较大一半，大小差不超过一。

边界反例。偶数个元素、负数、重复值、平均值溢出。

变体迁移。若还要删除任意元素，需要延迟删除哈希表。

LeetCode 300 最长递增子序列

题面模型。二次 DP 固定结尾，二分优化维护每个长度的最小结尾。

暴力瓶颈。tails 不是实际答案序列，而是未来扩展潜力表。

边界反例。重复元素、严格递增和非递减区别、空数组。

变体迁移。若要还原路径，需要额外前驱数组，不能只看 tails。

LeetCode 309 最佳买卖股票含冷冻期

题面模型。冷冻期让买入来源受昨天卖出状态限制。

暴力瓶颈。状态机维护持有、刚卖出、休息三种日终状态。

边界反例。空价格、一天、连续上涨、卖出后不能次日买入。

变体迁移。手续费版本可以在买入或卖出转移中扣费。

LeetCode 322 零钱兑换

题面模型。金额是容量，硬币可无限使用，目标是最少硬币数。

暴力瓶颈。dp[value] 从 value 减 coin 的已知状态转移。

边界反例。无法凑出、amount 为零、硬币面额大于目标、哨兵溢出。

变体迁移。零钱兑换 II 问组合数，循环语义不同。

LeetCode 337 打家劫舍 III

题面模型。树上相邻节点不能同时选，每个节点需要偷和不偷两个状态。

暴力瓶颈。后序汇总：偷当前则孩子不能偷，不偷当前则孩子可偷可不偷。

边界反例。空树、负值是否可能、链式树、递归深度。

变体迁移。可用显式栈后序保存节点状态，避免调用栈。

LeetCode 347 前 K 个高频元素

题面模型。先统计频次，再只维护前 K 个候选。

暴力瓶颈。小顶堆大小超过 K 就弹出最低频；桶排序利用频次上界。

边界反例。K 等于不同元素数、频次相同、结果顺序不要求。

变体迁移。若要按频次和字典序排序，比较器要同时处理两个键。

LeetCode 394 字符串解码

题面模型。嵌套结构需要保存上一层字符串和重复次数。

暴力瓶颈。遇到左括号入栈当前状态，右括号弹出并拼接展开。

边界反例。多位数字、嵌套、空片段、数字后必须跟括号。

变体迁移。递归下降解析更通用，但迭代栈更符合工程栈控制。

LeetCode 416 分割等和子集

题面模型。总和一半是背包容量，每个数只能使用一次。

暴力瓶颈。0-1 背包可达性，一维容量必须倒序。

边界反例。总和为奇数、目标为零、数值较大、复杂度依赖 target。

变体迁移。负数会破坏普通容量模型，需要重新建模。

LeetCode 417 太平洋大西洋水流问题

题面模型。从每个格子向海搜索很慢，反向从海边沿非递减高度搜索。

暴力瓶颈。分别标记能到太平洋和大西洋的格子，交集就是答案。

边界反例。单行单列、平台高度、边界重复入队、方向条件反向。

变体迁移。反向搜索是很多可达性题的重要技巧。

LeetCode 435 无重叠区间

题面模型。保留最多不重叠区间等价于删除最少区间。

暴力瓶颈。按终点升序选择，结束越早给后续留下空间越多。

边界反例。端点相接是否重叠、完全覆盖、相同终点、空列表。

变体迁移。用交换论证说明最早结束的选择不劣于其他第一选择。

LeetCode 438 找到字符串中所有字母异位词

题面模型。固定长度窗口内字符频次等于目标频次即为答案。

暴力瓶颈。右进左出维护计数，窗口长度达到目标长度后检查差异。

边界反例。目标比源串长、重复字符、字符集固定、答案重叠。

变体迁移。维护差异计数可以避免每次比较整个数组。

LeetCode 450 删除二叉搜索树中的节点

题面模型。删除节点后仍要保持 BST 中序有序。

暴力瓶颈。若有两个孩子，用后继或前驱替换当前值，再删除那个后继节点。

边界反例。删除叶子、一个孩子、两个孩子、删除根节点。

变体迁移。工程实现要清楚是移动值还是重连节点。

LeetCode 452 用最少数量的箭引爆气球

题面模型。一支箭的位置可以覆盖所有包含该位置的区间。

暴力瓶颈。按右端点排序，当前箭放在最早结束气球的右端。

边界反例。端点相同、负坐标、区间包含、闭区间边界。

变体迁移。它和无重叠区间是同一类最早结束贪心。

LeetCode 494 目标和

题面模型。正负号选择可代数转换为选若干数凑出特定正和。

暴力瓶颈。若 S 加 target 为奇数或目标绝对值过大，直接无解。

边界反例。零元素会让方案数翻倍、目标为负、容量为零。

变体迁移。也可用哈希 DP 保存所有可能和，但背包转换更高效。

LeetCode 496 下一个更大元素 I

题面模型。第二个数组提供每个值右侧第一个更大值的全局关系。

暴力瓶颈。单调栈预处理值到下一个更大值映射，再回答第一个数组查询。

边界反例。不存在更大值、递减数组、值唯一假设。

变体迁移。若值不唯一，应按下标而不是值建映射。

LeetCode 503 下一个更大元素 II

题面模型。环形数组可以通过遍历两倍下标模拟绕回。

暴力瓶颈。第一遍入栈，第二遍只帮助未解决下标找到答案。

边界反例。全递减、全相等、长度一、取模下标。

变体迁移。不要在第二遍重复入栈，否则可能死循环或重复计算。

LeetCode 518 零钱兑换 II

题面模型。硬币无限使用，问组合数而不是最少数量。

暴力瓶颈。外层遍历硬币、内层金额正序，保证组合按固定硬币顺序统计。

边界反例。amount 为零、无硬币、组合数较大、循环顺序反例。

变体迁移。若外层金额内层硬币，会统计排列数。

LeetCode 543 二叉树直径

题面模型。直径经过某节点时等于左高加右高，但向父亲只能贡献单边高度。

暴力瓶颈。后序遍历同时更新全局直径并返回高度。

边界反例。空树、单节点、链式树、边数还是节点数定义。

变体迁移。最大路径和是同型题，但贡献值和全局更新有负数处理。

LeetCode 547 省份数量

题面模型。城市连接关系形成无向图，省份就是连通分量。

暴力瓶颈。并查集合并所有直接连接的城市，最后统计根数量。

边界反例。邻接矩阵对称、自连接、孤立城市、只扫描上三角。

变体迁移。BFS 也可做；并查集更突出动态合并。

LeetCode 560 和为 K 的子数组

题面模型。当前前缀和减去目标值，就是需要匹配的历史前缀。

暴力瓶颈。哈希表保存前缀和出现次数，先查询再更新。

边界反例。负数、目标为零、从下标零开始的区间、重复前缀。

变体迁移。若要求最长区间，哈希表应保存最早位置而不是次数。

LeetCode 581 最短无序连续子数组

题面模型。要找破坏全局有序性的最左和最右位置。

暴力瓶颈。从左维护历史最大值确定右边界，从右维护历史最小值确定左边界。

边界反例。已经有序、逆序、重复值、局部乱序。

变体迁移。排序比较法更直观但复杂度更高。

LeetCode 621 任务调度器

题面模型。相同任务之间有冷却间隔，核心受最高频任务骨架限制。

暴力瓶颈。可用大顶堆模拟，也可用最高频公式计算最短时间。

边界反例。冷却为零、多个最高频、空闲被其他任务填满。

变体迁移。若任务有不同释放时间或执行时长，就变成更一般的调度问题。

LeetCode 647 回文子串

题面模型。每个回文都有唯一中心或中心间隙。

暴力瓶颈。对每个中心向两边扩展，扩展成功一次就计数一次。

边界反例。空串、全相同字符、偶数回文、奇数回文。

变体迁移。Manacher 算法可线性解决，但中心扩展更适合面试基础。

LeetCode 684 冗余连接

题面模型。树加一条边会形成唯一环。

暴力瓶颈。按顺序加边，若一条边两端已连通，则它就是冗余边。

边界反例。节点编号、最后一条成环边、重复边、并查集初始化大小。

变体迁移。有向冗余连接需要同时处理入度为二和环，复杂度更高。

LeetCode 695 岛屿的最大面积

题面模型。面积就是一个连通块中陆地格子的数量。

暴力瓶颈。每发现新陆地就搜索完整连通块并计数，取最大值。

边界反例。全水、全陆、细长岛、是否允许修改输入。

变体迁移。与岛屿数量相比，只是连通块聚合值不同。

LeetCode 704 二分查找

题面模型。基础题的重点是固定区间语义。

暴力瓶颈。左闭右开写法能统一 lower bound 和存在性检查。

边界反例。空数组、长度一、目标在边界、目标不存在。

变体迁移。所有复杂二分都应该能退回到这道题的边界不变量。

LeetCode 714 买卖股票含手续费

题面模型。手续费改变交易收益，仍可用持有和不持有状态。

暴力瓶颈。卖出时扣费或买入时扣费都可以，但不要重复扣。

边界反例。手续费为零、价格震荡、小利润不足手续费。

变体迁移。这是状态机比局部贪心更稳的例子。

LeetCode 739 每日温度

题面模型。每一天等待右侧第一个更高温度。

暴力瓶颈。单调递减栈保存尚未找到答案的下标。

边界反例。没有更高温、相等温度、递增、递减。

变体迁移。下一个更大元素与它同型，只是输出值或距离不同。

LeetCode 743 网络延迟时间

题面模型。有向正权图从起点到所有节点的最短路最大值。

暴力瓶颈。Dijkstra 求每个节点最短距离，最后取最大，若有无穷则不可达。

边界反例。节点编号从一开始、不可达、重边、过期堆元素。

变体迁移。边权相同才可用普通 BFS。

LeetCode 752 打开转盘锁

题面模型。四位密码是状态，每次转动一位形成边。

暴力瓶颈。BFS 从起点扩展，死亡状态不能入队，访问状态避免重复。

边界反例。起点死亡、目标就是起点、数字环绕、死亡集合很大。

变体迁移。双向 BFS 能明显减少状态展开。

LeetCode 763 划分字母区间

题面模型。每个片段必须覆盖其中所有字符的最后出现位置。

暴力瓶颈。扫描维护当前片段最远右端，到达右端即可切分。

边界反例。单字符、所有字符相同、字符多次跨段。

变体迁移。这是最早合法切分贪心，能最大化片段数量。

LeetCode 875 爱吃香蕉的珂珂

题面模型。速度越大，总耗时越小，答案具有单调可行性。

暴力瓶颈。二分速度，检查函数用向上取整累加小时数。

边界反例。速度下界、堆很大、h 等于堆数、乘法加法溢出。

变体迁移。船运包裹和分割数组都是同类最小可行上限。

LeetCode 973 最接近原点的 K 个点

题面模型。距离只需比较平方，避免开方误差和成本。

暴力瓶颈。大小为 K 的大顶堆保存当前最近 K 个点中最远者。

边界反例。K 为一、K 为全部、距离相同、坐标平方溢出。

变体迁移。快速选择可平均线性，但堆更适合数据流。

LeetCode 994 腐烂的橘子

题面模型。所有初始腐烂橘子同时作为 BFS 源点。

暴力瓶颈。按层扩散，分钟数等于最后一个新鲜橘子被首次访问的层数。

边界反例。没有新鲜橘子、没有腐烂源、隔离新鲜橘子、空格阻断。

变体迁移。多源 BFS 也适用于最近零距离、地图扩散等题。

LeetCode 1011 在 D 天内送达包裹

题面模型。容量越大，需要天数越少，存在最小可行容量。

暴力瓶颈。下界是最大包裹，上界是总重量，检查函数按顺序贪心装船。

边界反例。D 为一、D 等于包裹数、单个包裹超过猜测容量。

变体迁移。它和分割数组最大值都是连续分段最大和模型。

LeetCode 1143 最长公共子序列

题面模型。状态表示两个前缀的最长公共子序列长度。

暴力瓶颈。末尾相同取左上加一，不同取上方和左方最大。

边界反例。空串、完全相同、无公共字符、子序列不是子串。

变体迁移。若要求还原序列，需要从表右下角反向追踪选择。

LeetCode 1288 删除被覆盖区间

题面模型。被覆盖要求另一区间起点不晚且终点不早。

暴力瓶颈。按起点升序、终点降序排序，扫描维护最大右端。

边界反例。同起点区间、完全覆盖、无覆盖、端点相等。

变体迁移。终点降序是关键，否则同起点小区间会先出现造成误判。

LeetCode 1631 最小体力消耗路径

题面模型。路径代价是沿途最大高度差，不是边权和。

暴力瓶颈。Dijkstra 的松弛改成取当前代价和新边代价的最大值。

边界反例。单格、平坦网格、必须绕路、多个相同代价。

变体迁移。也可二分体力上限后 BFS 检查可达。

逐题推导与实验工作坊

这一节把前面的题号索引继续展开。每道题都从自己的题面模型出发，先说明慢在哪里，再说明状态怎样保存、哪些候选被安全排除、哪些边界会打破错误实现。题型共性只作为背景，真正要读的是每道题自己的瓶颈和反例。

数组与原地维护工作坊

先写一个不会漏答案的枚举或辅助数组版本，再观察哪些位置已经被前缀、后缀、慢指针或边界确定。优化时把数组划成语义清楚的区域，每次循环只扩大已确认区域。

LeetCode 5 最长回文子串

先从位置关系入手。本题的单点模型是回文围绕中心对称，中心可以是字符也可以是两个字符之间。朴素版可以为每个位置重新寻找最终状态，或者用辅助数组把答案先做出来；它虽然慢，却能说明哪些位置必须被覆盖。本题真正浪费的部分是中心扩展避免枚举子串后再逐个检查；每次扩展只比较新增左右边界，后面的优化只能消掉这部分重复，不能改变输出语义。

优化时把数组切成几段：已经确认的前缀、未知区、尚未处理的后缀，或者左侧贡献和右侧贡献。本题落到代码上就是中心扩展避免枚举子串后再逐个检查；每次扩展只比较新增左右边界。循环中每改一个下标，都要能说出哪一段扩大了，哪一段缩小了。放到“数组与原地维护”这条主线里看，关键原则是：优化要把数组切成若干语义明确的区间：已经处理好的前缀、未知区、已经确定的后缀，或者左侧贡献和右侧贡献。双指针、前后缀数组、原地交换和稳定写入，本质都是让每个元素只进入少数几次状态转移。

正确性可以按区间不变量证明：已处理部分满足题意，未知部分还没承诺，循环每次只把一个元素移入正确区域。对本题来说，回文围绕中心对称，中心可以是字符也可以是两个字符之间给出了答案范围，中心扩展避免枚举子串后再逐个检查；每次扩展只比较新增左右边界给出了推进方式。这道题的边界不能留到最后补丁式处理，实验时把偶数长度回文、单字符、全相同字符、多个同长答案列成表，再打印关键下标和有效区间。若某个元素被写入后又被未来步骤破坏，说明区间不变量没有成立。

C++ 实现上，C++ 中优先使用 `std::vector` 和清楚的整型下标；涉及乘积、面积、和或距离时提前考虑 `long long`。如果题目要求原地修改，返回长度、容器容量和有效区间要分开讲。如果追问变成“若要求回文子序列，应转成区间 DP，不能用中心扩展”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 26 删除有序数组重复项

先从位置关系入手。本题的单点模型是有序数组让重复元素相邻，慢指针表示下一个唯一值写入位置。朴素版可以为每个位置重新寻找最终状态，或者用辅助数组把答案先做出来；它虽然慢，却能

说明哪些位置必须被覆盖。本题真正浪费的部分是快指针扫描，只有发现新值才写入慢指针并推进，后面的优化只能消掉这部分重复，不能改变输出语义。

优化时把数组切成几段：已经确认的前缀、未知区、尚未处理的后缀，或者左侧贡献和右侧贡献。本题落到代码上就是快指针扫描，只有发现新值才写入慢指针并推进。循环中每改一个下标，都要能说出哪一段扩大了，哪一段缩小了。放到“数组与原地维护”这条主线里看，关键原则是：优化要把数组切成若干语义明确的区间：已经处理好的前缀、未知区、已经确定的后缀，或者左侧贡献和右侧贡献。双指针、前后缀数组、原地交换和稳定写入，本质都是让每个元素只进入少数几次状态转移。

正确性可以按区间不变量证明：已处理部分满足题意，未知部分还没承诺，循环每次只把一个元素移入正确区域。对本题来说，有序数组让重复元素相邻，慢指针表示下一个唯一值写入位置给出了答案范围，快指针扫描，只有发现新值才写入慢指针并推进给出了推进方式。这道题的边界不能留到最后补丁式处理，实验时把空数组、全重复、无重复、返回长度不等于物理容量列成表，再打印关键下标和有效区间。若某个元素被写入后又被未来步骤破坏，说明区间不变量没有成立。

C++ 实现上，C++ 中优先使用 `std::vector` 和清楚的整型下标；涉及乘积、面积、和或距离时提前考虑 `long long`。如果题目要求原地修改，返回长度、容器容量和有效区间要分开讲。如果追问变成“若允许每个元素最多保留两次，只需改变写入条件为和前两个比较”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 27 移除元素

先从位置关系入手。本题的单点模型是原地过滤，慢指针左侧始终是不等于目标值的稳定前缀。朴素版可以为每个位置重新寻找最终状态，或者用辅助数组把答案先做出来；它虽然慢，却能说明哪些位置必须被覆盖。本题真正浪费的部分是保持顺序用快慢指针；不保持顺序可用左右交换减少写入，后面的优化只能消掉这部分重复，不能改变输出语义。

优化时把数组切成几段：已经确认的前缀、未知区、尚未处理的后缀，或者左侧贡献和右侧贡献。本题落到代码上就是保持顺序用快慢指针；不保持顺序可用左右交换减少写入。循环中每改一个下标，都要能说出哪一段扩大了，哪一段缩小了。放到“数组与原地维护”这条主线里看，关键原则是：优化要把数组切成若干语义明确的区间：已经处理好的前缀、未知区、已经确定的后缀，或者左侧贡献和右侧贡献。双指针、前后缀数组、原地交换和稳定写入，本质都是让每个元素只进入少数几次状态转移。

正确性可以按区间不变量证明：已处理部分满足题意，未知部分还没承诺，循环每次只把一个元素移入正确区域。对本题来说，原地过滤，慢指针左侧始终是不等于目标值的稳定前缀给出了答案范围，保持顺序用快慢指针；不保持顺序可用左右交换减少写入给出了推进方式。这道题的边界不能留到最后补丁式处理，实验时把所有元素都被移除、没有元素被移除、目标值在尾部密集列成表，再打印关键下标和有效区间。若某个元素被写入后又被未来步骤破坏，说明区间不变量没有成立。

C++ 实现上，C++ 中优先使用 `std::vector` 和清楚的整型下标；涉及乘积、面积、和或距离时提前考虑 `long long`。如果题目要求原地修改，返回长度、容器容量和有效区间要分开讲。如果追问变成“工程里要先确认是否要求稳定顺序，不同要求对应不同写法”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 31 下一个排列

先从位置关系入手。本题的单点模型是字典序结构由最长非递增后缀决定，枢轴是后缀前一个位置。朴素版可以为每个位置重新寻找最终状态，或者用辅助数组把答案先做出来；它虽然慢，却能说明哪些位置必须被覆盖。本题真正浪费的部分是从右找第一个上升点，再从右找刚好更大的元素交换，最后反转后缀，后面的优化只能消掉这部分重复，不能改变输出语义。

优化时把数组切成几段：已经确认的前缀、未知区、尚未处理的后缀，或者左侧贡献和右侧贡献。本题落到代码上就是从右找第一个上升点，再从右找刚好更大的元素交换，最后反转后缀。循环中每改一个下标，都要能说出哪一段扩大了，哪一段缩小了。放到“数组与原地维护”这条主线里看，关键原则是：优化要把数组切成若干语义明确的区间：已经处理好的前缀、未知区、已经确定的后缀，或者左侧贡献和右侧贡献。双指针、前后缀数组、原地交换和稳定写入，本质都是让每个元素只进入少数几次状态转移。

正确性可以按区间不变量证明：已处理部分满足题意，未知部分还没承诺，循环每次只把一个

元素移入正确区域。对本题来说，字典序结构由最长非递增后缀决定，枢轴是后缀前一个位置给出了答案范围，从右找第一个上升点，再从右找刚好更大的元素交换，最后反转后缀给出了推进方式。这道题的边界不能留到最后补丁式处理，实验时把完全降序、重复数字、只有一个元素、交换后后缀必须最小化列成表，再打印关键下标和有效区间。若某个元素被写入后又被未来步骤破坏，说明区间不变量没有成立。

C++ 实现上，C++ 中优先使用 `std::vector` 和清楚的整型下标；涉及乘积、面积、和或距离时提前考虑 `long long`。如果题目要求原地修改，返回长度、容器容量和有效区间要分开讲。如果追问变成“若要求上一个排列，比较方向和后缀处理对称变化”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 54 螺旋矩阵

先从位置关系入手。本题的单点模型是四个边界收缩模拟一圈一圈访问。朴素版可以为每个位置重新寻找最终状态，或者用辅助数组把答案先做出来；它虽然慢，却能说明哪些位置必须被覆盖。本题真正浪费的部分是每走完一条边就收缩对应边界，并在走反方向前检查是否仍合法，后面的优化只能消掉这部分重复，不能改变输出语义。

优化时把数组切成几段：已经确认的前缀、未知区、尚未处理的后缀，或者左侧贡献和右侧贡献。本题落到代码上就是每走完一条边就收缩对应边界，并在走反方向前检查是否仍合法。循环中每改一个下标，都要能说出哪一段扩大了，哪一段缩小了。放到“数组与原地维护”这条主线里看，关键原则是：优化要把数组切成若干语义明确的区间：已经处理好的前缀、未知区、已经确定的后缀，或者左侧贡献和右侧贡献。双指针、前后缀数组、原地交换和稳定写入，本质都是让每个元素只进入少数几次状态转移。

正确性可以按区间不变量证明：已处理部分满足题意，未知部分还没承诺，循环每次只把一个元素移入正确区域。对本题来说，四个边界收缩模拟一圈一圈访问给出了答案范围，每走完一条边就收缩对应边界，并在走反方向前检查是否仍合法给出了推进方式。这道题的边界不能留到最后补丁式处理，实验时把单行、单列、空矩阵、最后一圈只剩一个元素列成表，再打印关键下标和有效区间。若某个元素被写入后又被未来步骤破坏，说明区间不变量没有成立。

C++ 实现上，C++ 中优先使用 `std::vector` 和清楚的整型下标；涉及乘积、面积、和或距离时提前考虑 `long long`。如果题目要求原地修改，返回长度、容器容量和有效区间要分开讲。如果追问变成“若改成生成矩阵，边界逻辑相同，只是读变成写”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 75 颜色分类

先从位置关系入手。本题的单点模型是三指针维护零区、一号区、未知区和二区。朴素版可以为每个位置重新寻找最终状态，或者用辅助数组把答案先做出来；它虽然慢，却能说明哪些位置必须被覆盖。本题真正浪费的部分是遇到二时只移动右边界，不移动扫描指针，因为换回来的元素未知，后面的优化只能消掉这部分重复，不能改变输出语义。

优化时把数组切成几段：已经确认的前缀、未知区、尚未处理的后缀，或者左侧贡献和右侧贡献。本题落到代码上就是遇到二时只移动右边界，不移动扫描指针，因为换回来的元素未知。循环中每改一个下标，都要能说出哪一段扩大了，哪一段缩小了。放到“数组与原地维护”这条主线里看，关键原则是：优化要把数组切成若干语义明确的区间：已经处理好的前缀、未知区、已经确定的后缀，或者左侧贡献和右侧贡献。双指针、前后缀数组、原地交换和稳定写入，本质都是让每个元素只进入少数几次状态转移。

正确性可以按区间不变量证明：已处理部分满足题意，未知部分还没承诺，循环每次只把一个元素移入正确区域。对本题来说，三指针维护零区、一号区、未知区和二区给出了答案范围，遇到二时只移动右边界，不移动扫描指针，因为换回来的元素未知给出了推进方式。这道题的边界不能留到最后补丁式处理，实验时把全零、全二、已经有序、逆序、交换后漏检查列成表，再打印关键下标和有效区间。若某个元素被写入后又被未来步骤破坏，说明区间不变量没有成立。

C++ 实现上，C++ 中优先使用 `std::vector` 和清楚的整型下标；涉及乘积、面积、和或距离时提前考虑 `long long`。如果题目要求原地修改，返回长度、容器容量和有效区间要分开讲。如果追问变成“这是荷兰国旗问题，能推广到三类分区的原地维护”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 88 合并两个有序数组

先从位置关系入手。本题的单点模型是第一个数组尾部有空位，适合从后往前写入最终最大值。朴素版可以为每个位置重新寻找最终状态，或者用辅助数组把答案先做出来；它虽然慢，却能说明哪些位置必须被覆盖。本题真正浪费的部分是逆向填充避免覆盖尚未读取的 `nums1` 元素，后面的优化只能消掉这部分重复，不能改变输出语义。

优化时把数组切成几段：已经确认的前缀、未知区、尚未处理的后缀，或者左侧贡献和右侧贡献。本题落到代码上就是逆向填充避免覆盖尚未读取的 `nums1` 元素。循环中每改一个下标，都要能说出哪一段扩大了，哪一段缩小了。放到“数组与原地维护”这条主线里看，关键原则是：优化要把数组切成若干语义明确的区间：已经处理好的前缀、未知区、已经确定的后缀，或者左侧贡献和右侧贡献。双指针、前后缀数组、原地交换和稳定写入，本质都是让每个元素只进入少数几次状态转移。

正确性可以按区间不变量证明：已处理部分满足题意，未知部分还没承诺，循环每次只把一个元素移入正确区域。对本题来说，第一个数组尾部有空位，适合从后往前写入最终最大值给出了答案范围，逆向填充避免覆盖尚未读取的 `nums1` 元素给出了推进方式。这道题的边界不能留到最后补丁式处理，实验时把第二个数组为空、第一个有效元素为空、重复值、尾部剩余列成表，再打印关键下标和有效区间。若某个元素被写入后又被未来步骤破坏，说明区间不变量没有成立。

C++ 实现上，C++ 中优先使用 `std::vector` 和清楚的整型下标；涉及乘积、面积、和或距离时提前考虑 `long long`。如果题目要求原地修改，返回长度、容器容量和有效区间要分开讲。如果追问变成“若没有额外空位，只能新开数组或使用更复杂原地归并”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 `value` 语义，还是换成另一类结构。

LeetCode 189 轮转数组

先从位置关系入手。本题的单点模型是右移 k 位等价于把数组切成两段后换序。朴素版可以为每个位置重新寻找最终状态，或者用辅助数组把答案先做出来；它虽然慢，却能说明哪些位置必须被覆盖。本题真正浪费的部分是三次反转能原地完成：整体、前 k 、后 n 减 k ，后面的优化只能消掉这部分重复，不能改变输出语义。

优化时把数组切成几段：已经确认的前缀、未知区、尚未处理的后缀，或者左侧贡献和右侧贡献。本题落到代码上就是三次反转能原地完成：整体、前 k 、后 n 减 k 。循环中每改一个下标，都要能说出哪一段扩大了，哪一段缩小了。放到“数组与原地维护”这条主线里看，关键原则是：优化要把数组切成若干语义明确的区间：已经处理好的前缀、未知区、已经确定的后缀，或者左侧贡献和右侧贡献。双指针、前后缀数组、原地交换和稳定写入，本质都是让每个元素只进入少数几次状态转移。

正确性可以按区间不变量证明：已处理部分满足题意，未知部分还没承诺，循环每次只把一个元素移入正确区域。对本题来说，右移 k 位等价于把数组切成两段后换序给出了答案范围，三次反转能原地完成：整体、前 k 、后 n 减 k 给出了推进方式。这道题的边界不能留到最后补丁式处理，实验时把 k 为零、 k 大于 n 、空数组、单元元素列成表，再打印关键下标和有效区间。若某个元素被写入后又被未来步骤破坏，说明区间不变量没有成立。

C++ 实现上，C++ 中优先使用 `std::vector` 和清楚的整型下标；涉及乘积、面积、和或距离时提前考虑 `long long`。如果题目要求原地修改，返回长度、容器容量和有效区间要分开讲。如果追问变成“环状替换也可做，但要处理最大公约数个循环”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 `value` 语义，还是换成另一类结构。

LeetCode 238 除自身以外数组的乘积

先从位置关系入手。本题的单点模型是不能用除法时，答案由左侧乘积和右侧乘积组成。朴素版可以为每个位置重新寻找最终状态，或者用辅助数组把答案先做出来；它虽然慢，却能说明哪些位置必须被覆盖。本题真正浪费的部分是先写入左侧前缀积，再从右向左乘右侧后缀积，后面的优化只能消掉这部分重复，不能改变输出语义。

优化时把数组切成几段：已经确认的前缀、未知区、尚未处理的后缀，或者左侧贡献和右侧贡献。本题落到代码上就是先写入左侧前缀积，再从右向左乘右侧后缀积。循环中每改一个下标，都要能说出哪一段扩大了，哪一段缩小了。放到“数组与原地维护”这条主线里看，关键原则是：优化要把数组切成若干语义明确的区间：已经处理好的前缀、未知区、已经确定的后缀，或者左侧贡献和右侧贡献。双指针、前后缀数组、原地交换和稳定写入，本质都是让每个元素只进入少数几次状

态转移。

正确性可以按区间不变量证明：已处理部分满足题意，未知部分还没承诺，循环每次只把一个元素移入正确区域。对本题来说，不能用除法时，答案由左侧乘积和右侧乘积组成给出了答案范围，先写入左侧前缀积，再从右向左乘右侧后缀积给出了推进方式。这道题的边界不能留到最后补丁式处理，实验时把一个零、多个零、负数、乘积溢出列成表，再打印关键下标和有效区间。若某个元素被写入后又被未来步骤破坏，说明区间不变量没有成立。

C++ 实现上，C++ 中优先使用 `std::vector` 和清楚的整型下标；涉及乘积、面积、和或距离时提前考虑 `long long`。如果题目要求原地修改，返回长度、容器容量和有效区间要分开讲。如果追问变成“这个题展示输出数组可作为状态存储的一部分”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 `value` 语义，还是换成另一类结构。

LeetCode 283 移动零

先从位置关系入手。本题的单点模型是稳定保留非零元素顺序，把零集中到尾部。朴素版可以为每个位置重新寻找最终状态，或者用辅助数组把答案先做出来；它虽然慢，却能说明哪些位置必须被覆盖。本题真正浪费的部分是慢指针写入下一个非零位置，最后补零或用交换，后面的优化只能消掉这部分重复，不能改变输出语义。

优化时把数组切成几段：已经确认的前缀、未知区、尚未处理的后缀，或者左侧贡献和右侧贡献。本题落到代码上就是慢指针写入下一个非零位置，最后补零或用交换。循环中每改一个下标，都要能说出哪一段扩大了，哪一段缩小了。放到“数组与原地维护”这条主线里看，关键原则是：优化要把数组切成若干语义明确的区间：已经处理好的前缀、未知区、已经确定的后缀，或者左侧贡献和右侧贡献。双指针、前后缀数组、原地交换和稳定写入，本质都是让每个元素只进入少数几次状态转移。

正确性可以按区间不变量证明：已处理部分满足题意，未知部分还没承诺，循环每次只把一个元素移入正确区域。对本题来说，稳定保留非零元素顺序，把零集中到尾部给出了答案范围，慢指针写入下一个非零位置，最后补零或用交换给出了推进方式。这道题的边界不能留到最后补丁式处理，实验时把全零、无零、零在开头结尾、稳定顺序要求列成表，再打印关键下标和有效区间。若某个元素被写入后又被未来步骤破坏，说明区间不变量没有成立。

C++ 实现上，C++ 中优先使用 `std::vector` 和清楚的整型下标；涉及乘积、面积、和或距离时提前考虑 `long long`。如果题目要求原地修改，返回长度、容器容量和有效区间要分开讲。如果追问变成“若目标是移动指定值，模型与移除元素相同”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 `value` 语义，还是换成另一类结构。

LeetCode 581 最短无序连续子数组

先从位置关系入手。本题的单点模型是要找破坏全局有序性的最左和最右位置。朴素版可以为每个位置重新寻找最终状态，或者用辅助数组把答案先做出来；它虽然慢，却能说明哪些位置必须被覆盖。本题真正浪费的部分是从左维护历史最大值确定右边界，从右维护历史最小值确定左边界，后面的优化只能消掉这部分重复，不能改变输出语义。

优化时把数组切成几段：已经确认的前缀、未知区、尚未处理的后缀，或者左侧贡献和右侧贡献。本题落到代码上就是从左维护历史最大值确定右边界，从右维护历史最小值确定左边界。循环中每改一个下标，都要能说出哪一段扩大了，哪一段缩小了。放到“数组与原地维护”这条主线里看，关键原则是：优化要把数组切成若干语义明确的区间：已经处理好的前缀、未知区、已经确定的后缀，或者左侧贡献和右侧贡献。双指针、前后缀数组、原地交换和稳定写入，本质都是让每个元素只进入少数几次状态转移。

正确性可以按区间不变量证明：已处理部分满足题意，未知部分还没承诺，循环每次只把一个元素移入正确区域。对本题来说，要找破坏全局有序性的最左和最右位置给出了答案范围，从左维护历史最大值确定右边界，从右维护历史最小值确定左边界给出了推进方式。这道题的边界不能留到最后补丁式处理，实验时把已经有序、逆序、重复值、局部乱序列成表，再打印关键下标和有效区间。若某个元素被写入后又被未来步骤破坏，说明区间不变量没有成立。

C++ 实现上，C++ 中优先使用 `std::vector` 和清楚的整型下标；涉及乘积、面积、和或距离时提前考虑 `long long`。如果题目要求原地修改，返回长度、容器容量和有效区间要分开讲。如果追问变成“排序比较法更直观但复杂度更高”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 `value` 语义，还是换成另一类结构。

LeetCode 647 回文子串

先从位置关系入手。本题的单点模型是每个回文都有唯一中心或中心间隙。朴素版可以为每个位置重新寻找最终状态，或者用辅助数组把答案先做出来；它虽然慢，却能说明哪些位置必须被覆盖。本题真正浪费的部分是对每个中心向两边扩展，扩展成功一次就计数一次，后面的优化只能消掉这部分重复，不能改变输出语义。

优化时把数组切成几段：已经确认的前缀、未知区、尚未处理的后缀，或者左侧贡献和右侧贡献。本题落到代码上就是对每个中心向两边扩展，扩展成功一次就计数一次。循环中每改一个下标，都要能说出哪一段扩大了，哪一段缩小了。放到“数组与原地维护”这条主线里看，关键原则是：优化要把数组切成若干语义明确的区间：已经处理好的前缀、未知区、已经确定的后缀，或者左侧贡献和右侧贡献。双指针、前后缀数组、原地交换和稳定写入，本质都是让每个元素只进入少数几次状态转移。

正确性可以按区间不变量证明：已处理部分满足题意，未知部分还没承诺，循环每次只把一个元素移入正确区域。对本题来说，每个回文都有唯一中心或中心间隙给出了答案范围，对每个中心向两边扩展，扩展成功一次就计数一次给出了推进方式。这道题的边界不能留到最后补丁式处理，实验时把空串、全相同字符、偶数回文、奇数回文列成表，再打印关键下标和有效区间。若某个元素被写入后又被未来步骤破坏，说明区间不变量没有成立。

C++ 实现上，C++ 中优先使用 `std::vector` 和清楚的整型下标；涉及乘积、面积、和或距离时提前考虑 `long long`。如果题目要求原地修改，返回长度、容器容量和有效区间要分开讲。如果追问变成“Manacher 算法可线性解决，但中心扩展更适合面试基础”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 `value` 语义，还是换成另一类结构。

滑动窗口与双指针工作坊

先枚举所有窗口，再确认右端扩展和左端收缩是否具有单调性。只有窗口状态可以增量维护，且移动左端不会漏掉未来更优答案时，滑动窗口才成立。

LeetCode 3 无重复字符的最长子串

先枚举所有连续窗口，明确左端、右端和窗口内容怎样决定答案。本题的模型是窗口保存当前无重复子串，右端每次加入一个字符；暴力版会反复统计相邻窗口中相同的内容。慢点具体落在用上次出现位置直接跳过无效左端，左边界只能向右不能回退，所以后面要证明左端或右端移动时不会漏掉可能答案。

窗口题只在条件有单调性或窗口长度固定时成立。本题用上次出现位置直接跳过无效左端，左边界只能向右不能回退维护窗口，含义是右端只带来一个新字符或新元素，左端只移走一个旧元素；如果输入条件改变导致状态不再单调，就必须换方法。放到“滑动窗口与双指针”这条主线里看，关键原则是：滑动窗口成立必须有单调性或固定长度边界。右端扩展让某个条件单调变好或变坏，左端收缩可以恢复合法性或缩短答案。若数组含负数、状态会来回震荡，就不能硬套窗口。

证明重点是排除左端。若某个左端被移过，它对应的窗口已经被完整检查，或继续保留只会让窗口更长、更不合法。本题的窗口保存当前无重复子串，右端每次加入一个字符和用上次出现位置直接跳过无效左端，左边界只能向右不能回退必须一起说明这一点。这道题的边界不能留到最后补丁式处理，实验时重点跑空串、全相同字符、重复字符出现在窗口左侧之外、扩展字符集，并打印左右端、窗口计数和答案。若左端发生回退，或者窗口失去合法性后仍更新答案，就能很快定位错误。

C++ 实现上，窗口计数若字符集固定，可以用 `std::array<int, 128>` 或 26 长度数组；字符集不固定再用哈希表。左闭右闭和左闭右开选一种坚持到底，避免窗口长度和删除位置错位。如果追问变成“若要求恰好包含 k 种字符，窗口条件从无重复变成种类计数”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 `value` 语义，还是换成另一类结构。

LeetCode 11 盛最多水的容器

先枚举所有连续窗口，明确左端、右端和窗口内容怎样决定答案。本题的模型是左右指针代表当前选取的两条边，面积由短板和宽度共同决定；暴力版会反复统计相邻窗口中相同的内容。慢点具体落在每次移动短板，因为移动长板只会缩小宽度且不会提高短板，所以后面要证明左端或右端移动时不会漏掉可能答案。

窗口题只在条件有单调性或窗口长度固定时成立。本题用每次移动短板，因为移动长板只会缩小宽度且不会提高短板维护窗口，含义是右端只带来一个新字符或新元素，左端只移走一个旧元素；

如果输入条件改变导致状态不再单调，就必须换方法。放到“滑动窗口与双指针”这条主线里看，关键原则是：滑动窗口成立必须有单调性或固定长度边界。右端扩展让某个条件单调变好或变坏，左端收缩可以恢复合法性或缩短答案。若数组含负数、状态会来回震荡，就不能硬套窗口。

证明重点是排除左端。若某个左端被移过，它对应的窗口已经被完整检查，或继续保留只会让窗口更长、更不合法。本题的左右指针代表当前选取的两条边，面积由短板和宽度共同决定和每次移动短板，因为移动长板只会缩小宽度且不会提高短板必须一起说明这一点。这道题的边界不能留到最后补丁式处理，实验时重点跑两端高度相等、数组长度为二、面积乘法可能溢出，并打印左右端、窗口计数和答案。若左端发生回退，或者窗口失去合法性后仍更新答案，就能很快定位错误。

C++ 实现上，窗口计数若字符集固定，可以用 `std::array<int, 128>` 或 26 长度数组；字符集不固定再用哈希表。左闭右闭和左闭右开选一种坚持到底，避免窗口长度和删除位置错位。如果追问变成“接雨水计算每个位置贡献，不能把本题双指针证明直接搬过去”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 15 三数之和

先枚举所有连续窗口，明确左端、右端和窗口内容怎样决定答案。本题的模型是排序后固定第一个数，剩余两数转成有序双指针；暴力版会反复统计相邻窗口中相同的内容。慢点具体落在和太小移动左指针，和太大移动右指针；固定值和左右值都要跳过重复，所以后面要证明左端或右端移动时不会漏掉可能答案。

窗口题只在条件有单调性或窗口长度固定时成立。本题用和太小移动左指针，和太大移动右指针；固定值和左右值都要跳过重复维护窗口，含义是右端只带来一个新字符或新元素，左端只移走一个旧元素；如果输入条件改变导致状态不再单调，就必须换方法。放到“滑动窗口与双指针”这条主线里看，关键原则是：滑动窗口成立必须有单调性或固定长度边界。右端扩展让某个条件单调变好或变坏，左端收缩可以恢复合法性或缩短答案。若数组含负数、状态会来回震荡，就不能硬套窗口。

证明重点是排除左端。若某个左端被移过，它对应的窗口已经被完整检查，或继续保留只会让窗口更长、更不合法。本题的排序后固定第一个数，剩余两数转成有序双指针和和太小移动左指针，和太大移动右指针；固定值和左右值都要跳过重复必须一起说明这一点。这道题的边界不能留到最后补丁式处理，实验时重点跑全零、重复元素很多、没有答案、结果顺序不要求但组合不能重复，并打印左右端、窗口计数和答案。若左端发生回退，或者窗口失去合法性后仍更新答案，就能很快定位错误。

C++ 实现上，窗口计数若字符集固定，可以用 `std::array<int, 128>` 或 26 长度数组；字符集不固定再用哈希表。左闭右闭和左闭右开选一种坚持到底，避免窗口长度和删除位置错位。如果追问变成“四数之和再外层固定一个数，并用 long long 防止目标和溢出”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 16 最接近的三数之和

先枚举所有连续窗口，明确左端、右端和窗口内容怎样决定答案。本题的模型是排序后固定一数，双指针寻找最接近目标的两数和；暴力版会反复统计相邻窗口中相同的内容。慢点具体落在每次根据当前和与目标的大小移动指针，同时更新最小绝对差，所以后面要证明左端或右端移动时不会漏掉可能答案。

窗口题只在条件有单调性或窗口长度固定时成立。本题用每次根据当前和与目标的大小移动指针，同时更新最小绝对差维护窗口，含义是右端只带来一个新字符或新元素，左端只移走一个旧元素；如果输入条件改变导致状态不再单调，就必须换方法。放到“滑动窗口与双指针”这条主线里看，关键原则是：滑动窗口成立必须有单调性或固定长度边界。右端扩展让某个条件单调变好或变坏，左端收缩可以恢复合法性或缩短答案。若数组含负数、状态会来回震荡，就不能硬套窗口。

证明重点是排除左端。若某个左端被移过，它对应的窗口已经被完整检查，或继续保留只会让窗口更长、更不合法。本题的排序后固定一数，双指针寻找最接近目标的两数和和每次根据当前和与目标的大小移动指针，同时更新最小绝对差必须一起说明这一点。这道题的边界不能留到最后补丁式处理，实验时重点跑差值相等、负数、目标很大、三数和溢出，并打印左右端、窗口计数和答案。若左端发生回退，或者窗口失去合法性后仍更新答案，就能很快定位错误。

C++ 实现上，窗口计数若字符集固定，可以用 `std::array<int, 128>` 或 26 长度数组；字符集不固定再用哈希表。左闭右闭和左闭右开选一种坚持到底，避免窗口长度和删除位置错位。如

果追问变成“若要求返回所有最接近组合，需要记录同差值并去重”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 18 四数之和

先枚举所有连续窗口，明确左端、右端和窗口内容怎样决定答案。本题的模型是两层固定加一层双指针，关键是剪枝和去重；暴力版会反复统计相邻窗口中相同的内容。慢点具体落在排序提供单调性，外层可用最小可能和和最大可能和提前跳过，所以后面要证明左端或右端移动时不会漏掉可能答案。

窗口题只在条件有单调性或窗口长度固定时成立。本题用排序提供单调性，外层可用最小可能和和最大可能和提前跳过维护窗口，含义是右端只带来一个新字符或新元素，左端只移走一个旧元素；如果输入条件改变导致状态不再单调，就必须换方法。放到“滑动窗口与双指针”这条主线里看，关键原则是：滑动窗口成立必须有单调性或固定长度边界。右端扩展让某个条件单调变好或变坏，左端收缩可以恢复合法性或缩短答案。若数组含负数、状态会来回震荡，就不能硬套窗口。

证明重点是排除左端。若某个左端被移过，它对应的窗口已经被完整检查，或继续保留只会让窗口更长、更不合法。本题的两层固定加一层双指针，关键是剪枝和去重和排序提供单调性，外层可用最小可能和和最大可能和提前跳过必须一起说明这一点。这道题的边界不能留到最后补丁式处理，实验时重点跑重复元素、目标超出 int、答案很多导致输出空间主导复杂度，并打印左右端、窗口计数和答案。若左端发生回退，或者窗口失去合法性后仍更新答案，就能很快定位错误。

C++ 实现上，窗口计数若字符集固定，可以用 `std::array<int, 128>` 或 26 长度数组；字符集不固定再用哈希表。左闭右闭和左闭右开选一种坚持到底，避免窗口长度和删除位置错位。如果追问变成“k 数之和可以递归降维，但工程实现要控制重复和边界”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 76 最小覆盖子串

先枚举所有连续窗口，明确左端、右端和窗口内容怎样决定答案。本题的模型是窗口内字符计数必须覆盖目标计数，满足后尽量收缩左端；暴力版会反复统计相邻窗口中相同的内容。慢点具体落在用 formed 记录满足需求的字符种类，避免每次全量比较，所以后面要证明左端或右端移动时不会漏掉可能答案。

窗口题只在条件有单调性或窗口长度固定时成立。本题用 formed 记录满足需求的字符种类，避免每次全量比较维护窗口，含义是右端只带来一个新字符或新元素，左端只移走一个旧元素；如果输入条件改变导致状态不再单调，就必须换方法。放到“滑动窗口与双指针”这条主线里看，关键原则是：滑动窗口成立必须有单调性或固定长度边界。右端扩展让某个条件单调变好或变坏，左端收缩可以恢复合法性或缩短答案。若数组含负数、状态会来回震荡，就不能硬套窗口。

证明重点是排除左端。若某个左端被移过，它对应的窗口已经被完整检查，或继续保留只会让窗口更长、更不合法。本题的窗口内字符计数必须覆盖目标计数，满足后尽量收缩左端和用 formed 记录满足需求的字符种类，避免每次全量比较必须一起说明这一点。这道题的边界不能留到最后补丁式处理，实验时重点跑目标有重复字符、源串缺字符、最小窗口在开头或结尾、大小写，并打印左右端、窗口计数和答案。若左端发生回退，或者窗口失去合法性后仍更新答案，就能很快定位错误。

C++ 实现上，窗口计数若字符集固定，可以用 `std::array<int, 128>` 或 26 长度数组；字符集不固定再用哈希表。左闭右闭和左闭右开选一种坚持到底，避免窗口长度和删除位置错位。如果追问变成“若字符集固定，数组计数更快；若是 Unicode，哈希表更通用”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 209 长度最小的子数组

先枚举所有连续窗口，明确左端、右端和窗口内容怎样决定答案。本题的模型是正数数组让窗口和随右端增加、随左端减少；暴力版会反复统计相邻窗口中相同的内容。慢点具体落在右端扩展到满足目标后，不断左移缩短并更新答案，所以后面要证明左端或右端移动时不会漏掉可能答案。

窗口题只在条件有单调性或窗口长度固定时成立。本题用右端扩展到满足目标后，不断左移缩短并更新答案维护窗口，含义是右端只带来一个新字符或新元素，左端只移走一个旧元素；如果输入条件改变导致状态不再单调，就必须换方法。放到“滑动窗口与双指针”这条主线里看，关键原则是：滑动窗口成立必须有单调性或固定长度边界。右端扩展让某个条件单调变好或变坏，左端收缩可以恢复合法性或缩短答案。若数组含负数、状态会来回震荡，就不能硬套窗口。

证明重点是排除左端。若某个左端被移过，它对应的窗口已经被完整检查，或继续保留只会让窗口更长、更不合法。本题的正数数组让窗口和随右端增加、随左端减少和右端扩展到满足目标后，不断左移缩短并更新答案必须一起说明这一点。这道题的边界不能留到最后补丁式处理，实验时重点跑不存在答案、单元素达到目标、全正是必要条件，并打印左右端、窗口计数和答案。若左端发生回退，或者窗口失去合法性后仍更新答案，就能很快定位错误。

C++ 实现上，窗口计数若字符集固定，可以用 `std::array<int, 128>` 或 26 长度数组；字符集不固定再用哈希表。左闭右闭和左闭右开选一种坚持到底，避免窗口长度和删除位置错位。如果追问变成“若含负数，需要前缀和加单调队列”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 438 找到字符串中所有字母异位词

先枚举所有连续窗口，明确左端、右端和窗口内容怎样决定答案。本题的模型是固定长度窗口内字符频次等于目标频次即为答案；暴力版会反复统计相邻窗口中相同的内容。慢点具体落在右进左出维护计数，窗口长度达到目标长度后检查差异，所以后面要证明左端或右端移动时不会漏掉可能答案。

窗口题只在条件有单调性或窗口长度固定时成立。本题用右进左出维护计数，窗口长度达到目标长度后检查差异维护窗口，含义是右端只带来一个新字符或新元素，左端只移走一个旧元素；如果输入条件改变导致状态不再单调，就必须换方法。放到“滑动窗口与双指针”这条主线里看，关键原则是：滑动窗口成立必须有单调性或固定长度边界。右端扩展让某个条件单调变好或变坏，左端收缩可以恢复合法性或缩短答案。若数组含负数、状态会来回震荡，就不能硬套窗口。

证明重点是排除左端。若某个左端被移过，它对应的窗口已经被完整检查，或继续保留只会让窗口更长、更不合法。本题的固定长度窗口内字符频次等于目标频次即为答案和右进左出维护计数，窗口长度达到目标长度后检查差异必须一起说明这一点。这道题的边界不能留到最后补丁式处理，实验时重点跑目标比源串长、重复字符、字符集固定、答案重叠，并打印左右端、窗口计数和答案。若左端发生回退，或者窗口失去合法性后仍更新答案，就能很快定位错误。

C++ 实现上，窗口计数若字符集固定，可以用 `std::array<int, 128>` 或 26 长度数组；字符集不固定再用哈希表。左闭右闭和左闭右开选一种坚持到底，避免窗口长度和删除位置错位。如果追问变成“维护差异计数可以避免每次比较整个数组”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

前缀和与哈希表工作坊

先用双层枚举区间，再把区间条件改写成两个前缀状态的关系。哈希表保存历史前缀的次数、最早位置或存在性，具体保存什么由题目目标决定。

LeetCode 1 两数之和

先用双层循环枚举区间或候选对，再把条件写成历史状态和当前状态的关系。本题的模型是当前元素只需要寻找唯一补数，历史值到下标的映射就是最小状态；暴力版的问题不是不会找，而是每个当前位置都回头扫太多历史。优化目标就是把先查再插入，避免同一个下标被用两次；若数组有序才考虑双指针变成一次查表或一次状态读取。

哈希表保存的是当前下标之前的历史，不是随手放进去的缓存。本题需要先查再插入，避免同一个下标被用两次；若数组有序才考虑双指针，所以要先决定 value 是次数、最早位置、最近位置还是布尔值。这个决定直接改变答案类型。放到“前缀和与哈希表”这条主线里看，关键原则是：前缀和把区间问题改写成两个前缀状态的差；哈希表把寻找匹配前缀改成查表。频次表、最早位置表、最近位置表对应不同目标：计数、最长区间、最短区间不能混用。

证明回到代数关系：任意合法答案都能对应到一个当前状态和一个历史状态；算法查到的历史状态也一定构成合法答案。本题的当前元素只需要寻找唯一补数，历史值到下标的映射就是最小状态说明要找到什么，先查再插入，避免同一个下标被用两次；若数组有序才考虑双指针说明历史表怎样保存它。这道题的边界不能留到最后补丁式处理，实验时把重复数字、目标为偶数且补数等于当前值、没有答案时返回空结果加入对拍集合，用平方暴力和优化版比较。失败时先看初始历史状态、查询更新顺序和哈希表 value 类型。

C++ 实现上，哈希表查询优先用 `find`，只在确实要插入或累加时使用 `operator[]`。前缀和、区间和、计数乘法常需要 `long long`；模运算还要处理负余数归一化。如果追问变成“若改成返回

所有不重复数对，要先排序或用频次表处理重复”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 49 字母异位词分组

先用双层循环枚举区间或候选对，再把条件写成历史状态和当前状态的关系。本题的模型是异位词共享同一个规范表示，可以是排序字符串或字符计数；暴力版的问题不是不会找，而是每个当前位置都回头扫太多历史。优化目标就是把哈希表按规范键分桶，避免两两比较字符串变成一次查表或一次状态读取。

哈希表保存的是当前下标之前的历史，不是随手放进去的缓存。本题需要哈希表按规范键分桶，避免两两比较字符串，所以要先决定 value 是次数、最早位置、最近位置还是布尔值。这个决定直接改变答案类型。放到“前缀和与哈希表”这条主线里看，关键原则是：前缀和把区间问题改写成两个前缀状态的差；哈希表把寻找匹配前缀改成查表。频次表、最早位置表、最近位置表对应不同目标：计数、最长区间、最短区间不能混用。

证明回到代数关系：任意合法答案都能对应到一个当前状态和一个历史状态；算法查到的历史状态也一定构成合法答案。本题的异位词共享同一个规范表示，可以是排序字符串或字符计数说明要找什么，哈希表按规范键分桶，避免两两比较字符串说明历史表怎样保存它。这道题的边界不能留到最后补丁式处理，实验时把空字符串、重复单词、字符集不止小写字母、计数键必须无歧义加入对拍集合，用平方暴力和优化版比较。失败时先看初始历史状态、查询更新顺序和哈希表 value 类型。

C++ 实现上，哈希表查询优先用 `find`，只在确实要插入或累加时使用 `operator[]`。前缀和、区间和、计数乘法常需要 `long long`；模运算还要处理负余数归一化。如果追问变成“若字符串很长且字符集固定，计数键通常比排序键更稳定”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 128 最长连续序列

先用双层循环枚举区间或候选对，再把条件写成历史状态和当前状态的关系。本题的模型是连续段只需要从没有前驱的数开始扩展；暴力版的问题不是不会找，而是每个当前位置都回头扫太多历史。优化目标就是把哈希集合判断 `x 减一` 是否存在，跳过非起点避免重复扫描变成一次查表或一次状态读取。

哈希表保存的是当前下标之前的历史，不是随手放进去的缓存。本题需要哈希集合判断 `x 减一` 是否存在，跳过非起点避免重复扫描，所以要先决定 value 是次数、最早位置、最近位置还是布尔值。这个决定直接改变答案类型。放到“前缀和与哈希表”这条主线里看，关键原则是：前缀和把区间问题改写成两个前缀状态的差；哈希表把寻找匹配前缀改成查表。频次表、最早位置表、最近位置表对应不同目标：计数、最长区间、最短区间不能混用。

证明回到代数关系：任意合法答案都能对应到一个当前状态和一个历史状态；算法查到的历史状态也一定构成合法答案。本题的连续段只需要从没有前驱的数开始扩展说明要找什么，哈希集合判断 `x 减一` 是否存在，跳过非起点避免重复扫描说明历史表怎样保存它。这道题的边界不能留到最后补丁式处理，实验时把重复元素、负数、空数组、整数边界加入对拍集合，用平方暴力和优化版比较。失败时先看初始历史状态、查询更新顺序和哈希表 value 类型。

C++ 实现上，哈希表查询优先用 `find`，只在确实要插入或累加时使用 `operator[]`。前缀和、区间和、计数乘法常需要 `long long`；模运算还要处理负余数归一化。如果追问变成“若数据流动态插入，可以用并查集或区间合并维护长度”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 202 快乐数

先用双层循环枚举区间或候选对，再把条件写成历史状态和当前状态的关系。本题的模型是反复变换数字会进入一或循环，状态空间最终有限；暴力版的问题不是不会找，而是每个当前位置都回头扫太多历史。优化目标就是把用集合检测重复状态，或用快慢指针看成函数图变成一次查表或一次状态读取。

哈希表保存的是当前下标之前的历史，不是随手放进去的缓存。本题需要用集合检测重复状态，或用快慢指针看成函数图，所以要先决定 value 是次数、最早位置、最近位置还是布尔值。这个决定直接改变答案类型。放到“前缀和与哈希表”这条主线里看，关键原则是：前缀和把区间问题改写成

两个前缀状态的差；哈希表把寻找匹配前缀改成查表。频次表、最早位置表、最近位置表对应不同目标：计数、最长区间、最短区间不能混用。

证明回到代数关系：任意合法答案都能对应到一个当前状态和一个历史状态；算法查到的历史状态也一定构成合法答案。本题的反复变换数字会进入一或循环，状态空间最终有限说明要找什么，用集合检测重复状态，或用快慢指针看成函数图说明历史表怎样保存它。这道题的边界不能留到最后补丁式处理，实验时把输入为一、进入循环、位平方和函数、负数不在题意内加入对拍集合，用平方暴力和优化版比较。失败时先看初始历史状态、查询更新顺序和哈希表 value 类型。

C++ 实现上，哈希表查询优先用 `find`，只在确实要插入或累加时使用 `operator[]`。前缀和、区间和、计数乘法常需要 `long long`；模运算还要处理负余数归一化。如果追问变成“快慢指针版本能训练把数值迭代看成链表”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 217 存在重复元素

先用双层循环枚举区间或候选对，再把条件写成历史状态和当前状态的关系。本题的模型是判断是否出现过是哈希集合最直接的职责；暴力版的问题不是不会找，而是每个当前位置都回头扫太多历史。优化目标就是把扫描时若插入前已经存在则立即返回真变成一次查表或一次状态读取。

哈希表保存的是当前下标之前的历史，不是随手放进去的缓存。本题需要扫描时若插入前已经存在则立即返回真，所以要先决定 value 是次数、最早位置、最近位置还是布尔值。这个决定直接改变答案类型。放到“前缀和与哈希表”这条主线里看，关键原则是：前缀和把区间问题改写成两个前缀状态的差；哈希表把寻找匹配前缀改成查表。频次表、最早位置表、最近位置表对应不同目标：计数、最长区间、最短区间不能混用。

证明回到代数关系：任意合法答案都能对应到一个当前状态和一个历史状态；算法查到的历史状态也一定构成合法答案。本题的判断是否出现过是哈希集合最直接的职责说明要找什么，扫描时若插入前已经存在则立即返回真说明历史表怎样保存它。这道题的边界不能留到最后补丁式处理，实验时把空数组、全唯一、重复在末尾、值域很小加入对拍集合，用平方暴力和优化版比较。失败时先看初始历史状态、查询更新顺序和哈希表 value 类型。

C++ 实现上，哈希表查询优先用 `find`，只在确实要插入或累加时使用 `operator[]`。前缀和、区间和、计数乘法常需要 `long long`；模运算还要处理负余数归一化。如果追问变成“若允许修改输入，排序后看相邻可以降低额外空间”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 560 和为 K 的子数组

先用双层循环枚举区间或候选对，再把条件写成历史状态和当前状态的关系。本题的模型是当前前缀和减去目标值，就是需要匹配的历史前缀；暴力版的问题不是不会找，而是每个当前位置都回头扫太多历史。优化目标就是把哈希表保存前缀和出现次数，先查询再更新变成一次查表或一次状态读取。

哈希表保存的是当前下标之前的历史，不是随手放进去的缓存。本题需要哈希表保存前缀和出现次数，先查询再更新，所以要先决定 value 是次数、最早位置、最近位置还是布尔值。这个决定直接改变答案类型。放到“前缀和与哈希表”这条主线里看，关键原则是：前缀和把区间问题改写成两个前缀状态的差；哈希表把寻找匹配前缀改成查表。频次表、最早位置表、最近位置表对应不同目标：计数、最长区间、最短区间不能混用。

证明回到代数关系：任意合法答案都能对应到一个当前状态和一个历史状态；算法查到的历史状态也一定构成合法答案。本题的当前前缀和减去目标值，就是需要匹配的历史前缀说明要找什么，哈希表保存前缀和出现次数，先查询再更新说明历史表怎样保存它。这道题的边界不能留到最后补丁式处理，实验时把负数、目标为零、从下标零开始的区间、重复前缀加入对拍集合，用平方暴力和优化版比较。失败时先看初始历史状态、查询更新顺序和哈希表 value 类型。

C++ 实现上，哈希表查询优先用 `find`，只在确实要插入或累加时使用 `operator[]`。前缀和、区间和、计数乘法常需要 `long long`；模运算还要处理负余数归一化。如果追问变成“若要求最长区间，哈希表应保存最早位置而不是次数”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

二分查找与答案空间工作坊

先线性扫描或线性试答案，再找出能把候选一分为二的单调谓词。二分的核心不是 mid 写法，而是被排除的一半确实不可能包含答案。

LeetCode 33 搜索旋转排序数组

先用线性扫描或线性试答案保证正确，再找边界。本题的模型是旋转数组每次二分至少有一半保持有序；二分不是为了套循环，而是因为判断哪一半有序，再看目标是否落在有序半边范围内给出了一条能排除候选的边界线。若这条边界线说不清，宁愿先保留线性版本。

二分的状态不是数组本身，而是候选区间。本题用判断哪一半有序，再看目标是否落在有序半边范围内缩小区间；每次更新左右边界时，都要能说清被丢掉的一半为什么没有答案。边界不变量比 mid 公式更重要。放到“二分查找与答案空间”这条主线里看，关键原则是：二分前必须先定义谓词：某个位置是否已经满足边界条件，或者某个答案是否可行。数组二分找的是边界，答案二分找的是最小可行值或最大可行值。

证明二分时，不要只说复杂度是对数。要证明当判断哪一半有序，再看目标是否落在有序半边范围内成立或不成立时，被排除的一侧确实没有目标。本题的旋转数组每次二分至少有一半保持有序提供候选空间，循环只是在这个空间里保留答案。这道题的边界不能留到最后补丁式处理，实验时覆盖未旋转、长度一、目标不存在、边界等号、存在重复元素时退化，逐轮打印 left、mid、right 和谓词值。只要有一次被排除区间仍含答案，二分不变量就错了。

C++ 实现上，中点写成 `left + (right - left) / 2`；平方、乘法和总量比较避免溢出。答案二分最好把检查函数单独写出，让单调谓词可读。如果追问变成“若重复元素很多，需要在无法判断有序半边时收缩边界”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 34 排序数组中查找首尾位置

先用线性扫描或线性试答案保证正确，再找边界。本题的模型是首尾位置可以拆成两个边界：第一个大于等于和第一个大于；二分不是为了套循环，而是因为不要找到一个目标后向两边线性扩展，否则重复元素很多时退化给出了一条能排除候选的边界线。若这条边界线说不清，宁愿先保留线性版本。

二分的状态不是数组本身，而是候选区间。本题用不要找到一个目标后向两边线性扩展，否则重复元素很多时退化缩小区间；每次更新左右边界时，都要能说清被丢掉的一半为什么没有答案。边界不变量比 mid 公式更重要。放到“二分查找与答案空间”这条主线里看，关键原则是：二分前必须先定义谓词：某个位置是否已经满足边界条件，或者某个答案是否可行。数组二分找的是边界，答案二分找的是最小可行值或最大可行值。

证明二分时，不要只说复杂度是对数。要证明当不要找到一个目标后向两边线性扩展，否则重复元素很多时退化成立或不成立时，被排除的一侧确实没有目标。本题的首尾位置可以拆成两个边界：第一个大于等于和第一个大于提供候选空间，循环只是在这个空间里保留答案。这道题的边界不能留到最后补丁式处理，实验时覆盖目标不存在、目标在开头或结尾、数组为空、全是目标，逐轮打印 left、mid、right 和谓词值。只要有一次被排除区间仍含答案，二分不变量就错了。

C++ 实现上，中点写成 `left + (right - left) / 2`；平方、乘法和总量比较避免溢出。答案二分最好把检查函数单独写出，让单调谓词可读。如果追问变成“这个模型能迁移到计数某值出现次数和插入区间边界”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 35 搜索插入位置

先用线性扫描或线性试答案保证正确，再找边界。本题的模型是题目等价于寻找第一个大于等于目标的位置；二分不是为了套循环，而是因为统一用 lower bound 语义，不在相等时提前返回也能保持边界一致给出了一条能排除候选的边界线。若这条边界线说不清，宁愿先保留线性版本。

二分的状态不是数组本身，而是候选区间。本题用统一用 lower bound 语义，不在相等时提前返回也能保持边界一致缩小区间；每次更新左右边界时，都要能说清被丢掉的一半为什么没有答案。边界不变量比 mid 公式更重要。放到“二分查找与答案空间”这条主线里看，关键原则是：二分前必须先定义谓词：某个位置是否已经满足边界条件，或者某个答案是否可行。数组二分找的是边界，答案二分找的是最小可行值或最大可行值。

证明二分时，不要只说复杂度是对数。要证明当统一用 lower bound 语义，不在相等时提前返回也能保持边界一致成立或不成立时，被排除的一侧确实没有目标。本题的题目等价于寻找第一个大于等于目标的位置提供候选空间，循环只是在这个空间里保留答案。这道题的边界不能留到最后补丁式处理，实验时覆盖目标小于所有元素、目标大于所有元素、重复值、空数组，逐轮打印 left、mid、right 和谓词值。只要有一次被排除区间仍含答案，二分不变量就错了。

C++ 实现上，中点写成 $\text{left} + (\text{right} - \text{left}) / 2$ ；平方、乘法和总量比较避免溢出。答案二分最好把检查函数单独写出，让单调谓词可读。如果追问变成“手写二分时用左闭右开可直接返回 left”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 153 寻找旋转排序数组中的最小值

先用线性扫描或线性试答案保证正确，再找边界。本题的模型是最小值是旋转断点，右端点可帮助判断中点在哪一段；二分不是为了套循环，而是因为若 mid 大于 right，最小值在右侧；否则在左侧含 mid 给出了一条能排除候选的边界线。若这条边界线说不清，宁愿先保留线性版本。

二分的状态不是数组本身，而是候选区间。本题用若 mid 大于 right，最小值在右侧；否则在左侧含 mid 缩小区间；每次更新左右边界时，都要能说清被丢掉的一半为什么没有答案。边界不变量比 mid 公式更重要。放到“二分查找与答案空间”这条主线里看，关键原则是：二分前必须先定义谓词：某个位置是否已经满足边界条件，或者某个答案是否可行。数组二分找的是边界，答案二分找的是最小可行值或最大可行值。

证明二分时，不要只说复杂度是对数。要证明当若 mid 大于 right，最小值在右侧；否则在左侧含 mid 成立或不成立时，被排除的一侧确实没有目标。本题的最小值是旋转断点，右端点可帮助判断中点在哪一段提供候选空间，循环只是在这个空间里保留答案。这道题的边界不能留到最后补丁式处理，实验时覆盖未旋转、长度一、旋转一位、无重复条件，逐轮打印 left、mid、right 和谓词值。只要有一次被排除区间仍含答案，二分不变量就错了。

C++ 实现上，中点写成 $\text{left} + (\text{right} - \text{left}) / 2$ ；平方、乘法和总量比较避免溢出。答案二分最好把检查函数单独写出，让单调谓词可读。如果追问变成“有重复时遇到相等只能收缩 right，最坏退化”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 154 寻找旋转排序数组中的最小值 II

先用线性扫描或线性试答案保证正确，再找边界。本题的模型是重复元素会让 mid 和 right 相等时无法判断最小值方向；二分不是为了套循环，而是因为相等时只能安全地丢掉一个 right，因为它不是唯一必要候选给出了一条能排除候选的边界线。若这条边界线说不清，宁愿先保留线性版本。

二分的状态不是数组本身，而是候选区间。本题用相等时只能安全地丢掉一个 right，因为它不是唯一必要候选缩小区间；每次更新左右边界时，都要能说清被丢掉的一半为什么没有答案。边界不变量比 mid 公式更重要。放到“二分查找与答案空间”这条主线里看，关键原则是：二分前必须先定义谓词：某个位置是否已经满足边界条件，或者某个答案是否可行。数组二分找的是边界，答案二分找的是最小可行值或最大可行值。

证明二分时，不要只说复杂度是对数。要证明当相等时只能安全地丢掉一个 right，因为它不是唯一必要候选成立或不成立时，被排除的一侧确实没有目标。本题的重复元素会让 mid 和 right 相等时无法判断最小值方向提供候选空间，循环只是在这个空间里保留答案。这道题的边界不能留到最后补丁式处理，实验时覆盖全相等、重复值跨断点、最小值出现多次，逐轮打印 left、mid、right 和谓词值。只要有一次被排除区间仍含答案，二分不变量就错了。

C++ 实现上，中点写成 $\text{left} + (\text{right} - \text{left}) / 2$ ；平方、乘法和总量比较避免溢出。答案二分最好把检查函数单独写出，让单调谓词可读。如果追问变成“这题说明二分依赖的信息不足时会退化，不是所有旋转题都严格对数”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 162 寻找峰值

先用线性扫描或线性试答案保证正确，再找边界。本题的模型是比较 mid 和 mid 加一的斜率能判断某侧一定存在峰值；二分不是为了套循环，而是因为上升则右侧有峰，下降则左侧含 mid 有峰给出了一条能排除候选的边界线。若这条边界线说不清，宁愿先保留线性版本。

二分的状态不是数组本身，而是候选区间。本题用上升则右侧有峰，下降则左侧含 mid 有峰缩小区间；每次更新左右边界时，都要能说清被丢掉的一半为什么没有答案。边界不变量比 mid 公式更重要。放到“二分查找与答案空间”这条主线里看，关键原则是：二分前必须先定义谓词：某个位置是否已经满足边界条件，或者某个答案是否可行。数组二分找的是边界，答案二分找的是最小可行值或最大可行值。

证明二分时，不要只说复杂度是对数。要证明当上升则右侧有峰，下降则左侧含 mid 有峰成立或不成立时，被排除的一侧确实没有目标。本题的比较 mid 和 mid 加一的斜率能判断某侧一定存在峰值提供候选空间，循环只是在这个空间里保留答案。这道题的边界不能留到最后补丁式处理，实验时覆盖长度一、峰在边界、多个峰、相邻不相等假设，逐轮打印 left、mid、right 和谓词值。只要有一次被排除区间仍含答案，二分不变量就错了。

C++ 实现上，中点写成 $\text{left} + (\text{right} - \text{left}) / 2$ ；平方、乘法和总量比较避免溢出。答案二分最好把检查函数单独写出，让单调谓词可读。如果追问变成“二维峰值需要按行列最大值进行更复杂二分”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 240 搜索二维矩阵 II

先用线性扫描或线性试答案保证正确，再找边界。本题的模型是右上角同时具有向左变小、向下变大的排除能力；二分不是为了套循环，而是因为当前值大于目标就左移，小于目标就下移给出了一条能排除候选的边界线。若这条边界线说不清，宁愿先保留线性版本。

二分的状态不是数组本身，而是候选区间。本题用当前值大于目标就左移，小于目标就下移缩小区间；每次更新左右边界时，都要能说清被丢掉的一半为什么没有答案。边界不变量比 mid 公式更重要。放到“二分查找与答案空间”这条主线里看，关键原则是：二分前必须先定义谓词：某个位置是否已经满足边界条件，或者某个答案是否可行。数组二分找的是边界，答案二分找的是最小可行值或最大可行值。

证明二分时，不要只说复杂度是对数。要证明当当前值大于目标就左移，小于目标就下移成立或不成立时，被排除的一侧确实没有目标。本题的右上角同时具有向左变小、向下变大的排除能力提供候选空间，循环只是在这个空间里保留答案。这道题的边界不能留到最后补丁式处理，实验时覆盖空矩阵、单行、单列、目标在角落、重复值，逐轮打印 left、mid、right 和谓词值。只要有一次被排除区间仍含答案，二分不变量就错了。

C++ 实现上，中点写成 $\text{left} + (\text{right} - \text{left}) / 2$ ；平方、乘法和总量比较避免溢出。答案二分最好把检查函数单独写出，让单调谓词可读。如果追问变成“从左上角无法一次排除一行或一列”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 704 二分查找

先用线性扫描或线性试答案保证正确，再找边界。本题的模型是基础题的重点是固定区间语义；二分不是为了套循环，而是因为左闭右开写法能统一 lower bound 和存在性检查给出了一条能排除候选的边界线。若这条边界线说不清，宁愿先保留线性版本。

二分的状态不是数组本身，而是候选区间。本题用左闭右开写法能统一 lower bound 和存在性检查缩小区间；每次更新左右边界时，都要能说清被丢掉的一半为什么没有答案。边界不变量比 mid 公式更重要。放到“二分查找与答案空间”这条主线里看，关键原则是：二分前必须先定义谓词：某个位置是否已经满足边界条件，或者某个答案是否可行。数组二分找的是边界，答案二分找的是最小可行值或最大可行值。

证明二分时，不要只说复杂度是对数。要证明当左闭右开写法能统一 lower bound 和存在性检查成立或不成立时，被排除的一侧确实没有目标。本题的基础题的重点是固定区间语义提供候选空间，循环只是在这个空间里保留答案。这道题的边界不能留到最后补丁式处理，实验时覆盖空数组、长度一、目标在边界、目标不存在，逐轮打印 left、mid、right 和谓词值。只要有一次被排除区间仍含答案，二分不变量就错了。

C++ 实现上，中点写成 $\text{left} + (\text{right} - \text{left}) / 2$ ；平方、乘法和总量比较避免溢出。答案二分最好把检查函数单独写出，让单调谓词可读。如果追问变成“所有复杂二分都应该能退回到这道题的边界不变量”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 875 爱吃香蕉的珂珂

先用线性扫描或线性试答案保证正确，再找边界。本题的模型是速度越大，总耗时越小，答案具有单调可行性；二分不是为了套循环，而是因为二分速度，检查函数用向上取整累加小时数给出了一条能排除候选的边界线。若这条边界线说不清，宁愿先保留线性版本。

二分的状态不是数组本身，而是候选区间。本题用二分速度，检查函数用向上取整累加小时数缩小区间；每次更新左右边界时，都要能说清被丢掉的一半为什么没有答案。边界不变量比 mid 公式更重要。放到“二分查找与答案空间”这条主线里看，关键原则是：二分前必须先定义谓词：某个位置是否已经满足边界条件，或者某个答案是否可行。数组二分找的是边界，答案二分找的是最小可行值或最大可行值。

证明二分时，不要只说复杂度是对数。要证明当二分速度，检查函数用向上取整累加小时数成立或不成立时，被排除的一侧确实没有目标。本题的速度越大，总耗时越小，答案具有单调可行性提供候选空间，循环只是在这个空间里保留答案。这道题的边界不能留到最后补丁式处理，实验时覆盖速度下界、堆很大、h 等于堆数、乘法加法溢出，逐轮打印 left、mid、right 和谓词值。只要有一次被排除区间仍含答案，二分不变量就错了。

C++ 实现上，中点写成 $\text{left} + (\text{right} - \text{left}) / 2$ ；平方、乘法和总量比较避免溢出。答案二分最好把检查函数单独写出，让单调谓词可读。如果追问变成“船运包裹和分割数组都是同类最小可行上限”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 1011 在 D 天内送达包裹

先用线性扫描或线性试答案保证正确，再找边界。本题的模型是容量越大，需要天数越少，存在最小可行容量；二分不是为了套循环，而是因为下界是最大包裹，上界是总重量，检查函数按顺序贪心装船给出了一条能排除候选的边界线。若这条边界线说不清，宁愿先保留线性版本。

二分的状态不是数组本身，而是候选区间。本题用下界是最大包裹，上界是总重量，检查函数按顺序贪心装船缩小区间；每次更新左右边界时，都要能说清被丢掉的一半为什么没有答案。边界不变量比 mid 公式更重要。放到“二分查找与答案空间”这条主线里看，关键原则是：二分前必须先定义谓词：某个位置是否已经满足边界条件，或者某个答案是否可行。数组二分找的是边界，答案二分找的是最小可行值或最大可行值。

证明二分时，不要只说复杂度是对数。要证明当下界是最大包裹，上界是总重量，检查函数按顺序贪心装船成立或不成立时，被排除的一侧确实没有目标。本题的容量越大，需要天数越少，存在最小可行容量提供候选空间，循环只是在这个空间里保留答案。这道题的边界不能留到最后补丁式处理，实验时覆盖 D 为一、D 等于包裹数、单个包裹超过猜测容量，逐轮打印 left、mid、right 和谓词值。只要有一次被排除区间仍含答案，二分不变量就错了。

C++ 实现上，中点写成 $\text{left} + (\text{right} - \text{left}) / 2$ ；平方、乘法和总量比较避免溢出。答案二分最好把检查函数单独写出，让单调谓词可读。如果追问变成“它和分割数组最大值都是连续分段最大和模型”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

栈、单调栈与表达式工作坊

先对每个位置向左或向右扫描，再识别还没有被解决的候选。栈保存等待未来元素解决的对象，新元素到来时一次性结算被它支配的一批旧候选。

LeetCode 42 接雨水

先让每个位置向左或向右暴力寻找答案，观察哪些候选一直等到未来才被解决。本题的模型是每个位置的水由左侧最高和右侧最高的较小者决定；慢点集中在前后缀、双指针、单调栈都是不同状态维护方式；要能说明各自结算对象。栈的作用不是保存所有历史，而是保存仍未被当前输入解决的那一小段历史。

栈里应保存还没确定答案的对象。本题用前后缀、双指针、单调栈都是不同状态维护方式；要能说明各自结算对象触发结算；弹出时要说清当前元素是右边界、左边界还是运算触发点。栈里存值还是下标，要由距离、宽度和过期判断决定。放到“栈、单调栈与表达式”这条主线里看，关键原则是：当新元素到来时，它会一次性解决栈顶一串被它支配的旧元素。被弹出的元素以后再也不会参与比较，因此总体线性。

正确性来自弹出时机。一旦前后缀、双指针、单调栈都是不同状态维护方式；要能说明各自结算对象触发，栈顶对象的答案已经由当前边界和新的栈顶共同确定；未来元素不会再改变它。本题的每个位置的水由左侧最高和右侧最高的较小者决定说明栈中对象等待的是什么。这道题的边界不能留到最后补丁式处理，实验时准备单调递增、单调递减、平台高度、多个凹槽、数组长度不足三，打印每次入栈、弹栈和结算对象。重点观察相等元素、空栈、哨兵和最后一次结算。

C++ 实现上，栈题多数时候用 `std::vector<int>` 保存下标，比适配器更方便查看栈顶和计算宽度。表达式题要注意左右操作数顺序，柱状图题要注意哨兵和宽度。如果追问变成“若柱子变成二维网格，需要最小堆从边界向内扩展”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 84 柱状图中最大的矩形

先让每个位置向左或向右暴力寻找答案，观察哪些候选一直等到未来才被解决。本题的模型是每根柱子作为最矮高度时，需要知道左右第一个更矮位置；慢点集中在单调递增栈在遇到更矮柱时结算被弹出柱子的最大宽度。栈的作用不是保存所有历史，而是保存仍未被当前输入解决的那一小段历史。

栈里应保存还没确定答案的对象。本题用单调递增栈在遇到更矮柱时结算被弹出柱子的最大宽度触发结算；弹出时要说清当前元素是右边界、左边界还是运算触发点。栈里存值还是下标，要由距离、宽度和过期判断决定。放到“栈、单调栈与表达式”这条主线里看，关键原则是：当新元素到来时，它会一次性解决栈顶一串被它支配的旧元素。被弹出的元素以后再也不会参与比较，因此总体线性。

正确性来自弹出时机。一旦单调递增栈在遇到更矮柱时结算被弹出柱子的最大宽度触发，栈顶对象的答案已经由当前边界和新的栈顶共同确定；未来元素不会再改变它。本题的每根柱子作为最矮高度时，需要知道左右第一个更矮位置说明栈中对象等待的是什么。这道题的边界不能留到最后补丁式处理，实验时准备递增、递减、相等高度、哨兵零、宽度计算，打印每次入栈、弹栈和结算对象。重点观察相等元素、空栈、哨兵和最后一次结算。

C++ 实现上，栈题多数时候用 `std::vector<int>` 保存下标，比适配器更方便查看栈顶和计算宽度。表达式题要注意左右操作数顺序，柱状图题要注意哨兵和宽度。如果追问变成“最大矩形可以作为二维矩阵最大矩形的行压缩子问题”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 150 逆波兰表达式求值

先让每个位置向左或向右暴力寻找答案，观察哪些候选一直等到未来才被解决。本题的模型是后缀表达式把优先级和括号都编码进 token 顺序；慢点集中在遇到数字入栈，遇到运算符弹出右操作数和左操作数计算。栈的作用不是保存所有历史，而是保存仍未被当前输入解决的那一小段历史。

栈里应保存还没确定答案的对象。本题用遇到数字入栈，遇到运算符弹出右操作数和左操作数计算触发结算；弹出时要说清当前元素是右边界、左边界还是运算触发点。栈里存值还是下标，要由距离、宽度和过期判断决定。放到“栈、单调栈与表达式”这条主线里看，关键原则是：当新元素到来时，它会一次性解决栈顶一串被它支配的旧元素。被弹出的元素以后再也不会参与比较，因此总体线性。

正确性来自弹出时机。一旦遇到数字入栈，遇到运算符弹出右操作数和左操作数计算触发，栈顶对象的答案已经由当前边界和新的栈顶共同确定；未来元素不会再改变它。本题的后缀表达式把优先级和括号都编码进 token 顺序说明栈中对象等待的是什么。这道题的边界不能留到最后补丁式处理，实验时准备负数 token、多位数、除法向零截断、减法顺序，打印每次入栈、弹栈和结算对象。重点观察相等元素、空栈、哨兵和最后一次结算。

C++ 实现上，栈题多数时候用 `std::vector<int>` 保存下标，比适配器更方便查看栈顶和计算宽度。表达式题要注意左右操作数顺序，柱状图题要注意哨兵和宽度。如果追问变成“中缀表达式求值需要另一个运算符栈处理优先级”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 155 最小栈

先让每个位置向左或向右暴力寻找答案，观察哪些候选一直等到未来才被解决。本题的模型是普通栈只能看到栈顶，不能常数时间知道历史最小值；慢点集中在辅助栈同步保存每个深度对应的最小

值。栈的作用不是保存所有历史，而是保存仍未被当前输入解决的那一小段历史。

栈里应保存还没确定答案的对象。本题用辅助栈同步保存每个深度对应的最小值触发结算；弹出时要说清当前元素是右边界、左边界还是运算触发点。栈里存值还是下标，要由距离、宽度和过期判断决定。放到“栈、单调栈与表达式”这条主线里看，关键原则是：当新元素到来时，它会一次性解决栈顶一串被它支配的旧元素。被弹出的元素以后再也不会参与比较，因此总体线性。

正确性来自弹出时机。一旦辅助栈同步保存每个深度对应的最小值触发，栈顶对象的答案已经由当前边界和新的栈顶共同确定；未来元素不会再改变它。本题的普通栈只能看到栈顶，不能常数时间知道历史最小值说明栈中对象等待的是什么。这道题的边界不能留到最后补丁式处理，实验时准备重复最小值、弹出最小值、空栈操作约定，打印每次入栈、弹栈和结算对象。重点观察相等元素、空栈、哨兵和最后一次结算。

C++ 实现上，栈题多数时候用 `std::vector<int>` 保存下标，比适配器更方便查看栈顶和计算宽度。表达式题要注意左右操作数顺序，柱状图题要注意哨兵和宽度。如果追问变成“也可保存差值压缩空间，但可读性和溢出处理更复杂”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 232 用栈实现队列

先让每个位置向左或向右暴力寻找答案，观察哪些候选一直等到未来才被解决。本题的模型是两个栈分别负责输入顺序和输出顺序；慢点集中在只有输出栈为空时，才把输入栈整体倒过去，保证均摊常数。栈的作用不是保存所有历史，而是保存仍未被当前输入解决的那一小段历史。

栈里应保存还没确定答案的对象。本题用只有输出栈为空时，才把输入栈整体倒过去，保证均摊常数触发结算；弹出时要说清当前元素是右边界、左边界还是运算触发点。栈里存值还是下标，要由距离、宽度和过期判断决定。放到“栈、单调栈与表达式”这条主线里看，关键原则是：当新元素到来时，它会一次性解决栈顶一串被它支配的旧元素。被弹出的元素以后再也不会参与比较，因此总体线性。

正确性来自弹出时机。一旦只有输出栈为空时，才把输入栈整体倒过去，保证均摊常数触发，栈顶对象的答案已经由当前边界和新的栈顶共同确定；未来元素不会再改变它。本题的两个栈分别负责输入顺序和输出顺序说明栈中对象等待的是什么。这道题的边界不能留到最后补丁式处理，实验时准备连续 peek、空队列、交替 push pop，打印每次入栈、弹栈和结算对象。重点观察相等元素、空栈、哨兵和最后一次结算。

C++ 实现上，栈题多数时候用 `std::vector<int>` 保存下标，比适配器更方便查看栈顶和计算宽度。表达式题要注意左右操作数顺序，柱状图题要注意哨兵和宽度。如果追问变成“用队列实现栈则要通过轮转保持最近元素在队首”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 239 滑动窗口最大值

先让每个位置向左或向右暴力寻找答案，观察哪些候选一直等到未来才被解决。本题的模型是每个窗口最大值来自仍未过期且没有被新元素支配的候选；慢点集中在单调队列保存下标，队尾小于等于新值的候选被弹出。栈的作用不是保存所有历史，而是保存仍未被当前输入解决的那一小段历史。

栈里应保存还没确定答案的对象。本题用单调队列保存下标，队尾小于等于新值的候选被弹出触发结算；弹出时要说清当前元素是右边界、左边界还是运算触发点。栈里存值还是下标，要由距离、宽度和过期判断决定。放到“栈、单调栈与表达式”这条主线里看，关键原则是：当新元素到来时，它会一次性解决栈顶一串被它支配的旧元素。被弹出的元素以后再也不会参与比较，因此总体线性。

正确性来自弹出时机。一旦单调队列保存下标，队尾小于等于新值的候选被弹出触发，栈顶对象的答案已经由当前边界和新的栈顶共同确定；未来元素不会再改变它。本题的每个窗口最大值来自仍未过期且没有被新元素支配的候选说明栈中对象等待的是什么。这道题的边界不能留到最后补丁式处理，实验时准备 k 为一、k 等于数组长度、重复最大值、队首过期，打印每次入栈、弹栈和结算对象。重点观察相等元素、空栈、哨兵和最后一次结算。

C++ 实现上，栈题多数时候用 `std::vector<int>` 保存下标，比适配器更方便查看栈顶和计算宽度。表达式题要注意左右操作数顺序，柱状图题要注意哨兵和宽度。如果追问变成“受限子序列和使用同样队列维护最近 k 个 DP 最大值”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 394 字符串解码

先让每个位置向左或向右暴力寻找答案，观察哪些候选一直等到未来才被解决。本题的模型是嵌套结构需要保存上一层字符串和重复次数；慢点集中在遇到左括号入栈当前状态，右括号弹出并拼接展开。栈的作用不是保存所有历史，而是保存仍未被当前输入解决的那一小段历史。

栈里应保存还没确定答案的对象。本题用遇到左括号入栈当前状态，右括号弹出并拼接展开触发结算；弹出时要说清当前元素是右边界、左边界还是运算触发点。栈里存值还是下标，要由距离、宽度和过期判断决定。放到“栈、单调栈与表达式”这条主线里看，关键原则是：当新元素到来时，它会一次性解决栈顶一串被它支配的旧元素。被弹出的元素以后再也不会参与比较，因此总体线性。

正确性来自弹出时机。一旦遇到左括号入栈当前状态，右括号弹出并拼接展开触发，栈顶对象的答案已经由当前边界和新的栈顶共同确定；未来元素不会再改变它。本题的嵌套结构需要保存上一层字符串和重复次数说明栈中对象等待的是什么。这道题的边界不能留到最后补丁式处理，实验时准备多位数字、嵌套、空片段、数字后必须跟括号，打印每次入栈、弹栈和结算对象。重点观察相等元素、空栈、哨兵和最后一次结算。

C++ 实现上，栈题多数时候用 `std::vector<int>` 保存下标，比适配器更方便查看栈顶和计算宽度。表达式题要注意左右操作数顺序，柱状图题要注意哨兵和宽度。如果追问变成“递归下降解析更通用，但迭代栈更符合工程栈控制”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 496 下一个更大元素 I

先让每个位置向左或向右暴力寻找答案，观察哪些候选一直等到未来才被解决。本题的模型是第二个数组提供每个值右侧第一个更大值的全局关系；慢点集中在单调栈预处理值到下一个更大值映射，再回答第一个数组查询。栈的作用不是保存所有历史，而是保存仍未被当前输入解决的那一小段历史。

栈里应保存还没确定答案的对象。本题用单调栈预处理值到下一个更大值映射，再回答第一个数组查询触发结算；弹出时要说清当前元素是右边界、左边界还是运算触发点。栈里存值还是下标，要由距离、宽度和过期判断决定。放到“栈、单调栈与表达式”这条主线里看，关键原则是：当新元素到来时，它会一次性解决栈顶一串被它支配的旧元素。被弹出的元素以后再也不会参与比较，因此总体线性。

正确性来自弹出时机。一旦单调栈预处理值到下一个更大值映射，再回答第一个数组查询触发，栈顶对象的答案已经由当前边界和新的栈顶共同确定；未来元素不会再改变它。本题的第二个数组提供每个值右侧第一个更大值的全局关系说明栈中对象等待的是什么。这道题的边界不能留到最后补丁式处理，实验时准备不存在更大值、递减数组、值唯一假设，打印每次入栈、弹栈和结算对象。重点观察相等元素、空栈、哨兵和最后一次结算。

C++ 实现上，栈题多数时候用 `std::vector<int>` 保存下标，比适配器更方便查看栈顶和计算宽度。表达式题要注意左右操作数顺序，柱状图题要注意哨兵和宽度。如果追问变成“若值不唯一，应按下标而不是值建映射”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 503 下一个更大元素 II

先让每个位置向左或向右暴力寻找答案，观察哪些候选一直等到未来才被解决。本题的模型是环形数组可以通过遍历两倍下标模拟绕回；慢点集中在第一遍入栈，第二遍只帮助未解决下标找到答案。栈的作用不是保存所有历史，而是保存仍未被当前输入解决的那一小段历史。

栈里应保存还没确定答案的对象。本题用第一遍入栈，第二遍只帮助未解决下标找到答案触发结算；弹出时要说清当前元素是右边界、左边界还是运算触发点。栈里存值还是下标，要由距离、宽度和过期判断决定。放到“栈、单调栈与表达式”这条主线里看，关键原则是：当新元素到来时，它会一次性解决栈顶一串被它支配的旧元素。被弹出的元素以后再也不会参与比较，因此总体线性。

正确性来自弹出时机。一旦第一遍入栈，第二遍只帮助未解决下标找到答案触发，栈顶对象的答案已经由当前边界和新的栈顶共同确定；未来元素不会再改变它。本题的环形数组可以通过遍历两倍下标模拟绕回说明栈中对象等待的是什么。这道题的边界不能留到最后补丁式处理，实验时准备全递减、全相等、长度一、取模下标，打印每次入栈、弹栈和结算对象。重点观察相等元素、空栈、哨兵和最后一次结算。

C++ 实现上，栈题多数时候用 `std::vector<int>` 保存下标，比适配器更方便查看栈顶和计算宽度。表达式题要注意左右操作数顺序，柱状图题要注意哨兵和宽度。如果追问变成“不要在第二遍重复入栈，否则可能死循环或重复计算”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 739 每日温度

先让每个位置向左或向右暴力寻找答案，观察哪些候选一直等到未来才被解决。本题的模型是每一天等待右侧第一个更高温度；慢点集中在单调递减栈保存尚未找到答案的下标。栈的作用不是保存所有历史，而是保存仍未被当前输入解决的那一小段历史。

栈里应保存还没确定答案的对象。本题用单调递减栈保存尚未找到答案的下标触发结算；弹出时要说清当前元素是右边界、左边界还是运算触发点。栈里存值还是下标，要由距离、宽度和过期判断决定。放到“栈、单调栈与表达式”这条主线里看，关键原则是：当新元素到来时，它会一次性解决栈顶一串被它支配的旧元素。被弹出的元素以后再也不用参与比较，因此总体线性。

正确性来自弹出时机。一旦单调递减栈保存尚未找到答案的下标触发，栈顶对象的答案已经由当前边界和新的栈顶共同确定；未来元素不会再改变它。本题的每一天等待右侧第一个更高温度说明栈中对象等待的是什么。这道题的边界不能留到最后补丁式处理，实验时准备没有更高温度、相等温度、递增、递减，打印每次入栈、弹栈和结算对象。重点观察相等元素、空栈、哨兵和最后一次结算。

C++ 实现上，栈题多数时候用 `std::vector<int>` 保存下标，比适配器更方便查看栈顶和计算宽度。表达式题要注意左右操作数顺序，柱状图题要注意哨兵和宽度。如果追问变成“下一个更大元素与它同型，只是输出值或距离不同”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

堆与动态极值工作坊

先每轮扫描所有候选找极值，再把动态极值查询交给堆。堆只保证堆顶，不保证全局遍历有序；有过期元素时要在使用堆顶前清理。

LeetCode 215 数组中的第 K 个最大元素

先每轮扫描所有候选找极值，确认答案选择的对象。本题的模型是只关心第 K 大，不需要完整排序；暴力版慢在动态候选集合不断变化，却每次重新找最值。本题的优化点是大小为 K 的小顶堆保存当前前 K 大边界；快速选择可做到平均线性，因此堆中只能放足以比较下一步选择的轻量状态。

堆只保证堆顶，不保证内部全局有序。本题的大小为 K 的小顶堆保存当前前 K 大边界；快速选择可做到平均线性要转成堆顶可回答的问题；如果元素会过期，还要在真正使用堆顶前清理或跳过旧状态。放到“堆与动态极值”这条主线里看，关键原则是：堆把插入和弹出极值控制在对数复杂度。Top K 用固定大小堆维护边界，合并多路序列用堆保存每一路当前头部，数据流中位数用双堆维护两半。

证明堆题时，要说明堆覆盖了所有可能的下一步候选，并且堆顶过期时不会被使用。本题的大小为 K 的小顶堆保存当前前 K 大边界；快速选择可做到平均线性是选择顺序，只关心第 K 大，不需要完整排序是候选集合来源。这道题的边界不能留到最后补丁式处理，实验时覆盖 K 为一、K 为 n、重复值、随机 pivot，记录堆顶是否过期、比较器是否让正确候选在堆顶、K 或窗口大小为边界值时是否稳定。

C++ 实现上，`std::priority_queue` 默认大顶堆，小顶堆需要比较器。堆中保存大对象时尽量保存下标或轻量记录；若不能删除任意元素，就用懒删除或过期检查。如果追问变成“若数据流持续到来，堆方案比一次性快速选择更自然”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 295 数据流的中位数

先每轮扫描所有候选找极值，确认答案选择的对象。本题的模型是数据流需要动态维护较小一半和较大一半；暴力版慢在动态候选集合不断变化，却每次重新找最值。本题的优化点是大顶堆保存较小一半，小顶堆保存较大一半，大小差不超过一，因此堆中只能放足以比较下一步选择的轻量状态。

堆只保证堆顶，不保证内部全局有序。本题的大顶堆保存较小一半，小顶堆保存较大一半，大小差不超过一要转成堆顶可回答的问题；如果元素会过期，还要在真正使用堆顶前清理或跳过旧状态。

放到“堆与动态极值”这条主线里看，关键原则是：堆把插入和弹出极值控制在对数复杂度。Top K 用固定大小堆维护边界，合并多路序列用堆保存每一路当前头部，数据流中位数用双堆维护两半。

证明堆题时，要说明堆覆盖了所有可能的下一步候选，并且堆顶过期时不会被使用。本题的大顶堆保存较小一半，小顶堆保存较大一半，大小差不超过一是选择顺序，数据流需要动态维护较小一半和较大一半是候选集合来源。这道题的边界不能留到最后补丁式处理，实验时覆盖偶数个元素、负数、重复值、平均值溢出，记录堆顶是否过期、比较器是否让正确候选在堆顶、K 或窗口大小为边界值时是否稳定。

C++ 实现上，`std::priority_queue` 默认大顶堆，小顶堆需要比较器。堆中保存大对象时尽量保存下标或轻量记录；若不能删除任意元素，就用懒删除或过期检查。如果追问变成“若还要删除任意元素，需要延迟删除哈希表”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 347 前 K 个高频元素

先每轮扫描所有候选找极值，确认答案选择的对象。本题的模型是先统计频次，再只维护前 K 个候选；暴力版慢在动态候选集合不断变化，却每次重新找最值。本题的优化点是小顶堆大小超过 K 就弹出最低频；桶排序利用频次上界，因此堆中只能放足以比较下一步选择的轻量状态。

堆只保证堆顶，不保证内部全局有序。本题的小顶堆大小超过 K 就弹出最低频；桶排序利用频次上界要转成堆顶可回答的问题；如果元素会过期，还要在真正使用堆顶前清理或跳过旧状态。放到“堆与动态极值”这条主线里看，关键原则是：堆把插入和弹出极值控制在对数复杂度。Top K 用固定大小堆维护边界，合并多路序列用堆保存每一路当前头部，数据流中位数用双堆维护两半。

证明堆题时，要说明堆覆盖了所有可能的下一步候选，并且堆顶过期时不会被使用。本题的小顶堆大小超过 K 就弹出最低频；桶排序利用频次上界是选择顺序，先统计频次，再只维护前 K 个候选是候选集合来源。这道题的边界不能留到最后补丁式处理，实验时覆盖 K 等于不同元素数、频次相同、结果顺序不要求，记录堆顶是否过期、比较器是否让正确候选在堆顶、K 或窗口大小为边界值时是否稳定。

C++ 实现上，`std::priority_queue` 默认大顶堆，小顶堆需要比较器。堆中保存大对象时尽量保存下标或轻量记录；若不能删除任意元素，就用懒删除或过期检查。如果追问变成“若要按频次和字典序排序，比较器要同时处理两个键”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 621 任务调度器

先每轮扫描所有候选找极值，确认答案选择的对象。本题的模型是相同任务之间有冷却间隔，核心受最高频任务骨架限制；暴力版慢在动态候选集合不断变化，却每次重新找最值。本题的优化点是可用大顶堆模拟，也可用最高频公式计算最短时间，因此堆中只能放足以比较下一步选择的轻量状态。

堆只保证堆顶，不保证内部全局有序。本题的可用大顶堆模拟，也可用最高频公式计算最短时间要转成堆顶可回答的问题；如果元素会过期，还要在真正使用堆顶前清理或跳过旧状态。放到“堆与动态极值”这条主线里看，关键原则是：堆把插入和弹出极值控制在对数复杂度。Top K 用固定大小堆维护边界，合并多路序列用堆保存每一路当前头部，数据流中位数用双堆维护两半。

证明堆题时，要说明堆覆盖了所有可能的下一步候选，并且堆顶过期时不会被使用。本题的可用大顶堆模拟，也可用最高频公式计算最短时间是选择顺序，相同任务之间有冷却间隔，核心受最高频任务骨架限制是候选集合来源。这道题的边界不能留到最后补丁式处理，实验时覆盖冷却为零、多个最高频、空闲被其他任务填满，记录堆顶是否过期、比较器是否让正确候选在堆顶、K 或窗口大小为边界值时是否稳定。

C++ 实现上，`std::priority_queue` 默认大顶堆，小顶堆需要比较器。堆中保存大对象时尽量保存下标或轻量记录；若不能删除任意元素，就用懒删除或过期检查。如果追问变成“若任务有不同释放时间或执行时长，就变成更一般的调度问题”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 973 最接近原点的 K 个点

先每轮扫描所有候选找极值，确认答案选择的对象。本题的模型是距离只需比较平方，避免开方误差和成本；暴力版慢在动态候选集合不断变化，却每次重新找最值。本题的优化点是大小为 K 的大

顶堆保存当前最近 K 个点中最远者，因此堆中只能放足以比较下一步选择的轻量状态。

堆只保证堆顶，不保证内部全局有序。本题的大小为 K 的大顶堆保存当前最近 K 个点中最远者要转成堆顶可回答的问题；如果元素会过期，还要在真正使用堆顶前清理或跳过旧状态。放到“堆与动态极值”这条主线里看，关键原则是：堆把插入和弹出极值控制在对数复杂度。Top K 用固定大小堆维护边界，合并多路序列用堆保存每一路当前头部，数据流中位数用双堆维护两半。

证明堆题时，要说明堆覆盖了所有可能的下一步候选，并且堆顶过期时不会被使用。本题的大小为 K 的大顶堆保存当前最近 K 个点中最远者是选择顺序，距离只需比较平方，避免开方误差和成本是候选集合来源。这道题的边界不能留到最后补丁式处理，实验时覆盖 K 为一、 K 为全部、距离相同、坐标平方溢出，记录堆顶是否过期、比较器是否让正确候选在堆顶、 K 或窗口大小为边界值时是否稳定。

C++ 实现上，`std::priority_queue` 默认大顶堆，小顶堆需要比较器。堆中保存大对象时尽量保存下标或轻量记录；若不能删除任意元素，就用懒删除或过期检查。如果追问变成“快速选择可平均线性，但堆更适合数据流”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

链表与指针重连工作坊

先用数组或多次扫描得到直观答案，再把操作改成节点重连。链表题的关键是保存后继、明确前驱和当前段边界，不能靠值交换掩盖指针关系。

LeetCode 2 两数相加

先把链表节点拷到数组里得到一个容易验证的版本，再回到节点重连。本题的模型是链表按低位到高位保存数字，本质是竖式加法而不是整数转换；暴力版能帮助确认节点顺序或节点身份。优化时关注维护两个游标和进位，任一链未结束或进位非零都继续生成节点，不要用值交换掩盖链表结构要求。

链表优化要把指针角色固定下来。本题用维护两个游标和进位，任一链未结束或进位非零都继续生成节点推进，至少要画出前驱、当前节点、后继节点和已处理链尾。每次改 next 前先保存后继，防止链断掉。放到“链表与指针重连”这条主线里看，关键原则是：优化来自哨兵节点、快慢指针、前驱指针和局部反转。链表题的核心不是值交换，而是保持断开和接回的顺序正确。

链表题的正确性靠结构不变量：已经处理好的链仍然连通，未处理部分仍然可达，二者之间只有明确连接点。本题的维护两个游标和进位，任一链未结束或进位非零都继续生成节点每执行一次，都不能破坏这三个条件。这道题的边界不能留到最后补丁式处理，实验时覆盖两链长度不同、最后进位、输入为零、不能用内置整数承载超长数字，每步画出前驱、当前、后继和下一段头。链表题不要只看输出值，还要检查节点是否丢失或成环。

C++ 实现上，平台管理输入节点时不要随意 `delete`。使用虚拟头节点统一头部边界；每次重连前保存后继。比较交点时比较节点地址，不比较节点值。如果追问变成“若链表高位在前，可以用栈反向处理或先反转链表”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 141 环形链表

先把链表节点拷到数组里得到一个容易验证的版本，再回到节点重连。本题的模型是快慢指针在有环时必然相遇，无环时快指针先到空；暴力版能帮助确认节点顺序或节点身份。优化时关注不用哈希表也能常数空间检测环，不要用值交换掩盖链表结构要求。

链表优化要把指针角色固定下来。本题用不用哈希表也能常数空间检测环推进，至少要画出前驱、当前节点、后继节点和已处理链尾。每次改 next 前先保存后继，防止链断掉。放到“链表与指针重连”这条主线里看，关键原则是：优化来自哨兵节点、快慢指针、前驱指针和局部反转。链表题的核心不是值交换，而是保持断开和接回的顺序正确。

链表题的正确性靠结构不变量：已经处理好的链仍然连通，未处理部分仍然可达，二者之间只有明确连接点。本题的不用哈希表也能常数空间检测环每执行一次，都不能破坏这三个条件。这道题的边界不能留到最后补丁式处理，实验时覆盖空链、单节点自环、两节点环、尾部无环，每步画出前驱、当前、后继和下一段头。链表题不要只看输出值，还要检查节点是否丢失或成环。

C++ 实现上，平台管理输入节点时不要随意 `delete`。使用虚拟头节点统一头部边界；每次重连前保存后继。比较交点时比较节点地址，不比较节点值。如果追问变成“若要求环入口，用相遇

后双指针同步走”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 142 环形链表 II

先把链表节点拷到数组里得到一个容易验证的版本，再回到节点重连。本题的模型是相遇点到入口的距离关系来自快慢指针速度差；暴力版能帮助确认节点顺序或节点身份。优化时关注相遇后一个指针回头，两个指针每次走一步，再次相遇就是入口，不要用值交换掩盖链表结构要求。

链表优化要把指针角色固定下来。本题用相遇后一个指针回头，两个指针每次走一步，再次相遇就是入口推进，至少要画出前驱、当前节点、后继节点和已处理链尾。每次改 next 前先保存后继，防止链断掉。放到“链表与指针重连”这条主线里看，关键原则是：优化来自哨兵节点、快慢指针、前驱指针和局部反转。链表题的核心不是值交换，而是保持断开和接回的顺序正确。

链表题的正确性靠结构不变量：已经处理好的链仍然连通，未处理部分仍然可达，二者之间只有明确连接点。本题的相遇后一个指针回头，两个指针每次走一步，再次相遇就是入口每执行一次，都不能破坏这三个条件。这道题的边界不能留到最后补丁式处理，实验时覆盖入口在头节点、长尾短环、无环、单节点环，每步画出前驱、当前、后继和下一段头。链表题不要只看输出值，还要检查节点是否丢失或成环。

C++ 实现上，平台管理输入节点时不要随意 `delete`。使用虚拟头节点统一头部边界；每次重连前保存后继。比较交点时比较节点地址，不比较节点值。如果追问变成“也可用哈希集合，但空间更高”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 146 LRU 缓存

先把链表节点拷到数组里得到一个容易验证的版本，再回到节点重连。本题的模型是哈希表负责按 key 定位，双向链表负责维护新旧顺序；暴力版能帮助确认节点顺序或节点身份。优化时关注访问和更新都把节点移动到头部，容量满时删除尾部最旧节点，不要用值交换掩盖链表结构要求。

链表优化要把指针角色固定下来。本题用访问和更新都把节点移动到头部，容量满时删除尾部最旧节点推进，至少要画出前驱、当前节点、后继节点和已处理链尾。每次改 next 前先保存后继，防止链断掉。放到“链表与指针重连”这条主线里看，关键原则是：优化来自哨兵节点、快慢指针、前驱指针和局部反转。链表题的核心不是值交换，而是保持断开和接回的顺序正确。

链表题的正确性靠结构不变量：已经处理好的链仍然连通，未处理部分仍然可达，二者之间只有明确连接点。本题的访问和更新都把节点移动到头部，容量满时删除尾部最旧节点每执行一次，都不能破坏这三个条件。这道题的边界不能留到最后补丁式处理，实验时覆盖容量为一、重复 put、get 不存在、更新已有值，每步画出前驱、当前、后继和下一段头。链表题不要只看输出值，还要检查节点是否丢失或成环。

C++ 实现上，平台管理输入节点时不要随意 `delete`。使用虚拟头节点统一头部边界；每次重连前保存后继。比较交点时比较节点地址，不比较节点值。如果追问变成“C++ 可用 `std::list` 加迭代器，也可手写双向链表理解所有权”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 148 排序链表

先把链表节点拷到数组里得到一个容易验证的版本，再回到节点重连。本题的模型是链表不能随机访问，适合归并排序而不是快速排序数组 partition；暴力版能帮助确认节点顺序或节点身份。优化时关注自底向上迭代归并能避免递归栈，同时保持 $O(n \log n)$ ，不要用值交换掩盖链表结构要求。

链表优化要把指针角色固定下来。本题用自底向上迭代归并能避免递归栈，同时保持 $O(n \log n)$ 推进，至少要画出前驱、当前节点、后继节点和已处理链尾。每次改 next 前先保存后继，防止链断掉。放到“链表与指针重连”这条主线里看，关键原则是：优化来自哨兵节点、快慢指针、前驱指针和局部反转。链表题的核心不是值交换，而是保持断开和接回的顺序正确。

链表题的正确性靠结构不变量：已经处理好的链仍然连通，未处理部分仍然可达，二者之间只有明确连接点。本题的自底向上迭代归并能避免递归栈，同时保持 $O(n \log n)$ 每执行一次，都不能破坏这三个条件。这道题的边界不能留到最后补丁式处理，实验时覆盖空链、单节点、重复值、奇数长度、切断子链，每步画出前驱、当前、后继和下一段头。链表题不要只看输出值，还要检查节点是否丢失或成环。

C++ 实现上，平台管理输入节点时不要随意 `delete`。使用虚拟头节点统一头部边界；每次重连前保存后继。比较交点时比较节点地址，不比较节点值。如果追问变成“若把值复制到数组排序，会改变节点身份语义”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 `value` 语义，还是换成另一类结构。

LeetCode 160 相交链表

先把链表节点拷到数组里得到一个容易验证的版本，再回到节点重连。本题的模型是相交是节点地址相同，不是节点值相同；暴力版能帮助确认节点顺序或节点身份。优化时关注两个指针走完各自链表后切换到另一条链，走过总长度相同后相遇，不要用值交换掩盖链表结构要求。

链表优化要把指针角色固定下来。本题用两个指针走完各自链表后切换到另一条链，走过总长度相同后相遇推进，至少要画出前驱、当前节点、后继节点和已处理链尾。每次改 `next` 前先保存后继，防止链断掉。放到“链表与指针重连”这条主线里看，关键原则是：优化来自哨兵节点、快慢指针、前驱指针和局部反转。链表题的核心不是值交换，而是保持断开和接回的顺序正确。

链表题的正确性靠结构不变量：已经处理好的链仍然连通，未处理部分仍然可达，二者之间只有明确连接点。本题的两个指针走完各自链表后切换到另一条链，走过总长度相同后相遇每执行一次，都不能破坏这三个条件。这道题的边界不能留到最后补丁式处理，实验时覆盖不相交、头节点相交、尾节点相交、长度差很大，每步画出前驱、当前、后继和下一段头。链表题不要只看输出值，还要检查节点是否丢失或成环。

C++ 实现上，平台管理输入节点时不要随意 `delete`。使用虚拟头节点统一头部边界；每次重连前保存后继。比较交点时比较节点地址，不比较节点值。如果追问变成“哈希集合更直观但空间更高”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 `value` 语义，还是换成另一类结构。

LeetCode 206 反转链表

先把链表节点拷到数组里得到一个容易验证的版本，再回到节点重连。本题的模型是每次把当前节点的 `next` 指向已经反转好的前缀头；暴力版能帮助确认节点顺序或节点身份。优化时关注先保存后继，再改指针，再推进 `previous` 和 `current`，不要用值交换掩盖链表结构要求。

链表优化要把指针角色固定下来。本题用先保存后继，再改指针，再推进 `previous` 和 `current` 推进，至少要画出前驱、当前节点、后继节点和已处理链尾。每次改 `next` 前先保存后继，防止链断掉。放到“链表与指针重连”这条主线里看，关键原则是：优化来自哨兵节点、快慢指针、前驱指针和局部反转。链表题的核心不是值交换，而是保持断开和接回的顺序正确。

链表题的正确性靠结构不变量：已经处理好的链仍然连通，未处理部分仍然可达，二者之间只有明确连接点。本题的先保存后继，再改指针，再推进 `previous` 和 `current` 每执行一次，都不能破坏这三个条件。这道题的边界不能留到最后补丁式处理，实验时覆盖空链、单节点、两个节点、不要丢失剩余链，每步画出前驱、当前、后继和下一段头。链表题不要只看输出值，还要检查节点是否丢失或成环。

C++ 实现上，平台管理输入节点时不要随意 `delete`。使用虚拟头节点统一头部边界；每次重连前保存后继。比较交点时比较节点地址，不比较节点值。如果追问变成“K 组反转是在这段局部反转外增加分组边界”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 `value` 语义，还是换成另一类结构。

二叉树与树形状态工作坊

先想每个节点重复扫描子树会得到什么信息，再把这些信息压成节点向父节点返回的状态。树题要区分自顶向下约束、后序汇总和按层传播。

LeetCode 94 二叉树中序遍历

先想最直接的树遍历会在每个节点重复求什么。本题的模型是中序访问顺序是左子树、根、右子树，BST 中会得到有序序列；如果每个节点都重新扫描子树，慢点就会非常明显。本题要压缩的状态是用显式栈模拟一路向左，再回退访问根并转向右子树，也就是节点和父子方向之间真正传递的信息。

树题要先判定状态方向。本题的用显式栈模拟一路向左，再回退访问根并转向右子树可能是自顶向下的约束、后序向上的汇总，或队列中的层号。返回值或栈元素不能只有节点指针，还要带上题目需要的信息。放到“二叉树与树形状态”这条主线里看，关键原则是：树形 DP 把每个节点看

成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。

树题常用对子树归纳。假设子节点状态已经正确，父节点只按用显式栈模拟一路向左，再回退访问根并转向右子树合并或传递信息。本题的中序访问顺序是左子树、根、右子树，BST 中会得到有序序列决定空节点、叶子和极端链式树如何处理。这道题的边界不能留到最后补丁式处理，实验时覆盖空树、单节点、链式左树、链式右树，尤其关注空节点、单节点、链式树和全负值。递归版本和显式栈版本可以互相对拍。

C++ 实现上，极端深树可能造成递归栈风险，工程写法可以用 `std::vector<TreeNode*>` 或队列显式保存状态。BST 边界最好用更宽整数或可选前驱值。如果追问变成“Morris 遍历可做到常数额外空间，但会临时改树结构”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 98 验证二叉搜索树

先想最直接的树遍历会在每个节点重复求什么。本题的模型是 BST 约束是全局上下界，不是只比较父子节点；如果每个节点都重新扫描子树，慢点就会非常明显。本题要压缩的状态是中序严格递增或显式传递上下界都可以；中序版本要保存前一个访问值，也就是节点和父子方向之间真正传递的信息。

树题要先判定状态方向。本题的中序严格递增或显式传递上下界都可以；中序版本要保存前一个访问值可能是自顶向下的约束、后序向上的汇总，或队列中的层号。返回值或栈元素不能只有节点指针，还要带上题目需要的信息。放到“二叉树与树形状态”这条主线里看，关键原则是：树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。

树题常用对子树归纳。假设子节点状态已经正确，父节点只按中序严格递增或显式传递上下界都可以；中序版本要保存前一个访问值合并或传递信息。本题的 BST 约束是全局上下界，不是只比较父子节点决定空节点、叶子和极端链式树如何处理。这道题的边界不能留到最后补丁式处理，实验时覆盖重复值、int 最小最大值、局部合法但全局非法，尤其关注空节点、单节点、链式树和全负值。递归版本和显式栈版本可以互相对拍。

C++ 实现上，极端深树可能造成递归栈风险，工程写法可以用 `std::vector<TreeNode*>` 或队列显式保存状态。BST 边界最好用更宽整数或可选前驱值。如果追问变成“若允许重复值，需要明确重复放左还是放右”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 102 二叉树层序遍历

先想最直接的树遍历会在每个节点重复求什么。本题的模型是队列在每轮开始时保存当前层所有节点；如果每个节点都重新扫描子树，慢点就会非常明显。本题要压缩的状态是记录 level size 固定层边界，避免下一层节点混入当前层，也就是节点和父子方向之间真正传递的信息。

树题要先判定状态方向。本题的记录 level size 固定层边界，避免下一层节点混入当前层可能是自顶向下的约束、后序向上的汇总，或队列中的层号。返回值或栈元素不能只有节点指针，还要带上题目需要的信息。放到“二叉树与树形状态”这条主线里看，关键原则是：树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。

树题常用对子树归纳。假设子节点状态已经正确，父节点只按记录 level size 固定层边界，避免下一层节点混入当前层合并或传递信息。本题的队列在每轮开始时保存当前层所有节点决定空节点、叶子和极端链式树如何处理。这道题的边界不能留到最后补丁式处理，实验时覆盖空树、只有根、最后一层不满、左右顺序，尤其关注空节点、单节点、链式树和全负值。递归版本和显式栈版本可以互相对拍。

C++ 实现上，极端深树可能造成递归栈风险，工程写法可以用 `std::vector<TreeNode*>` 或队列显式保存状态。BST 边界最好用更宽整数或可选前驱值。如果追问变成“锯齿形层序只是在每层输出顺序上增加方向控制”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 103 二叉树锯齿形层序遍历

先想最直接的树遍历会在每个节点重复求什么。本题的模型是 BFS 层边界不变，变化的是当前层写入结果的方向；如果每个节点都重新扫描子树，慢点就会非常明显。本题要压缩的状态是可以层内反转，也可以预分配数组按方向写入目标下标，也就是节点和父子方向之间真正传递的信息。

树题要先判定状态方向。本题的可以层内反转，也可以预分配数组按方向写入目标下标可能是自顶向下的约束、后序向上的汇总，或队列中的层号。返回值或栈元素不能只有节点指针，还要带上题目需要的信息。放到“二叉树与树形状态”这条主线里看，关键原则是：树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。

树题常用对子树归纳。假设子节点状态已经正确，父节点只按可以层内反转，也可以预分配数组按方向写入目标下标合并或传递信息。本题的 BFS 层边界不变，变化的是当前层写入结果的方向决定空节点、叶子和极端链式树如何处理。这道题的边界不能留到最后补丁式处理，实验时覆盖单层、偶数层、奇数层、空树，尤其关注空节点、单节点、链式树和全负值。递归版本和显式栈版本可以互相对拍。

C++ 实现上，极端深树可能造成递归栈风险，工程写法可以用 `std::vector<TreeNode*>` 或队列显式保存状态。BST 边界最好用更宽整数或可选前驱值。如果追问变成“若要求从底向上输出，收集后反转层列表即可”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 104 二叉树最大深度

先想最直接的树遍历会在每个节点重复求什么。本题的模型是深度可以看成层数，也可以作为栈中携带的状态；如果每个节点都重新扫描子树，慢点就会非常明显。本题要压缩的状态是显式栈保存节点和深度，避免极端链式树递归栈过深，也就是节点和父子方向之间真正传递的信息。

树题要先判定状态方向。本题的显式栈保存节点和深度，避免极端链式树递归栈过深可能是自顶向下的约束、后序向上的汇总，或队列中的层号。返回值或栈元素不能只有节点指针，还要带上题目需要的信息。放到“二叉树与树形状态”这条主线里看，关键原则是：树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。

树题常用对子树归纳。假设子节点状态已经正确，父节点只按显式栈保存节点和深度，避免极端链式树递归栈过深合并或传递信息。本题的深度可以看成层数，也可以作为栈中携带的状态决定空节点、叶子和极端链式树如何处理。这道题的边界不能留到最后补丁式处理，实验时覆盖空树返回零、根深度是一、链式树，尤其关注空节点、单节点、链式树和全负值。递归版本和显式栈版本可以互相对拍。

C++ 实现上，极端深树可能造成递归栈风险，工程写法可以用 `std::vector<TreeNode*>` 或队列显式保存状态。BST 边界最好用更宽整数或可选前驱值。如果追问变成“最小深度不能简单取左右最小，因为空子树不构成叶子路径”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 105 前序中序构造二叉树

先想最直接的树遍历会在每个节点重复求什么。本题的模型是前序给根，中序把左右子树切开；如果每个节点都重新扫描子树，慢点就会非常明显。本题要压缩的状态是用哈希表记录中序下标，避免每次线性寻找根位置，也就是节点和父子方向之间真正传递的信息。

树题要先判定状态方向。本题的用哈希表记录中序下标，避免每次线性寻找根位置可能是自顶向下的约束、后序向上的汇总，或队列中的层号。返回值或栈元素不能只有节点指针，还要带上题目需要的信息。放到“二叉树与树形状态”这条主线里看，关键原则是：树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。

树题常用对子树归纳。假设子节点状态已经正确，父节点只按用哈希表记录中序下标，避免每次线性寻找根位置合并或传递信息。本题的前序给根，中序把左右子树切开决定空节点、叶子和极端链式树如何处理。这道题的边界不能留到最后补丁式处理，实验时覆盖节点值唯一、空子树、左右子树长度计算、输入不合法，尤其关注空节点、单节点、链式树和全负值。递归版本和显式栈版本

可以互相对拍。

C++ 实现上, 极端深树可能造成递归栈风险, 工程写法可以用 `std::vector<TreeNode*>` 或队列显式保存状态。BST 边界最好用更宽整数或可选前驱值。如果追问变成“后序中序构造类似, 只是根来自后序末尾”, 先判断原来的状态是否仍然覆盖新答案集合, 再决定是微调边界、改 value 语义, 还是换成另一类结构。

LeetCode 106 中序后序构造二叉树

先想最直接的树遍历会在每个节点重复求什么。本题的模型是后序末尾是根, 中序位置确定左右子树规模; 如果每个节点都重新扫描子树, 慢点就会非常明显。本题要压缩的状态是构造时要小心后序左右区间边界, 避免切分错位, 也就是节点和父子方向之间真正传递的信息。

树题要先判定状态方向。本题的构造时要小心后序左右区间边界, 避免切分错位可能是自顶向下的约束、后序向上的汇总, 或队列中的层号。返回值或栈元素不能只有节点指针, 还要带上题目需要的信息。放到“二叉树与树形状态”这条主线里看, 关键原则是: 树形 DP 把每个节点看成子问题根; BFS 把层次边界显式放进队列; BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。

树题常用对子树归纳。假设子节点状态已经正确, 父节点只按构造时要小心后序左右区间边界, 避免切分错位合并或传递信息。本题的后序末尾是根, 中序位置确定左右子树规模决定空节点、叶子和极端链式树如何处理。这道题的边界不能留到最后补丁式处理, 实验时覆盖空树、单节点、全左链、全右链, 尤其关注空节点、单节点、链式树和全负值。递归版本和显式栈版本可以互相对拍。

C++ 实现上, 极端深树可能造成递归栈风险, 工程写法可以用 `std::vector<TreeNode*>` 或队列显式保存状态。BST 边界最好用更宽整数或可选前驱值。如果追问变成“若给前序和后序, 二叉树通常不能唯一确定, 除非额外限制”, 先判断原来的状态是否仍然覆盖新答案集合, 再决定是微调边界、改 value 语义, 还是换成另一类结构。

LeetCode 108 有序数组转二叉搜索树

先想最直接的树遍历会在每个节点重复求什么。本题的模型是有序数组中点作为根能让左右规模接近; 如果每个节点都重新扫描子树, 慢点就会非常明显。本题要压缩的状态是每次选择区间中点, 左右子区间构造左右子树, 也就是节点和父子方向之间真正传递的信息。

树题要先判定状态方向。本题的每次选择区间中点, 左右子区间构造左右子树可能是自顶向下的约束、后序向上的汇总, 或队列中的层号。返回值或栈元素不能只有节点指针, 还要带上题目需要的信息。放到“二叉树与树形状态”这条主线里看, 关键原则是: 树形 DP 把每个节点看成子问题根; BFS 把层次边界显式放进队列; BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。

树题常用对子树归纳。假设子节点状态已经正确, 父节点只按每次选择区间中点, 左右子区间构造左右子树合并或传递信息。本题的有序数组中点作为根能让左右规模接近决定空节点、叶子和极端链式树如何处理。这道题的边界不能留到最后补丁式处理, 实验时覆盖空区间、偶数长度选左中还是右中、高度平衡定义, 尤其关注空节点、单节点、链式树和全负值。递归版本和显式栈版本可以互相对拍。

C++ 实现上, 极端深树可能造成递归栈风险, 工程写法可以用 `std::vector<TreeNode*>` 或队列显式保存状态。BST 边界最好用更宽整数或可选前驱值。如果追问变成“工程上递归可读, 若严格避免递归可用队列保存区间和父指针”, 先判断原来的状态是否仍然覆盖新答案集合, 再决定是微调边界、改 value 语义, 还是换成另一类结构。

LeetCode 110 平衡二叉树

先想最直接的树遍历会在每个节点重复求什么。本题的模型是每个节点需要知道子树高度以及是否已经失衡; 如果每个节点都重新扫描子树, 慢点就会非常明显。本题要压缩的状态是后序汇总时一旦子树失衡就向上返回失败标记, 避免重复算高度, 也就是节点和父子方向之间真正传递的信息。

树题要先判定状态方向。本题的后序汇总时一旦子树失衡就向上返回失败标记, 避免重复算高度可能是自顶向下的约束、后序向上的汇总, 或队列中的层号。返回值或栈元素不能只有节点指针, 还要带上题目需要的信息。放到“二叉树与树形状态”这条主线里看, 关键原则是: 树形 DP 把每个节点看成子问题根; BFS 把层次边界显式放进队列; BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。

树题常用对子树归纳。假设子节点状态已经正确，父节点只按后序汇总时一旦子树失衡就向上返回失败标记，避免重复算高度合并或传递信息。本题的每个节点需要知道子树高度以及是否已经失衡决定空节点、叶子和极端链式树如何处理。这道题的边界不能留到最后补丁式处理，实验时覆盖空树、单节点、左右高度差正好一、链式树，尤其关注空节点、单节点、链式树和全负值。递归版本和显式栈版本可以互相对拍。

C++ 实现上，极端深树可能造成递归栈风险，工程写法可以用 `std::vector<TreeNode*>` 或队列显式保存状态。BST 边界最好用更宽整数或可选前驱值。如果追问变成“可以把返回值设计成 optional 高度，空表示不平衡”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 111 二叉树最小深度

先想最直接的树遍历会在每个节点重复求什么。本题的模型是最小深度是到最近叶子的节点数，不是左右深度简单取最小；如果每个节点都重新扫描子树，慢点就会非常明显。本题要压缩的状态是 BFS 第一次遇到叶子就是最小深度，因为按层扩展，也就是节点和父子方向之间真正传递的信息。

树题要先判定状态方向。本题的 BFS 第一次遇到叶子就是最小深度，因为按层扩展可能是自顶向下的约束、后序向上的汇总，或队列中的层号。返回值或栈元素不能只有节点指针，还要带上题目需要的信息。放到“二叉树与树形状态”这条主线里看，关键原则是：树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。

树题常用对子树归纳。假设子节点状态已经正确，父节点只按 BFS 第一次遇到叶子就是最小深度，因为按层扩展合并或传递信息。本题的最小深度是到最近叶子的节点数，不是左右深度简单取最小决定空节点、叶子和极端链式树如何处理。这道题的边界不能留到最后补丁式处理，实验时覆盖只有左子树、只有右子树、根为空、根是叶子，尤其关注空节点、单节点、链式树和全负值。递归版本和显式栈版本可以互相对拍。

C++ 实现上，极端深树可能造成递归栈风险，工程写法可以用 `std::vector<TreeNode*>` 或队列显式保存状态。BST 边界最好用更宽整数或可选前驱值。如果追问变成“DFS 写法必须特殊处理空子树不能作为最小路径”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 112 路径总和

先想最直接的树遍历会在每个节点重复求什么。本题的模型是路径必须从根到叶子，状态是剩余目标值；如果每个节点都重新扫描子树，慢点就会非常明显。本题要压缩的状态是沿路径减去当前节点值，到叶子时检查剩余是否等于叶子值，也就是节点和父子方向之间真正传递的信息。

树题要先判定状态方向。本题的沿路径减去当前节点值，到叶子时检查剩余是否等于叶子值可能是自顶向下的约束、后序向上的汇总，或队列中的层号。返回值或栈元素不能只有节点指针，还要带上题目需要的信息。放到“二叉树与树形状态”这条主线里看，关键原则是：树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。

树题常用对子树归纳。假设子节点状态已经正确，父节点只按沿路径减去当前节点值，到叶子时检查剩余是否等于叶子值合并或传递信息。本题的路径必须从根到叶子，状态是剩余目标值决定空节点、叶子和极端链式树如何处理。这道题的边界不能留到最后补丁式处理，实验时覆盖负数节点、目标为零、非叶子提前达到目标、空树，尤其关注空节点、单节点、链式树和全负值。递归版本和显式栈版本可以互相对拍。

C++ 实现上，极端深树可能造成递归栈风险，工程写法可以用 `std::vector<TreeNode*>` 或队列显式保存状态。BST 边界最好用更宽整数或可选前驱值。如果追问变成“若要求列出所有路径，需要保存路径数组并在回退时弹出”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 113 路径总和 II

先想最直接的树遍历会在每个节点重复求什么。本题的模型是相比判定题，状态还要保存当前路径；如果每个节点都重新扫描子树，慢点就会非常明显。本题要压缩的状态是到叶子且和满足时复制路径；回溯时恢复路径，也就是节点和父子方向之间真正传递的信息。

树题要先判定状态方向。本题的到叶子且和满足时复制路径；回溯时恢复路径可能是自顶向下的约束、后序向上的汇总，或队列中的层号。返回值或栈元素不能只有节点指针，还要带上题目需要的信息。放到“二叉树与树形状态”这条主线里看，关键原则是：树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。

树题常用对子树归纳。假设子节点状态已经正确，父节点只按到叶子且和满足时复制路径；回溯时恢复路径合并或传递信息。本题的相比判定题，状态还要保存当前路径决定空节点、叶子和极端链式树如何处理。这道题的边界不能留到最后补丁式处理，实验时覆盖多个答案、负数、空树、路径数组复制时机，尤其关注空节点、单节点、链式树和全负值。递归版本和显式栈版本可以互相对拍。

C++ 实现上，极端深树可能造成递归栈风险，工程写法可以用 `std::vector<TreeNode*>` 或队列显式保存状态。BST 边界最好用更宽整数或可选前驱值。如果追问变成“若路径不要求从根到叶，模型会变成前缀和加哈希”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 114 二叉树展开为链表

先想最直接的树遍历会在每个节点重复求什么。本题的模型是目标是按前序顺序把树原地改成右指针链；如果每个节点都重新扫描子树，慢点就会非常明显。本题要压缩的状态是显式栈按右后左先入顺序处理，逐个接到前驱右侧，也就是节点和父子方向之间真正传递的信息。

树题要先判定状态方向。本题的显式栈按右后左先入顺序处理，逐个接到前驱右侧可能是自顶向下的约束、后序向上的汇总，或队列中的层号。返回值或栈元素不能只有节点指针，还要带上题目需要的信息。放到“二叉树与树形状态”这条主线里看，关键原则是：树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。

树题常用对子树归纳。假设子节点状态已经正确，父节点只按显式栈按右后左先入顺序处理，逐个接到前驱右侧合并或传递信息。本题的目标是按前序顺序把树原地改成右指针链决定空节点、叶子和极端链式树如何处理。这道题的边界不能留到最后补丁式处理，实验时覆盖空树、单节点、原有右子树不能丢、左指针置空，尤其关注空节点、单节点、链式树和全负值。递归版本和显式栈版本可以互相对拍。

C++ 实现上，极端深树可能造成递归栈风险，工程写法可以用 `std::vector<TreeNode*>` 或队列显式保存状态。BST 边界最好用更宽整数或可选前驱值。如果追问变成“也可用寻找左子树最右节点的原地方法”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 124 二叉树最大路径和

先想最直接的树遍历会在每个节点重复求什么。本题的模型是路径可以从任意节点到任意节点，但不能分叉向父节点贡献两边；如果每个节点都重新扫描子树，慢点就会非常明显。本题要压缩的状态是每个节点向父亲贡献单边最大增益，全局答案可用左右两边加当前节点更新，也就是节点和父子方向之间真正传递的信息。

树题要先判定状态方向。本题的每个节点向父亲贡献单边最大增益，全局答案可用左右两边加当前节点更新可能是自顶向下的约束、后序向上的汇总，或队列中的层号。返回值或栈元素不能只有节点指针，还要带上题目需要的信息。放到“二叉树与树形状态”这条主线里看，关键原则是：树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。

树题常用对子树归纳。假设子节点状态已经正确，父节点只按每个节点向父亲贡献单边最大增益，全局答案可用左右两边加当前节点更新合并或传递信息。本题的路径可以从任意节点到任意节点，但不能分叉向父节点贡献两边决定空节点、叶子和极端链式树如何处理。这道题的边界不能留到最后补丁式处理，实验时覆盖全负数、单节点、左右增益为负时取零、答案初始值，尤其关注空节点、单节点、链式树和全负值。递归版本和显式栈版本可以互相对拍。

C++ 实现上，极端深树可能造成递归栈风险，工程写法可以用 `std::vector<TreeNode*>` 或队列显式保存状态。BST 边界最好用更宽整数或可选前驱值。如果追问变成“若路径必须从根到叶，状态和答案位置完全不同”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改

value 语义，还是换成另一类结构。

LeetCode 199 二叉树右视图

先想最直接的树遍历会在每个节点重复求什么。本题的模型是每层从右侧能看到的就是该层最后访问或最先访问的右侧节点；如果每个节点都重新扫描子树，慢点就会非常明显。本题要压缩的状态是 BFS 固定层边界，取每层最后一个；或 DFS 按右优先首次到达深度，也就是节点和父子方向之间真正传递的信息。

树题要先判定状态方向。本题的 BFS 固定层边界，取每层最后一个；或 DFS 按右优先首次到达深度可能是自顶向下的约束、后序向上的汇总，或队列中的层号。返回值或栈元素不能只有节点指针，还要带上题目需要的信息。放到“二叉树与树形状态”这条主线里看，关键原则是：树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。

树题常用对子树归纳。假设子节点状态已经正确，父节点只按 BFS 固定层边界，取每层最后一个；或 DFS 按右优先首次到达深度合并或传递信息。本题的每层从右侧能看到的就是该层最后访问或最先访问的右侧节点决定空节点、叶子和极端链式树如何处理。这道题的边界不能留到最后补丁式处理，实验时覆盖空树、左链、右链、左右交错，尤其关注空节点、单节点、链式树和全负值。递归版本和显式栈版本可以互相对拍。

C++ 实现上，极端深树可能造成递归栈风险，工程写法可以用 `std::vector<TreeNode*>` 或队列显式保存状态。BST 边界最好用更宽整数或可选前驱值。如果追问变成“若求左视图，只改变层内取值方向”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 208 实现 Trie

先想最直接的树遍历会在每个节点重复求什么。本题的模型是前缀树把字符串公共前缀共享成路径；如果每个节点都重新扫描子树，慢点就会非常明显。本题要压缩的状态是插入和查询都逐字符沿边移动，不存在的边按需要创建，也就是节点和父子方向之间真正传递的信息。

树题要先判定状态方向。本题的插入和查询都逐字符沿边移动，不存在的边按需要创建可能是自顶向下的约束、后序向上的汇总，或队列中的层号。返回值或栈元素不能只有节点指针，还要带上题目需要的信息。放到“二叉树与树形状态”这条主线里看，关键原则是：树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。

树题常用对子树归纳。假设子节点状态已经正确，父节点只按插入和查询都逐字符沿边移动，不存在的边按需要创建合并或传递信息。本题的前缀树把字符串公共前缀共享成路径决定空节点、叶子和极端链式树如何处理。这道题的边界不能留到最后补丁式处理，实验时覆盖空字符串约定、单词结束标记、前缀存在不等于单词存在，尤其关注空节点、单节点、链式树和全负值。递归版本和显式栈版本可以互相对拍。

C++ 实现上，极端深树可能造成递归栈风险，工程写法可以用 `std::vector<TreeNode*>` 或队列显式保存状态。BST 边界最好用更宽整数或可选前驱值。如果追问变成“若字符集很大，用哈希子节点；小写字母用数组更快”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 211 添加与搜索单词

先想最直接的树遍历会在每个节点重复求什么。本题的模型是点号通配符让查询可能分叉，但前缀树仍能剪掉不匹配前缀；如果每个节点都重新扫描子树，慢点就会非常明显。本题要压缩的状态是普通字符沿唯一边走，点号枚举当前节点所有孩子，也就是节点和父子方向之间真正传递的信息。

树题要先判定状态方向。本题的普通字符沿唯一边走，点号枚举当前节点所有孩子可能是自顶向下的约束、后序向上的汇总，或队列中的层号。返回值或栈元素不能只有节点指针，还要带上题目需要的信息。放到“二叉树与树形状态”这条主线里看，关键原则是：树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。

树题常用对子树归纳。假设子节点状态已经正确，父节点只按普通字符沿唯一边走，点号枚举当前节点所有孩子合并或传递信息。本题的点号通配符让查询可能分叉，但前缀树仍能剪掉不匹配

前缀决定空节点、叶子和极端链式树如何处理。这道题的边界不能留到最后补丁式处理，实验时覆盖连续通配符、空结果、单词结束标记、字符集大小，尤其关注空节点、单节点、链式树和全负值。递归版本和显式栈版本可以互相对拍。

C++ 实现上，极端深树可能造成递归栈风险，工程写法可以用 `std::vector<TreeNode*>` 或队列显式保存状态。BST 边界最好用更宽整数或可选前驱值。如果追问变成“大量通配符会接近遍历整棵 Trie，需要规模约束”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 226 翻转二叉树

先想最直接的树遍历会在每个节点重复求什么。本题的模型是每个节点交换左右孩子即可，遍历顺序不影响最终结构；如果每个节点都重新扫描子树，慢点就会非常明显。本题要压缩的状态是显式队列或栈逐个节点交换，避免递归深度，也就是节点和父子方向之间真正传递的信息。

树题要先判定状态方向。本题的显式队列或栈逐个节点交换，避免递归深度可能是自顶向下的约束、后序向上的汇总，或队列中的层号。返回值或栈元素不能只有节点指针，还要带上题目需要的信息。放到“二叉树与树形状态”这条主线里看，关键原则是：树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。

树题常用对子树归纳。假设子节点状态已经正确，父节点只按显式队列或栈逐个节点交换，避免递归深度合并或传递信息。本题的每个节点交换左右孩子即可，遍历顺序不影响最终结构决定空节点、叶子和极端链式树如何处理。这道题的边界不能留到最后补丁式处理，实验时覆盖空树、单节点、只有一侧子树、重复值，尤其关注空节点、单节点、链式树和全负值。递归版本和显式栈版本可以互相对拍。

C++ 实现上，极端深树可能造成递归栈风险，工程写法可以用 `std::vector<TreeNode*>` 或队列显式保存状态。BST 边界最好用更宽整数或可选前驱值。如果追问变成“这题简单但适合训练树节点指针修改”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 230 二叉搜索树中第 K 小

先想最直接的树遍历会在每个节点重复求什么。本题的模型是 BST 中序遍历按升序访问节点；如果每个节点都重新扫描子树，慢点就会非常明显。本题要压缩的状态是显式栈中序，访问第 k 个节点时返回，也就是节点和父子方向之间真正传递的信息。

树题要先判定状态方向。本题的显式栈中序，访问第 k 个节点时返回可能是自顶向下的约束、后序向上的汇总，或队列中的层号。返回值或栈元素不能只有节点指针，还要带上题目需要的信息。放到“二叉树与树形状态”这条主线里看，关键原则是：树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。

树题常用对子树归纳。假设子节点状态已经正确，父节点只按显式栈中序，访问第 k 个节点时返回合并或传递信息。本题的 BST 中序遍历按升序访问节点决定空节点、叶子和极端链式树如何处理。这道题的边界不能留到最后补丁式处理，实验时覆盖 k 为一、k 为节点数、树不平衡、是否有重复值，尤其关注空节点、单节点、链式树和全负值。递归版本和显式栈版本可以互相对拍。

C++ 实现上，极端深树可能造成递归栈风险，工程写法可以用 `std::vector<TreeNode*>` 或队列显式保存状态。BST 边界最好用更宽整数或可选前驱值。如果追问变成“若频繁查询，可在节点维护子树大小成为顺序统计树”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 235 二叉搜索树最近公共祖先

先想最直接的树遍历会在每个节点重复求什么。本题的模型是 BST 的值域关系能决定两个目标在当前节点的哪一侧；如果每个节点都重新扫描子树，慢点就会非常明显。本题要压缩的状态是两个值都小去左，都大去右，否则当前节点就是分叉点，也就是节点和父子方向之间真正传递的信息。

树题要先判定状态方向。本题的两个值都小去左，都大去右，否则当前节点就是分叉点可能是自顶向下的约束、后序向上的汇总，或队列中的层号。返回值或栈元素不能只有节点指针，还要带上题目需要的信息。放到“二叉树与树形状态”这条主线里看，关键原则是：树形 DP 把每个节点

看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。

树题常用对子树归纳。假设子节点状态已经正确，父节点只按两个值都小去左，都大去右，否则当前节点就是分叉点合并或传递信息。本题的 BST 的值域关系能决定两个目标在当前节点的哪一侧决定空节点、叶子和极端链式树如何处理。这道题的边界不能留到最后补丁式处理，实验时覆盖一个节点是另一个祖先、目标顺序、重复值约定，尤其关注空节点、单节点、链式树和全负值。递归版本和显式栈版本可以互相对拍。

C++ 实现上，极端深树可能造成递归栈风险，工程写法可以用 `std::vector<TreeNode*>` 或队列显式保存状态。BST 边界最好用更宽整数或可选前驱值。如果追问变成“普通二叉树不能用值域剪枝，需要父指针或后序状态”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 236 二叉树最近公共祖先

先想最直接的树遍历会在每个节点重复求什么。本题的模型是普通树没有有序性，只能依靠祖先关系；如果每个节点都重新扫描子树，慢点就会非常明显。本题要压缩的状态是父指针集合或后序返回目标命中状态都可行，也就是节点和父子方向之间真正传递的信息。

树题要先判定状态方向。本题的父指针集合或后序返回目标命中状态都可行可能是自顶向下的约束、后序向上的汇总，或队列中的层号。返回值或栈元素不能只有节点指针，还要带上题目需要的信息。放到“二叉树与树形状态”这条主线里看，关键原则是：树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。

树题常用对子树归纳。假设子节点状态已经正确，父节点只按父指针集合或后序返回目标命中状态都可行合并或传递信息。本题的普通树没有有序性，只能依靠祖先关系决定空节点、叶子和极端链式树如何处理。这道题的边界不能留到最后补丁式处理，实验时覆盖目标不存在、p 等于 q、一个是另一个祖先、比较节点地址，尤其关注空节点、单节点、链式树和全负值。递归版本和显式栈版本可以互相对拍。

C++ 实现上，极端深树可能造成递归栈风险，工程写法可以用 `std::vector<TreeNode*>` 或队列显式保存状态。BST 边界最好用更宽整数或可选前驱值。如果追问变成“多次 LCA 查询可预处理倍增祖先表”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 337 打家劫舍 III

先想最直接的树遍历会在每个节点重复求什么。本题的模型是树上相邻节点不能同时选，每个节点需要偷和不偷两个状态；如果每个节点都重新扫描子树，慢点就会非常明显。本题要压缩的状态是后序汇总：偷当前则孩子不能偷，不偷当前则孩子可偷可不偷，也就是节点和父子方向之间真正传递的信息。

树题要先判定状态方向。本题的后序汇总：偷当前则孩子不能偷，不偷当前则孩子可偷可不偷可能是自顶向下的约束、后序向上的汇总，或队列中的层号。返回值或栈元素不能只有节点指针，还要带上题目需要的信息。放到“二叉树与树形状态”这条主线里看，关键原则是：树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。

树题常用对子树归纳。假设子节点状态已经正确，父节点只按后序汇总：偷当前则孩子不能偷，不偷当前则孩子可偷可不偷合并或传递信息。本题的树上相邻节点不能同时选，每个节点需要偷和不偷两个状态决定空节点、叶子和极端链式树如何处理。这道题的边界不能留到最后补丁式处理，实验时覆盖空树、负值是否可能、链式树、递归深度，尤其关注空节点、单节点、链式树和全负值。递归版本和显式栈版本可以互相对拍。

C++ 实现上，极端深树可能造成递归栈风险，工程写法可以用 `std::vector<TreeNode*>` 或队列显式保存状态。BST 边界最好用更宽整数或可选前驱值。如果追问变成“可用显式栈后序保存节点状态，避免调用栈”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 450 删除二叉搜索树中的节点

先想最直接的树遍历会在每个节点重复求什么。本题的模型是删除节点后仍要保持 BST 中序有序；如果每个节点都重新扫描子树，慢点就会非常明显。本题要压缩的状态是若有两个孩子，用后继或前驱替换当前值，再删除那个后继节点，也就是节点和父子方向之间真正传递的信息。

树题要先判定状态方向。本题的若有两个孩子，用后继或前驱替换当前值，再删除那个后继节点可能是自顶向下的约束、后序向上的汇总，或队列中的层号。返回值或栈元素不能只有节点指针，还要带上题目需要的信息。放到“二叉树与树形状态”这条主线里看，关键原则是：树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。

树题常用对子树归纳。假设子节点状态已经正确，父节点只按若有两个孩子，用后继或前驱替换当前值，再删除那个后继节点合并或传递信息。本题的删除节点后仍要保持 BST 中序有序决定空节点、叶子和极端链式树如何处理。这道题的边界不能留到最后补丁式处理，实验时覆盖删除叶子、一个孩子、两个孩子、删除根节点，尤其关注空节点、单节点、链式树和全负值。递归版本和显式栈版本可以互相对拍。

C++ 实现上，极端深树可能造成递归栈风险，工程写法可以用 `std::vector<TreeNode*>` 或队列显式保存状态。BST 边界最好用更宽整数或可选前驱值。如果追问变成“工程实现要清楚是移动值还是重连节点”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 543 二叉树直径

先想最直接的树遍历会在每个节点重复求什么。本题的模型是直径经过某节点时等于左高加右高，但向父亲只能贡献单边高度；如果每个节点都重新扫描子树，慢点就会非常明显。本题要压缩的状态是后序遍历同时更新全局直径并返回高度，也就是节点和父子方向之间真正传递的信息。

树题要先判定状态方向。本题的后序遍历同时更新全局直径并返回高度可能是自顶向下的约束、后序向上的汇总，或队列中的层号。返回值或栈元素不能只有节点指针，还要带上题目需要的信息。放到“二叉树与树形状态”这条主线里看，关键原则是：树形 DP 把每个节点看成子问题根；BFS 把层次边界显式放进队列；BST 题利用中序有序或上下界约束。工程实现优先考虑显式栈避免深树调用栈风险。

树题常用对子树归纳。假设子节点状态已经正确，父节点只按后序遍历同时更新全局直径并返回高度合并或传递信息。本题的直径经过某节点时等于左高加右高，但向父亲只能贡献单边高度决定空节点、叶子和极端链式树如何处理。这道题的边界不能留到最后补丁式处理，实验时覆盖空树、单节点、链式树、边数还是节点数定义，尤其关注空节点、单节点、链式树和全负值。递归版本和显式栈版本可以互相对拍。

C++ 实现上，极端深树可能造成递归栈风险，工程写法可以用 `std::vector<TreeNode*>` 或队列显式保存状态。BST 边界最好用更宽整数或可选前驱值。如果追问变成“最大路径和是同型题，但贡献值和全局更新有负数处理”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

图建模、BFS 与 DFS 工作坊

先定义节点、边和状态，再决定 BFS、DFS、拓扑或并查集。图题失败常常不是搜索模板错，而是状态维度不完整或邻居生成过慢。

LeetCode 127 单词接龙

先画图，不先选搜索模板。本题的模型是每个单词是节点，一次修改一个字符形成边；朴素做法通常从多个起点反复搜索，或者把状态维度压得太少。优化首先要围绕通配模式桶加速邻居查询，BFS 第一次到达终点就是最短转换长度明确节点、边和访问标记。

图状态由节点和附加资源共同组成。本题的通配模式桶加速邻居查询，BFS 第一次到达终点就是最短转换长度要决定访问标记何时设置、状态是否只按位置去重、邻居如何生成。建模错了，BFS 或 DFS 再标准也会错。放到“图建模、BFS 与 DFS”这条主线里看，关键原则是：无权最短步数用 BFS，连通块和状态检测用 DFS 或显式栈，有向无环依赖用拓扑排序。多源问题把所有源同时入队，状态带资源的问题不能只按位置去重。

图搜索证明要回到可达性或层次。一次搜索必须覆盖从起点按每个单词是节点，一次修改一个

字符形成边可达的全部状态,同时不能越过非法边;通配模式桶加速邻居查询,BFS 第一次到达终点就是最短转换长度保证重复状态不会反复入队或入栈。这道题的边界不能留到最后补丁式处理,实验时覆盖终点不在词表、起点和终点相同、桶重复扫描、单词长度一致,并打印入队时的完整状态。若同一位置但资源不同的状态被合并,很多图题会出现隐蔽错误。

C++ 实现上,连续编号图用 `std::vector<std::vector<int>>`,字符串节点先编号或使用哈希表。BFS 通常入队时标记访问,方向数组用固定常量集中维护。如果追问变成“双向 BFS 可以进一步降低搜索空间”,先判断原来的状态是否仍然覆盖新答案集合,再决定是微调边界、改 value 语义,还是换成另一类结构。

LeetCode 130 被围绕的区域

先画图,不先选搜索模板。本题的模型是只有不连通到边界的 O 才会被翻转;朴素做法通常从多个起点反复搜索,或者把状态维度压得太少。优化首先要围绕从边界 O 出发标记安全区域,再翻转剩余 O 明确节点、边和访问标记。

图状态由节点和附加资源共同组成。本题的从边界 O 出发标记安全区域,再翻转剩余 O 要决定访问标记何时设置、状态是否只按位置去重、邻居如何生成。建模错了,BFS 或 DFS 再标准也会错。放到“图建模、BFS 与 DFS”这条主线里看,关键原则是:无权最短步数用 BFS,连通块和状态检测用 DFS 或显式栈,有向无环依赖用拓扑排序。多源问题把所有源同时入队,状态带资源的问题不能只按位置去重。

图搜索证明要回到可达性或层次。一次搜索必须覆盖从起点按只有不连通到边界的 O 才会被翻转可达的全部状态,同时不能越过非法边;从边界 O 出发标记安全区域,再翻转剩余 O 保证重复状态不会反复入队或入栈。这道题的边界不能留到最后补丁式处理,实验时覆盖全边界、无 O、单行单列、内部岛屿,并打印入队时的完整状态。若同一位置但资源不同的状态被合并,很多图题会出现隐蔽错误。

C++ 实现上,连续编号图用 `std::vector<std::vector<int>>`,字符串节点先编号或使用哈希表。BFS 通常入队时标记访问,方向数组用固定常量集中维护。如果追问变成“这是反向思维:找不能被翻转的区域比直接判断每块区域更简单”,先判断原来的状态是否仍然覆盖新答案集合,再决定是微调边界、改 value 语义,还是换成另一类结构。

LeetCode 133 克隆图

先画图,不先选搜索模板。本题的模型是每个原节点需要唯一对应一个新节点,边按原图连接;朴素做法通常从多个起点反复搜索,或者把状态维度压得太少。优化首先要围绕哈希表保存原节点到克隆节点的映射,BFS 或 DFS 遍历图明确节点、边和访问标记。

图状态由节点和附加资源共同组成。本题的哈希表保存原节点到克隆节点的映射,BFS 或 DFS 遍历图要决定访问标记何时设置、状态是否只按位置去重、邻居如何生成。建模错了,BFS 或 DFS 再标准也会错。放到“图建模、BFS 与 DFS”这条主线里看,关键原则是:无权最短步数用 BFS,连通块和状态检测用 DFS 或显式栈,有向无环依赖用拓扑排序。多源问题把所有源同时入队,状态带资源的问题不能只按位置去重。

图搜索证明要回到可达性或层次。一次搜索必须覆盖从起点按每个原节点需要唯一对应一个新节点,边按原图连接可达的全部状态,同时不能越过非法边;哈希表保存原节点到克隆节点的映射,BFS 或 DFS 遍历图保证重复状态不会反复入队或入栈。这道题的边界不能留到最后补丁式处理,实验时覆盖空图、自环、重边、环结构、不能按节点值唯一假设,并打印入队时的完整状态。若同一位置但资源不同的状态被合并,很多图题会出现隐蔽错误。

C++ 实现上,连续编号图用 `std::vector<std::vector<int>>`,字符串节点先编号或使用哈希表。BFS 通常入队时标记访问,方向数组用固定常量集中维护。如果追问变成“若图节点值不唯一,必须用地址作为键”,先判断原来的状态是否仍然覆盖新答案集合,再决定是微调边界、改 value 语义,还是换成另一类结构。

LeetCode 200 岛屿数量

先画图,不先选搜索模板。本题的模型是陆地格子是节点,四方向相邻是边,岛屿是连通块;朴素做法通常从多个起点反复搜索,或者把状态维度压得太少。优化首先要围绕扫描遇到未访问陆地就启动搜索并标记整块明确节点、边和访问标记。

图状态由节点和附加资源共同组成。本题的扫描遇到未访问陆地就启动搜索并标记整块要决定

访问标记何时设置、状态是否只按位置去重、邻居如何生成。建模错了，BFS 或 DFS 再标准也会错。放到“图建模、BFS 与 DFS”这条主线里看，关键原则是：无权最短步数用 BFS，连通块和状态检测用 DFS 或显式栈，有向无环依赖用拓扑排序。多源问题把所有源同时入队，状态带资源的问题不能只按位置去重。

图搜索证明要回到可达性或层次。一次搜索必须覆盖从起点按陆地格子是节点，四方向相邻是边，岛屿是连通块可达的全部状态，同时不能越过非法边；扫描遇到未访问陆地就启动搜索并标记整块保证重复状态不会反复入队或入栈。这道题的边界不能留到最后补丁式处理，实验时覆盖空网格、全水、全陆、斜对角不连通、是否允许修改输入，并打印入队时的完整状态。若同一位置但资源不同的状态被合并，很多图题会出现隐蔽错误。

C++ 实现上，连续编号图用 `std::vector<std::vector<int>>`，字符串节点先编号或使用哈希表。BFS 通常入队时标记访问，方向数组用固定常量集中维护。如果追问变成“也可用并查集合并相邻陆地，适合动态岛屿扩展”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 207 课程表

先画图，不先选搜索模板。本题的模型是课程是节点，先修关系是有向边，问题是判断是否有环；朴素做法通常从多个起点反复搜索，或者把状态维度压得太少。优化首先要围绕入度拓扑排序不断学习当前无依赖课程明确节点、边和访问标记。

图状态由节点和附加资源共同组成。本题的入度拓扑排序不断学习当前无依赖课程要决定访问标记何时设置、状态是否只按位置去重、邻居如何生成。建模错了，BFS 或 DFS 再标准也会错。放到“图建模、BFS 与 DFS”这条主线里看，关键原则是：无权最短步数用 BFS，连通块和状态检测用 DFS 或显式栈，有向无环依赖用拓扑排序。多源问题把所有源同时入队，状态带资源的问题不能只按位置去重。

图搜索证明要回到可达性或层次。一次搜索必须覆盖从起点按课程是节点，先修关系是有向边，问题是判断是否有环可达的全部状态，同时不能越过非法边；入度拓扑排序不断学习当前无依赖课程保证重复状态不会反复入队或入栈。这道题的边界不能留到最后补丁式处理，实验时覆盖边方向、孤立课程、环、自依赖、重复边，并打印入队时的完整状态。若同一位置但资源不同的状态被合并，很多图题会出现隐蔽错误。

C++ 实现上，连续编号图用 `std::vector<std::vector<int>>`，字符串节点先编号或使用哈希表。BFS 通常入队时标记访问，方向数组用固定常量集中维护。如果追问变成“DFS 三色标记也能判环，但要明确访问状态”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 210 课程表 II

先画图，不先选搜索模板。本题的模型是在判环基础上还要输出一个可行拓扑序；朴素做法通常从多个起点反复搜索，或者把状态维度压得太少。优化首先要围绕每弹出一个入度为零课程，就把它加入顺序并删除出边明确节点、边和访问标记。

图状态由节点和附加资源共同组成。本题的每弹出一个入度为零课程，就把它加入顺序并删除出边要决定访问标记何时设置、状态是否只按位置去重、邻居如何生成。建模错了，BFS 或 DFS 再标准也会错。放到“图建模、BFS 与 DFS”这条主线里看，关键原则是：无权最短步数用 BFS，连通块和状态检测用 DFS 或显式栈，有向无环依赖用拓扑排序。多源问题把所有源同时入队，状态带资源的问题不能只按位置去重。

图搜索证明要回到可达性或层次。一次搜索必须覆盖从起点按在判环基础上还要输出一个可行拓扑序可达的全部状态，同时不能越过非法边；每弹出一个入度为零课程，就把它加入顺序并删除出边保证重复状态不会反复入队或入栈。这道题的边界不能留到最后补丁式处理，实验时覆盖有环返回空数组、多个可行顺序、边方向，并打印入队时的完整状态。若同一位置但资源不同的状态被合并，很多图题会出现隐蔽错误。

C++ 实现上，连续编号图用 `std::vector<std::vector<int>>`，字符串节点先编号或使用哈希表。BFS 通常入队时标记访问，方向数组用固定常量集中维护。如果追问变成“若需要字典序最小拓扑序，把队列换成小顶堆”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 417 太平洋大西洋水流问题

先画图，不先选搜索模板。本题的模型是从每个格子向海搜索很慢，反向从海边沿非递减高度搜索；朴素做法通常从多个起点反复搜索，或者把状态维度压得太少。优化首先要围绕分别标记能到太平洋和大西洋的格子，交集就是答案明确节点、边和访问标记。

图状态由节点和附加资源共同组成。本题的分别标记能到太平洋和大西洋的格子，交集就是答案要决定访问标记何时设置、状态是否只按位置去重、邻居如何生成。建模错了，BFS 或 DFS 再标准也会错。放到“图建模、BFS 与 DFS”这条主线里看，关键原则是：无权最短步数用 BFS，连通块和状态检测用 DFS 或显式栈，有向无环依赖用拓扑排序。多源问题把所有源同时入队，状态带资源的问题不能只按位置去重。

图搜索证明要回到可达性或层次。一次搜索必须覆盖从起点按从每个格子向海搜索很慢，反向从海边沿非递减高度搜索可达的全部状态，同时不能越过非法边；分别标记能到太平洋和大西洋的格子，交集就是答案保证重复状态不会反复入队或入栈。这道题的边界不能留到最后补丁式处理，实验时覆盖单行单列、平台高度、边界重复入队、方向条件反向，并打印入队时的完整状态。若同一位置但资源不同的状态被合并，很多图题会出现隐蔽错误。

C++ 实现上，连续编号图用 `std::vector<std::vector<int>>`，字符串节点先编号或使用哈希表。BFS 通常入队时标记访问，方向数组用固定常量集中维护。如果追问变成“反向搜索是很多可达性题的重要技巧”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 695 岛屿的最大面积

先画图，不先选搜索模板。本题的模型是面积就是一个连通块中陆地格子的数量；朴素做法通常从多个起点反复搜索，或者把状态维度压得太少。优化首先要围绕每发现新陆地就搜索完整连通块并计数，取最大值明确节点、边和访问标记。

图状态由节点和附加资源共同组成。本题的每发现新陆地就搜索完整连通块并计数，取最大值要决定访问标记何时设置、状态是否只按位置去重、邻居如何生成。建模错了，BFS 或 DFS 再标准也会错。放到“图建模、BFS 与 DFS”这条主线里看，关键原则是：无权最短步数用 BFS，连通块和状态检测用 DFS 或显式栈，有向无环依赖用拓扑排序。多源问题把所有源同时入队，状态带资源的问题不能只按位置去重。

图搜索证明要回到可达性或层次。一次搜索必须覆盖从起点按面积就是一个连通块中陆地格子的数量可达的全部状态，同时不能越过非法边；每发现新陆地就搜索完整连通块并计数，取最大值保证重复状态不会反复入队或入栈。这道题的边界不能留到最后补丁式处理，实验时覆盖全水、全陆、细长岛、是否允许修改输入，并打印入队时的完整状态。若同一位置但资源不同的状态被合并，很多图题会出现隐蔽错误。

C++ 实现上，连续编号图用 `std::vector<std::vector<int>>`，字符串节点先编号或使用哈希表。BFS 通常入队时标记访问，方向数组用固定常量集中维护。如果追问变成“与岛屿数量相比，只是连通块聚合值不同”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 752 打开转盘锁

先画图，不先选搜索模板。本题的模型是四位密码是状态，每次转动一位形成边；朴素做法通常从多个起点反复搜索，或者把状态维度压得太少。优化首先要围绕 BFS 从起点扩展，死亡状态不能入队，访问状态避免重复明确节点、边和访问标记。

图状态由节点和附加资源共同组成。本题的 BFS 从起点扩展，死亡状态不能入队，访问状态避免重复要决定访问标记何时设置、状态是否只按位置去重、邻居如何生成。建模错了，BFS 或 DFS 再标准也会错。放到“图建模、BFS 与 DFS”这条主线里看，关键原则是：无权最短步数用 BFS，连通块和状态检测用 DFS 或显式栈，有向无环依赖用拓扑排序。多源问题把所有源同时入队，状态带资源的问题不能只按位置去重。

图搜索证明要回到可达性或层次。一次搜索必须覆盖从起点按四位密码是状态，每次转动一位形成边可达的全部状态，同时不能越过非法边；BFS 从起点扩展，死亡状态不能入队，访问状态避免重复保证重复状态不会反复入队或入栈。这道题的边界不能留到最后补丁式处理，实验时覆盖起点死亡、目标就是起点、数字环绕、死亡集合很大，并打印入队时的完整状态。若同一位置但资源不

同的状态被合并，很多图题会出现隐蔽错误。

C++ 实现上，连续编号图用 `std::vector<std::vector<int>>`，字符串节点先编号或使用哈希表。BFS 通常入队时标记访问，方向数组用固定常量集中维护。如果追问变成“双向 BFS 能明显减少状态展开”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 994 腐烂的橘子

先画图，不先选搜索模板。本题的模型是所有初始腐烂橘子同时作为 BFS 源点；朴素做法通常从多个起点反复搜索，或者把状态维度压得太少。优化首先要围绕按层扩散，分钟数等于最后一个新鲜橘子被首次访问的层数明确节点、边和访问标记。

图状态由节点和附加资源共同组成。本题的按层扩散，分钟数等于最后一个新鲜橘子被首次访问的层数要决定访问标记何时设置、状态是否只按位置去重、邻居如何生成。建模错了，BFS 或 DFS 再标准也会错。放到“图建模、BFS 与 DFS”这条主线里看，关键原则是：无权最短步数用 BFS，连通块和状态检测用 DFS 或显式栈，有向无环依赖用拓扑排序。多源问题把所有源同时入队，状态带资源的问题不能只按位置去重。

图搜索证明要回到可达性或层次。一次搜索必须覆盖从起点按所有初始腐烂橘子同时作为 BFS 源点可达的全部状态，同时不能越过非法边；按层扩散，分钟数等于最后一个新鲜橘子被首次访问的层数保证重复状态不会反复入队或入栈。这道题的边界不能留到最后补丁式处理，实验时覆盖没有新鲜橘子、没有腐烂源、隔离新鲜橘子、空格阻断，并打印入队时的完整状态。若同一位置但资源不同的状态被合并，很多图题会出现隐蔽错误。

C++ 实现上，连续编号图用 `std::vector<std::vector<int>>`，字符串节点先编号或使用哈希表。BFS 通常入队时标记访问，方向数组用固定常量集中维护。如果追问变成“多源 BFS 也适用于最近零距离、地图扩散等题”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

最短路与带权图工作坊

先确认目标是步数最少还是代价最小，再检查边权是否非负。非负权用 Dijkstra，等权用 BFS，限制边数用分层松弛或 Bellman-Ford 思想。

LeetCode 743 网络延迟时间

先区分步数和代价。本题的模型是有向正权图从起点到所有节点的最短路最大值；若暴力枚举路径，路径数量会爆炸。优化点 Dijkstra 求每个节点最短距离，最后取最大，若有无穷则不可达说明下一步扩展必须按某种代价顺序组织，而不是普通队列随便推进。

最短路状态至少包含当前节点和当前代价。本题的 Dijkstra 求每个节点最短距离，最后取最大，若有无穷则不可达决定优先队列按什么排序，以及旧状态如何被识别。非负边条件、代价合成方式和不可达处理都要写清楚。放到“最短路与带权图”这条主线里看，关键原则是：非负权图用 Dijkstra，小顶堆保存当前距离最小的候选；边权为一用 BFS；有负边要考虑 Bellman-Ford 或重新建模。路径代价若是最大边权，也可用 Dijkstra 或二分答案。

最短路证明依赖扩展顺序。若本题使用 Dijkstra，就要说明非负代价让弹出的未过期状态已经最优；若用二分可达性，就要说明阈值越大可用边只会增加。这道题的边界不能留到最后补丁式处理，实验时覆盖节点编号从一开始、不可达、重边、过期堆元素，打印每次弹出的代价和是否过期。若普通 BFS 与 Dijkstra 结果不同，要能解释边权条件造成的差异。

C++ 实现上，Dijkstra 用邻接表保存 `(to, weight)`，距离数组用 `long long`。由于 `priority_queue` 没有 `decrease-key`，压入新状态后弹出时跳过旧状态。如果追问变成“边权相同才可用普通 BFS”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 1631 最小体力消耗路径

先区分步数和代价。本题的模型是路径代价是沿途最大高度差，不是边权和；若暴力枚举路径，路径数量会爆炸。优化点 Dijkstra 的松弛改成取当前代价和新边代价的最大值说明下一步扩展必须按某种代价顺序组织，而不是普通队列随便推进。

最短路状态至少包含当前节点和当前代价。本题的 Dijkstra 的松弛改成取当前代价和新边代价

的最大值决定优先队列按什么排序，以及旧状态如何被识别。非负边条件、代价合成方式和不可达处理都要写清楚。放到“最短路与带权图”这条主线里看，关键原则是：非负权图用 Dijkstra，小顶堆保存当前距离最小的候选；边权为一用 BFS；有负边要考虑 Bellman-Ford 或重新建模。路径代价若是最大边权，也可用 Dijkstra 或二分答案。

最短路证明依赖扩展顺序。若本题使用 Dijkstra，就要说明非负代价让弹出的未过期状态已经最优；若用二分可达性，就要说明阈值越大可用边只会增加。这道题的边界不能留到最后补丁式处理，实验时覆盖单格、平坦网格、必须绕路、多个相同代价，打印每次弹出的代价和是否过期。若普通 BFS 与 Dijkstra 结果不同，要能解释边权条件造成的差异。

C++ 实现上，Dijkstra 用邻接表保存 (to, weight)，距离数组用 long long。由于 priority_queue 没有 decrease-key，压入新状态后弹出时跳过旧状态。如果追问变成“也可二分体力上限后 BFS 检查可达”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

并查集与动态连通性工作坊

先用搜索回答连通性，再发现查询和合并交替出现。并查集只维护集合代表，如果题目还要路径、距离、比例或奇偶性，就必须扩展状态。

LeetCode 547 省份数量

先用搜索回答每次连通性查询，观察合并和查询如何交替出现。本题的模型是城市连接关系形成无向图，省份就是连通分量；慢点在每次都重新遍历已有连接。并查集只适合并查集合并所有直接连接的城市，最后统计根数量这类集合代表问题，若题目要路径本身就不能硬套。

并查集状态是代表元和集合附加信息。本题的并查集合并所有直接连接的城市，最后统计根数量只要求合并集合和查询同组关系；如果还要大小、边数、比例或奇偶性，就要在合并根时同步维护额外数组。放到“并查集与动态连通性”这条主线里看，关键原则是：路径压缩把查找路径上的节点直接挂到根，按大小或按秩合并控制树高。邮箱、等式、边、网格都可以映射成元素集合。

并查集正确性来自等价关系。合并两个代表后，两个集合中任意元素的代表都会一致；不合并时，集合关系保持不变。本题的并查集合并所有直接连接的城市，最后统计根数量必须只依赖这种代表关系。这道题的边界不能留到最后补丁式处理，实验时覆盖邻接矩阵对称、自连接、孤立城市、只扫描上三角，打印每个元素的代表元和集合大小。重复合并、已经连通的边和字符串编号最容易暴露实现问题。

C++ 实现上，并查集类中成员访问保持 this->, find 可以写成迭代路径压缩避免深递归。若维护集合大小、边数或权值差，合并根时同步更新。如果追问变成“BFS 也可做；并查集更突出动态合并”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 684 冗余连接

先用搜索回答每次连通性查询，观察合并和查询如何交替出现。本题的模型是树加一条边会形成唯一环；慢点在每次都重新遍历已有连接。并查集只适合按顺序加边，若一条边两端已连通，则它就是冗余边这类集合代表问题，若题目要路径本身就不能硬套。

并查集状态是代表元和集合附加信息。本题的按顺序加边，若一条边两端已连通，则它就是冗余边只要求合并集合和查询同组关系；如果还要大小、边数、比例或奇偶性，就要在合并根时同步维护额外数组。放到“并查集与动态连通性”这条主线里看，关键原则是：路径压缩把查找路径上的节点直接挂到根，按大小或按秩合并控制树高。邮箱、等式、边、网格都可以映射成元素集合。

并查集正确性来自等价关系。合并两个代表后，两个集合中任意元素的代表都会一致；不合并时，集合关系保持不变。本题的按顺序加边，若一条边两端已连通，则它就是冗余边必须只依赖这种代表关系。这道题的边界不能留到最后补丁式处理，实验时覆盖节点编号、最后一条成环边、重复边、并查集初始化大小，打印每个元素的代表元和集合大小。重复合并、已经连通的边和字符串编号最容易暴露实现问题。

C++ 实现上，并查集类中成员访问保持 this->, find 可以写成迭代路径压缩避免深递归。若维护集合大小、边数或权值差，合并根时同步更新。如果追问变成“有向冗余连接需要同时处理入度为二和环，复杂度更高”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

回溯、枚举与剪枝工作坊

先写搜索树：路径是什么、选择列表是什么、什么时候结束。然后用排序、剩余容量、同层去重或合法性预处理剪掉不可能分支。

LeetCode 78 子集

先画前三层搜索树。本题的模型是每个元素都有选或不选两种选择，答案是所有路径节点；暴力搜索要明确路径、选择列表和终止条件。优化点是回溯按起点向后枚举，避免不同顺序生成同一集合，也就是哪些分支在展开前已经不可能产生合法答案。

回溯状态由路径和当前位置共同定义。本题的回溯按起点向后枚举，避免不同顺序生成同一集合要落成剪枝条件或同层去重条件。剪枝只能删掉不可能成功的分支，不能删掉只是暂时看起来不优的分支。放到“回溯、枚举与剪枝”这条主线里看，关键原则是：剪枝来自容量、剩余长度、排序后去重、合法性预检查和提前终止。组合题关注起点推进，排列题关注元素是否使用，分割题关注当前片段是否合法。

回溯证明分两半：搜索树覆盖所有合法答案，同层去重或剪枝不会删除合法答案。本题的每个元素都有选或不选两种选择，答案是所有路径节点给出选择来源，回溯按起点向后枚举，避免不同顺序生成同一集合给出提前停止的理由。这道题的边界不能留到最后补丁式处理，实验时覆盖空数组、单元素、输出数量为二的 n 次方、顺序不要求，统计搜索节点数量和剪枝次数。重复元素题要打印同一层跳过哪些值，确认没有误删合法路径。

C++ 实现上，路径容器修改要成对出现：加入、搜索、撤销。重复元素题先排序，再用同层去重条件；输出规模大时，复杂度要把答案数量算进去。如果追问变成“也可用位掩码枚举所有子集，适合 n 较小的场景”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 79 单词搜索

先画前三层搜索树。本题的模型是网格路径不能重复使用同一格，状态包含位置、匹配下标和访问标记；暴力搜索要明确路径、选择列表和终止条件。优化点是剪枝包括首字符不匹配、剩余长度不足、字符频次不足和越界访问，也就是哪些分支在展开前已经不可能产生合法答案。

回溯状态由路径和当前位置共同定义。本题的剪枝包括首字符不匹配、剩余长度不足、字符频次不足和越界访问要落成剪枝条件或同层去重条件。剪枝只能删掉不可能成功的分支，不能删掉只是暂时看起来不优的分支。放到“回溯、枚举与剪枝”这条主线里看，关键原则是：剪枝来自容量、剩余长度、排序后去重、合法性预检查和提前终止。组合题关注起点推进，排列题关注元素是否使用，分割题关注当前片段是否合法。

回溯证明分两半：搜索树覆盖所有合法答案，同层去重或剪枝不会删除合法答案。本题的网格路径不能重复使用同一格，状态包含位置、匹配下标和访问标记给出选择来源，剪枝包括首字符不匹配、剩余长度不足、字符频次不足和越界访问给出提前停止的理由。这道题的边界不能留到最后补丁式处理，实验时覆盖单字符单词、路径需要回退、重复字母、单元格不能复用，统计搜索节点数量和剪枝次数。重复元素题要打印同一层跳过哪些值，确认没有误删合法路径。

C++ 实现上，路径容器修改要成对出现：加入、搜索、撤销。重复元素题先排序，再用同层去重条件；输出规模大时，复杂度要把答案数量算进去。如果追问变成“若要查很多单词，应使用 Trie 合并搜索前缀”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 90 子集 II

先画前三层搜索树。本题的模型是重复元素会让不同下标路径生成相同集合；暴力搜索要明确路径、选择列表和终止条件。优化点是排序后在同一层跳过相同值，保留不同层使用重复值的可能，也就是哪些分支在展开前已经不可能产生合法答案。

回溯状态由路径和当前位置共同定义。本题的排序后在同一层跳过相同值，保留不同层使用重复值的可能要落成剪枝条件或同层去重条件。剪枝只能删掉不可能成功的分支，不能删掉只是暂时看起来不优的分支。放到“回溯、枚举与剪枝”这条主线里看，关键原则是：剪枝来自容量、剩余长度、排序后去重、合法性预检查和提前终止。组合题关注起点推进，排列题关注元素是否使用，分割题关注当前片段是否合法。

回溯证明分两半：搜索树覆盖所有合法答案，同层去重或剪枝不会删除合法答案。本题的重复元

素会让不同下标路径生成相同集合给出选择来源，排序后在同一层跳过相同值，保留不同层使用重复值的可能给出提前停止的理由。这道题的边界不能留到最后补丁式处理，实验时覆盖全重复、空集、重复值相邻、去重条件写成同层而不是全局，统计搜索节点数量和剪枝次数。重复元素题要打印同一层跳过哪些值，确认没有误删合法路径。

C++ 实现上，路径容器修改要成对出现：加入、搜索、撤销。重复元素题先排序，再用同层去重条件；输出规模大时，复杂度要把答案数量算进去。如果追问变成“组合总和 II 使用同样的同层去重思想”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 131 分割回文串

先画前三层搜索树。本题的模型是每次选择一个从当前位置开始的回文前缀；暴力搜索要明确路径、选择列表和终止条件。优化点是预处理回文 DP 表，让回溯合法性检查为常数，也就是哪些分支在展开前已经不可能产生合法答案。

回溯状态由路径和当前位置共同定义。本题的预处理回文 DP 表，让回溯合法性检查为常数要落成剪枝条件或同层去重条件。剪枝只能删掉不可能成功的分支，不能删掉只是暂时看起来不优的分支。放到“回溯、枚举与剪枝”这条主线里看，关键原则是：剪枝来自容量、剩余长度、排序后去重、合法性预检查和提前终止。组合题关注起点推进，排列题关注元素是否使用，分割题关注当前片段是否合法。

回溯证明分两半：搜索树覆盖所有合法答案，同层去重或剪枝不会删除合法答案。本题的每次选择一个从当前位置开始的回文前缀给出选择来源，预处理回文 DP 表，让回溯合法性检查为常数给出提前停止的理由。这道题的边界不能留到最后补丁式处理，实验时覆盖空串、单字符、全相同字符、重复分割路径，统计搜索节点数量和剪枝次数。重复元素题要打印同一层跳过哪些值，确认没有误删合法路径。

C++ 实现上，路径容器修改要成对出现：加入、搜索、撤销。重复元素题先排序，再用同层去重条件；输出规模大时，复杂度要把答案数量算进去。如果追问变成“若只求最少切割次数，可改成 DP 最短切分”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 216 组合总和 III

先画前三层搜索树。本题的模型是从一到九中选 k 个数和为 n，每个数最多用一次；暴力搜索要明确路径、选择列表和终止条件。优化点是按递增起点枚举，利用剩余个数和剩余和剪枝，也就是哪些分支在展开前已经不可能产生合法答案。

回溯状态由路径和当前位置共同定义。本题的按递增起点枚举，利用剩余个数和剩余和剪枝要落成剪枝条件或同层去重条件。剪枝只能删掉不可能成功的分支，不能删掉只是暂时看起来不优的分支。放到“回溯、枚举与剪枝”这条主线里看，关键原则是：剪枝来自容量、剩余长度、排序后去重、合法性预检查和提前终止。组合题关注起点推进，排列题关注元素是否使用，分割题关注当前片段是否合法。

回溯证明分两半：搜索树覆盖所有合法答案，同层去重或剪枝不会删除合法答案。本题的从一到九中选 k 个数和为 n，每个数最多用一次给出选择来源，按递增起点枚举，利用剩余个数和剩余和剪枝给出提前停止的理由。这道题的边界不能留到最后补丁式处理，实验时覆盖 n 太小、n 太大、k 为零、路径长度已经超过 k，统计搜索节点数量和剪枝次数。重复元素题要打印同一层跳过哪些值，确认没有误删合法路径。

C++ 实现上，路径容器修改要成对出现：加入、搜索、撤销。重复元素题先排序，再用同层去重条件；输出规模大时，复杂度要把答案数量算进去。如果追问变成“可预估剩余最小和最大和进一步剪枝”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

动态规划与状态定义工作坊

先写递归搜索并找重复状态，再给状态命名。状态必须包含未来决策所需信息，遍历顺序必须保证依赖状态已经算完。

LeetCode 53 最大子数组和

先写递归或枚举，把重复子问题暴露出来。本题的模型是状态是必须以当前位置结尾的最大连续和；慢点不是递归形式，而是相同状态被反复求。本题需要命名的状态来自若前一个后缀为负，接上它只会拖累当前元素，因此可以重新开始，状态定义必须让未来决策不再依赖完整历史。

DP 状态必须足够描述未来。本题的若前一个后缀为负，接上它只会拖累当前元素，因此可以重新开始要写成数组下标、容量、前缀长度、持有状态或区间边界。初始化对应空状态，遍历顺序对应依赖方向。放到“动态规划与状态定义”这条主线里看，关键原则是：状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择，二维 DP 看两个前缀或网格位置，背包看物品阶段和容量，区间 DP 看最后合并点。

DP 证明通常看最后一步。任意最优解或合法方案的最后一步必在转移枚举中；每个转移来源又已经按遍历顺序算完。本题的若前一个后缀为负，接上它只会拖累当前元素，因此可以重新开始要支撑这个依赖关系。这道题的边界不能留到最后补丁式处理，实验时覆盖全负数组、单元素、和溢出、答案不能初始化为零，先打印完整 DP 表，再比较空间压缩版本。若压缩版不同，优先检查遍历顺序和旧值保存。

C++ 实现上，DP 表初始化要对应状态语义，不可达哨兵不能溢出。空间压缩前先写完整表；压缩后要说清读到的是上一轮、本轮左侧还是左上角旧值。如果追问变成“若要求返回区间下标，同步记录重新开始位置和最优左右端”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 62 不同路径

先写递归或枚举，把重复子问题暴露出来。本题的模型是网格状态由上方和左方两种最后一步转移而来；慢点不是递归形式，而是相同状态被反复求。本题需要命名的状态来自二维表可压缩为一维，因为当前行只依赖上一行同列和当前行左侧，状态定义必须让未来决策不再依赖完整历史。

DP 状态必须足够描述未来。本题的二维表可压缩为一维，因为当前行只依赖上一行同列和当前行左侧要写成数组下标、容量、前缀长度、持有状态或区间边界。初始化对应空状态，遍历顺序对应依赖方向。放到“动态规划与状态定义”这条主线里看，关键原则是：状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择，二维 DP 看两个前缀或网格位置，背包看物品阶段和容量，区间 DP 看最后合并点。

DP 证明通常看最后一步。任意最优解或合法方案的最后一步必在转移枚举中；每个转移来源又已经按遍历顺序算完。本题的二维表可压缩为一维，因为当前行只依赖上一行同列和当前行左侧要支撑这个依赖关系。这道题的边界不能留到最后补丁式处理，实验时覆盖一行、一列、较大网格结果溢出、障碍物版本边界，先打印完整 DP 表，再比较空间压缩版本。若压缩版不同，优先检查遍历顺序和旧值保存。

C++ 实现上，DP 表初始化要对应状态语义，不可达哨兵不能溢出。空间压缩前先写完整表；压缩后要说清读到的是上一轮、本轮左侧还是左上角旧值。如果追问变成“带障碍物时遇到障碍状态清零，第一行第一列要按阻断处理”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 63 不同路径 II

先写递归或枚举，把重复子问题暴露出来。本题的模型是障碍物让某些格子的路径数变为零；慢点不是递归形式，而是相同状态被反复求。本题需要命名的状态来自滚动数组中遇到障碍直接把当前位置清零，否则累加左侧，状态定义必须让未来决策不再依赖完整历史。

DP 状态必须足够描述未来。本题的滚动数组中遇到障碍直接把当前位置清零，否则累加左侧要写成数组下标、容量、前缀长度、持有状态或区间边界。初始化对应空状态，遍历顺序对应依赖方向。放到“动态规划与状态定义”这条主线里看，关键原则是：状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择，二维 DP 看两个前缀或网格位置，背包看物品阶段和容量，区间 DP 看最后合并点。

DP 证明通常看最后一步。任意最优解或合法方案的最后一步必在转移枚举中；每个转移来源又已经按遍历顺序算完。本题的滚动数组中遇到障碍直接把当前位置清零，否则累加左侧要支撑这个依赖关系。这道题的边界不能留到最后补丁式处理，实验时覆盖起点或终点是障碍、整行被截断、单行障碍，先打印完整 DP 表，再比较空间压缩版本。若压缩版不同，优先检查遍历顺序和旧值保

存。

C++ 实现上, DP 表初始化要对应状态语义, 不可达哨兵不能溢出。空间压缩前先写完整表; 压缩后要说清读到的是上一轮、本轮左侧还是左上角旧值。如果追问变成“若允许向上或向左走, 图会有环, 普通填表不再成立”, 先判断原来的状态是否仍然覆盖新答案集合, 再决定是微调边界、改 value 语义, 还是换成另一类结构。

LeetCode 64 最小路径和

先写递归或枚举, 把重复子问题暴露出来。本题的模型是状态是到达当前格子的最小代价, 最后一步来自上方或左方; 慢点不是递归形式, 而是相同状态被反复求。本题需要命名的状态来自先初始化第一行和第一列, 再处理内部, 主转移保持简单, 状态定义必须让未来决策不再依赖完整历史。

DP 状态必须足够描述未来。本题的先初始化第一行和第一列, 再处理内部, 主转移保持简单要写成数组下标、容量、前缀长度、持有状态或区间边界。初始化对应空状态, 遍历顺序对应依赖方向。放到“动态规划与状态定义”这条主线里看, 关键原则是: 状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择, 二维 DP 看两个前缀或网格位置, 背包看物品阶段和容量, 区间 DP 看最后合并点。

DP 证明通常看最后一步。任意最优解或合法方案的最后一步必在转移枚举中; 每个转移来源又已经按遍历顺序算完。本题的先初始化第一行和第一列, 再处理内部, 主转移保持简单要支撑这个依赖关系。这道题的边界不能留到最后补丁式处理, 实验时覆盖单行、单列、权值为零、路径和溢出, 先打印完整 DP 表, 再比较空间压缩版本。若压缩版不同, 优先检查遍历顺序和旧值保存。

C++ 实现上, DP 表初始化要对应状态语义, 不可达哨兵不能溢出。空间压缩前先写完整表; 压缩后要说清读到的是上一轮、本轮左侧还是左上角旧值。如果追问变成“若允许四方向移动, 变成最短路而不是普通网格 DP”, 先判断原来的状态是否仍然覆盖新答案集合, 再决定是微调边界、改 value 语义, 还是换成另一类结构。

LeetCode 70 爬楼梯

先写递归或枚举, 把重复子问题暴露出来。本题的模型是最后一步来自前一阶或前两阶, 是最小的重叠子问题模型; 慢点不是递归形式, 而是相同状态被反复求。本题需要命名的状态来自滚动变量保存最近两个状态即可, 不需要完整数组, 状态定义必须让未来决策不再依赖完整历史。

DP 状态必须足够描述未来。本题的滚动变量保存最近两个状态即可, 不需要完整数组要写成数组下标、容量、前缀长度、持有状态或区间边界。初始化对应空状态, 遍历顺序对应依赖方向。放到“动态规划与状态定义”这条主线里看, 关键原则是: 状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择, 二维 DP 看两个前缀或网格位置, 背包看物品阶段和容量, 区间 DP 看最后合并点。

DP 证明通常看最后一步。任意最优解或合法方案的最后一步必在转移枚举中; 每个转移来源又已经按遍历顺序算完。本题的滚动变量保存最近两个状态即可, 不需要完整数组要支撑这个依赖关系。这道题的边界不能留到最后补丁式处理, 实验时覆盖 n 为一、 n 为二、题目是否允许零阶、结果范围, 先打印完整 DP 表, 再比较空间压缩版本。若压缩版不同, 优先检查遍历顺序和旧值保存。

C++ 实现上, DP 表初始化要对应状态语义, 不可达哨兵不能溢出。空间压缩前先写完整表; 压缩后要说清读到的是上一轮、本轮左侧还是左上角旧值。如果追问变成“若每次可走一到 k 阶, 转移变成固定宽度窗口求和”, 先判断原来的状态是否仍然覆盖新答案集合, 再决定是微调边界、改 value 语义, 还是换成另一类结构。

LeetCode 72 编辑距离

先写递归或枚举, 把重复子问题暴露出来。本题的模型是二维状态表示两个前缀之间的最少编辑操作; 慢点不是递归形式, 而是相同状态被反复求。本题需要命名的状态来自末尾字符相同继承左上; 不同则枚举插入、删除、替换, 状态定义必须让未来决策不再依赖完整历史。

DP 状态必须足够描述未来。本题的末尾字符相同继承左上; 不同则枚举插入、删除、替换要写成数组下标、容量、前缀长度、持有状态或区间边界。初始化对应空状态, 遍历顺序对应依赖方向。放到“动态规划与状态定义”这条主线里看, 关键原则是: 状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择, 二维 DP 看两个前缀或网格位置, 背包看物品阶段和容量, 区间 DP 看最后合并点。

DP 证明通常看最后一步。任意最优解或合法方案的最后一步必在转移枚举中; 每个转移来源

又已经按遍历顺序算完。本题的末尾字符相同继承左上；不同则枚举插入、删除、替换要支撑这个依赖关系。这道题的边界不能留到最后补丁式处理，实验时覆盖空串、完全相同、完全不同、操作代价不同，先打印完整 DP 表，再比较空间压缩版本。若压缩版不同，优先检查遍历顺序和旧值保存。

C++ 实现上，DP 表初始化要对应状态语义，不可达哨兵不能溢出。空间压缩前先写完整表；压缩后要说清读到的是上一轮、本轮左侧还是左上角旧值。如果追问变成“若只允许插入删除，问题退化到和最长公共子序列相关”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 121 买卖股票的最佳时机

先写递归或枚举，把重复子问题暴露出来。本题的模型是卖出日固定时，最佳买入日是此前最低价格；慢点不是递归形式，而是相同状态被反复求。本题需要命名的状态来自扫描维护历史最低价和最大利润，一次交易无需复杂状态机，状态定义必须让未来决策不再依赖完整历史。

DP 状态必须足够描述未来。本题的扫描维护历史最低价和最大利润，一次交易无需复杂状态机要写成数组下标、容量、前缀长度、持有状态或区间边界。初始化对应空状态，遍历顺序对应依赖方向。放到“动态规划与状态定义”这条主线里看，关键原则是：状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择，二维 DP 看两个前缀或网格位置，背包看物品阶段和容量，区间 DP 看最后合并点。

DP 证明通常看最后一步。任意最优解或合法方案的最后一步必在转移枚举中；每个转移来源又已经按遍历顺序算完。本题的扫描维护历史最低价和最大利润，一次交易无需复杂状态机要支撑这个依赖关系。这道题的边界不能留到最后补丁式处理，实验时覆盖单日价格、一直下跌、一直上涨、价格相同，先打印完整 DP 表，再比较空间压缩版本。若压缩版不同，优先检查遍历顺序和旧值保存。

C++ 实现上，DP 表初始化要对应状态语义，不可达哨兵不能溢出。空间压缩前先写完整表；压缩后要说清读到的是上一轮、本轮左侧还是左上角旧值。如果追问变成“多次交易、冷冻期和手续费版本都要扩展成状态机”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 123 买卖股票 III

先写递归或枚举，把重复子问题暴露出来。本题的模型是最多两次交易，状态需要包含交易次数和持有状态；慢点不是递归形式，而是相同状态被反复求。本题需要命名的状态来自维护 buy1、sell1、buy2、sell2 四个状态，按天更新，状态定义必须让未来决策不再依赖完整历史。

DP 状态必须足够描述未来。本题的维护 buy1、sell1、buy2、sell2 四个状态，按天更新要写成数组下标、容量、前缀长度、持有状态或区间边界。初始化对应空状态，遍历顺序对应依赖方向。放到“动态规划与状态定义”这条主线里看，关键原则是：状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择，二维 DP 看两个前缀或网格位置，背包看物品阶段和容量，区间 DP 看最后合并点。

DP 证明通常看最后一步。任意最优解或合法方案的最后一步必在转移枚举中；每个转移来源又已经按遍历顺序算完。本题的维护 buy1、sell1、buy2、sell2 四个状态，按天更新要支撑这个依赖关系。这道题的边界不能留到最后补丁式处理，实验时覆盖价格为空、只够一次交易、两次交易间不能重叠，先打印完整 DP 表，再比较空间压缩版本。若压缩版不同，优先检查遍历顺序和旧值保存。

C++ 实现上，DP 表初始化要对应状态语义，不可达哨兵不能溢出。空间压缩前先写完整表；压缩后要说清读到的是上一轮、本轮左侧还是左上角旧值。如果追问变成“最多 k 次交易是它的泛化，k 很大时退化为无限次交易”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 139 单词拆分

先写递归或枚举，把重复子问题暴露出来。本题的模型是状态表示前缀能否由字典单词拼出；慢点不是递归形式，而是相同状态被反复求。本题需要命名的状态来自枚举最后一个单词的起点，左侧前缀可达且子串在字典中则当前可达，状态定义必须让未来决策不再依赖完整历史。

DP 状态必须足够描述未来。本题的枚举最后一个单词的起点，左侧前缀可达且子串在字典中则当前可达要写成数组下标、容量、前缀长度、持有状态或区间边界。初始化对应空状态，遍历顺序对应依赖方向。放到“动态规划与状态定义”这条主线里看，关键原则是：状态定义要包含足够信息

但不能过度膨胀。线性 DP 看最后一步选择，二维 DP 看两个前缀或网格位置，背包看物品阶段和容量，区间 DP 看最后合并点。

DP 证明通常看最后一步。任意最优解或合法方案的最后一步必在转移枚举中；每个转移来源又已经按遍历顺序算完。本题的枚举最后一个单词的起点，左侧前缀可达且子串在字典中则当前可达要支撑这个依赖关系。这道题的边界不能留到最后补丁式处理，实验时覆盖空串、重复单词、长单词、substr 复制成本，先打印完整 DP 表，再比较空间压缩版本。若压缩版不同，优先检查遍历顺序和旧值保存。

C++ 实现上，DP 表初始化要对应状态语义，不可达哨兵不能溢出。空间压缩前先写完整表；压缩后要说清读到的是上一轮、本轮左侧还是左上角旧值。如果追问变成“若要输出所有句子，需要 DFS 加记忆化或前驱列表”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 152 乘积最大子数组

先写递归或枚举，把重复子问题暴露出来。本题的模型是负数会让最大乘积和最小乘积互换；慢点不是递归形式，而是相同状态被反复求。本题需要命名的状态来自同时维护以当前位置结尾的最大和最小乘积，状态定义必须让未来决策不再依赖完整历史。

DP 状态必须足够描述未来。本题的同时维护以当前位置结尾的最大和最小乘积要写成数组下标、容量、前缀长度、持有状态或区间边界。初始化对应空状态，遍历顺序对应依赖方向。放到“动态规划与状态定义”这条主线里看，关键原则是：状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择，二维 DP 看两个前缀或网格位置，背包看物品阶段和容量，区间 DP 看最后合并点。

DP 证明通常看最后一步。任意最优解或合法方案的最后一步必在转移枚举中；每个转移来源又已经按遍历顺序算完。本题的同时维护以当前位置结尾的最大和最小乘积要支撑这个依赖关系。这道题的边界不能留到最后补丁式处理，实验时覆盖零、负数个数奇偶、单元素、乘积溢出，先打印完整 DP 表，再比较空间压缩版本。若压缩版不同，优先检查遍历顺序和旧值保存。

C++ 实现上，DP 表初始化要对应状态语义，不可达哨兵不能溢出。空间压缩前先写完整表；压缩后要说清读到的是上一轮、本轮左侧还是左上角旧值。如果追问变成“如果要求返回区间，同步记录状态来源”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 198 打家劫舍

先写递归或枚举，把重复子问题暴露出来。本题的模型是每间房只有偷和不偷两种结局，偷当前就不能偷前一间；慢点不是递归形式，而是相同状态被反复求。本题需要命名的状态来自滚动保存前一状态和前两状态，状态定义必须让未来决策不再依赖完整历史。

DP 状态必须足够描述未来。本题的滚动保存前一状态和前两状态要写成数组下标、容量、前缀长度、持有状态或区间边界。初始化对应空状态，遍历顺序对应依赖方向。放到“动态规划与状态定义”这条主线里看，关键原则是：状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择，二维 DP 看两个前缀或网格位置，背包看物品阶段和容量，区间 DP 看最后合并点。

DP 证明通常看最后一步。任意最优解或合法方案的最后一步必在转移枚举中；每个转移来源又已经按遍历顺序算完。本题的滚动保存前一状态和前两状态要支撑这个依赖关系。这道题的边界不能留到最后补丁式处理，实验时覆盖空数组、单间、全零、金额很大，先打印完整 DP 表，再比较空间压缩版本。若压缩版不同，优先检查遍历顺序和旧值保存。

C++ 实现上，DP 表初始化要对应状态语义，不可达哨兵不能溢出。空间压缩前先写完整表；压缩后要说清读到的是上一轮、本轮左侧还是左上角旧值。如果追问变成“环形房屋要拆成不含首和不含尾两个线性问题”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 221 最大正方形

先写递归或枚举，把重复子问题暴露出来。本题的模型是以当前位置为右下角的最大正方形由上、左、左上三个状态限制；慢点不是递归形式，而是相同状态被反复求。本题需要命名的状态来自当前位置为一时取三者最小加一，否则为零，状态定义必须让未来决策不再依赖完整历史。

DP 状态必须足够描述未来。本题的当前位置为一时取三者最小加一，否则为零要写成数组下

标、容量、前缀长度、持有状态或区间边界。初始化对应空状态，遍历顺序对应依赖方向。放到“动态规划与状态定义”这条主线里看，关键原则是：状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择，二维 DP 看两个前缀或网格位置，背包看物品阶段和容量，区间 DP 看最后合并点。

DP 证明通常看最后一步。任意最优解或合法方案的最后一步必在转移枚举中；每个转移来源又已经按遍历顺序算完。本题的当前位置为一时取三者最小加一，否则为零要支撑这个依赖关系。这道题的边界不能留到最后补丁式处理，实验时覆盖全零、全一、单行单列、字符一和数字一不要混淆，先打印完整 DP 表，再比较空间压缩版本。若压缩版不同，优先检查遍历顺序和旧值保存。

C++ 实现上，DP 表初始化要对应状态语义，不可达哨兵不能溢出。空间压缩前先写完整表；压缩后要说清读到的是上一轮、本轮左侧还是左上角旧值。如果追问变成“最大矩形则需要按行构造柱状图再用单调栈”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 300 最长递增子序列

先写递归或枚举，把重复子问题暴露出来。本题的模型是二次 DP 固定结尾，二分优化维护每个长度的最小结尾；慢点不是递归形式，而是相同状态被反复求。本题需要命名的状态来自 tails 不是实际答案序列，而是未来扩展潜力表，状态定义必须让未来决策不再依赖完整历史。

DP 状态必须足够描述未来。本题的 tails 不是实际答案序列，而是未来扩展潜力表要写成数组下标、容量、前缀长度、持有状态或区间边界。初始化对应空状态，遍历顺序对应依赖方向。放到“动态规划与状态定义”这条主线里看，关键原则是：状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择，二维 DP 看两个前缀或网格位置，背包看物品阶段和容量，区间 DP 看最后合并点。

DP 证明通常看最后一步。任意最优解或合法方案的最后一步必在转移枚举中；每个转移来源又已经按遍历顺序算完。本题的 tails 不是实际答案序列，而是未来扩展潜力表要支撑这个依赖关系。这道题的边界不能留到最后补丁式处理，实验时覆盖重复元素、严格递增和非递减区别、空数组，先打印完整 DP 表，再比较空间压缩版本。若压缩版不同，优先检查遍历顺序和旧值保存。

C++ 实现上，DP 表初始化要对应状态语义，不可达哨兵不能溢出。空间压缩前先写完整表；压缩后要说清读到的是上一轮、本轮左侧还是左上角旧值。如果追问变成“若要还原路径，需要额外前驱数组，不能只看 tails”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 309 最佳买卖股票含冷冻期

先写递归或枚举，把重复子问题暴露出来。本题的模型是冷冻期让买入来源受昨天卖出状态限制；慢点不是递归形式，而是相同状态被反复求。本题需要命名的状态来自状态机维护持有、刚卖出、休息三种日终状态，状态定义必须让未来决策不再依赖完整历史。

DP 状态必须足够描述未来。本题的状态机维护持有、刚卖出、休息三种日终状态要写成数组下标、容量、前缀长度、持有状态或区间边界。初始化对应空状态，遍历顺序对应依赖方向。放到“动态规划与状态定义”这条主线里看，关键原则是：状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择，二维 DP 看两个前缀或网格位置，背包看物品阶段和容量，区间 DP 看最后合并点。

DP 证明通常看最后一步。任意最优解或合法方案的最后一步必在转移枚举中；每个转移来源又已经按遍历顺序算完。本题的状态机维护持有、刚卖出、休息三种日终状态要支撑这个依赖关系。这道题的边界不能留到最后补丁式处理，实验时覆盖空价格、一天、连续上涨、卖出后不能次日买入，先打印完整 DP 表，再比较空间压缩版本。若压缩版不同，优先检查遍历顺序和旧值保存。

C++ 实现上，DP 表初始化要对应状态语义，不可达哨兵不能溢出。空间压缩前先写完整表；压缩后要说清读到的是上一轮、本轮左侧还是左上角旧值。如果追问变成“手续费版本可以在买入或卖出转移中扣费”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 714 买卖股票含手续费

先写递归或枚举，把重复子问题暴露出来。本题的模型是手续费改变交易收益，仍可用持有和不持有状态；慢点不是递归形式，而是相同状态被反复求。本题需要命名的状态来自卖出时扣费或买入

时扣费都可以，但不要重复扣，状态定义必须让未来决策不再依赖完整历史。

DP 状态必须足够描述未来。本题的卖出时扣费或买入时扣费都可以，但不要重复扣要写成数组下标、容量、前缀长度、持有状态或区间边界。初始化对应空状态，遍历顺序对应依赖方向。放到“动态规划与状态定义”这条主线里看，关键原则是：状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择，二维 DP 看两个前缀或网格位置，背包看物品阶段和容量，区间 DP 看最后合并点。

DP 证明通常看最后一步。任意最优解或合法方案的最后一步必在转移枚举中；每个转移来源又已经按遍历顺序算完。本题的卖出时扣费或买入时扣费都可以，但不要重复扣要支撑这个依赖关系。这道题的边界不能留到最后补丁式处理，实验时覆盖手续费为零、价格震荡、小利润不足手续费，先打印完整 DP 表，再比较空间压缩版本。若压缩版不同，优先检查遍历顺序和旧值保存。

C++ 实现上，DP 表初始化要对应状态语义，不可达哨兵不能溢出。空间压缩前先写完整表；压缩后要说清读到的是上一轮、本轮左侧还是左上角旧值。如果追问变成“这是状态机比局部贪心更稳的例子”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 1143 最长公共子序列

先写递归或枚举，把重复子问题暴露出来。本题的模型是状态表示两个前缀的最长公共子序列长度；慢点不是递归形式，而是相同状态被反复求。本题需要命名的状态来自末尾相同取左上加一，不同取上方和左方最大，状态定义必须让未来决策不再依赖完整历史。

DP 状态必须足够描述未来。本题的末尾相同取左上加一，不同取上方和左方最大要写成数组下标、容量、前缀长度、持有状态或区间边界。初始化对应空状态，遍历顺序对应依赖方向。放到“动态规划与状态定义”这条主线里看，关键原则是：状态定义要包含足够信息但不能过度膨胀。线性 DP 看最后一步选择，二维 DP 看两个前缀或网格位置，背包看物品阶段和容量，区间 DP 看最后合并点。

DP 证明通常看最后一步。任意最优解或合法方案的最后一步必在转移枚举中；每个转移来源又已经按遍历顺序算完。本题的末尾相同取左上加一，不同取上方和左方最大要支撑这个依赖关系。这道题的边界不能留到最后补丁式处理，实验时覆盖空串、完全相同、无公共字符、子序列不是子串，先打印完整 DP 表，再比较空间压缩版本。若压缩版不同，优先检查遍历顺序和旧值保存。

C++ 实现上，DP 表初始化要对应状态语义，不可达哨兵不能溢出。空间压缩前先写完整表；压缩后要说清读到的是上一轮、本轮左侧还是左上角旧值。如果追问变成“若要求还原序列，需要从表右下角反向追踪选择”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

背包模型工作坊

先枚举物品选择，再把剩余容量作为状态。0-1 背包倒序容量，完全背包正序容量，组合和排列的循环顺序不能混用。

LeetCode 279 完全平方数

先枚举每个物品选或不选、选几次。本题的模型是完全平方数可以看成无限使用的物品，目标是凑出 n 的最少数目；暴力慢在相同阶段和容量反复出现。优化点完全背包最少物品数，或把数字看成图节点做 BFS 决定容量循环方向、状态值含义和是否允许当前物品在同一轮被再次使用。

背包状态围绕阶段和容量。本题的完全背包最少物品数，或把数字看成图节点做 BFS 决定倒序还是正序、求最值还是计数、组合还是排列。循环顺序不是写法偏好，而是题意的一部分。放到“背包模型”这条主线里看，关键原则是：0-1 背包每个物品只能用一次，一维压缩时容量倒序；完全背包物品可无限使用，容量正序。计数组合和计数排列的循环顺序不同。

背包证明围绕当前物品使用次数。0-1 只有选和不选，完全背包允许同一物品继续转移。本题的完全背包最少物品数，或把数字看成图节点做 BFS 要与循环方向一致，否则会把题意改掉。这道题的边界不能留到最后补丁式处理，实验时覆盖 n 为零或一、完全平方数本身、较大 n ，并故意把容量方向写反构造最小反例。这样能确认循环方向来自题意，不是背模板。

C++ 实现上，一维背包数组的容量方向要和题意一致。方案数用 `long long` 或题目要求的取模；容量来自数值大小时，复杂度是伪多项式。如果追问变成“数学四平方定理可给更强解法，但面试题 DP 更通用”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，

还是换成另一类结构。

LeetCode 322 零钱兑换

先枚举每个物品选或不选、选几次。本题的模型是金额是容量，硬币可无限使用，目标是最少硬币数；暴力慢在相同阶段和容量反复出现。优化点 $dp[value]$ 从 $value$ 减 $coin$ 的已知状态转移决定容量循环方向、状态值含义和是否允许当前物品在同一轮被再次使用。

背包状态围绕阶段和容量。本题的 $dp[value]$ 从 $value$ 减 $coin$ 的已知状态转移决定倒序还是正序、求最值还是计数、组合还是排列。循环顺序不是写法偏好，而是题意的一部分。放到“背包模型”这条主线里看，关键原则是：0-1 背包每个物品只能用一次，一维压缩时容量倒序；完全背包物品可无限使用，容量正序。计数组合和计数排列的循环顺序不同。

背包证明围绕当前物品使用次数。0-1 只有选和不选，完全背包允许同一物品继续转移。本题的 $dp[value]$ 从 $value$ 减 $coin$ 的已知状态转移要与循环方向一致，否则会把题意改掉。这道题的边界不能留到最后补丁式处理，实验时覆盖无法凑出、amount 为零、硬币面额大于目标、哨兵溢出，并故意把容量方向写反构造最小反例。这样能确认循环方向来自题意，不是背模板。

C++ 实现上，一维背包数组的容量方向要和题意一致。方案数用 `long long` 或题目要求的取模；容量来自数值大小时，复杂度是伪多项式。如果追问变成“零钱兑换 II 问组合数，循环语义不同”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 $value$ 语义，还是换成另一类结构。

LeetCode 416 分割等和子集

先枚举每个物品选或不选、选几次。本题的模型是总和一半是背包容量，每个数只能使用一次；暴力慢在相同阶段和容量反复出现。优化点 0-1 背包可达性，一维容量必须倒序决定容量循环方向、状态值含义和是否允许当前物品在同一轮被再次使用。

背包状态围绕阶段和容量。本题的 0-1 背包可达性，一维容量必须倒序决定倒序还是正序、求最值还是计数、组合还是排列。循环顺序不是写法偏好，而是题意的一部分。放到“背包模型”这条主线里看，关键原则是：0-1 背包每个物品只能用一次，一维压缩时容量倒序；完全背包物品可无限使用，容量正序。计数组合和计数排列的循环顺序不同。

背包证明围绕当前物品使用次数。0-1 只有选和不选，完全背包允许同一物品继续转移。本题的 0-1 背包可达性，一维容量必须倒序要与循环方向一致，否则会把题意改掉。这道题的边界不能留到最后补丁式处理，实验时覆盖总和为奇数、目标为零、数值较大、复杂度依赖 $target$ ，并故意把容量方向写反构造最小反例。这样能确认循环方向来自题意，不是背模板。

C++ 实现上，一维背包数组的容量方向要和题意一致。方案数用 `long long` 或题目要求的取模；容量来自数值大小时，复杂度是伪多项式。如果追问变成“负数会破坏普通容量模型，需要重新建模”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 $value$ 语义，还是换成另一类结构。

LeetCode 494 目标和

先枚举每个物品选或不选、选几次。本题的模型是正负号选择可代数转换为选若干数凑出特定正和；暴力慢在相同阶段和容量反复出现。优化点若 S 加 $target$ 为奇数或目标绝对值过大，直接无解决定容量循环方向、状态值含义和是否允许当前物品在同一轮被再次使用。

背包状态围绕阶段和容量。本题的若 S 加 $target$ 为奇数或目标绝对值过大，直接无解决定倒序还是正序、求最值还是计数、组合还是排列。循环顺序不是写法偏好，而是题意的一部分。放到“背包模型”这条主线里看，关键原则是：0-1 背包每个物品只能用一次，一维压缩时容量倒序；完全背包物品可无限使用，容量正序。计数组合和计数排列的循环顺序不同。

背包证明围绕当前物品使用次数。0-1 只有选和不选，完全背包允许同一物品继续转移。本题的若 S 加 $target$ 为奇数或目标绝对值过大，直接无解要与循环方向一致，否则会把题意改掉。这道题的边界不能留到最后补丁式处理，实验时覆盖零元素会让方案数翻倍、目标为负、容量为零，并故意把容量方向写反构造最小反例。这样能确认循环方向来自题意，不是背模板。

C++ 实现上，一维背包数组的容量方向要和题意一致。方案数用 `long long` 或题目要求的取模；容量来自数值大小时，复杂度是伪多项式。如果追问变成“也可用哈希 DP 保存所有可能和，但背包转换更高效”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 $value$ 语义，还是换成另一类结构。

LeetCode 518 零钱兑换 II

先枚举每个物品选或不选、选几次。本题的模型是硬币无限使用，问组合数而不是最少数量；暴力在相同阶段和容量反复出现。优化点外层遍历硬币、内层金额正序，保证组合按固定硬币顺序统计决定容量循环方向、状态值含义和是否允许当前物品在同一轮被再次使用。

背包状态围绕阶段和容量。本题的外层遍历硬币、内层金额正序，保证组合按固定硬币顺序统计决定倒序还是正序、求最值还是计数、组合还是排列。循环顺序不是写法偏好，而是题意的一部分。放到“背包模型”这条主线里看，关键原则是：0-1 背包每个物品只能用一次，一维压缩时容量倒序；完全背包物品可无限使用，容量正序。计数组合和计数排列的循环顺序不同。

背包证明围绕当前物品使用次数。0-1 只有选和不选，完全背包允许同一物品继续转移。本题的外层遍历硬币、内层金额正序，保证组合按固定硬币顺序统计要与循环方向一致，否则会把题意改掉。这道题的边界不能留到最后补丁式处理，实验时覆盖 amount 为零、无硬币、组合数较大、循环顺序反例，并故意把容量方向写反构造最小反例。这样能确认循环方向来自题意，不是背模板。

C++ 实现上，一维背包数组的容量方向要和题意一致。方案数用 `long long` 或题目要求的取模；容量来自数值大小时，复杂度是伪多项式。如果追问变成“若外层金额内层硬币，会统计排列数”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

贪心与交换证明工作坊

先把选择空间完整枚举，再寻找可以交换到最优解前面的局部选择。贪心必须能证明当前选择不会让后续资源变差，而不是只看排序后代码短。

LeetCode 55 跳跃游戏

先把选择顺序完整枚举，哪怕这个版本只能处理很小输入。本题的模型是扫描过程中只关心当前能到达的最远位置；真正要证明的是如果下标超过最远可达，任何之前路径都不能到达它；否则用它扩展边界为什么不会伤害后续选择。没有这一步证明，排序或局部选择都只是猜测。

贪心状态要表达当前方案为什么不落后。本题围绕如果下标超过最远可达，任何之前路径都不能到达它；否则用它扩展边界保留局部最优边界；排序只是把可比较的候选排到一起，证明仍要落在交换或领先关系上。放到“贪心与交换证明”这条主线里看，关键原则是：贪心需要找到可交换的局部最优：最早结束、最小右端、当前最远覆盖、当前最大收益或最小代价。排序只是手段，不是证明。

贪心证明要么用交换，要么用领先性。围绕如果下标超过最远可达，任何之前路径都不能到达它；否则用它扩展边界说明把任意最优解的第一步换成贪心选择不会变差，或说明当前方案在同等步数下始终不落后。这道题的边界不能留到最后补丁式处理，实验时除了第一个元素为零、数组长度一、恰好到达最后、远超最后，还要故意替换成一个看似合理但错误的排序规则，构造反例。能解释错误贪心为何失败，才算理解正确贪心。

C++ 实现上，比较器必须满足严格弱序，相同关键字的处理要服务于证明。涉及区间端点、收益和代价时，排序键要写成业务含义，不要只凭直觉写小于号。如果追问变成“求最少跳数时把可达区间看成 BFS 层”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 122 买卖股票 II

先把选择顺序完整枚举，哪怕这个版本只能处理很小输入。本题的模型是允许多次交易且不能同时持有，所有正收益差都可以收集；真正要证明的是把长上涨段拆成每天买卖收益相同，因此累加正差值最优为什么不会伤害后续选择。没有这一步证明，排序或局部选择都只是猜测。

贪心状态要表达当前方案为什么不落后。本题围绕把长上涨段拆成每天买卖收益相同，因此累加正差值最优保留局部最优边界；排序只是把可比较的候选排到一起，证明仍要落在交换或领先关系上。放到“贪心与交换证明”这条主线里看，关键原则是：贪心需要找到可交换的局部最优：最早结束、最小右端、当前最远覆盖、当前最大收益或最小代价。排序只是手段，不是证明。

贪心证明要么用交换，要么用领先性。围绕把长上涨段拆成每天买卖收益相同，因此累加正差值最优说明把任意最优解的第一步换成贪心选择不会变差，或说明当前方案在同等步数下始终不落后。这道题的边界不能留到最后补丁式处理，实验时除了下降段、平台价格、只有一天、手续费不存在，还要故意替换成一个看似合理但错误的排序规则，构造反例。能解释错误贪心为何失败，才算

理解正确贪心。

C++ 实现上，比较器必须满足严格弱序，相同关键字的处理要服务于证明。涉及区间端点、收益和代价时，排序键要写成业务含义，不要只凭直觉写小于号。如果追问变成“有手续费时不能简单累加正差，需要状态机或调整贪心阈值”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 169 多数元素

先把选择顺序完整枚举，哪怕这个版本只能处理很小输入。本题的模型是超过一半的元素可以和其他元素两两抵消后仍然剩下；真正要证明的是 Boyer-Moore 投票维护候选和计数为什么不会伤害后续选择。没有这一步证明，排序或局部选择都只是猜测。

贪心状态要表达当前方案为什么不落后。本题围绕 Boyer-Moore 投票维护候选和计数保留局部最优边界；排序只是把可比较的候选排到一起，证明仍要落在交换或领先关系上。放到“贪心与交换证明”这条主线里看，关键原则是：贪心需要找到可交换的局部最优：最早结束、最小右端、当前最远覆盖、当前最大收益或最小代价。排序只是手段，不是证明。

贪心证明要么用交换，要么用领先性。围绕 Boyer-Moore 投票维护候选和计数说明把任意最优解的第一步换成贪心选择不会变差，或说明当前方案在同等步数下始终不落后。这道题的边界不能留到最后补丁式处理，实验时除了题目是否保证存在多数、计数归零、全相同，还要故意替换成一个看似合理但错误的排序规则，构造反例。能解释错误贪心为何失败，才算理解正确贪心。

C++ 实现上，比较器必须满足严格弱序，相同关键字的处理要服务于证明。涉及区间端点、收益和代价时，排序键要写成业务含义，不要只凭直觉写小于号。如果追问变成“若找超过三分之一的元素，需要维护两个候选”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 435 无重叠区间

先把选择顺序完整枚举，哪怕这个版本只能处理很小输入。本题的模型是保留最多不重叠区间等价于删除最少区间；真正要证明的是按终点升序选择，结束越早给后续留下空间越多为什么不会伤害后续选择。没有这一步证明，排序或局部选择都只是猜测。

贪心状态要表达当前方案为什么不落后。本题围绕按终点升序选择，结束越早给后续留下空间越多保留局部最优边界；排序只是把可比较的候选排到一起，证明仍要落在交换或领先关系上。放到“贪心与交换证明”这条主线里看，关键原则是：贪心需要找到可交换的局部最优：最早结束、最小右端、当前最远覆盖、当前最大收益或最小代价。排序只是手段，不是证明。

贪心证明要么用交换，要么用领先性。围绕按终点升序选择，结束越早给后续留下空间越多说明把任意最优解的第一步换成贪心选择不会变差，或说明当前方案在同等步数下始终不落后。这道题的边界不能留到最后补丁式处理，实验时除了端点相接是否重叠、完全覆盖、相同终点、空列表，还要故意替换成一个看似合理但错误的排序规则，构造反例。能解释错误贪心为何失败，才算理解正确贪心。

C++ 实现上，比较器必须满足严格弱序，相同关键字的处理要服务于证明。涉及区间端点、收益和代价时，排序键要写成业务含义，不要只凭直觉写小于号。如果追问变成“用交换论证说明最早结束的选择不劣于其他第一选择”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 452 用最少数量的箭引爆气球

先把选择顺序完整枚举，哪怕这个版本只能处理很小输入。本题的模型是一支箭的位置可以覆盖所有包含该位置的区间；真正要证明的是按右端点排序，当前箭放在最早结束气球的右端为什么不会伤害后续选择。没有这一步证明，排序或局部选择都只是猜测。

贪心状态要表达当前方案为什么不落后。本题围绕按右端点排序，当前箭放在最早结束气球的右端保留局部最优边界；排序只是把可比较的候选排到一起，证明仍要落在交换或领先关系上。放到“贪心与交换证明”这条主线里看，关键原则是：贪心需要找到可交换的局部最优：最早结束、最小右端、当前最远覆盖、当前最大收益或最小代价。排序只是手段，不是证明。

贪心证明要么用交换，要么用领先性。围绕按右端点排序，当前箭放在最早结束气球的右端说明把任意最优解的第一步换成贪心选择不会变差，或说明当前方案在同等步数下始终不落后。这道题的边界不能留到最后补丁式处理，实验时除了端点相同、负坐标、区间包含、闭区间边界，还要故

意替换成一个看似合理但错误的排序规则，构造反例。能解释错误贪心为何失败，才算理解正确贪心。

C++ 实现上，比较器必须满足严格弱序，相同关键字的处理要服务于证明。涉及区间端点、收益和代价时，排序键要写成业务含义，不要只凭直觉写小于号。如果追问变成“它和无重叠区间是同一类最早结束贪心”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 763 划分字母区间

先把选择顺序完整枚举，哪怕这个版本只能处理很小输入。本题的模型是每个片段必须覆盖其中所有字符的最后出现位置；真正要证明的是扫描维护当前片段最远右端，到达右端即可切分为什么不会伤害后续选择。没有这一步证明，排序或局部选择都只是猜测。

贪心状态要表达当前方案为什么不落后。本题围绕扫描维护当前片段最远右端，到达右端即可切分保留局部最优边界；排序只是把可比较的候选排到一起，证明仍要落在交换或领先关系上。放到“贪心与交换证明”这条主线里看，关键原则是：贪心需要找到可交换的局部最优：最早结束、最小右端、当前最远覆盖、当前最大收益或最小代价。排序只是手段，不是证明。

贪心证明要么用交换，要么用领先性。围绕扫描维护当前片段最远右端，到达右端即可切分说明把任意最优解的第一步换成贪心选择不会变差，或说明当前方案在同等步数下始终不落后。这道题的边界不能留到最后补丁式处理，实验时除了单字符、所有字符相同、字符多次跨段，还要故意替换成一个看似合理但错误的排序规则，构造反例。能解释错误贪心为何失败，才算理解正确贪心。

C++ 实现上，比较器必须满足严格弱序，相同关键字的处理要服务于证明。涉及区间端点、收益和代价时，排序键要写成业务含义，不要只凭直觉写小于号。如果追问变成“这是最早合法切分贪心，能最大化片段数量”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

区间、排序与合并工作坊

先两两比较或完整重排，再利用排序后相交区间连续出现的性质。动态区间问题还要关注前驱、后继、端点闭开和覆盖关系。

LeetCode 56 合并区间

先两两比较区间，确认相交、覆盖或冲突的定义。本题的模型是排序后可能与当前区间相交的只会是结果尾部；暴力慢在每个区间都回头看太多无关区间。优化点按起点排序，扫描时维护结果最后区间的右端点通常依赖排序后相邻关系，必须先定义端点闭开。

区间结构的状态是当前右端、结果尾区间或动态有序表的相邻关系。本题的按起点排序，扫描时维护结果最后区间的右端点只允许丢掉已经不可能再冲突的区间；端点相等是否合并要先定义。放到“区间、排序与合并”这条主线里看，关键原则是：合并按起点排序，调度按终点排序，覆盖题常用起点升序且终点降序。扫描时维护当前最远右端、结果尾区间或已选区间终点。

区间问题证明依赖排序后的邻接关系。若当前区间无法影响结果尾部或相邻区间，它就不会再影响更远的区间。本题的按起点排序，扫描时维护结果最后区间的右端点要说明哪些区间已经安全归档。这道题的边界不能留到最后补丁式处理，实验时覆盖端点相接、完全覆盖、无重叠、空列表、输入未排序，画出排序后的区间序列和当前结果尾部。动态版本要专门测前驱、后继和端点相接。

C++ 实现上，区间可用 `std::vector<std::array<int, 2>` 或结构体表达端点，排序比较器要处理相同起点。动态区间需要 `std::map` 的前驱后继，而不是哈希表。如果追问变成“若区间是半开区间，端点相等不一定合并”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 57 插入区间

先两两比较区间，确认相交、覆盖或冲突的定义。本题的模型是新区间只会影响与它相交的一段连续区间；暴力慢在每个区间都回头看太多无关区间。优化点先追加左侧完全不相交区间，再合并重叠段，最后追加右侧通常依赖排序后相邻关系，必须先定义端点闭开。

区间结构的状态是当前右端、结果尾区间或动态有序表的相邻关系。本题的先追加左侧完全不相交区间，再合并重叠段，最后追加右侧只允许丢掉已经不可能再冲突的区间；端点相等是否合并要先定义。放到“区间、排序与合并”这条主线里看，关键原则是：合并按起点排序，调度按终点排

序，覆盖题常用起点升序且终点降序。扫描时维护当前最远右端、结果尾区间或已选区间终点。

区间问题证明依赖排序后的邻接关系。若当前区间无法影响结果尾部或相邻区间，它就不会再影响更远的区间。本题的先追加左侧完全不相交区间，再合并重叠段，最后追加右侧要说明哪些区间已经安全归档。这道题的边界不能留到最后补丁式处理，实验时覆盖新区间在最前、最后、覆盖全部、刚好相邻、原列表为空，画出排序后的区间序列和当前结果尾部。动态版本要专门测前驱、后继和端点相接。

C++ 实现上，区间可用 `std::vector<std::array<int, 2>` 或结构体表达端点，排序比较器要处理相同起点。动态区间需要 `std::map` 的前驱后继，而不是哈希表。如果追问变成“若输入不是有序且不重叠，必须先排序或换模型”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 1288 删除被覆盖区间

先两两比较区间，确认相交、覆盖或冲突的定义。本题的模型是被覆盖要求另一区间起点不晚且终点不早；暴力慢在每个区间都回头看太多无关区间。优化点按起点升序、终点降序排序，扫描维护最大右端通常依赖排序后相邻关系，必须先定义端点闭开。

区间结构的状态是当前右端、结果尾区间或动态有序表的相邻关系。本题的按起点升序、终点降序排序，扫描维护最大右端只允许丢掉已经不可能再冲突的区间；端点相等是否合并要先定义。放到“区间、排序与合并”这条主线里看，关键原则是：合并按起点排序，调度按终点排序，覆盖题常用起点升序且终点降序。扫描时维护当前最远右端、结果尾区间或已选区间终点。

区间问题证明依赖排序后的邻接关系。若当前区间无法影响结果尾部或相邻区间，它就不会再影响更远的区间。本题的按起点升序、终点降序排序，扫描维护最大右端要说明哪些区间已经安全归档。这道题的边界不能留到最后补丁式处理，实验时覆盖同起点区间、完全覆盖、无覆盖、端点相等，画出排序后的区间序列和当前结果尾部。动态版本要专门测前驱、后继和端点相接。

C++ 实现上，区间可用 `std::vector<std::array<int, 2>` 或结构体表达端点，排序比较器要处理相同起点。动态区间需要 `std::map` 的前驱后继，而不是哈希表。如果追问变成“终点降序是关键，否则同起点小区间会先出现造成误判”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

位运算与状态压缩工作坊

先用计数或哈希保证正确，再寻找位运算的代数抵消、按位统计或函数图结构。位题的难点是证明被抵消的信息确实不影响答案。

LeetCode 136 只出现一次的数字

先用计数、集合或普通 DP 写出清楚版本。本题的模型是成对元素可以用异或抵消，剩下单个元素；位运算只是把状态压进二进制表示。优化点异或满足交换结合，扫描顺序不影响答案必须说明每一位或每次异或到底保留了什么信息。

位状态要给每一位固定含义。本题的异或满足交换结合，扫描顺序不影响答案如果依赖异或抵消，就要说明成对元素为什么完全消失；如果依赖掩码，就要说明同一位置不同掩码为什么不能合并。放到“位运算与状态压缩”这条主线里看，关键原则是：异或解决成对抵消，按位计数处理出现三次，掩码表示访问集合或可用集合，低位提取用于枚举候选。

位运算证明要逐位说明。异或、按位计数或掩码转移都不能只靠直觉；本题的异或满足交换结合，扫描顺序不影响答案必须保证被压掉的信息不会影响成对元素可以用异或抵消，剩下单个元素要求的答案。这道题的边界不能留到最后补丁式处理，实验时覆盖零、负数、唯一值在开头或结尾、所有其他值恰好两次，把数字写成二进制逐位检查。负数、最高位、空掩码和全集掩码都要单独测。

C++ 实现上，移位时优先使用无符号或足够宽的整数，位数超过 31 时考虑 `long long` 或 `std::bitset`。位运算表达式加括号，避免优先级误读。如果追问变成“若其他值出现三次，要改为按位计数取模”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 value 语义，还是换成另一类结构。

LeetCode 287 寻找重复数

先用计数、集合或普通 DP 写出清楚版本。本题的模型是数组值域使下标到值形成函数图，重复数是环入口；位运算只是把状态压进二进制表示。优化点快慢指针不修改数组且常数空间；值域二分

用抽屉原理必须说明每一位或每次异或到底保留了什么信息。

位状态要给每一位固定含义。本题的快慢指针不修改数组且常数空间；值域二分用抽屉原理如果依赖异或抵消，就要说明成对元素为什么完全消失；如果依赖掩码，就要说明同一位置不同掩码为什么不能合并。放到“位运算与状态压缩”这条主线里看，关键原则是：异或解决成对抵消，按位计数处理出现三次，掩码表示访问集合或可用集合，低位提取用于枚举候选。

位运算证明要逐位说明。异或、按位计数或掩码转移都不能只靠直觉；本题的快慢指针不修改数组且常数空间；值域二分用抽屉原理必须保证被压掉的信息不会影响数组值域使下标到值形成函数图，重复数是环入口要求的答案。这道题的边界不能留到最后补丁式处理，实验时覆盖重复数出现多次、不能排序、不能改数组、长度为 n 加一，把数字写成二进制逐位检查。负数、最高位、空掩码和全集掩码都要单独测。

C++ 实现上，移位时优先使用无符号或足够宽的整数，位数超过 31 时考虑 `long long` 或 `std::bitset`。位运算表达式加括号，避免优先级误读。如果追问变成“快慢指针要求值都能作为合法下标”，先判断原来的状态是否仍然覆盖新答案集合，再决定是微调边界、改 `value` 语义，还是换成另一类结构。

工作坊使用方式

不要一次性把本节当题解背完。建议每次选一个题型，先遮住优化落点，只看题面模型和暴力基线，自己写出慢在哪里；再看结构观察和正确性；最后用边界实验做对拍。能完成这个过程，才说明题目进入了自己的知识体系。

数据结构变种、工程应用和实验专题

这一组专题补齐数据结构的变种、工程应用和实验要求。它不是题号清单，而是把同一类结构放到刷题、C++ 容器和系统源码之间比较，帮助读者理解为什么同一个复杂度在不同场景下表现差异很大。

数组、字符串和连续内存的变种

数组的优势不是语法简单，而是连续内存、随机访问和下标可计算。刷题里常见的前缀和、差分、双指针、二分、原地交换，都依赖下标能在常数时间跳转。工程里数组还带来缓存局部性：顺序扫描通常比链表快很多，即使二者同为线性复杂度。学习数组题时，要把每个下标变量写成区间语义，例如左闭右开窗口、已处理前缀、未知区、已确认后缀。只要区间语义清楚，代码中的加一减一就有根据。

数组变种很多。前缀和把区间求和变成两个端点相减，适合静态区间查询；差分把区间加法变成两个端点标记，适合批量更新后一次还原；二维前缀和把矩形查询变成四个角组合；二维差分适合批量矩形更新；坐标压缩把大值域映射到紧凑下标，适合树状数组、线段树和计数数组。每个变种都要问一个问题：它把哪类重复工作提前保存了，代价是多了什么空间和初始化步骤。

字符串本质上也是字符数组，但它多了编码、复制和匹配成本。力扣基础题常假设小写字母，此时 `std::array<int, 26>` 比哈希表更直接；若扩展到 ASCII，可以用 128 或 256 长度数组；若扩展到 Unicode，就不能简单按字节计数。字符串切片也要注意成本，`substr` 可能创建新字符串，热循环里更适合用下标区间、`string_view` 或 Trie。面试时如果能主动说明字符集假设，答案会更稳。

数组题的实验很直接。第一，给一个窗口题打印 `left`、`right`、窗口内容和计数表，观察每次移动只改变一个元素。第二，把二维矩阵随机生成后，用暴力矩形求和和二维前缀和对拍，确认四角公式。第三，把 `vector` 和 `list` 各存一百万个整数，分别顺序求和，比较时间，理解缓存局部性。第四，故意保存 `vector` 元素地址后继续 `push_back`，观察扩容后旧地址失效。

链表、侵入式节点和稳定身份

链表题的难点不是没有下标，而是节点身份和连接关系分离。数组元素的位置由下标决定，移动元素会改变位置；链表节点的位置由前后指针决定，重连节点不会改变节点自身地址。这个差异让链表适合 LRU、任务队列、空闲块链、内核对象列表和需要稳定迭代器的场景。刷题里的 `ListNode` 是最简模型，真实工程里一个业务对象可能带多个链表节点，分别挂进不同索引。

链表有多种变种。单链表空间少，适合反转、合并、快慢指针和环检测；双链表能从已知节点 $O(1)$ 删除，适合 LRU 和双端队列；循环链表适合轮转和哨兵化边界；跳表在多层链表上提供期望

对数查找；侵入式链表把链接字段放进业务对象，减少额外分配。每个变种都在回答访问模式：是否需要前驱，是否需要随机查找，是否需要稳定节点，是否会频繁移动到头尾。

链表面试要能讲副作用。反转链表会改变输入结构，回文链表若反转后半段，工程里最好恢复；合并链表可以复用节点，也可以新建节点，二者所有权不同；删除节点时如果平台管理内存，不应该随意释放输入节点；相交链表比较的是地址不是值；环形链表的快慢指针证明来自相对速度，而不是经验。只要这些边界说不清，代码短也不代表理解深。

链表实验建议做三组。第一，把反转链表每一步的 `prev`、`cur`、`next` 打印成边关系，确认没有丢失剩余链。第二，写 LRU 的 `get` 后移动到头部，统计哈希查询和链表移动次数。第三，写一个侵入式节点对象，同时挂入按时间排序链表和按 `key` 哈希桶，体会同一对象进入多个索引时删除顺序有多重要。

栈、队列、双端队列和单调结构

栈的核心是最近未完成结构。括号匹配保存最近未闭合左括号，表达式求值保存上一层上下文，单调栈保存还在等待未来元素解决的下标。不要把“用栈”当结论，必须说清栈顶代表什么，什么事件触发入栈，什么事件触发出栈，出栈时答案是否已经确定。柱状图最大矩形、每日温度、下一个更大元素、最长有效括号虽然都用栈，但弹出的语义完全不同。

队列的核心是按到达顺序处理。BFS 队列保证距离层次，任务队列保证提交顺序，滑动窗口队列保存候选下标。双端队列进一步允许两端进出，适合 0-1 BFS 和单调队列。单调队列不是普通队列加排序，而是在队尾删除被新元素支配的候选，在队首删除过期候选。每个元素最多进出一次，所以线性复杂度来自摊还分析。

单调结构最容易被写成模板。真正要问的是支配关系：一个旧候选为什么被新候选删除后永远不可能成为答案？滑动窗口最大值里，新值更大且更晚，旧小值在未来窗口中既不更大也不更持久；受限子序列和里，新 DP 值更大且下标更晚，也支配旧候选；柱状图中，遇到更矮柱时，被弹出柱子的右边界已经确定。支配关系说不清，单调结构就不可靠。

实验可以把队列状态打印出来。对滑动窗口最大值，记录每轮右端进入、队尾弹出、队首过期和答案读取。对 0-1 BFS，构造零权边和一边权混合的小图，观察零权边入队首如何保证距离顺序。对表达式求值，打印栈中数字和运算符，观察乘除为什么要立即处理，加减为什么可以延迟。

哈希、前缀状态和等价类

哈希表的本质是把对象归入等价类。两数之和把历史值映射到下标，字母异位词把字符串映射到规范签名，前缀和把区间条件映射成历史前缀，账户合并把邮箱映射到账户集合。哈希不是魔法，它只把等值查询变便宜；如果题目需要顺序、前驱、后继或范围最值，哈希表就不是合适结构。

前缀哈希题要先写代数关系。和为 K 的子数组来自 `prefix[j] - prefix[i] == k`；连续子数组和来自两个前缀同余；连续数组来自把 0 当成 -1 后找相同前缀；优美子数组来自奇数计数差。关系不同，哈希表保存内容也不同：计数题保存频次，最长题保存最早下标，判定题保存是否出现，最短题可能需要单调队列而不是普通哈希。

哈希 `key` 的设计决定正确性。异位词可以排序字符串做 `key`，也可以用字符计数做 `key`；计数 `key` 必须带分隔符，否则 `1,11` 和 `11,1` 可能混淆；结构化 `key` 可以用 `pair` 或自定义结构，但要提供哈希函数和相等比较；浮点数不适合直接做哈希 `key`，工程里要转成可比较的离散表示。面试中讲 `key` 比讲容器名字更重要。

实验建议做哈希对拍。第一，写和为 K 的子数组暴力版和哈希版，用包含负数、零和重复前缀的随机数组对拍。第二，把先查询后更新改成先更新后查询，观察 `target` 为 0 时如何出错。第三，给异位词分组设计没有分隔符的计数 `key`，构造冲突案例。第四，对 `unordered_map` 调用和不调用 `reserve`，比较大量插入时 `rehash` 次数。

堆、优先队列和动态候选集

堆解决的是动态候选集的极值查询。排序能给全局顺序，但每次插入删除后重新排序成本高；堆只维护足够的信息让堆顶是最大或最小。前 K 高频、数据流中位数、合并 K 个链表、任务调度、最小加油次数，都是把“下一步选谁”变成堆顶查询。要注意堆不能回答任意元素删除，也不能直接给有序遍历。

双堆是数据流中位数的核心。大顶堆保存较小一半，小顶堆保存较大一半，大小差不超过一，且大顶堆堆顶不大于小顶堆堆顶。两个不变量缺一不可。若要支持滑动窗口删除，就必须加延迟删除

表，删除请求先记录，等元素到堆顶时再真正弹出。这和 Linux RCU 的延迟释放不是同一机制，但都体现“逻辑删除”和“物理清理”分离。

堆题要警惕比较器。C++ `priority_queue` 默认大顶堆，小顶堆要用 `greater` 或自定义比较器。结构体比较器应该明确谁的优先级低，而不是凭直觉写小于号。若堆中保存 `pair`，默认比较会先比第一维再比第二维；如果业务只看频次或距离，就要写清楚 `tie-breaker`。比较器一旦不满足严格弱序，结果可能不稳定。

堆实验可以从三个版本比较。第 K 大元素用排序、大小为 K 的小顶堆和快速选择分别实现，比较复杂度和适用场景。合并 K 个有序链表用每轮扫描 K 个头、堆和分治三种方式实现，观察 N 和 K 对性能的影响。任务调度器同时写模拟堆版本和最高频公式版本，解释公式为什么不需要逐步模拟。

树、Trie、平衡树和区间树视角

树题的关键是状态方向。自顶向下问题把父节点约束传给子节点，例如验证 BST 的上下界；自底向上问题让子树向父节点汇报信息，例如平衡二叉树、最大路径和、直径；按层问题用队列维护层边界，例如层序遍历和最小深度。把方向说清楚，递归或迭代只是实现选择。

BST 的价值是有序性。中序遍历得到升序序列，查找可以按大小排除半棵树，前驱后继可以沿树结构寻找。普通二叉树没有这些性质，不能把 BST 的值域剪枝搬过去。红黑树、AVL、Treap、跳表都是为了在动态更新时保持近似平衡；力扣大多不要求实现平衡树，但 `map`、`set` 和 Linux `rbtree` 背后都依赖这一类思想。

Trie 把字符串集合按前缀共享。实现 Trie、单词搜索 II、搜索建议系统、异或最大值位 Trie 都是这个思想。数组子节点适合小固定字符集，哈希子节点适合稀疏大字符集；单词结束标记不能省，因为前缀存在不等于单词存在。Trie 的空间可能很大，工程里会考虑压缩 Trie、Radix Tree 或 DAWG 等变体。

树实验要同时做递归和迭代。用显式栈写中序遍历，比较它和递归版本的状态；用队列写层序遍历，打印每层 `size`；用 Trie 插入一批单词，统计节点数和共享前缀数；用 `std::map` 实现日程安排，理解前驱和后继为什么足以判断新区间冲突。

图、状态图和搜索维度

图题第一步永远是建模。网格题的节点可以是格子，也可以是格子加钥匙集合、剩余障碍次数或时间；课程表的节点是课程，边是依赖；单词接龙的节点是单词，边是一位字符变化；转盘锁的节点是四位状态，边是一次拨动。节点维度少了会漏解，节点维度多了会超时。

BFS 适合无权最短路，因为队列按层扩展，第一次到达就是最短。DFS 适合连通块、路径枚举和拓扑状态检测。拓扑排序适合有向依赖，入度为零代表当前可执行。并查集适合动态连通，但不保存路径。最短路算法选择取决于边权：等权 BFS，非负 Dijkstra，负边 Bellman-Ford 或 SPFA 风险评估，边权只有 0 和 1 则 0-1 BFS。

图的性能常败在邻居生成。单词接龙如果每次和所有单词比较，会很慢；通配模式桶把邻居查询降到共享模式。网格 BFS 如果出队才标记访问，可能被同层多个父节点重复入队；入队时标记能控制规模。课程表如果边方向建反，判环可能仍然对，但输出顺序会错。图题必须把邻接表、访问标记和状态编码讲清楚。

实验建议做状态维度对比。打开转盘锁写普通 BFS 和双向 BFS，统计扩展节点数。障碍消除最短路分别用二维 `visited` 和三维 `visited`，对比二维为何错误剪枝。课程表分别按两种边方向建图，观察输出顺序差异。最小体力路径写 Dijkstra、二分 BFS、并查集三版，比较它们利用的是哪种单调性。

动态规划状态表和空间压缩

DP 的起点不是公式，而是暴力搜索树。每个搜索节点描述处理到哪里、剩余资源是什么、已经做了什么选择；如果不同路径到达同一个节点后未来选择完全一样，就有重叠子问题。状态定义就是给这种节点命名。定义不稳定时，代码会偷偷依赖历史，边界也会变成补丁。

线性 DP 看最后一步，二维 DP 看两个前缀或网格位置，背包 DP 看物品阶段和容量，区间 DP 看最后合并点，树形 DP 看子树向父节点汇报的信息，状态压缩 DP 看集合中哪些元素已访问。每一类 DP 都要问：状态是否足够，转移是否覆盖全部选择，遍历顺序是否保证依赖已计算，初始化是否对应空状态。

空间压缩是高风险优化。0-1 背包倒序容量，是为了让当前物品只能用一次；完全背包正序容量，是为了允许当前物品重复使用；LCS 一维压缩需要保存左上角旧值；股票状态机需要先保存昨天状态，避免同一天污染。初学时先写清二维或完整状态，确认正确后再压缩。

DP 实验可以非常具体。打印编辑距离表，手动解释每个格子来自插入、删除还是替换。把 0-1 背包容量循环改成正序，用一个物品重复使用的反例验证错误。把零钱兑换 II 的循环顺序调换，观察组合数如何变成排列数。给股票冷冻期状态机打印每天 hold、sold、rest，观察买入来源为什么受限制。

综合案例、实验与源码对照

这一章专门补齐三个容易被刷题书忽略的环节：第一，经典题不能直接给最终答案，而要从暴力方法、瓶颈定位、结构观察和正确性证明一步步推出来；第二，C++ 容器不是语法糖，容器的内存模型、迭代器失效、比较器和分配成本会改变工程质量；第三，Linux 源码里的数据结构不是孤立技巧，而是访问模式、生命周期、并发和内存分配共同作用的结果。读本章时，不要把它当题号清单，而要把它当作一组可以反复做的实验。

经典题完整推导

LeetCode 1 两数之和

这道题看似简单，却是“从暴力到结构”的最小样本。题面要求找两个下标，值相加等于目标。最直接的暴力做法是两层循环枚举 i 和 j ，检查 `nums[i] + nums[j] == target`。这个版本一定正确，因为它覆盖了所有下标对；它的问题是同一个历史值会被未来很多位置反复扫描。把瓶颈说清楚后，优化就自然出现：当扫描到当前值 x 时，需要的另一个数只可能是 `target - x`，所以不必再遍历所有历史下标，只要查历史表即可。

这里的哈希表不是“模板”，而是把暴力中的历史扫描换成等值查询。表的 key 是已经出现过的数，value 是它的下标。顺序必须先查再插入当前数，因为题目不允许同一个下标使用两次。如果先插入再查，输入 `[3]`、目标 6 这种边界就会把当前元素和自己配对。重复数字也要靠这个顺序处理：输入 `[3,3]`、目标 6 时，扫描第二个 3 才能查到第一个 3。

正确性可以对照暴力证明。任意合法答案有两个下标 $i < j$ 。算法扫描到 j 时， i 的值已经插入历史表；由于 `nums[i] + nums[j] == target`，查 `target - nums[j]` 一定能找到 i 。反过来，算法找到的历史下标一定早于当前下标，且值满足目标和，所以返回结果合法。复杂度从 $O(n^2)$ 降到平均 $O(n)$ ，代价是 $O(n)$ 额外空间。实验时要覆盖没有答案、重复值、负数、目标为零、补数等于当前值这些输入。

LeetCode 3 无重复字符的最长子串

暴力版本可以枚举每个左端和右端，再检查子串内是否有重复字符。如果检查也用集合扫描，复杂度会很高。优化的第一步不是立刻说“滑动窗口”，而是观察连续子串的变化：右端向右扩展时，只新增一个字符；如果这个字符在当前窗口里重复，左端只需要移动到它上一次出现位置之后。左端永远不会向左回退，因为已经被排除的左端只会让窗口更长、更容易包含重复字符。

高质量实现通常保存字符上一次出现的位置。扫描右端 `right` 时，如果字符 `s[right]` 的上次位置在当前窗口内部，就把 `left` 更新为 `last[s[right]] + 1`；如果上次位置在窗口左侧之外，就不能把 `left` 拉回去。这里最常见的错误是直接写 `left = last[c] + 1`，导致输入 `abba` 时，处理最后一个 `a` 会把左端从 2 错误拉回 1。

这题的正确性来自窗口不变量：任意时刻，`[left, right]` 内没有重复字符，且 `left` 是在当前 `right` 下能保持无重复的最左合法边界。每次右端加入一个字符，只可能因为这个新字符破坏合法性；把左端移过它上次出现位置后，窗口重新合法。答案在每次窗口合法后更新。实验应打印 `left`、`right`、当前字符、上次位置和答案，特别观察 `abba`、空串、全相同字符、全部不同字符以及扩展字符集的情况。

LeetCode 15 三数之和

暴力方法三层循环枚举三个下标，能够覆盖全部答案，但复杂度是 $O(n^3)$ ，并且重复组合处理很麻烦。优化从排序开始。排序的意义不是让数组变好看，而是制造单调性：固定第一个数以后，剩下两个数在有序数组中寻找目标和。如果左指针和右指针的和太小，左指针右移才能增大；如果和太大，

右指针左移才能减小。这样一层固定加一层双指针就能覆盖所有二元组合。

去重是这题的核心边界。固定第一个数时，如果它和前一个固定值相同，就跳过，否则会生成重复三元组。找到一个三元组后，左右指针也要分别跳过相同值，避免同一组值被多个下标组合重复加入。注意，去重不是随便用集合过滤结果；集合过滤可以让答案看起来对，但会掩盖推导过程，也可能带来额外排序和哈希成本。

正确性可以从固定值证明。排序后，对于每个固定位置 i ，双指针会在区间 (i, n) 中按单调规则排除不可能的配对。若当前和小于目标，所有使用当前左指针和更小右指针的组合只会更小，所以左指针可以安全右移；若当前和大于目标，所有使用当前右指针和更大左指针的组合只会更大，所以右指针可以安全左移。实验要覆盖全零、大量重复、无解、正负混合、答案很多以及目标和可能溢出的四数扩展。

LeetCode 42 接雨水

这题最容易被误讲成“背双指针”。先从暴力出发：对每个位置 i ，它能接的水量由左侧最高柱和右侧最高柱的较小者决定，再减去当前高度。暴力每个位置都向两边扫描，复杂度 $O(n^2)$ 。第一层优化是预处理左右最高数组，令 $left_max[i]$ 表示 i 左侧含自身最高， $right_max[i]$ 表示右侧含自身最高。这样每个位置的贡献能常数时间计算。

双指针进一步把左右最高数组压缩成两个变量。左指针和右指针从两端向中间走，同时维护 $left_max$ 和 $right_max$ 。如果 $left_max \leq right_max$ ，左侧当前位置的水位已经由 $left_max$ 决定，因为右侧至少存在一个不低于 $left_max$ 的边界；此时可以结算左指针并右移。反之结算右指针。这个证明比代码更重要：结算的一侧必须已经知道较小边界，否则会提前计算。

单调栈解法则从另一角度结算凹槽。栈中保存递减高度下标，遇到更高的右边界时，弹出槽底，用当前柱作为右边界、新栈顶作为左边界计算水量。三种解法都正确，但状态不同：前后缀数组保存每个位置的边界，双指针保存当前两侧边界，单调栈保存尚未找到右边界的柱子。实验时分别打印三个版本的状态，对比单调递增、单调递减、平台高度、多个凹槽和长度不足三的情况。

LeetCode 76 最小覆盖子串

暴力方法枚举所有子串，再检查它是否覆盖目标字符串需求。它的问题有两层：枚举窗口多，检查窗口也重复统计。优化要抓住覆盖关系的单调性。右端扩展时，窗口字符只会增加，覆盖条件更容易满足；一旦满足覆盖，继续扩展只会让窗口更长，不可能改善当前左端下的答案，所以应该尽量收缩左端，直到刚好失去覆盖。

实现中需要两个计数结构：目标需求 $need$ 和窗口计数 $window$ 。为了避免每次比较全部字符，可以维护 $formed$ ，表示有多少种字符已经达到需求次数。注意这里是“达到需求次数的字符种类”，不是窗口里匹配的字符总数。目标串包含重复字符时尤其容易错，例如 $AABC$ 需要两个 A ，窗口里一个 A 不算满足。

正确性来自最短窗口的收缩。每当右端使窗口满足覆盖，内层循环会尽可能移动左端，并在每个合法窗口上更新答案。对于某个固定右端，循环结束前已经检查了所有能覆盖目标的左端中最短的那个；对于所有右端扫描完成后，全局最短自然被覆盖。实验要覆盖目标字符缺失、目标含重复、最短窗口在开头、最短窗口在结尾、大小写混合和源串短于目标串。C++ 中若字符集固定，可用数组；若字符集不确定，用哈希表但要注意 `operator[]` 的默认插入。

LeetCode 84 柱状图中最大的矩形

暴力方法枚举每根柱子作为矩形高度，再向左右扩展直到遇到更矮柱子。这个做法清楚地暴露了瓶颈：每根柱子都在寻找左右第一个更矮的位置。单调栈优化就是一次扫描求出“某根柱子作为最矮高度时的最大宽度”。栈中保存高度递增的下标；当遇到一个更矮柱子时，栈顶柱子的右边界已经确定为当前位置，左边界是弹出后新的栈顶。

宽度计算是这题最容易错的地方。弹出下标 mid 后，如果栈为空，说明它左侧没有更矮柱子，宽度是 $right$ ；如果栈不空，左边界是 $stack.top()$ ，宽度是 $right - stack.top() - 1$ 。为了统一结算，可以在末尾补一个高度为零的哨兵，让所有柱子最终都被弹出。相等高度如何处理也要固定：常见写法遇到小于当前栈顶才弹，或者遇到小于等于时弹，只要宽度语义和去重一致即可。

正确性来自结算时机。某根柱子被弹出，说明当前柱子是它右侧第一个更矮位置；弹出后新的栈顶是它左侧第一个更矮位置。两个边界之间所有柱子高度都不低于它，所以以它为最矮柱的最大矩形已经确定，未来再向右也会被当前更矮柱阻断。实验要覆盖递增、递减、全相等、单柱、空数

组、哨兵和宽度计算。这个题也是 LeetCode 85 最大矩形的基础：二维矩阵每一行都能转成一组柱状图高度。

LeetCode 146 LRU 缓存

LRU 不是“哈希表加链表”六个字。题目要求 `get` 和 `put` 都是常数时间，并且容量满时淘汰最久未使用项。先看暴力：用数组或向量保存访问顺序，每次访问都线性查找并移动到头部；定位和移动都会慢。哈希表能按 `key` 常数定位，但无法维护新旧顺序；双向链表能常数删除已知节点并移动到头部，但无法按 `key` 查找。两个结构职责互补，组合起来才满足要求。

高质量实现要先定义节点身份。链表节点里保存 `key` 和 `value`；哈希表从 `key` 映射到链表迭代器或节点指针。为什么节点里还要保存 `key`？因为淘汰尾部节点时，需要从哈希表删除对应 `key`。如果节点只保存 `value`，尾部删除后无法常数时间更新哈希表。访问已有 `key` 时，更新 `value` 后要把节点移动到头部；插入新 `key` 时，容量超过上限就删除尾部。

正确性来自两个不变量：哈希表中的每个 `key` 都指向链表中的一个节点，链表从头到尾按最近使用到最久未使用排序。每次 `get` 或成功更新都会把节点移到头部，所以尾部始终是最久未使用项。实验要覆盖容量为一、重复 `put`、访问不存在 `key`、更新已有值、连续淘汰和插入后立即访问。对照 Linux 源码时，可以进一步理解侵入式链表：业务对象自己带链表节点，哈希索引只保存对象位置，链表不拥有对象。

LeetCode 200 岛屿数量

这题的第一步是建模：网格中的陆地格子是节点，四方向相邻陆地之间有边，岛屿就是连通分量。暴力错误做法常常从每个陆地都重新搜索，或者不标记访问导致重复计数。正确方法是扫描每个格子，遇到一个未访问陆地，就启动 BFS 或 DFS，把整个连通块标记为已访问，并把岛屿数量加一。

BFS 和 DFS 在这题里都可以。BFS 用队列保存待扩展格子，入队时标记访问，避免同一格子被多个邻居重复加入；DFS 可以递归，也可以用显式栈。工程上大网格可能导致递归栈风险，因此显式栈更稳。若允许修改输入，可以把访问过的 `1` 改成 `0`；若不允许修改，就使用单独访问数组。这个选择不是风格问题，而是题目副作用要求。

正确性来自连通块定义。一次搜索从某个未访问陆地出发，会沿合法边访问所有与它连通的陆地；搜索不会越过水或边界，所以不会把两个不同岛屿合并。外层扫描保证每个连通块至少会被第一次遇到的陆地启动一次，而访问标记保证同一连通块不会重复计数。实验覆盖全水、全陆、单个陆地、斜角相邻但不连通、细长岛、输入是否可修改以及大网格递归深度。

LeetCode 207 课程表

课程表题不是普通遍历，而是有向依赖图判环。每门课是节点，先修关系是有向边。如果学习课程 `b` 后才能学 `a`，可以建边 `b -> a`，表示 `b` 完成后减少 `a` 的依赖。暴力思路是反复找没有先修要求的课程并删除它；这正是拓扑排序的朴素模型。优化做法用入度数组保存每门课尚未满足的依赖数量，用队列保存当前入度为零的课程。

拓扑排序的过程是：初始把所有入度为零的节点入队；每弹出一门课，就把它指向的后继课程入度减一；如果某个后继入度变成零，就入队。最后如果弹出的课程数等于总课程数，说明所有依赖都能被满足；否则剩余课程处在环里或依赖环。边方向要讲清楚，否则输出顺序会反。判能否完成时，边方向反了有时也能判环，但到了课程表 II 输出顺序就会错。

正确性来自入度语义。入度为零表示当前没有未完成先修，可以安全安排；安排它只会减少后继依赖，不会破坏其他课程合法性。若最终还有节点未输出，这些节点的入度无法降到零，说明它们之间存在循环依赖。实验要覆盖无依赖、单链依赖、简单环、多个连通分量、自环、重复边和边方向检查。DFS 三色标记也能做，但需要区分“当前路径中”和“已经完成”的节点。

LeetCode 300 最长递增子序列

先写二次 DP。令 `dp[i]` 表示必须以 `nums[i]` 结尾的最长递增子序列长度。对每个 `i`，枚举所有 `j < i`，若 `nums[j] < nums[i]`，就可以从 `j` 转移到 `i`。这个状态非常清楚，复杂度是 $O(n^2)$ 。它的问题是每个位置都要扫描所有历史前驱，很多历史状态只是在比较“能否成为某个长度的更小结尾”。

二分优化维护 `tails[len]`：长度为 `len + 1` 的递增子序列，当前能达到的最小结尾值。这个数组不是某条真实子序列，而是未来扩展潜力表。扫描新数 `x` 时，用 `lower_bound` 找第一个大于等于 `x` 的位置并替换；如果位置在末尾，就把 `x` 追加，说明能形成更长序列。为什么替换安全？同

样长度下，结尾越小，未来越容易接上更大的数，不会比原结尾差。

正确性要强调两个层次。第一，`tails` 的长度始终是已经扫描前缀中可形成的某个递增子序列长度；第二，`tails` 每个位置保存该长度的最小可能结尾，因此替换不会减少当前答案，只会改善未来扩展空间。重复元素时使用 `lower_bound` 保持严格递增；若题目改成非递减，就要用 `upper_bound`。若要还原路径，单靠 `tails` 不够，需要前驱数组和位置记录。

LeetCode 322 零钱兑换

这题要求凑出金额的最少硬币数，硬币可以无限使用。暴力搜索会枚举每种硬币使用次数或递归尝试所有选择，很多剩余金额会重复出现。状态可以定义为 `dp[value]`：凑出金额 `value` 的最少硬币数。初始化 `dp[0] = 0`，其他为无穷。对每个金额，枚举硬币，如果 `value >= coin` 且 `dp[value - coin]` 可达，就尝试更新 `dp[value]`。

这里要区分“最少数量”和“组合数量”。零钱兑换 I 求最少硬币数，内外循环顺序有多种写法，只要转移语义正确即可；零钱兑换 II 求组合数，循环顺序决定是否把不同顺序算作不同方案。哨兵也要谨慎：如果无穷值设得太大，`dp[value - coin] + 1` 可能溢出。常见做法是把无穷设为 `amount + 1`，因为最坏情况下使用面额一的硬币也不会超过这个数量。

正确性来自最后一枚硬币。任意最优方案最后使用某枚 `coin`，去掉它后剩余金额是 `value - coin` 的最优方案；否则如果剩余部分不是最优，就能替换得到更少硬币。实验覆盖金额为零、无法凑出、硬币面额大于目标、重复硬币、存在面额一和不存在面额一。复杂度是 $O(\text{amount} \cdot \text{coins})$ ，这是伪多项式复杂度，依赖金额数值而不是输入字符串长度。

LeetCode 560 和为 K 的子数组

暴力枚举所有左右端点并求和，若每次重新求和是 $O(n^3)$ ，用前缀和可以降到 $O(n^2)$ 。继续优化的关键是代数关系：区间 $(i, j]$ 的和等于 `prefix[j] - prefix[i]`。当扫描到当前前缀 `prefix[j]` 时，需要寻找多少个历史前缀等于 `prefix[j] - k`。这就是哈希表保存历史前缀频次的理由。

先查询还是先更新是本题核心边界。我们要找的是当前下标之前的历史前缀，所以应该先用当前前缀查询 `prefix - k`，再把当前前缀加入表。初始表里要放 `0 -> 1`，表示从数组开头到当前下标的区间也可能满足条件。若漏掉这个初始值，输入 `[k]` 会错误返回零。

这题能处理负数，是它和正数滑动窗口的关键区别。滑动窗口依赖右端扩展和左端收缩带来的和单调变化；一旦有负数，窗口和可能来回变化，不能安全排除左端。前缀和加哈希不需要单调性，只需要代数等式。实验覆盖负数、零、目标为零、重复前缀、从下标零开始的答案和答案数量很多。若题目改成最长区间，哈希表应保存最早位置；若改成是否存在，保存出现与否即可。

LeetCode 743 网络延迟时间

题目给有向带权图，要求从起点到所有节点的最短路最大值。暴力枚举路径会爆炸；普通 BFS 只适用于等权边，因为它按边数层次扩展，而题目边权可能不同。边权非负时，Dijkstra 是合适选择：维护每个节点当前已知最短距离，用小顶堆每次取距离最小的候选扩展。

C++ 的 `priority_queue` 不支持 `decrease-key`，所以常用“压入新状态、弹出时跳过过期状态”的写法。每次松弛边 `u -> v`，如果 `dist[u] + w < dist[v]`，就更新 `dist[v]` 并把新状态放入堆。堆里可能还保留旧距离；弹出时如果堆顶距离不等于当前 `dist[node]`，说明它已经过期，直接跳过。这个懒处理是容器限制下的工程实现，不是算法偷懒。

正确性依赖非负边。每次弹出的未过期节点拥有当前全图最小的临时距离，由于所有边权非负，从其他未确定节点绕回来不可能得到更短路径，因此它的距离可以确定。实验要覆盖不可达节点、重边、起点到自己、零权边、多条等长路径和堆中过期状态。最终若有节点距离仍为无穷，返回 -1；否则返回最大最短距离。

LeetCode 862 和至少为 K 的最短子数组

如果数组全是正数，长度最小且和至少为 K 可以用滑动窗口；但本题允许负数，窗口和不再单调。一个负数进入窗口可能让和变小，一个负数离开窗口可能让和变大，所以单纯移动左右端无法安全排除候选。先写前缀和：需要找 `prefix[j] - prefix[i] >= k` 且 `j - i` 最短。对每个 `j`，我们希望历史前缀尽量小且下标尽量靠近。

单调队列维护候选前缀下标，队列中的前缀和严格递增。处理当前 `j` 时，若队首和当前前缀差至少 K，就可以更新答案并弹出队首，因为这个队首作为左端已经得到最短的当前右端，未来右端更远只会长度更长。然后维护队尾：如果队尾前缀和大于等于当前前缀，且当前下标更靠右，那么

队尾被当前下标支配，未来用当前下标只会得到更大的区间和和更短长度。

正确性核心是支配关系。前缀和更大且下标更早的候选不如前缀和更小且下标更晚的候选；满足条件的队首在当前右端下已经结算完毕。实验要覆盖负数破坏普通窗口、答案从开头开始、无答案、多个候选同时满足、队尾连续弹出和 K 为负或零的边界。这个题是单调队列最值得精读的一题，因为它把前缀和、最短长度和支配关系放到一起。

LeetCode 1631 最小体力消耗路径

这题的路径代价不是边权和，而是路径上最大高度差。暴力可以枚举路径，但路径数量巨大。第一种高质量建模是 Dijkstra 变体：节点是格子，边权是相邻格子高度差，路径代价是当前路径最大边权。从一个格子扩展到邻居时，新代价是 $\max(\text{current_cost}, \text{edge_cost})$ 。这个代价随着路径延长不会下降，因此仍然满足按当前最小代价优先扩展的单调性。

第二种建模是二分答案。猜一个体力上限 `limit`，只允许走高度差不超过 `limit` 的边，检查左上角是否能到右下角。上限越大，可用边越多，可达性单调，所以可以二分最小可行上限。第三种建模是并查集：把所有边按高度差排序，从小到大加入，直到起点和终点连通，此时加入的边权就是答案。

三种方法利用的是不同结构。Dijkstra 把路径代价作为动态极值扩展；二分 BFS 利用可行性单调；并查集利用从小到大开放边的连通性。面试中能比较三者，比只写一种代码更有说服力。实验覆盖单格、平坦网格、必须绕路、局部高度差很大但可绕开、多个同代价路径和大网格性能。注意不要把路径代价误写成边权和，否则就变成了另一道最短路径题。

C++ 容器底层实验

实验一：vector 的容量和地址

很多刷题代码默认 `vector` 是“无限数组”，但工程上必须理解它的容量模型。写一个实验：从空 `vector<int>` 开始连续 `push_back`，每次当 `capacity` 改变时打印旧容量、新容量和 `data()` 地址。你会看到容量不是每次加一，而是按某种增长策略跳变；地址也可能变化。这解释了为什么保存元素指针、引用或迭代器后继续插入是危险的。

第二步比较 `reserve` 和 `resize`。`reserve(1000)` 只保证容量，不改变 `size`，不能直接访问 `v[999]`；`resize(1000)` 会构造一千个元素，`size` 变为一千。很多同学为了避免扩容使用 `resize`，结果把未写入元素也当成有效数据，答案被默认零污染。高质量题解要说明自己需要的是容量还是大小。

第三步比较顺序扫描 `vector` 和 `list`。两者理论上都是线性，但 `vector` 连续内存通常有更好的缓存局部性，`list` 每个节点分散分配，指针追踪成本高。这个实验能纠正一个常见误解：链表插入删除复杂度好，不代表所有场景都快。若题目主要是顺序扫描和随机访问，`vector` 往往是更好的默认选择。

实验二：unordered_map 的查询语义

哈希表题最常见错误不是哈希函数，而是 API 语义。写三版前缀和计数：第一版用 `operator[]` 查询，第二版用 `find`，第三版用 `contains` 后再取值。观察不存在 `key` 时，`operator[]` 会插入默认值，导致表大小变化。计数累加时这很方便，但只想判断存在时可能污染状态，也会让调试输出误以为某些前缀已经真实出现过。

再做 `reserve` 实验。向 `unordered_map` 插入大量元素，记录桶数量和负载因子变化。没有 `reserve` 时，插入过程中可能多次 rehash，迭代器失效，性能也会波动。刷题中不一定必须 `reserve`，但当输入规模很大、哈希表是热路径时，主动预留容量可以降低常数成本。需要强调的是，`reserve` 不能修复错误 key 设计；如果 key 构造昂贵或容易冲突，复杂度仍然会差。

最后检查 key 设计。异位词分组若把计数直接拼成字符串而不加分隔符，`1,11` 和 `11,1` 可能混淆；二维坐标若用 `x * M + y` 编码，要确认不会溢出且 `M` 选择正确；自定义结构作为 key 时，哈希函数和相等比较必须表达同一个等价关系。哈希表只是查询工具，真正决定正确性的是 key 的语义。

实验三：priority_queue 和过期状态

`priority_queue` 只支持访问和弹出堆顶，不支持删除任意元素，也不支持 decrease-key。Dijkstra、滑动窗口中位数、任务调度和 Top K 题都必须面对这个限制。写一个 Dijkstra 小实验：每当发现更

短距离，就把新状态压入堆；弹出时如果距离和 `dist` 数组不一致，就统计一次过期弹出。这个实验能让你直观看到“堆中有旧状态”并不影响正确性，只要使用前检查。

再写滑动窗口最大值或中位数的懒删除实验。窗口滑动时，一些元素已经过期，但它们可能不在堆顶，无法立即删除。可以用哈希表记录待删除次数，等它们浮到堆顶时再真正弹出。这个思想和 RCU 的延迟释放相似但目的不同：堆懒删除是容器能力不足下的工程技巧，RCU 是并发读者安全要求。

比较器也要单独实验。C++ 默认 `priority_queue<int>` 是大顶堆；小顶堆要写 `greater<int>`。如果存的是结构体，比较器含义是“谁的优先级更低”，很多人会写反。若比较器对相等元素处理不满足严格弱序，排序和堆行为都可能不稳定。任何堆题都要先在纸上写出堆顶应该谁，再写比较器。

实验四：map、set 和动态区间

有序容器适合前驱、后继和范围边界。写一个“我的日程安排”实验：用 `map<int,int>` 保存不重叠区间，key 是起点，value 是终点。插入新区间 `[l,r)` 时，先找第一个起点不小于 `l` 的后继，再检查前驱和后继是否与新区间重叠。只要这两个邻居合法，其他区间就不可能冲突，因为已有区间按起点有序且互不重叠。

这个实验能解释为什么哈希表不适合动态区间。哈希表能查某个起点是否存在，却不能快速找到离 `l` 最近的前驱和后继。排序数组适合离线处理，但动态插入会移动大量元素。红黑树类结构牺牲一些常数，换来稳定的对数级顺序操作。Linux 的 `rbtree` 和 `Maple Tree` 也服务于类似需求，只是多了侵入式节点、并发和内存管理约束。

实验报告要记录闭区间和半开区间的差异。若使用 `[l,r)`，前一个区间终点等于新区间起点不算重叠；若使用闭区间，端点相等可能算重叠。很多区问题错误都来自没定义端点语义。写比较器时也要小心相同起点：覆盖类题常需要起点升序、终点降序；合并类题只按起点升序通常足够。

Linux 源码数据结构对照

`list_head` 和 LeetCode 146

Linux 的 `list_head` 和力扣 LRU 的双向链表有共同点：都需要已知节点常数时间删除、移动到头部和从尾部淘汰。区别在所有权。力扣常用 `std::list<Node>` 让容器拥有节点，哈希表保存迭代器；内核常把 `list_head` 嵌进业务对象，链表只保存链接关系，不负责对象分配和释放。这个侵入式设计让同一个对象可以同时挂进多个链表，例如一个页面对象可以进入 LRU 链表，也可以进入其他状态链表。

源码阅读时不要先追每个宏，而要先画对象布局：业务对象里有哪些 `list_head` 字段，每个字段对应哪条链，插入和删除由谁保护，删除后对象是否还能被其他索引找到。刷题中的 LRU 删除尾部只要维护哈希表和链表；内核里删除一个对象可能还要处理引用计数、锁、RCU 或延迟释放。相同点是“节点身份必须稳定”，不同点是“生命周期复杂得多”。

实验可以写两个版本的 LRU。第一版使用 `std::list` 和 `unordered_map`；第二版让 `Entry` 自己带 `prev`、`next` 指针，哈希表保存 `Entry*`。比较两版的删除顺序：先从链表摘除，再从哈希表删除，最后释放对象。如果先释放对象，链表或哈希表就会留下悬空指针。这个实验能把链表题、缓存题和内核侵入式结构连起来。

`hlist` 和哈希冲突

哈希表的桶内冲突可以用链表处理。Linux 的 `hlist` 面向大量桶头做了空间优化：桶头较轻，节点保存足够信息以支持已知节点删除。刷题时 `unordered_map` 把桶数组、负载因子和冲突链隐藏起来，但理解桶结构能帮助你解释为什么哈希表平均常数不是无条件保证。

对照题包括两数之和、字母异位词分组、和为 K 的子数组、账户合并。每道题都不是“用了哈希就完了”，而是要回答 key 是什么、冲突时如何确认相等、value 保存次数还是下标、什么时候插入、什么时候删除。内核里这些问题更具体：对象节点是否已在桶中，删除时是否有读者正在遍历，key 对应的对象生命周期由谁保证。

实验可以写一个固定 16 个桶的小哈希表。第一版桶内使用普通单链表，删除已知节点时必须从桶头找前驱；第二版让节点保存前驱链接位置，删除时直接改指针。这个实验能解释 `hlist` 的设计动机：不是为了让算法更神秘，而是为了在桶很多、删除频繁的场景下减少空间和扫描成本。

rbtree 和动态有序集合

Linux rbtree 与 C++ `map` 都是动态有序结构，但接口风格不同。C++ 容器把节点、比较器和内存管理封装起来；Linux rbtree 通常把树节点嵌入业务对象，比较逻辑由调用者在插入和查找时写出。这样做适合内核对象按不同字段进入不同索引，也避免通用容器隐藏分配和生命周期。

对照刷题时，可以把 rbtree 看作“需要前驱后继的动态集合”的底层代表。日程安排、区间冲突、滑动窗口有序统计、数据流排名，都不是哈希表擅长的问题。红黑树的旋转不会改变中序顺序，只是在保持有序语义的前提下修复平衡。理解这一点后，再看 `lower_bound`、前驱、后继和区间插入，会更容易说清为什么只检查相邻区间。

源码阅读建议选一条简单路径：从查找插入位置开始，看调用者如何比较 key；再看链接新节点；最后看插入修复如何通过旋转和变色维持平衡。不要第一天就背红黑树五条性质的所有修复分支。刷题阶段也一样：先证明有序性和相邻关系，再谈容器实现。

XArray、Trie 和稀疏整数索引

XArray 面向整数索引到指针的映射，适合稀疏 ID 空间和内核对象管理。它和 Trie 有相似思想：把 key 分段，沿层级向下查找，只为实际存在的路径分配节点。与普通哈希相比，它更重视按索引组织、范围推进、标记和并发语义；与巨大数组相比，它避免为空洞分配大量空间。

刷题中可以用位 Trie、前缀树、坐标压缩来建立直觉。最大异或题把整数按二进制位分层；单词搜索 II 把字符串按字符分层；坐标压缩把稀疏大值域映射到紧凑下标。XArray 的工程环境更复杂，但核心问题仍是：key 空间很大，实际对象稀疏，等值查找之外还可能需要按序扫描和状态标记。

实验可以写一个简化的二进制 Trie 保存整数集合，再支持查找某个整数是否存在、查找不小于某个值的下一个元素。这个实验不会复刻 XArray，但能训练分层索引的感觉。报告中要比较数组、哈希表、平衡树和 Trie：它们分别适合连续小值域、稀疏等值查找、动态有序查找、可分段 key。

Maple Tree 和动态区间

Maple Tree 服务于范围映射场景，典型直觉是虚拟地址区间管理。区间问题比单点 key 更难：查询一个地址需要找到覆盖它的区间；插入新区间需要检查前后邻居是否重叠；删除可能删除整段，也可能把原区间切成两段。力扣 56 和 57 是离线区间问题，Maple Tree 面对的是动态范围集合和频繁查询。

刷题阶段可以先用 `std::map` 写简化区间表。key 是区间起点，value 是终点和属性。点查询时找起点不大于地址的最后一个区间，再判断终点是否覆盖；插入时检查前驱和后继；删除中间一段时要拆分。这个小实验能帮助理解 Maple Tree 的问题动机，而不是把源码当成神秘结构背诵。

工程差异在于读写并发、内存分配和查询路径。一个简单 `map` 版本可以帮助学习区间不变量，却不能替代内核结构。源码阅读时要问：节点里保存哪些范围信息，读路径如何快速找到覆盖区间，更新如何保持相邻关系，旧节点什么时候释放。这些问题比记字段名更稳定。

伙伴系统、SLUB 和分配成本

算法题通常把内存分配当成免费，但 Linux 内存管理会把分配成本放到中心。伙伴系统按 2 的幂次阶管理连续页，分配时从合适阶取块，不够就拆更高阶；释放时检查伙伴是否空闲，能合并就合并。它强调连续性和对齐，不只是总空闲量。一个系统可能总空闲页很多，却没有足够大的连续块。

SLUB 解决小对象频繁分配问题。同类型对象从缓存中分配，避免每次都走通用页分配路径，也提升局部性。回到刷题，链表节点、哈希节点、树节点都可能带来大量小对象分配；这就是为什么连续数组有时远快于节点容器。复杂度分析给出数量级，分配器和缓存局部性决定常数。

实验可以写简化伙伴系统：管理 2^{10} 个页框，维护 0 到 10 阶空闲链表。分配某阶时逐级拆分，释放时按伙伴编号尝试合并。再写一个对象池，为固定大小节点重复分配和释放，比较它与频繁 `new` 的差异。这个实验能把位运算、链表、区间合并和工程性能放在同一张图里。

错误用例库

数组和窗口

数组题的错误用例要专门打破区间语义。删除重复项要测空数组、全重复、无重复和只保留两次的变体；颜色分类要测全二、全零、逆序和交换后二次检查；接雨水要测单调递增、单调递减和平台高度。窗口题则要测试左端不能回退的情况，例如无重复子串的 `abba`；还要测试目标字符重复，例如最小覆盖子串中目标含两个相同字符。

设计反例时，不要只看最终答案，要让错误状态暴露出来。滑动窗口题可以打印左端、右端、计数和答案；双指针题可以打印每次为什么移动某一侧。一个好的反例能说明错误发生在初始化、循环条件、状态更新还是答案读取，而不是只告诉你“答案错了”。这也是把题解变成教材的关键。

哈希和前缀和

哈希题的错误用例集中在查询和更新顺序、key 设计、保存次数还是下标。两数之和要测补数等于当前值；和为 K 的子数组要测从下标零开始的答案和目标为零；连续子数组和要测长度至少为二；连续数组要测重复前缀且要求最长。异位词分组要测空字符串、重复单词和计数 key 歧义。

前缀和题尤其适合对拍。写一个平方暴力版，再随机生成包含负数、零和重复值的数组，与哈希版比较。很多窗口解法在正数数组上正确，一加负数就失败；对拍能强迫你说明算法依赖的是单调性还是代数等式。若目标从“计数”改成“最长”，哈希表 value 也要从频次改成最早下标，这类变化必须进反例库。

二分和答案空间

二分错误不是只差一，而是区间语义没有固定。测试要覆盖空数组、长度一、长度二、目标小于所有元素、目标大于所有元素、目标重复出现、答案在边界。答案二分还要测试刚好可行和刚好不可行的相邻值。每个测试都打印 `left`、`mid`、`right` 和谓词值，检查被排除的一半是否真的不含答案。

对于船运包裹、爱吃香蕉、分割数组这类题，检查函数本身就是贪心题。反例要针对检查函数，而不是只针对外层二分。比如容量小于最大包裹必须不可行；天数为一时容量必须至少是总和；天数等于包裹数时容量下界就是最大包裹。若检查函数没有这些边界，二分循环再标准也会返回错误答案。

栈、堆和单调结构

单调栈要测试严格递增、严格递减、相等元素和哨兵。柱状图最大矩形中，递增数组能暴露末尾未结算问题；递减数组能暴露宽度计算问题；全相等能暴露相等元素处理策略。每日温度要测试没有更高温、相同温度不算更高、最后一个元素答案为零。

堆题要测试过期状态和比较器方向。Top K 高频要测试频次相同和 K 等于不同元素数；数据流中位数要测试偶数个元素平均溢出；Dijkstra 要测试重边和旧堆状态；滑动窗口中位数要测试要删除的元素不在堆顶。每个堆实验都应该打印堆顶是否真实可用，而不是默认堆顶一定有效。

树、图和并查集

树题要测试空树、单节点、链式树、完全树、重复值、全负值和极端深度。验证 BST 的反例必须包含“局部父子合法但全局不合法”，例如某个右子树深处节点小于根。最大路径和要测试全负数，答案不能初始化为零。构造树题要测试节点值唯一性，如果值不唯一，单靠值到中序下标的哈希表就不成立。

图题要测试自环、重边、不连通、多个源点、状态资源不同但位置相同。打开转盘锁要测试起点死亡和目标就是起点；障碍消除最短路要测试同一格子剩余消除次数不同；课程表要测试自环和两个节点互相依赖。并查集要测试重复合并、已经连通的边、字符串编号和合并失败代表环，而不是路径本身。

动态规划

DP 反例要打状态定义。最大子数组和要测试全负数；不同路径要测试一行一列；障碍物版本要测试起点或终点障碍；编辑距离要测试空串；背包要测试容量为零、总和奇偶、一个物品被重复使用的循环方向错误；零钱兑换 II 要测试组合和排列的差异。空间压缩题还要测试更新顺序，例如 0-1 背包正序会错误重复使用当前物品。

打印 DP 表是最有效的实验。编辑距离表能看到插入、删除、替换；LCS 表能看到不连续子序列和子串的区别；股票状态机能看到持有、卖出、休息之间的约束。若一个状态无法解释表中某个格子的来源，就说明状态定义还不够稳定。不要急着压缩空间，先让完整状态表正确。

复盘报告模板

每道题复盘写一页即可，但必须有固定信息。第一行写题号和题名，例如 LeetCode 560 和为 K 的子数组。第二行写题面模型：对象、关系、输出。第三行写暴力基线：如何保证不漏候选，复杂度是多少。第四行写瓶颈：哪一步重复扫描、重复计算或无法快速查询。第五行写优化结构：保存什么，

查询什么，更新什么，删除什么。第六行写正确性：覆盖和排除各一句。第七行写边界用例：至少三个能打破错误实现的输入。第八行写 C++ 注意点：容器、溢出、迭代器、所有权或副作用。

这份报告不需要文学化，越具体越好。比如“用哈希优化”不是合格描述，“哈希表保存当前下标之前每个前缀和出现次数，扫描到前缀 s 时先查询 $s-k$ 再累加当前前缀”才是合格描述。比如“注意边界”也不是合格描述，“初始放入 $0 \rightarrow 1$ ，否则从下标零开始的区间会漏掉”才是合格描述。

最后要有实验记录。把暴力版和优化版同时保留，用随机小数据对拍；把边界用例写成单元测试；把大规模输入用于观察复杂度增长。对拍不是浪费时间，它能把“我觉得对”变成“我知道错在哪里”。当你能用一页报告解释一道题，再用相似报告解释变体，刷题才真正转化为面试能力和工程能力。

高阶经典题补充推导

LeetCode 4 寻找两个正序数组的中位数

这题经常被当成二分难题背答案，其实先从暴力看更清楚。两个数组已经有序，最直接的方法是归并到一半位置，复杂度 $O(m+n)$ 或 $O((m+n)/2)$ 。它没有利用“只需要中位数而不是完整合并结果”这个条件。中位数的本质是把全部元素切成左右两半，左半元素数量等于右半或多一个，并且左半最大值不大于右半最小值。

优化做法是在较短数组上二分切分位置。假设第一个数组左边取 i 个元素，第二个数组左边就必须取 $half - i$ 个元素。这样数量条件自动满足，只剩大小条件：第一个数组左侧最大不大于第二个数组右侧最小，第二个数组左侧最大不大于第一个数组右侧最小。如果第一个数组左侧太大，说明 i 取多了，要向左；如果第二个数组左侧太大，说明 i 取少了，要向右。

边界处理依赖哨兵。切分在数组开头时，左侧最大可视作负无穷；切分在数组末尾时，右侧最小可视作正无穷。这样不需要为每个边界写特殊分支。正确性来自切分条件：数量正确且左右有序时，中位数只可能由左侧最大和右侧最小决定。实验要覆盖一个数组为空、两个数组长度差很大、总长度奇偶、重复元素、所有元素都在一个数组左侧或右侧。讲这题时重点不是背代码，而是说明二分的对象是切分位置，不是某个具体值。

LeetCode 10 正则表达式匹配

这题的困难在于星号。暴力递归从字符串下标和模式下标出发，遇到普通字符或点号就匹配一个字符，遇到星号就尝试匹配零次、一次、两次甚至更多次。暴力会在同一对下标上反复尝试，复杂度爆炸。DP 状态可以定义为 $dp[i][j]$ ：字符串前 i 个字符能否匹配模式前 j 个字符。这个定义把递归中的下标对固定下来，避免重复搜索。

转移要围绕模式最后一个字符。如果 $p[j-1]$ 不是星号，那么只有当前字符能匹配时，才能从 $dp[i-1][j-1]$ 转移。若 $p[j-1]$ 是星号，它修饰的是 $p[j-2]$ 。星号匹配零次时，看 $dp[i][j-2]$ ；星号匹配一次或多次时，必须当前字符能被 $p[j-2]$ 匹配，然后看 $dp[i-1][j]$ ，表示消耗一个字符串字符后仍由这个星号继续处理。

初始化是最容易漏的地方。空字符串可以被 a^* 、 a^*b^* 这类模式匹配，所以第一行需要按星号零次匹配向前传播。正确性来自模式末尾分类：任意合法匹配的最后一步要么消耗一个普通模式字符，要么让星号匹配零次，要么让星号至少匹配一次，这三类覆盖全部可能。实验覆盖空串、只含星号组合的模式、点号、星号连续修饰的结构、完全不匹配和长重复字符串。

LeetCode 25 K 个一组翻转链表

这题不是单纯反转链表，而是“先确定边界，再局部反转，再接回”。暴力直觉可以把链表节点放进数组，每 K 个一组反转数组片段，再重连节点。但原地链表解法要训练的是指针边界。每一轮从上一组尾部出发，向后找第 K 个节点；如果不足 K 个，剩余节点保持原样。找到组尾后，先保存下一组头，否则反转过程中会丢失后续链表。

指针关系可以固定成四个名字： $group_prev$ 是上一组尾部， $group_head$ 是当前组头， $group_tail$ 是当前组尾， $next_group$ 是下一组头。反转当前组后，原来的 $group_tail$ 成为新头，原来的 $group_head$ 成为新尾。接回时， $group_prev \rightarrow next$ 指向新头，新尾的 $next$ 指向 $next_group$ ，然后 $group_prev$ 移到新尾。

正确性来自链表结构不变量：已经处理的前缀是按 K 分组翻转后的合法链；当前组在反转前已经确认长度足够；未处理后缀仍然由 $next_group$ 保存且可达。实验要覆盖空链表、长度小于 K 、

长度等于 K、长度不是 K 倍数、K 为一、连续多组和删除头部附近边界。C++ 中不要随意释放输入节点，除非题目明确要求；这里重连节点即可。

LeetCode 32 最长有效括号

暴力方法枚举每个子串并检查括号是否合法，复杂度很高。栈解法的核心是保存边界。栈里放尚未匹配的左括号下标，同时保留最后一个无法匹配的右括号位置。一个常见写法是在栈中先放 `-1` 作为哨兵：遇到左括号入栈；遇到右括号先弹出一个左括号位置，如果弹完后栈空，说明当前右括号无法匹配，把当前下标作为新边界入栈；否则当前有效长度就是 `i - stack.top()`。

为什么这个长度公式正确？栈顶保存的是当前有效后缀左侧最近的非法边界，或者尚未匹配的左括号位置。当前右括号匹配成功后，从栈顶之后到当前下标之间都是一个有效括号串，因此长度是差值。如果没有哨兵，边界会被很多特殊分支打散。DP 解法也可做，`dp[i]` 表示以 `i` 结尾的最长有效括号长度，但转移需要回看匹配左括号位置，初学阶段栈更直观。

实验要覆盖空串、全左括号、全右括号、简单嵌套、相邻有效段、非法右括号打断后重新开始。调试时打印栈内容和当前答案，特别观察 `)()()` 这类样例。正确性不是“栈能匹配括号”这么简单，而是栈顶始终保存当前有效后缀左侧的边界。

LeetCode 41 缺失的第一个正数

排序能做，哈希集合也能做，但题目要求线性时间和常数额外空间。关键观察是：长度为 `n` 的数组，答案只可能在 `1..n+1` 之间。小于一和大于 `n` 的数不会影响前 `n` 个正整数是否存在；每个值 `x` 如果在 `1..n` 内，它的理想位置是下标 `x-1`。于是可以把数组本身当作哈希表，通过交换把每个合法值放到自己的位置。

循环时，对每个下标 `i`，只要 `nums[i]` 在 `1..n` 内，并且目标位置上的值不是它，就交换 `nums[i]` 和 `nums[nums[i]-1]`。这里必须检查目标位置是否已经有相同值，否则重复数字会导致无限交换。所有能放到正确位置的数处理完后，再从左到右找第一个 `nums[i] != i+1` 的位置，答案就是 `i+1`；如果都正确，答案是 `n+1`。

正确性来自位置映射。任何在 `1..n` 内出现过的值，只要经过交换循环，就会被移动到对应位置，除非该位置已经有相同值。无关值被留在某些位置，不影响其他合法值继续归位。实验覆盖空数组、全负数、含零、重复正数、已经连续、缺一、答案为 `n+1` 和极端乱序。这个题训练的是原地哈希，不是普通排序。

LeetCode 51 N 皇后

暴力可以在棋盘每个格子上选择放或不放皇后，但这会产生巨大搜索树。第一层优化是逐行放置，因为合法解中每行必须恰好一个皇后；这样每层只需要选择列。再进一步，用三个集合判断冲突：列集合、主对角线集合和副对角线集合。对于位置 `row, col`，主对角线可用 `row-col` 标识，副对角线可用 `row+col` 标识。

回溯状态包括当前行号、已经放置的列、两类对角线和棋盘路径。进入某个选择时，把对应列和对角线加入集合；递归到下一行；返回时撤销。剪枝正确性来自皇后的攻击规则完全由同列和两类对角线覆盖。不要用“遍历已有皇后逐个检查”作为最终写法，它虽然正确，但每次检查重复扫描；集合把冲突判断降成常数时间。

实验可以统计搜索节点数。先写无剪枝版本或只检查列的版本，再加入对角线集合，比较 `n` 增大时节点数差异。边界包括 `n` 为一、`n` 为二和三无解、`n` 为四有两个解。输出棋盘时要注意路径拷贝成本：每次找到答案再构造字符串，或维护可修改棋盘并在答案处复制。这个题的重点是搜索树建模和剪枝证明。

LeetCode 72 编辑距离

编辑距离是二维 DP 的代表。暴力递归从两个字符串末尾开始，如果末尾相同，就处理两个更短前缀；如果不同，就尝试删除、插入、替换三种操作。不同路径会反复处理同一对前缀长度，于是定义 `dp[i][j]`：把 `word1` 前 `i` 个字符变成 `word2` 前 `j` 个字符的最少操作数。

初始化来自空串。把前 `i` 个字符变成空串，需要删除 `i` 次；把空串变成前 `j` 个字符，需要插入 `j` 次。转移时，如果两个末尾字符相同，`dp[i][j] = dp[i-1][j-1]`；否则三种候选分别是删除 `word1` 末尾、向 `word1` 插入 `word2` 末尾、替换末尾，对应 `dp[i-1][j] + 1`、`dp[i][j-1] + 1`、`dp[i-1][j-1] + 1`。

正确性来自最后一步分类。任意最优编辑序列的最后一步必然是三种操作之一，或者末尾字符

相同不需要操作；每种操作前的子问题都由更短前缀定义。实验要打印 DP 表，手动解释每个格子来源。若操作代价不同，转移权重要改；若只允许插入删除，则问题和最长公共子序列相关。空间压缩可以做，但初学时先写二维表，避免左上角旧值被覆盖。

LeetCode 212 单词搜索 II

如果对每个单词单独在棋盘上 DFS，会重复搜索大量相同前缀。优化来自 Trie：把所有单词的前缀合并成一棵树，网格 DFS 时同步沿 Trie 前进。如果当前路径在 Trie 中已经没有对应子节点，就可以立即剪枝；如果 Trie 节点标记了完整单词，就把它加入答案。这样搜索空间由“单词数乘以棋盘搜索”变成“棋盘路径和词典前缀共同约束”。

状态包括网格位置、当前 Trie 节点和访问标记。进入一个格子前，先看当前 Trie 节点是否有该字符子节点；没有就返回。有则进入子节点，标记格子已访问，向四周扩展，退出时恢复访问标记。找到单词后，为避免重复加入，可以把节点上的单词标记清空；进一步优化还可以在子树为空时剪掉 Trie 叶子，但要小心节点生命周期。

正确性来自前缀共享。任意答案单词对应 Trie 中从根到终止节点的一条路径，网格 DFS 会枚举所有不重复格子的相邻路径；只有当前缀存在时才继续，不会漏掉可能单词。实验覆盖重复单词、同一单词多条路径、棋盘字符频次不足、单字符单词、访问标记恢复和 Trie 剪枝。C++ 实现中，固定小写字母可以用 26 个指针或下标数组；若字符集更大，哈希子节点更通用但常数更高。

LeetCode 312 戳气球

这题最重要的转折是从“第一个戳谁”改成“最后戳谁”。如果按第一个戳的气球思考，左右邻居会随着后续操作变化，状态不稳定。若考虑某个开区间内最后被戳的气球 `mid`，那么在它最后被戳时，区间左右边界气球仍然存在，收益可以稳定写成 `nums[left] * nums[mid] * nums[right]`，左右两侧已经独立处理完。

令 `dp[left][right]` 表示开区间 `(left, right)` 内所有气球戳完的最大收益。枚举最后戳的 `mid`，转移为左区间收益加右区间收益加最后一次收益。为了处理边界，在数组两端补一。遍历顺序按区间长度从小到大，因为大区间依赖更短区间。这个状态定义看起来绕，但它解决了邻居变化的问题。

正确性来自最后一步分解。任意最优方案在区间内都有最后一个被戳的气球；在它之前，左右两侧气球的操作互不影响，因为最后时左右边界固定。枚举所有 `mid` 就覆盖全部最优方案可能的最后一步。实验覆盖一个气球、两个气球、包含一、较大值在中间、不同最后戳选择。这个题是区间 DP 的核心样本：只要正向操作让状态变化复杂，就尝试倒过来看最后一步。

LeetCode 460 LFU 缓存

LFU 比 LRU 多一个频次维度。题目要求淘汰访问频率最低的 key；若频率相同，淘汰最久未使用的 key。暴力可以每次淘汰时扫描所有 key 找最低频和最旧时间，复杂度太高。要做到常数时间，需要两个索引：key 到节点信息的哈希表，频次到双向链表的哈希表。每个频次链表内部按最近使用顺序排列。

访问一个 key 时，节点频次从 `f` 变成 `f+1`，要从旧频次链表删除，再插入新频次链表头部。如果旧频次链表为空，并且它正好是当前最小频次，就把最小频次加一。插入新 key 时，频次为一，放入频次一的链表头部，最小频次重置为一。容量满时，从最小频次链表尾部淘汰最旧节点，并从 key 哈希表删除。

正确性来自两个不变量：key 表能定位每个节点；频次表中，每个链表只保存该频次节点，且从头到尾是新到旧顺序；`min_freq` 始终指向当前存在节点的最低频次。实验覆盖容量为零、容量为一、重复访问同一 key、更新已有 key、频次相同按 LRU 淘汰、旧频次链表清空。这个题非常接近工程索引维护：多个结构必须同步更新，任何一步漏掉都会产生悬空节点或错误淘汰。

Linux 专题阅读路线

第一周：链表和对象身份

第一周只读链表，不要贪多。目标是理解 `list_head` 为什么嵌进对象，`container_of` 为什么能从成员地址回到外层对象，遍历时删除为什么需要 `safe` 版本。读源码时画三张图：空链表哨兵、自身在链表中的节点、删除当前节点前保存下一个节点。然后写一个小实验：同一个对象带两个链表节点，分别挂入 LRU 链和脏链。实验报告说明删除其中一条链的节点不会自动从另一条链删除。

这一周要和力扣链表题对照。反转链表训练保存后继，K 个一组翻转训练分组边界，LRU 训练已知节点移动和尾部淘汰。内核链表把这些小技巧放到真实生命周期里：对象可能被多个模块引用，删除链表节点不等于释放对象。读懂这一点，再看内核其他数据结构就不会只停留在指针操作。

第二周：哈希链和有序树

第二周读 `hlist` 和 `rbtree`。`hlist` 关注桶头空间、冲突链和已知节点删除；`rbtree` 关注动态有序索引和调用者比较。读 `hlist` 时，写一个小哈希表，比较普通单链表删除和保存前驱链接位置的删除。读 `rbtree` 时，不要先背全部修复情况，而是先看调用者如何找到插入位置，再看节点如何链接到父节点，最后理解旋转保持中序顺序。

对应刷题题型是哈希与区间。两数之和、前缀和计数、异位词分组训练 `key` 设计；日程安排、合并区间动态版、滑动窗口有序统计训练前驱后继。你要能说出什么时候哈希表不够：只需要“小于等于某值的最大 `key`”“大于等于某值的最小 `key`”或按顺序扫描，哈希表就不是合适结构。

第三周：XArray、Maple Tree 和范围

第三周读整数索引和范围索引。`XArray` 可以先用 `Trie` 类比：把整数 `key` 分段，沿层级定位对象，适合稀疏空间。`Maple Tree` 可以先用 `std::map` 区间表类比：给地址找覆盖区间，插入区间检查邻居，删除区间可能拆分。源码细节很多，但第一遍只回答三个问题：`key` 或范围如何组织，查找路径如何走，更新后旧节点如何处理。

对应刷题题型包括 `Trie`、坐标压缩、区间合并、插入区间、删除被覆盖区间。区别在于内核还要处理并发和对象生命周期。不要把 `Maple Tree` 简化成“高级红黑树”，它面向的是范围映射和读路径效率。读完这一周，应能写出一个简化 `VMA` 表，支持点查询、插入不重叠区间、删除子区间，并解释它缺少哪些内核级能力。

第四周：RCU 和内存分配

第四周读 `RCU`、伙伴系统和 `SLUB`。`RCU` 的关键词是读侧轻量、写侧复制或替换、旧对象延迟释放。把删除拆成逻辑删除和物理释放：先从索引摘除，让新读者找不到；再等旧读者离开；最后释放。这个思想能帮助你理解为什么源码里看似“已经删掉”的对象还不能马上归还内存。

伙伴系统和 `SLUB` 则回答分配成本。伙伴系统关心连续页和阶数，`SLUB` 关心固定大小对象复用。对应刷题里的对象池、链表节点分配、哈希表节点常数、数组缓存局部性。第四周实验可以写一个读多写少配置表：读者只读全局指针，写者复制新表并发布旧表到延迟列表；再写简化伙伴分配器，观察总空闲量足够但连续块不足的情况。

专题实验清单

实验一：暴力与优化对拍框架

为每类题保留一个暴力版。数组和窗口题用小规模枚举；图题用小图全搜索或慢 `BFS`；`DP` 题用记忆化递归和表格互相校验；哈希题用平方枚举校验。对拍框架随机生成小输入，比较暴力版和优化版输出，一旦不同就打印输入、两版结果和关键状态。这个实验能极大减少“看起来对”的错觉。

对拍不是为了替代证明。它的作用是暴露边界，证明的作用是解释为什么所有边界都被覆盖。报告里要写：随机输入如何生成，哪些约束被覆盖，暴力版为什么可信，优化版错在哪里。比如和为 `K` 的子数组，要让随机数组包含负数和零；滑动窗口正数题，则要分别生成全正和含负两组，观察算法适用边界。

实验二：复杂度曲线

选三道题画复杂度曲线：两数之和的平方枚举和哈希版，柱状图最大矩形的暴力扩展和单调栈版，岛屿数量的重复搜索和访问标记版。每个版本运行多个输入规模，记录耗时中位数。不要把打印放进计时代码，不要只运行一次，不要在 `Debug` 编译下得出结论。实验结果不必精确符合数学曲线，但增长趋势应能说明优化替换了哪一步。

复杂度曲线能训练工程判断。有时 $O(n \log n)$ 的排序在小数据上比复杂的线性算法更快；有时哈希表因为 `key` 构造成本高而慢；有时链表理论删除快，但缓存局部性差导致整体慢。教材里的复杂度是第一层判断，实验让你看到常数和内存布局。

实验三：边界测试表

为每个题型建立边界表。二分题固定测试空、一、二、边界外、重复；窗口题固定测试空串、全重复、目标缺失、刚好覆盖、窗口多次收缩；图题固定测试不连通、自环、重边、多源、资源维度；DP 题固定测试空状态、全负、不可达、空间压缩顺序。每次写题先从表里选用例，不要等错了再补。

边界表的价值在于复用思维，而不是复用代码。比如“重复元素”在三数之和里是去重问题，在二分里是边界问题，在堆里是比较器 `tie-breaker` 问题，在哈希里是频次问题。表格要记录同一个输入特征在不同题型中的含义，这样复习时才能横向迁移。

实验四：源码阅读报告

每次读源码只回答一个小问题。例如读 `list_head` 时，只回答删除节点改了哪几条边；读 `rbtree` 时，只回答插入前调用者如何比较 `key`；读 `XArray` 时，只回答索引如何分层；读 `RCU` 时，只回答旧对象什么时候释放。报告必须包含文件路径、接口名、对象结构、不变量、和一个刷题类比。

不要把源码报告写成字段翻译。字段名会随版本变，设计问题更稳定。一个合格报告应写出：这个结构服务的访问模式是什么，为什么普通数组、哈希表或链表不够，插入和删除维护什么关系，并发或生命周期带来什么额外约束。把这四个问题写清楚，比背十个宏更有价值。

面试讲解脚本

从题目到方案

高质量口述可以固定成六句。第一句复述题面模型，不加算法名；第二句给暴力解，说明它如何覆盖所有候选；第三句指出瓶颈，说明哪一步重复；第四句给关键观察，说明某个状态可以复用或某些候选可以排除；第五句讲数据结构职责，说明保存、查询、更新、删除；第六句讲正确性和复杂度。这个顺序能防止直接背答案，也能让面试官看到推导过程。

以 LeetCode 560 为例：题面模型是区间和计数；暴力枚举所有左右端点；瓶颈是每个右端要找很多历史左端；关键观察是 `prefix[j] - prefix[i] == k`；数据结构是哈希表保存历史前缀和频次；正确性来自任意合法区间都对应一对前缀。这个脚本比“用前缀和加哈希表”更完整。

面对追问

追问通常改一个条件。数组从全正改成含负，滑动窗口可能失效；返回一个答案改成返回所有答案，输出规模和去重会变化；只判断存在改成计数，哈希表 `value` 要从布尔变成频次；求最短改成求最长，保存最早位置或单调队列；普通树改成 BST，可以利用中序和上下界。面对追问先问条件变了哪里，再判断原状态是否仍然足够。

不要用“这个也差不多”回答变体。三数之和和四数之和相似，但四数之和要注意外层剪枝和 `long long`；接雨水和盛水最多相似处很少，一个计算每个位置贡献，一个选择两条边；零钱兑换 I 和 II 都是背包，但一个求最少数量，一个求组合数，循环语义不同。能指出相似题的不同点，比会背更多题更重要。

讲代码质量

面试代码要能说明工程取舍。为什么用 `vector` 而不是 `list`，为什么哈希表要先查再插入，为什么堆里保存下标而不是值，为什么距离用 `long long`，为什么比较器这样写，为什么递归可能有栈风险。代码短不是唯一目标，状态清楚、边界自然、复杂度可信才是高质量。

如果写类，例如 LRU、并查集、Trie，要说明成员职责。并查集的 `parent` 保存代表关系，`size` 或 `rank` 控制合并；LRU 的哈希表负责定位，链表负责顺序；Trie 节点负责前缀转移和单词结束。类方法内部访问成员时保持一致风格，复杂逻辑拆成小函数，如 `move_to_front`、`remove_node`、`link_after`。这不仅让代码好看，也让不变量有固定位置。

最后的学习路线

第一轮：题型地图

第一轮目标不是做难题，而是建立题型地图。数组、窗口、哈希、二分、栈、堆、树、图、回溯、DP、贪心、并查集都要各做一组代表题。每题写出暴力、瓶颈、优化结构、复杂度和两个边界。第一轮可以看题解，但看完必须用自己的话写一页复盘。不会证明也没关系，但要知道证明缺口在哪里。

第二轮：证明和边界

第二轮不追求新题数量，而是补证明。二分题证明排除一半，窗口题证明左端不会漏，单调栈证明弹出时答案确定，贪心题证明局部选择不劣，DP 题证明最后一步覆盖全部可能，图题证明首次到达或最短距离。每个题型至少准备三个反例，能解释错误代码为什么会被打破。

第三轮：工程化表达

第三轮把代码改成工程上也能接受的版本。统一变量语义，处理整数溢出，避免不必要复制，确认容器 API 副作用，给复杂状态写辅助函数。对照 C++ 容器实验和 Linux 源码专题，重新审视每道题的数据结构选择。最终目标是：你不仅能 AC，还能解释为什么这个结构适合这个访问模式，为什么边界不会漏，为什么复杂度可信。

高频综合题继续精读

LeetCode 85 最大矩形

最大矩形是柱状图最大矩形的二维化。暴力枚举左上角和右下角，再检查矩形内是否全是一，复杂度会非常高；即使用二维前缀和把检查降成常数，枚举矩形仍然是四次方级别。真正的观察是把每一行当作矩形底边：如果当前格子是 1，它上方连续一的数量可以累加成柱高；如果当前格子是 0，柱高清零。于是每一行都转化成一组柱状图高度，对这组高度运行 LeetCode 84 的单调栈算法。

这个转化有两个关键点。第一，任何全一矩形都有一个底边行，当扫描到这行时，它会在柱状图中表现为一段连续高度至少为矩形高度的柱子；单调栈会在某根柱子作为最矮高度时结算出它。第二，遇到零必须把高度清零，因为以当前行为底的矩形不能穿过零。不要把每行高度重新向上扫描，那会把复杂度又拉高；高度数组应当逐行增量维护。

实验时先在纸上画一个小矩阵，逐行写出高度数组，再运行柱状图栈。边界包括全零、全一、单行、单列、零把高度截断、最大矩形不在最后一行。C++ 中矩阵字符常是 '0' 和 '1'，不要和整数零一混淆。讲复杂度时要写 $O(mn)$ ，因为每一行的柱状图算法是线性的。

LeetCode 97 交错字符串

交错字符串的暴力递归非常直观：目标串当前位置可以来自第一个字符串，也可以来自第二个字符串，只要字符相等就递归尝试。问题是同一对下标 i, j 会被不同路径反复到达。状态定义应当保留两个来源串各用了多少字符： $dp[i][j]$ 表示 s_1 前 i 个字符和 s_2 前 j 个字符，能否交错组成 s_3 前 $i+j$ 个字符。

转移来自最后一个字符。若 $s_1[i-1] == s_3[i+j-1]$ ，并且 $dp[i-1][j]$ 为真，则当前状态可达；若 $s_2[j-1] == s_3[i+j-1]$ ，并且 $dp[i][j-1]$ 为真，也可达。两个来源是或关系。开始前必须先判断长度和，否则无论 DP 表如何都不可能匹配。第一行和第一列分别表示只使用一个来源串的情况，遇到第一个不匹配后后面都不能再真。

这题最容易错在把“交错”误解成“保持两串各自相对顺序即可，但不要求用完全部字符”。题目要求刚好用完两个字符串并组成整个 s_3 。实验覆盖空串、一个来源为空、两个来源字符大量相同、两条路径都可行、长度不等、最后一个字符只能来自某一侧。空间可以压缩成一维，但压缩后要小心 $dp[j]$ 代表上一行还是当前行。

LeetCode 115 不同的子序列

这题和最长公共子序列很像，但目标不是长度，而是方案数。暴力递归从 s 中选择或不选择字符，判断能否组成 t ，搜索树会非常大。状态定义为 $dp[i][j]$ ：在 s 的前 i 个字符中，有多少个子序列等于 t 的前 j 个字符。这里 i 是候选来源长度， j 是目标匹配长度，两者角色不能交换。

转移分两种。无论当前 $s[i-1]$ 是否使用，都可以继承 $dp[i-1][j]$ ，表示不用当前字符；如果 $s[i-1] == t[j-1]$ ，还可以用它匹配目标末尾，增加 $dp[i-1][j-1]$ 种方案。初始化 $dp[i][0] = 1$ ，因为任何前缀都能用“什么都不选”组成空目标；而 $dp[0][j>0] = 0$ 。

正确性来自最后一个来源字符是否被使用的划分，两类互不重叠且覆盖全部方案。方案数可能很大，C++ 中要根据题目范围考虑 `long long` 或无符号长整型。实验覆盖目标为空、来源为空、来源比目标短、重复字符大量产生多方案、完全无匹配。空间压缩时， j 必须倒序，否则同一个来源字符会在一轮中被重复使用。

LeetCode 126 单词接龙 II

单词接龙 II 不要求最短长度，还要求输出所有最短路径。暴力 DFS 枚举所有变换路径会陷入指数爆炸，并且容易先走到较长路径。正确思路是先用 BFS 确定最短层级，再用前驱关系回溯所有最短路径。BFS 的层次性保证第一次到达某个单词时是最短距离；但本题必须保留同一层的多个前驱，否则会漏掉另一条同样短的路径。

实现时可以建立通配模式桶，例如 `hot` 生成 `*ot`、`h*t`、`ho*`，把共享模式的单词放在同一桶里。这样找邻居时不需要和所有单词逐个比较。BFS 时维护 `distance` 和 `parents`，当发现一个未访问单词，把距离设为当前距离加一并记录父亲；当发现一个已经在下一层的单词，也要追加当前父亲。不能因为它已经被本层访问就丢掉同层前驱。

BFS 完成后，从终点按 `parents` 反向回溯到起点，路径反转后加入答案。边界包括终点不在词典、起点等于终点、多个最短路径、词典中存在大量共享模式、同一层访问标记时机。报告中要说明为什么不能边 BFS 边 DFS 输出路径：那样会混合搜索层级和路径枚举，容易保留非最短路径或漏掉同层父亲。

LeetCode 139 单词拆分

单词拆分的暴力递归会枚举每个切分点：如果前缀在字典中，就递归处理后缀。重复后缀会被多次处理，例如很多不同前缀都能走到同一个剩余位置。状态可以定义为 `dp[i]`：字符串前 `i` 个字符能否被字典拼出。转移枚举最后一个单词的起点 `j`，如果 `dp[j]` 为真且 `s[j:i]` 在字典中，则 `dp[i]` 为真。

优化细节有两个。第一，字典放进哈希集合做等值查询，但 `substr` 会创建新字符串，热循环里可能增加成本；可以先写清楚版本，再用最大单词长度限制 `j` 范围，或用 Trie 避免大量切片。第二，`dp[0]` 必须为真，表示空前缀可以被拼出，这是所有第一段单词的起点。如果漏掉这个初始化，任何以开头单词开始的方案都无法转移。

正确性来自最后一个单词。任意合法拆分都有最后一段 `s[j:i]`，其前缀 `s[0:j]` 必须也可拆；枚举所有 `j` 就覆盖全部可能。实验覆盖空串、字典为空、重复单词、长字符串无法拆、多个拆法和最大单词长度剪枝。若题目改成输出所有句子，就不能只保存布尔值，需要在可达性基础上 DFS 枚举路径，并考虑输出规模可能指数级。

LeetCode 188 买卖股票 IV

股票 IV 的关键是交易次数。暴力搜索每天买、卖或不操作，会产生大量状态。状态必须包含天数、已完成交易次数和是否持有股票。常见压缩写法维护 `buy[t]` 和 `sell[t]`：`buy[t]` 表示完成至多 `t` 次交易后持有股票的最大收益，`sell[t]` 表示完成至多 `t` 次交易后不持有股票的最大收益。买入不增加完成交易次数，卖出才完成一次交易。

若 $k \geq n/2$ ，交易次数限制失去意义，问题退化为无限次交易，可以把所有上涨差价累加，或用两状态 DP。否则每天更新所有 `t`。转移时 `buy[t] = max(buy[t], sell[t-1] - price)`，`sell[t] = max(sell[t], buy[t] + price)`。更新顺序要防止同一天买卖污染状态；一种稳妥方式是使用昨天状态的副本，或者按经过验证的循环顺序更新。

边界包括空价格、只有一天、`k` 为零、`k` 很大、连续上涨、连续下跌、价格震荡。正确性来自状态穷尽：每天结束时，要么持有，要么不持有；完成交易次数从零到 `k`；所有合法操作都在买入、卖出、休息中。面试里要主动说清“交易次数”是约束维度，不是最后统计能补出来的东西。

LeetCode 295 数据流中位数

如果每次加入数字后完整排序，插入成本太高。中位数只关心较小一半的最大值和较大一半的最小值，所以用两个堆维护两半。大顶堆保存较小一半，小顶堆保存较大一半；大小差不超过一；并且大顶堆堆顶不大于小顶堆堆顶。两个不变量同时成立时，中位数可以由堆顶直接得到。

插入时可以先根据堆顶把元素放入某一边，再做重平衡；也可以统一先放大顶堆，再把大顶堆堆顶移到小顶堆，最后根据大小移回。无论写法如何，都必须同时维护大小和平序关系。若只维护大小，可能较大元素在左堆；若只维护顺序，大小差过大时堆顶不再代表中位数。

实验覆盖第一个元素、偶数个元素、负数、重复值、升序输入、降序输入、平均值溢出。求偶数中位数时两个 `int` 相加可能溢出，应先转成 `long long` 或分别除以浮点。若题目扩展为滑动窗口中位数，还要支持删除过期元素，普通堆不支持任意删除，需要懒删除表或有序多重集合。

LeetCode 329 矩阵中的最长递增路径

暴力从每个格子 DFS 寻找递增路径，会重复计算同一个后续格子的最长长度。由于路径必须严格递增，状态图是有向无环图：从低值指向高值，不可能回到原值或更低值形成环。可以定义 `memo[r][c]` 为从该格子出发的最长递增路径长度。DFS 访问邻居时，只走高度更大的格子，并取邻居结果加一的最大值。

记忆化的正确性来自子问题稳定：从某个格子出发，未来可走路径只由它的位置和矩阵值决定，与之前如何到达它无关。因此计算一次后可以复用。也可以把所有格子按值排序，从大到小或小到大做 DAG DP；本质都是利用严格递增带来的无环性。如果题目允许不下降，就可能出现相等值环，原证明失效。

实验覆盖单格、全相等、严格递增矩阵、严格递减矩阵、多个局部峰值、长蛇形路径。递归实现要注意深度，极端矩阵可能路径很长；工程上可考虑排序 DP 或显式栈。调试时打印 `memo` 表，检查每个格子是否只计算一次。

LeetCode 394 字符串解码

字符串解码是解析类问题。暴力可以遇到右括号时向前寻找匹配左括号并展开，但嵌套结构会让查找和字符串拼接复杂。更自然的模型是栈保存进入新括号前的上下文：上一层已经构造的字符串、当前重复次数。扫描字符时，数字累积成多位数；遇到左括号，把当前字符串和次数入栈，重置当前字符串；遇到右括号，弹出上一层，把当前片段重复后接回去。

这里的状态必须区分“当前层字符串”和“上一层字符串”。如果只用一个字符串，遇到嵌套时无法知道展开后接回哪里。数字也要累积，例如 `12[a]` 不是 `1` 和 `2` 两个次数。题目通常保证输入合法，但教材实现仍应知道非法输入会在哪里出错：括号不匹配、数字后没有括号、重复次数为空等。

实验覆盖多位数字、嵌套、相邻编码段、普通字符混合、空片段。C++ 中大量字符串重复拼接可能成本高，可以先写清晰版本，再讨论用 `reserve` 或输出缓冲优化。这个题和表达式求值同属“上下文栈”：遇到进入嵌套结构时保存旧上下文，退出时恢复并合并。

LeetCode 416 分割等和子集

题面问能否把数组分成和相等的两部分。先做代数转换：总和若为奇数，直接不可能；否则问题变成能否选出若干数，使其和为 `sum/2`。每个数只能使用一次，所以这是 0-1 背包可达性问题。状态 `dp[cap]` 表示是否能用已处理数字凑出容量 `cap`。

一维压缩时容量必须倒序遍历。倒序保证当前数字只会基于上一轮状态转移，最多使用一次；如果正序遍历，当前数字刚更新出的状态会在同一轮再次被使用，变成完全背包。这个循环方向不是模板差异，而是物品使用次数的语义。初始化 `dp[0] = true`，表示什么都不选可以凑出零。

实验要专门构造循环方向反例，例如只有一个数却能在正序中被重复使用凑出更大容量。边界包括总和为奇数、目标为零、数组中有零、数值很大导致容量表很大。复杂度是伪多项式 $O(n \cdot target)$ ，如果数值范围很大，即使元素个数不多也可能慢。

LeetCode 494 目标和

每个数前面放正号或负号，暴力枚举所有符号是指数级。设正号集合和为 `P`，负号集合和为 `N`，总和为 `S`，则有 `P - N = target`，且 `P + N = S`，推出 `P = (S + target) / 2`。于是问题变成从数组中选若干数凑出 `P` 的方案数。这一步转换要先检查 `S + target` 是否非负且为偶数，否则无解。

转成背包后，状态 `dp[cap]` 表示凑出容量 `cap` 的方案数。每个数只能选择一次，所以容量倒序遍历。零元素会自然让方案数翻倍：当 `num = 0` 时，`dp[cap] += dp[cap]`，表示给零加正号或负号都不改变和，但属于不同符号方案。很多手写特殊分支反而容易错，标准计数转移可以自然处理。

正确性来自代数一一对应。任意符号方案对应一个正号集合，正号集合和必须是 `P`；任意选出和为 `P` 的集合，也能给集合内元素正号、集合外元素负号得到目标。实验覆盖目标绝对值大于总和、奇偶不匹配、含多个零、全零、空数组语义。它和分割等和子集很像，但一个求可达性，一个求方案数。

LeetCode 518 零钱兑换 II

零钱兑换 II 问组合数，硬币可以无限使用。如果外层遍历金额、内层遍历硬币，会把不同顺序算作不同方案；例如一加二和二加一会被重复统计。组合数要求每组硬币按固定顺序被考虑，因此外层应遍历硬币，内层正序遍历金额。状态 `dp[value]` 表示使用已经处理过的硬币种类，凑出金额 `value` 的组合数。

初始化 `dp[0] = 1`，表示凑出金额零有一种方案：什么都不选。处理某个硬币 `coin` 时，金额从 `coin` 到 `amount` 正序遍历，`dp[value] += dp[value - coin]`。正序表示当前硬币可以重复使用；外层硬币顺序固定表示组合不会因为排列不同重复计数。

实验要把循环顺序故意写反，观察组合数如何变成排列数。边界包括金额为零、硬币数组为空、硬币面额大于金额、组合数较大。和 LeetCode 322 的区别必须说清：322 求最少硬币数，518 求组合数量；一个是最值 DP，一个是计数 DP；循环语义和初始化都不同。

LeetCode 621 任务调度器

任务调度器可以用堆模拟，也可以用公式。暴力模拟每个时间片，选择当前可执行且剩余次数最多的任务，不够时插入空闲。堆模拟比较直观，但公式能揭示结构：最短时间主要由最高频任务决定。设最高频为 `maxFreq`，有 `maxCount` 种任务达到最高频。前 `maxFreq - 1` 轮需要形成骨架，每轮长度至少为 `n + 1`，最后放所有最高频任务的尾部。

公式为 `max(tasks.size(), (maxFreq - 1) * (n + 1) + maxCount)`。为什么还要和任务总数取最大？因为其他任务足够多时，它们可以填满所有冷却空位，甚至让整体没有空闲，此时总时间就是任务数。若其他任务不够，骨架长度决定必须插入多少空闲。这个证明比记公式更重要。

实验覆盖冷却为零、只有一种任务、多个最高频任务、其他任务刚好填满空隙、其他任务超过空隙。堆模拟可以作为验证公式的对拍版本。若题目扩展为每个任务有不同执行时间、释放时间或不同冷却间隔，公式不再适用，需要更一般的调度模型。

LeetCode 815 公交线路

公交线路题的建模很关键。如果把站点作为 BFS 节点，边表示同一公交线路内任意两站可达，那么一条路线内站点两两连边会非常多。更合适的模型是把“乘坐路线”作为 BFS 层级的一部分：从起点站找到所有可乘路线，乘上一条路线算一次公交；这条路线经过的所有站点都可以到达，并继续找到这些站点可换乘的其他路线。

实现通常建立站点到路线列表的哈希表。BFS 队列里可以放路线，也可以放站点加换乘次数；无论如何，都要分别标记访问过的路线和站点，避免重复展开。同一条路线一旦处理过，就不必再次扫描它的所有站点。目标站如果在当前路线中出现，就可以返回当前乘车次数。

正确性来自 BFS 层次：每一层代表乘坐公交次数增加一，第一次到达目标站就是最少乘车次数。实验覆盖起点等于终点、目标不可达、多条路线共享站点、路线很长、站点编号很大且稀疏。容器选择上，站点编号不一定连续，站点到路线列表适合哈希表；路线访问适合布尔数组，因为路线数量连续。

LeetCode 895 最大频率栈

最大频率栈要求弹出出现频率最高的元素；频率相同，弹出最近压入的元素。暴力每次 `pop` 扫描所有元素找最高频和最近时间，太慢。需要两个维度的索引：`value` 到频次的哈希表，频次到栈的哈希表，以及当前最大频次 `maxFreq`。`push` 某个值时，先把它频次加一，再压入对应频次的栈，并更新最大频次。

`pop` 时，从 `maxFreq` 对应的栈弹出栈顶值。由于每次达到某个频次时都按时间压入该频次栈，栈顶自然是该频次下最近的元素。弹出后把该值频次减一；如果最大频次栈为空，`maxFreq` 减一。注意，不需要从较低频次栈中删除旧记录，因为那些记录代表该元素历史上达到该频次的时刻；只有当它再次成为当前最大频次栈顶时才会被使用。

正确性来自频次分层和栈内时间顺序。哈希表给出当前真实频次，频次栈保存达到该频次的时间顺序，`maxFreq` 指向最高非空层。实验覆盖频次并列、同一元素连续 `push`、`pop` 后最大频次下降、多个历史记录存在。这个题能训练“按维度拆索引”：一个结构回答频次，一个结构回答同频最近。

LeetCode 981 基于时间的键值存储

题目要求对每个 key 保存多个时间戳版本，并查询不超过给定时间戳的最新值。暴力可以把所有记录放到一个列表里，每次查询扫描；更好的模型是按 key 分组。每个 key 对应一个按时间递增的数组，元素是时间戳和值。查询时先用哈希表定位 key，再在该 key 的时间数组中二分找第一个大于目标时间的位置，答案是它前一个元素。

这个结构利用了题目常见条件：同一个 key 的 set 调用时间戳递增。如果没有这个条件，就需要插入时保持有序，可能使用有序容器或插入后二分位置。二分边界要处理目标时间小于该 key 最早时间，此时返回空字符串；目标时间大于所有记录，返回最后一个值。不要把所有 key 的记录混在一个全局有序数组里，否则查询某个 key 时还要过滤，复杂度和实现都变差。

实验覆盖 key 不存在、时间戳早于所有记录、等于某个记录、落在两个记录之间、晚于所有记录、多个 key 交错 set。C++ 中可以用 `unordered_map<string, vector<pair<int, string>>>`，二分比较 pair 的第一维，或者手写二分避免构造哨兵 pair。这个题的重点是“先按 key 分组，再在组内二分”。

LeetCode 1235 规划兼职工作

每个工作有开始时间、结束时间和收益，目标是选择互不重叠工作最大化收益。暴力枚举子集会指数爆炸。排序后可以做 DP：按结束时间排序，`dp[i]` 表示考虑前 *i* 个工作能获得的最大收益。第 *i* 个工作要么不选，收益 `dp[i-1]`；要么选，则需要找到最后一个结束时间不大于当前开始时间的工作 *p*，收益为 `dp[p] + profit[i]`。

二分用于寻找 *p*。因为工作按结束时间排序，所有结束时间不大于当前开始时间的工作构成前缀，可以用 `upper_bound` 找边界。这里的二分不是查目标值是否存在，而是找兼容工作的数量。若端点语义是前一个工作结束时间等于当前开始时间可兼容，比较条件应使用不大于。

正确性来自最后一个决策：最优解对当前工作只有选和不选两类，选当前工作后，之前能选择的工作只来自兼容前缀，且这部分最优值已经在 DP 中。实验覆盖工作完全不重叠、完全重叠、端点相接、相同结束时间、收益很高但阻塞多个小工作。不要用按收益贪心，局部最高收益可能破坏全局组合。

LeetCode 1438 绝对差不超过限制的最长连续子数组

窗口合法性由最大值和最小值决定：窗口内最大值减最小值不超过限制。暴力每个窗口重新找最大最小会很慢；平衡树可以维护动态有序集合，复杂度 $O(n \log n)$ ；单调队列可以做到线性。维护两个队列：一个递减队列保存最大值候选，一个递增队列保存最小值候选，队列里存下标以便判断过期。

右端加入新元素时，最大队列弹出所有小于新值的尾部，因为它们既更小又更早，不可能再成为最大值；最小队列弹出所有大于新值的尾部。然后检查队首差值是否超过限制；若超过，就移动左端，并把过期下标从队首弹出。注意，左端可能需要连续移动，直到窗口重新合法。

正确性来自支配关系和窗口单调收缩。队首始终是当前窗口最大和最小；被队尾弹出的旧候选被新候选支配，不会在未来窗口中重新有用。实验覆盖限制为零、全相等、严格递增、严格递减、需要多次收缩、最大最小交替变化。这个题说明滑动窗口状态不一定是和或频次，也可以是动态极值。

LeetCode 1514 概率最大的路径

这题是最短路思想的变体。路径概率是边概率乘积，目标是最大乘积。暴力枚举路径不可行；因为边概率在零到一之间，沿路径继续乘只会不增，所以可以用 Dijkstra 类似思想：每次扩展当前概率最大的节点。用大顶堆保存当前到达某节点的最大概率候选，若弹出的概率不是当前记录值，就跳过过期状态。

松弛边时，候选概率是 `prob[u] * edgeProb`。如果它大于 `prob[v]`，就更新并压入堆。也可以把概率取负对数，把乘积最大转成权重和最小；但直接最大堆更贴近题意。正确性来自单调性：当前堆顶概率最高，未来从其他更低概率路径继续乘边，不可能变成更高概率回头改写它。

实验覆盖起点等于终点、不可达、概率为零、多个同概率路径、浮点比较。C++ 中要使用 `double`，并避免用整数处理概率。这个题适合训练“框架迁移”：不是所有 Dijkstra 都叫最短距离，只要代价扩展满足合适单调性，优先队列扩展思想就可能成立。

LeetCode 1675 数组的最小偏移量

每个数可以做有限变化：奇数可以乘二一次变成偶数，偶数可以反复除二。直接搜索所有状态组合会爆炸。关键是先把每个数规范化到它能达到的最大偶数形态：奇数乘二，偶数保持。之后每个数只允许不断减小。维护当前所有数中的最大值和最小值，每次尝试降低当前最大偶数，因为只有降低最大值才可能缩小偏移。

用大顶堆保存当前值，同时维护当前最小值。每次堆顶是最大值，先用最大值减最小值更新答案；如果最大值是偶数，可以除二后放回堆，并更新最小值；如果最大值是奇数，它已经不能再降低，过程结束。为什么只处理最大值？偏移由最大和最小决定，降低非最大值不会降低当前最大，反而可能降低最小使偏移变大。

正确性来自状态空间方向。规范化后，每个数的可达序列都是单调下降的；从当前最大值开始下降，枚举了所有可能改善最大边界的步骤。当最大值无法继续下降时，当前及未来都无法缩小最大边界。实验覆盖全奇数、全偶数、已经偏移为零、最大值多次下降、下降后成为新的最小值。注意值乘二可能溢出，要根据输入范围使用足够整数类型。

跨题型迁移和混淆点

盛水最多和接雨水不能混用

LeetCode 11 盛最多水的容器和 LeetCode 42 接雨水都出现左右指针和高度数组，但问题模型完全不同。盛水最多只选择两条边，面积由短板和宽度决定；每次移动短板，是因为移动长板无法提高短板且宽度必然变小。接雨水则计算每个位置的水量，位置贡献由左右最高边界的较小值决定。前者在候选对之间做排除，后者在每个位置上做边界结算。

如果把盛水最多的证明搬到接雨水，会漏掉中间位置的贡献；如果把接雨水的左右最高思路搬到盛水最多，也会计算不存在的“内部蓄水”。复盘时要写两张状态表：盛水最多保存的是当前左右边界和最大面积，接雨水保存的是左侧最高、右侧最高和当前位置贡献。它们的共同点只是都利用了高度边界，不能因为代码都有双指针就当成同一模板。

最小覆盖子串和找到所有异位词

LeetCode 76 最小覆盖子串是可变长度窗口，窗口满足覆盖后要尽量收缩；LeetCode 438 找到字符串中所有字母异位词是固定长度窗口，只需要维护长度等于目标串长度的频次。两者都使用字符计数，但窗口边界语义不同。最小覆盖子串的左端移动由“覆盖条件仍然满足”驱动；异位词窗口的左端移动由“窗口长度固定”驱动。

混用这两题会出现典型错误。把固定窗口写成可变窗口，可能在频次超过需求时过度收缩，漏掉长度刚好的异位词；把可变窗口写成固定窗口，则无法找到更短覆盖。实验时分别打印窗口长度、字符差异和答案更新点。讲题时要先说输出目标：一个要求最短子串，一个要求所有起点；输出目标不同，状态读取位置也不同。

普通 BFS、Dijkstra 和 0-1 BFS

看到“最短”不能直接写 BFS。普通 BFS 成立的条件是每条边代价相同，队列层数就是路径长度。Dijkstra 成立的条件是边权非负，每次扩展当前代价最小的未确定状态。0-1 BFS 则针对边权只有零和一的图，用双端队列把零权边放队首、一权边放队尾，从而保持距离顺序。三者都是按某种代价顺序扩展，但容器和证明不同。

网络延迟时间有不同正权边，普通 BFS 会按边数而不是时间扩展，可能错；打开转盘锁每次拨动代价相同，普通 BFS 正合适；带零一代价的状态图若用普通队列，会破坏距离顺序，若用普通 Dijkstra 又能做但常数更高。复盘图题时先写边权范围，再选队列、双端队列或堆。这个判断比背算法名字重要。

并查集和 DFS 连通块

岛屿数量、朋友圈、省份数量、冗余连接都和连通性有关，但并查集和 DFS 的职责不同。DFS/BFS 适合从一个起点找完整连通块，能顺便统计面积、边界、路径等信息。并查集适合动态合并和快速回答两个元素是否同组，但它不保存具体路径，也不擅长枚举某个连通块的所有节点，除非额外维护集合成员。

冗余连接按边顺序加入，发现两个端点已经连通时，这条边形成环，因此并查集非常合适。岛屿最大面积需要统计每个连通块大小，BFS/DFS 更直接；当然也可以用并查集维护 size，但实现成

本更高。选择结构时先问查询是什么：要遍历块，还是只问同组；要动态加边，还是静态扫描；要路径，还是只要集合代表。

背包三题的状态区别

分割等和子集、目标和、零钱兑换 II 都能归到背包，但状态含义不同。分割等和是可达性，`dp[cap]` 是布尔值；目标和是方案数，`dp[cap]` 是计数；零钱兑换 II 是完全背包组合计数，硬币可无限使用且组合不区分顺序。循环方向和外层顺序都由这些语义决定。

若把分割等和的布尔状态搬到目标和，就只能知道是否存在，无法统计方案；若把零钱兑换 II 的正序容量搬到 0-1 背包，就会重复使用同一个物品；若把组合计数的循环顺序反过来，就会统计排列数。背包题不要背“倒序或正序”，而要先写一句：物品能用几次，答案是可达、最值还是计数，顺序是否有意义。

源码思想迁移到题解表达

对象生命周期

Linux 源码读多以后，刷题里也会更重视对象生命周期。链表节点是否由题目平台管理，返回的新链表是否复用输入节点，哈希表保存的是值、下标、迭代器还是指针，vector 扩容后旧引用是否还有效，这些都属于生命周期问题。很多面试题不会直接考内存释放，但会通过 LRU、链表反转、合并链表、复制随机链表等题考察你是否理解节点身份。

题解表达中可以主动说明副作用。例如“本题允许修改输入，所以我把访问过的陆地改成水”；“本题不应释放输入链表节点，只需要重连”；“LRU 的哈希表保存链表迭代器，链表节点必须稳定，所以不能用会移动元素的 vector 存节点”。这些话能把算法答案提升到工程答案。

索引组合

内核对象常同时进入多个索引：一个对象可能在链表中表示顺序，在树中表示有序键，在哈希表中表示快速查找。刷题中的很多设计题也是索引组合。LRU 是 key 到节点的哈希表加新旧顺序链表；LFU 是 key 到节点、频次到链表、最小频次；时间键值存储是 key 到有序时间数组；账户合并是邮箱到集合编号再按根收集。

讲设计题时，不要只列容器，要说明每个索引回答什么问题。哈希表回答“这个 key 在哪里”，链表回答“谁最旧”，堆回答“当前极值是谁”，有序表回答“前驱后继是谁”，并查集回答“两个对象是否同组”。一个容器若回答不了所有操作，就需要组合；组合后每次更新都要维护所有索引的一致性。

逻辑删除和物理删除

RCU、堆懒删除、滑动窗口过期元素、哈希表延迟清理都体现了一个思想：逻辑上不可用不等于物理上马上移除。Dijkstra 堆里的旧状态逻辑上过期，但直到弹出才物理删除；滑动窗口中位数里，过期元素可能埋在堆中，需要延迟清理；RCU 中，旧对象从新索引不可达后，还要等旧读者离开才能释放。

题解里要说清使用前检查。堆顶如果可能过期，就在读取答案前清理；哈希表如果保存历史频次，就要明确什么时候递减或删除；链表节点如果从一个索引摘除，还要确认其他索引是否仍引用它。逻辑删除能降低操作成本，但必须有可靠的清理时机，否则会读到脏状态。

更多实验报告样例

样例一：二分答案

题目选择 LeetCode 1011 在 D 天内送达包裹。暴力基线是从容量下界开始逐个尝试，检查能否在 D 天内完成。瓶颈是答案空间可能很大，线性试容量太慢。关键观察是容量越大，需要天数越少，可行性单调。检查函数按顺序装包，当前天能放就放，放不下再开新天。这个贪心正确，因为提前开新天只会让剩余天数更紧，不会减少所需天数。

边界测试包括 D 为一、D 等于包裹数、单个包裹等于最大值、容量小于最大包裹、所有包裹相同。实验记录 `left`、`right`、`mid` 和需要天数，检查每次排除是否符合“最小可行容量”语义。C++ 注意总重量可能较大，用足够整数类型。最终复杂度是 $O(n \log S)$ ，S 是容量范围。

样例二：单调队列

题目选择 LeetCode 239 滑动窗口最大值。暴力基线是每个窗口扫描 K 个元素取最大，复杂度 $O(nk)$ 。优化观察是：若新元素比队尾候选更大且下标更晚，那么旧候选在未来任何同时包含二者的窗口中都不可能成为最大值；新元素既更大又更持久。单调队列保存下标，队首是当前窗口最大值。

边界测试包括 K 为一、 K 等于数组长度、严格递增、严格递减、重复最大值。调试时打印右端进入、队尾弹出、队首过期和答案读取。证明分两步：队尾弹出不会丢答案，因为旧候选被新候选支配；队首过期必须删除，因为它不再属于窗口。C++ 用 `deque<int>` 保存下标，而不是保存值，因为需要判断过期。

样例三：树形 DP

题目选择 LeetCode 337 打家劫舍 III。暴力递归对每个节点尝试偷或不偷，会重复计算子树。状态应由子树向父节点汇报两个值：偷当前节点的最大收益、不偷当前节点的最大收益。若偷当前节点，孩子不能偷；若不偷当前节点，孩子可以偷也可以不偷，取各自最大值。

边界包括空树、单节点、链式树、左右子树收益差异大。正确性来自树形后序归纳：左右子树已经给出在偷或不偷根时的最优结果，父节点只按相邻不能同偷的约束合并。工程上递归清晰，但极端深树有栈风险；可用显式栈做后序遍历并用哈希表保存每个节点状态。这个实验把 DP 状态和树遍历顺序联系起来。

样例四：区间贪心

题目选择 LeetCode 452 用最少数量的箭引爆气球。暴力可以尝试所有射箭位置组合，但选择空间巨大。贪心观察是：按右端点排序后，第一支箭放在最早结束气球的右端，不会比任何其他能射爆这个气球的位置更差，因为它给后续气球留下最大可能覆盖范围。然后跳过所有被这支箭覆盖的气球，继续处理下一个未覆盖气球。

正确性用交换论证。任意最优解中，必须有一支箭射爆最早结束的气球；把这支箭移动到该气球右端，仍能射爆原来被它覆盖且起点不大于该右端的气球，并且不会更早结束。因此存在一个最优解采用贪心第一步。边界包括端点相同、负坐标、区间包含、闭区间端点相接。比较器按右端升序，注意端点可能接近整数边界。

全书复盘索引

题型到能力

数组训练区间语义和缓存局部性；窗口训练增量维护和单调条件；哈希训练等价类和历史状态；二分训练边界谓词；栈训练最近未完成结构；堆训练动态极值；树训练状态方向；图训练建模和访问标记；DP 训练状态定义和依赖顺序；贪心训练交换证明；并查集训练动态连通；Linux 源码训练对象生命周期和索引组合。

复习时不要按题号孤立记忆，而要问每题训练哪种能力。LeetCode 560 训练前缀代数 and 哈希频次；LeetCode 862 训练前缀和上的单调队列；LeetCode 1631 训练同一问题的多模型；LeetCode 146 训练索引组合；LeetCode 312 训练区间 DP 的最后一步。把能力写在题号旁边，复习效率会明显高于只刷通过率。

源码到能力

Linux 链表训练侵入式节点和稳定身份；hlist 训练桶和删除成本；rbtree 训练动态有序索引；XArray 训练稀疏整数映射；Maple Tree 训练范围查询；RCU 训练逻辑删除和物理释放分离；伙伴系统训练连续性、阶数和合并；SLUB 训练小对象分配成本。它们不是为了替代刷题算法，而是帮助你把数据结构选择讲到工程约束层面。

最终目标是形成统一语言：访问模式决定结构，结构维护不变量，不变量支撑正确性，容器实现带来工程成本，边界实验验证理解。无论面对力扣题、C++ 面试题还是 Linux 源码阅读，都按这条线分析，就不会退回“复制模板”的学习方式。

代码审阅清单

状态是否真的必要

每写完一道题，先检查状态是否最小且足够。最小是指没有保存未来不会再用的信息；足够是指未来决策所需的信息没有缺失。两数之和只需要历史值到下标，不需要保存所有数对；和为 K 的子数组计数需要历史前缀频次，不是只保存出现与否；最长区间需要最早位置，不是最后位置。状态一旦不匹配输出目标，代码可能在简单样例上通过，却在变体或边界上失败。

审阅时可以问四个问题：状态何时进入，何时更新，何时删除，何时读取答案。窗口题中，右端进入、左端删除和答案读取顺序非常关键；堆题中，堆顶读取前可能需要清理过期元素；DP 题中，状态更新顺序决定读到旧值还是新值。只要这四个问题有一个答不上来，就不应该把题标记为掌握。

复杂度是否按真实维度说明

复杂度不能只写一个 n 。图题要区分节点数和边数，字符串 DP 要区分两个字符串长度，背包题要说明容量来自数值大小，答案二分要说明对答案范围取对数，输出所有路径的题要把输出规模算进去。比如单词接龙 II 的 BFS 只是找到最短层级，最终输出所有路径可能本身就很大；N 皇后输出数量也是复杂度的一部分。

审阅复杂度时还要看容器操作。哈希表平均常数不等于最坏常数，`substr` 可能复制字符串，`priority_queue` 插入和弹出是对数，`map` 查询前驱后继也是对数。若代码在循环里构造大量临时字符串，表面 DP 复杂度可能被隐藏成本放大。复杂度说明要和实际代码一致。

边界是否进入主逻辑

好的代码不是主体一段、后面一堆补丁分支。边界最好进入初始化和不变量。前缀和把 $0 \rightarrow 1$ 放进哈希表，就自然处理从开头开始的区间；括号栈放 -1 哨兵，就自然处理第一个有效段；链表使用虚拟头节点，就自然处理删除头节点；二分使用左闭右开，就自然处理插入到末尾。审阅时要优先寻找能消除特殊分支的状态设计。

当然，有些入口边界应当直接返回，例如空数组、起点终点被阻塞、容量为零、目标长度不匹配。这些判断不是补丁，而是题意的不可行条件。区别在于：入口判断处理的是根本无解或无输入；不变量处理的是算法过程中的普通边界。把二者混在一起，会让代码难以解释。

C++ 实现是否隐藏风险

C++ 审阅至少看五类问题。第一，整数溢出：面积、路径距离、前缀和、二分平方、概率转换都可能超出 `int`。第二，迭代器和引用失效：`vector` 扩容、`erase`、`unordered_map` rehash 都可能让保存的引用失效。第三，容器副作用：`operator[]` 会插入默认值。第四，比较器：排序和堆比较器必须满足严格弱序。第五，复制成本：循环中不要无意复制大 `vector`、`string` 或复杂节点。审阅必须具体。

设计类时还要看成员职责。并查集的 `find` 是否路径压缩，`unite` 是否按大小合并；LRU 的移动节点是否只改链表而不丢哈希；Trie 的析构和节点所有权是否清楚；图算法的邻接表是否按输入规模预分配。刷题代码不一定要写成工程库，但高质量教材必须告诉读者这些风险在哪里。

十周训练安排

第一、二周：线性结构

第一周集中数组、字符串、双指针和窗口。每天选择两道基础题和一道变体题，必须写暴力版和优化版。重点不是数量，而是把区间语义写清楚：左闭右开还是左闭右闭，慢指针左侧保存什么，窗口何时合法，答案何时更新。实验必须打印指针移动过程。推荐题包括两数之和、无重复字符最长子串、盛水最多、三数之和、最小覆盖子串、长度最小子数组、找到所有异位词。

第二周集中链表、栈、队列和单调结构。链表题每一步都画前驱、当前、后继；栈题说明栈顶代表什么；单调栈和单调队列说明支配关系。推荐题包括反转链表、K 个一组翻转链表、有效括号、最长有效括号、每日温度、柱状图最大矩形、滑动窗口最大值。周末写一个 LRU，把哈希表和双向链表的职责讲清楚。

第三、四周：查找和有序结构

第三周集中哈希、前缀和和二分。哈希题必须写 key 和 value 的语义，前缀题必须写代数等式，二分题必须写谓词和区间不变量。推荐题包括字母异位词分组、和为 K 的子数组、连续子数组和、连续数组、搜索插入位置、查找首尾位置、爱吃香蕉、船运包裹、分割数组最大值。

第四周集中排序、区间、堆和有序容器。排序题说明排序换来了什么关系，区问题说明端点闭开，堆题说明堆顶是否可能过期，有序容器题说明前驱后继。推荐题包括合并区间、插入区间、无重叠区间、用箭引爆气球、前 K 高频、数据流中位数、最接近原点 K 个点、任务调度器。周末写 map 动态区间实验，对照 Maple Tree 思维。

第五、六周：树和图

第五周集中二叉树、BST、Trie。每道树题先判断状态方向：自顶向下、后序汇总还是层序传播。递归写法要能说出迭代替代方案。推荐题包括层序遍历、验证 BST、最大深度、最近公共祖先、二叉树直径、打家劫舍 III、单词搜索 II、实现 Trie。实验打印显式栈和队列状态。

第六周集中图、拓扑、并查集和最短路。每道图题先写节点、边、边权和状态维度。推荐题包括岛屿数量、岛屿最大面积、腐烂橘子、打开转盘锁、课程表、省份数量、冗余连接、网络延迟时间、最小体力路径、概率最大路径。周末比较 BFS、Dijkstra、二分 BFS 和并查集在同一题上的建模差异。

第七、八周：动态规划

第七周集中线性 DP、二维 DP 和字符串 DP。每题先写暴力递归，再圈出重复状态。推荐题包括爬楼梯、打家劫舍、最大子数组和、不同路径、最小路径和、最长公共子序列、编辑距离、交错字符串、不同子序列。每题至少打印一个小 DP 表，解释每个格子来源。

第八周集中背包、区间 DP、状态机和状态压缩。推荐题包括零钱兑换、零钱兑换 II、分割等和子集、目标和、买卖股票含冷冻期、买卖股票 IV、戳气球、最长递增子序列。重点检查循环方向、初始化、空间压缩旧值来源和伪多项式复杂度。周末把三道背包题放在一张表里比较状态含义。

第九、十周：综合和源码

第九周做综合设计题和高频难题。推荐题包括 LFU 缓存、最大频率栈、时间键值存储、公交线路、最短子数组至少为 K、数组最小偏移量、规划兼职工作。每题都写索引组合图，说明每个结构回答什么查询，更新时如何保持一致。

第十周读源码和复盘。按链表、hlist、rbtree、XArray、Maple Tree、RCU、伙伴系统、SLUB 的顺序，每天只读一个小问题，写一页报告。最后选十道自己最容易混淆的题，重新讲一遍暴力、瓶颈、状态、不变量、边界和 C++ 风险。十周结束后，不要求记住所有题号，但要能看到题面就判断访问模式、状态维度和可疑边界。

质量底线

不能用模板替代理解

任何题解如果只有“这题用哈希”“这题用 DP”“这题用双指针”，都不合格。必须补上为什么：哈希保存什么等价类，DP 状态包含哪些未来信息，双指针排除了哪一批候选。模板只能作为复习索引，不能作为解释本身。读者应该在每道题里看到暴力版本如何演化，而不是看到一个突然出现的最终答案。

教材写作也一样。章节之间要有衔接：数组的区间语义过渡到窗口，窗口的增量维护过渡到单调队列，前缀和的代数关系过渡到哈希历史状态，树的后序汇总过渡到树形 DP，图的层次扩展过渡到最短路。只有这样，书才像教材，而不是题解堆。

不能用字数掩盖空洞

字数目标只是底线，不是质量本身。有效内容应当增加背景、推导、证明、边界、实验或源码对照；无效内容只是换词重复同一句话。审阅时要删掉不能回答新问题的段落。一个段落如果不能说明“为什么这样做”“错在哪里”“边界如何处理”“和另一个结构有什么区别”，就应该重写。

本书后续继续扩展时，应优先补三类内容：第一，更多经典题的完整推导；第二，真实 C++ 实验和性能对拍；第三，Linux 源码阅读报告。它们能自然增加深度，不需要硬凑。最终读者拿到的应是一套可执行的学习系统：能刷题、能讲题、能写代码、能读源码。

口述和代码审查样例

样例一：讲清前缀和

以前缀和题为例，合格口述不能停在“我用前缀和”。应当这样讲：我先把任意区间和写成两个前缀的差，因此当前右端固定时，问题变成寻找某个历史前缀。若题目问数量，历史表保存频次；若题目问最长，历史表保存最早位置；若题目问是否存在，保存布尔即可。这个区分直接决定代码里的 value 类型。

审查代码时，第一看初始化是否包含前缀零；第二看先查询还是先更新；第三看是否使用足够整数类型；第四看负数是否影响算法。和为 K 的子数组不需要数组全正，因为它只依赖代数等式；长度最小且至少为 K 的问题若含负数，普通窗口失效，要换成前缀和加单调队列。能说清这两个题的差异，就说明前缀和不是死记公式。

样例二：讲清二分

二分题的口述要先定义谓词。比如船运包裹，谓词是“容量为 cap 时，能否在 D 天内按顺序运完”。这个谓词随 cap 增大从假变真，因此要找第一个真。区间语义可以固定为左闭右开：答案始终在 $[left, right)$ 内，若 mid 可行，就把右边界收到 mid；否则把左边界移到 mid+1。

代码审查时，不要只看循环是否会停。要看下界是否为最大包裹，上界是否为总重量，检查函数是否在容量小于某个包裹时正确失败，求和是否溢出。很多二分错题的根源不在二分循环，而在谓词写错。二分只是加速寻找边界，不能修复错误的可行性判断。

样例三：讲清单调栈

单调栈题的口述要说明“栈里保存的是还没有被解决的候选”。每日温度中，栈里是还没找到更高温的日期；柱状图中，栈里是还没确定右侧第一个更矮位置的柱子；下一个更大元素中，栈里是等待未来更大值的元素。新元素到来时，如果它能解决栈顶，就不断弹出并结算。

代码审查时，第一看栈里存值还是下标。只要需要距离、宽度或过期判断，就必须存下标。第二看相等元素如何处理。第三看是否需要哨兵。第四看弹出时结算对象是谁。柱状图最大矩形里，当前更矮柱不是答案高度，它只是右边界；被弹出的柱子才是本次结算的最矮高度。

样例四：讲清图建模

图题口述第一句必须定义节点和边。岛屿数量的节点是陆地格子，边是四方向相邻；课程表的节点是课程，边是先修关系；公交路线题更适合把路线或“乘坐路线”作为 BFS 层次，而不宜把同一路线内所有站点两两连边；最小体力路径的边权是高度差，路径代价是最大边权，不是和。

代码审查时，第一看访问标记时机。BFS 通常入队时标记，避免重复入队。第二看状态维度。位置相同但钥匙集合不同、剩余障碍次数不同，都不能合并。第三看边权。等权用 BFS，非负权用 Dijkstra，零一权用双端队列。第四看邻居生成成本。单词接龙如果每次扫描所有单词找邻居，复杂度会很差，应考虑通配桶。

样例五：讲清 DP

DP 口述要从暴力搜索树开始。先说每个搜索节点需要哪些信息，再说哪些节点会重复出现，最后给状态命名。编辑距离的状态是两个前缀长度；背包的状态是物品阶段和容量；股票的状态是天数、持有状态和交易次数；树形 DP 的状态是子树向父节点返回的信息。状态不是为了填表而填表，而是为了让未来决策只依赖少量参数。

代码审查时，第一看初始化是否对应空状态；第二看转移是否覆盖所有最后一步；第三看遍历顺序是否满足依赖；第四看空间压缩是否读旧值。0-1 背包倒序容量，完全背包正序容量，LCS 一维压缩保存左上角旧值，股票状态机保存昨天状态。只要空间压缩解释不清，就先退回完整二维或多状态表。

源码阅读样例报告

报告一：链表删除

问题：删除一个已知链表节点需要维护什么不变量。对象：业务结构体中嵌入一个链表节点。索引：链表按某种顺序组织对象。插入时要让前驱、后继和新节点三者互相指向；删除时要让前驱直接指向后继，并让后继回指前驱。若遍历中删除当前节点，必须先保存下一个节点，否则删除后当前节点的链接可能被重置。

刷题类比：LRU 缓存移动节点到头部，K 个一组翻转链表接回前后段。工程差异：内核对象可

能同时挂在多条链上，删除某条链的节点不代表对象释放。并发情况下，还要考虑锁或 RCU。实验结论：链表操作的难点不是复杂度，而是节点身份、链接顺序和生命周期。

报告二：红黑树插入

问题：动态有序索引如何插入新对象。对象：业务结构体中嵌入树节点。索引：按某个 key 维护二叉搜索树顺序。插入前，调用者沿树比较 key，找到空孩子位置；链接新节点后，库函数通过旋转和变色修复平衡。旋转不改变中序顺序，所以查找语义保持不变。

刷题类比：日程安排用有序表找前驱后继，数据流排名维护动态有序集合。工程差异：C++ map 隐藏节点分配和比较器，Linux rbtree 让调用者负责比较和对象嵌入。实验结论：有序树解决的是顺序查询，不是比哈希表更高级；只要题目需要前驱后继，哈希表就不够。

报告三：RCU 删除

问题：读多写少结构中，删除对象为什么不能立刻释放。对象：读者可能正在访问的共享节点。索引：链表、树或哈希表让新读者找到对象。删除第一步是从索引摘除，让后续读者不可达；第二步等待旧读者离开读侧临界区；第三步释放内存。若摘除后马上释放，旧读者可能访问悬空指针。

刷题类比：Dijkstra 堆中的旧状态、滑动窗口堆中的过期元素。相同点是逻辑不可用和物理清理分离；不同点是 RCU 为并发安全服务，堆懒删除多为容器能力限制服务。实验结论：删除语义要拆开看，不能把“从答案集合移除”和“销毁对象”混成一步。

单点深挖：把模板拆成问题

这一节专门回应一个写作底线：单点问题要单点分析，不能把同一套话粘到每道题下面。真正的教材应该让读者看到每个问题自己的结构。接雨水不是“又一个双指针”，它的难点是何时可以结算某个位置的水；柱状图不是“又一个单调栈”，它的难点是被弹出的柱子在那一刻同时知道左右边界；LRU 不是“哈希加链表”，它的难点是节点身份、两个索引同步和对象生命周期。下面每个小节只抓一个具体难点，把暴力、优化、反例、实验和工程迁移连成一条线。

LeetCode 42 接雨水：先问结算对象

接雨水最常见的错误讲法是直接背“左右指针，谁小移谁”。这句话不解释为什么能算水，也不解释什么时候不能算。先从暴力出发：对位置 i ，水位等于左侧最高柱和右侧最高柱的较小值，再减去当前位置高度。暴力每个位置都向两边扫描，所以慢在反复寻找左右边界。前后缀数组的优化很直观：左边界数组保存每个位置左侧最高，右边界数组保存每个位置右侧最高，最后逐点结算。这个版本空间高一些，但最容易证明，因为每个位置结算时两个边界都已经明确。

双指针版本必须继承这个结算条件。设左侧已经看到的最高柱是 left max ，右侧已经看到的最高柱是 right max 。如果 left max 不大于 right max ，那么左指针位置的右侧至少存在一个不低于 left max 的边界，所以它的水位上限已经由 left max 决定，此时可以结算左侧并右移。反过来，如果 right max 更小，就结算右侧。注意，结算的是较低已知边界那一侧的位置，不是简单因为某个指针当前高度小就移动。代码里可以用当前高度更新边界，再根据边界大小结算，也可以先比较再更新，但不变量必须保持一致。

这道题的实验要同时跑三版：暴力版、前后缀版、双指针版。输入选择单调递增、单调递减、平台高度、多个凹槽、长度不足三、两端高而中间低。打印每一轮 left 、 right 、 left max 、 right max 和本轮新增水量。若某个实现提前结算了右侧边界未知的位置，平台高度和多凹槽会立刻暴露问题。把这组实验做完，读者才能明白双指针不是口诀，而是把“每个位置需要左右最高”压缩成两个移动边界。

单调栈解法是另一种结算对象。栈中保存还没有找到右边界的柱子，且高度保持单调。当遇到更高的当前柱时，栈顶作为槽底被弹出，当前柱是右边界，弹出后的新栈顶是左边界。此时槽底水量才确定。这个版本特别适合解释“弹出时机”：不是当前柱接水，而是被弹出的低柱终于知道了左右边界。把双指针和单调栈放在一起比较，能训练一个重要能力：同一题可以有不同状态，但每个状态都必须回答“现在能结算谁”。

LeetCode 76 最小覆盖子串：覆盖不是匹配总数

最小覆盖子串的暴力做法是枚举所有子串，再检查是否覆盖目标字符需求。这个暴力版清楚地暴露两个重复：窗口候选太多，检查需求时又反复统计。滑动窗口能成立，是因为右端扩展只会增加窗

口里的字符，覆盖条件更容易满足；当窗口已经覆盖目标时，继续扩展只会更长，所以应当收缩左端尝试变短。这个推理依赖“覆盖”是一个可增量维护的条件。

很多错误实现把覆盖理解成匹配字符总数。目标串如果是 AABC，只看到一个 A 不能算完成 A 的需求；窗口里多出来的无关字符也不能增加完成度。更稳的状态是 `need` 保存目标每个字符所需次数，`window` 保存当前窗口次数，`formed` 表示有多少种字符已经达到需求。一个字符从少于需求变成恰好达到需求时，`formed` 加一；从恰好达到需求被左端移走变成不足时，`formed` 减一。`formed` 等于 `need` 中字符种类数，才表示窗口覆盖目标。这里的“种类达标”比“总匹配数”更不容易被重复字符误导。

证明时按固定右端来看。当右端扩展到覆盖目标，内层循环会把左端一直右移到再移就不覆盖为止。在这个过程中，每个合法窗口都被检查，最短的那个被记录。对所有右端重复这个过程，全局最短一定被看见。被移过的左端不会再产生更短答案，因为未来右端只会更大，窗口只会更长。这个排除逻辑必须讲清楚，否则读者只能记住一段滑动窗口代码。

实验输入要包含目标字符重复、目标字符缺失、答案在开头、答案在结尾、大小写混合、源串短于目标串。调试时打印 `left`、`right`、当前字符、`formed`、`need size` 和当前最优区间。若用总匹配数写法，目标含重复字符的输入会暴露错误；若左端收缩后忘记更新 `formed`，答案在开头或结尾的用例会出错。C++ 里字符集固定时用数组即可；若题目不保证字符集，再用哈希表。容器选择只是实现细节，核心仍是“覆盖状态”到底表示什么。

LeetCode 862 最短子数组至少为 K：负数为什么击穿窗口

很多人看到“连续子数组”和“至少为 K”就想用滑动窗口。若数组全为正数，右端扩展会让窗口和变大，左端收缩会让窗口和变小，所以窗口条件具有单调性。但一旦允许负数，右端加入一个负数可能让和变小，左端移走一个负数可能让和变大，左端不再能安全排除。用一个小反例就能说明问题：窗口当前不满足 K，不代表继续扩展一定更接近；窗口当前满足 K，移走左端也不一定破坏或改善。普通窗口失去依据。

这题正确的切入点是前缀和。区间和等于 `prefix[j]` 减 `prefix[i]`，目标是找到 `j` 固定时最靠右、同时 `prefix[i]` 不大于 `prefix[j]` 减 K 的历史下标 `i`。为了让长度最短，历史下标越大越好；为了让未来更容易满足条件，前缀和越小越好。单调队列保存候选历史下标，队列里的前缀和递增。扫描当前前缀时，若当前前缀减队首前缀至少为 K，就可以更新答案，并弹出队首，因为对未来来说，更靠后的当前 `j` 已经让这个队首不可能给出更短长度。随后，若队尾前缀和大于等于当前前缀，则队尾被当前下标支配：当前下标更靠后，前缀和还不大，未来任何右端选择当前下标都不差。

这道题的重点是两个弹出条件含义完全不同。队首弹出是“已经形成一个答案，为了找更短长度继续丢旧左端”；队尾弹出是“旧候选被新候选支配，以后没必要保留”。如果只背单调队列，很容易把两个条件混成一个。实验要准备含负数、全正、答案不存在、答案为一、前缀和反复下降、K 很大这些输入。打印 `prefix`、队列下标、队列前缀值和答案。看到队尾被支配的过程，读者才会理解单调队列不是排序结构，而是保留一组仍有可能成为最优左端的历史状态。

这题也适合作为“模板失效”的案例。LeetCode 209 长度最小的子数组可以用正数窗口，LeetCode 560 和为 K 的子数组可以用前缀和加哈希计数，LeetCode 862 要用前缀和加单调队列。三个题都在说连续区间，但保存的历史状态不同：正数窗口保存当前和，计数题保存历史前缀频次，最短且含负数保存单调的历史前缀下标。把它们放在一起，读者才能学会根据条件换结构。

LeetCode 84 柱状图：弹栈时到底算谁

柱状图最大矩形的暴力模型很清楚：把每根柱子当作矩形最矮高度，向左右扩展到第一个更矮柱子之间，面积就是高度乘宽度。暴力慢在每根柱子都重复找左右第一个更矮位置。单调栈的目的就是在一次扫描中确定这些边界。栈中保存高度递增的下标，表示这些柱子还没遇到右侧第一个更矮位置。

当当前柱子高度小于栈顶柱子时，栈顶柱子的右边界已经确定为当前下标。弹出栈顶后，新的栈顶就是它左侧第一个更矮柱子。于是被弹出的柱子作为最矮高度时，最大宽度就是右边界减左边界再减一。如果弹出后栈为空，说明左侧没有更矮柱，宽度从零延伸到当前下标之前。这里最容易错的是把当前柱子当成答案高度。当前柱子只是触发结算的右边界；真正被结算的是被弹出的柱子。

相等高度也要有一致策略。可以遇到小于栈顶才弹，也可以遇到小于等于时弹，只要宽度语义和重复高度处理一致。教材阶段建议先用末尾哨兵零，让所有柱子最终都被结算，避免循环结束后再写一段补丁逻辑。实验输入必须包括严格递增、严格递减、全相等、单柱、空数组和中间凹槽。严

格递增会暴露末尾未结算，严格递减会暴露宽度计算，全相等会暴露相等高度策略。

这道题还连接 LeetCode 85 最大矩形。二维矩阵中每一行都可以看成柱状图底部，连续一的高度向下累积，然后对每一行运行柱状图算法。这里的迁移不是复制代码，而是迁移“某个高度作为最矮高度时找左右边界”的模型。读者若能从柱状图自然推到最大矩形，就说明已经掌握了单调栈的结算对象，而不是只记住一段弹栈代码。

LeetCode 15 和 18：排序、去重和输出规模

三数之和最容易被讲成“排序加双指针”，但排序为什么有用，去重为什么必须分层，往往没有讲透。暴力三层循环覆盖所有下标三元组，慢在组合数量太多，也慢在重复值会生成大量相同答案。排序后的第一层收益是制造单调性：固定第一个数后，剩下两个数的和随左指针右移而增大，随右指针左移而减小。这样可以安全排除一批配对，而不是逐个枚举。

第二层收益是去重变得局部可见。固定值如果和前一个固定值相同，跳过当前固定值，因为它会生成同一组数值答案。找到一组三元组后，左指针和右指针也要跳过相同值，避免相同数值组合重复进入结果。注意，去重不是最后把答案塞进集合。集合过滤能让结果看起来正确，但它掩盖了算法结构，也增加了额外成本。高质量题解应该在生成阶段就避免重复答案。

四数之和是三数之和的压力测试。外层多固定一个数，目标和可能超出 int，答案数量也可能很大。排序后可以用最小可能和与最大可能和提前剪枝：若当前固定前缀加上后面最小几个数已经大于目标，后续更大固定值也不必继续；若加上最大几个数仍小于目标，则当前固定值太小，可以跳过。这个剪枝必须建立在排序上，也必须注明溢出。C++ 中目标和、中间和最好用更宽整数。

实验要包含全零、大量重复、无答案、正负混合、目标极大、答案很多。还要专门测输出规模：当答案本身很多时，复杂度不能只写计算部分，结果数组也占空间和时间。面试追问 k 数之和时，不要直接写递归模板。先说明递归每层固定一个数，最后降到双指针；再说明每层去重和剪枝如何保持不重复、不漏解。这样讲，三数、四数和 k 数才是一条连续的教材线。

答案二分：真正要审查的是谓词

二分答案题表面上在写循环，实际难点在检查函数。以 LeetCode 410 分割数组的最大值为例，猜一个最大段和上限 limit，检查能否把数组按原顺序分成不超过 k 段，使每段和不超过 limit。检查函数的贪心是尽量把当前数字放进当前段，放不下才开新段。为什么这样正确？因为顺序固定，提前开新段只会让剩余段数更紧，不会减少所需段数。这个证明比外层二分更重要。

LeetCode 875 爱吃香蕉也是一样。速度 speed 越大，吃完所需时间越少，因此可行性从假到真。检查函数里每堆耗时要向上取整，不能用普通除法。边界也由谓词决定：速度下界是一，上界可以取最大堆；船运包裹的容量下界应是最大包裹，上界是总重量；分割数组的 limit 下界是最大元素，上界是总和。若下界设错，检查函数可能处理本不该出现的非法状态。

二分实验要把外层循环和检查函数分开测。先写一个线性枚举答案的慢版本，再写二分版本，两者对拍。对每个 mid 打印 left、right、谓词值和检查函数返回的段数或小时数。若二分结果错，先怀疑谓词，不要立刻改加一减一。很多错误都是“谓词并不单调”或者“检查函数没有表达题意”，外层循环再标准也无济于事。

这类题还要说明复杂度维度。它通常不是 $O(n \log n)$ ，而是 $O(n \log M)$ ，其中 M 是答案范围。背包、金额、容量、速度、最大段和这类数值进入复杂度时，要主动说明它和输入长度不是同一个维度。面试中能讲清这个点，会明显区别于只背模板的人。

LeetCode 146 LRU：两个索引和一个节点身份

LRU 缓存的暴力实现可以用数组保存访问顺序。每次 get 或 put 时线性查找 key，再把元素移动到头部；容量满时删除尾部。这个版本正确但慢，瓶颈在定位 key 和移动节点。哈希表能常数定位 key，但不能表达最近使用顺序；双向链表能常数删除已知节点并移动到头部，但不能按 key 查找。组合结构的必要性就在这里：哈希表负责定位，链表负责顺序，二者共享同一个节点身份。

节点里必须保存 key 和 value。很多人疑惑哈希表已经有 key，链表节点为什么还要保存 key。原因在淘汰尾部时，链表先告诉我们最久未使用的节点，但要从哈希表中删除对应条目，必须知道它的 key。若节点只保存 value，就无法常数时间同步删除哈希索引。更新已有 key 时也不能新建重复节点，否则哈希表和链表会出现两个身份不一致的对象。正确做法是更新节点值，并把同一个节点移动到头部。

这题可以直接对照 Linux 侵入式链表。刷题版常用 `std::list` 让容器拥有节点，哈希表保存

迭代器。内核常把 `list_head` 嵌进业务对象，链表只组织对象，不拥有对象。共同点是节点身份必须稳定，删除必须同步多个索引；区别是内核还要处理引用计数、锁、RCU 和分配上下文。把这种差异讲出来，LRU 就不再是六个字，而是一个小型索引系统。

实验要跑操作序列而不是只测单次 `get`。容量为一、重复 `put`、`get` 不存在、更新已有值、`get` 后再 `put`、连续淘汰都要覆盖。每步打印链表从头到尾的 `key` 顺序和哈希表大小，检查两个不变量：哈希表每个 `key` 都能在链表中找到唯一节点，链表尾部就是最久未使用节点。若某个操作后链表有重复 `key` 或哈希表指向失效迭代器，说明节点身份没有维护好。

并查集：代表元不是路径

并查集适合回答“两个元素是否在同一集合”以及“把两个集合合并”。它不保存两点之间的具体路径，也不适合回答最短距离。LeetCode 684 冗余连接中，按顺序加入无向边；若一条边的两个端点已经同属一个集合，再加入它就会形成环。这道题不需要知道环上所有边，也不需要给出路径，所以并查集正好匹配。暴力每加一条边都 DFS 检查连通性，会重复扫描已有结构。

并查集的正确性来自等价关系。每个集合有代表元，`find` 返回代表；`unite` 只把两个代表连起来。路径压缩改变的是树形实现，不改变集合语义；按大小合并降低树高，也不改变集合语义。初学者容易把 `parent` 数组看成图的真实父边，这是错误的。压缩后 `parent` 会直接指向根，原图路径信息已经不存在。如果题目要输出路径，不能只靠普通并查集。

账户合并是另一个代表性例子。邮箱是连接账户的证据，姓名通常不能作为合并依据。可以给每个邮箱编号，把同一账户中的邮箱合并；最后按根收集邮箱并排序。这里并查集解决的是动态连通，哈希表解决的是字符串到编号的映射，排序解决的是输出格式。三个结构各司其职。若题目变成等式除法或带奇偶约束，并查集就要扩展权值或颜色；普通代表元不够。

实验要打印合并前后每个元素的根、集合大小和 `parent` 树高度。重复合并、已经连通的边、孤立点、字符串编号映射都要测。还可以故意去掉按大小合并或路径压缩，观察 `find` 路径变长。这个实验的目的不是追求近乎常数的理论证明，而是让读者看见代表元如何把“许多连接关系”压缩成一个可查询的集合状态。

Dijkstra：旧堆状态不是错误

带正权图不能用普通 BFS，因为 BFS 按边数扩展，不按总代价扩展。LeetCode 743 网络延迟时间中，从起点到各点的最短时间取决于边权和。Dijkstra 的核心状态是当前已知最短距离，优先队列每次弹出距离最小的候选。若边权非负，弹出的未过期状态已经不可能被其他路径再改小，因此可以扩展它的出边。

C++ 的 `priority_queue` 没有 `decrease-key`。发现更短距离时，常见写法不是修改堆中旧元素，而是把新距离再压入堆。这样堆里会同时存在同一个节点的旧状态和新状态。弹出时比较堆顶距离和 `distance` 数组，若不一致，说明它已经过期，跳过即可。旧堆状态不是错误，只要使用前检查，它只是容器接口限制下的工程实现。

这点可以和 RCU 或懒删除做概念对照，但不能混为一谈。Dijkstra 的旧状态是算法实现中的过期候选，跳过即可；RCU 的旧对象是并发读者可能仍在访问的内存，必须等宽限期后释放。共同点是逻辑状态和物理存在分离；不同点是安全目标完全不同。教材要把相似性和差异都说清楚，避免把源码概念泛化成空话。

实验要构造重边、零权边、不可达节点、多条等长路径和明显会产生旧堆状态的图。打印每次弹出的节点、堆中距离、当前 `distance`、是否过期。再用普通 BFS 跑同一张带权图，观察结果差异。最后让读者回答：如果边权为负，为什么这个证明失效？这个问题能把 Dijkstra 的条件边界讲清楚。

DP：从暴力搜索树命名状态

动态规划最怕直接给公式。以编辑距离为例，暴力思路是从两个字符串末尾开始，如果末尾字符相同，就处理剩余前缀；如果不同，就尝试删除、插入、替换。这个递归会反复处理同一对前缀长度。于是状态自然命名为 `dp[i][j]`，表示第一个字符串前 `i` 个字符变成第二个字符串前 `j` 个字符的最少操作。公式不是凭空出现，而是暴力选择的缓存。

背包也是同样逻辑。暴力枚举每个物品选或不选，未来只依赖“处理到第几个物品”和“剩余容量”。0-1 背包每个物品只能用一次，所以一维压缩时容量倒序，防止当前物品在同一轮被重复使用；完全背包允许重复使用当前物品，所以容量正序。若把循环方向背成模板，不构造反例，就很容易在分割等和子集或零钱兑换之间写错。

最长递增子序列展示了另一类优化。二次 DP 的状态是必须以 i 结尾的最长长度，枚举所有前驱。二分优化维护 tails，表示某个长度的递增子序列能拥有的最小结尾。tails 不是答案序列，而是未来扩展潜力表。这个区别必须说清楚，否则读者会误以为 tails 中的值就是实际选择路径。若要还原路径，还要记录前驱和每个长度对应的位置。

DP 实验不要一上来压缩空间。先打印完整表，让读者解释每个格子来自哪里。编辑距离表能看到插入、删除、替换；LCS 表能看到不连续子序列；背包表能看到每个物品阶段如何影响容量；股票状态机能看到持有、卖出、休息的约束。只有完整表能讲清，再做滚动数组。空间压缩是优化，不是理解的起点。

Trie 和单词搜索：前缀剪枝不等于路径合法

单词搜索 II 的暴力做法是对每个单词在棋盘上单独 DFS。慢点在于不同单词共享大量前缀，却被重复搜索。Trie 把所有单词的前缀合并，棋盘 DFS 时沿 Trie 节点前进；如果当前路径在 Trie 中没有对应前缀，就立即剪枝。这个优化节省的是“重复检查前缀”的工作，而不是改变网格路径规则。

路径合法性仍然由访问标记控制。同一条路径中，一个格子不能重复使用；进入格子要标记，递归返回要恢复。Trie 节点上存在单词结束标记，只表示当前路径构成一个词；它不允许你忽略网格访问约束。找到一个词后，可以清空该节点的单词标记避免重复加入答案，也可以在工程上做叶子删除减少搜索，但这些都不能破坏其他单词共享的前缀。

实验要准备重复字母棋盘、单字符单词、一个单词多条路径、多个单词共享前缀、路径必须回退的用例。打印当前坐标、路径字符串、Trie 深度和访问矩阵。若忘记恢复访问标记，其他分支会漏答案；若没有访问标记，同一格子会被重复使用；若找到词后错误删除共享节点，另一个共享前缀的词会丢失。这个单点能把回溯、Trie 和状态恢复三者联系起来。

C++ 实现时要考虑节点所有权。教学代码可以用数组下标池或智能指针；若用裸指针，必须说明释放责任。字符集固定为小写字母时，子节点数组更快；字符集稀疏时，哈希表更省空间。不要在 DFS 热路径里反复构造大字符串，可以用路径数组或直接在 Trie 节点保存完整单词。容器选择仍然服务于访问模式。

动态区间和 Maple Tree 思维

离线区间问题和动态区间问题的难点不同。LeetCode 56 合并区间只需排序后线性扫描；所有区间一次性给出，排序把可能相交的区间放到相邻位置。日程安排、内存区间表或虚拟地址区域管理则是动态问题：区间会插入、删除、查询，不能每次都重新排序全部数据。此时需要有序结构维护相邻关系。

用 `std::map` 写简化动态区间表，key 是区间起点，value 是终点和属性。点查询时，找起点不大于目标地址的最后一个区间，再判断终点是否覆盖。插入新区间时，只需检查前驱和后继是否重叠；如果它们都不冲突，更远区间也不可能冲突。删除一段时更复杂：可能删除整个区间，可能截掉左侧或右侧，可能从中间挖掉一段并分裂成两个区间。

Maple Tree 的源码背景比这个实验复杂得多，但问题动机相通：大地址空间、稀疏区间、频繁查询、动态更新、读路径性能和并发安全。刷题阶段不需要复刻 Maple Tree，但可以通过 map 区间表理解范围映射的基本不变量。读源码时要问节点保存了哪些范围信息，查询如何找到覆盖区间，更新如何保持相邻关系，旧节点何时释放。

实验用例要包括插入到最前、插入到最后、端点相接、完全覆盖、跨多个区间、删除中间子段、删除不存在区间。报告要写清半开区间和闭区间的差别。半开区间下，前一个终点等于后一个起点不重叠；闭区间下，端点相等可能冲突。很多区间问题的错误都来自端点语义没有先定下来。

伙伴系统：总量够不代表连续块够

算法题常把内存分配当成免费，但系统里分配器本身就是算法。伙伴系统把连续页按 2 的幂次阶管理。分配某阶块时，若该阶没有空闲块，就向更高阶找，找到后逐层拆分；释放时，如果伙伴块同阶且空闲，就合并成高一阶块。这个机制强调连续性和对齐，而不是只看空闲页总量。

一个系统可能总空闲页很多，却没有足够大的连续块。比如很多小块分散在不同位置，总和超过请求大小，但无法合并成所需阶数。这就是碎片问题。刷题里可以把它类比为区间合并和二进制对齐，但不能简化成普通计数。每个空闲块必须只出现在它当前阶的链表里，拆分和合并都要维护这个不变量。

实验可以设总页数为 1024，维护从零阶到十阶的空闲链表。分配时打印从哪个高阶拆下，释放时打印伙伴编号和是否合并。构造一个碎片案例：先分配多个小块，再释放其中一部分，让总空闲

量看似足够，但高阶请求失败。这个实验能把位运算、链表、区间、贪心分配和工程性能放到一起理解。

把这个视角带回 C++ 容器，读者会更容易理解为什么大量链表节点、哈希节点和树节点的分配成本不能忽略。复杂度都是线性或对数，连续数组可能因为缓存局部性和少分配而快很多；节点容器可能因为稳定身份和常数删除而必要。算法复杂度给出数量级，分配器和内存局部性决定真实表现。

题解写作：每道题至少回答一个自己的问题

写题解时，不要让所有题都长成同一张表。可以有统一的复盘框架，但正文必须回答本题自己的问题。两数之和要回答为什么先查再插入；无重复子串要回答左端为什么不能回退；接雨水要回答何时能结算某个位置；柱状图要回答弹栈时算谁；LRU 要回答为什么节点里要保存 key；Dijkstra 要回答堆中旧状态为什么可以跳过；背包要回答容量循环方向从哪里来。

一个合格段落应当增加新信息。它可以解释一个错误实现为什么错，可以给一个最小反例，可以把暴力中的重复工作定位出来，可以说明一个容器的副作用，可以把源码结构和刷题模型做有限对照。如果一个段落只是在说“注意边界”“使用哈希优化”“复杂度降低”，但没有说明具体边界、具体 key、具体降低了哪一步，就应当重写。

题解还要有实验意识。每个高频题至少保留一个暴力对拍版本或小规模枚举器。前缀和题用随机数组对拍，二分题用线性枚举答案对拍，单调栈题打印入栈弹栈，DP 题打印小表，图题打印完整状态，链表题画节点连接。实验不是附属内容，它是把“我觉得对”变成“我知道为什么错不了”的过程。

最后是语言衔接。章节不是题号仓库，应该从一个问题过渡到下一个问题：连续内存引出双指针和窗口，窗口失效引出前缀和与单调队列，栈的最近未完成结构引出表达式和柱状图，动态候选集引出堆和最短路，子问题重复引出 DP，动态有序关系引出区间树和源码结构。这样的书读起来才像教材，而不是把许多答案贴在一起。

源码与系统结构精读

刷题教材加入 Linux 源码，不是为了让读者背内核字段名，而是为了让读者看到同一类数据结构在真实系统中为何会变复杂。力扣题通常默认单线程、对象生命周期简单、内存分配随时可用；内核代码必须同时考虑并发读写、引用计数、缓存局部性、分配上下文和对象同时进入多个索引。下面这些小节仍然坚持单点分析，每节只追一个问题。

list head：为什么节点嵌进对象

普通 C++ 初学者会把链表理解为“容器里存元素”。Linux 的侵入式链表不是这样。业务对象中嵌入链表节点，链表节点只负责前后指针，不拥有对象，也不知道对象类型。遍历时通过成员偏移从链表节点回到外层对象。这个设计看起来底层，但它直接解决三个工程问题：减少额外分配，让对象可以同时进入多条链表，保持对象地址稳定。

把它和 LRU 放在一起看会很清楚。刷题版 LRU 中，缓存条目既需要按 key 被哈希表找到，又需要按新旧顺序进入链表。如果链表节点和业务对象分离，删除和移动时要处理额外间接层；如果对象自己带链表节点，哈希表可以直接指向对象，链表只改变对象中的链接字段。内核里一个页面对象可能同时出现在 LRU 链、回写链或其他状态链上，所以对象中可以有多条链表节点字段，每个字段代表一条不同关系。

单点实验是写一个 Item，里面有 key、value、lru node、dirty node。先把三个 Item 挂入 LRU 链，再把其中两个挂入 dirty 链。删除一个对象的 dirty node，不应影响它在 LRU 链中的位置；释放对象前，必须先确认它已经从所有链和索引中摘除。这个实验比单纯反转链表更接近源码思维，因为它把“节点身份”和“对象生命周期”放在同一处观察。

hlist：哈希桶为什么不一定用完整双链

哈希表有大量桶，很多桶可能为空。如果每个桶头都使用完整双向链表节点，桶数组本身就会消耗更多空间。hlist 的设计可以理解为桶头较轻，节点保存足够信息以支持已知节点删除。桶头只需要知道第一个节点，节点除了 next，还保存指向自己的前驱链接位置。这样删除已知节点时，可以直接修改前驱的 next 或桶头的 first，不必从桶头扫描找前驱。

刷题里哈希冲突通常被标准库隐藏，但很多题都在使用同一思想：key 先定位桶，桶内再比较实

际 key。两数之和的 key 是历史值，字母异位词的 key 是规范签名，前缀和计数的 key 是前缀值，账户合并的 key 是邮箱。高质量题解要说清 key 和 value，不应只说“用 unordered map”。如果 key 设计错了，哈希表再快也会得到错误答案。

实验可以写两个固定桶数的小哈希表。第一版桶内普通单链表，删除已知节点必须找前驱；第二版节点保存前驱链接位置，删除时不用扫描。再设计一个遍历中删除的场景：如果读者正在走桶链，写者删除节点后什么时候能释放对象？单线程实验只需保存 next，内核并发场景则需要锁或 RCU。这个单点把哈希冲突、删除成本和并发生命周期自然连起来。

rbtree：比较逻辑为什么交给调用者

C++ 的 map 把比较器和节点管理封装在容器里。Linux rbtree 更像一个底层平衡树工具：树节点嵌在业务对象里，插入和查找时，调用者自己沿树比较 key，找到位置后再调用库函数链接和修复平衡。这样设计的好处是业务对象可以按不同字段进入不同树，内存分配和对象生命周期也不被通用容器隐藏。

红黑树单点要抓住“旋转不改变中序顺序”。插入新节点后，树可能违反颜色和平衡约束，修复函数通过变色和旋转调整局部结构。旋转前后，中序遍历得到的 key 顺序相同，所以查找语义不变。很多教材一上来讲红黑树性质，读者很快迷路；源码阅读可以先问一个更小的问题：调用者如何比较 key，库函数如何在不破坏有序性的前提下修复平衡。

刷题对应的是动态有序集合。日程安排要找前驱后继，滑动窗口有序统计要插入删除并取极值，时间键值存储在每个 key 内部按时间二分，动态区间表需要按起点维护顺序。哈希表适合等值查找，不能回答前驱后继。实验用 map 实现日程安排，再手写一个简化 BST 插入，不要求实现红黑修复，只要求打印中序序列。等读者理解有序语义稳定后，再看平衡修复才不会把重点放错。

XArray：稀疏整数索引不是普通哈希

XArray 面向整数索引到对象指针的映射，适合大而稀疏的索引空间。巨大数组会为空洞浪费空间，普通哈希表能做等值查找，但对按序推进、范围扫描、标记和并发读写支持不一定合适。XArray 这类结构把整数索引按位分层，只为实际存在的路径分配节点。它和 Trie 的直觉相似：字符串 Trie 每层消耗一个字符，整数分层索引每层消耗几位。

刷题里可以用三个模型建立直觉。位 Trie 用二进制位分层，常见于最大异或；坐标压缩把稀疏大值域映射到紧凑数组下标；有序 map 支持稀疏 key 的顺序遍历。XArray 和它们都不完全相同，但共享一个问题：key 空间很大，实际对象稀疏，需要在空间、查找、遍历和更新之间折中。

实验可以写一个简化二进制 Trie 保存整数集合，支持插入、查找、寻找不小于某个值的下一个元素。再和数组、哈希表、map 比较：数组适合小连续值域，哈希适合稀疏等值查找，map 适合动态有序，Trie 适合可按位分解的 key。读 XArray 时不要先背节点字段，先回答索引如何分层、空洞如何表示、读者什么时候能看到新对象。

Maple Tree：范围映射的三个操作

范围映射比单点映射更容易出错。给定地址要找到覆盖它的区间，插入新区间要检查重叠，删除区间可能把原区间切开。Maple Tree 服务的正是这种范围索引场景。它比普通 map 复杂，是因为真实内核还要追求读路径性能、处理并发、减少分配和维护更复杂的节点布局。但教材阶段先抓三个操作：点查询、插入、删除。

用简化 map 区间表可以训练这些操作。点查询先找起点不大于地址的最后一个区间，再看终点是否覆盖；插入只需检查前驱和后继，因为已有区间不重叠且按起点有序；删除有四种情况：删除整段，截掉左侧，截掉右侧，从中间挖掉一段并分裂。每种情况都要画图，否则代码很容易在端点处错。

这和力扣 56、57、729、731 是一条线。合并区间是离线排序扫描，插入区间是一次性合并连续重叠段，我的日程安排是动态插入并检查冲突。Maple Tree 是系统级动态范围集合。读者不需要一开始理解全部源码，但要知道为什么一个普通哈希表无法回答“哪个区间覆盖这个地址”，也无法按地址顺序找下一个区域。

RCU：删除要拆成摘除和释放

RCU 最适合从链表删除理解。单线程删除一个节点，可以改前后指针后立刻释放节点。读多写少并发场景不行：读者可能已经拿到旧指针，写者把节点从新索引中摘除后，旧读者仍可能继续访问它。于是删除必须拆成两个阶段：先从结构中摘除，让后来的读者找不到；再等待所有可能看到旧节点

的读者离开；最后释放内存。

这个拆分和刷题里的懒删除有相似直觉，但目的不同。滑动窗口中位数的堆懒删除是因为 priority queue 不能删除任意元素；Dijkstra 的旧堆状态是因为没有 decrease-key；RCU 延迟释放是为了内存安全。相似点只是“逻辑不可用”和“物理存在”分离，不能把它们说成同一种机制。教材要防止概念类比过头。

实验可以模拟一个全局配置表。读者进入读侧临界区后读取当前指针；写者复制旧表，修改副本，发布新指针，把旧表放入延迟释放列表。只有当活跃读者计数为零时，才释放旧表。这个实验能让读者看到，读路径轻量的代价是写路径更复杂。回到算法题，读者也会更谨慎地使用“删除”这个词：从集合不可达、从答案中移除、从内存释放，三者不是同一件事。

调度队列：为什么动态最小值不总是堆

调度器需要在可运行任务中选择下一个运行者。一个简化模型是每个任务有虚拟运行时间，每次选择虚拟运行时间最小的任务运行，运行后它的虚拟运行时间增加，再放回候选集合。直觉上堆可以取最小值，但真实调度还需要删除任意任务、处理睡眠唤醒、按顺序遍历、维护权重和调度类。堆只解决“取最小”，不解决全部操作。

刷题中类似问题很多。数据流中位数用双堆，因为只需要两半极值；日程安排用有序表，因为需要前驱后继；任务调度器可以用堆模拟，也可以用最高频公式；滑动窗口最大值用单调队列，因为候选只按窗口顺序过期。结构选择必须列出操作集合：插入、删除任意元素、取最值、找前驱后继、范围遍历、按时间过期。只看一个操作很容易选错结构。

实验写一个简化调度模拟器。第一版用 priority queue，每次任务运行后压入新状态，睡眠任务只能懒删除；第二版用 set，可以按迭代器删除任意任务，再重新插入更新后的任务。比较两版代码，读者会看到接口能力的差异。复杂度同为对数，维护语义却不同。这个单点能把堆、有序树和系统队列联系起来。

页缓存和 LRU：真实淘汰不是一道缓存题

力扣 LRU 缓存的规则很干净：访问就移动到头部，容量满就淘汰尾部。页面回收里的 LRU 思想复杂得多。页面可能是文件页或匿名页，可能干净或脏，可能正在写回，可能被引用，可能属于不同内存区域。系统不能简单删除“最旧页面”，还要考虑 I/O 成本、回收收益和并发状态。

教材中引入这个差异，是为了防止读者把刷题结构绝对化。哈希表加双向链表是 LRU 的核心索引组合，但真实系统可能使用多条链表达状态分层，例如活跃和非活跃，文件和匿名。页面从一条链移动到另一条链，不仅是顺序变化，也是状态变化。删除页面也不是释放一个普通对象，还要处理引用、脏页写回和映射关系。

实验可以从简化两级 LRU 开始。把页面分成 active 和 inactive 两条链，访问 inactive 页面时提升到 active，周期性把 active 尾部降级，回收从 inactive 尾部开始。再加入 dirty 标记：脏页不能立即释放，需要先模拟写回。这个实验不追求复刻内核，只让读者看到“链表表达状态，哈希表达定位，标记表达附加约束”。再回到 LRU 题，读者会更理解为什么节点要稳定、为什么多个索引要同步。

高频设计题精读

设计题比普通单题更容易露出模板化问题，因为它们通常不是一个数据结构就能完成，而是多个索引协同维护。每次读设计题，都要写出操作表：每个操作需要什么查询、什么更新、什么删除、什么顺序。下面几个题都按这个方式分析。

LeetCode 460 LFU 缓存：频次和时间两个维度

LFU 缓存要求淘汰使用频次最低的项；如果频次相同，淘汰最久未使用的项。它比 LRU 多一个频次维度。暴力做法可以每次淘汰时扫描所有 key，找最低频次和最旧时间，复杂度太高。要做到常数时间，需要两个索引：key 到节点信息，frequency 到同频节点链表。还要维护当前最小频次 min freq。

节点信息至少包含 key、value、freq，以及它在对应频次链表中的位置。get 命中后，节点频次加一：先从旧频次链表删除，再加入新频次链表头部；若旧频次链表空且旧频次等于 min freq，min freq 加一。put 新 key 时，若容量满，从 min freq 对应链表尾部淘汰最旧节点。插入新节点的频次为一，因此 min freq 重置为一。

这题最容易错在只更新频次，不更新同频时间顺序；或者删除节点后忘记维护 min freq。实验序列要包含容量为零、容量为一、同频淘汰、get 后频次变化、put 已存在 key、低频链表清空。每步打印 min freq、每个频次链表顺序和 key 表大小。LFU 的核心不是“哈希加链表”四个字，而是频次索引和时间索引同时一致。

LeetCode 895 最大频率栈：为什么 frequency 到 stack 足够

最大频率栈要求 pop 时返回出现频率最高的元素；若频率相同，返回最近压入的元素。暴力可以每次 pop 扫描所有元素，找最高频和最近时间。优化时观察两个维度：value 到当前频率，frequency 到达该频率的元素栈。push 一个值时，它的频率从 f 变成 f+1，把这个值压入 group[f+1]，同时更新 max freq。

pop 时直接从 group[max freq] 弹出最近达到最高频的值，然后把它的当前频率减一。若 group[max freq] 变空，max freq 减一。为什么不需要从旧频率栈中删除这个值？因为一个值每次频率增加都会在新频率栈留下记录，这些记录对应历史时刻；pop 只会从当前最高频栈弹出合法的最近记录。频率减一后，低频栈中旧记录仍然代表它之前达到低频时的顺序，后续在那个频率层可能被使用。

实验序列要让同一个值多次 push，并和另一个值形成同频竞争。打印 value frequency、max freq、每个 group 栈。若实现使用单个堆，也能做，但要处理过期记录；frequency 到 stack 的结构更贴合题意，因为“同频最近”天然就是栈。这个题训练的是把一个复杂排序规则拆成两级索引，而不是盲目使用优先队列。

LeetCode 981 时间键值存储：按 key 分组再二分

时间键值存储支持 set(key,value,timestamp) 和 get(key,timestamp)，返回该 key 在不超过 timestamp 的最新值。暴力做法把所有记录放在一个列表里，每次 get 扫描。优化的第一步是按 key 分组，因为不同 key 的记录互不影响。对每个 key，保存按时间递增的 pair 列表。get 时在该 key 的列表中二分，找最后一个时间不超过目标的记录。

这题的关键条件是 set 的 timestamp 通常按递增顺序到来。如果不递增，就需要插入时保持有序，或者最后排序。二分查找的是 upper bound：第一个大于目标时间的位置，答案是它前一个。若位置为开头，说明没有可用记录。这里不能用哈希表直接按 timestamp 查，因为查询是“不超过”的前驱问题，不是等值问题。

实验要覆盖 key 不存在、时间早于第一条记录、等于某条记录、落在两条记录之间、晚于最后记录、多个 key 交错写入。报告里要写清为什么结构是 hash map 到 vector，而不是一个全局 map。这个题非常适合训练“先按独立维度分组，再在组内使用有序结构”的思想。

LeetCode 632 最小区间：多路归并里的覆盖条件

最小区间覆盖 K 个列表，要求区间内至少包含每个列表一个数。暴力可以从所有元素中枚举左右端，再检查是否覆盖每个列表。优化从多路归并视角看：每个列表当前选一个元素，则当前 K 个元素天然覆盖所有列表，区间由其中最小值和最大值决定。为了缩小区间，应当推进当前最小值所在的列表，因为当前最大值不变时，只有提高最小值才可能缩短区间。

小顶堆保存每个列表当前元素，同时维护当前最大值。每次弹出最小元素，更新答案区间，然后把该元素所在列表的下一个元素压入堆，并更新最大值。如果某个列表耗尽，就无法再保持覆盖所有列表，算法结束。这个过程和合并 K 个有序链表相似，但多了当前最大值和覆盖区间的目标。

证明时看当前状态覆盖了所有列表。若不推进最小元素，而推进其他列表，当前最小值仍在，区间左端不变，最大值还可能变大，不可能得到更优区间。因此推进最小所在列表是安全的。实验覆盖某个列表只有一个元素、多个相同值、答案在开头、答案在中间、列表长度差异很大。C++ 中堆元素保存值、列表编号和下标，避免复制整个列表。

LeetCode 815 公交线路：节点选错会爆炸

公交线路题要求从起点站到终点站最少乘坐几辆公交。若把站点作为节点，边表示两个站在同一路线上相邻，建图会非常大；若把同一路线内所有站两两连边，更会爆炸。更稳的建模是站点到路线列表的哈希表，BFS 的层次表示乘坐公交次数。起点站能乘坐的所有路线作为第一层路线，访问一条路线时，可以到达这条路线上的所有站，再从这些站扩展到其他路线。

这里的访问标记要分路线和站点两个层次。路线一旦乘坐过，就不需要再次乘坐；站点如果已经用于扩展过，也可以避免重复扫描它对应的路线列表。目标是到达终点站，不一定要构造完整的

路线图。若起点等于终点，答案为零；若终点站不在任何可达路线中，返回失败。

实验要比较两种建模的扩展数量。构造多条长路线共享少量换乘站的输入，站点图会产生大量邻接关系，而路线 BFS 只在需要时通过站点找到可换乘路线。这个题说明图题第一步不是写 BFS，而是选择节点含义。节点选错，后面所有优化都只是在错误模型上补救。

LeetCode 1235 规划兼职工作：排序、二分和 DP 合体

规划兼职工作给出若干工作，每个工作有开始时间、结束时间和收益，要求选择不重叠工作最大化收益。按收益贪心会失败，因为高收益工作可能阻塞多个组合后更高的工作。暴力枚举子集会指数爆炸。正确切入点是按结束时间排序，定义 $dp[i]$ 表示考虑前 i 个按结束时间排序的工作时的最大收益。

对于第 i 个工作，有两种选择：不选它，收益是 $dp[i-1]$ ；选它，则要找到最后一个结束时间不超过当前开始时间的工作 j ，收益是 $dp[j]$ 加当前收益。因为工作已按结束时间排序， j 可以用二分找到。这个二分不是在答案空间上，而是在已经排序的结束时间数组上找前驱边界。

实验要覆盖完全不重叠、全部重叠、端点相接是否允许、同结束时间、同开始时间、收益很大的单个工作和多个小工作组合。打印排序后的工作、每个 i 找到的 j 、选择不选择的收益。这个题很好地展示了多技术衔接：排序制造可二分的结束时间，二分找到兼容前驱，DP 比较选或不选。任何一个环节单独背模板都不够。

LeetCode 1675 最小偏移量：先规范化状态空间

数组最小偏移量的操作规则有点绕：偶数可以除以二，奇数可以乘以二。若直接搜索所有可能状态，分支很多。关键观察是每个奇数最多先乘二一次，变成偶数后就只能不断除以二；每个偶数也只能不断除以二。于是可以先把所有数规范化到“可往下减少的最大形态”：奇数乘二，偶数保持。之后每次只尝试降低当前最大偶数，同时维护当前最小值。

用大顶堆保存当前所有数中的最大值，另有变量保存当前最小值。每轮用最大值减最小值更新答案；如果最大值是偶数，就把它除以二放回堆，并更新最小值；如果最大值是奇数，就无法继续降低它，算法结束。为什么每次只动最大值？偏移量由最大和最小决定，降低非最大值不会降低当前最大值，反而可能降低最小值让偏移更大。

实验覆盖全奇数、全偶数、混合、已经最优、最大值很大、多个相同值。打印堆顶、当前最小值和答案。这个题的单点是“规范化状态空间”：先把所有数推到统一方向可操作的边界，再用堆做贪心推进。它和很多搜索题相通，先把可逆或双向操作改成单向可控过程，复杂度才降下来。

实验报告样例库

下面给几份更具体的实验报告样例。它们不是为了增加形式，而是给读者一个可执行标准：每个实验都能复现一个算法为什么对，或者一个错误实现为什么错。

报告样例：前缀和对拍

问题：验证和为 K 的子数组哈希版是否漏掉边界。暴力基线：两层枚举左右端点，内部用累加变量得到区间和。优化版本：扫描前缀和，哈希表保存历史前缀频次。输入生成：数组长度从零到二十，元素范围从负三到三， K 从负五到五随机选取。每组数据同时跑暴力和优化，结果不同就打印数组、 K 、当前前缀、哈希表内容和下标。

预期发现：若漏掉初始前缀零，所有从下标零开始的答案会少计；若先更新再查询， K 为零时可能多算空区间；若用滑动窗口替代哈希，含负数输入会失败。结论：本题依赖代数等式，不依赖单调性；哈希表 value 必须是频次，因为题目问数量；查询更新顺序是算法语义的一部分。

报告样例：单调栈状态打印

问题：验证柱状图最大矩形宽度计算。暴力基线：每根柱子向左右找第一个更矮位置。优化版本：单调递增栈加末尾哨兵。输入集合：空数组、单柱、严格递增、严格递减、全相等、中间低谷。每次弹栈时打印当前下标、被弹出下标、高度、左边界、右边界、宽度和面积。

预期发现：若没有末尾哨兵，严格递增数组会漏算；若宽度写成 $right-left$ ，递减数组会面积过大；若把当前柱高度当作结算高度，中间低谷会错。结论：弹出时被弹出的柱子才是矩形高度，当前柱只提供右边界，弹出后的栈顶提供左边界。

报告样例：二分谓词审查

问题：验证船运包裹容量二分。暴力基线：从最大包裹到总重量线性枚举容量，找到第一个可行容量。优化版本：左闭右开二分第一个可行容量。检查函数：按顺序装包，当前天放不下就开新天，统计天数是否不超过 D。输入集合：一天运完、每天一个包裹、最大包裹等于答案、多个相同重量、容量小于最大包裹的非法 mid。

预期发现：若下界不是最大包裹，检查函数必须额外处理单包超过容量；若天数统计初始值错，边界会偏一；若二分返回未验证的 left，也可能隐藏谓词错误。结论：答案二分的核心是可行性谓词单调，外层循环只是利用这个谓词找边界。

报告样例：LRU 索引一致性

问题：验证 LRU 的哈希表和链表是否一致。暴力基线：用 vector 保存从新到旧的 key，每次操作线性移动。优化版本：哈希表加双向链表。操作序列：容量为二，依次 put 一、put 二、get 一、put 三、get 二、put 一更新、put 四。每步打印链表顺序、哈希表 key 集合、每个 key 指向的节点值。

预期发现：若 get 后不移动节点，淘汰顺序会错；若 put 已存在 key 时新建节点，链表会出现重复 key；若淘汰尾部时忘记删哈希表，后续 get 会访问失效节点。结论：LRU 的不变量有两个，哈希表到链表节点一一对应，链表顺序表达最近使用到最久未使用。

报告样例：图状态维度

问题：验证障碍消除最短路是否错误合并状态。暴力基线：小网格上用 BFS 保存行、列、剩余消除次数。错误版本：visited 只保存行和列。输入集合：必须绕路保留消除次数的网格、起点终点相邻、K 为零、K 大于障碍数。每次入队时打印位置、剩余次数和距离。

预期发现：二维 visited 会把“到达同一格但剩余资源更多”的状态丢掉，导致错误不可达或路径过长。结论：BFS 第一次到达最短，只对完整状态成立。状态维度少了，访问标记就会错误剪枝。

报告样例：背包循环方向

问题：验证 0-1 背包一维压缩方向。暴力基线：二维 dp，阶段是物品下标，容量是当前目标。错误版本：一维数组容量正序。输入集合：一个物品重量一、容量二；两个物品重量一和二、容量三；分割等和子集中的小数组。打印每轮物品处理后的 dp 数组。

预期发现：正序容量会让同一个物品在同一轮被重复使用。倒序容量读取的是上一轮阶段状态，符合每个物品只能用一次。结论：循环方向不是模板差异，而是题目对物品使用次数的约束。

章节衔接重写原则

本书后续扩展时，章节应该按能力递进，而不是按题号堆叠。线性结构部分先讲连续内存、下标和缓存局部性，再讲双指针和窗口；窗口失效后，自然引出前缀和、哈希和单调队列。栈和队列部分先讲最近未完成结构，再讲单调栈、表达式解析和柱状图。堆部分先讲动态极值，再讲 Dijkstra、数据流中位数和多路归并。

树和图部分也要有顺序。树先讲遍历方向：自顶向下传约束、后序向上汇总、层序传播；再讲 BST 的有序性和 Trie 的前缀共享。图先讲节点和边的建模，再讲 BFS 的层次、DFS 的连通块、拓扑的依赖、并查集的集合代表，最后讲带权最短路。这样读者能看见一个概念如何扩展，而不是在相邻页里突然切换题型。

动态规划部分必须从暴力搜索树开始。线性 DP 解释最后一步，二维 DP 解释两个前缀，背包解释阶段和容量，区间 DP 解释最后合并点，状态机解释约束转移，状态压缩解释集合。每个小节都要有一个完整表格实验。空间压缩只能在完整状态讲清后出现，不能一开始就给滚动数组代码。

Linux 源码专题放在基础算法之后更合适。读者先掌握链表、哈希、有序树、区间、队列、堆、分配器这些算法结构，再去看内核里的侵入式节点、hlist、rbtree、XArray、Maple Tree、RCU、伙伴系统和 SLUB，才不会被字段名淹没。源码章节不是另一本操作系统书，而是把算法结构放进真实工程约束里重新审视。

目录也要克制。章标题保留大的学习路线，小节标题只在正文中服务阅读，不把每个微型点都塞进目录。一本教材的视觉舒适来自层级稳定、段落长度适中、代码块清晰、图示服务推理。章节过碎会让读者感觉像在翻题库；章节过长没有锚点又会迷路。后续排版应在这两者之间取平衡。

新增专题候选清单

如果继续扩写，优先补这些不是模板、能提升质量的专题。第一，字符串算法专题：KMP 的失败函数为什么能跳转，Z 函数如何维护右边界，Manacher 如何把回文半径线性化，后缀数组和后缀自动机只作为进阶引导。每个算法都必须从暴力匹配出发，而不是直接给公式。

第二，数学和位运算专题：快速幂、欧几里得、质数筛、组合数、模逆元、同余、异或线性基。面试刷题不一定高频，但它们能训练代数不变量。写法上要强化适用条件，例如模逆元要求互质或模数为质数，位移要注意有符号整数。

第三，图高级专题：强连通分量、割点桥、最小生成树、二分图匹配、网络流只做求职向概览。重点不是铺算法名，而是说明它们分别回答什么问题：强连通回答有向互达，桥回答删边连通性，最小生成树回答连接全部点的最低总成本，匹配回答两侧资源配对。

第四，工程性能专题：缓存局部性、分配器、哈希退化、字符串拷贝、迭代器失效、递归栈、输入输出性能。每个点都要配小实验。算法书如果不讲这些，读者容易以为复杂度相同的代码真实表现也相同。

第五，面试表达专题：如何在五分钟内讲清一题，如何在追问中判断条件变化，如何承认不确定并回到暴力，如何用反例纠正错误贪心。这个专题能把刷题能力转成沟通能力，但必须用具体题演示，不能写成空泛建议。

反模板质量审查

这一节给后续写作和修订使用。算法教材最容易出现的低质量问题，不是少写几个题，而是把每道题都写成相同段落：先说暴力，再说优化，再说边界，再说 C++。这种结构本身可以作为复盘框架，但不能成为正文内容。正文必须回答本题自己的问题，否则读者读十页之后只会记住一套空话。

审查一：每题是否有自己的核心疑问

每道题提交前，先写一句“本题真正要解释的问题是什么”。两数之和的问题是为什么历史表要先查再插入；无重复子串的问题是为什么左边界不能回退；接雨水的问题是何时能结算某一侧；柱状图的问题是弹栈时谁的左右边界确定；课程表的问题是边方向如何影响输出顺序；零钱兑换的问题是最少数量和组合数量的状态含义不同。若这句话写不出来，说明题解还停留在题型标签上。

核心疑问必须能被一个反例检验。比如左边界回退用 `abba` 检验，前缀和初始零用单元元素等于目标检验，柱状图宽度用递减数组检验，背包循环方向用单物品容量二检验，图状态维度用剩余资源不同的同位置状态检验。没有反例的“注意边界”不是分析，只是提醒。

一个题可以有多个解法，但每个解法都要对应自己的疑问。接雨水的前后缀数组回答“每个位置的左右最高如何预处理”，双指针回答“何时能用较低已知边界结算”，单调栈回答“什么时候槽底知道左右边界”。如果把三种解法都写成“维护状态，线性扫描”，就是在抹掉差异。

审查二：暴力版本是否真的服务推导

暴力版本不是形式。它应该说明候选空间是什么，检查条件是什么，复杂度慢在哪一步。三数之和暴力枚举三元组，慢在组合数量和重复答案；排序双指针正是利用单调性和相邻重复来减少这两类问题。编辑距离暴力尝试三种操作，慢在相同前缀对反复出现；DP 状态正是给前缀对命名。若暴力版本只写“暴力会超时”，它没有提供推导价值。

暴力也不一定要写代码，但必须能口头执行。读者应当能拿一个小输入，按暴力过程跑完，看到哪些操作被重复。窗口题可以枚举所有左右端；图题可以从每个起点重新搜索；区间题可以两两比较；设计题可以每次操作扫描全部元素。只有低效过程清楚，优化才不是突然出现。

审稿时可以问一句：优化到底替换了暴力中的哪一行工作？哈希替换历史扫描，前缀和替换重复区间求和，单调栈替换左右边界寻找，堆替换每轮找极值，DP 替换重复递归，并查集替换重复连通搜索。若回答不上来，就说明题解在堆算法名。

审查三：状态是否有不变量

高质量题解必须写出状态含义，而不是只写变量名。slow 左侧保存什么，窗口内满足什么，哈希表保存当前下标之前还是包含当前下标，栈里是等待右边界的柱子还是等待更高温的日子，堆顶是否可能过期，`dp[i][j]` 表示哪个前缀，`parent` 数组是不是原图路径。这些问题都应在正文中说清。

不变量要能指导代码顺序。前缀和题先查再更新，是因为哈希表表示历史前缀；颜色分类遇到二不移动扫描指针，是因为换回来的元素仍属于未知区；LRU 更新已有 key 要移动节点，是因为访

问也改变最近使用顺序；拓扑排序入度变零才入队，是因为所有先修已满足。若状态含义不能决定代码顺序，说明它还不够具体。

不变量还要能解释结束条件。二分结束时 left 为什么是答案，BFS 第一次到达为什么最短，单调队列队首为什么是当前窗口最大值，DP 表最后一个格子为什么是全局答案，回溯到叶子时为什么可以收集路径。很多错误代码不是主体循环错，而是答案读取时机错。

审查四：C++ 内容是否和题目相关

C++ 提醒不能每题都写同一句“注意 long long”。只有涉及求和、乘积、面积、距离、时间和路径代价时，才重点讨论溢出。只有哈希查询语义会影响状态时，才强调 `find` 和 `operator[]` 的区别。只有保存节点身份时，才强调 `list` 迭代器稳定。只有比较器决定排序不变量时，才重点检查严格弱序。

容器选择也要跟访问模式绑定。`vector` 适合连续扫描和随机访问，`deque` 适合两端操作，`unordered map` 适合等值历史查询，`map` 适合前驱后继，`priority queue` 适合动态极值，`list` 适合已知节点常数移动，`array` 适合固定字符集计数。若题解只说“用某容器更方便”，没有说明它回答哪个查询，就应该重写。

代码块要少而精。一本教材不需要每道题都贴完整最终代码；更重要的是给关键骨架、状态更新和易错行。代码排版必须稳定：缩进清晰，命名表达状态，辅助函数拆出检查逻辑，复杂条件不要塞在一行。代码服务推理，不是用来填满页面。

审查五：Linux 源码对照是否克制

源码对照只能在确实能解释访问模式时出现。LRU 对照 `list head`，动态区间对照 `Maple Tree`，哈希桶对照 `hlist`，动态有序集合对照 `rbtree`，读多写少删除对照 `RCU`，分配连续页对照伙伴系统。这些对照有价值，因为它们回答“真实系统为什么比题目复杂”。如果只是写“Linux 里也用了某结构”，没有说明对象生命周期、并发、分配或多索引，就没有教学价值。

源码字段名会随版本变化，教材不应把字段名当重点。重点应是接口语义、对象关系、热路径和不变量。读 `list head` 就问节点如何嵌入对象；读 `rbtree` 就问比较逻辑在哪里；读 `XArray` 就问稀疏整数索引如何分层；读 `Maple Tree` 就问范围查询如何保持相邻关系；读 `RCU` 就问删除为何不能立刻释放。每次只追一个问题，读者才不会迷路。

源码实验也要可执行。写最小侵入式链表，写固定桶数哈希表，写 `map` 区间表，写简化伙伴分配器，写读写配置表模拟 `RCU`。实验不复刻内核，而是复现一个不变量。报告里必须写明和刷题题目的相同点和差异点，防止类比过度。

审查六：重复句子必须删除

修订时可以直接搜索一些危险句式：这题用某结构、注意边界、时间复杂度降低、维护状态、不要套模板、实验时对拍、C++ 要注意容器。这些句子不是绝对不能出现，但如果没有跟具体题目绑定，就应删除或改写。比如“注意边界”要改成“目标串含重复字符时 formed 表示达标字符种类，不是匹配总数”；“维护状态”要改成“哈希表保存当前下标之前每个前缀和的出现次数”。

同一章节内，相邻题目的开头也要避免同构。可以用题面冲突开头，可以用反例开头，可以用暴力慢点开头，可以用错误实现开头，可以用源码类比开头。读者需要节奏变化。若十道题都以“先从暴力开始”开头，即使内容不同，也会产生复制感。

真正必要的重复是术语稳定，而不是段落重复。比如“不变量”“候选”“历史状态”“前驱后继”“过期状态”这些词可以反复出现，因为它们构成学习框架；但每次出现都要落到不同对象上。稳定词汇让书有体系，重复段落会让书像拼接稿。

审查七：章节是否能串起来

每章开头要告诉读者上一章解决了什么问题，本章接着解决什么问题。数组讲连续内存和下标，窗口讲连续区间的增量维护，前缀和讲窗口失效后如何用代数历史状态补救，单调结构讲如何保存未解决候选，堆讲动态极值，图讲状态扩展，DP 讲重复子问题，源码讲真实工程约束。这条线如果断了，读者就会觉得每章都在换话题。

章节结尾要给迁移任务，而不是只总结。学完前缀和，让读者比较 560、523、525、862；学完单调栈，让读者比较每日温度、柱状图、下一个更大元素；学完背包，让读者比较 416、494、518、322；学完图，让读者比较岛屿、课程表、网络延迟、公交路线。迁移任务能强迫读者看到条件变化，而不是只记题号。

目录层级也要服务这种串联。大章标题表达学习阶段，小节标题表达核心问题，过碎的小节不要全部进入目录。正文中的图示、表格和实验框可以帮助定位，但不应把每个提醒都变成标题。一本书的排版舒适，来自稳定层级和连续叙述，而不是标题数量。

审查八：如何判断一段应该扩写还是删除

一段文字如果回答了为什么、如何证明、哪里会错、如何实验、如何迁移、如何和源码差异化，就值得扩写。比如“Dijkstra 堆里有旧状态”值得扩，因为它解释了 priority queue 的接口限制和正确性检查；“窗口要维护计数”只有在说明 formed、need、重复字符时才值得扩；“DP 可以空间压缩”只有在说明旧值来源时才值得扩。

一段文字如果只是换词重复前文，就应该删除。比如已经在题卡里讲过“哈希保存历史”，后面再写“哈希能快速查询历史”就没有新信息，除非加入具体 key、value、查询顺序或反例。已经在章节里讲过“数组连续内存缓存友好”，后面再重复这句话也没有意义，除非加入 vector 扩容、迭代器失效或和 list 的实验对比。

扩写和删除都要服务读者。字数目标是最低体量，不是写作目标。一本三十万字的教材如果每一万字都能教会一个新判断，就是扎实；如果只是同一句话重复三十遍，就是负担。后续修订时宁愿删除一千字模板，也要补一千字能解释一个具体错误的内容。

审查九：最终交付前的抽样方法

最终交付前，不要只看总字数和编译成功。随机抽十道题，遮住标题之外的内容，只读正文第一段，判断是否能看出本题自己的问题。若第一段换到另一道题也能成立，就说明太模板。再抽五个代码块，检查缩进、字体、行宽、注释和变量名。再抽五个实验，确认它们能复现一个错误或证明一个不变量。

还要抽查跨章节引用。接雨水在数组窗口章节讲过，在单点深挖里又讲，两个地方应该互补，而不是重复。LRU 在链表、设计题和 Linux 对照里都出现，三个地方应分别强调链表移动、索引一致性和侵入式节点。Dijkstra 在图章节和设计实验里出现，应分别强调非负边证明和旧堆状态。重复题可以出现，但角度必须不同。

最后检查手机阅读体验。段落不能太碎，代码不能横向溢出，目录不要塞满细碎小标题，彩色强调不能影响正文黑色阅读，图示要解释状态流动。手机上读算法书最怕目录混乱和代码挤压，所以构建 PDF 和 EPUB 后都要实际打开检查。技术内容和排版体验必须一起达标。

抽查结果要写进修订记录。发现一个模板段，就回到对应题目重新补模型、反例和实验，而不是只换几个词。

审查十：术语统一但例子必须变化

全书可以统一使用一些核心术语，例如候选、状态、不变量、历史信息、支配关系、边界、前驱、后继、过期状态、逻辑删除、物理释放。这些词能让读者在不同章节之间建立联系。但术语统一不等于例子重复。每次出现“不变量”，都要落到具体对象：窗口题是不重复窗口，数组题是已写入前缀，栈题是仍未结算的下标，DP 题是已经计算完的依赖状态，源码题是对象已经从索引摘除但还没释放。

同样，每次出现“支配关系”也要换成题目自己的语言。单调队列里，新下标前缀和更小且位置更靠后，所以支配旧下标；滑动窗口最大值里，新值更大且更晚，所以支配旧值；柱状图里，当前更矮柱不是支配旧柱，而是触发旧柱结算；贪心题里的支配常常要靠交换证明。若这些差异没有写出来，术语就会变成空壳。

最后抽查一句话是否合格，可以问它能否直接指导代码。比如“维护历史状态”不合格；“哈希表保存当前下标之前每个前缀和出现次数，扫描当前前缀时先查询再更新”合格。比如“注意链表指针”不合格；“改 current next 前先保存 next，反转后 previous 指向已处理前缀头，current 指向未处理第一节点”合格。教材中的每个关键句都应该尽量达到这种可执行程度。

最终自测题

自测一：给一个含负数数组，问和至少为 K 的最短子数组。请说明为什么普通滑动窗口失效，为什么前缀和单调队列成立，队尾弹出条件是什么，队首弹出条件是什么。

自测二：给一个动态日程表，要求插入区间并拒绝冲突。请说明为什么哈希表不合适，为什么只检查前驱和后继，半开区间下端点相等是否冲突。

自测三：给一个带正权图，要求从起点到所有点的最短时间。请说明为什么 BFS 不够，Dijkstra

的过期堆状态如何处理，距离数组使用什么类型。

自测四：给一个链表设计题，要求 $O(1)$ 移动节点。请说明节点由谁拥有，哈希表保存什么，删除节点时哪些索引要同步更新，是否能用 `vector` 保存节点。

自测五：给一个背包题，要求统计方案数。请说明物品能用几次，方案是否区分顺序，容量循环方向是什么，`dp[0]` 为什么初始化为 1。

这些自测题不要求背出固定代码，但要求能完整说出模型、暴力、瓶颈、状态、不变量、边界和 C++ 风险。如果说不完整，就回到对应章节重写复盘报告。

错题复盘样例

错题一：窗口左端回退

错误现象：无重复字符最长子串在输入 `abba` 上返回三。错误原因通常是看到重复字符后直接把左端设为上次位置加一，没有检查上次位置是否仍在当前窗口内。处理最后一个 `a` 时，它上次出现在下标零，但当前窗口左端已经在下标二；如果把左端拉回一，就会让已经排除的字符重新进入窗口，破坏窗口无重复不变量。

复盘写法：窗口左端只能右移，不能回退。更新公式应当是取当前左端和上次位置后一位的较大值。这个错误说明我们没有把“窗口保存当前无重复子串”落实成不变量，只是在机械处理重复字符。修复后，再用 `abba`、`tmmzuxt`、全相同字符和全不同字符测试。报告中要记录每一步左端变化，而不是只写最终答案。

错题二：前缀和漏掉开头区间

错误现象：和为 K 的子数组在输入 `[3]`、目标 `3` 时返回零。错误原因是哈希表没有预先放入前缀和零的一次出现。区间从下标零开始时，它对应的历史前缀是空前缀；如果不记录空前缀，就等于告诉算法“从开头开始的区间不存在”。

复盘写法：前缀和题必须解释历史状态的时间点。扫描到当前前缀 `s` 时，答案增加历史前缀 `s-k` 的出现次数；空前缀是合法历史状态，位置在数组开始之前。先查询再更新当前前缀，是为了避免把当前前缀当成历史前缀。修复后测试目标为零、负数、重复前缀和从开头开始的多段答案。

错题三：二分检查函数错误

错误现象：船运包裹二分循环看似标准，但某些用例返回容量小于最大包裹。错误原因不是二分边界，而是检查函数没有在当前包裹超过容量时立即判不可行，或者下界没有设为最大包裹。二分只能在正确谓词上找边界，不能修复错误谓词。

复盘写法：先写谓词定义，再写二分。容量 `cap` 可行，含义是按原顺序把包裹分成不超过 D 段，每段和不超过 `cap`。如果某个包裹重量已经超过 `cap`，谓词必假。下界设为最大包裹，上界设为总重量，可以让检查函数更简单。每次失败用例都要打印 `cap` 和需要天数，确认单调性是否存在。

错题四：柱状图宽度少一

错误现象：柱状图最大矩形在递减数组或含哨兵用例上面积偏小。错误原因常在弹栈后的宽度计算。弹出柱子后，当前下标是右侧第一个更矮位置；弹出后新的栈顶是左侧第一个更矮位置。有效宽度不包含左右两个更矮边界，因此是 `right - left - 1`。如果栈为空，说明左侧没有更矮边界，宽度是当前下标。

复盘写法：单调栈弹出时结算的是被弹出的柱子，不是当前柱子。当前柱子只是右边界。每一次弹出都要能说出左边界、右边界、高度和宽度。修复后测试严格递增、严格递减、全相等、单个柱子和末尾哨兵。这个错误说明我们背了栈模板，却没有理解弹出时机。

错题五：背包循环方向写反

错误现象：分割等和子集在只有一个物品时却能凑出两倍容量。原因是 0-1 背包一维压缩时容量正序遍历，导致当前物品刚更新出的状态又被当前物品再次使用。这个错误在小样例上可能不明显，但一旦构造单物品反例就会暴露。

复盘写法：循环方向来自物品能使用几次。0-1 背包每个物品只能用一次，所以容量倒序，读取上一轮物品阶段的状态；完全背包物品可以无限使用，所以容量正序，允许当前物品在同一轮继续贡献。报告里不要写“背包模板”，要写“当前题每个数只能选一次，因此倒序容量”。再用零钱兑换 II 对比，说明组合计数为什么外层遍历硬币。

错题六：图状态维度缺失

错误现象：带钥匙、障碍消除或剩余资源的网格最短路返回过大或错误不可达。原因是访问标记只按位置记录，把“同一位置但资源不同”的状态合并了。位置相同不代表未来能力相同；剩余障碍次数更多或钥匙集合不同，会改变后续可走边。

复盘写法：图题先定义状态，不只是节点位置。若未来决策依赖资源，就把资源放进状态和访问标记。障碍消除题的 `visited` 至少应包含行、列、剩余消除次数；钥匙迷宫要包含行、列、钥匙集合。BFS 的第一次到达最短，只对完整状态成立，不对被错误压缩的位置成立。测试要构造必须绕路保留资源的用例。

错题七：LRU 更新已有 key

错误现象：LRU 在 `put` 已存在 key 后淘汰错误元素。原因是只更新了 `value`，没有把节点移动到最新位置，或者新建了一个重复节点而没有删除旧节点。LRU 的“使用”包括 `get`，也包括更新已有 key；只要访问或更新成功，它就应成为最近使用。

复盘写法：LRU 有两个不变量：哈希表中每个 key 对应链表中唯一节点，链表顺序从最新到最旧。更新已有 key 时，不能增加容量，不能产生重复节点，必须移动到头部。容量满淘汰时，要从链表尾找到 key，再从哈希表删除。测试容量为一、重复 `put`、`get` 后再 `put`、更新后淘汰等序列。

错题八：Trie 搜索没有恢复访问标记

错误现象：单词搜索在某些路径上漏答案，或者同一格子被重复使用。原因通常是 DFS 进入格子后没有在返回时恢复访问标记，导致其他分支误以为该格不可用；或者根本没有访问标记，导致一条路径中重复使用同一格。Trie 剪枝只能判断前缀是否存在，不能替代网格路径的访问约束。

复盘写法：回溯代码必须成对出现：做选择、递归、撤销选择。访问标记属于路径状态，而不是全局永久状态。找到一个单词后可以清空 Trie 节点上的单词标记避免重复答案，但不能清空网格访问规则。测试棋盘有重复字母、同一单词多条路径、单字符单词和路径必须回退的用例。

最终质量检查表

交付一本算法教材前，逐章检查以下问题。第一，是否每个核心算法都有暴力基线；第二，是否解释了优化替换了哪一步；第三，是否给出至少一个反例或边界实验；第四，是否说明 C++ 容器选择和风险；第五，是否有 Linux 或工程源码视角帮助读者迁移；第六，是否避免同一句话在大量题卡中机械重复。

对单题检查也用同样标准。题面模型是否一句话说明清；输入规模是否决定复杂度预算；状态是否足够；不变量是否能反驳错误实现；边界是否进入测试表；代码是否有溢出、迭代器失效、默认插入、递归深度或比较器风险；变体是否重新证明。只有这些都通过，才算高质量题解。

最后检查阅读体验。章节标题不要过碎，小节应服务于连续讲解；目录只保留大结构，不把每个细碎点都塞进去；代码块使用稳定宽度和清晰缩进；正文以黑色为主，颜色用于提示、强调和代码；图示用于解释状态流动，而不是装饰。一本教材的质量不仅取决于内容量，也取决于读者能否顺着推理读下去。

读者自查问答

看到新题先问什么

第一问输入规模。规模决定暴力能否作为最终答案，也决定是否需要取对数、线性、平方或伪多项式方法。第二问数据条件。数组是否有序，元素是否全正，字符集是否固定，图边权是否相同，树是否是搜索树，区间端点是闭区间还是半开区间。第三问输出目标。是判断、计数、最值、路径、所有方案还是修改后的结构。输出目标不同，保存的状态就不同。

第四问能否从暴力中看出重复工作。重复扫描历史值，可能用哈希；重复求区间和，可能用前缀；重复找极值，可能用堆或单调队列；重复计算子问题，可能用 DP；重复判断连通，可能用并查集。第五问优化依赖的条件是否稳定。如果条件变了，比如正数变成有负数，有序变成无序，等权变成带权，原算法就要重新证明。

如何判断自己不是背模板

把答案遮住，只保留题面，尝试写出暴力版。如果暴力版说不清，说明还没理解候选空间。再尝试说出瓶颈是哪一步，而不是直接说某个算法名。然后说明优化结构维护什么状态，为什么这个状态足

够回答输出目标。最后拿一个反例解释错误写法为什么错。能完成这四步，才说明不是背模板。

另一个检查方法是改条件。把数组加入负数，把返回一个答案改成返回所有答案，把判断存在改成统计数量，把静态输入改成动态更新，把普通树改成 BST 或把 BST 改成普通树。若你能判断原解法哪些部分保留、哪些部分失效，就说明理解的是结构。若只能说“这个题我见过”，说明还停留在题号记忆。

如何把代码写得更像工程答案

工程答案要把风险前置。函数入口检查不可行条件，变量名表达状态语义，复杂判断拆成辅助函数，容器选择和访问模式一致。二分答案把检查函数单独写；图算法把邻居生成集中；链表操作把摘除和插入封装；DP 把初始化和转移分开。这样写不是为了增加行数，而是让不变量有清晰位置。

还要避免隐式成本。不要在热循环里反复构造大字符串；不要在 range-for 中复制复杂对象；不要把可能溢出的乘法留在 int；不要保存会被 vector 扩容失效的指针；不要用 `operator[]` 做纯查询；不要写不满足严格弱序的比较器。面试时间有限，也应主动说明这些风险，哪怕代码里只写最核心版本。

如何读 Linux 源码不迷路

读源码前先写问题，不要漫无目的打开文件。比如“链表删除怎么保持前后指针”“红黑树插入时比较逻辑在哪里”“XArray 如何把整数索引分层”“RCU 为什么延迟释放”。每次只追一条路径，画出对象、索引、插入、查找、删除和释放。字段名可以暂时不背，但对象关系必须画清楚。

读完以后找一个刷题类比。list 对照 LRU，rbtree 对照动态区间，XArray 对照稀疏索引，RCU 对照逻辑删除和物理删除分离，伙伴系统对照分层空闲块。类比不是说二者完全相同，而是帮助理解访问模式。真正的源码能力来自差异：内核多了并发、分配上下文、引用计数和生命周期。

如何长期复习

每周固定回看错题，而不是只做新题。错题报告按八项写：题面模型、暴力基线、瓶颈、优化结构、正确性、边界、C++ 风险、变体。一个月后随机抽十道题，只看题名重讲推导。如果讲不出暴力和瓶颈，就回到原章节；如果讲不出边界，就补实验；如果讲不出 C++ 风险，就补容器实验。

复习时把题按能力分组。前缀和组、单调结构组、答案二分组、拓扑和最短路组、背包组、区间 DP 组、设计题索引组合组。每组挑一题作为代表，再列两道变体。这样复习能建立迁移能力，而不是只增加题号数量。最终目标是看到新题时能自己搭桥：从题面条件到暴力，从暴力到瓶颈，从瓶颈到结构，从结构到证明。

第零部分

Linux 源码专题：内核数据结构与算法

Linux 内核数据结构与算法专题

学习 Linux 内核数据结构，不能把它当成另一套刷题模板。内核里的链表、哈希链、红黑树、XArray、Maple Tree、RCU、伙伴系统和 LRU 链表，都是在同一个工程环境里长出来的：对象生命周期由内核控制，很多对象不能随便移动，内存分配可能发生在严格上下文中，读写并发非常频繁，某个对象还常常要同时挂进多个索引。理解这些约束以后，再回看力扣题，会更容易明白为什么一个高质量答案必须先问访问模式，而不是先背容器名字。

本章的阅读目标不是背某个内核版本的字段名。Linux 头文件和内部结构会随着版本变化，字段、辅助宏和封装层都可能调整；真正稳定的是设计原则：节点常嵌入业务对象，比较逻辑常由调用者提供，读路径尽量减少分配和锁竞争，删除对象常分成“从索引摘除”和“等读者离开后释放”两个阶段。读源码时请优先看接口语义、热路径和不变量，再看具体字段。

源码阅读路线

链表	读 <code>include/linux/list.h</code> ，先看 <code>struct list_head</code> 、初始化、插入、删除和遍历宏。
哈希链	读 <code>hlist</code> 相关接口，理解桶头和节点为什么分开，为什么节点里要保存前驱指针位置。
有序树	读 <code>include/linux/rbtree.h</code> ，看节点嵌入、旋转由库完成、比较由调用者完成。
整数索引	读 <code>include/linux/xarray.h</code> ，关注整数 ID 到指针的稀疏映射。
区间索引	读 <code>include/linux/maple_tree.h</code> ，关注范围查询、VMA 这类区间对象为什么不适合普通数组。
并发读	读 <code>include/linux/rcupdate.h</code> ，先理解读侧临界区、发布新版本和延迟释放。

本章建议对照 Linux 官方文档阅读：链表见 <https://docs.kernel.org/core-api/list.html>，红黑树见 <https://docs.kernel.org/core-api/rbtree.html>，XArray 见 <https://docs.kernel.org/core-api/xarray.html>，Maple Tree 见 https://docs.kernel.org/core-api/maple_tree.html，RCU 入门见 <https://docs.kernel.org/RCU/whatisRCU.html>。书中代码是教学简化版，用来解释不变量和访问模式，不代表内核源码的逐行复制。

为什么内核不用 STL 容器

在用户态 C++ 里，`std::vector`、`std::list`、`std::map` 和 `std::unordered_map` 会把元素所有权、分配器、迭代器、异常安全和泛型接口封装在容器内部。这个模型非常适合普通应用：你把元素放进容器，容器负责管理节点和内存，算法通过迭代器访问元素。内核的环境不一样。内核主要用 C 写成，不能依赖 C++ 异常和模板；很多路径不能随意阻塞分配；同一个对象常常同时属于多个结构；对象释放必须和锁、引用计数、RCU 宽限期配合。于是内核更偏向侵入式数据结构（intrusive data structure）：链接节点嵌进业务对象，数据结构只负责组织对象，不拥有对象。

以一个进程、一个页缓存对象或一个定时器对象为例，它可能同时需要按链表顺序被扫描，按哈希键被快速查找，按时间或地址范围被有序索引。如果每种索引都单独分配一个外部节点，再让节点指向业务对象，内存分配次数会增加，缓存局部性更差，释放路径也更复杂。侵入式结构让对象自己带着多个链接字段：一个字段挂进 LRU 链表，另一个字段挂进哈希桶，另一个字段挂进树。代价是调用者必须非常清楚节点属于哪个结构、何时插入、何时删除、对象何时还能被访问。

这和刷题中的 LRU 缓存有直接关系。普通写法是 `unordered_map<Key, list<Node>::iterator>` 加 `list<Node>`。哈希表负责按 key 定位，链表负责维护新旧顺序。内核场景常进一步把链表节点嵌到缓存对象内部，哈希索引也嵌到对象内部；对象本身不因为

进入链表而被复制。你在面试中说“哈希表加双向链表”时，如果能补一句“哈希表负责定位，链表负责顺序，节点身份必须稳定”，就已经接近源码思维。

侵入式链表：list_head

Linux 的双向链表不是一个拥有元素的容器，而是一个很小的链接节点。典型形式可以抽象成下面这样：

Listing 15.1 教学化简版：侵入式双向链表节点

```
1 struct ListHead {
2     ListHead* next;
3     ListHead* prev;
4 };
5
6 struct PageLike {
7     int id;
8     bool active;
9     ListHead lru_node;
10    ListHead dirty_node;
11 };
```

PageLike 里有两个链表节点，意味着同一个对象可以同时挂进两个不同链表。链表节点不保存 **id**，也不知道外层类型。遍历链表时，源码会通过成员地址反推外层对象地址，这就是常说的 **container_of** 思想：已知一个成员在结构体中的偏移，就能从成员指针减去偏移得到结构体起始地址。它不是魔法，而是 C 语言对象布局和偏移计算的组合。

图示：从节点回到对象

业务对象	<code>PageLike \{ id, active, lru_node, dirty_node \}</code>
链表视角	链表只看见一串 <code>ListHead</code> ，不知道 <code>PageLike</code> 的其他字段。
反推对象	遍历到 <code>lru_node</code> 后，用成员偏移回到包含它的 <code>PageLike</code> 。
设计收益	不额外分配链表节点，同一对象能进入多个链表，节点地址稳定。

链表操作的核心不变量很简单：对任意节点 **x**，如果它在链表中，那么 `x->next->prev == x` 且 `x->prev->next == x`。插入节点就是同时改四条边；删除节点也是同时改四条边。错误通常不来自算法难，而来自顺序和生命周期：删除前是否还在链表中，删除后是否把节点重新初始化，遍历过程中删除当前节点是否提前保存了下一个节点。

Listing 15.2 教学化简版：在两个节点之间插入新节点

```
1 void insert_between(ListHead* previous, ListHead* next, ListHead* node)
2 {
3     node->next = next;
4     node->prev = previous;
5     previous->next = node;
6     next->prev = node;
7 }
8
9 void remove_node(ListHead* node)
10 {
11     ListHead* previous = node->prev;
12     ListHead* next = node->next;
13     previous->next = next;
```

```

14     next->prev = previous;
15     node->next = node;
16     node->prev = node;
17 }

```

这段代码要和力扣链表题联系起来看。反转链表强调“改 `next` 前保存后继”，K 个一组翻转强调“先确定分组边界”，LRU 强调“已知节点常数时间移动到头部”。内核链表把这些细节放大了：因为节点可能被很多模块引用，错误删除不是返回错误答案，而是破坏内核对象关系。刷题阶段写链表题，也应该养成明确前驱、当前、后继和已处理前缀的习惯。

实验：写一个最小侵入式链表。定义 `struct Item { int key; ListHead node; }`，创建三个对象并把它们插入同一个链表。遍历时不要在 `ListHead` 里存 `key`，而是通过成员偏移回到 `Item`。再让 `Item` 增加第二个 `ListHead`，同时挂入另一个链表，观察同一对象如何拥有两种顺序。

hlist：哈希桶里的瘦链表

普通双向链表每个头节点也是一个 `list_head`，空链表时头节点的 `next` 和 `prev` 都指向自己。哈希表有大量桶，如果每个桶头都放完整双向节点，桶头本身会占用更多空间。Linux 的 `hlist` 可以理解成一种面向哈希桶优化的链表：桶头只保存第一个节点，节点保存下一个节点以及“指向当前节点指针的位置”。这样删除已知节点时仍然可以常数时间更新前驱链接，同时桶头更轻。

这背后的算法思想和刷题哈希表相同：哈希函数把 `key` 映射到桶，桶内用链表处理冲突。区别在于内核更关心节点分配、缓存局部性和并发删除。刷题里你使用 `unordered_map` 时，桶和节点都由标准库隐藏；读内核源码时，桶数组、节点链接和删除顺序都暴露在接口语义中。

哈希链的不变量

桶定位	哈希值决定先进入哪个桶，桶只缩小候选范围，不保证桶内有序。
冲突处理	同桶节点继续按 <code>key</code> 比较，不能只比较哈希值。
删除节点	已知节点删除要能改前驱的 <code>next</code> ，否则必须从桶头重新扫描。
并发风险	读者遍历时删除节点，需要锁、引用计数或 RCU 保护生命周期。

把它映射到面试题，两数之和和前缀和计数都在使用“key 到历史状态”的映射。高质量题解不应只写 `unordered_map`，还要说明 `key` 是什么、`value` 是什么、什么时候查询、什么时候插入、是否允许当前元素和自己匹配。内核里的 `hlist` 把这些语义拆得更底层：桶只是入口，节点才是业务对象，生命周期由调用者控制。

红黑树：有序索引和调用者比较

红黑树解决的是动态有序集合问题：插入、删除、查找、前驱、后继都能在对数复杂度内完成。C++ 的 `std::map` 把 `key`、`value`、比较器和树节点封装起来；Linux 的 `rbtree` 更底层，树节点通常嵌入业务对象，库函数负责旋转和颜色维护，但“往左还是往右”的比较逻辑由调用者写。这样做让内核对象可以按自己的字段组织成树，也避免为每个节点额外包装一个通用容器节点。

Listing 15.3 教学化简版：业务对象嵌入树节点

```

1 struct RbNode {
2     RbNode* left;
3     RbNode* right;
4     RbNode* parent;
5     bool red;
6 };
7

```

```

8 struct TimerLike {
9     long long expires;
10    RbNode tree_node;
11 };

```

红黑树的细节可以很复杂，但教材阶段先抓三件事。第一，它是二叉搜索树，左子树 key 小，右子树 key 大。第二，它通过颜色约束和旋转保持近似平衡，避免退化成链。第三，插入和删除之后，局部旋转不会破坏中序顺序，只是在保持排序的前提下修复高度和颜色约束。刷题中的“搜索旋转排序数组”和这里的旋转不是同一件事；树旋转是局部结构调整，中序序列不变。

红黑树和刷题有序容器

需要顺序	只要题目问前驱、后继、最小大于、最大小于，哈希表就不够。
动态更新	如果元素会插入删除，排序数组每次移动成本高，有序树更合适。
比较责任	内核调用者自己决定 key；C++ <code>map</code> 把比较器藏进容器类型。
正确性	树旋转保持中序顺序，所以查找语义不变，只改善平衡。

在内核中，有序树常用于按时间、地址或其他可比较 key 组织对象。具体模块会随版本演进而变化，例如某些历史结构可能从红黑树迁移到 Maple Tree 或其他结构。读源码时不要死记“某模块一定使用某树”，而要问：这个模块是否需要动态有序查找，是否需要范围查询，是否需要前驱后继，是否需要频繁插入删除。

XArray：稀疏整数索引

很多内核对象需要用整数 ID 查找：页缓存按页索引查找，文件描述符按编号查找，设备和 ID 分配也常见“整数到对象”的映射。直接开一个巨大数组很浪费，因为 ID 空间可能稀疏；用普通哈希表能查找，但范围扫描、按序遍历和某些并发语义不一定理想。XArray 可以理解作为一种面向无符号长整数索引的稀疏数组结构，它以树状分层方式把大整数索引拆开，按需分配中间节点。

从算法上看，这和 Trie 有相似味道：字符串 Trie 每层消耗一个字符，XArray 这类整数索引结构每层消耗索引的一部分位。若索引空间很大但实际对象不多，树只为存在对象的路径分配节点。若需要查找某个 ID，沿着索引位段向下走；若需要扫描一段 ID，结构可以按顺序推进。它比“巨大数组”省空间，比“无序哈希”更重视索引顺序和内核并发接口。

从力扣题理解 XArray

数组	下标连续且范围小，直接数组最快。
哈希表	key 稀疏且只需要等值查找，哈希表简单。
Trie / 分层索引	key 可以按位或按字符分层，适合共享前缀和稀疏空间。
XArray	面向内核对象的整数到指针映射，还要考虑遍历、标记和并发读写。

刷题中类似的思想出现在前缀树、位 Trie、稀疏表和坐标压缩。区别是刷题往往只考虑单线程和答案复杂度，内核还要考虑节点分配、锁、RCU、标记位和对象生命周期。如果你读 XArray 感到细节很多，先不要陷进每个宏，先回答三个问题：索引如何分层，空洞如何表示，对象指针何时对读者可见。

Maple Tree：范围映射和 VMA 思维

普通映射结构处理的是点查询：给定 key 找对象。内存管理中大量问题是范围查询：给定地址，找覆盖它的虚拟内存区域；插入一个新区间，需要检查与相邻区间是否重叠；删除或分裂区间后，还要保持后续查询高效。Maple Tree 是 Linux 中用于范围映射的一类重要结构，现代内核用它承载诸如虚拟内存区域管理这类范围索引场景。你不需要在刷题里复刻它，但要理解为什么“区间对象”比

“单点 key”更复杂。

力扣里的合并区间、插入区间、删除被覆盖区间和日程安排，都是范围结构的简化版。排序数组适合离线一次性处理；如果区间会动态插入、删除、查询，就需要有序结构维护相邻关系。Maple Tree 的动机也是如此：地址空间很大，区间数量动态变化，查询频繁，且读路径性能很关键。它的实现细节比普通红黑树复杂，但读者先抓住范围映射语义即可。

区间结构的单点分析

点查询	给定地址，找到起点不大于它且终点覆盖它的区间。
插入	新区间必须和前后相邻区间比较，判断重叠、合并或拒绝。
删除	可能删除整个区间，也可能把一个区间切成左右两段。
遍历	常需要按地址顺序扫描下一段区域，不能只支持等值查找。

学习这部分时，建议先用 `std::map<int, Interval>` 写一个小型区间表：key 是区间起点，value 是终点和属性。实现 `find(point)`、`insert(l, r)`、`erase(l, r)` 三个操作。写完以后再问自己：如果区间数量巨大、读多写少、对象释放有并发读者，这个简单 map 还缺什么？这个问题会自然引出内核结构的复杂性。

RCU：读多写少下的数据结构生命周期

RCU 的全称是 Read-Copy-Update。它不是某个容器，而是一种并发读写思想：读者在读侧临界区内尽量不加重锁；写者创建或修改新版本，把新指针发布给后来的读者；旧对象不能立刻释放，必须等所有可能还看见旧版本的读者离开之后再回收。这个“延迟释放”会直接改变数据结构删除操作的含义。

普通单线程链表删除可以理解为两步：改前后指针、释放节点。RCU 场景下必须拆得更细：第一步从索引中摘除，让新读者找不到它；第二步等待宽限期，让旧读者不再持有它；第三步才释放内存。如果把摘除和释放混成一步，读者可能拿着旧指针访问已经释放的对象。很多内核数据结构的复杂性，都来自这类生命周期问题。

RCU 删除的三阶段

逻辑删除	从链表、树或哈希表中摘除，使后续查找不再得到该对象。
等待读者	等待读侧临界区结束，确保旧读者不再访问旧对象。
物理释放	释放内存或归还缓存，此时对象生命周期才真正结束。

这和刷题中的“懒删除”有相似但不相同的味道。滑动窗口中位数用堆时，过期元素先标记，等它到堆顶再弹出；RCU 中对象先从新索引中不可达，再等旧读者离开后释放。二者都说明删除不一定等于立刻销毁，区别在于刷题懒删除主要为了弥补容器能力，RCU 延迟释放主要为了并发安全。

调度器：从队列到虚拟运行时间

操作系统调度要解决的问题是：大量可运行任务中，下一个应该让谁运行，运行多久，如何兼顾公平性、响应性和吞吐。最简单的模型是队列：先来先服务或轮转调度。但真实系统里任务权重不同、交互需求不同、睡眠和唤醒频繁，单纯 FIFO 不够。Linux 的完全公平调度思想可以粗略理解为维护任务的虚拟运行时间，优先选择虚拟运行时间较小的任务，让每个任务按权重获得相对公平的 CPU 时间。

这个模型和算法题里的“动态最小值”有关。你可以把可运行任务看成候选集合，每次取虚拟运行时间最小者运行，运行后它的虚拟运行时间增加，再放回候选集合。若只需要取最小值，堆似乎可行；若还需要删除任意任务、按顺序遍历、处理权重和复杂的队列状态，有序树或更专门的数据结构会更合适。具体内核实现会随调度类变化，但核心问题始终是动态候选集合和公平性指标。

实验：写一个简化调度模拟器。每个任务有 `id`、`weight`、`vruntime` 和剩余运行时间。用 `std::set` 按 `vruntime` 排序，每次取最小者运行一个时间片，再更新 `vruntime` 放回。然后把 `std::set` 换成 `priority_queue`，尝试支持“任务睡眠时从队列删除”，比较两种结构的接口差异。

伙伴系统、SLUB 和 LRU：内存管理里的算法

内存管理是数据结构密度很高的地方。伙伴系统把物理页按 2 的幂次阶组织成空闲块：需要分配小块时，从更大块拆分；释放时，如果相邻伙伴也空闲，就合并成更大块。这个算法的目标不是最优装箱，而是在可接受碎片下快速分配和释放连续页。它和刷题里的区间合并、二进制分解、堆式层级都有联系，但更强调页框编号、阶数和合并条件。

伙伴系统的思维模型

阶数	第 k 阶块包含 2^k 个连续页。
拆分	没有合适小块时，从更高阶块拆成两个伙伴。
合并	释放后若伙伴同阶且空闲，就合并成高一阶块。
不变量	每个空闲块只出现在它当前阶数对应的空闲链表中。

小对象分配又是另一层问题。频繁为内核对象调用通用页分配器成本高，也会造成碎片。SLAB/SLUB 这类分配器把同类型对象放进缓存，从页中切出固定大小对象，复用已经初始化过的对象槽位。刷题里你通常不关心对象分配成本，但工程里大量小节点会让哈希表、链表和树的常数差异变得很明显。理解 SLUB 后，再看 `list` 节点分配、`unordered_map` 节点分配，就会知道“理论 $O(1)$ ”背后还有分配器和缓存局部性。

页面回收中的 LRU 思想也值得和力扣 LRU 缓存比较。力扣 LRU 通常要求 `get` 和 `put` 都是 $O(1)$ ，结构是哈希表加双向链表。内核页面回收更复杂：页面可能分为活跃和非活跃，文件页和匿名页也可能有不同链表，访问位、脏页、写回和引用计数都会影响回收判断。它不是一个简单的“最久未使用就删”，而是在可测量近似、并发成本和 I/O 成本之间折中。教材阶段要学到的不是所有策略细节，而是多链表如何表达状态分层。

网络、I/O 和定时器中的队列结构

系统里还有大量队列结构。网络包需要排队、分类、限速和调度；块 I/O 请求需要合并相邻请求、排序或分发到设备队列；定时器需要按到期时间组织事件；工作队列需要把任务从中断或提交路径转移到可睡眠上下文执行。每种队列都不是“先进先出”四个字能解释完的，关键仍然是访问模式：是否只从头出队，是否需要按优先级出队，是否需要按时间查找最早事件，是否需要取消任意节点。

这部分和刷题题型也能对应。普通 BFS 队列只需要 FIFO；0-1 BFS 需要双端队列，因为零权边放队首、一权边放队尾；Dijkstra 需要按当前距离取最小，适合堆；任务调度器可能需要最大频次、冷却时间和时间推进；区间调度需要按结束时间排序。真实内核把这些问题放在更严格的并发和生命周期约束下，但算法选择的出发点完全一致。

如何把内核源码读成能力

读内核源码最怕两种极端：一种是只看博客结论，觉得“内核用了红黑树”就结束；另一种是一打开头文件就追每个宏，最后忘了自己为什么读。正确方法是单点问题单点读。比如今天只读链表删除，就只回答：删除已知节点要改几条边，遍历时删除为什么要 `safe` 版本，删除后节点是否重新初始化。明天只读红黑树插入，就只回答：调用者如何找到插入位置，库函数如何链接节点，修复函数为什么不改变中序顺序。

五个源码实验案例

这一节把 Linux 数据结构压成五个可以动手的实验。实验不要求完全复刻内核，只要求把源码里的访问模式和不变量跑出来。每个实验都要写报告：对象是什么，索引是什么，插入、查询、删除分别维护哪个不变量，和对应力扣题有什么相同点和不同点。

实验一：侵入式 LRU

从力扣 146 LRU 缓存开始。普通答案通常是 `list` 加 `unordered_map`，链表节点由 `list` 管理，哈希表保存迭代器。侵入式版本改成对象自己带两个字段：`key`、`value` 和双向链表节点。哈希表从 `key` 映射到对象指针，链表只负责顺序。这样能直观看到内核设计的第一原则：对象不因为进入链表而被复制，链表也不拥有对象。

实验步骤如下。第一，写一个 `CacheEntry`，内部包含 `ListHead lru`。第二，写

`move_to_front`、`remove_tail`、`insert_front` 三个链表操作。第三，用 `unordered_map<int, CacheEntry*>` 定位对象。第四，容量满时先从链表尾部找到对象，再从哈希表删除 key，最后释放对象。报告里要特别说明删除顺序：如果先释放对象，再从哈希表或链表删除，就会留下悬空指针。

这个实验对应内核里的 LRU 思维，但不能简单说“内核也是 LRU 缓存”。真实页面回收会考虑活跃、非活跃、脏页、写回、引用和 NUMA 等因素；实验只保留最核心的索引组合：哈希表负责定位，双向链表负责顺序，节点身份必须稳定。

实验二：哈希桶和 hlist 删除

写一个小哈希表，桶数组大小固定为 16，每个桶保存一条链。第一版用普通单链表，只在节点里保存 `next`。当你拿到一个节点指针并想删除它时，必须从桶头重新扫描找到前驱。第二版让节点额外保存“指向当前节点的前驱链接位置”，也就是可以修改前驱的 `next` 或桶头的 `first`。这样已知节点删除就不需要重新扫描。

报告要回答三个问题。第一，为什么桶头不一定需要完整双向节点？第二，为什么节点保存前驱链接位置可以支持常数时间删除？第三，如果另一个读者正在遍历这个桶，删除节点后什么时候才能释放对象？第三个问题会自然连接到 RCU：摘除只是让新查找找不到它，释放还要等旧读者离开。

这个实验对应的力扣题不是某一道固定题，而是一类哈希题：两数之和、前缀和计数、字母异位词分组、四数相加 II。刷题里桶结构被标准库隐藏；实验把桶、冲突和删除成本显式暴露出来。

实验三：区间表和 Maple Tree 思维

用 `std::map<long long, Interval>` 写一个简化虚拟地址区间表。key 是区间起点，value 保存终点和权限字符串。实现三个操作：`find(addr)` 找覆盖某个地址的区间；`insert(left, right)` 插入不重叠区间；`erase(left, right)` 删除一个子区间，必要时把原区间切成左右两段。

这个实验的关键不是 map 代码，而是区间不变量。任意两个相邻区间必须不重叠，查询地址时只需要找起点不大于它的最后一个区间，再判断终点是否覆盖。插入时只需要检查前驱和后继；删除时要处理四种情况：删除整段、删除左侧、删除右侧、从中间挖掉一段。每种情况都要写一个用例。

Maple Tree 比这个实验复杂得多，但问题同源：地址空间是稀疏范围集合，查询频繁，插入删除会改变相邻关系。力扣 56、57、435、986 是离线区间处理；这个实验是动态区间索引。两者的桥梁是“排序后相邻区间决定合法性”。

实验四：简化伙伴分配器

设总页数为 $2^{\{10\}}$ ，每个空闲块用起始页号和阶数表示。维护 11 个空闲链表，第 k 个链表保存大小为 2^k 的空闲块。分配 k 阶块时，如果当前阶没有空闲块，就向上找更大阶，逐层拆分；释放时，计算伙伴块编号，如果伙伴同阶且空闲，就从空闲链表删除伙伴并合并成高一阶。

这个实验训练的是二进制边界。伙伴块的起始页号不是随便算的，它和阶数对齐；两个伙伴合并后起点取较小者，阶数加一。报告要打印每次操作后各阶空闲块数量，并给出一个碎片案例：虽然总空闲页数足够，但没有足够大的连续块，导致高阶分配失败。

对应的算法题包括区间合并、位运算对齐、堆式层级和内存池模拟。它们都说明一个原则：分配器不是只问总量，还问连续性、对齐和合并条件。

实验五：RCU 风格读写模拟

写一个读多写少的配置表。读者只读一个全局指针指向的不可变表；写者复制旧表，修改副本，然后原子发布新指针。旧表不要立即释放，而是放入延迟释放列表。为了模拟宽限期，可以让每个读者进入读侧临界区时增加计数，离开时减少计数；当读者计数为零时，写者再释放延迟列表里的旧表。

这个实验会让你看到 RCU 和普通锁的不同。普通互斥锁让读者和写者互相等待，逻辑简单但读路径重；RCU 让读者轻量，写者承担复制、发布和延迟释放成本。数据结构上最大的变化是删除不等于释放：先让新读者不可达，再等旧读者结束，最后释放对象。

把它和刷题懒删除比较。滑动窗口中位数里，堆中旧元素暂时不删，是因为 `priority_queue` 不支持删除任意元素；RCU 旧对象暂时不释放，是因为并发读者可能仍然持有旧指针。相同点是“逻辑不可见”和“物理释放”分离，不同点是目的和安全边界完全不同。

习题与作业

本章作业：

1. 用 C++ 写一个侵入式链表小实验，要求同一个对象同时挂入两个链表，并说明两个链表节点字段分别代表什么。
2. 用 `std::map` 写一个区间表，支持点查询、插入不重叠区间、删除一个子区间。完成后解释它和 Maple Tree 解决的问题有什么相同点，工程约束有什么不同。
3. 写一个简化伙伴系统，只管理 2^{10} 个页框，支持按阶分配和释放。每次操作后打印各阶空闲链表长度，观察拆分和合并。
4. 写一个 RCU 思想模拟：读者线程只读取共享指针，写者复制新对象并发布，旧对象放进延迟释放列表。即使不真正实现内核 RCU，也要把“摘除”和“释放”拆成两步。
5. 选一道力扣题和一个内核结构做对照。例如 LRU 缓存对照页面回收链表，合并区间对照 VMA 区间管理，前 K 高频对照动态候选极值。要求写出相同点和不同点，不能只写“都是链表”。

本章复盘

Linux 内核数据结构不是为了展示技巧，而是为访问模式、生命周期、并发和内存分配服务。侵入式链表强调对象身份稳定，hlist 强调哈希桶空间和删除效率，红黑树强调动态有序索引，XArray 强调稀疏整数映射，Maple Tree 强调范围查询，RCU 强调读多写少下的延迟释放。刷题时把这些思想降维使用，就是高质量题解里的状态职责、不变量和容器选择。

术语表

暴力法 直接枚举解空间的方法，是理解题目和建立正确性的起点。

不变量 算法执行过程中始终成立的性质，用来证明优化解正确。

滑动窗口 用左右边界维护连续区间，并增量更新窗口状态的方法。

前缀和 把区间和查询转化为两个前缀差值的数据表示。

哈希表 通过哈希函数把 key 映射到桶，平均支持常数时间查询的数据结构。

BFS 广度优先搜索，适合无权最短路和层次扩展。

DFS 深度优先搜索，适合连通块、路径搜索和回溯。

拓扑排序 对有向无环图按依赖顺序排列节点的方法。

索引

algorithm, ii

data structure, ii

intrusive data structure, 349

侵入式数据结构, 349

数据结构, ii

算法, ii